

Also Available

FREE DVD



Free with This Book Professor Teaches

**MS® Office 2010
PC DVD-ROM**

- Over 494 Learning Topics.
- The Fast, Easy, Interactive way to learn
- The fastest way to learn MS Office 2010
- #1 Best Seller in Office 2010 Training
- Step-By-Step Subject Based Training
- Covers... Word, Excel, Access, PowerPoint, Outlook and Publisher...In depth Training

BPB's

COMPUTER COURSE

Windows 7 with MS Office 2010

- ✓ Fundamentals
- ✓ Networking
- ✓ Windows 7
- ✓ MS Office 2010
 - Word 2010
 - Excel 2010
 - PowerPoint 2010
 - Access 2010
- ✓ Tally.ERP 9
- ✓ Internet
- ✓ Chatting, E-mail
- ✓ CorelDRAW Graphics Suite X5
- ✓ Photoshop CS5
- ✓ PageMaker 7
- ✓ Social Networking

FREE DVD

Free with This Book
Professor Teaches
MS® Office 2010
PC DVD-ROM

**BPB's
COMPUTER COURSE**

Windows 7 with MS Office 2010

Fundamentals

Networking

Windows 7

MS Office 2010

Word 2010

Excel 2010

PowerPoint 2010

Access 2010

Tally.ERP 9

Internet

Chatting, E-mail

CorelDRAW Graphics Suite X5

Photoshop CS5

PageMaker 7

Social Networking



Prof. Satish Jain

M. Tech. (I.I.T. Kanpur)

M. Geetha

Kratika

By : Prof. Satish Jain
M. Tech. (I.I.T. Kanpur)

ISBN : 978-81-8333-477-8 • Pages : 668

BPB PUBLICATIONS
B-14, CONNAUGHT PLACE, NEW DELHI-110001

Tel: 011-23254990 / 91, 23325760
E-mail : bpbpublications@gmail.com
Visit us at : www.bpbonline.com

Available at All Leading Book Stores

C in Depth

(3rd Revised and updated Edition)

By

S. K. Srivastava
Deepali Srivastava

Naveen chandra



BPB PUBLICATIONS
B-14, CONNAUGHT PLACE, NEW DELHI-110001

REPRINT 2014

FIRST EDITION 2013

Copyright © BPB Publications, INDIA

ISBN : 978-81-8333-048-0

All Rights Reserved. No part of this publication can be stored in any retrieval system, or reproduced in any form or by any means without the prior written permission of the publishers.

LIMITS OF LIABILITY AND DISCLAIMER OF WARRANTY

The Author and Publisher of this book have tried their best to ensure that the programmes, procedures and functions described in the book are correct. However, the author and publishers make no warranty of any kind, expressed or implied, with regard to these programmes or the documentation contained in the book. The author and publishers shall not be liable in any event of any damages, incidental or consequential, in connection with, or arising out of the furnishing, performance or use of these programmes, procedures and functions. Product name mentioned are used for identification purposes only and may be trademarks of their respective companies.

All trademarks referred to in the book are acknowledged as properties of their respective owners.

Distributors:

COMPUTER BOOK CENTRE

12, Shrungar Shopping Centre,
M.G.Road, BENGALURU-560001
Ph: 25587923/25584641

MICRO BOOKS

Shanti Niketan Building,
8, Camac Street, KOLKATA-700017
Ph: 22826518/22826519

BPB PUBLICATIONS

B-14, Connaught Place,
NEW DELHI-110001
Ph: 23325760/43526249

MICRO MEDIA

Shop No. 5, Mahendra Chambers,
150 DN Rd. Next to Capital Cinema,
J.T. (C.S.T.) Station, MUMBAI-400 001
Ph: 22078296/22078297

BPB BOOK CENTRE

376 Old Lajpat Rai Market,
DELHI-110006
Ph: 23861747

DECCAN AGENCIES

4-3-329, Bank Street,
HYDERABAD-500195
Ph: 24756967/24756400

INFOTECH

G-2, Sidhartha Building,
96 Nehru Place,
NEW DELHI-110019
Ph: 26438245

BPB PUBLICATIONS

20, Ansari Road, Darya Ganj
New Delhi-110002
Ph: 23254990/23254991

Dedicated

To the great leader
Subhash Chandra Bose

Published by Manish Jain for BPB Publications, B-14, Connaught Place, New Delhi - 110001
and Printed by him at Rajeev Book Binding House, New Delhi.

Preface

The first two editions of this book were appreciated by the students for their simplicity. This new edition maintains this feature along with inclusion of new topics. Every topic has been explained in depth without compromising over the lucidity of the text and programs. This approach makes this book suitable for both novices and advanced programmers; the well-structured programs are easily understandable by the beginners and useful for the experienced programmers. The book can be used as tool for self-study as it provides step by step explanation and comes with solutions of all exercises. It explains all the basic concepts and doesn't assume that you know how to program.

Development of logic and familiarity with the syntax and features of the language are the two pillars of excellent programming skills. The comprehensive contents of the chapters along with the numerous example programs help you to develop your logic in a stepwise manner and make you feel comfortable with the syntax of the language. Remember that you can't learn swimming by just reading a book on how to swim; you have to jump into water for that, similarly if you want to learn programming, just reading a book is not sufficient, it is essential for you to make your own programs. So start by understanding the programs given in the book, work on them and try to make your own programs.

Each chapter is complemented by exercises with solutions that act as a review of the chapter. We strongly recommend that you solve all the exercises before switching over to another topic. Exercises have been structured in such a way that you can test and implement the knowledge acquired from the chapter, and this is really important for getting full hold over a topic. We are sure if you understand the concepts, you'll enjoy doing the exercises.

New features in the 3rd edition include a chapter on Recursion, thorough explanation of Bitwise Manipulation, new and improved programming examples, lots of new exercises ranging in difficulty, solutions to all the exercises and a CD that includes the code of all the programming examples and exercises. The book contains about 310 well explained programming examples to drive the concepts home and nearly 450 exercises which include many interesting and challenging programming exercises that will help you sharpen your programming skill. Our aim of writing this book is to enable any student emerge as a full-fledged C programmer who can withstand the challenges of the industry. This is the reason for inclusion of chapters on project building, library development and code optimization.

We are thankful to our family and friends for their love and support. We would like to express our gratitude to our publishers for having faith in our work and publishing our books.

If you have any problems or suggestions, please feel free to contact us at- bpb@vsnl.com

Suresh Kumar Srivastava
Deepali Srivastava

CONTENTS

Chapter 1. Introduction 1

- 1.1 Design Methods 1
 - 1.1.1 Top-Down Design 1
 - 1.1.2 Bottom-Up Design 2
 - 1.1.3 Modular Approach 2
- 1.2 Programming Languages 2
- 1.3 Low Level Languages 2
 - 1.3.1 Machine Level Language 2
 - 1.3.2 Assembly Language 2
 - 1.3.3 High-Level Languages 2
- 1.4 Translators 3
- 1.4 History of C 3
- 1.5 Characteristics of C 3
- 1.6 Structure of C Programs 4
- 1.7 Compilation and Execution of C Programs 4
- 1.8 Environment for C 5

Chapter 2. Elements of C 8

- 2.1 C Character Set 8
- 2.2 Escape Sequences/Execution Characters 9
- 2.3 Trigraph Characters 9
- 2.4 Delimiters 9
- 2.5 Reserved Words/ Keywords 10
- 2.6 Identifiers 10
- 2.7 Data Types 10
- 2.8 Constants 12
 - 2.8.1 Numeric Constants 12
 - 2.8.2 Integer Constants 12
 - 2.8.3 Real (floating point) Constants 13
 - 2.8.4 Character Constants 13
 - 2.8.5 String Constants 14
 - 2.8.6 Symbolic Constants 14
- 2.9 Variables 14
 - 2.9.1 Declaration of Variables 15
 - 2.9.2 Initialization of Variables 15
- 2.10 Expressions 15
- 2.11 Statements 15
- 2.12 Expression Statement 16
- 2.13 Compound Statement 16
- 2.14 Comments 16

Chapter 3. Input-Output in C 17

- 3.1 Conversion Specifications 17
- 3.2 Reading Input Data 18
- 3.3 Writing Output Data 21

3.4 Formatted Input and Output	23
3.4.1 Format for Integer Input	23
3.4.2 Format for Integer Output	24
3.4.3 Format for Floating Point Numeric Input	25
3.4.4 Format for Floating Point Numeric Output	25
3.4.5 Format for String Input	25
3.4.6 Format for String Output	26
3.4.7 Suppression Character in scanf()	26
3.5 Character I/O	26
Exercise	27

Chapter 4. Operators and Expressions 29

4.1 Arithmetic Operators	29
4.2 Assignment Operators	31
4.3 Increment and Decrement Operators	31
4.3.1 Prefix Increment / Decrement	32
4.3.2 Postfix Increment / Decrement	32
4.4 Relational Operators	33
4.5 Logical or Boolean Operators	34
4.5.1 AND (&&) Operator	34
4.5.2 OR () Operator	35
4.5.3 Not (!) Operator	35
4.6 Conditional Operator	35
4.7 Comma Operator	36
4.8 sizeof Operator	37
4.9 Bitwise Operators	37
4.10 Type Conversion	38
4.10.1 Implicit Type Conversions	38
4.10.1.1 Automatic Conversions	38
4.10.1.2 Type Conversion in Assignment	39
4.10.2 Explicit Type Conversion or Type Casting	40
4.11 Precedence and Associativity of Operators	41
4.12 Role of Parentheses in Evaluating Expressions	42
4.13 Order of Evaluation of Operands	44
Exercise	45

Chapter 5. Control Statements 49

5.1 Compound Statement or Block	49
5.2 if-else	49
5.2.1 Nesting of if...else	52
5.2.2 Dangling else problem	54
5.2.3 else-if Ladder	54
5.3 Loops	56
5.3.1 while statement	56
5.3.2 do-while statement	60
5.3.3 for statement	62
5.3.4 Nesting of Loops	65
5.3.5 Infinite Loops	66

5.4 break statement	67
5.5 continue statement	68
5.6 goto	70
5.7 switch	71
Some Additional Problems	76
Pyramids	83
Exercise	87

Chapter 6. Functions 96

6.1 Advantages of using Functions	96
6.2 Library Functions	96
6.3 User-Defined Functions	97
6.4 Function Definition	97
6.5 Function Call or Function Invocation	98
6.6 return statement	100
6.7 Function Parameters and Arguments	102
6.8 Order of Evaluation of Function Arguments	103
6.9 Function Declaration	103
6.10 main() Function	108
6.11 Library Functions	108
6.12 Old Style of Function Definition	109
6.13 Old Style of Function Declaration	109
6.14 More about function declaration	110
6.15 Local, Global and Static variables	110
6.15.1 Local Variables	110
6.15.2 Global Variables	111
6.15.3 Static Variables	111
Some Additional Problems	112
Exercise	120

Chapter 7. Recursion 127

7.1 Writing a recursive function	127
7.2 Flow of control in Recursive functions	128
7.3 Winding and unwinding phase	130
7.4 Examples of Recursion	130
7.4.1 Factorial	130
7.4.2 Summation of numbers from 1 to n	132
7.4.3 Displaying numbers from 1 to n	132
7.4.4 Display and Summation of series	134
7.4.5 Sum of digits of an integer and displaying an integer as sequence of characters	135
7.4.6 Base conversion	136
7.4.7 Exponentiation of a float by a positive integer	136
7.4.8 Prime Factorization	137
7.4.9 Greatest Common Divisor	137
7.4.10 Fibonacci Series	138
7.4.11 Checking Divisibility by 9 and 11	139
7.4.12 Tower of Hanoi	140
7.5 Implementation of Recursion	142

- 7.6 Recursion vs. Iteration 143
- 7.7 Tail recursion 143
- 7.8 Indirect and direct Recursion 145
- Exercise 146

Chapter 8. Arrays 150

- 8.1 One Dimensional Array 150
 - 8.1.1 Declaration of 1-D Array 150
 - 8.1.2 Accessing 1-D Array Elements 151
 - 8.1.3 Processing 1-D Arrays 152
 - 8.1.4 Initialization of 1-D Array 153
 - 8.1.5 1-D Arrays and Functions 155
 - 8.1.5.1 Passing Individual Array Elements to a Function 155
 - 8.1.5.2 Passing whole 1-D Array to a Function 155
- 8.2 Two Dimensional Arrays 157
 - 8.2.1 Declaration and Accessing Individual Elements of a 2-D array 157
 - 8.2.2 Processing 2-D Arrays 157
 - 8.2.3 Initialization of 2-D Arrays 158
- 8.3 Arrays with more than two Dimensions 161
 - 8.3.1 Multidimensional Array and Functions 162
- 8.4 Introduction to Strings 162
 - 8.4.1 Input and output of strings 162
- Some Additional Problems 163
- Exercise 182

Chapter 9. Pointers 187

- 9.1 About Memory 187
- 9.2 Address Operator 187
- 9.3 Pointer Variables 188
 - 9.3.1 Declaration of Pointer Variables 188
 - 9.3.2 Assigning Address to Pointer Variables 189
 - 9.3.4 Dereferencing Pointer Variables 189
- 9.4 Pointer Arithmetic 191
- 9.5 Precedence of Dereferencing Operator and Increment/Decrement Operators 194
- 9.6 Pointer Comparisons 195
- 9.7 Pointer to Pointer 195
- 9.8 Pointers and One Dimensional Arrays 196
- 9.9 Subscripting Pointer Variables 199
- 9.10 Pointer to an Array(Array pointer) 200
- 9.11 Pointers and Two Dimensional Arrays 201
- 9.12 Subscripting Pointer to an Array 203
- 9.13 Pointers and Three Dimensional Arrays 204
- 9.14 Pointers and Functions 205
- 9.15 Returning more than one value from a Function 207
- 9.16 Function Returning Pointer 208
- 9.17 Passing a 1-D Array to a Function 209
- 9.18 Passing a 2-D Array to a Function 211
- 9.19 Array of Pointers(Pointer Array) 212

- 9.20 void Pointers 214
- 9.21 Dynamic Memory Allocation 215
 - 9.21.1 malloc() 215
 - 9.21.2 calloc() 216
 - 9.21.3 realloc() 217
 - 9.21.4 free() 218
- 9.22 Dynamic Arrays 219
- 9.23 Pointers to Functions 221
 - 9.23.1 Declaring a Pointer to a Function 222
 - 9.23.2 Calling a Function through Function Pointer 222
 - 9.23.3 Passing a function's address as an argument to other function 223
 - 9.23.4 Arrays of function pointers 224
- Exercise 225

Chapter 10. Strings 233

- 10.1 String Constant (String Literal) 233
- 10.2 String Variables 234
- 10.3 String Library Functions 236
 - 10.3.1 strlen() 236
 - 10.3.2 strcmp() 237
 - 10.3.3 strcpy() 238
 - 10.3.4 strcat() 239
- 10.4 String Pointers 240
- 10.5 Array of Strings or Two Dimensional Array of Characters 242
- 10.6 Array of Pointers to Strings 244
- 10.7 String Library Functions 247
 - 10.7.1 strncpy() 247
 - 10.7.2 strncat() 248
 - 10.7.3 strncmp() 248
 - 10.7.4 strchr() and strchr() 249
 - 10.7.5 strpbrk() 249
 - 10.7.6 strspn() and strspn() 249
 - 10.7.7 strstr() 250
 - 10.7.8 strtok() 250
 - 10.7.9 sprintf() 251
 - 10.7.10 sscanf() 251
- Some Additional Problems 252
- Exercise 263

Chapter 11. Structure and Union 270

- 11.1 Declaring a Structure type 270
- 11.2 Declaring Structure Variables 271
- 11.3 Initialization of Structure Variables 271
- 11.4 Accessing Members of a Structure 272
- 11.5 Assignment of Structure Variables 273
- 11.6 Storage allocation and Size of Structure 273
- 11.7 Array of Structures 274
- 11.8 Arrays within Structures 275

1.9 Nested Structures (Structure within Structure)	276
1.10 Pointers to Structures	277
1.11 Pointers within Structures	278
1.12 Structures and Functions	279
11.12.1 Passing Structure Members as Arguments	279
11.12.2 Passing Structure Variable as Argument	279
11.2.3 Passing Pointers to Structures as Arguments	280
11.2.4 Returning a Structure Variable from Function	281
11.2.5 Returning a Pointer to Structure from a Function	281
11.2.6 Passing Array of Structures as Argument	282
1.3 Self Referential Structures	286
1.4 Linked List	287
11.4.1 Traversing a Linked List	290
11.4.2 Searching in a Linked List	291
11.4.3 Insertion in a Linked List	291
11.4.3.1 Insertion at the beginning of the list	291
11.4.3.2 Insertion in an empty list	292
11.4.3.3 Insertion at the end of the list	293
11.4.3.4 Insertion in between the list nodes	294
11.4.3.4.1 Insertion after a node	294
11.4.3.4.2 Insertion before a node	295
11.4.3.4.3 Insertion at a given position	296
11.4.4 Creation of a Linked List	296
11.4.5 Deletion in a Linked List	297
11.4.5.1 Deletion of first node	297
11.4.5.2 Deletion of the only node	297
11.4.5.3 Deletion in between the list nodes	298
11.4.5.4 Deletion at the end of the list	298
11.4.6 Reversing a Linked List	299
1.5 union	301
1.6 typedef	304
Exercise	306

Chapter 12. Files 310

2.1 Text and Binary Streams	310
2.2 Buffer	311
2.3 Opening a File	311
2.4 Errors in Opening Files	312
2.5 Closing a File	313
2.6 End of File	314
2.7 Structure of a File Program	314
2.8 Standard streams	314
2.9 Character I/O	314
12.9.1 fputc()	314
12.9.2 fgetc()	315
12.9.3 getc() and putc()	316
2.10 String I/O	316
12.10.1 fputs()	316
12.10.2 fgets()	317

12.11 Formatted I/O	318
12.11.1 fprintf()	318
12.11.2 fscanf()	319
12.12 Block Read/Write	320
12.12.1 fwrite()	320
12.12.2 fread()	321
12.13 Random Access to File	322
12.13.1 fseek()	323
12.13.2 ftell()	324
12.13.3 rewind()	325
12.14 Other File Functions	334
12.14.1 feof()	334
12.14.2 ferror()	334
12.14.3 clearerr()	335
12.14.4 perror()	336
12.14.5 rename()	336
12.14.6 remove()	336
12.14.7 fflush()	336
12.14.8 tmpfile()	337
12.14.9 tmpnam()	337
12.14.10 freopen()	337
12.15 Command Line Arguments	337
Some Additional Problems	338
Exercise	343

Chapter 13. The C Preprocessor 346

13.1 #define	346
13.2 Parameterized Macros	348
13.3 Nesting in Macros	349
13.4 Problems with Macros	350
13.5 Macros vs. Functions	352
13.6 Generic Functions	353
13.7 #undef	354
13.8 Stringizing Operator (#)	354
13.9 Token Pasting Operator (##)	355
13.10 Including Files	355
13.11 Conditional Compilation	356
13.11.1 #if and #endif	356
13.11.2 #else and #elif	357
13.11.3 defined Operator	358
13.11.4 #ifdef and #ifndef	359
13.11.5 Writing Portable Code	360
13.11.6 Debugging	361
13.11.7 Commenting a Part of Code	362
13.11.8 Other Uses of conditional compilation	362
13.12 Predefined Macro Names	363
13.12.1 #line	363
13.12.2 #error	364
13.12.3 #pragma	365

Exercise 365

Chapter 14. Operations on Bits 369

- 14.1 Logical bitwise operators (&, |, ^, ~) 370
- 14.2 Shift Operators 371
 - 14.2.1 Multiplication and Division by 2 using shift operators 372
- 14.3 Masking 373
 - 14.3.1 Masking Using Bitwise AND 373
 - 14.3.2 Masking Using Bitwise OR 374
 - 14.3.3 Masking Using Bitwise XOR 374
- Some Additional Problems 376
- 14.4 Bit Fields 389
- Exercise 391

Chapter 15. Miscellaneous Features 398

- 15.1 Enumeration 398
- 15.2 const 401
- 15.3 volatile 402
- 15.4 lvalue and rvalue 403
- 15.5 Scope 403
 - 15.5.1 Block Scope 404
 - 15.5.2 File Scope 404
 - 15.5.3 Prototype Scope 404
 - 15.5.4 Function Scope 404
- 15.6 Storage Classes 405
 - 15.6.1 Automatic 405
 - 15.6.2 Register 407
 - 15.6.3 External 407
 - 15.6.4 Static 409
 - 15.6.4.1 Local Static Variables 410
 - 15.6.4.2 Global Static Variables 411
- 15.7 Storage Class Specifiers in Functions 411
- 15.8 Linkage 412
- 15.9 Memory During Program Execution 412
- 15.10 Functions with Variable Number of Arguments 413
 - 15.10.1 Passing variable number of arguments to another function 417
- Exercise 419

Chapter 16. Building Project and Creation of Library 424

- 16.1 Requirement Analysis 424
- 16.2 Top Level Design 424
- 16.3 Detail Design 425
- 16.4 Coding 428
 - 16.4.1 Dtmnip.h 428
 - 16.4.2 Datefmt.c 428
 - 16.4.3 Valid.c 429

- 16.4.4 Leap.c 429
- 16.4.5 Julian.c 429
- 16.4.6 Weekday.c 430
- 16.4.7 Cmpdate.c 431
- 16.4.8 Diffymd.c 431
- 16.4.9 Diffdays.c 432
- 16.4.10 Addyears.c 432
- 16.4.11 Subyears.c 432
- 16.4.12 Addmonths.c 433
- 16.4.13 Submonths.c 433
- 16.4.14 Adddays.c 433
- 16.4.15 Subdays.c 434
- 16.4.16 Main.c 435
- 16.5 Building Project in Turbo C 438
- 16.6 Testing 438
- 16.7 Creation of Library and using it in your Program in Turbo C 438
- 16.8 Deletion of a Module from Library 439
- 16.9 Getting Modules from Library 439
- 16.10 Changing Version of Module in Library 439
- 16.11 Building Project on Unix 440
- 16.12 Writing Makefile 440
- 16.13 Building Project with Make 442
- 16.14 Creation of Library and Using in your Program in Unix 442

Chapter 17. Code Optimization in C 444

- 17.1 Optimization with Tool 444
- 17.2 Optimization with Loop 444
 - 17.2.1 Loop Unrolling 444
 - 17.2.2 Avoiding Calculations in Loops 445
- 17.3 Fast Mathematics 445
 - 17.3.1 Avoid Unnecessary Integer Division 445
 - 17.3.2 Multiplication and Division by Power of 2 445
 - 17.3.3 Simplifying Expressions 446
- 17.4 Declare prototypes for Functions 446
- 17.5 Better Way of Calling Function 446
- 17.6 Prefer int to char or short 446
- 17.7 Use of Register Variables 446
- 17.8 Optimization with Switch Statement 446
- 17.9 Avoid Pointer Dereference 448
- 17.10 Prefer Pre Increment/Decrement to Post Increment/Decrement 448
- 17.11 Prefer Array to Memory Allocation 448
- 17.12 Expression Order Understanding 448
- 17.13 Declaration of Local Function 449
- 17.14 Breaking Loop with Parallel Coding 449
- 17.15 Trick to use Common Expression 449
- 17.16 Declaring Local Variables Based On Size 450
- 17.17 Prefer Integer Comparison 450
- 17.18 Avoid String Comparison 450

Chapter 18. C and Assembly Interaction 451

- 18.1 Inline Assembly Language 451
- 18.2 Linking of Two Assembly Files 454
- 18.3 Memory Models 455
- 18.4 C and Segments in Library 455
- 18.5 Linking Assembly Procedure in C Program 457

Chapter 19. Library Functions 459

19.1 Mathematical Functions <math.h> 459

- pow() 459
- sqrt() 459
- ceil() 459
- floor() 459
- fmod() 459
- log() 459
- log10() 459
- exp() 460
- ldexp() 460
- frexp() 460
- modf() 460
- sin() 460
- cos() 460
- tan() 460
- asin() 460
- acos() 460
- atan() 460
- atan2() 460
- sinh() 461
- cosh() 461
- tanh() 461
- fabs() 461

19.2 Character Type Functions <ctype.h> 461

- isalnum() 461
- isalpha() 461
- iscentrl() 461
- isdigit() 461
- isgraph() 461
- islower() 461
- isprint() 461
- ispunct() 462
- isspace() 462
- isupper() 462
- isxdigit() 462
- tolower() 462
- toupper() 462

19.3 String Handling Functions <string.h> 462

- strlen() 462

- strcpy() 462
 - strncpy() 462
 - strcat() 463
 - strncat() 463
 - strchr() 463
 - strchr() 463
 - strcmp() 464
 - strncmp() 464
 - strspn() 464
 - strspn() 464
 - strpbrk() 464
 - strstr() 465
 - strtok() 465
- ### 19.4 Input/Output Functions <stdio.h> 465
- fopen() 465
 - freopen() 465
 - fclose() 465
 - clearerr() 466
 - feof() 466
 - ferror() 466
 - fflush() 466
 - fputc() 466
 - fgetc() 466
 - fgets() 466
 - fputs() 466
 - fprintf() 467
 - fscanf() 467
 - fread() 467
 - fwrite() 467
 - fseek() 467
 - ftell() 467
 - rewind() 468
 - remove() 468
 - rename() 468
 - tmpfile() 468
 - tmpnam() 468
 - setbuf() 468
 - perror() 468
 - sprintf() 468
 - sscanf() 469
- ### 19.5 Utility functions <stdlib.h> 469
- malloc() 469
 - calloc() 469
 - realloc() 469
 - free() 469
 - abs() 469
 - labs() 469
 - rand() 469
 - srand() 469
 - atoi() 470
 - atol() 470

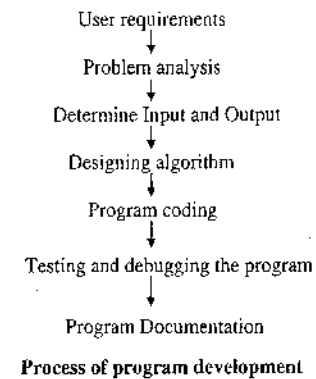
atof() 470
 atexit() 470
 exit() 470

Solutions 471

Index 494

Introduction

Software is a collection of programs and a program is a collection of instructions given to the computer. Development of software is a stepwise process, before developing a software number of processes are done. The first step is to understand the user requirements. Problem analysis arises during the requirement phase of software development. Problem analysis is done for obtaining the user requirements and to determine the input and output of the program. For solving any problem, an "algorithm" is implemented. Algorithm is a sequence of steps that gives method of solving a problem. This algorithm creates the logic of program. On the basis of the algorithm, program code is written. Program testing is performed to make sure that it gives correct results. Proper documentation of the program should be done so that it is easier to modify or enhance the program whenever required.



1.1 Design Methods

Designing is the first step for obtaining solution of a given problem. The purpose of designing is to represent a solution for a system. It is difficult to design a large system because the complexity of system cannot be represented easily. Thus various methods have been evolved for designing.

1.1.1 Top-Down Design

Every system has several hierarchies of components. The top-level component represents the whole system. Top-Down design method starts from top-level component to lowest level (bottom) component. In this design method, the system is divided into some major components. Then each major component is divided into lower level components. Similarly other components are divided till the lowest level component.

1.1.2 Bottom-Up Design

Bottom-Up design method is the reverse of Top-Down approach. It starts from the lowest level component to the highest-level component. It first designs the basic components and from these basic components the higher-level components are designed.

1.1.3 Modular Approach

It is better to divide a large system into modules. In terms of programming, module is logically a well-defined part of program. Each module is a separate part of the program. It is easy to modify a program written with modular approach because changes in one module don't affect other modules of program. It is also easy to check bugs in the program in module level programming.

1.2 Programming Languages

Before learning any language, it is important to know about the various types of languages and their features. It is interesting to know what the basic requirements of the programmers were and what difficulties they faced with the existing languages. The programming languages can be classified into two types-

1. Low level languages
2. High level languages

1.3 Low Level Languages

The languages in this category are the Machine level language and Assembly language.

1.3.1 Machine Level Language

Computers can understand only digital signals, which are in binary digits i.e. 0 and 1. So the instructions given to the computer can be only in binary codes. The machine language consists of instructions that are in binary 0 or 1. Computers can understand only machine level language.

Writing a program in machine level language is a difficult task because it is not easy for programmers to write instructions in binary code. A machine level language program is error-prone and its maintenance is very difficult. Furthermore machine language programs are not portable. Every computer has its own machine instructions, so the programs written for one computer are not valid for other computers.

1.3.2 Assembly Language

The difficulties faced in machine level language were reduced to some extent by using a modified form of machine level language called assembly language. In assembly language instructions are given in English like words, such as MOV, ADD, SUB etc. So it is easier to write and understand assembly programs. Since a computer can understand only machine level language, assembly language program must be translated into machine language. The translator that is used for translating is called "assembler".

Although writing programs in assembly language is a bit easier, the programmer has to know all the low level details related with the hardware of a computer. In assembly language, data is stored in computer registers and each computer has different set of registers. Thus the assembly language program is also not portable. Since the low level languages are related with the hardware, the execution of a low-level program is faster.

1.3.3 High-Level Languages

High-level languages are designed keeping in mind the features of portability i.e. these languages are machine independent. These are English like languages, so it is easy to write and understand the programs of high-level language. While programming in a high level language, the programmer is not concerned with the low level details, and so the whole attention can be paid to the logic of the problem being solved. For translating a high-

level language program into machine language, compiler or interpreter is used. Every language has its own compiler or interpreter. Some languages in this category are- FORTRAN, COBOL, BASIC, Pascal etc.

1.4 Translators

We know that computers can understand only machine level language, which is in binary 1 or 0. The code written in any high-level or low-level languages should be translated into machine level language and translators are used for this purpose. These translators are just computer programs, which accept a program written in high level or low-level language and produce an equivalent machine language program as output. The three types of translators used are-

- Assembler
- Compiler
- Interpreter

Assembler is used for converting the code of low-level language (assembly language) into machine level language.

Compilers and interpreters are used to convert the code of high-level language into machine language. The high level program is known as source program and the corresponding machine language program is known as object program. Although both compilers and interpreters perform the same task, there is a difference in their working.

A compiler searches all the errors of program and lists them. If the program is error free then it converts the code of program into machine code and then the program can be executed by separate commands. An interpreter checks the errors of program statement by statement. After checking one statement, it converts the statement into machine code and then executes that statement. This process continues until the last statement of program or an erroneous statement occurs.

1.4 History of C

In earlier days, every language was designed for some specific purpose. For example FORTRAN (Formula Translator) was used for scientific and mathematical applications, COBOL(Common Business Oriented Language) was used for business applications. So need of such a language was felt which could be useful for most of the purposes. The C language was developed by Dennis Ritchie in 1970s at AT&T Bell laboratories Murray Hill, New Jersey. Initially it was designed for programming in the operating system called UNIX. After the advent of C, the whole UNIX operating system was rewritten using it. Now almost the entire UNIX operating system and the tools supplied with it including the C compiler itself are written in C.

The C language is derived from the B language, which was written by Ken Thompson at AT&T Bell laboratories. The B language was adopted from a language called BCPL (Basic Combined Programming Language), which was developed by Martin Richards at Cambridge University.

In 1982 a committee was formed by ANSI(American National Standards Institute) to standardize the C language. Finally in 1989, the standard for C language was introduced known as ANSI C. Generally most of the modern compilers conform to this standard.

1.5 Characteristics of C

C is a middle level language. It has the simplicity of a high level language as well as the power of a low level language. This aspect of C makes it suitable for writing both application programs and system programs. Thus it is an excellent, efficient and general-purpose language for most of the applications such as mathematical, scientific, business and system software applications.

C is small language, consisting of only 32 English words known as keywords(if, else, for, break etc). The power of C is augmented by the library functions provided with it. Moreover, the language is extendible since it allows the users to add their own library functions to the library.

C contains control constructs needed to write a structured program so it is considered a structured programming language. It includes structures for selection (if...else, switch), repetition (while, for, do...while) and for loop exit (break). C provides bitwise operators for bit level manipulation. This makes it suitable for system programming.

1.6 Structure of C Programs

A C program is a collection of one or more functions. Every function is a collection of statements and performs some specific task. The general structure of a C program is-

```

Comments
Preprocessor directives
Global variables
int main(void)
{
    local variables
    statements
    .....
    return 0;
}
func1()
{
    local variables
    statements
    .....
}
func2()
{
    local variables
    statements
    .....
}

```

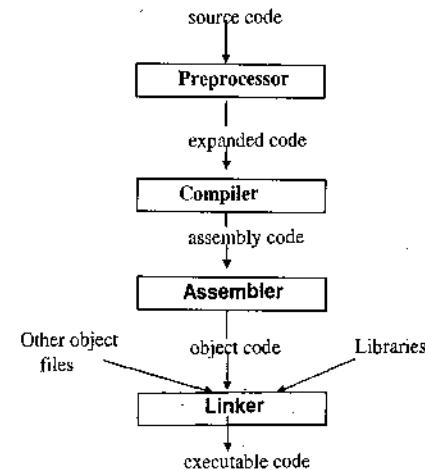
Comments can be placed anywhere in a program and are enclosed between the delimiters `/*` and `*/`. Comments are generally used for documentation purposes.

Preprocessor directives are processed through the preprocessor before the C source code passes through the compiler. The commonly used preprocessor directives are `#include` and `#define`. `#include` is used for including header files. `#define` is used to define symbolic constants and macros.

Every C program has one or more functions. If a program has only one function then it must be `main()`. Execution of every C program starts with `main()` function. It has two parts, declaration of local variables and statements. Statements in the `main()` function are executed one by one. The statement `return 0;` at the end of `main()` means that the function has terminated successfully. Other functions are user-defined functions, which also have local variables and C statements. They can be defined before or after `main()`. It may be possible that some variables have to be used in many functions, so it is necessary to declare them globally. These variables are called global variables.

1.7 Compilation and Execution of C Programs

There are different phases through which our program passes before being transformed into an executable form. The following figure shows these phases-



Preprocessor

The source code is the code that we write using any text editor and the source code file is given an extension `.c`. This source code is firstly passed through the preprocessor. The preprocessor expands this code and passes it to the compiler. We have discussed the role of preprocessor in chapter 13.

Compiler

The expanded code is passed to the compiler. The compiler converts this code into the machine's assembly language code.

Assembler

This assembly language code is converted to object code by the system's assembler. The name of object file is same as that of source file. In DOS the object file has extension `.obj` and in UNIX the extension is `.o`.

Linker

Generally all programs written in C, use library functions. Library functions are precompiled and their object code is stored in library files with `.lib` (or `.a`) extension. The linker combines this object code of the library functions with the object code of our program. Our program may also contain references to functions that are defined in other files. The linker links the object code of these files also to our program. So the job of the linker is to combine the object code of our program with the object code of other files and object code of library functions. The output of the linker is an executable file. In DOS the executable file has same name as that of source code file and has extension `.exe`, and in UNIX the executable file is named as `a.out` or the name of output with `-o` option.

1.8 Environment for C

The steps involved in developing a C program are-

1. Program creation
2. Program compilation
3. Program execution

6 Unix/Linux Environment

Generally a command line C compiler is provided with the Unix/Linux operating system. This compiler is named cc or gcc.

a) Program creation

In unix environment, file can be created with vi editor as-
\$ vi filename.c

Here \$ is the unix prompt. The file can be saved by pressing ESC and SHIFT+zz.

b) Program compilation

After creation of C program, it can be compiled as-

\$cc filename.c

If the program has mathematical function then it is compiled as-
\$cc filename.c -lm

After compilation, the executable code is stored in the file a.out.

Another way of compilation providing executable file name is-

\$cc filename.c -o executablename

After compilation, the executable code is stored in the file executablename.

c) Program execution-

After the compilation of program, it can be executed as-

\$ a.out

If compiled with -o option then it can be executed as-

\$ executablename

MS-DOS Environment

In MS-DOS environment creation, compilation and execution can be done using command line or IDE (Integrated Development Environment).

Command Line

In Borland C, the command line compiler is bcc.exe and in Turbo C the command line compiler is tcc.exe.

a) Program creation

The program file can be created using any editor and should be saved with .c extension

b) Program compilation

After saving the file, C program can be compiled at DOS prompt by writing-

\>tcc filename (in Turbo C)

\>bcc filename (in Borland C)

c) Program execution

After compilation of C program, the executable file filename.exe is created. It is executed at DOS prompt by writing-

\>filename

Integrated Development Environment

All these steps can be performed in an IDE using menu options or shortcut keys. In Borland C the program bcc.exe is the IDE and in Turbo C the program tcc.exe is the IDE. So we can open the IDE by typing bc or tc at the command prompt.

a) Program creation

A new file can be created from menu option New. The file can be saved by menu option Save. If the file is unnamed then it is saved by menu option Save as. An existing file can be opened from the menu option Open.

b) Program compilation

The file can be compiled by the menu option Compile.(Alt+F9)

c) Program execution

The file can be executed by the menu option Run.(Ctrl+F9). The output appears in the output window that can be seen using the keys Alt+F5.

We have given you just a preliminary knowledge of how to execute your programs. There are several other options that you can explore while working and it is best to check the manual of your compiler to know about these options. Similarly other command lines and IDEs have their own commands and menus options.

Every language has some basic elements and grammatical rules. Before programming in C, it is must to know the basic elements of the language. These basic elements are character set, variables, datatypes, constants, keywords (reserved words), variable declaration, expressions, statements etc. All of these are used to construct a C program.

2.1 C Character Set

The characters that are used in C programs are given below-

Uppercase and Lowercase Letters

A, B, C Z

a, b, c z

Digits

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Graphic characters

character	Name	character	Name
+	plus sign	-	minus sign(hyphen)
*	asterisk	%	percent sign
\	backward slash	/	forward slash
<	less than sign	=	equal to sign
>	greater than sign	_	underscore
(	left parenthesis	)	right parenthesis
{	left braces	}	right braces
[	left bracket	]	right bracket
,	comma	.	period
'	single quotes	"	double quotes
:	colon	;	semicolon
?	question mark	!	exclamation sign
&	ampersand		vertical bar
~	tilde sign	^	caret sign
#	hash		

Whitespace Characters

Space character, newline character, horizontal tab, vertical tab, and form feed.

Other characters

Alert, backspace, carriage return and null character.

Elements of C

2.2 Escape Sequences/Execution Characters

Some characters such as newline, tab, backspace cannot be displayed like other normal characters. C supports the combination of backslash(\) and some characters from the C character set for representing these characters in our C programs. These character combinations are known as escape sequences and are represented by backslash(\) followed by a character. The first character is " \ " and second character is from the C character set. Some escape sequences are given below-

Escape sequence	Meaning	ASCII Value	Purpose
\b	backspace	008	Moves the cursor to the previous position of the current line
\a	alert	007	Produces an audible or visible alert
\r	carriage return	013	Moves the cursor to beginning of the current line.
\n	newline	010	Moves the cursor to the beginning of the next line
\f	form feed	012	Moves the cursor to the initial position of the next logical page.
\0	null character	000	Used for termination of character string
\v	vertical tab	011	Moves the cursor to next vertical tab position
\t	Horizontal tab	009	Moves the cursor to the next horizontal tab position.
\\	backslash	092	Presents a character with backslash (\)

Blank, horizontal tab, vertical tab, newline, carriage return, form feed are known as whitespace in C language.

2.3 Trigraph Characters

Some keyboards may not have all the characters from the C character set. C supports the facility of "trigraph sequence" to print these characters. These trigraph sequences have three characters. First two are "?? " and third character is any character from C character set. Some trigraph sequences are as given below-

Trigraph sequence	Symbol
??<	{ left brace
??>	} right brace
??(	[left bracket
??)	] right bracket
??!	vertical bar
??\	\ backslash
??#	# hash sign
??~	~ tilde
??^	^ caret

2.4 Delimiters

Delimiters are used for syntactic purpose in C.

:	colon	used for label
;	semicolon	end of statement
()	parentheses	used in expressions
[]	square brackets	used for arrays

{ }	curly braces	used for block of statements
#	hash	preprocessor directive
,	comma	variable delimiter

2.5 Reserved Words / Keywords

There are certain words that are reserved for doing specific tasks. These words are known as keywords and they have standard, predefined meaning in C. They are always written in lowercase. There are only 32 keywords available in C which are given below-

auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

2.6 Identifiers

All the words that we will use in our C programs will be either keywords or identifiers. Keywords are predefined and can't be changed by the user, while identifiers are user defined words and are used to give names to entities like variables, arrays, functions, structures etc. Rules for naming identifiers are given below-

- (1) The name should consist of only uppercase and lower case letters, digits and underscore sign(_).
- (2) First character should be an alphabet or underscore.
- (3) The name should not be a keyword.
- (4) Since C is case sensitive, the uppercase and lowercase letters are considered different. For example code, Code and CODE are three different identifiers.
- (5) An identifier name may be arbitrarily long. Some implementations of C recognize only the first eight characters, though most implementations recognize 31 characters. ANSI standard compilers recognize 31 characters.

The identifiers are generally given meaningful names. Some examples of valid identifier names-

Value	a	net_pay	rec1	_data	MARKS
-------	---	---------	------	-------	-------

Some examples of invalid identifier names are-

abc	First character should be an alphabet or underscore
int	int is a keyword
rec#	# is a special character
avg no	blank space is not permitted

2.7 Data Types

A data type defines a domain of allowed values and the operations that can be performed on those values. Storage representation of different data types is different in memory. There are four fundamental datatypes in C, which are int, char, float and double.

char is used to store any single character, int is used to store integer value, float is used for storing single precision floating point number and double is used for storing double precision floating point number. We can use type qualifiers with these basic types to get some more types.

There are two types of type qualifiers-

1. Size qualifiers - short, long
2. Sign qualifiers - signed, unsigned

The qualifiers signed and unsigned can be applied to char and integer types. When the qualifier unsigned is used the number is always positive, and when signed is used number may be positive or

negative. If the sign qualifier is not mentioned in integers, then by default signed qualifier is assumed. If the sign qualifier is not mentioned for char type, then whether the char type is signed or unsigned is machine dependent. The range of values for signed data types is less than that of unsigned type. This is because in signed type, the leftmost bit is used to represent the sign, while in unsigned type this bit is also used to represent the value.

The qualifiers short and long can be applied to int type to get types short int and long int. The qualifier long can be applied to double to get type long double which stores extended precision floating point number.

The size and range of different data types on a 16-bit machine is given in the following table. The size and range may vary on machines with different word sizes.

Basic data types	Data types with type qualifiers	Size(bytes)	Range
char	signed char	1	-128 to 127
	unsigned char	1	0 to 255
int	int or signed int	2	-32768 to 32767
	unsigned int	2	0 to 65535
	short int or signed short int	1	-128 to 127
	unsigned short int	1	0 to 255
	long int or signed long int	4	-2147483648 to 2147483647
	unsigned long int	4	0 to 4294967295
float	float	4	3.4E-38 to 3.4E+38
double	double	8	1.7E-308 to 1.7E+308
	long double	10	3.4E-4932 to 1.1E+4932

The standard specifies that char should be at least 8 bits, short int at least 16 bits and long int at least 32 bits. The type short int should not be longer than int and int should not be longer than long int. The signed and unsigned versions of each type have same size.

If int takes 2 bytes then the range is -2^{15} to $2^{15}-1$ (-32768 to 32767), and if it takes 4 bytes then the range will be -2^{31} to $2^{31}-1$ (-2147483648 to 2147483647). Similarly the range for other types may also vary depending upon their size.

All character and integer types are known as integral types, and the types float, double, long double are known as floating point type. The size and range of different data types is machine and compiler dependent. The range of integral types is given in the header file limits.h, and the range and precision for floating point types is given in header file float.h. We can use the sizeof() operator to find out the size of various data types or our system. To find out the range of values we can see the macros defined in header files limits.h and float.h. The following program can be run to get the size and range of some of the types.

```
/*P2.1 Program to find out the size and limits of data types*/
#include<stdio.h>
#include<limits.h>
#include<float.h>
int main(void)
{
    printf("sizeof(char) = %u\n", sizeof(char));
    printf("sizeof(short) = %u\n", sizeof(short));
    printf("sizeof(int) = %u\n", sizeof(int));
    printf("sizeof(long) = %u\n", sizeof(long));
    printf("sizeof(float) = %u\n", sizeof(float));
    printf("sizeof(double) = %u\n", sizeof(double));
    printf("sizeof(long double) = %u\n", sizeof(long double));

    printf("SCHAR_MIN = %d\n", SCHAR_MIN);
    printf("SCHAR_MAX = %d\n", SCHAR_MAX);
    printf("UCHAR_MAX = %d\n", UCHAR_MAX);

    printf("SHRT_MIN = %d\n", SHRT_MIN);
```

```

printf("SHRT_MAX = %d\n", SHRT_MAX);
printf("USHRT_MAX = %u\n", USHRT_MAX);

printf("INT_MIN = %d\n", INT_MIN);
printf("INT_MAX = %d\n", INT_MAX);
printf("UINT_MAX = %u\n", UINT_MAX);

printf("LONG_MIN = %ld\n", LONG_MIN);
printf("LONG_MAX = %ld\n", LONG_MAX);
printf("ULONG_MAX = %lu\n", ULONG_MAX);

printf("FLT_MIN = %e\n", FLT_MIN);
printf("FLT_MAX = %e\n", FLT_MAX);

printf("DBL_MIN = %e\n", DBL_MIN);
printf("DBL_MAX = %e\n", DBL_MAX);

printf("LDBL_MIN = %e\n", LDBL_MIN);
printf("LDBL_MAX = %e\n", LDBL_MAX);

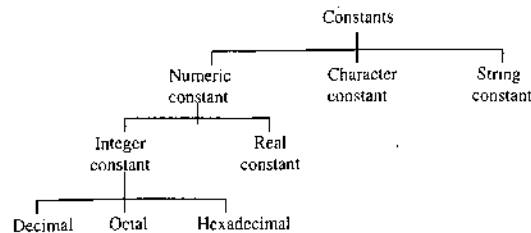
/*Number of digits of precision*/
printf("FLT_DIG = %d\n", FLT_DIG);
printf("DBL_DIG = %d\n", DBL_DIG);
printf("LDBL_DIG = %d\n", LDBL_DIG);

return 0;
}

```

2.8 Constants

Constant is a value that cannot be changed during execution of the program.



2.8.1 Numeric Constants

Numeric constants consist of numeric digits, they may or may not have decimal point (.). These are the rules for defining numeric constants-

1. Numeric constant should have at least one digit.
2. No comma or space is allowed within the numeric constant.
3. Numeric constants can either be positive or negative but default sign is always positive.

2.8.2 Integer Constants

There are three types of integer constants based on different number systems(decimal, octal, hexadecimal). The permissible characters that can be used in these constants are-

Decimal constants - 0,1,2,3,4,5,6,7,8,9 (base 10)

Octal constants - 0,1,2,3,4,5,6,7 (base 8)

Hex decimal constants - 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F,a,b,c,d,e,f (base 16)

Elements of C

Some valid decimal integer constants are-

```

0
123
3705
23759

```

Some invalid decimal integer constants are-

Invalid	Remark
2.5	illegal character (.)
3#5	illegal character (#)
98 5	No blank space allowed
0925	First digit cannot be zero
8,354	Comma is not allowed

In octal integer constants, first digit must be 0. Some examples are-

```

0
05
077
0324

```

In hexadecimal integer constants, first two characters should be 0x or 0X. Some examples are-

```

0X23
0x515
0XA15B
0xFFf
0xac

```

By default the type of an integer constant is int. But if the value of integer constant exceeds the range of int, the type is taken to be unsigned int or long int. We can also explicitly mention the type of the constant by suffixing it with l or L (for long), u or U (for unsigned), ul or UL (for unsigned long).

For example-

6453		Integer constant of type int
45238722UL	or	45238722ul
		Integer constant of type unsigned long int
66550	or	6655u
		Integer constant of type unsigned int

2.8.3 Real (floating point) Constants

Floating point constants are numeric constants that contain decimal point. Some valid floating point constants are-

```

0.5
5.3
4000.0
0.0073
5597.89

```

For expressing very large or very small real constants, exponential (scientific) form is used. Here the number is written in the mantissa and exponent form, which are separated by 'e' or 'E'. The mantissa can be an integer or a real number, while the exponent can be only an integer (positive or negative). For example the number 1800000 can be written as 1.8e6, here 1.8 is mantissa and 6 is the exponent. Some more examples are as-

Number		Exponential form
2500000000	→	2.5*10 ⁹
0.0000076	→	7.6*10 ⁻⁶
-670000	→	-6.7*10 ⁵

By default the type of a floating point constant is double. We can explicitly mention the type of constant by suffixing it with a f or F (for float type), l or L (for long double). For example-

2.3e5 floating point constant of type double

2.4e-9l or 2.4e-9L floating point constant of type long double
 3.52f or 3.52F floating point constant of type float

2.8.4 Character Constants

A character constant is a single character that is enclosed within single quotes. Some valid character constants are-

'g' 'D' 'S' ' ' '#'

Some invalid character constants are-

Invalid	Remark
'four'	There should be only one character within quotes
"d"	Double quotes are not allowed
' '	No character between single quotes
y	Single quotes missing

Every character constant has a unique integer value associated with it. This integer is the numeric value of the character in the machine's character code. If the machine is using ASCII (American Standard Code for Information Interchange), then the character 'G' represents integer value 71 and the character 'S' represents value 83. Some ASCII values are-

A - Z	ASCII value (65 - 90)
a - z	ASCII value (97 - 122)
0 - 9	ASCII value (48 - 57)
;	ASCII value (59)

2.8.5 String Constants

A string constant is enclosed within double quotes (""). At the end of string, the null character '\0' is automatically placed by the compiler. Some examples of string constants are-

"Kumar"
 "593"
 "8"
 " "
 "A"

Note that "A" and 'A' are different, the first one is a string constant which consists of character A and '\0' while the second one is a character constant which represents integer value 65 (in ASCII).

2.8.6 Symbolic Constants

If we want to use a constant several times, we can provide it a name. For example if we have to use the constant 3.14159265 at many places in our program, we can give it a name PI and use this name instead of writing the constant value everywhere. These types of constants are called symbolic constants or named constants.

A symbolic constant is a name that substitutes for a sequence of characters. The characters may represent a numeric constant, a character constant or a string constant. These constants are generally defined at the beginning of the program as-

```
#define name value
```

Here name is the symbolic name for the constant, and is generally written in uppercase letters. value can be numeric, character or string constant.

Some examples of symbolic constants are-

```
#define MAX 100
#define PI 3.14159265
#define CH 'y'
#define NAME "Suresh"
```

In the program, these names will be replaced by the corresponding values. These symbolic constants improve the readability and modifiability of the program.

2.9 Variables

Variable is a name that can be used to store values, it can take different values but one at a time. These values can be changed during execution of the program. A data type is associated with each variable and it decides what values the variable can take. The rules for naming variables are same as that for naming identifiers.

2.9.1 Declaration of Variables

It is must to declare a variable before it is used in the program. Declaration of a variable specifies its name and datatype. The type and range of values that a variable can store depends upon its datatype. The syntax of declaration of a variable is-

```
datatype variablename;
```

Here datatype may be int, float, char, double etc. Some examples of declaration of variables are-

```
int x;
float salary;
char grade;
long y;
short z;
```

Here x is a variable of type int, salary is a variable of type float, and grade is a variable of type char. The data type short int can be written as only short and long int can be written as only long. So y is declared as a variable of type long int and z is declared as a variable of type short int. We can also declare more than one variable in a single declaration. For example-

```
int x,y,z,total;
```

Here x, y, z and total are all variables of type int.

2.9.2 Initialization of Variables

When a variable is declared it contains undefined value commonly known as garbage value. If we want we can assign some initial value to the variable during the declaration itself, this is called initialization of the variable.

For example-

```
int a=5;
float x=0.9,y=10.5;
char ch='y';
double num=0.15197e-7;
int l,m,n,total=0;
```

In the last declaration, only the variable total is initialized.

2.10 Expressions

An expression is a combination of operators, constants, variables and function calls. An expression can be arithmetic, logical or relational.

Some examples of expressions are-

x+y	- arithmetic operation
a=b+c	- uses two operators (=) and (+)
a>b	- relational expression
a==b	- logical expression
func(a,b)	- function call

We will study about these operators and expressions in the next chapter.

2.11 Statements

In a C program, instructions are written in the form of statements. A statement is an executable part of the program and causes the computer to carry out some action. Statements can be categorized as-

- (i) Expression statements
- (ii) Compound statements
- (iii) Selection statements (if, if...else, switch)
- (iv) Iterative statements (for, while, do...while)
- (v) Jump statements (goto, continue, break, return)
- (vi) Label statements (case, default, label statement used in goto)

2.12 Expression Statement

Expression statement consists of an expression followed by a semicolon. For example-

```
x=5;
x=y+z;
func(a, b);
```

A statement that has only a semicolon is known as null statement.

2.13 Compound Statement

A compound statement consists of several statements enclosed within a pair of curly braces { }. Compound statement is also known as block of statements. Note that there is no semicolon after the closing brace. For example-

```
{
    int l=4, b=2, h=3;
    int area, volume;
    area=2*(l*b+b*h+h*l);
    volume=l*b*h;
}
```

If any variable is to be declared inside the block then it can be declared only at the beginning of the block. The variables that are declared inside a block can be used only inside that block.

All other categories of statements are discussed in further chapters.

2.14 Comments

Comments are used for increasing readability of the program. They explain the purpose of the program and are helpful in understanding the program. Comments are written inside the delimiters /* and */. There can be single line or multiple line comments. We can write comments anywhere in a program except inside a string constant or a character constant. Some examples of comments are-

```
/*Variable b represents basic salary*/ (single line comment)
/*This is a C program to calculate (multiple line comment)
   simple interest */
```

The closing delimiter */ can't be used inside a comment, as a result comments can't be nested i.e. we can't write a comment inside another comment.

Input-Output in C

There are three main functions of any program: it takes data as input, processes this data and gives output. The input operation involves movement of data from an input device (generally keyboard) to computer memory while in output operation the data moves from computer memory to the output device (generally screen). C language does not provide any facility for input-output operations. The input/output is performed through a set of library functions that are supplied with every C compiler. These functions are formally not a part of the language but they are considered standard for all input-output operations in C. The set of library functions that performs input-output operations is known as standard I/O library.

There are several header files that provide necessary information in support of the various library functions. These header files are entered in the program using the #include directive at the beginning of the program. For example if a program uses any function from the standard I/O library, then it should include the header file stdio.h as-

```
#include<stdio.h>
```

Similarly there are other header files like math.h, string.h, stdlib.h that should be included when certain library functions are used.

In this chapter we will discuss about the input functions scanf() and getchar() and the output functions printf() and putchar(). There are several other input-output functions that will be discussed in other chapters.

A simple method of providing value to variables is by assignment statement. For example-

```
int basic = 2000;
char ch = 'y';
```

The second method is to use the input function scanf(), which takes the input data from the keyboard. In this method we can give any value to the variables at run time. For output, we use the function printf().

3.1 Conversion Specifications

The functions scanf() and printf() make use of conversion specifications to specify the type and size of data. Each conversion specification must begin with a percent sign (%). Some conversion specifications are given below-

Conversion Specifications for printf()

Conversion Character	Meaning
%c	print a single character
%d, %i	print a decimal integer
%u	print an unsigned decimal integer
%o	print an unsigned octal integer
%x	print an unsigned hexadecimal integer using a,b,c,d,e,f
%X	print an unsigned hexadecimal integer using A,B,C,D,E,F
%f	print a floating point number (by default 6 digits printed after the point)

%e	-	print a floating point number in exponential format.
%E	-	same as %e, but it prints E for exponent
%g	-	print a floating point number in %f or %e format whichever is shorter
%G	-	print a floating point number in %f or %E format whichever is shorter
%s	-	print a string
%%	-	print a % sign
%p	-	print a pointer in hexadecimal format

Conversion Specifications for scanf()

Conversion Character	Meaning
%c	- read a single character
%d	- read a signed decimal integer
%i	- read a signed integer, base is decided by the prefix of the input number read a signed hexadecimal integer if prefix is 0x read a signed octal integer if prefix is 0 otherwise read a signed decimal integer
%u	- read an unsigned decimal integer
%o	- read an unsigned octal integer
%x, %X	- read an unsigned hexadecimal integer
%f	- read a floating point number
%e, %E	- read a floating point number
%g, %G	- read a floating point number
%s	- read a string

The modifier `h` can be used before conversion specifications `d`, `i`, `o`, `u`, `x`, `X` to specify short integer and the modifier `l` can be used before them to specify a long integer. The modifier `l` can be used before conversion specifications `f`, `e`, `E`, `g`, `G` to specify double while modifier `L` can be used before them to specify a long double. For example `%ld`, `%hd`, `%Lf`, `%hX` are valid conversion specifications.

3.2 Reading Input Data

Input data can be entered into the memory from a standard input device (keyboard). The `scanf()` library function can be used for entering input data. This function can take all types of values (numeric, character, string) as input. The `scanf()` function can be written as-

```
scanf("control string", address1, address2, ...);
```

This function should have at least two parameters. First parameter is a control string, which contains conversion specification characters. It should be within double quotes. The conversion specification characters may be one or more; it depends on the number of variables we want to input. The other parameters are addresses of variables. In the `scanf()` function at least one address should be present. The address of a variable is found by preceding the variable name by an ampersand (&) sign. This sign is called the address operator and it gives the starting address of the variable name in memory. A string variable is not preceded by an & sign to get the address. Some examples of `scanf()` function are-

```
#include<stdio.h>
int main(void)
{
    int marks;
    .....
    scanf("%d", &marks);
    .....
}
```

In this example, the control string contains only one conversion specification character `%d`, which implies that one integer value should be entered as input. This entered value will be stored in the variable `marks`.

```
#include<stdio.h>
int main(void)
{
    char ch;
    .....
    scanf("%c", &ch);
    .....
}
```

Here the control string contains conversion specification character `%c`, which means that a single character should be entered as input. This entered value will be stored in the variable `ch`.

```
#include<stdio.h>
int main(void)
{
    float height;
    .....
    scanf("%f", &height);
    .....
}
```

Here the control string contains the conversion specification character `%f`, which means that a floating point number should be entered as input. This entered value will be stored in the variable `height`.

```
#include<stdio.h>
int main(void)
{
    char str[30];
    .....
    scanf("%s", str);
    .....
}
```

In this example control string has conversion specification character `%s` implying that a string should be taken as input. Note that the variable `str` is not preceded by ampersand (&) sign. The entered string will be stored in the variable `str`. More than one value can also be entered by single `scanf()` function. For example-

```
#include<stdio.h>
int main(void)
{
    int basic, da;
    .....
    scanf("%d%d", &basic, &da);
    .....
}
```

Here the control string has two conversion specification characters implying that two integer values should be entered. These values are stored in the variables `basic` and `da`. The data can be entered with space as the delimiter.

1500 1200

```
#include<stdio.h>
int main(void)
{
    int basic;
    float hra;
    char grade;
    .....
    scanf("%d%f%c", &basic, &hra, &grade);
    .....
}
```

Here the control string has three conversion specifications characters `%d`, `%f` and `%c`, which means that one integer value, one floating point value and one single character can be entered as input. These values are stored in the variables `basic`, `hra` and `grade`. The input data can be entered as-

1500 200.50 A

When more than one values are input by `scanf()`, these values can be separated by whitespace characters like space, tab or newline (default). A specific character can also be placed between two conversion specification characters as a delimiter.

```
#include<stdio.h>
int main(void)
{
    int basic;
    float hra;
    .....
    scanf("%d:%f",&basic,&hra);
    .....
}
```

Here the delimiter is colon (:). The input data can be entered as-
1500:200.50

The value 1500 is stored in variable `basic` and 200.50 is stored in `hra`.

```
#include<stdio.h>
int main(void)
{
    int basic;
    float hra;
    .....
    scanf("%d,*f",&basic,&hra);
    .....
}
```

Here the delimiter is comma (,). The input data can be entered as-
1500, 200.40

```
#include<stdio.h>
int main(void)
{
    int day,month,year;
    int basic;
    .....
    scanf("%d-%d-%d",&day,&month,&year);
    scanf("%d",&basic);
    .....
}
```

Suppose the data is entered as-
24-5-1973
\$3000

24 is stored in variable `day`, 5 is stored in variable `month` and 1973 is stored in variable `year` and 3000 is stored in variable `basic`. If we include any spaces between the conversion specifications inside the control string, then they are just ignored.

```
#include<stdio.h>
int main(void)
{
    int x,y,z;
    .....
    scanf("%d %d %d",&x,&y,&z);
    .....
}
```

If the data is entered as-

12 34 56

Then 12 is stored in `x`, 34 is stored in `y` and 56 is stored in `z`.

3.3 Writing Output Data

Output data can be written from computer memory to the standard output device (monitor) using `printf` library function. With this function all type of values (numeric, character or string) can be written as output. The `printf()` function can be written as-

```
printf("control string",variable 1,variable 2,.....);
```

In this function, the control string contains conversion specification characters and text. It should be enclosed within double quotes. The names of variables should not be preceded by an ampersand (&) sign. If the control string does not contain any conversion specification, the variable names are not specified. Some examples of `printf()` function are-

```
#include<stdio.h>
int main(void)
{
    .....
    printf("C is excellent\n");
    return 0;
}
```

Output:
C is excellent

Here control string has only text and no conversion specification characters, so the output is only text.

```
#include<stdio.h>
int main(void)
{
    int age;
    printf("Enter your age : ");
    scanf("%d",&age);
    return 0;
}
```

Here also `printf()` does not contain any conversion specification character and is used to display a message that tells the user to enter his age.

```
#include<stdio.h>
int main(void)
{
    int basic=2000;
    .....
    printf("%d",basic);
    .....
}
```

In this example control string contains a conversion specification character `%d`, which implies that an integer value will be displayed. The variable `basic` has the integer value to be displayed as output.

```
#include<stdio.h>
int main(void)
{
    float height=5.6;
    .....
    printf("%f",height);
    .....
}
```

Here control string has conversion specification character `%f`, which means that a floating point number will be displayed. The variable `height` has the floating point value to be displayed as output.

```
#include<stdio.h>
int main(void)
{
    char ch='$';
    .....
    printf("%c",ch);
    .....
}
```

In the above example, the control string has conversion specification character %c, means that a single character will be displayed and variable ch has that character value.

```
#include<stdio.h>
int main(void)
{
    char str[30] = "CinDepth";
    printf("%s", str);
}
```

Here control string has conversion specification character %s, implying that a string will be displayed and variable name str is a character array, holding the string which will be displayed.

```
#include<stdio.h>
int main(void)
{
    int basic=2000;
    printf("Basic Salary = %d", basic);
    return 0;
}
```

Output:
Basic Salary = 2000

Here the control string contains text with conversion specification character %d. The text will be displayed as it is, and the value of variable basic will be displayed in place of %d.

```
#include<stdio.h>
int main(void)
{
    int b=1500;
    float h=200.50;
    char g='A';
    printf("Basic=%d, HRA=%f, Grade=%c", b, h, g);
    return 0;
}
```

Output:
Basic = 1500, HRA = 200.500000, Grade = A

Here control string contains text with three conversion specification characters %d, %f and %c. %d is for integer value, %f is for floating point number and %c is for a single character.

```
#include<stdio.h>
int main(void)
{
    int num=10;
    printf("Octal equivalent of decimal %d = %o", num, num);
    return 0;
}
```

Output:
Octal equivalent of decimal 10 = 12

Here the second conversion specification character is %o, thus the octal equivalent of the decimal number stored in the variable num is displayed.

```
#include<stdio.h>
int main(void)
{
    int num=10;
    printf("Hex equivalent of decimal %d = %x", num, num);
    return 0;
}
```

Output:
Hex equivalent of decimal 10 = a

Here the second conversion specification character is %x, thus the hexadecimal equivalent of the decimal number stored in the variable num is displayed.

In chapter 2 we had studied about escape sequences. Here we'll see how we can use them in the printf statement. The most commonly used escape sequences used are '\n' and '\t'.

```
#include<stdio.h>
int main(void)
{
    int b=1500;
    float h=200.50;
    char g='A';
    printf("Basic=%d\nHRA=%f\nGrade=%c\n", b, h, g);
    return 0;
}
```

Output:
Basic = 1500
HRA = 200.500000
Grade = A

'\n' moves the cursor to the beginning of next line.

```
#include<stdio.h>
int main(void)
{
    int b=1500;
    float h=200.50;
    char g='A';
    printf("Basic=%d\tHRA=%f\tGrade=%c\n", b, h, g);
    return 0;
}
```

Output:
Basic = 1500 HRA = 200.500000 Grade = A

'\t' moves the cursor to the next tab stop. Similarly we can use other escape sequences also. For example '\b' moves the cursor one position back, '\r' moves the cursor to the beginning of the current line and '\a' alerts the user by a beep sound. '\v' moves the cursor to the next vertical tab position (first column of the next line), and '\f' move the cursor to the next page. '\v' and '\f' are effective only when output is printed through a printer.

If we want to print characters like single quotes ('), double quotes (") or the backslash character (\), then we have to precede them by a backslash character in the format string.

For example-

```
printf("9\\11\\1978");           will print 9\11\1978
printf("She said,\"I have to go\"."); will print        She said, "I have to go".
```

3.4 Formatted Input and Output

Data can be entered and displayed in a particular format. Through format specifications, better presentation result can be obtained. Formats for different specifications are given next.

3.4.1 Format for Integer Input

%wd

Here 'd' is the conversion specification character for integer value and 'w' is an integer number specifying the maximum field width of input data. If the length of input is more than this maximum field width then the value are not stored correctly.

```
scanf("%2d%3d", &a, &b);
```

(i) When input data length is less than or equal to the given field width, the input values are unaltered and stored in given variables.

Input :

6 394

Result :

6 is stored in a and 394 is stored in b.

(ii) When input data length is more than the given field width, the input values are altered and stored in the variable as-

Input :

269 3845

Result :

26 is stored in a and 9 is stored in b and the rest of input is ignored.

3.4.2 Format for Integer Output

%wd

Here w is the integer number specifying the field width of the output data. If the length of the variable is less than the specified field width, the variable is right justified with leading blanks.

```
printf("a=%3d,b=%4d",a,b);
```

(i) When the length of variable is less than the width specifier.
Value of variables: 78 9

Output:

a	=		7	8	,	b	=				9
---	---	--	---	---	---	---	---	--	--	--	---

The width specifier of first data is 3 while there are only 2 digits in it, so there is one leading blank. The width specifier of second data is 4 while there is only 1 digit, so there are 3 leading blanks.

(ii) When the length of the variable is equal to the width specifier
Value of variables: 263 1941

Output:

a	=	2	6	3	,	b	=	1	9	4	1
---	---	---	---	---	---	---	---	---	---	---	---

(iii) When length of variable is more than the width specifier, then also the output is printed correctly.
Value of variables : 2691 19412

Output:

a	=	2	6	9	1	,	b	=	1	9	4	1	2
---	---	---	---	---	---	---	---	---	---	---	---	---	---

```
#include<stdio.h>
int main(void)
{
    int a=4000,b=200,c=15;
    printf("a=%d\nb=%d\nc=%d\n",a,b,c);
    printf("a=%d\nb=%d\nc=%d\n",a,b,c);
    return 0;
}
```

The output of the first printf() would be-

a=4000

b=200

c=15

while the output of second printf() would be-

a=4000

b=200

c=15

3.4.3 Format for Floating Point Numeric Input

%wf

Here 'w' is the integer number specifying the total width of the data (including the digits before and after the decimal and the decimal itself).

```
scanf("%3f%4f",&x,&y);
```

(i) When input data length is less than or equal to the given width, values are unaltered and stored in the variables.

Input :

5 5.92

Result :

5.0 is stored in x and 5.92 is stored in y.

(ii) When input data length is more than the given width, the entered values are altered and stored in the variables.

Input :

5.93 65.875

Result :

5.9 is stored in x and 3.00 is stored in y.

3.4.4 Format for Floating Point Numeric Output

%w.nf

Here w is the integer number specifying the total width of the input data and n is the number of digits printed after decimal point. By default 6 digits are printed after the decimal.

```
printf("x=%4.1f,y=%7.2f",x,y);
```

If the total length of the variable is less than the specified width w, then the value is right justified with leading blanks. If the number of digits after decimal is more than n, the digits are rounded off.

Value of variables: 8 5.9

Output:

x	=		8	.	0	,	y	=				5	.	9	0
---	---	--	---	---	---	---	---	---	--	--	--	---	---	---	---

Value of variables : 25.3 1635.92

Output:

x	=	2	5	.	3	,	y	=	1	6	3	5	.	9	2
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Value of variables : 15.231 65.875948

Output:

x	=	1	5	.	2	,	y	=			6	5	.	8	8
---	---	---	---	---	---	---	---	---	--	--	---	---	---	---	---

3.4.5 Format for String Input

%ws

Here w specifies the total number of characters that will be stored in the string.

```
char str[8];
scanf("%3s",str);
```

If the input is-

Srivastava

only first three characters of this input will be stored in the string, so the characters in the string will be 'S', 'r', 'i', '\0'

The null character('\0') is automatically stored at the end.

3.4.6 Format for String Output

%w.ns

Here *w* is the specified field width. Decimal point and *n* are optional. If present then *n* specifies that only first *n* characters of the string will be displayed and (*w*-*n*) leading blanks are displayed before string.

(i) `printf("%3s", "sureshkumar");`

s u r e s h k u m a r

(ii) `printf("%10s", "seeta");`

seeta

(ii) `printf("%.3s", "sureshkumar");`

s u r

(iii) `printf("%8.3s", "sureshkumar");`

s u r

(8 - 3 = 5 leading blanks)

3.4.7 Suppression Character in scanf()

If we want to skip any input field, we can specify *** between the *%* sign and the conversion specification. The input field is read but its value is not assigned to any address. This character *** is called the suppression character. For example-

`scanf("%d*d%d", &a, &b, &c);`

Input:

25 30 35

Here 25 is stored in *a*, 30 is skipped and 35 is stored in *b*. Since no data is available for *c*, it has garbage value.

`scanf("%d*c%d*c%d", &d, &m, &y);`

Input:

3/1/2003

Here 3 will be stored in *d*, then / will be skipped, 11 will be stored in *m*, again / will be skipped and finally 2003 will be stored in *y*.

`#include<stdio.h>`

```
int main(void)
{
    int a,b,c;
    printf("Enter three numbers : ");
    scanf("%d*d%d", &a, &b, &c);
    printf("%d %d %d", a,b,c);
    return 0;
}
```

Output:

Enter three numbers : 25 30 35

25 35 253810942

The variable *c* has garbage value.

3.5 Character I/O

getchar() and putchar()

These functions `getchar()` and `putchar()` can be used for character I/O. `getchar()` reads a single character from the standard input. `putchar()` outputs one character at a time to the standard output.

`#include<stdio.h>`

`int main(void)`

```
{
    char ch;
```

Input-Output in C

```
printf("Enter a character : ");
ch = getchar();
printf("The entered character is : ");
putchar(ch);
return 0;
```

Output:

Enter a character: B

The entered character is : B

Exercise

Find the output of the following programs; assume `stdio.h` is included in all programs.

```
1. #define MSG "Hello World\n"
   int main(void)
   {
       printf(MSG);
       return 0;
   }
```

```
2. int main(void)
   {
       printf("Indian\b \n");
       printf("New\rDelhi\n");
       return 0;
   }
```

```
3. int main(void)
   {
       int a=11;
       printf("a=%d\t", a);
       printf("a=%o\t", a);
       printf("a=%x\t", a);
       printf("a=%X\n", a);
       return 0;
   }
```

```
4. #include<limits.h>
   int main(void)
   {
       int a=4000000000;
       unsigned int b=4000000000;
       printf("a=%d, b=%u\n", a,b);
       printf("a=%d, b=%u\n", INT_MAX, UINT_MAX);
       return 0;
   }
```

```
5. int main(void)
   {
       char ch;
       printf("Enter a character:");
       scanf("%c", &ch);
       printf("%d\n", ch);
       return 0;
   }
```

```
6. int main(void)
   {
       float b=123.1265;
       printf("%f\t", b);
   }
```

```

        printf("%.2f\t",b);
        printf("%.3f\n",b);
        return 0;
    }

7. int main(void)
{
    int a=625,b=2394,c=12345;
    printf("%5d,%5d,%5d\n",a,b,c);
    printf("%3d,%4d,%5d\n",a,b,c);
    return 0;
}

8. int main(void)
{
    int a=98;
    char ch='c';
    printf("%c,%d\n",a,ch);
    return 0;
}

9. int main(void)
{
    float a1,b1,a2,b2,a3,b3;
    a1=2;
    b1=6.8;
    a2=4.2;
    b2=3.57;
    a3=9.82;
    b3=85.673;
    printf("%3.1f,%4.2f\n",a1,b1);
    printf("%5.1f,%6.2f\n",a2,b2);
    printf("%7.1f,%8.2f\n",a3,b3);
    return 0;
}

10. int main(void)
{
    printf("%10s\n","India");
    printf("%4s\n","India");
    printf("%.2s\n","India");
    printf("%5.2s\n","India");
    return 0;
}

```

Operators and Expressions

An operator specifies an operation to be performed that yields a value. The variables, constants can be joined by various operators to form an expression. An operand is a data item on which an operator acts. Some operators require two operands, while others act upon only one operand. C includes a large number of operators that fall under several different categories, which are-

1. Arithmetic operators
2. Assignment operators
3. Increment and Decrement operators
4. Relational operators
5. Logical operators
6. Conditional operator
7. Comma operator
8. sizeof operator
9. Bitwise operators
10. Other operators

4.1 Arithmetic Operators

Arithmetic operators are used for numeric calculations. They are of two types-

1. Unary arithmetic operators
2. Binary arithmetic operators

Unary Arithmetic Operators

Unary operators require only one operand. For example-

+x -y

Here '-' changes the sign of the operand y.

Binary Arithmetic Operators

Binary operators require two operands. There are five binary arithmetic operators-

Operator	Purpose
+	addition
-	subtraction
*	multiplication
/	division
%	gives the remainder in integer division

%(modulus operator) cannot be applied with floating point operands. There is no exponent operator in C, however there is a library function pow() to carry out exponentiation operation. Note that unary plus and minus operators are different from the addition and subtraction operators.

When both operands are integers, the resulting value is always an integer. Let us take two variables a and b. The value of a is 17 and b is 4, the results of the following operations are-

Expression	Result
a+b	21
a-b	13
a*b	68
a/b	4 (decimal part truncates)
a%b	1 (Remainder after integer division)

After division operation the decimal part will be truncated and result is only integer part of quotient. After modulus operation the result will be remainder part of integer division. The second operand must be nonzero for division and modulus operations.

```
/*P4.1 Integer arithmetic operations*/
#include<stdio.h>
int main(void)
{
    int a=17,b=4;
    printf("Sum=%d\n",a+b);
    printf("Difference=%d\n",a-b);
    printf("Product=%d\n",a*b);
    printf("Quotient=%d\n",a/b);
    printf("Remainder=%d\n",a%b);
    return 0;
}
```

Output:
Sum=21
Difference=13
Product=68
Quotient=4
Remainder=1

When both operands are floating point type, the result is also floating point type. Let us take two variables a and b. The value of a is 12.4 and b is 3.1, the results of the following operations are-

Expression	Result
a+b	15.5
a-b	9.3
a*b	38.44
a/b	4.0

The modulus operator % cannot be used with floating point numbers.

```
/*P4.2 Program to understand the floating point arithmetic operation*/
#include<stdio.h>
int main(void)
{
    float a=12.4,b=3.8;
    printf("Sum=%.2f\n",a+b);
    printf("Difference=%.2f\n",a-b);
    printf("Product=%.2f\n",a*b);
    printf("a/b=%.2f\n",a/b);
    return 0;
}
```

Output:
Sum=16.20
Difference=8.60
Product=47.12
a/b=3.26

When one operand is of integer type and the other is of floating type, the resulting value is floating type. If value of a is 12 and of b is 2.5,

Expression	Result
a+b	14.5
a-b	9.5
a*b	30.0
a/b	4.8

Sometimes mixed mode arithmetic can help in getting exact results. For example the result of expression 5/ will be 2, since integer arithmetic is applied. If we want exact result we can make one of the operands float type. For example 5.0/2 or 5/2.0, both will give result 2.5.

4.2 Assignment Operators

A value can be stored in a variable using assignment operator. The operand on the left hand side should be a variable, while the operand on the right hand side can be any variable, constant or expression. The value of right hand operand is assigned to the left hand operand. Here are some examples of assignment expressions-

```
x=8          /*8 is assigned to x*/
y=5          /*5 is assigned to y*/
s=x+y-2      /*Value of expression x+y-2 is assigned to s*/
y=x          /*Value of x is assigned to y*/
x=y          /*Value of y is assigned to x*/
```

The value that is being assigned is considered as value of the assignment expression. For example x=8 is an assignment expression whose value is 8.

We can have multiple assignment expressions also, for example-

```
x=y=z=20
```

Here all the three variables x, y, z will be assigned value 20, and the value of the whole expression will be 20. If we put a semicolon after the assignment expression then it becomes an assignment statement. For example these are assignment statements-

```
x=8;
y=5;
s=x+y-2;
x=y=z=20;
```

When the variable on the left hand side of assignment operator also occurs on right hand side, we can avoid writing the variable twice by using compound assignment operators. For example-

```
x=x+5      can also be written as-
x+=5
```

Here += is a compound assignment operator.

Similarly we have other compound assignment operators-

x-=5	is equivalent to	x=x-5
y*=5	is equivalent to	y=y*5
sum/=5	is equivalent to	sum=sum/5
k%=5+x	is equivalent to	k=k%(5+x)
a*=b+5;	is equivalent to	a=a*(b+5)

4.3 Increment and Decrement Operators

The increment (++) and decrement (--) operators are unary operators because they operate on a single operand. The increment operator increments the value of the variable by 1, while decrement operator decrements the value of the variable by 1.

++x	is equivalent to	x=x+1
--x	is equivalent to	x=x-1

These operators should be used only with variables; they can't be used with constants or expressions. For example the expressions ++5 or ++(x+y+z) are invalid.

These operators are of two types-

1. Prefix increment/decrement - operator is written before the operand (e.g. ++x or --x)
2. Postfix increment/decrement - operator is written after the operand (e.g. x++ or x--)

4.3.1 Prefix Increment / Decrement

Here first the value of variable is incremented/decremented then the new value is used in the operation. Let us take a variable x whose value is 3.

The statement `y=++x;` means first increment the value of x by 1, then assign the value of x to y. This single statement is equivalent to these two statements-

```
x=x+1;
y=x;
```

Now value of x is 4 and value of y is also 4.

The statement `y=--x;` means first decrement the value of x by 1 then assign the value of x to y.

This statement is equivalent to these two statements.

```
x=x-1;
y=x;
```

Now value of x is 3 and value of y is also 3.

```
/*P4.3 Prefix increment/decrement*/
#include<stdio.h>
int main(void)
{
    int x=8;
    printf("x=%d\t",x);
    printf("x=%d\t",++x); /*Prefix increment*/
    printf("x=%d\t",x);
    printf("x=%d\t",--x); /*Prefix decrement*/
    printf("x=%d\n",x);
    return 0;
}
```

Output:
x=8 x=9 x=9 x=8 x=8

In the second `printf()`, first the value of x is incremented and then printed; similarly in the fourth `printf()` first the value of x is decremented and then printed.

4.3.2 Postfix Increment / Decrement

Here first the value of variable is used in the operation and then increment/decrement is performed. Let us take a variable x whose value is 3.

The statement `y=x++;` means first the value of x is assigned to y and then x is incremented. This statement is equivalent to these two statements-

```
y=x;
x=x+1;
```

Now value of x is 4 and value of y is 3.

The statement `y=x--;` means first the value of x is assigned to y and then x is decremented.

This statement is equivalent to these two statements-

```
y=x;
x=x-1;
```

Now value of x is 3 and value of y is 4.

```
/*P4.4 Program to understand the use of postfix increment/decrement*/
#include<stdio.h>
int main(void)
{
    int x=8;
    printf("x=%d\t",x);
```

Operators and Expressions

```
printf("x=%d\t",x++); /*postfix increment*/
printf("x=%d\t",x);
printf("x=%d\t",x--); /*postfix decrement*/
printf("x=%d\n",x);
return 0;
```

Output:
x=8 x=8 x=9 x=9 x=8

In the second `printf()` statement, first the value of x is printed and then incremented; similarly in the fourth `printf()` statement first the value of x is printed and then decremented.

4.4 Relational Operators

Relational operators are used to compare values of two expressions. An expression that contains relational operators is called relational expression. If the relation is true then the value of relational expression is 1; if the relation is false then the value of expression is 0. The relational operators are-

Operator	Meaning
<	less than
<=	less than or equal to
==	equal to
!=	Not equal to
>	Greater than
>=	Greater than or equal to

Let us take two variables a=9 and b=5, and form simple relational expressions with them-

Expression	Relation	Value of Expression
a<b	False	0
a<=b	False	0
a==b	False	0
a!=b	True	1
a>b	True	1
a>=b	True	1
a==0	False	0
b!=0	True	1
a>8	True	1
2>4	False	0

The relational operators are generally used in `if...else` construct and loops. In our next program we will use `if` statement to illustrate the use of relational operators. The `if` control statement evaluates an expression; if this expression is true (non zero) then the next statement is executed, otherwise the next statement is skipped. The details of `if` statement are discussed in chapter 5, here we have used it to give you an idea of how relational operators are used.

```
/*P4.5 Program to understand the use of relational operators*/
#include<stdio.h>
int main(void)
{
    int a,b;
    printf("Enter values for a and b : ");
    scanf("%d%d",&a,&b);

    if(a<b)
        printf("%d is less than %d\n",a,b);
    if(a<=b)
        printf("%d is less than or equal to %d\n",a,b);
    if(a==b)
        printf("%d is equal to %d\n",a,b);
    if(a!=b)
```

```

printf("%d is not equal to %d\n", a, b);
if(a>b)
    printf("%d is greater than %d\n", a, b);
if(a==b)
    printf("%d is greater than or equal to %d\n", a, b);
return 0;
}

```

Output:

Enter values for a and b: 12 7
 12 is not equal to 7
 12 is greater than 7
 12 is greater than or equal to 7

It is important to note that the assignment operator(=) and equality operator(==) are entirely different. Assignment operator is used for assigning values while equality operator is used to compare two expressions. Beginners generally confuse between the two and use one in the place of another and this leads to an error difficult to find out. For example if in the above program we use = instead of ==, then we will get wrong output.

```

if(a=b)
    printf("%d is equal to %d\n", a, b);

```

Here a=b is treated as an assignment expression, so the value of b is assigned to variable a, and the value of the whole expression becomes 7 which is non-zero, and so the next statement is executed.

4.5 Logical or Boolean Operators

An expression that combines two or more expressions is termed as a logical expression. For combining these expressions we use logical operators. These operators return 0 for false and 1 for true. The operands may be constants, variables or expressions. C has three logical operators.

Operator	Name
&&	AND
	OR
!	NOT

Here logical NOT is a unary operator while the other two are binary operators. Before studying these operators let us understand the concept of true and false. In C any non-zero value is regarded as true and zero is regarded as false.

4.5.1 AND (&&) Operator

This operator gives the net result true, if both the conditions are true, otherwise the result is false.

Boolean Table		
Condition1	Condition2	Result
False	False	False
False	True	False
True	False	False
True	True	True

Let us take three variables a=10, b=5, c=0. Suppose we have a logical expression-
 (a==10) && (b<a)

Here both the conditions a==10 and b<a are true, and so this whole expression is true. Since the logical operators return 1 for true, the value of this expression is 1.

Expression		Result	Value of expression
(a==10) && (b>a)	true && false	false	0
(b>a) && (b==3)	false && false	false	0
a&&b	true && true	true	1
a&&c	true && false	false	0

In the last two expressions we have taken only variables. Since nonzero values are regarded as true and zero value is regarded as false, variables a and b are considered true and variable c is considered false.

4.5.2 OR (||) Operator

This operator gives the net result false, if both the conditions have the value false, otherwise the result is true.

Boolean Table		
Condition1	Condition2	Result
False	False	False
False	True	True
True	False	True
True	True	True

Let us take three variables a=10, b=5, c=0

Consider the logical expression-
 (a>=b) || (b>15)

This gives result true because one condition is true.

Expression		Result	Value of expression
a b	true true	true	1
a c	true false	true	1
(a<9) (b>10)	false false	false	0
(b!=7) c	true false	true	1

4.5.3 Not (!) Operator

This is a unary operator and it negates the value of the condition. If the value of the condition is false then gives the result true. If the value of the condition is true then it gives the result false.

Boolean Table	
Condition	Result
False	True
True	False

Let us take three variables a=10, b=5, c=0

Suppose we have this logical expression-
 !(a==10)

The value of the condition (a==10) is true. NOT operator negates the value of the condition, thus the result false.

Expression		Result	Value of expression
!a	!true	false	0
!c	!false	true	1
!(b>c)	!true	false	0
!(a&&c)	!false	true	1

4.6 Conditional Operator

Conditional operator is a ternary operator (? and :) which requires three expressions as operands. It is written as-

TestExpression ? expression1 : expression2

Firstly the TestExpression is evaluated.

(i) If TestExpression is true(nonzero), then expression1 is evaluated and it becomes the value of the overall conditional expression.

(ii) If TestExpression is false(zero), then expression2 is evaluated and it becomes the value of overall conditional expression.

For example consider this conditional expression-

```
a>b?a:b
```

Here first the expression $a>b$ is evaluated, if the value is true then the value of variable a becomes the value of conditional expression otherwise the value of b becomes the value of conditional expression.

Suppose $a=5$ and $b=8$, and we use the above conditional expression in a statement as-

```
max = a>b?a:b;
```

First the expression $a>b$ is evaluated, it is false so the value of b becomes the value of conditional expression and it is assigned to variable max .

In our next example we have written a conditional statement by putting a semicolon after the conditional expression.

```
a<b ? printf("a is smaller"):printf("b is smaller");
```

Since the expression $a<b$ is true, the first `printf()` function is executed.

```
/*P4.6 Program to print the larger of two numbers using conditional operator*/
#include<stdio.h>
```

```
int main(void)
{
    int a,b,max;
    printf("Enter values for a and b :");
    scanf("%d%d",&a,&b);
    max = a>b ? a : b; /*ternary operator*/
    printf("Larger of %d and %d is %d\n",a,b,max);
    return 0;
}
```

Output:
Enter values for a and b: 12 7
Larger of 12 and 7 is 12

4.7 Comma Operator

The comma operator (,) is used to permit different expressions to appear in situations where only one expression would be used. The expressions are separated by the comma operator. The separated expressions are evaluated from left to right and the type and value of the rightmost expression is the type and value of the compound expression.

For example consider this expression-

```
a=8, b=7, c=9, a+b+c
```

Here we have combined 4 expressions. Initially 8 is assigned to the variable a , then 7 is assigned to the variable b , 9 is assigned to variable c and after this $a+b+c$ is evaluated which becomes the value of whole expression. So the value of the above expression is 24. Now consider this statement-

```
sum = (a=8, b=7, c=9, a+b+c);
```

Here the value of the whole expression on right side will be assigned to variable sum i.e. sum will be assigned value 24. Since precedence of comma operator is lower than that of assignment operator, the parentheses are necessary here. The comma operator helps make the code more compact, for example without the use of comma operator the above task would have been done in 4 statements.

```
a=8;
b=7;
c=9;
sum=a+b+c;
```

```
/*P4.7 Program to understand the use of comma operator*/
#include<stdio.h>
int main(void)
{
```

Operators and Expressions

```
int a,b,c,sum;
sum = (a=8,b=7,c=9,a+b+c);
printf("Sum=%d\n",sum);
return 0;
```

Output:
Sum=24

```
/*P4.8 Program to interchange the value of two variables using comma operator*/
#include<stdio.h>
int main(void)
```

```
int a=8,b=7,temp;
printf("a=%d, b=%d\n",a,b);
temp=a,a=b,b=temp;
printf("a=%d, b=%d\n",a,b);
return 0;
```

Output:
 $a=8, b=7$
 $a=7, b=8$

The comma operator is also used in loops, we'll study about that in the next chapter.

4.8 sizeof Operator

`sizeof` is an unary operator that gives the size of its operand in terms of bytes. The operand can be a variable, constant or any datatype (`int`, `float`, `char` etc), for example `sizeof(int)` gives the bytes occupied by `int` datatype.

```
/*P4.9 Program to understand the sizeof operator*/
#include<stdio.h>
int main(void)
```

```
int var;
printf("Size of int=%u\n",sizeof(int));
printf("Size of float=%u\n",sizeof(float));
printf("Size of var=%u\n",sizeof(var));
printf("Size of an integer constant=%u\n",sizeof(45));
return 0;
```

Generally `sizeof` operator is used to make portable programs i.e. programs that can be run on different machines. For example if we write our program assuming `int` to be of 2 bytes, then it won't run correctly on machine on which `int` is of 4 bytes. So to make general code that can run on all machines we can use `sizeof` operator.

4.9 Bitwise Operators

C has the ability to support manipulation of data at the bit level. Bitwise operators operate on integers only they are used for operations on individual bits. The bitwise operators are-

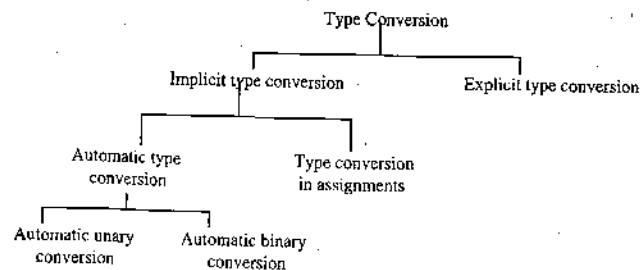
Bitwise operator	Name
&	bitwise AND
	bitwise OR
~	one's complement
<<	left shift
>>	right shift
^	bitwise XOR

These bitwise operators are discussed in detail in chapter 14.

There are many other operators that we'll study in further chapters. The last operator that we'll take up in this chapter is the type cast operator. Before studying that we'll discuss about type conversions in C.

4.10 Type Conversion

C provides the facility of mixing different types of variables and constants in an expression. In these types of operations, data type of one operand is converted into data type of another operand to get a uniform type. This is known as type conversion. The different types of type conversions are-



Implicit type conversions are done by the compiler while the explicit type conversions are user defined conversions.

4.10.1 Implicit Type Conversions

These conversions are done by the C compiler according to some predefined rules of C language. The two types of implicit type conversions are automatic type conversions and type conversion in assignments.

4.10.1.1 Automatic Conversions

Automatic unary conversions

All operands of type char and short are converted to int before any operation.

Automatic binary conversions

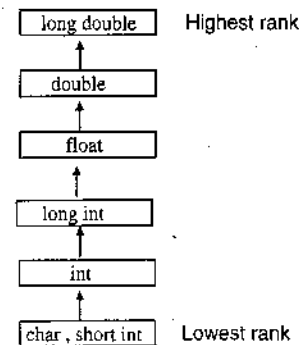
The rules for automatic binary conversions are as-

- (i) If one operand is long double, then the other will be converted to long double, and the result will be long double,
- (ii) Otherwise if one operand is double, then the other will be converted to double and the result will be double,
- (iii) Otherwise if one operand is float, the other will be converted to float and the result will be float,
- (iv) Otherwise if one operand is unsigned long int, then other will be converted to unsigned long int and the result will be unsigned long int.
- (v) Otherwise if one operand is long int and other is unsigned int
 - (a) If long int can represent all the values of an unsigned int, then unsigned int will be converted to long int and the result will be long int,
 - (b) Else both the operands will be converted to unsigned long int and the result will be unsigned long int,
- (vi) Otherwise if one operand is long int, then the other will be converted to long int and the result will be long int.

(vii) Otherwise if one operand is unsigned int, then the other will be converted to unsigned int and the result will be unsigned int.

(viii) Otherwise both operands will be int and the result will be int.

If we leave aside unsigned variables, then these rules are rather simple and can be summarized by assigning a rank to each data type. Whenever there are two operands of different data types the operand with a lower rank will be converted to the type of higher rank operand. This is called promotion of data type.



4.10.1.2 Type Conversion in Assignment

If the types of the two operands in an assignment expression are different, then the type of the right hand side operand is converted to the type of left hand operand. If the right hand operand is of lower rank then it will be promoted to the rank of left hand operand, and if it is of higher rank then it will be demoted to the rank of left hand operand.

Some consequences of these promotions and demotions are-

1. Some high order bits may be dropped when long is converted to int, or int is converted to short int or char.
2. Fractional part will be truncated during conversion of float type to int type
3. When double type is converted to float type, digits are rounded off.
4. When a signed type is changed to unsigned type, the sign may be dropped.
5. When an int is converted to float, or float to double there will be no increase in accuracy or precision.

*/*P4.10 Program to understand type conversion in assignment*/*

```

#include<stdio.h>
int main(void)
{
    char c1,c2;
    int i1,i2;
    float f1,f2;
    c1='H';
    i1=80.56; /*float converted to int, only 80 assigned to i1*/
    f1=12.6;
    c2=i1; /*int converted to char */
    i2=f1; /*float converted to int */
    /*Now c2 has the character with ASCII value 80, i2 is assigned value 12*/
    printf("c2=%c, i2=%d\n",c2,i2);
    f2=i1; /*int converted to float*/
    i2=c1; /*char converted to int */
    /*Now i2 contains ASCII value of character 'H' which is 72*/
    printf("f2=%.2f, i2=%d\n",f2,i2);
    return 0;
}
  
```

```

)
Output:
c2=P, i2=12
f2=80.00, i2=72

```

4.10.2 Explicit Type Conversion or Type Casting

There may be certain situations where implicit conversions may not solve our purpose. For example-

```

float z;
int x=20, y=3;
z=x/y;

```

The value of z will be 6.0 instead of 6.66.

In these types of cases we can specify our own conversions known as type casting or coercion. This is done with the help of cast operator. The cast operator is a unary operator that is used for converting an expression to a particular data type temporarily. The syntax of cast operator is-

```
(datatype) expression
```

Here the datatype along with the parentheses is called the cast operator.

If we write the above statement as-

```
z=(float)x/y;
```

The value of z will come out to be 6.66. This happens because the cast operator (float) temporarily converted the int variable x into float type and so mixed mode arithmetic took place and fractional part was not lost.

Note that the cast operator changes the data type of variable x only temporarily for the evaluation of this expression, everywhere else in the program it will be an int variable only.

*/*P4.11 Program to illustrate the use of cast operator*/*

```
#include<stdio.h>
```

```
int main(void)
```

```

{
    int x=5,y=2;
    float p,q;
    p=x/y;
    printf("p=%f\n",p);
    q=(float)x/y;
    printf("q=%f\n",q);
    return 0;
}

```

Output:

```

p=2.000000
q=2.500000

```

Initially the expression x/y is evaluated, both x and y are integers so according to integer arithmetic after division, decimal value is truncated and result is integer value 2. This value will be assigned to p, but p is a float variable so according to implicit type conversion in assignment the integer value 2 will be converted to float and then assigned to p. So finally the value of p is 2.0

When cast operator is used, mixed mode arithmetic is performed and thus the value of q is 2.5.

Here are some other examples of usage of cast operator-

```

(int)20.3      constant 20.3 converted to integer type
                and fractional part is lost(Result 20)

(float)20/3    constant 20 converted to float type, and
                then divided by 3 (Result 6.66)

(float)(20/3)  First 20 divided by 3 and then result of whole
                expression converted to float type(Result 6.00)

(double)(x+y-z) Result of expression x+y-z is converted to double

(double)x+y-z  First x is converted to double and then used in expression

```

4.11 Precedence and Associativity of Operators

For evaluation of expressions having more than one operator, there are certain precedence and associativity rules defined in C. Let us see what these rules are and why are they required. Consider the following expression-

2+3*5

Here we have two operators- addition and multiplication operators. If addition is performed before multiplication then result will be 25 and if multiplication is performed before addition then the result will be 17.

In C language, operators are grouped together and each group is given a precedence level. The precedence of the operators is given in the following table. The upper rows in the table have higher precedence and decreases as we move down the table. Thus the operators with precedence level 1 have highest precedence and operators with precedence level 15 have lowest precedence. So whenever an expression contains more than one operator with a higher precedence is evaluated first. For example in the above expression, multiplication will be performed before addition since multiplication operator has higher precedence than the addition operator.

Precedence and Associativity of operators

Operator	Description	Precedence level	Associativity
() [] -> .	Function call Array subscript Arrow operator Dot operator	1	Left to Right
+ - ++ -- ! ~ * & (datatype) sizeof	Unary plus Unary minus Increment Decrement Logical NOT One's complement Indirection Address Type cast Size in bytes	2	Right to Left
* / %	Multiplication Division Modulus	3	Left to Right
+ -	Addition Subtraction	4	Left to Right
<< >>	Left shift Right shift	5	Left to Right
< <= > >=	Less than Less than or equal to Greater than Greater than or equal to	6	Left to Right
= !=	Equal to Not equal to	7	Left to Right
& ^	Bitwise AND Bitwise XOR	8 9	Left to Right
 &&	Bitwise OR Logical AND	10 11	Left to Right
	Logical OR	12	Left to Right
?:	Conditional operator	13	Right to Left
= *= += &= &= &= <<= >>=	Assignment operators	14	Right to Left
,	Comma operator	15	Left to Right

Let us take some expressions and see how they will be evaluated-

(i) $x = a + b < c$

Here + operator has higher precedence than < and =, and < has more precedence than =, so first $a + b$ will be evaluated, then < operator will be evaluated, and at last the whole value will be assigned to x. If initial values are $a=2, b=6, c=9$, then final value of x will be 1.

(ii) $x * = a + b$

Here + operator has higher precedence than *=, so $a + b$ will be evaluated before compound assignment. This is interpreted as $x = x * (a + b)$ and not as $x = x * a + b$. If initial values are $x=5, a=2, b=6$, then final value of x will be 13.

(iii) $x = a <= b \mid b = c$

Here order of evaluation of operators will be <=, ==, ||, =. If initial values are $a=2, b=3, c=4$, then final value of x will be 1.

In the above examples we have considered expressions that contain operators having different precedence levels. Now consider a situation when two operators with the same precedence occur in an expression. For example-

$$5 + 16 / 2 * 4$$

Here / and * have higher precedence than + operator, so they will be evaluated before addition. But / and * have same precedence, so which one of them will be evaluated first still remains a problem. If / is evaluated before *, then the result is 37 otherwise the result is 7. Similarly consider this expression-

$$20 - 7 - 5 - 2 - 1$$

Here we have four subtraction operators, which of course have the same precedence level. If we decide to evaluate from left to right then answer will be 5, and if we evaluate from right to left the answer will be 17. To solve these types of problems, an associativity property is assigned to each operator. Associativity of the operators within same group is same. All the operators either associate from left to right or from right to left. The associativity of all operators is also given in the precedence table. Now again consider the above two expressions-

$$5 + 16 / 2 * 4$$

Since / and * operators associate from left to right, / will be evaluated before * and the correct result is 37.

$$20 - 7 - 5 - 2 - 1$$

The subtraction operator associates from left to right so the value of this expression is 5.

The assignment operator associates from right to left. Suppose we have a multiple assignment expression like this-

$$x = y = z = 5$$

Here initially the integer value 5 is assigned to variable z and then value of expression $z=5$ is assigned to variable y. The value of expression $z=5$ is 5, so 5 is assigned to variable y and now the value of expression $y=z=5$ becomes 5. Now value of this expression is assigned to x and the value of whole expression $x=y=z=5$ becomes 5.

The conditional operator also associates from right to left. Consider this expression-

$$x ? y : a ? b : c$$

Since the conditional operator is right associative, this expression is considered as-

$$x ? y : (a ? b : c)$$

If x is non zero then the value of this expression will be equal to y. If x is zero then value of this expression will be equal to the value of expression $(a ? b : c)$ i.e. equal to b if a is non zero and equal to c if a is zero.

4.12 Role of Parentheses in Evaluating Expressions

If we want to change the order of precedence of any operation, we can use parentheses. According to the parentheses rule, all the operations that are enclosed within parentheses are performed first. For example in

expression $24 / 2 + 4$, division will take place before addition according to precedence rule, but if we enclose $2 + 4$ inside parentheses then addition will be performed first. So the value of expression $24 / (2 + 4)$ is 4.

For evaluation of expression inside parentheses same precedence and associativity rules apply. For example- $(22 - 4) / (2 + 4 * 2 - 1)$

Here inside parentheses multiplication will be performed before addition.

There can be nesting of parentheses in expressions i.e. a pair of parentheses can be enclosed within another pair of parentheses. For example- $(4 * (3 + 2)) / 10$

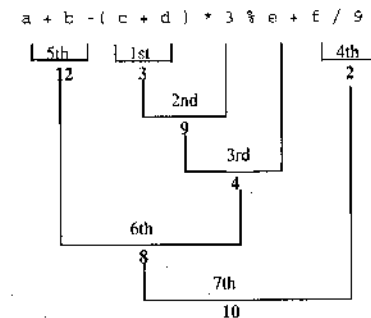
In these cases, expressions within innermost parentheses are always evaluated first, and then next innermost parentheses and so on, till outermost parentheses. After evaluation of all expressions within parentheses, the remaining expression is evaluated as usual. In the above expression $3 + 2$ is evaluated first and then $4 * 5$ and then $20 / 10$.

Sometimes in complex expressions, parentheses and spaces are used just to increase readability. For example compare the following two expressions-

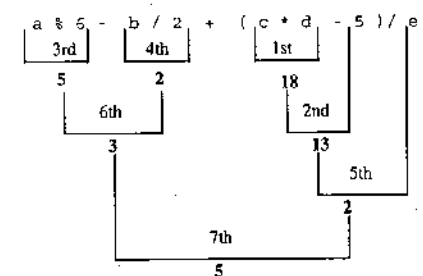
```
x = a! = b && c * d >= m & n
x = (a! = b) && ((c * d) >= (m & n))
```

These two expressions are evaluated in the same way but the operations performed in second one are clearer. Let us take some expressions and evaluate them according to the precedence, associativity and parentheses rules.

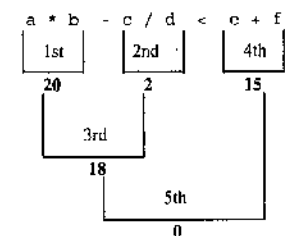
$a=8, b=4, c=2, d=1, e=5, f=20$
 $a + b - (c + d) * 3 \% e + f / 9$



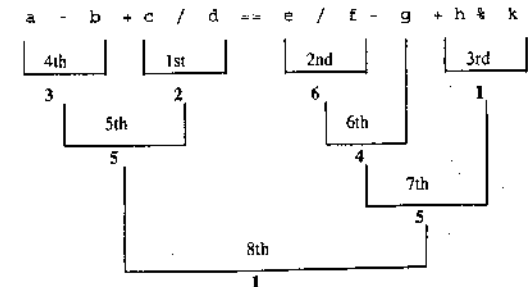
$a=17, b=5, c=6, d=3, e=5$
 $a \% 5 - b / 2 + (c * d - 5) / e$



$a=4, b=5, c=6, d=3, e=5, f=10$
 $a * b - c / d < c + f$



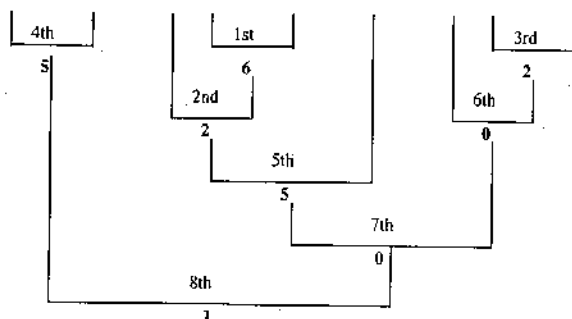
$a=8, b=5, c=8, d=3, e=65, f=10, g=2, h=5, k=2$
 $a - b + c / d == e / f - g + h \% k$



a=8, b=3, c=2, d=3, e=2, f=11

$$a-b \mid (a-b*c)+d \&\&e-f\%3$$

```
a - b || (a - b * c) + d && e - f % 3
```



The following program prints the results of above expressions-

```
/*P4.12 Program to evaluate some expressions*/
```

```
int main(void)
{
    int a,b,c,d,e,f,g,h,k;
    a=8, b=4, c=2, d=1, e=5, f=20;
    printf("%d\t", a+b-(c*d)*3%e+f/g);

    a=17, b=5, c=6, d=3, e=5;
    printf("%d\t", a+b-b/2*(c*d-5)/e);

    a=4, b=5, c=6, d=3, e=5, f=10;
    printf("%d\t", a*b-c/d*c*f);

    a=8, b=5, c=8, d=3, e=65, f=10, g=2, h=5, k=2;
    printf("%d\t", a-b*c/d=e/f-g+h*k);

    a=8, b=3, c=2, d=3, e=2, f=11;
    printf("%d\n", a-b|| (a-b*c)+d&&e-f%3);
    return 0;
}
```

Output:

10	5	0	1	1
----	---	---	---	---

4.13 Order of Evaluation of Operands

C does not specify the order in which the operands of an operator are evaluated. For example consider the expression-

```
y = (++x) * (--x); /*Assume value of x is 5*/
```

Here the two operands of the addition operator are $(++x)$ and $(--x)$. These two operands have to be added and then the result has to be assigned to variable y . Before addition, these two operands will be evaluated. Now if the first operand $(++x)$ is evaluated first then y will be assigned value 11 $(6+5)$, while if second operand $(--x)$ is evaluated first then y will be assigned the value 9 $(5+4)$. The answers may vary on different compilers so it is better to avoid such type of expressions.

There are four operators where C clearly specifies the order of evaluation of operands. These operators are logical AND (&), logical OR (|), conditional (? :) and comma operator (,). In all these cases the operand on the left side is evaluated first.

In the case of logical AND and logical OR operators, sometimes there is no need to evaluate second operand. In && operator, if the first operand evaluates to false(zero), then second operand is not evaluated and in the case of || operator if the first operand evaluates to true(non zero) then second operand is not evaluated.

Exercise

What is the output of the following programs; assume `stdio.h` is included in all programs.

```
1. int main(void)
{
    int a=-3;
    a = -a-a+!a;
    printf("%d\n",a);
    return 0;
}
```

```
2. int main(void)
{
    int a=2,b=1,c,d;
    c = a<b;
    d = (a>b) && (c<b);
    printf("c=%d, d=%d\n",c,d);
    return 0;
}
```

```
3. int main(void)
{
    int a=9,b=15,c=16,d=12,e,f;
    e = !(a<b || b<c);
    f = (a>b) ? a-b: b-a;
    printf("e=%d, f=%d\n",e,f);
    return 0;
}
```

```
4. int main(void)
{
    int a=5;
    a=6;
    a=a*5*a;
    printf("a=%d\n",a);
    return 0;
}
```

```
S, int main(void)
{
    int a=5, b=5;
    printf("%d,%d\t", ++a, b--);
    printf("%d,%d\t", a, b);
    printf("%d,%d\t", ++a, b++);
    printf("%d,%d\n", a, b);
    return 0;
}
```

```
6. int main(void)
{
    int x,y,z;
    x = 8++;
    y = ++x++;
    z = {x+y}--;
    printf("x=%d, y=%d, z=%d\n",x,y,z);
}
```

$$\angle b$$

$$a >$$
$$e = \begin{pmatrix} 1 & 9 & 15 & 11 & 15 & 16 \\ & & & 1 & 1 & 1 \\ & & & & 1 & 1 \end{pmatrix}$$

$$\begin{array}{r} 0? - 6:6 \\ 1? - 6:6 \end{array} \quad t = 6$$

17

```

return 0;
}

int main(void)
{
    int a=4, b=8, c=3, d=9, z;  $5+9 \times 2 - 8$ 
    z = a++ + ++b * c-- --d;  $14+0+3$ 
    printf("a=%d, b=%d, c=%d, d=%d, z=%d\n", a, b, c, d, z);
    return 0;
}

```

```

int main(void)
{
    int a=14, b, c;
    a=a%5;
    b=a/3;
    c=a/5%3;
    printf("a=%d, b=%d, c=%d\n", a, b, c);
    return 0;
}

```

```

int main(void)
{
    int a=15, b=13, c=16, x, y;
    x = a-3%2+c*2/4%2+b/4;
    y = a - b+5-b+9/3;
    printf("x=%d, y=%d\n", x, y);
    return 0;
}

```

$$\begin{aligned}
 &15 - 3 + 16 \times 0.2 + 3 \\
 &14 + 0 + 3 \\
 &x = 17 \\
 &15 = 13 + 5 - 13 + 9/3 \\
 &= 13 + 5 - 13 + 3 \\
 &= 8
 \end{aligned}$$

```

0. int main(void)
{
    int x, y, z, k=10;
    k+=(x=5, y=x+2, z=x+y);
    printf("x=%d, y=%d, z=%d, k=%d\n", x, y, z, k);
    return 0;
}

```

```

1. int main(void)
{
    float b;
    b=15/2;
    printf("%f\n", b);
    b=(float)15/2 + (15/2);
    printf("%f\n", b);
    return 0;
}

```

```

2. int main(void)
{
    int a=9;
    char ch='A';
    a = a+ch+24;
    printf("%d,%c\t%d,%c\n", ch, ch, a, a);
    return 0;
}

```

```

3. int main(void)
{
    int a, b, c, d;
    a=b=c=d=4;
}

```

```

a*=b+1;
c+=d*=3;
printf("a=%d, c=%d\n", a, c);
return 0;
}

```

```

14. int main(void)
{
    int a=5, b=10, temp;
    temp=a, a=b, b=temp;
    printf("a=%d, b=%d\n", a, b);
    return 0;
}

```

```

15. int main(void)
{
    int a=10, b=3, max;
    a>b ? max=a : max=b;
    printf("%d\n", max);
    return 0;
}

```

```

16. int main(void)
{
    int a=5, b=6;
    printf("%d\t", a=b);
    printf("%d\t", a=b);
    printf("%d %d\n", a, b);
    return 0;
}

```

```

17. int main(void)
{
    int a=3, b=4, c=3, d=4, x, y;
    x = (a=5) && (b=7);
    y = (c=5) || (d=8);
    printf("a=%d, b=%d, c=%d, d=%d, x=%d, y=%d\n", a, b, c, d, x, y);
    x = (a==6) && (b=9);
    y = (c==6) || (d=10);
    printf("a=%d, b=%d, c=%d, d=%d, x=%d, y=%d\n", a, b, c, d, x, y);
    return 0;
}

```

```

18. int main(void)
{
    int a=10;
    a=a++;
    a = a++ * a--;
    printf("%d\n", a);
    printf("%d\n", a++ * a++);
    return 0;
}

```

```

19. int main(void)
{
    int a=2, b=2, x, y;
    x = 4*(++a * 2 + 3);
    y = 4*(b++ * 2 + 3);
    printf("a=%d, b=%d, x=%d, y=%d\n", a, b, x, y);
    return 0;
}

```


20. Write a program that enters temperature in Celsius and converts that into Fahrenheit.
21. Write a program that accepts the radius of a circle and calculates the area and perimeter of that circle.
22. Write a program to accept a number in decimal and print the number in octal and hexadecimal.
23. Write a program to accept any number and print the value of remainder after dividing it by 3.
24. Accept any two numbers, if the first number is greater than second then print the difference of these two numbers, otherwise print their sum. Write this program using ternary operator.
25. Write a program that accepts marks in five subjects and calculates the total percentage marks.

Control Statements

In C programs, statements are executed sequentially in the order in which they appear in the program. But sometimes we may want to use a condition for executing only a part of program. Also many situations arise where we may want to execute some statements several times. Control statements enable us to specify the order in which the various instructions in the program are to be executed. They define how the control is transferred to other parts of the program. The different types of control statements in C language are-

Selection statements

- if...else
- switch

Iterative statements

- while statement
- do...while statement
- for statement

Jump statements

- break
- continue
- goto

5.1 Compound Statement or Block

A compound statement or a block is a group of statements enclosed within a pair of curly braces { }. The statements inside a block are executed sequentially. The general form is-

```
statement1;
statement2;
.....
.....
```

For example-

```
l=4;
b=2;
area=l*b;
printf("%d",area);
```

A compound statement is syntactically equivalent to a single statement and can appear anywhere in a program where a single statement is allowed.

5.2 if-else

This is a bi-directional selection control statement which is used to take one of the two possible actions. The syntax of an if-else statement is-

```
if (expression)
    statement1
else
    statement2
```

The expression inside the parentheses is evaluated and if it is true (has a non-zero value), then statement1 is executed and if the expression is false (zero) then statement2 is executed. This expression is often called the condition. The following figure shows the flow chart for an if-else statement.

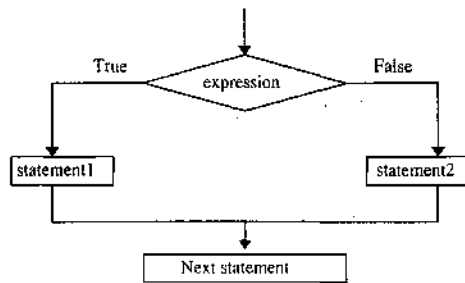


Figure 5.1 Flow chart of if...else control statement

Statement1 and statement2 can be single statements terminated by semicolon or compound statements.

```

if(expression)
{
    statement
}
else
{
    statement
}

if(expression)
    statement1;
else
    statement2;

if(expression)
{
    statement
}
else
{
    statement2
}
  
```

*/*P5.1 Program to print the bigger number*/*

```

#include<stdio.h>
int main(void)
{
    int a,b;
    printf("Enter two numbers : ");
    scanf("%d%d",&a,&b);
    if(a>b)
        printf("Bigger number=%d\n",a);
    else
        printf("Bigger number=%d\n",b);
    return 0;
}
  
```

Output :

Enter two numbers: 11 9
Bigger number=11

The following program prints whether a number is even or odd, and if the number is odd it is multiplied by 2.

*/*P5.2 Program to print whether a number is even or odd*/*

```

#include<stdio.h>
int main(void)
{
    int num;
    printf("Enter a number : ");
    scanf("%d",&num);
    if(num%2 == 0) /*test for even*/
        printf("Number is even\n");
    else
    {
        printf("Number is odd\n");
        num*=2;
        printf("Now the number is %d\n",num);
    }
    return 0;
}
  
```

Output :

Enter a number : 15

Control Statements

Number is odd
Now the number is,30

The else part in the if-else statement is optional, i.e. an if-else statement can be written with or without the else clause. In case of missing else the syntax would be-

```

if(expression)
    statement1;

if(expression)
{
    statement
}
  
```

As stated before, there can be a single statement or a block of statements after the if part. If the expression is true, then statement1 is executed and if it is false then the control transfers to the next statement which is immediately after the if control statement.

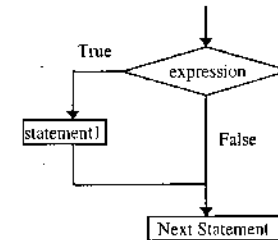


Figure 5.2 Flow chart of if control statement

*/*P5.3 Program to print a message if negative number is entered*/*

```

#include<stdio.h>
int main(void)
{
    int num;
    printf("Enter a number : ");
    scanf("%d",&num);
    if(num<0)
    {
        printf("Number entered is negative\n");
        num=-num;
    }
    printf("Value of num is : %d\n", num);
    return 0;
}
  
```

Output :

Enter a number : -6
Number entered is negative
Value of num is : 6
Output :
Enter a number : 8
Value of num is : 8

We can use boolean operators for combining conditions. For example, suppose we want to take some action if the value of variable is more than 10 and less than 25, then we can write an if statement as-

```

if(var>10 && var<25)
    statement1
  
```

Now the statement1 will be executed only when both var>10 and var<25 are true.

Sometimes coding shortcuts are used while writing the conditions.

(i) if (exp!=0) is often written as if (exp)

The value of expression (exp!=0) will be true if exp is non-zero and false if exp is zero. The value of expression (exp) will be true if exp is non-zero(as non-zero is true in C) and false if exp is zero. Hence

we can write `if (exp)` instead of `if (exp!=0)`.

So whenever you see code if this form-

```
if (exp)
    statement1
```

read it as - if exp is non-zero execute statement1.

(ii) `if (exp==0)` is often written as `if (!exp)`

The value of expression `(exp==0)` will be true if exp is zero and false if exp is non-zero. The expression `(!exp)` will be true if exp is zero and false if exp is non-zero. Hence we can write `if (!exp)` instead of `if (exp==0)`.

So whenever you see code if this form-

```
if (!exp)
    statement2
```

read it as - if exp is zero execute statement2.

*/*PS.4 Program to find quotient and remainder*/*

```
#include<stdio.h>
int main(void)
{
    int x,y,quo,rem;
    printf("Enter two numbers : ");
    scanf("%d%d",&x,&y);
    if (y) /*if y is non-zero*/
    {
        quo=x/y;
        rem=x%y;
        printf("Quotient = %d,Remainder=%d\n",quo,rem);
    }
    else
        printf("Divide by zero error\n");
    return 0;
}
```

We could write the if statement as-

```
if(!y) /*if y is zero*/
    printf("Divide by zero error\n");
else
{
    quo = x/y;
    rem = x%y;
    printf("Quotient = %d, remainder=%d\n",quo,rem);
}
```

5.2.1 Nesting of if...else

We can have another if...else statement in the if part or the else part. This is called nesting of if...else statements. Here is an example of nesting where we have if...else inside both if and else parts.

```
if (expression1)
{
    if (expression2)
        statementA1
    else
        statementA2
}
else
{
    if (expression3)
        statementB1
    else
        statementB2
}
```

The following two programs show nesting of if-else statements.

*/*PS.5 Program to find biggest number from three given numbers*/*

```
#include<stdio.h>
int main(void)
{
    int a,b,c,big;
    printf("Enter three numbers : ");
    scanf("%d%d%d",&a,&b,&c);
    if (a>b)
    {
        if (a>c)
            big=a;
        else
            big=c;
    }
    else
    {
        if (b>c)
            big=b;
        else
            big=c;
    }
    printf("Biggest number is %d\n",big);
    return 0;
} /*End of main()*/
```

The braces in this if-else structure can be dropped and we can write our code like this-

```
if (a>b)
    if (a>c)
        large=a;
    else
        large=c;
else
    if (b>c)
        large=b;
    else
        large=c;
```

It is valid to write so because an if-else statement is a single statement. In the `if (a>b)` part there is single if-else statement and in its else part also there is a single if-else statement. But it is advisable put the braces as it makes the code clear and also avoids the dangling else problem described in section 5.2.2.

The next program finds whether a given year is leap or not. A centennial(divisible by 100) year is leap if it divisible by 400, and a non-centennial year is leap if it is divisible by 4.

*/*PS.6 Program to find whether a year is leap or not*/*

```
#include<stdio.h>
int main(void)
{
    int year;
    printf("Enter year : ");
    scanf("%d",&year);
    if (year%100 != 0)
    {
        if (year%4 == 0)
            printf("Leap year\n");
        else
            printf("Not leap\n");
    }
    else
    {
        if (year%400 == 0)
            printf("Leap year\n");
        else
            printf("Not leap\n");
    }
    return 0;
}
```

Note that we can write this program using a single if-else statement and && and || operators.

```
if(year%100!=0 && year%4==0 || year%400==0)
    printf("Leap year\n");
else
    printf("Not leap\n");
```

5.2.2 Dangling else problem

Since the else part in an if-else statement is optional, it is possible that in the nested if-else statements an else is omitted. In this case confusion may arise in associating else part with the appropriate if part. Let us take an example-

```
if(grade=='A')
    if(marks>95)
        printf("Excellent");
else
    printf("Work hard for getting A grade");
```

The indentations and the printf statements suggest that we want to match the else with the first if. But remember the indentations are only for the readers of the program and not for the compiler. The compiler will always associate the else part with closest unmatched if part. So in this case the else part is matched with the second if which is if(marks>95).

```
if(grade=='A')
    if(marks>95)
        printf("Excellent");
else
    printf("Work hard for getting A grade");
```

Now suppose the value of grade is 'A' and value of marks is 94, then the output will be 'work hard for getting A grade'.

We get this problem because there is an else missing in the nested if-else structure. The if(marks>95) does not have an else part. We should tell the compiler about this by putting braces at proper place.

```
if(grade=='A')
{
    if(marks>95)
        printf("Excellent");
}
else
    printf("Work hard for getting A grade");
```

Whenever there is doubt regarding matching of if and else parts we can use braces to enclose each if and else part.

5.2.3 else-if Ladder

This is a type of nesting in which there is an if-else statement in every else part except the last else part. This type of nesting is frequently used in programs for writing multi-way decisions and is also known as else-if ladder or else-if chain.

```
if(expression1)
    statementA
else
    if(expression2)
        statementB
    else
        if(expression3)
            statementC
        else
            statementD
```

It is generally indented like this to avoid deep indentation.

```
if(expression 1)
    statementA
else if(expression 2)
    statementB
else if(expression 3)
    statementC
else
    statementD
```

The flow chart for this structure is-

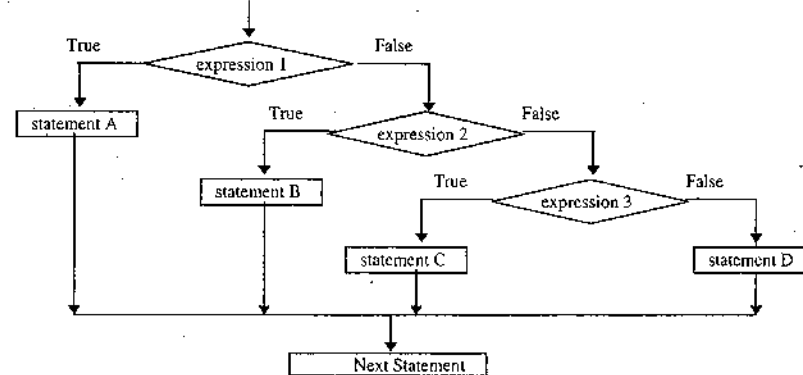


Figure 5.3 Flow chart of else-if ladder statement

Here each expression is evaluated one by one in order, and when an expression is found to be true, the statement or block of statements corresponding to that expression is executed, and the control comes out of the nested structure without checking remaining conditions. If none of the conditions is true then the last else part is executed. If the last else and its statement are absent, then no action takes place.

/*PS.7 Program to find out the grade of a student when the marks of 4 subjects are given.

```
The method of assigning grade is-
percentage >= 85      grade=A
percentage < 85 and percentage >= 70  grade=B
percentage < 70 and percentage >= 55  grade=C
percentage < 55 and percentage >= 40  grade=D
percentage < 40      grade=E
```

```
#include<stdio.h>
int main(void)

{
    float m1,m2,m3,m4,total,per;
    char grade;
    printf("Enter marks of 4 subjects : ");
    scanf("%f%f%f%f",&m1,&m2,&m3,&m4);
    total=m1+m2+m3+m4;
    per=total/4;
    if(per>=85)
        grade='A';
    else if(per>=70)
        grade='B';
    else if(per>=55)
        grade='C';
    else if(per>=40)
        grade='D';
    else
        grade='E';
}
```

```
printf("Percentage is %F, Grade is %c\n", per, grade);
return 0;
}
```

If we don't use the else-if ladder, the equivalent code for this problem would be-

```
if(per >= 85)
    grade = 'A';
if(per < 85 && per >= 70)
    grade = 'B';
if(per < 70 && per >= 55)
    grade = 'C';
if(per < 55 && per >= 40)
    grade = 'D';
if(per < 40)
    grade = 'E';
```

In else..if ladder whenever a condition is found true other conditions are not checked, while in this case all the conditions will always be checked wasting time, and moreover the conditions here are lengthy.

5.3 Loops

Loops are used when we want to execute a part of the program or a block of statements several times. For example, suppose we want to print "C is the best" 10 times. One way to get the desired output is to write 10 printf statements, which is not preferable. Other way out is- use loop. Using loop we can write one loop statement and only one printf statement, and this approach is definitely better than the first one. With the help of loop we can execute a part of the program repeatedly till some expression is true. In C, the loops can be written using these three iterative statements.

- (i) while statement
- (ii) do-while statement
- (iii) for statement

5.3.1 while statement

The syntax of while statement is-

```
while (expression)
    statement
```

Like if-else statement here also we can have either a single statement or a block of statements, and here it is known as the body of loop. Now let us see how this loop works.

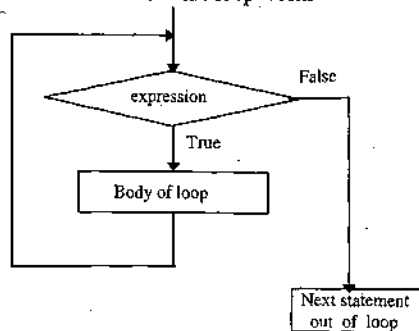


Figure 5.4 Flow chart of while loop

First the expression is evaluated; if it is true then the statements in the body of loop are executed. After the execution, again the expression is evaluated and if it is found to be true then again the statements in the body of loop are executed. This means that these statements are executed continuously till the expression is true and when it becomes false, the loop terminates and the control comes out of the loop. Each execution of the loop body is known as iteration, and the expression is known as loop condition.

```
/*P5.8 Program to print numbers from 1 to 10 using while loop*/
#include <stdio.h>
int main(void)
{
    int i=1;
    while(i <= 10)
    {
        printf("%d\t", i);
        i=i+1; /*Statement that changes the value of condition*/
    }
    printf("\n");
    return 0;
}
```

Output:
1 2 3 4 5 6 7 8 9 10

Here initially the condition ($i \leq 10$) is true, because the value of i is 1. So the loop body is executed i.e. value of i is printed and incremented by 1. Now one iteration of the loop is over and the loop condition is again checked. The condition ($i \leq 10$) is again true because i is 2. So the loop body is again executed i.e. value of i is again printed and increased. Now two iterations of the loop are over. The body of the loop will be executed long as the condition ($i \leq 10$) is true. In each iteration of the loop, value of i increases and when the value of i equals 11 the condition ($i \leq 10$) becomes false and the loop terminates i.e. control is transferred to the next statement following the loop, which is `printf("\n")`.

If we write the condition as ($i < 10$), then the numbers will be printed only upto 9, because the loop condition becomes false when the value of i equals 10. This type of logical error is known as off-by-one error.

Note that inside the body of the loop there should be a statement that alters the value of the condition, so that the condition becomes false ultimately at some point. In our program this statement is `i=i+1`.

```
/*P5.9 Program to print numbers in reverse order with a difference of 2*/
#include <stdio.h>
int main(void)
{
    int k=10;
    while(k >= 2)
    {
        printf("%d\t", k);
        k=k-2;
    }
    printf("\n");
    return 0;
}
```

Output:
10 8 6 4 2

```
/*P5.10 Program to print the sum of digits of any number*/
#include <stdio.h>
int main(void)
{
    int n, sum=0, rem;
    printf("Enter a number : ");
    scanf("%d", &n);
    while(n > 0)
    {
        rem = n%10; /*taking last digit of n*/
        sum += rem;
    }
}
```

```

    n/=10;      /*skipping last digit of n*/
}
printf("Sum of digits = %d\n",sum);
return 0;
}

```

Output :

Enter a number : 1452

Sum of digits = 12

Here we are extracting the digits of the number from right to left and then these digits are added one by one to the variable sum. Note that the variable sum is initialized to 0. This is because we are adding some numbers to it, and if not initialized then these numbers will be added to garbage value present in it. Let's see how the loop works when the value of n is 1452.

Before loop starts-	Value of rem : garbage value,	Value of sum : 0,	Value of n : 1452
After 1st iteration-	Value of rem : 1452%10 = 2,	Value of sum : 0+2 = 2,	Value of n : 1452/10=145
After 2nd iteration-	Value of rem : 145%10 = 5,	Value of sum : 2+5 = 7,	Value of n : 145/10=14
After 3rd iteration-	Value of rem : 14%10 = 4,	Value of sum : 7+4 = 11,	Value of n : 14/10=1
After 4th iteration-	Value of rem : 1%10 = 1,	Value of sum : 11+1 = 12,	Value of n : 1/10=0

Now since the value of n is equal to zero, the condition (n>0) becomes false and the loop stops.

/*P5.11 Program to find the product of digits of any number*/

```

#include<stdio.h>
int main(void)
{
    int n,prod=1,rem;
    printf("Enter a number : ");
    scanf("%d",&n);
    while(n>0)
    {
        rem = n%10; /*taking last digit of n*/
        prod*=rem;
        n/=10;      /*skipping last digit of n*/
    }
    printf("Product of digits = %d\n",prod);
    return 0;
}

```

Output :

Enter a number : 234

Product of digits = 24

The logic of extracting digits is similar to that in earlier program, but here we are multiplying the digits instead of adding, so here the variable prod is initialized to 1.

If the number entered is 0, the output should be 0 but this program will give output 1. To correct this problem we can add an if...else statement as given next or use do...while loop described in the next section.

```

if(n==0)
    prod=0;
else
    while(n>0)
    {
        rem = n%10; /*taking last digit of n*/
        prod*=rem;
        n/=10;      /*skipping last digit of n*/
    }
}

```

The next program finds the factorial of a number. The factorial of a number n is the product of numbers from 1 to n.

$n! = n * (n-1) * (n-2) * \dots * 2 * 1$

$4! = 4 * 3 * 2 * 1 = 24$

$8! = 8 * 7 * 6 * 5 * 4 * 3 * 2 * 1 = 40320$

/*P5.12 Program to find the factorial of any number*/

```

#include<stdio.h>
int main(void)
{
    int n,num;
    long fact=1;
    printf("Enter a number : ");
    scanf("%d",&n);
    num=n;
    if(n<0)
        printf("No factorial of negative number\n");
    else
    {
        while(n>1)
        {
            fact*=n;
            n--;
        }
        printf("Factorial of %d=%ld\n",num,fact);
    }
    return 0;
}

```

Output :

Enter a number : 4

Factorial of 4 = 24

We have taken the variable fact of type long instead of type int. This is so because the factorials grow at a very fast rate, and they may exceed the range of an int.

The next program converts a binary number to decimal. For this we extract binary digits from right and add them after multiplying by powers of 2. This is somewhat similar to the program P5.10, only we have to multiply the digits by powers of 2 before adding them.

/*P5.13 Program to convert a binary number to a decimal number*/

```

#include<stdio.h>
int main(void)
{
    int n,nsave,rem,d,j=1,dec=0;
    printf("Enter the number in binary : ");
    scanf("%d",&n);
    nsave=n;
    while(n>0)
    {
        rem=n%10; /*taking last digit of n*/
        d=rem*j;
        dec+=d;
        j*=2;
        n/=10; /*skipping last digit of n*/
    }
    printf("Binary number=%d,Decimal number=%d\n",nsave,dec);
    return 0;
}

```

Output :

Enter the number in binary : 1101

Binary number=1101, Decimal number=13

This is how the loop works for binary number 1101.

Before loop starts-	rem:garbage,	d: garbage,	dec:0,	j:1,	n:1101	nsave:1101
After 1st iteration-	rem:1101%10=1,	d:1*1,	dec:1,	j:2,	n:1101/10=110	nsave:1101
After 2nd iteration-	rem:110%10=0,	d:0*2,	dec:1+0,	j:4,	n:110/10=11	nsave:1101
After 3rd iteration-	rem:11%10=1,	d:1*4,	dec:1+0+4,	j:8,	n:11/10=1	nsave:1101
After 4th iteration-	rem:1%10=1,	d:1*8,	dec:1+0+4+8,	j:16,	n:1/10=0	nsave:1101

Now the value of *n* becomes zero so the loop terminates. We have taken a variable *nsave* to save the value of binary number, because the value of *n* gets changed after the loop execution. (For entering large binary take long int).

5.3.2 do-while statement

The do-while statement is also used for looping. The syntax of do-while statement is-

```
do
    statement
while(expression);

do
{
    statement
    statement
}while(expression);
```

Note that unlike while loop, here a semicolon is placed after the parentheses containing the expression.

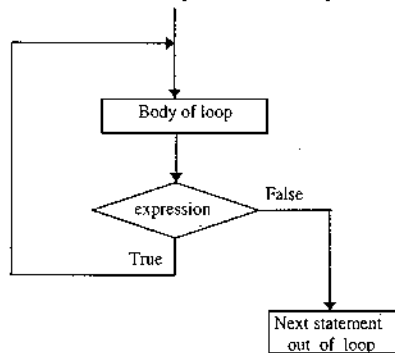


Figure 5.5 Flow chart of do-while loop

Here first the loop body is executed and then the expression is evaluated. If the expression is true, then again the loop body is executed and this process continues as long as the expression is true. When the expression becomes false the loop terminates.

In a while loop, first the expression is evaluated and then the loop body is executed whereas in a do-while loop, first the loop body is executed and then the expression is evaluated. If initially the condition is false then in the case of while loop, the loop body will not execute at all while in the case of do-while loop, the loop body will execute one time. So in a do-while loop, the loop body is guaranteed to execute at least once.

*/*P5.14 Program to print numbers from 1 to 10 using a do-while loop*/*

```
#include<stdio.h>
int main(void)
{
    int i=1;
    do
    {
        printf("%d\t",i);
        i=i+1;
    }while(i<=10);
    printf("\n");
    return 0;
}
```

Output: 1 2 3 4 5 6 7 8 9 10

Generally while loop is used more frequently than the do-while loop, but in some situations it is better to check the condition at the bottom of loop. For example in the next two programs it is better to use a do-while loop.

*/*P5.15 Program to count digits in a number*/*

```
#include<stdio.h>
int main(void)
{
    int n,count=0,rem;
    printf("Enter a number : ");
    scanf("%d",&n);
    do
    {
        n/=10;
        count++;
    }while(n>0);
    printf("Number of digits=%d\n",count);
    return 0;
}
```

If we write the same program using a while loop then we won't get correct answer when input number is 0.

```
while(n>0)
{
    n/=10;
    count++;
}
```

The count of digits in number 0 is 1, but using while loop the answer will come out to be zero. Similarly we can use do-while loop in program P5.11 to get correct output for input 0.

*/*P5.16 Program to find the sum of numbers entered*/*

```
#include<stdio.h>
int main(void)
{
    int num,sum=0;
    do
    {
        printf("Enter a number (enter 0 to stop) : ");
        scanf("%d",&num);
        sum+=num;
    }while(num!=0);
    printf("Sum is %d\n",sum);
    return 0;
}
```

In this program if we use while loop then the condition will be checked at the top, so either we'll have to give some initial value to the variable *num* or we'll have to write duplicate *printf* and *scanf* statements before the loop.

```
printf("Enter a number (enter 0 to stop) : ");
scanf("%d",&num);
sum+=num;
while(num!=0)
```

```
printf("Enter a number (enter 0 to stop) : ");
scanf("%d",&num);
sum+=num;
```

The do-while loop can be used for checking validity of entered data. Suppose we want the user to enter an employee ID, and the valid IDs are in the range 100 to 500 only. If the user enters an invalid ID, we want to ignore that ID and again ask him to enter another one, and we want this process to continue till he enters a valid ID. In this type of situation we can use do-while loop.

```
printf("Enter employee ID (100-500): ");
scanf("%d", &emp_id);
while(emp_id < 100 || emp_id > 500);
```

his loop will terminate only when the entered number is in the valid range 100 to 500.

3.3 for statement

The for statement has three expressions, semicolons are used for separating these expressions. The syntax of for statement is-

```
for(expression1; expression2; expression3)
    statement
for(expression1; expression2; expression3)
    statement
    statement
```

The loop body can be a single statement or block of statements.

expression1 is an initialization expression, expression2 is a test expression or condition and expression3 is an update expression. expression1 is executed only once when the loop starts and is used to initialize the loop variables. This expression is generally an assignment expression. expression2 is a condition and is tested before each iteration of the loop. This condition generally uses relational and logical operators. expression3 is an update expression and is executed each time after the body of the loop is executed.

Now let us see how this loop works. Firstly the initialization expression(expression1) is executed and the loop variables are initialized, and then the condition(expression2) is checked, if the condition is true then the body of loop is executed. After executing the loop body, control transfers to update expression(expression3) and it modifies the loop variables and then again the condition is checked, and if it is true, the body of loop is executed. This process continues till the condition is true and when the condition becomes false the loop is terminated and control is transferred to the statement following the loop.

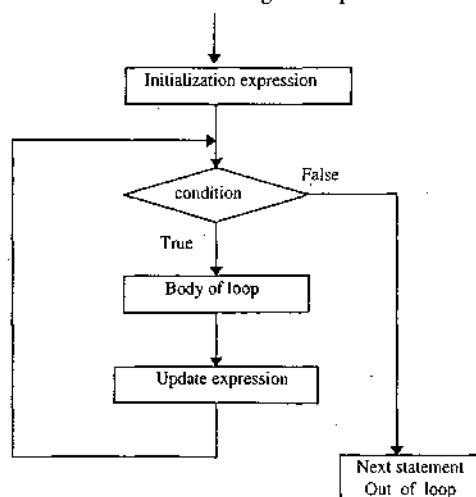


Figure 5.6 Flow chart of for loop

The work done by the for loop can be performed by writing a while loop as-

```
expression 1;
while(expression 2)
{
    statement
    .....
    expression 3;
}
```

Although the task of while and for loops is same, the for loop is generally used when the number of iterations are known in advance and when the initialization and update expressions are simple and related.

```
/*P5.17 Program to print numbers from 1 to 10 using for loop*/
#include<stdio.h>
int main(void)
```

```
{
    int i;
    for(i=1; i<=10; i++)
        printf("%d\t", i);
    printf("\n");
    return 0;
}
```

Output: 1 2 3 4 5 6 7 8 9 10

```
/*P5.18 Program to print numbers in reverse order with a difference of 2*/
int main(void)
```

```
{
    int k;
    for(k=10; k>=2; k-=2)
        printf("%d\t", k);
    printf("\n");
    return 0;
}
```

Output: 10 8 6 4 2

```
/*P5.19 Multiply two positive numbers without using * operator*/
#include<stdio.h>
int main(void)
```

```
{
    int a,b,i;
    int result=0;
    printf("Enter two numbers to be multiplied : ");
    scanf("%d%d", &a, &b);
    for(i=1; i<=b; i++)
        result=result+a;
    printf("%d * %d = %d\n", a,b,result);
    return 0;
}
```

a 6
b 3
Result = 0+2+2+2+2+2
Res = 2+2+2
Ans = 6

```
/*P5.20 Find the sum of this series upto n terms - 1 + 2 + 4 + 7 + 11 + 16 + .....*/
#include<stdio.h>
int main(void)
```

```
{
    int i,n,sum=0,term=1;
    printf("Enter number of terms : ");
    scanf("%d", &n);
    for(i=1; i<=n; i++)
    {
        sum+=term;
        term = term+i;
    }
    printf("The sum of series upto %d terms is %d\n", n,sum);
}
```



```

return 0;
}

/*P5.21 Program to generate fibonacci series
1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89.....
In this series each number is a sum of the previous two numbers*/
#include<stdio.h>
int main(void)
{
    long x,y,z;
    int i,n;
    x=0;
    y=1;
    printf("Enter the number of terms : ");
    scanf("%d",&n);
    printf("%ld ",y);
    for(i=1; i<n; i++)
    {
        z=x+y;
        printf("%ld ",z);
        x=y;
        y=z;
    }
    printf("\n");
    return 0;
}

```

All the three expressions of the for loop are optional. We can omit any one or all the three expressions in the for loop but in any case the two separating semicolons should always be present. expression1 is omitted when the initialization work is done before entering the loop. expression2 is a condition and if omitted, it is always assumed to be true and so this type of loop will never stop executing. This type of loop is infinite and to avoid it there should be a statement inside the loop that takes the control out of the loop.

expression3 is an update expression and is omitted when it is present inside the body of the loop.

```

/*P5.22 Program to print the sum of digits of any number using for loop*/
#include<stdio.h>
int main(void)
{
    int n,sum=0,rem;
    printf("Enter a number : ");
    scanf("%d",&n);
    for( ; n>0; n/=10)
    {
        rem=n%10;      /*taking last digit of number*/
        sum+=rem;
    }
    printf("Sum of digits=%d\n",sum);
    return 0;
}

```

The initialization expression and update expression in a for loop can have any number of expressions separated by comma operator. The use of comma operator ensures that the expressions are evaluated from left to right. In this way we can have more than one loop control variable which can be initialized and updated in a single for loop.

```

/*P5.23 Program to show the use of comma operator in for loop*/
#include<stdio.h>
int main(void)
{
    int i,j;
    for(i=0,j=10; i<=j; i++,j--)

```

```

        printf("i=%d j=%d\n",i,j);
    return 0;
}

```

Output:

```

i=0 j=10
i=1 j=9
i=2 j=8
i=3 j=7
i=4 j=6
i=5 j=5

```

5.3.4 Nesting of Loops

When a loop is written inside the body of another loop, it is known as nesting of loops. Any type of loop can be nested inside any other type of loop. For example a for loop may be nested inside another for loop or inside a while or do-while loop. Similarly while and do-while loops can be nested.

```

/*P5.24 Program to understand nesting in for loop*/
#include<stdio.h>
int main(void)
{
    int i,j;
    for(i=1; i<=3; i++)          /*outer loop*/
    {
        printf("i=%d\n",i);
        for(j=1; j<=4; j++)      /*inner loop*/
            printf("j=%d\t",j);
        printf("\n");
    }
    return 0;
}

```

Output:

```

i=1
j=1 j=2 j=3 j=4
i=2
j=1 j=2 j=3 j=4
i=3
j=1 j=2 j=3 j=4

```

The outer loop iterates 3 times and for each iteration of the outer loop, the inner loop is executed 4 times.

The next program prints Armstrong numbers. Armstrong number is a three digit number in which the sum of cube of all digits is equal to the number, for example 371 is an Armstrong number since $371 = 3^3 + 7^3 + 1^3 = 27 + 343 + 1$

```

/*P5.25 Program to print Armstrong numbers*/
#include<stdio.h>
int main(void)
{
    int num,n,cube,d,sum;
    printf("Armstrong numbers are :\n");

    for(num=100; num<=999; num++) /*outer loop to generate numbers*/
    {
        n=num;
        sum=0;
        while(n>0) /*inner loop to calculate sum of cube of digits*/
        {
            d=n%10;
            n/=10;
            cube=d*d*d;
            sum=sum+cube;
        } /*End of while loop*/

```

```

    if(num==sum)
        printf("%d\n",num);
}/*End of for loop*/
return 0;

```

Here the outer for loop is used to generate numbers from 100 to 999, and the inner while loop is used to extract digits and find the sum of cube of those digits.

*/*P5.26 Program to find the sum of digits of a number until the sum is reduced to 1 digit.
for example: 538769->38->11->2*/*

```

#include<stdio.h>
int main(void)

```

```

{
    long num;
    int dig,sum;
    printf("Enter a number : ");
    scanf("%ld",&num);
    printf("%ld->",num);
    do
    {
        sum=0;
        while(num!=0)
        {
            dig=num%10;
            sum+=dig;
            num/=10;
        }
        printf("%d\t",sum);
        num=sum;
    }while(num/10!=0); /*while num is more than one digit*/
    return 0;
}

```

Output:
Enter a number : 789988
789988 -> 49 13 4

5.3.5 Infinite Loops

The loops that go on executing infinitely and never terminate are called infinite loops. These loops can only be stopped by ending the program. Sometimes we write these loops by mistake while sometimes we deliberately make use of these loops in our programs. Let us take some examples and see what type of mistakes lead to infinite loops.

(A) `for(i=0; i<=5; i--)`
`printf("%d",i);`

This loop will execute till the value of `i` is less than or equal to 5 i.e. the loop will terminate only when `i` becomes greater than 5. The initial value of `i` is 0 and after each iteration its value is decreasing, hence it will never become greater than 5. So the loop condition will never become false and the loop will go on executing infinitely. For the loop to work correctly we should write `i++` instead of `i--`.

(B) There should be a statement inside the loop body that changes the value of loop variable after each iteration. In for loop this work is done by the update expression but in while and do-while we may forget to change the loop variable and this can lead to infinite loop.

```

int k=1;
do
{
    printf("%d ",k);
    sum = sum+k;
}while(k<5);

```

Here we are not changing the value of `k` inside the loop body and hence the loop becomes infinite.

(C) A common mistake made by beginners is to use the assignment operator(=) where equality operator(==) should have been used. If this mistake is made in the loop condition then it may cause the loop to execute infinitely. For example consider this loop-

```

while(n=2)
{
    .....
}

```

Here we wanted the loop to execute if the value of `n` is equal to 2. So we should have written `while(n==2)` but mistakenly we have written `while(n=2)`. Now `n=2` is an assignment expression and the value of this expression is 2, which is a nonzero(true) value and hence the loop condition is always true.

(D) `float k=2.0;`
`while(k!=3.0)`
`{`
 `printf("%f\n",k);`
 `k = k+0.2;`
`}`

This loop is infinite because the computer represents a floating point value as an approximation of the real value. The computer may represent the value 3.0 as 2.999999 or may be as 3.000001. So our condition (`k!=3.0`) never becomes false. The solution for this problem is to write the condition as (`k<=3.0`).

(E) `int i=1;`
`while(i<=5);`
`printf("%d",i++);`

This loop will produce no output and will go on executing infinitely. The mistake here is that we have put a semicolon after the condition. So this loop is treated as-

```

while(i<=5)
;

```

This null statement is treated as the body of loop and it keeps on executing.

These were some of the examples where infinite loops occur due to mistakes, but sometimes infinite loops are intentionally used in programs. The termination of these loops is controlled and to come out of these loops break, goto or return statements are used.

```

while(1)          for( ; ; )          do
{
    .....
    .....
}                  }                  }while(1);

```

5.4 break statement

break statement is used inside loops and switch statements. Sometimes it becomes necessary to come out of the loop even before the loop condition becomes false. In such a situation, break statement is used to terminate the loop. This statement causes an immediate exit from that loop in which this statement appears. It can be written as-

```
break;
```

When break statement is encountered, loop is terminated and the control is transferred to the statement immediately after the loop. The break statement is generally written along with a condition. If break is written inside a nested loop structure then it causes exit from the innermost loop.

*/*P5.27 Program to understand the use of break*/*
`#include<stdio.h>`
`int main(void)`

```

{
    int n;
    for(n=1; n<=5; n++)
    {

```

```

    if(n==3)
        break;
    printf("Number=%d\n",n);
}
printf("Out of for loop\n");
return 0;

```

Output:
Number=1
Number=2
Out of for loop

This is a simple program, not of much use but illustrates the use of break statement. If there had been no break statement, this loop would have executed 5 times, but here as the value of n becomes equal to 3, break is encountered and loop is terminated.

Now we take a program which checks whether a number is prime or not. A number is prime if it is divisible only by 1 and itself. A number n will be prime if remainders of $n/2$, $n/3$, $n/4$, ..., $n/\sqrt{n}$ are all non-zero, or in other words n is not divisible by any number from 2 to $\sqrt{n}$.

```

/*P5.28 Program to find whether a number is prime or not*/
#include<stdio.h>
#include<math.h>
int main(void)
{
    int i,n;
    printf("Enter a number : ");
    scanf("%d",&n);

    for(i=2; i<=sqrt(n); i++)
        if(n%i==0)
            break;

    if(i>sqrt(n))
        printf("%d is prime\n",n);
    else
        printf("%d is not prime\n",n);
    return 0;
}

```

sqrt() is a library function that returns the square root of a number, we have to include header file math.h when we use this function. The range of for loop is from 2 to $\sqrt{n}$, if the number is divisible by any number from this range, it means the number is not prime and there is no need to check divisibility by other numbers, hence the control is transferred out of the loop by break statement. Control can come out of loop in two situations, after the full execution of loop when loop condition becomes false i.e. when i becomes greater than sqrt(n), or due to break statement. If control comes out after full execution of loop, it means that number is not divisible by any number from 2 to $\sqrt{n}$, and the number is prime.

5.5 continue statement

The continue statement is used when we want to go to the next iteration of the loop after skipping some statements of the loop. The continue statement can be written simply as-

```
continue;
```

It is generally used with a condition. When continue statement is encountered all the remaining statements (statements after continue) in the current iteration are not executed and the loop continues with the next iteration.

The difference between break and continue is that when break is encountered the loop terminates and the control is transferred to the next statement following the loop, but when a continue statement is encountered the loop is not terminated and the next iteration of the loop starts.

In while and do-while loops, after continue statement the control is transferred to the condition and the loop continues if the condition is true, whereas in for loop after continue statement the control is transferred to update expression and then the condition is tested.

```

/*P5.29 Program to understand the use of continue statement*/
#include<stdio.h>
int main(void)
{
    int n;
    for(n=1; n<=5; n++)
    {
        if(n==3)
            continue;
        printf("Number=%d\n",n);
    }
    printf("Out of for loop\n");
    return 0;
}

```

Output:
Number=1
Number=2
Number=4
Number=5
Out of for loop

In this program, when the continue statement is encountered, the control is transferred to the update expression of the for loop.

The following program finds the sum and average of 10 positive integers. If any negative number is entered then a message is displayed and the control is transferred to the test condition of the while loop.

```

/*P5.30 Program to find the sum and average of 10 positive integers*/
#include<stdio.h>
int main(void)
{
    int i=1,n,sum=0;
    float avg;
    printf("Enter 10 positive numbers : \n");
    while(i<=10)
    {
        printf("Enter number %d : ",i);
        scanf("%d",&n);
        if(n<0)
        {
            printf("Enter only positive numbers\n");
            continue;
        }
        sum+=n;
        i++;
    }
    avg=sum/10.0;
    printf("Sum=%d Avg=%f\n",sum,avg);
    return 0;
}

```

The continue statement is used when we don't want to execute the statements in the loop under some abnormal conditions. These abnormal conditions are checked in the beginning of the loop and if they are true then we skip the rest of the loop body using continue.

If we write the loop in this program using an if-else statement, then the condition to be written in the if statement is the opposite of the condition written for continue.

```

while(i<=10)
{
    printf("Enter number %d : ",i);
    scanf("%d",&n);
}

```

```

if (n >= 0)
{
    sum += n;
    i++;
}
else
    printf("Enter only positive numbers\n");

```

The continue statement is generally used to avoid excessive nesting inside a loop.

5.6 goto

This is an unconditional control statement that transfers the flow of control to another part of the program.

The goto statement can be used as-

```

goto label;

.....

label:
    statement
    .....

```

Here label is any valid C identifier and it is followed by a colon. Whenever the statement goto label; is encountered, the control is transferred to the statement that is immediately after the label.

The following program finds whether a number is even or odd. Although it is not a good idea to use goto for this purpose, it tells us about the syntax of goto statement.

```

/*PS.31 Program to find whether a number is even or odd*/
#include <stdio.h>
int main(void)

{
    int n;
    printf("Enter a number : ");
    scanf("%d", &n);
    if (n % 2 == 0)
        goto even;
    else
        goto odd;
even:
    printf("Number is even\n");
    goto end;
odd:
    printf("Number is odd\n");
    goto end;
end:
    printf("\n");
    return 0;
}

```

Output:
Enter a number : 14
Number is even

The label can be placed anywhere. If the label is after goto then the control is transferred forward and it is known as forward jump or forward goto, and if the label is before goto then the control is transferred backwards and it is known as backward jump or backward goto. In forward goto, the statements between goto and label will not be executed and in backward goto statements between goto and label will be executed repeatedly. More than one goto can be associated with the same label but we cannot have same label at more than one place.

The control can be transferred only within a function using goto statement. (Concept of functions will be introduced in next chapter)

There should always be a statement after any label. If label is at the end of program, and no statements are to be written after it, we can write the null statement (single semicolon) after the label because a program can't end with a label.

The use of goto should be avoided, as it is difficult to understand where the control is being transferred. Sometimes it leads to "spaghetti" code, which is not understandable and is very difficult to debug and maintain. We can always perform all our jobs without using goto, and the use of goto is not favoured in structured programming.

Although the use of goto is not preferred, there is a situation where goto can actually make the code simpler and more readable. This situation arises when we have to exit from deeply nested loops. To exit from a single loop we can use the break statement, but in nested loops break will take the control only out of the innermost loop

```

for (.....)
{
    while (.....)
    {
        for (.....)
        {
            if (.....)
                goto stop;
        }
    }
}
stop:
.....

```

As mentioned earlier, we can always write any code without using goto, so here also we have another way of exiting out of the deeply nested loop.

```

flag=0;
for (.....)
{
    while (.....)
    {
        for (.....)
        {
            if (.....)
            {
                flag=1;
                break; /*out of innermost for loop*/
            }
        }
        if (flag==1)
            break; /*out of while loop*/
    }
    if (flag==1)
        break; /*out of outer for loop*/
}

```

We can see that in the case of nested loops, the code using goto is more readable and if it is not used then many tests have to be performed.

5.7 switch

This is a multi-way conditional control statement which helps us to make choice among a number of alternatives. The syntax of switch statement is-

```

switch(expression)
{
    case constant1:
        statement
        .....
    case constant2:
        statement
        .....
    .....
    case constantN:
        statement
        .....
    default:
        statement
        .....
}

```

Here switch, case and default are keywords. The expression following the switch keyword can be any C integral expression. The constant expressions in the case labels should be of integer or character type. These constants must be different from one another i.e. each case label should have a unique value. We can't use floating point or string constants. Multiple constants in a single case are not allowed; each case should be followed by only one constant.

Each case label can be followed by any number of statements. It is also possible that a case has no statement under it. If a case is followed by multiple statements, then it is not necessary to enclose them within pair of curly braces, but it is not an error if we do so. The statements under case can be any valid C statements like if-else, while, for or even another switch statement. Writing a switch statement inside another is called nesting of switches.

Now let us see how the switch statement works. Firstly the switch control expression is evaluated, if the value of expression matches with any case constant expression, then the control is transferred to that case label. If none of the case constant matches with the value of the expression then the control is transferred to the default label. The label default is optional, if it is not present and no case matches then control is transferred to the statement following the switch. These case labels and default label can occur in any order, but it is a good programming practice to put default at the last.

When control is transferred to a case label, all the statements following that case label till the end of the switch statement are executed.

*/*P5.32 Program to understand the switch control statement*/*

```

#include<stdio.h>
int main(void)
{
    int choice;
    printf("Enter your choice : ");
    scanf("%d", &choice);
    switch(choice)
    {
        case 1:
            printf("First\n");
        case 2:
            printf("Second\n");
        case 3:
            printf("Third\n");
        default:
            printf("Wrong choice\n");
    }
    return 0;
}

```

Output :

Control Statements

Enter your choice : 2
Second
Third
Wrong choice

5 + 0 = 5
4 + 1 = 5
3 + 2 = 5

Here the value of choice matches with second case, so control is transferred there and all the statements after case 2 are executed sequentially. The statements under case 3 and default are also executed in addition to the statements of case 2. This is known as falling through cases.

Suppose we don't want the control to fall through the statements of all the cases under the matching case then we can use break statement. If a break statement is encountered inside a switch, all the statements following break are not executed and the control jumps out of the switch. Let us write the previous program using break.

*/*P5.33 Program to understand the switch with break statement*/*

```

#include<stdio.h>
int main(void)
{
    int choice;
    printf("Enter your choice : ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            printf("First\n");
            break;          /*break statement in switch*/
        case 2:
            printf("Second\n");
            break;
        case 3:
            printf("Third\n");
            break;
        default:
            printf("Wrong choice\n");
    }

    return 0;
}
/*End of main()*/
Output :
Enter your choice : 2
Second

```

The value of choice matches with second case, so control is transferred there and after executing printf("Second\n"), the break statement is encountered which takes the control out of the switch. This way all the cases are separated using the break statements. Now when the switch expression matches a case constant, only the statements in that case are executed.

It is not necessary that if break is present inside switch, then it should be present after all case statements. Some cases may have break while others may not have. The use of break inside switch is optional, it may or may not be used depending on the requirement and logic of the program.

The break statement is not logically needed after the last case (or default if it is at the last) but it is a good programming practice to do so because in future we may need to add new cases at the last.

*/*P5.34 Program to perform arithmetic calculations on integers*/*

```

#include<stdio.h>
int main(void)
{
    char op;
    int a, b;
    printf("Enter number, operator and another number : ");
    scanf("%d%c%d", &a, &op, &b);
    switch(op)
    {
        case '+':

```

```

        printf("Result=%d\n", a+b);
        break;
    case '-':
        printf("Result=%d\n", a-b);
        break;
    case '*':
        printf("Result=%d\n", a*b);
        break;
    case '/':
        printf("Result=%d\n", a/b);
        break;
    case '%':
        printf("Result=%d\n", a%b);
        break;
    default:
        printf("Enter valid operator\n");
    } /*End of switch*/
    return 0;
} /*End of main()*/

Output:
Enter number operator and another number : 2*5
Result = 7

```

In this program, the valid operator for multiplication is only '*', if we want to make 'x', 'X' also valid operators for multiplication, then we can modify the switch statement like this.

```

switch(op)
{
    .....
    case '-':
        printf("Result=%d\n", a-b);
        break;
    case 'x':
    case 'X':
    case '*':
        printf("Result=%d\n", a*b);
        break;
    case '/':
    .....
} /*End of switch*/

```

Here we have added two cases which have no statements. In both these cases, statements of case '*' will be executed since there is no break after case 'x' and case 'X'. This is how we can have the same set of statements executed for different case constants. In these types of cases where we deliberately leave out the break statement, it is better to write a comment like FALL THRU or NO BREAK to make our intentions clear.

P5.35 Program to find whether the alphabet is a vowel or consonant

```

#include<stdio.h>
int main(void)
{
    char ch;
    printf("Enter an alphabet : ");
    scanf("%c", &ch);
    switch(ch)
    {
        case 'a':
        case 'e':
        case 'i':
        case 'o':
        case 'u':
            printf("Alphabet is a vowel\n");
            break;
        default:

```

```

        printf("Alphabet is a consonant\n");
    }
    return 0;
}

```

In this program, if someone enters a vowel in uppercase then it will be regarded as a consonant. One solution to this problem is to add 5 more cases for uppercase vowels. But we can solve this problem in another way without adding any cases. The switch expression can be a function call returning an int or char. So change the switch expression to `tolower(ch)` and now our problem will be solved. `tolower()` is a library function which converts uppercase to lowercase, header file `ctype.h` has to be included to use this function.

Switch statement is very important in menu driven programs. Here we'll make a menu driven (dummy) program which is identical to a database application. Although this program performs nothing useful, it shows us the structure of menu driven programs.

```

/*P5.36 A menu driven program using infinite loop and switch*/
#include<stdio.h>
int main(void)
{
    int choice;
    while(1)
    {
        printf("1.Create Database\n");
        printf("2.Insert new record\n");
        printf("3.Modify a record\n");
        printf("4.Delete a record\n");
        printf("5.Display all records\n");
        printf("6.Exit\n");
        printf("Enter your choice : ");

        scanf("%d", &choice);
        switch(choice)
        {
            case 1:
                printf("Database created.....\n\n");
                break;
            case 2:
                printf("Record inserted.....\n\n");
                break;
            case 3:
                printf("Record modified.....\n\n");
                break;
            case 4:
                printf("Record deleted.....\n\n");
                break;
            case 5:
                printf("Records displayed.....\n\n");
                break;
            case 6:
                exit(1);
            default:
                printf("Wrong choice\n");
        }
    } /*End of switch*/
} /*End of while*/
return 0;
} /*End of main()*/

```

Here we have taken switch inside an infinite loop. The function `exit()` is used to terminate the program. Each time this loop executes, all options will be displayed and the user will be asked to enter his choice. Appropriate action will be taken depending on that choice. If choice 6 is entered, then the program will be terminated. In real programs we will write full code inside the cases instead of simple `printf` statements.

The use of continue statement inside a switch statement is not valid. If the switch statement is written inside a loop, then the continue statement can appear in the switch, and this continue will apply to loop and start the next iteration of the loop.

The work done by a switch can be done by using an else-if ladder but using switch is more readable. So if we have to make choices based on different values of an integral expression we should use switch.

Some Additional Problems

Problem 1

Write a program to find whether a date entered is valid or not. Assume that dates between years 1850 and 2050 are valid.

```
/*P5.37 Program to check whether a date is valid or not*/
#include<stdio.h>
int main(void)
{
    int d,m,y;
    int flag=1,isleap=0;

    printf("Enter date(dd/mm/yyyy): ");
    scanf("%d/%d/%d",&d,&m,&y);
    if(y%100!=0 && y%4==0 || y%400==0)
        isleap=1;

    if(y<1850 || y>2050 || m<1 || m>12 || d<1 || d>31)
        flag=0;
    else if(m==2) /*Check for number of days in February*/
    {
        if(d==30 || d==31 || (d==29 && !isleap))
            flag=0;
    }
    else if(m==4 || m==6 || m==9 || m==11)/*Check days in April,June,Sept,Nov*/
    {
        if(d==31)
            flag=0;
    }

    if(flag==0)
        printf("Not a valid date\n");
    else
        printf("Valid Date \n");
    return 0;
}/*End of main()*/
```

Problem 2

Write a program to find difference of two dates in years, months and days. Assume that the dates entered are in valid range and that the first date falls before second date.

```
/*P5.38 Program to get difference of two dates in years, months, days*/
#include<stdio.h>
int main(void)
{
    int d1,d2,d,m1,m2,m,y1,y2,y;
    printf("Enter first date(dd/mm/yyyy): ");
    scanf("%d/%d/%d",&d1,&m1,&y1);
    printf("Enter second date(dd/mm/yyyy): ");
    scanf("%d/%d/%d",&d2,&m2,&y2);
    if(d2<d1)
```

```

    if(m2==3)
    {
        if(y2%100!=0 && y2%4==0 || y2%400==0) /*check for leap year*/
            d2=d2+29;
        else
            d2=d2+28;
    }
    else if(m2==5 || m2==7 || m2==10 || m2==12)
        d2=d2+30;
    else
        d2=d2+31;
    m2=m2-1;
}
if(m2<m1)
{
    y2=y2-1;
    m2=m2+12;
}
y=y2-y1;
m=m2-m1;
d=d2-d1;
printf("Difference of the two dates is : ");
printf("%d years %d months %d days\n",y,m,d);
return 0;
}/*End of main()*/
```

Here we are subtracting first date from second date, so we can get the difference in years, months and day by writing-

```
y=y2-y1;
m=m2-m1;
d=d2-d1;
```

We have assumed that the second date falls after first date, so y_2 will always be greater or equal to y_1 and will always come out positive.

It is possible that m_2 is less than m_1 , in this case m will come out negative, and if d_2 is less than d_1 then will come out to be negative. So before calculating y , m and d we should make sure that m_2 is greater than or equal to m_1 and d_2 is greater than or equal to d_1 .

If d_2 is less than d_1 , then we borrow a month from m_2 and add the days of that month to d_2 . Now since months have different number of days, the days added to d_2 will depend on the month borrowed. We'll always borrow a month that is before m_2 . For example if m_2 is 5(May), then we'll borrow month April from m_2 , so we'll add 30 to d_2 .

```
d2=d2+30;
m2=m2-1;
```

If $m_2 = 3$ (March), then we'll borrow month February from m_2 , so we'll add 28 or 29 (if it is leap year) to d_2 .

```
d2=d2+28;
m2=m2-1;
```

In the program we've done this through nested if-else statements.

If m_2 is less than m_1 , then we borrow 1 year(12 months) from y_2 and add it to m_2 .

```
m2=m2+12; y2=y2-1;
```

Now m_2 will become greater than m_1 .

Problem 3

Write a program to multiply two numbers by Russian peasant method. Russian peasant method multiplies two positive numbers using multiplication by 2, division by 2 and addition. Here the first number is divided by 2(integer division), and the second is multiplied by 2 repeatedly until the first number reduces to 1. Suppose we have to multiply 19 by 25, we write the result of division and multiplication by 2, in the two columns like this-

```

19    25 Add
9     50 Add
4     100
2     200
1     400 Add
      475

```

To get the product we add those values of the right hand column, for which the corresponding left column values are odd. So 25, 50, 400 will be added to get 475, which is the product of 19 and 25. Now we will see how this method can be implemented in program.

```

/*P5.39 Program to multiply two numbers by Russian peasant method*/
#include<stdio.h>
int main(void)
{
    int a,b,x,y,s=0;
    printf("Enter two numbers to be multiplied : ");
    scanf("%d%d",&x,&y);
    a = x;
    b = y;
    while(a>=1)
    {
        if(a%2!=0)
            s=s+b;
        a/=2;
        b*=2;
    }
    printf("%d * %d = %d\n",x,y,s);
    return 0;
}

```

Problem 4

Write a currency program, that tells you how many 100, 50, 20, 10, 5, 2 and 1 Rs notes will be needed for a given amount of money. For example if the total amount is Rs 545 then five 100 Rs notes, two 20 Rs note and one 5 Rs note will be needed.

```

/*P5.40 Program to find out the number of notes required for a given amount of money*/
#include<stdio.h>
int main(void)
{
    int n,choice,notes;
    printf("Enter the total amount in Rs : ");
    scanf("%d",&n);
    printf("Enter the value of note from which you want to begin : ");
    scanf("%d",&choice);
    switch(choice)
    {
        case 100:
            notes=n/100;
            printf("Number of 100 Rs notes = %d\n", notes);
            n=n%100;
        case 50:
            notes=n/50;
            printf("Number of 50 Rs notes = %d\n", notes);
            n=n%50;
        case 20:
            notes=n/20;
            printf("Number of 20 Rs notes = %d\n", notes);
            n=n%20;
        case 10:
            notes=n/10;

```

```

        printf("Number of 10 Rs notes = %d\n", notes);
        n=n%10;
    case 5:
        notes=n/5;
        printf("Number of 5 Rs notes = %d\n", notes);
        n=n%5;
    case 2:
        notes=n/2;
        printf("Number of 2 Rs notes = %d\n", notes);
        n=n%2;
    case 1:
        notes=n/1;
        printf("Number of 1 Rs notes = %d\n", notes);
        break;
    default:
        printf("Enter only valid values");
        break;
    }
    printf("\n");
    return 0;
}

```

Output:

```

Enter the total amount in Rs : 748
Enter the value of note from which you want to begin : 50
Number of 50 Rs notes = 14
Number of 20 Rs notes = 2
Number of 10 Rs notes = 0
Number of 5 Rs notes = 1
Number of 2 Rs notes = 1
Number of 1 Rs notes = 1

```

Problem 5

Write a program that finds out the day of week from a given date. The formula for calculating the day is-

$$\text{day} = (y + j + f - h + fh) \% 7;$$

j = julian day of the date

y = year of given date(in 4 digits)

f = int part of (y-1)/4

h = int part of (y-1)/100

fh = int part of (y-1)/400

The value of variable day tells us the day of week.

Value of variable day	Name of day of week
0	Saturday
1	Sunday
2	Monday
3	Tuesday
4	Wednesday
5	Thursday
6	Friday

In the above formula, julian day of a date represents the day of year on which the date falls. Julian day of 1st Jan is 1, of 2nd Feb is 33, of 31st Dec is 365(366 if leap year).

Now let's decide the structure of our program. Once we get the value of variable day, we can use switch to print the name of day of the week from the value of day.

To calculate the value of day we will have to find out the value of julian day. Let's assume that the day, month and year of entered date are stored in variables d, m and y respectively. We take another variable j for the value of julian day.

Initially the value of j is taken equal to d, and then days of previous months are added to j to get the value of julian day. We can use switch statement for this and take m as the switch variable.

```
j=d;
switch(m)
{
    case 2: j+=31; break;
    case 3: j+=31+28; break;
    case 4: j+=31+28+31; break;
    case 5: j+=31+28+31+30; break;
    case 6: j+=31+28+31+30+31; break;
    case 7: j+=31+28+31+30+31+30; break;
    case 8: j+=31+28+31+30+31+30+31; break;
    case 9: j+=31+28+31+30+31+30+31+31; break;
    case 10: j+=31+28+31+30+31+30+31+31+30; break;
    case 11: j+=31+28+31+30+31+30+31+31+30+31; break;
    case 12: j+=31+28+31+30+31+30+31+31+30+31+30; break;
}
```

Suppose the date is 2nd May 2002, then d=2, m=5, so the control will go in case 5, and julian day will be 2+31+28+31+30 = 122

The above logic applied in switch statement was simple but the whole switch statement is confusing to read and understand. If we omit the break statements, take the cases in descending order, and take the switch variable as m-1 then we can accomplish our job by writing a very concise switch statement.

```
j=d;
switch(m-1)
{
    case 11: j+=30; /*Fall thru in all cases*/
    case 10: j+=31;
    case 9: j+=30;
    case 8: j+=31;
    case 7: j+=31;
    case 6: j+=30;
    case 5: j+=31;
    case 4: j+=30;
    case 3: j+=31;
    case 2: j+=28;
    case 1: j+=31;
}
```

Here if d=2, m=5 then control will go in case 4(5-1), and since there is no break, all statements of cases 4, 3, 2, 1 will be executed. Hence $j = 2 + 30 + 31 + 28 + 31 = 122$. This shows that we can use the flexibility of switch statement according to the need and logic of the program.

We have found out the julian day, but what if the year is leap. In that case we'll have to add 1 to the julian day, if the month is other than January or February. This is how we'll do it.

```
if(y%100!=0 && y%4==0 || y%400==0) /*check for leap year*/
    if(m>2)
        j=j+1;
```

Recall that a non centennial year is leap if it is divisible by 4, and a centennial year (divisible by 100) is leap if it is divisible by 400.

/*P5.41 Program to find day of week from a given date*/

```
#include<stdio.h>
int main(void)
{
    int d,m,y,j,f,h,fh,day;
    printf("Enter date(dd/mm/yyyy): ");
    scanf("%d/%d/%d",&d,&m,&y);
    j=d;
```

```
switch(m-1)
{
    case 11: j+=30; /*Fall thru in all cases*/
    case 10: j+=31;
    case 9: j+=30;
    case 8: j+=31;
    case 7: j+=31;
    case 6: j+=30;
    case 5: j+=31;
    case 4: j+=30;
    case 3: j+=31;
    case 2: j+=28;
    case 1: j+=31;
}
if(y%100!=0 && y%4==0 || y%400==0)
    if(m>2)
        j=j+1;
f=(y-1)/4;
h=(y-1)/100;
fh=(y-1)/400;
day=(y+j+f-h+fh)%7;
switch(day)
{
    case 0: printf("Saturday\n"); break;
    case 1: printf("Sunday\n"); break;
    case 2: printf("Monday\n"); break;
    case 3: printf("Tuesday\n"); break;
    case 4: printf("Wednesday\n"); break;
    case 5: printf("Thursday\n"); break;
    case 6: printf("Friday\n"); break;
}
return 0;
} /*End of main()*/
```

Problem 6

Write a program to print triad numbers. Any three numbers will be triad numbers if they satisfy the following conditions-

1. Each number is a three digit number.
2. All the digits in the three numbers (total 9 digits) should be different.
3. Second number should be twice the first number and third number should be thrice the first number. For example-

219	438	657
267	534	801

/*P5.42 Program to print triad numbers*/

```
#include<stdio.h>
int main(void)
{
    int m,n,p,num;
    int i,k,d1,d2,d3;
    for(num=100; num<=999/3; num++) /*Loop A*/
    {
        for(i=num; i<=3*num; i+=num) /*loop B*/
        {
            k=i;
            d1=k%10; k/=10;
            d2=k%10; k/=10;
            d3=k%10; k/=10;
            if(d1==d2 || d2==d3 || d3==d1)
                goto nextnum;
        } /*End of loop B*/
    }
```

```

for(m=num; m>0; m/=10)/*Loop C*/
{
    d1=m%10;
    for(n=num*2; n>0; n/=10)/*Loop D*/
    {
        d2=n%10;
        for(p=num*3; p>0; p/=10)/*Loop E*/
        {
            d3=p%10;
            if(d1==d2 || d2==d3 || d1==d3)
                goto nextnum;
        }/*End of Loop E*/
    }/*End of Loop D*/
}/*End of loop C*/
printf("%d %d %d\t", num, num*2, num*3);
nextnum:

}/*End of loop A*/
return 0;
}/*End of main()*/

```

Loop A is used to generate numbers. Since triad numbers are three digit numbers, this loop starts from 100. The upper limit of this loop is 333 (999/3) because for numbers more than 333, 3*num will not be a three digit number.

Loop B is inside loop A, and it executes three times, for $i = \text{num}$, $i = 2 * \text{num}$ and $i = 3 * \text{num}$. It finds out whether the three digits in a number(i) are different or not. In this loop d1, d2, d3 are digits of a single number, and if any two digits are found to be same then we again go to the update expression of loop A and check for next number.

Loops C, D, E are nested loops which find out whether there are any common digits in the three numbers(num, 2*num, 3*num). Here d1, d2, d3 are digits of num, 2*num, 3*num respectively. If any two digits are same then we go to the update expression of loop A to check for next number.

Note that in loops C, D and E, the digits of different numbers are compared, and in loop B digits of a single number are compared.

Problem 7

Write a program to find out the Least Common Multiple and Highest Common Factor of two numbers.

*/*P5.43 Program to find the LCM and HCF of two numbers*/*

```

#include<stdio.h>
int main(void)
{
    int x,y,a,b;
    printf("Enter two numbers : ");
    scanf("%d%d",&x,&y);
    a=x; b=y;
    while(a!=b)
    {
        if(a<b)
            a=a+x;
        else
            b=b+y;
    }
    printf("LCM of %d and %d is %d\n",x,y,a);
    a=x; b=y;
    while(a!=b)
    {
        if(a>b)
            a=a-b;
        else
            b=b-a;
    }
}

```

```

}
printf("HCF of %d and %d is %d\n",x,y,a);
return 0;
}

```

Output:

```

Enter two numbers : 60 135
LCM of 60 and 135 is 540
HCF of 60 and 135 is 15

```

Pyramids

*	1	1	2	1	1	5	5
**	2 2	1 2	3 4	0 1	2 3	5 4	4 4
***	3 3 3	1 2 3	4 5 6	1 0 1	4 5 6	5 4 3	3 3 3
****	4 4 4 4	1 2 3 4	5 6 7 8	0 1 0 1	7 8 9 10	5 4 3 2	2 2 2 2
*****	5 5 5 5 5	1 2 3 4 5	6 7 8 9 10	1 0 1 0 1	11 12 13 14 15	5 4 3 2 1	1 1 1 1 1
(a)	(b)	(c)	(d)	(e)	(f)	(g)	(h)

In pyramid (a), there are five lines and the stars printed on each line are equal to the line number, i.e. there is 1 star on first line, two stars on second line and so on. To print this pyramid we will take two nested for loops, the outer for loop will be for number of lines and the inner for loop will be for number of stars in each line.

```

for(i=1; i<=5; i++)
{
    for(j=1; j<=i; j++)
        printf("* ");
    printf("\n"); /*for next line of pyramid*/
}

```

The number of lines is 5, so we will make the outer loop iterate 5 times. The inner loop will execute once for first line($i=1$), twice for second line($i=2$), thrice for third line($i=3$) and so on. So the inner loop will print 1 star for first line, 2 stars for second line and so on. After full execution of the inner for loop we need to write `printf("\n")`, so that the stars for next line are printed on a new line.

The following program prints pyramid (a) and it asks for the number of lines instead of taking fixed number of lines.

```

int main(void)
{
    int i,j,n;
    printf("Enter number of lines : ");
    scanf("%d",&n);
    for(i=1; i<=n; i++) /*Loop for number of lines*/
    {
        for(j=1; j<=i; j++) /*Loop for number of stars on a line*/
            printf("* ");
        printf("\n"); /*for next line of pyramid*/
    }
    return 0;
}

```

In the above program if we print the value of i , then we get the pyramid (b), and on printing the value of j we get the pyramid(c). For pyramid (d) we have to print the value of $i+j$, for pyramid (e) we have to print 1 if $(i+j)$ is even and print 0 if $(i+j)$ is odd.

For pyramid (f) we'll take a variable $p=1$, and write the `printf` statement as-
`printf("%3d", p++);`

For pyramid (g) we have to print $(n+1-j)$ and for pyramid (h) we have to print $(n+1-i)$.

```

*****  11111  12345  55555  54321
****    2222  1234   4444   5432
***     333   123    333    543
**      22    12     22     54
*       1     1      1      5
(i)      (j)      (k)      (l)      (m)

```

The outer loop should iterate n times for n number of lines, now let us see how many times the inner loop should iterate. There are 5 stars in line 1, 4 stars in line 2, 3 stars in line 3 and so on. If we talk in terms of n where n is the number of lines, then there are n stars in line 1, $n-1$ stars in line 2, $n-2$ stars in line 3, $n-3$ stars in line 4, $n-4$ stars in line 5. In general we can say that there are $n+1-i$ stars in line i . So the inner loop should iterate $n+1-i$ times. The code for pyramid (i) is-

```

for(i=1; i<=n; i++) /*loop for number of lines*/
{
    for(j=1; j<=n+1-i; j++) /*loop for number of stars on a line*/
        printf("*");
    printf("\n"); /*for next line of pyramid*/
}

```

For pyramid (j), we have to print the value of i , for pyramid (k) we have to print the value of j , for pyramid (l) we have to print the value of $n+1-i$, for pyramid (m) we have to print the value of $n+1-j$.

```

*       1       *       1
**      12      **      12
***     123     ***     123
****    1234    ****    1234
*****  12345   *****  12345
(n)      (o)      (p)      (q)

```

The code for pyramid (n) is similar to that of pyramid (a), but here we have to print spaces before printing stars. The code for it is-

```

for(i=1; i<=n; i++) /*loop for number of lines*/
{
    for(j=1; j<=n-i; j++) /*loop for printing spaces*/
        printf(" ");
    for(j=1; j<=i; j++) /*loop for printing stars*/
        printf("** ");
    printf("\n"); /*for next line of pyramid*/
}

```

In this code if we print the value of j , we will get the pyramid (o). The code for pyramids (p) and (q) is similar to that of (n) and (o), except that the space after star in (n) and space after number in (o) should not be there.

```

*       1       1       *
***     222     123     ***
*****  33333   12345   *****
*****  4444444 1234567 *****
*****  55555555 123456789 *****
(r)      (s)      (t)      (u)

```

In pyramid (r) there are $2*i-1$ stars on line number i . So the code for pyramid (r) is-

```

for(i=1; i<=n; i++) /*loop for number of lines*/
{
    for(j=1; j<=n-i; j++) /*loop for printing spaces*/
        printf(" ");
    for(j=1; j<=2*i-1; j++) /*loop for printing stars*/
        printf("*");
    printf("\n"); /*for next line of pyramid*/
}

```

In this code if we print the value of i we will get pyramid (s) and if we print the value of j we will get pyramid (t). The code for pyramid (u) is-

```

for(i=1; i<=n; i++)
{
    for(j=1; j<=2*(n-i); j++)
        printf(" ");
    for(j=1; j<=2*i-1; j++)
        printf("*");
    printf("\n");
}

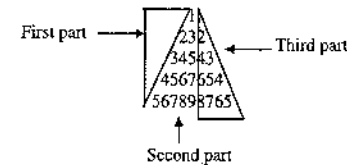
```

```

1       5
232     545
34543   54345
4567654 5432345
567898765 543212345
(v)      (w)

```

In pyramids (v) and (w) also we have $2*i-1$ numbers on line i . In first part spaces are printed, in second part i numbers are printed on line i and in third part $i-1$ numbers are printed on line i .



We will take a variable p and print its value.

```

for(i=1; i<=n; i++) /*loop for number of lines in pyramid*/
{
    for(j=1; j<=n-i; j++) /*loop for spaces (first part)*/
        printf(" ");
    p=i;
    for(j=1; j<=i; j++) /*loop for second part*/
        printf("%d", p++);
    p=p-2;
    for(j=1; j<=i-1; j++) /*loop for third part*/
        printf("%d", p--);
    printf("\n"); /*for next line of pyramid*/
}

```

For pyramid (w) we'll initialize the value of p with n before second inner for loop, and then print the value of $p--$ in second loop and $p++$ in third loop. The value of p has to be increased by 2 before third for loop. The code for pyramid (w) is-

```

for(i=1; i<=n; i++) /*loop for number of lines in pyramid*/
{
    for(j=1; j<=n-i; j++) /*loop for spaces (first part)*/
        printf(" ");
    p=n;
    for(j=1; j<=i; j++) /*loop for second part*/
        printf("%d", p--);
    p=p+2;
    for(j=1; j<=i-1; j++) /*loop for third part*/
        printf("%d", p++);
    printf("\n"); /*for next line of pyramid*/
}

```

```

*****      55555555      123456789      567898765      543212345
*****      4444444      1234567      4567654      5432345
****       33333      12345      34543      54345
***        222      123      232      545
*          1      1      1      5
(x)         (y)         (z)         (z1)         (z2)

```

In pyramid (x), there are 9 stars in first line, 7 in second, 5 in 3rd and so on. In line i there are $2(n-i)+1$ stars. Before printing stars, i spaces are printed.

```

for(i=1; i<=n; i++)
{
    for(j=1; j<=i; j++)
        printf(" ");
    for(j=1; j<=2*(n-i)+1; j++)
        printf("*");
    printf("\n");
}

```

For pyramid (y) print the value of $n+1-i$, and for pyramid (z) print the value of j .

For pyramids (z1) and z2 we have three parts. First part prints i spaces, second part prints $n-i+1$ numbers and third part prints $n-i$ numbers. The code for pyramid (z1) is-

```

for(i=1; i<=n; i++)
{
    for(j=1; j<=i; j++)
        printf(" ");
    p=n+1-i;
    for(j=1; j<=n-i+1; j++)
        printf("%d", p--);
    p=p-2;
    for(j=1; j<=n-i; j++)
        printf("%d", p--);
    printf("\n");
}

```

The code for pyramid (z2) is-

```

for(i=1; i<=n; i++)
{
    for(j=1; j<=i; j++)
        printf(" ");
    p=n;
    for(j=1; j<=n-i+1; j++)
        printf("%d", p--);
    p=p+2;
    for(j=1; j<=n-i; j++)
        printf("%d", p--);
    printf("\n");
}

```

```

*          1          1          1          5
***        222        123        232        545
*****     33333     12345     34543     54345
*****     4444444   1234567   4567654   5432345
*****     55555555  123456789  567898765  543212345
*****     4444444   1234567   4567654   5432345
*****     33333     12345     34543     54345
***        222      123      232      545
*          1        1        1        5
(z3)        (z4)        (z5)        (z6)        (z7)

```

The diamond (z3) can be obtained by joining pyramids (r) and (x). Note that if $n = 5$ then the upper pyramid has 5 lines while inverted pyramid has only 4 lines in this diamond. The code for pyramid (z3) is-

```

int main(void)
{
    int i, j, n;
    printf("Enter n : ");
    scanf("%d", &n);

    for(i=1; i<=n; i++)
    {
        for(j=1; j<=n-i; j++)
            printf(" ");
        for(j=1; j<=2*i-1; j++)
            printf("*");
        printf("\n");
    }
    n--;
    for(i=1; i<=n; i++)
    {
        for(j=1; j<=i; j++)
            printf(" ");
        for(j=1; j<=2*(n-i)+1; j++)
            printf("*");
        printf("\n");
    }
}

```

Diamond (z4) can be obtained by joining pyramids (s) and (y), diamond (z5) by joining pyramids (t) and (z), diamond (z6) by joining pyramids (v) and (z1), diamond (z7) by joining pyramids (w) and (z2).

Exercise

Assume `stdio.h` is included in all programs.

- ```

int main(void)
{
 int a=9;
 if(a=5)
 printf("a is five\t");
 else
 printf("a is not five\t");
 printf("Value of a is %d\n", a);
 return 0;
}

```
- ```

int main(void)
{
    int a=0;
    if(a=0)
        printf("a is zero\t");
    else
        printf("a is not zero\t");
    printf("Value of a is %d\n", a);
    return 0;
}

```
- ```

int main(void)
{
 int i=10;
 i==50;
 if(i==50)
 printf("i is fifty\n");
 else
 printf("i is not fifty\n");
 return 0;
}

```
- ```

int main(void)
{
    int a=9, b=0, c=0;
    if(!a<10 && !b||c)
        printf("C in depth\n");
    else
        printf("See in depth\n");
    return 0;
}

```
- ```

int main(void)
{
 int i=1, j=9;
 if(i>5 && j<5);
 i=j+2;
 printf("%d\n", i);
 return 0;
}

```

```

7. int main(void)
{
 int a=0,b=0;
 if(!a)
 {
 b=!a;
 if(b)
 a=!b;
 }
 printf("%d, %d\n",a,b);
 return 0;
}

```

9. Write this statement using an if-else.

```
x += a < b ? (-x) : 100;
```

10. Write this piece of code using a single if statement.

```

if(i==3)
 if(j==k)
 if(l<m)
 z=100;

```

11. Write this piece of code in an efficient way.

```

if(x==0)
 y++;
if(x>0)
 y--;
if(x<0)
 y+=2;

```

12. Solve this dangling else problem given in the chapter without using braces. Make the else match with the first if but without inserting any braces.

```

if(grade=='A')
 if(marks > 95)
 printf("Excellent\n");
else
 printf("Work hard for getting A grade\n");

```

13. In the following code, add braces so that the printf statements make sense and there is no dangling else problem, also do proper indentation.

```

if(a==10)
if(b==20)
if(c==30)
printf("a is 10, b is 20, c is 30\n");
else
printf("a is 10, b is not 20\n");
else
printf("a is not 10\n");

```

14. int main(void)

```

{
 int k=10;
 switch(k)
 {
 case '5':
 case '10':
 k++;
 continue;
 case '15':
 case '20':
 k--;
 }
 return 0;
}

```

8. int main(void)

```

{
 int a=2,x=10;
 if(a==2)
 if(x==8)
 printf("a is 2 and x is 8\n");
 else
 printf("a is not 2\n");
 return 0;
}

```

15. int main(void)

```

{
 int var=2,x=1,y=2;
 switch(var)
 {
 case x:
 x++;
 break;
 case y:
 y++;
 break;
 }
 return 0;
}

```

16. int main(void)

```

{
 int i,total=0;
 for(i=1; i<=10; i++)
 {
 switch(i)
 {
 case 1:
 case 4:
 case 5:
 case 7:
 total+=i;
 break;
 default:
 continue;
 }
 printf("%d ",i);
 }
 printf("total=%d\n",total);
 return 0;
}

```

18. Rewrite the following else..if chains using switch.

```

(i) if(x==1)
 y=x+1;
else if(x==2)
{
 y=0;
 x=0;
}
else if(x==3 || x==4 || x==5)
 y++;
else if(x==6)
 y+=4;
else
 y--;

```

17. int main(void)

```

{
 int x=2,y=20;
 switch(x)
 {
 case 1:
 y++;
 break;
 case 2:
 y--;
 break;
 default:
 y=y+2;
 }
 printf("y is %d\n",y);
 return 0;
}

```

```

(ii) if(x==1)
 y=x+1;
else if(x==2 || x==3 || x==4)
 y++;
else if(x==5)
 y--;
else if(x==6)
 y=0;

```

19. Write equivalent if...else statements for the following nested switch statements.

```

int x,y;
x=30, y=5,z;
switch(x)
{
 case 10:
 z++;
 break;
 case 20:
 z--;
 break;
 case 30:
 switch(y)
 {
 case 2:
 z=z+3;
 break;
 case 4:
 case 5:
 z=z+10;
 break;
 default:
 z=z-2;
 }
 break;
 default:
 z=0;
}

```

20. What is the output of the following programs.

```
(i) int main(void)
{
 int a=5;
 begin:
 if(a)
 {
 printf("%d\n",a);
 a--;
 goto begin;
 }
 return 0;
}
```

```
(ii) int main(void)
{
 int a=5;
 begin:
 if(a)
 printf("%d\n",a);
 a--;
 goto begin;
 return 0;
}
```

21. How many times will the following while loops iterate.

a) while(-1) { printf("Test Loop\n"); }  
 b) while(!0) { printf("Test Loop\n"); }  
 c) while(0) { printf("Test Loop\n"); }  
 d) a=0; while(a=0) { printf("Test Loop\n"); }  
 e) a=0; while(a=2) { printf("Test Loop\n"); }  
 f) a=0,b=1; while(a=b) { printf("Test Loop\n"); }  
 g) a=1,b=0; while(a=b) { printf("Test Loop\n"); }

22. How many times will the following for loops iterate and what values will be printed.

(i) for(i=0; i<10; i++) printf("%d ",i);  
 (ii) for(i=1; i<=10; i++) printf("%d ",i);  
 (iii) for(i=0; i<=10; i++) printf("%d ",i);  
 (iv) for(i=1; i<10; i++) printf("%d ",i);  
 (v) for(i=0; i<=10; i--) printf("%d ",i);  
 (vi) for(i=10; i>=1; i--) printf("%d ",i);  
 (vii) for(i=10; i>1; i--) printf("%d ",i);  
 (viii) for(i=10; i>0; i--) printf("%d ",i);  
 (ix) for(i=15; i>=0; i=i-3) printf("%d ",i);  
 (x) for(i=10; i>=0; i--); printf("%d ",i);  
 (xi) for(i=0; i>10; i++) printf("%d ",i);  
 (xii) for(i=0; i<=10; i+=20) printf("%d ",i);  
 (xiii) for(i=1; i!=10; i=i+2) printf("%d ",i);  
 (xiv) for(j=10; i=j; j--=2) printf("%d ",i);  
 (xv) for(i=10; i--; ) printf("%d ",i);

3. Write for loops for printing these sequences.

i) 10 20 30 40 50 60 70  
 ii) -10 -20 -30 -40 -50 -60 -70  
 iii) 1 11 111 1111 11111  
 iv) 1 2 4 7 11 16  
 v) 10 7 4 1 -2 -5  
 vi) 1 2 4 8 16 32

## Control Statements

```
24. int main(void)
{
 int i;
 for(i=1; i<5; i++);
 printf("%d ",i);
 i=10;
 while(i<5);
 printf("%d ",i++);
 i=0;
 while(i<5);
 printf("%d ",i++);
 return 0;
}
```

```
25. int main(void)
{
 int i=1;
 while(i++ < 5)
 printf("%d ",i);
 printf("\n");
 i=1;
 while(++i < 5)
 printf("%d ",i);
 printf("\n");
 i=6;
 while(i--> 5)
 printf("%d ",i);
 return 0;
}
```

```
26. int main(void)
{
 int a=6,b=4;
 while(a+b)
 {
 printf("a=%d, b=%d\n",a,b);
 a/=2;
 b*=3;
 }
 return 0;
}
```

```
27. int main(void)
{
 int i=10;
 do
 {
 printf("i=%d\t",i);
 i=i-3;
 }while(i);
 return 0;
}
```

```
28. int main(void)
{
 int i,j,x=0;
 for(i=0; i<5; i++)
 for(j=i; j>0; j--)
 x=i+j+1;
 printf("x=%d\n",x);
 return 0;
}
```

```
29. int main(void)
{
 int i,sum;
 for(i=0; i<10; i+=3)
 sum+=i*i;
 printf("%d\n",sum);
 return 0;
}
```

```
30. int main(void)
{
 int i=1;
 for(;;)
 if(i++==3)
 break;
 printf("%d ",i);
 i=1;
 while(1)
 if(i++==3)
 break;
 printf("%d ",i);
 return 0;
}
```

```
31. int main(void)
{
 int i,sum1=0,sum2=0;
 for(i=1; i<5; i++)
 sum1+=i;
 i=1;
 while(i<5)
 {
 i++;
 sum2+=i;
 }
 printf("%d %d\n",sum1,sum2);
 return 0;
}
```

```
32. int main(void)
{
 int i,j;
 for(j=i+1,i=1; i<=5; j++,i++)
 printf("%d %d\n",i,j);
 return 0;
}
```

```
33. int main(void)
{
 int i=1;
 do;
 while(++i<10);
 printf("%d\n",i);
 return 0;
}
```

34. What is the output of the following-

```
(i) for(i=1; i<=5; i++)
 printf("++");

(ii) for(j=1; j<=5; j++)
 for(i=1; i<=5; i++)
 printf("++");
 printf("\n");

(iii) for(j=1; j<=5; j++)
{
 for(i=1; i<=5; i++)
 printf("++");
 printf("\n");
}

(iv) for(k=1; k<=3; k++)
{
 for(j=1; j<=5; j++)
 {
 for(i=1; i<=5; i++)
 printf("++");
 printf("\n");
 }
 printf("\n");
}
```

35. Write the following loops without break statement.

```
(i) for(i=1; ; i++)
{
 if(i>5)
 break;
 printf("%d ", i);
}

(ii) i=1, j=3;
while(i<5)
{
 if(j==0)
 break;
 printf("%d ", i++);
 j--;
}

(iii) for(i=2; i<=sqrt(n); i++)
{
 if(n%i == 0)
 break;
}
```

36. Will the two programs given below give same output.

```
(i) int main(void)
{
 int i=1, j=3;
 while(i<5)
 {
 i++;
 if(j==0)
 break;
 j--;
 }
 printf("%d %d\n", i, j);
 return 0;
}

(ii) int main(void)
{
 int i=1, j=3;
 while(i<5 && j!=0)
 {
 i++;
 j--;
 }
 printf("%d %d\n", i, j);
 return 0;
}
```

37. Rewrite these for loops so that the initialization, test and update expressions are inside the parentheses of for statement.

```
(i) i=1;
for(; ;)
{
 if(i==5)
 break;
 printf("%d\n", i);
 i++;
}

(ii) sum=0;
for(; ;)
{
 if(sum>100)
 break;
 scanf("%d", &n);
 sum+=n;
}
```

38. Write this code using a for loop.

```
i=0, j=10, k=6;
while(1)
{
 if(i==j || k<0)
 break;
 printf("%d %d %d\n", i, j, k);
 i++;
 j--;
 k=k-2;
}
```

39. Write these pieces of code using while loops.

```
(i) i=1, z=0;
do
{
 if(i<5)
 printf("%d ", i++);
 z=z+2;
}while(i<5);

(ii) do
{
 printf("Enter employee ID (100-500): ");
 scanf("%d", &emp_id);
}while(emp_id < 100 || emp_id > 500);
```

40. Rewrite this program using a do-while loop.

```
#include<stdio.h>
int main(void)
{
 int l, b, h, sarea, vol;
 printf("Enter length, width, height of a cuboid : ");
 scanf("%d%d%d", &l, &b, &h);
 while(l<=0 || b<=0 || h<=0)
 {
 printf("Enter length, width, height of a cuboid : ");
 scanf("%d%d%d", &l, &b, &h);
 }
 sarea=2*(l*b + b*h + h*l);
 vol=l*b*h;
 printf("Surface Area=%d, Volume=%d\n", sarea, vol);
 return 0;
}
```

41. Will these three loops give the same output.

```
for(i=1; i<=5; i++)
 printf("%d ", i);

i=1;
while(i<=5)
{
 printf("%d ", i);
 i++;
}

i=1;
do
{
 printf("%d ", i);
 i++;
}while(i<=5);
```

42. Will these three loops give the same output.

```
for(i=1; i<=5; i++)
{
 if(i==3)
 continue;
 printf("%d\n", i);
}

i=1;
while(i<=5)
{
 if(i==3)
 continue;
 printf("%d\n", i);
 i++;
}

i=1;
do
{
 if(i==3)
 continue;
 printf("%d\n", i);
 i++;
}while(i<=5);
```

13. Compare the output given by these two loops which were written to display numbers from 1 to 5, but by mistake the braces of the loop body are not included.

```
(i) i=0;
while(i<=5)
 i++;
 printf("%d ", i);
```

```
(ii) i=0;
do
 i++;
 printf("%d ", i);
while(i<=5);
```

4. Write these loops without using continue.

```
(i) for(i=1; i<6; i++)
{
 if(i==3)
 continue;
 printf("%d ", i);
}
```

```
(ii) x=0, sum=0;
while(x<10)
{
 x++;
 if(x%2==0)
 continue;
 sum+=x;
}
```

5. Write the following programs using continue statement.

```
(i) int main(void)
{
 int i, j, k;
 for(i=1; i<8; i++)
 {
 if(i!=3 && i!=5)
 {
 for(j=1; j<=i; j++)
 {
 for(k=j; k>0; k--)
 printf("%d ", k);
 printf("\n");
 }
 printf("\n");
 }
 }
 return 0;
}
```

```
(ii) int main(void)
{
 int i, j, x;
 for(i=1; i<10; i++)
 {
 x=i+5;
 if(i%2==0)
 {
 x+=i;
 for(j=i; j>0; j--)
 {
 if(i==5)
 x++;
 else
 x--;
 printf("%d ", x);
 }
 printf("\n");
 }
 else
 printf("%d\n", x);
 }
 return 0;
}
```

1. Write a program to print numbers from 1 to 80 separated by tab, 8 numbers per line.
2. Write a program to input 10 numbers and find the largest number.
3. Write a program to enter 10 numbers ranging from 1 to 50, and draw a histogram by displaying adjacent '=' signs for each number entered. For example if the number entered is 12 then a line of 12 equal to signs should be displayed.
4. Write a program to print all prime numbers from 1 to 99.
5. Write a program to enter a number and find the reverse of that number. Also display the double of the reverse number.
6. Write a program to find whether a number is palindrome or not.
7. Write a program to display all four digit palindromes.
8. Write a program to input a number and count the digits in it. Use while loop and the program should work correctly for also.
9. Input a number and a digit, and count the number of times the digit occurs in the number.
10. Write a program to accept any number n and print the sum of square of all numbers from 1 to n.
11. Write a program to accept any number n and print the cube of all numbers from 1 to n which are divisible by 3. Rewrite the program using a continue statement.
12. Write a program to find out the value of x raised to the power y, where x and y are positive integers.
13. Write a program to enter a number and test whether it is a fibonacci number or not.

59. Find the sum of these series up to n terms where x is an integer entered by the user.

(i)  $1 + 11 + 111 + 1111 + \dots$

(ii)  $x + x^2 + x^3 + x^4 + \dots$

(iii)  $x - x^2 + x^3 - x^4 + \dots$

60. Write a program to read roll number and marks of 10 students in 3 subjects. The valid range for roll numbers is 1000-9999, if the roll number entered is not in this range, the user should be asked to enter again. Calculate total marks of only those students who get more than or equal to 40 marks in each subject. Count the number of students whose total is more than 200. Print the roll number of the student who gets the highest total.

61. Write a program that inputs an integer n and displays an n-by-n checkerboard pattern. If the value of n is 5, the pattern would be-

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

62. Write a program to print the following-

|       |       |         |
|-------|-------|---------|
| ***** | ***** | /*****/ |
| ***** | * *   | /* */   |
| ***** | * *   | /* */   |
| ***** | * *   | /* */   |
| ***** | * *   | /* */   |
| ***** | * *   | /* */   |
| ***** | * *   | /* */   |
| ***** | ***** | /*****/ |



# Functions

## Functions

A function is a self-contained subprogram that performs some specific, well-defined task. Any C program consists of one or more functions. If a program has only one function then it must be the `main()` function. It is the first function to be executed when the program starts, all other functions are called directly or indirectly from `main()`.

### 6.1 Advantages of using Functions

1. Generally a difficult problem is divided into subproblems and then solved. This divide and conquer technique is implemented in C through functions. A program can be divided into functions, each of which performs some specific task. Thus the use of functions modularizes and divides the work of a program.
2. When some specific code is to be used more than once and at different places in the program, the use of functions avoids repetition of that code. The function can be written once and called at different places in the program.
3. The program becomes easily understandable, modifiable and easy to debug and test. It becomes simple to write the program and understand *what* work is done by each part of the program. The details of *how* the work is done are hidden inside appropriate functions. This is called abstraction.
4. Functions can be stored in a library and can be reused.

C programs have two types of functions-

1. Library functions
2. User-defined functions

### 6.2 Library Functions

C has the facility to provide library functions for performing some operations. These functions are present in the C library and they are predefined. For example `sqrt()` is a mathematical library function which is used for finding out the square root of any number. The functions `scanf()` and `printf()` are input output library functions. Similarly we have functions like `strlen()`, `strcmp()` for string manipulations.

To use a library function we have to include corresponding header file using the preprocessor directive `#include`. For example to use input output functions like `printf()`, `scanf()` we have to include `stdio.h`, for mathematical library functions we have to include `math.h`, for string library functions `string.h` should be included. The following program illustrates the use of library function `sqrt()`.

```
/*P6.1 Program to find the square root of a number*/
#include<stdio.h>
#include<math.h>
int main(void)
{
 double n,s;
 printf("Enter a number : ");
 scanf("%lf",&n);
 s=sqrt(n);
```

```
 printf("Square root of %.2lf is %.2lf\n",n,s);
 return 0;
```

Output :

```
Enter a number : 16
Square root of 16.00 is 4.00
```

In this program we have used three library functions – `printf()`, `scanf()` and `sqrt()`. We will learn more about library functions later in this chapter.

### 6.3 User-Defined Functions

Programmers can create their own functions for performing any specific task of the program. These types of functions are called user-defined functions. To create and use these functions, we should know about these three things-

1. Function definition
2. Function declaration
3. Function call

Before discussing these three points let us write two simple programs that will be used for reference. The first program draws a line and the second one adds two numbers.

```
/*P6.2 Program to draw a line*/
#include<stdio.h>
void drawline(void); /*Function Declaration*/
int main(void)
{
 drawline(); /*Function Call*/
 return 0;

 void drawline(void) /*Function Definition*/
 {
 int i;
 for(i=1; i<=80; i++)
 printf("-");
 }
}
```

```
/*P6.3 Program to find the sum of two numbers*/
#include<stdio.h>
int sum(int x,int y); /*Function declaration*/
int main(void)
{
 int a,b,s;
 printf("Enter values for a and b : ");
 scanf("%d%d",&a,&b);
 s=sum(a,b); /*Function call*/
 printf("Sum of %d and %d is %d\n",a,b,s);
 return 0;

 int sum(int x,int y) /*Function definition*/
 {
 int s;
 s = x+y;
 return s;
 }
}
```

### 6.4 Function Definition

The function definition consists of the whole description and code of a function. It tells what the function is doing and what is its input and output. A function definition consists of two parts - a function header and a function body. The general syntax of a function definition is-

```
return_type func_name(parameter declarations)
```

```

local variables declarations;
statement;
.....
return (expression);
}

```

The first line in the function definition is known as the function header. After function header, body of the function is written enclosed in curly braces.

The `return_type` denotes the type of the value that will be returned by the function. A function can return either one value or no value. If a function does not return any value then `void` should be written in place of `return_type`. If no return type is specified in the definition then `int` return type is assumed, however it is not a good practice to omit the return type.

`func_name` specifies the name of the function and it can be any valid C identifier. After function name, a comma separated list of parameter declarations is given inside parentheses, which mentions the types and names of the parameters. These parameters are also known as formal parameters and are used to accept values which are provided to the function in the function call. A function can have any number of parameters or even no parameter at all. If the parameter list is empty, then `void` is written inside the parentheses (parameter list may be left empty in this case but it is a good practice to write `void`).

The body of function is a block which consists of declarations of variables, and C statements followed by optional return statement. The variables declared inside the function are known as local variables, since they are local to that function only. i.e. they have existence only in the function in which they are declared, they can not be used anywhere else in the program. They are created when the function body is entered, exist while the function is executing and are destroyed when the function is exited. The parameters of a function are also local variables of the function. There can be any number of valid C statements inside a function body. The return statement is optional, it may be absent if the function does not return any value.

The function definition can be placed anywhere in the program. But generally all definitions are placed after the `main()` function. Note that a function definition cannot be placed inside another function definition. Function definitions can also be placed in different source files.

In P6.2 the function definition is-

```

void drawline(void)
{
 int i;
 for(i=1; i<=80; i++)
 printf("-");
}

```

Here the function is not returning any value so `void` is written at the place of `return_type`, and since it does not accept any arguments, `void` is written inside parentheses. The `int` variable `i` is declared inside the function body so it is a local variable and can be used inside this function only.

In P6.3 the function definition is-

```

int sum(int x, int y)
{
 int s;
 s=x+y;
 return s;
}

```

This function returns a value of type `int` because `int` is written at the place of `return_type`. There are two formal parameters `x` and `y`, both of type `int`. The variable `s` is declared inside the function so it is a local variable. The formal parameters `x` and `y` are also used as local variables inside this function.

## 6.5 Function Call or Function Invocation

The function definition describes what a function can do, but to actually use it in the program the function should be called somewhere. A function is called by simply writing its name followed by the argument list inside the parentheses.

```
func_name(arg1, arg2, arg3,)
```

The arguments `arg1, arg2, .....`  are called actual arguments as these are the values which are *actually* sent to the called function. Actual arguments can be written in the form of variables, constants, expressions or any function call that returns a value. For example-

```

fun(x)
func(a*b, c*d+k)
func(22, 43)
func1(1, 2, sum(a, b))

```

`func_name` is the called function while the function in which this function call is placed is known as the calling function. For example in the program P6.3, `main()` is the calling function, `sum()` is the called function and `a, b` are actual arguments. In this program, the function call is written on the right hand side of the assignment operator as-

```
s=sum(a, b);
```

Even if there are no actual arguments, the function call should have the empty parentheses. For example in program P6.2, the function call is written as-

```
drawline();
```

When a function is called, the control passes to the called function which is executed, and after that the control is transferred to the calling function. Now let us see what exactly happens when a function is called. The compiler allocates storage space for the parameters specified in the function definition. The argument expressions are evaluated and the values of these expressions are assigned to the corresponding formal parameters. If the type of any argument and its corresponding formal parameter does not match, then default conversions take place before assigning the value to that parameter. After this the code in the function body is executed. The function terminates when a return statement is encountered, and if there is no return statement then the code till the closing brace is executed. After this control jumps back to the calling function. Let us take an example program to understand this whole procedure.

*/\*P6.4 Program to find the sum of two numbers\*/*

```

#include<stdio.h>
int sum(int x, int y); /*Function declaration*/
int main(void)

```

```

{
 int a=10, b=20, k;
 k=sum(a, b); /*Function call*/
 printf("%d\n", k);
 k=sum(4, 5); /*Function call*/
 printf("%d\n", k);
 k=sum(a+b, b*2); /*Function call*/
 printf("%d\n", k);
 return 0;
}

```

```

int sum(int x, int y) /*Function definition*/
{

```

```

 int s;
 s=x+y;
 return s;
}

```

In `main()` when the first function call is encountered, the control is passed to the function `sum()`. Space is allocated for parameters `x` and `y` and they are initialized with the values of variables `a` and `b` of `main()`. So `x` gets the value 10 and `y` gets the value 20. Now space is allocated for the local variable `s` and then the statement `s=x+y` is executed. The next statement is a return statement which will return the value of `s` to the calling function and stop the execution of the function. Now since the function is exited the variables `x, y` and `s` are

destroyed. Control passes to `main()` and the return value becomes the value of the expression `sum(a,b)` which is assigned to the variable `k`. The next statement of `main()` is executed which prints the value of `k`.

After this there is a call to the function `sum()` again, and this time the actual arguments are 4 and 5. So once more the control is transferred to the function `sum()`. The parameters `x` and `y` are created and this time initialized with values 4 and 5. The variable `s` is created and the statement after it is executed, then the value of `s` is returned. The variables `x`, `y` and `s` are destroyed and control returns to `main()`. The variable `k` of `main()` is assigned the return value of the function call which is 9. The next statement of `main()` is executed and after this again there is a function call. The whole procedure will be repeated again and this time parameters will be initialized with values of expressions `a+b` and `b*2`.

If a function returns a value, any call to this function call can be used like an operand in an expression anywhere in the program. The type and value of this operand will be the type and value of the return value of the function. For example-

```
s=sum(a,b); /*Assigning the return value of sum() to variable s */
b=max(x,y)*10; /*Return value of max() is multiplied by 10 and assigned to variable b*/
if(isprime(x)=1) /*Return value of isprime() is used in if condition*/
 printf("Number is prime");
printf("%d\n",sum(a,b)); /*Return value of sum() is printed*/
```

If the return type of function is `void` then it cannot be used in this way in any expression. For example it would be meaningless and invalid to write an expression like this-

```
s=drawline();
```

A function call can be used as a statement, if a semicolon is placed after it. In this case if the function returns any value, it is just discarded. For example-

```
draw(x,y);
display(a,b,c);
printf("prime");
```

A function call cannot occur on the left hand of an assignment statement.

```
func(x,y)=a; /*Invalid*/
```

The code of a function is executed only when it is called by some other function. If the function is defined and not called even once then its code will never be executed. A function can be called more than once, so the code is executed each time it is called. The execution of a function finishes either when the closing braces of the function body are reached or if return statement is encountered. Now let us learn in detail about the return statement.

## 6.6 return statement

The return statement is used for immediate exit from the called function to the calling function and to return a value to the calling function. This statement can appear anywhere inside the body of the function. There are two ways in which it can be used-

```
return;
return (expression);
```

Here `return` is a keyword. The first form of return statement is used to terminate the function without returning any value. In this case only `return` keyword is written and this form should be used only in functions that have `void` as return type. The following program uses this form of return statement.

```
/*P6.5 Program to understand the use of return statement*/
```

```
#include<stdio.h>
void selection(int age,float ht);
int main(void)
{
 int age;
 float ht;
 printf("Enter age and height: ");
 scanf("%d%f",&age,&ht);
```

```
 selection(age,ht);
 return 0;
```

```
void selection(int age,float ht)
```

```
{
 if(age>25)
 {
 printf("Age should be less than 25\n");
 return;
 }
 if(ht<5)
 {
 printf("Height should be more than 5\n");
 return;
 }
 printf("Selected\n");
}
```

If a function has no return statement, the code of that function is executed till the closing brace encountered. This is called falling off the end. In the function `selection()` we can exit the function even before the closing braces are encountered, with the help of the `return` statement.

The second form of return statement is used to terminate a function and return a value to the calling function. The program P6.3 uses this second form of return statement. The value returned by the `return` statement may be any constant, variable, any other function call or any valid C expression. For example in program P6.3 we can directly write `return (x+y)` instead of taking a variable and returning it.

Some other examples of valid return statements are-

```
return 1;
return ++x;
return (x+y*z);
return (3*sum(a,b));
```

It is optional to enclose the returning value in parentheses. The following program shows another example of a function that returns a value.

```
/*P6.6 Program to find the larger number*/
```

```
#include<stdio.h>
int larger(int x,int y);
int main(void)
{
 int x,y;
 printf("Enter two numbers : ");
 scanf("%d%d",&x,&y);
 printf("Larger number=%d\n",larger(x,y));
 return 0;
}
int larger(int x,int y)
{
 return x>y?x:y;
}
```

The expression returned is generally of the same type as specified in the function definition. If it is of some other type the compiler will try to convert it to the appropriate type, if the conversion is possible. For example a function returns `int` value and return type is specified as `double` then the return value will be converted to `double`.

```
double func(void)
```

```
{
 int x;

 return x;
}
```

Here the value returned is of int type, so it will be converted to double and then returned.

If the return type of a function is not void and no value is returned using return statement, the value returned to the caller is undefined, i.e. a garbage value is returned to the caller. It is also possible that a function returns value from only some places, in this case also the result is undefined.

We can't return value from a function that has void as the return type i.e. we can't use second form of return statement in void functions. For example it would be invalid to write a function like this-

```
void add(int a, int b)
{
 printf("%d ", a+b);
 return a+b; /*Error: returning value from void function*/
}
```

A void function should not be used as operand in any expression.

We can use multiple return statements in a function but as soon as first return statement is encountered the function terminates and all the statements following it are not executed.

The next function compares two dates (d1/m1/y1 and d2/m2/y2) and returns 1 if first date is smaller, returns -1 if second date is smaller, returns 0 if both dates are same. In this function we have used 7 return statements.

```
int cmpdate(int d1, int m1, int y1, int d2, int m2, int y2)
```

```
{
 if (y1 < y2)
 return 1;
 if (y1 > y2)
 return -1;
 if (m1 < m2)
 return 1;
 if (m1 > m2)
 return -1;
 if (d1 < d2)
 return 1;
 if (d1 > d2)
 return -1;
 return 0;
}
```

Note that there is no need to write else...if because return immediately takes control out of the function.

A function can return only one value. If we want to return more than one value then we have to use another technique discussed in further chapters. It is incorrect to try to return more than one value using comma.

```
return 1, 2, 3;
```

Here this expression is using comma operator and hence only the rightmost value will be returned.

## 6.7 Function Parameters and Arguments

We have seen before that parameters are mentioned in the function definition and they are used to hold the values that are sent by the calling function. These parameters are like other local variables of the function which are created when the function call starts and are destroyed when the function ends. However there are two differences. First is that parameters are declared inside parentheses while other local variables are declared at the beginning of the function block. The second difference is that parameters are automatically initialized with the values of the corresponding arguments provided in the function call while other local variables are assigned values through the statements written inside the function body.

Any changes made to the parameters inside the function do not affect the actual arguments. This mechanism of passing arguments is called call-by-value. The other mechanism where arguments passed can be changed by changing parameters is called call-by-reference. In C language, all arguments are passed using call-by-value only. We can simulate call-by-reference using pointers, which will be discussed in chapter 9.

In the following program x and y are the actual arguments sent to function func() and a, b are formal parameters of func(). Inside func(), values of a and b are changed but this does not affect the values of x and y in main().

```
/*P6.7*/
#include <stdio.h>
void func(int a, int b)
{
 int main(void)

 int x=10, y=20;
 func(x, y);
 printf("x=%d, y=%d\n", x, y);
 return 0;

void func(int a, int b)

 a++;
 b++;
 printf("a=%d, b=%d\n", a, b);
}
```

Output:

a=11, b=19

x=10, y=20

If the number of arguments in the call is not equal to the number of parameters in the definition, the behaviour is undefined.

If there is a type mismatch between a corresponding argument and parameter, then compiler tries to convert the argument to the type of the parameter if the conversion is legal, otherwise a garbage value is passed to the parameter.

## 6.8 Order of Evaluation of Function Arguments

The order of evaluation of arguments and subexpressions within the arguments is unspecified. This order of evaluation is not important in function calls like multiply(m, n) or multiply(m+n, m-n). But suppose we have a function call like this-

```
int m=3, k;
k=multiply(m, m++);
```

Now here if the first argument is evaluated first then value of k will be 9, and if the second argument is evaluated first, the value of k will be 12. Since the order of evaluation of arguments is unspecified in C and depends on compiler, the result may be different on different compilers. Similarly if we write-

```
int i=10;
printf("%d %d %d", --i, i++, i);
```

Here also the result is unpredictable. So it is better to avoid these types of argument expressions that can produce different results on different compilers.

## 6.9 Function Declaration

A function declaration is used to give specific information to the compiler about the function so that it can check the function calls.

The calling function needs information about the called function. If definition of the called function is placed before the calling function, then declaration is not needed. For example in program P6.3, if we write the definition of sum() before main(), then declaration is not required.

```
/*P6.8 Program to find the sum of two numbers*/
#include <stdio.h>
int sum(int x, int y) /*Function definition*/

 int s;
```

```

s=x+y;
return s;
}

int main(void)
{
 int a,b,s;
 printf("Enter values for a and b : ");
 scanf("%d %d", &a, &b);
 s=sum(a,b); /*Function call*/
 printf("Sum of %d and %d is %d\n", a,b,s);
 return 0;
}

```

Here the definition of sum() is written before main(), so main() knows everything about the function sum(). But generally the function main() is placed at the top and all other functions are placed after it. In this case, function declaration is required. The function declaration informs the compiler about the following three things-

1. Name of the function.
2. Number and types of arguments received by the function and their order.
3. Type of value returned by the function.

Function declaration tells the compiler that a function with these features will be defined and used later in the program. The general syntax of a function declaration is-

```
return_type func_name(type1 par1, type2 par2,);
```

This looks just like the header of function definition, except that there is a semicolon at the end. The names of the parameters while declaring a function are optional. These are only used for descriptive reasons, i.e. they inform the programmer about their purpose. The compiler ignores these names. So we can write the declaration in this way also -

```
return_type func_name(type1, type2,);
```

In program P6.2 the declaration is written as-

In program P6.3 the declaration is written as-

The main thing about the placing of function declaration is that it should be before the function call. Generally all the function declarations are written before the main() function. This way the function declaration is accessible to all functions. It is also possible that we declare the function only inside those functions that call it.

Now we'll write two more example programs that use functions.

*/\*P6.9 Program that finds whether a number is even or odd\*/*

```

#include<stdio.h>
void find(int n);
int main(void)
{
 int num;
 printf("Enter a number : ");
 scanf("%d", &num);
 find(num);
 return 0;
}

void find(int n)
{
 if(n%2==0)
 printf("%d is even\n", n);
 else
 printf("%d is odd\n", n);
}

```

*/\*P6.10 Program to find out the factorial of a number\*/*

```

#include<stdio.h>
long int factorial(int n);
int main(void)
{
 int num;
 printf("Enter a number : ");
 scanf("%d", &num);
 if(num<0)
 printf("No factorial of negative number\n");
 else
 printf("Factorial of %d is %d\n", num, factorial(num));
 return 0;
}

long int factorial(int n)
{
 int i;
 long int fact=1;
 if(n==0)
 return 1;
 for(i=n; i>1; i--)
 fact*=i;
 return fact;
}

```

We can classify functions into four types depending on the return value and arguments.

1. Functions with no arguments and no return value
2. Functions with no arguments but a return value
3. Functions with arguments but no return value
4. Functions with arguments and return value

### Functions with no arguments and no return value

Functions that have no arguments and no return value are written as-

```

void func(void);
int main(void)
{

 func();

}

void func(void)
{

}

```

In the above example, the function func() is called by main() and the function definition is written after the main() function. As the function func() has no arguments, main() can not send any data to func() and since it has no return statement, function can not return any value to main(). There is no communication between the calling and the called function. Since there is no return value, these types of functions cannot be used as operands in expressions. The function drawline() in P6.2 is an example of these types of functions. Let us see one more example program-

*/\*P6.11 Program that uses a function with no arguments and no return values\*/*

```

#include<stdio.h>
void displaymenu(void);
int main(void)
{
 int choice;
 displaymenu();
 printf("Enter your choice : ");
 scanf("%d", &choice);
 return 0;
}

```

```
void displaymenu(void) {
```

```
 printf("1.Create database\n");
 printf("2.Insert new record\n");
 printf("3.Modify a record\n");
 printf("4.Delete a record\n");
 printf("5.Display all records\n");
 printf("6.Exit\n");
}
```

### Functions with no arguments but a return value

These types of functions do not receive any arguments but they can return a value.

```
int func(void);
int main(void)
{
 int r;

 r=func();

 return 0;
}

int func(void)
{

 return expression;
}
```

The next program uses a function of this type.

```
/*P6.12*/
#include<stdio.h>
int func(void);
int main(void)
{
 printf("%d\n",func());
 return 0;
}

/*Returns the sum of squares of all odd numbers from 1 to 25*/
int func(void)
{
 int num,s=0;
 for(num=1; num<=25; num++)
 {
 if(num%2!=0)
 s+=num*num;
 }
 return s;
}
```

Output:  
925

### Functions with arguments but no return value

These types of functions have arguments, so the calling function can send data to the called function but the called function does not return any value.

```
void func(int,int);
int main(void)
```

```
.....
func(a,b);
.....
```

```
return 0;
```

```
void func(int c,int d)
```

Here a and b are actual arguments which are used for sending the value, c and d are the formal parameters, which accept values from the actual arguments.

```
/*P6.13 Program to find the type and area of a triangle*/
#include<stdio.h>
#include<math.h>
void type(float a,float b,float c);
void area(float a,float b,float c);
int main(void)
{
 float a,b,c;
 printf("Enter the sides of triangle : ");
 scanf("%f%f%f",&a,&b,&c);
 if(a<b+c && b<c+a && c<a+b)
 {
 type(a,b,c);
 area(a,b,c);
 }
 else
 printf("No triangle possible with these sides\n");
 return 0;
}

void type(float a, float b, float c)
{
 if((a*a)+(b*b)==(c*c) || (b*b)+(c*c)==(a*a) || (c*c)+(a*a)==(b*b))
 printf("Right angled triangle\n");
 if(a==b && b==c)
 printf("Equilateral triangle\n");
 else if(a==b || b==c || c==a)
 printf("Isosceles triangle\n");
 else
 printf("Scalene triangle\n");
}

void area(float a,float b,float c)
{
 float s,area;
 s=(a+b+c)/2;
 area=sqrt(s*(s-a)*(s-b)*(s-c));
 printf("Area of triangle=%f\n",area);
}
```

### Functions with arguments and return value

These types of functions have arguments, so the calling function can send data to the called function. The called function can return any value to the calling function using return statement.

```
int func(int,int);
int main(void)
{
 int r;

 r=func(a,b);

 func(c,d);
 return 0;
}
```

The definition, declaration and call of library functions-

Function Definition - Predefined, precompiled and present in the library.

Function Declaration - In header files (files with a .h extension)

Function Call - By the programmer

To use a library function in our program we should know-

- (i) Name of the function and its purpose
- (ii) Type and number of arguments it accepts
- (iii) Type of the value it returns
- (iv) Name of the header file to be included.

We can define any function of our own with the same name as that of any function in the C library. If we do so then the function that we have defined will take precedence over the library function of the same name. The next few sections discuss the old style of definition and declaration. We should write all our functions with proper definition, declaration as discussed in previous sections. The old style is still supported because of backward compatibility and you might come across code written in this style.

## 6.12 Old Style of Function Definition

The old style or K&R-style of writing function definition is slightly different from the one we have studied. Although this style is no longer used, the compilers still support it. Here the function header contains only the names of the parameters, and they are declared between the function header and the function body.

```
return_type func_name(parameter_list)
parameter declarations
```

```
{
 local variables declarations;
 statements;
 return(expression);
}
```

Let us see an example of function definition written in old style-

```
float func1(a,b,z)
int a,b;
float z;
{
 float s;
 s=(a+b)/z;
 return z;
}
```

## 6.13 Old Style of Function Declaration

The old style declaration of functions does not inform the compiler about the type and number of arguments. It only specifies the return type of the function. The syntax of old style declaration is-

```
return_type func_name();
```

The parentheses are always empty even if the function receives arguments. Since there is no information about the parameters, the compiler is not able to check calls to this function. It is possible to call the function with number and types of arguments which are different from the number and types given in function definition. We don't get any error while compiling but it can lead to run-time errors.

With function prototype declaration (section 6.9) it will not be possible to call the function incorrectly and the compiler will be able to detect the error in function call.

This style is supported in new compilers also so that the old programs can be compiled using the new compilers. However this style is not used now since the new style provides more information to the compiler. The new style is also known as function prototype.

```
int func(int a, int b)
{
 // ...
 return expression;
}
```

Here return statement returns the value of expression to the calling function. The functions sum(), larger(), factorial() that we had written earlier are examples of these types of functions. Let us take one more example program-

```
/*6.14 Program to find the sum of digits of any number*/
#include <stdio.h>
int sum(int n);
int main(void)
{
 int num;
 printf("Enter a number : ");
 scanf("%d", &num);
 printf("Sum of digits of %d is %d\n", num, sum(num));
 return 0;
}

int sum(int n)
{
 int sum=0;
 while(n>0)
 {
 sum+=n%10;
 n/=10; /*Skip the last digit of number*/
 }
 return sum;
}
```

While describing the different type of functions, we have assumed arguments and return values of type int for simplicity. These values can be of any data type.

## 6.10 main() Function

Execution of every C program always begins with the function main(). Each function is called directly or indirectly in main() and after all functions have done their operations, control returns back to main(). There can be only one main() function in a program.

The main() function is a user defined function but the name, number and type of arguments are predefined in the language. The operating system calls the main function and main() returns a value of integer type to the operating system. If the value returned is zero, it implies that the function has terminated successfully and any nonzero return value indicates an error. If no return value is specified in main() then any garbage value will be returned automatically. Calling the function exit() with an integer value is equivalent to returning that value from main(). The function main() can also take arguments, which will be discussed in further chapters.

The definition, declaration and call of main() function-

Function Declaration - By the C compiler

Function Definition - By the programmer

Function Call - By the operating system

## 6.11 Library Functions

The library functions are formally not a part of the C language, but they are supplied with every C compiler. The source code of the library functions is not given to the user. These functions are precompiled and the user gets only the object code. This object code is linked to the object code of your program by the linker. Different categories of library functions are grouped together in separate library files. When we call a library function in our program, the linker selects the code of that function from the library file and adds it to the program.

## 6.14 More about function declaration

If function declaration is not present and function definition is present after the function call, then the function is implicitly declared when it is first called in the program. Its return type is assumed to be `int` and nothing is assumed about its parameters.

```
s=sum(a,b);
multiply(2,3);
display();
func(1.2, 3.5, 2);
```

All these functions will be implicitly declared as-

```
int sum();
int multiply();
int display();
int func();
```

The empty parentheses mean that there is no information about the parameters. So these declarations will help in checking only the return type of the function. If the return type specified in the function definition is `int` then we will not get any error, otherwise we get an error. Now suppose the definitions of these functions were written in the file as-

```
sum(int a,int b)
{
 return a+b;
}
int multiply(int a,int b,int c)
{
 return a*b*c;
}
void display(int x)
{
 printf("%d\n",x);
}
float func(float x, float y, int z)
{
 return (x+y)/z;
}
```

For the first two functions we will not get any error since their return type is `int`. For the last two functions we will get error since return types specified in the function definition and declaration don't match.

So if we write a function returning `int` and don't provide a declaration for it, we won't get any error. If the function that we are writing returns something other than `int`, then it is compulsory to declare it otherwise we get an error.

It is not good to rely on the declaration given by the compiler as it does not give any information about parameters. For example in the call to function `multiply()`, we have provided only 2 arguments while the definition has 3 parameters. The compiler can't detect this problem because in the declaration of `multiply()` there is no information about parameters. So in this way we get bugs which occur due to wrong number and type of arguments in the function call.

## 6.15 Local, Global and Static variables

### 6.15.1 Local Variables

The variables that are defined within the body of a function or a block are local to that function or block only and are called local variables. These variables are created when function is entered and are destroyed when the function is exited. For example-

```
void func(void)
{
```

### Functions

```
int a,b;
```

Here `a` and `b` are local variables which are defined within the body of the function `func()`. Local variables can be used only in those functions or blocks, in which they are declared. The same variable name may be used in different functions. For example-

```
void func1(void)
{
 int a=2,b=4;
}
void func2(void)
{
 int a=15,b=20;
```

Here values of `a=2, b=4` are local to the function `func1()` and `a=15, b=20` are local to the function `func2()`.

### 6.15.2 Global Variables

The variables that are defined outside any function are called global variables. All functions in the program can access and modify global variables. It is useful to declare a variable global if it is to be used by many functions in the program. Global variables are automatically initialized to 0 at the time of declaration.

*/\*P6.15 Program to understand the use of global variables\*/*

```
#include<stdio.h>
void func1(void);
void func2(void);
int a,b=6;
int main(void)
{
 printf("Inside main() : a=%d,b=%d\n",a,b);
 func1();
 func2();
 return 0;
}
void func1(void)
{
 printf("Inside func1() : a=%d,b=%d\n",a,b);
}
void func2(void)
{
 int a=8;
 printf("Inside func2() : a=%d,b=%d\n",a,b);
}
```

Output:

```
Inside main() : a=0,b=6
Inside func1() : a=0,b=6
Inside func2() : a=8,b=6
```

Here `a` and `b` are declared outside all functions so they are global variables. The variable `a` will be initialized to 0 automatically since it is a global variable. Now we can use these variables in any function. In `func2()`, there is local variable with the same name as global variable. Whenever there is a conflict between a local and global variable, the local variable gets the precedence. So inside `func2()` the value of local variable gets printed.

### 6.15.3 Static Variables

Static variables are declared by writing keyword `static` in front of the declaration. If a static variable is not initialized then it is automatically initialized to 0.



```
static type var_name;
```

A static variable is initialized only once and its value is retained between function calls.

*/\*P6.16 Program to understand the use of static variables\*/*

```
#include<stdio.h>
void func(void);
int main(void)
{
 func();
 func();
 func();
 return 0;
}
void func(void)
{
 int a=10;
 static int b=10;
 printf("a=%d, b=%d\n",a,b);
 a++;
 b++;
}
```

Output:  
a=10, b=10  
a=10, b=11  
a=10, b=12

Here b is a static variable. First time when the function is called b is initialized to 10. Inside the function, value of b becomes 11. This value is retained and when next time the function is called, value of b is 11 and the initialization is neglected. Similarly when third time function is called, value of b is 12. Note that the variable a, which is not static is initialized on each call and its value is not retained.

## Some Additional Problems

### Problem 1

Write a function reverse() that inputs a number and returns the reverse of that number, for example if the input is 1293, function should return 3921.

```
/*P6.17*/
#include<stdio.h>
int reverse(int n);
int main(void)
{
 int num;
 printf("Enter a number : ");
 scanf("%d", &num);
 printf("%d\n", reverse(num));
 return 0;
}
int reverse(int n)
{
 int rev=0;
 do
 {
 rev=rev*10+n%10;
 n/=10;
 }while(n>0);
 return rev;
}
```

### Problem 2

Using the function reverse() in previous program, make a function isPalindrome() that inputs a number and checks whether it is a palindrome number. A number is palindrome number if it remains same after reversing its digits, for example 949, 82328 are palindromes.

```
/*P6.18*/
#include<stdio.h>
long int reverse(long int n);
int isPalindrome(long int num);
int main(void)
{
 long int num;
 printf("Enter a number : ");
 scanf("%ld", &num);
 if(isPalindrome(num))
 printf("Number is a palindrome\n");
 else
 printf("Number is not a palindrome\n");
 return 0;
}
int isPalindrome(long int num)
{
 if(num==reverse(num))
 return 1;
 return 0;
}
long int reverse(long int n)
{
 long int rev=0;
 do
 {
 rev=rev*10+n%10;
 n/=10;
 }while(n>0);
 return rev;
}
```

### Problem 3

Write a function isprime() which takes a number and returns 1 if the number is prime and 0 otherwise.

```
/*P6.19*/
#include<stdio.h>
#include<math.h>
int isprime(int n);
int main(void)
{
 int num;
 printf("Enter a number : ");
 scanf("%d", &num);
 if(isprime(num))
 printf("Number is prime\n");
 else
 printf("Number is not prime\n");
 return 0;
}
int isprime(int n)
{
 int i;
 for(i=2; i<=sqrt(n); i++)
 if(n%i==0)
 return 0;
 return 1;
}
```

### Problem 4

Write a function that inputs two numbers and prints all prime numbers between those numbers.

```
/*P6.20*/
#include<stdio.h>
#include<math.h>
void printPrimes(int num1, int num2);
int isprime(int n);
int main(void)
{
 int num1, num2;
 printf("Enter two numbers : ");
 scanf("%d%d", &num1, &num2);
 printf("Prime number between %d and %d are : ", num1, num2);
 printPrimes(num1, num2);
 return 0;
}

void printPrimes(int num1, int num2)
{
 int i;
 for(i=num1; i<=num2; i++)
 if(isprime(i))
 printf("%d ", i);
}

int isprime(int n)
{
 int i;
 for(i=2; i<=sqrt(n); i++)
 if(n%i==0)
 return 0;
 return 1;
}
```

### Problem 5

Write a function that converts a decimal number to binary number.

```
/*P6.21*/
#include<stdio.h>
long int binary(int num);
int main(void)
{
 int num;
 printf("Enter a number : ");
 scanf("%d", &num);
 printf("Decimal=%d, Binary=%ld\n", num, binary(num));
 return 0;
}

long int binary(int num)
{
 long a=1, bin=0, rem;
 while(num>0)
 {
 rem=num%2;
 bin=bin+rem*a;
 num/=2;
 a*=10;
 }
 return bin;
}
```

The logic used in this program is discussed below-  
Suppose decimal number is 29

|      | Quotient | Remainder |      |     |
|------|----------|-----------|------|-----|
| 29/2 | 14       | 1         | a[0] | MSB |
| 14/2 | 7        | 0         | a[1] |     |
| 7/2  | 3        | 1         | a[2] |     |
| 3/2  | 1        | 1         | a[3] |     |
| 1/2  | 0        | 1         | a[4] | LSB |

$$1*1 + 0*10 + 1*100 + 1*1000 + 1*10000 = 11101$$

By this method we can get binary equivalents of decimal numbers upto 1023 only, since for numbers more than 1023 (Binary - 111111111), the binary equivalents will exceed the range of long int. In the next chapter we'll see a better method to print binary numbers.

### Problem 6

Write a program to raise a floating point number to an integer power (eg.  $a^n$  where  $a$  is floating point number and  $n$  is an integer value).

```
/*P6.22*/
#include<stdio.h>
double power(double a, int n);
#include<math.h>
int main(void)
{
 double a;
 int n;
 printf("Enter base : ");
 scanf("%lf", &a);
 printf("Enter exponent : ");
 scanf("%d", &n);
 printf("%lf raised to power %d is %lf\n", a, n, power(a, n));
 return 0;
}

double power(double a, int n)
{
 int i;
 double p=1;
 if(n==0)
 return 1;
 else
 {
 for(i=1; i<=abs(n); i++)
 p=p*a;
 if(n>0)
 return p;
 else
 return 1/p;
 }
}
```

### Problem 7

Write a function that inputs a binary or octal number and converts to decimal number.

```
/*P6.23 Program to convert a binary or octal number to a decimal number*/
#include<stdio.h>
int func(int n, int base);
```

Handwritten notes on page 115:

$r = 1$   
 $b = 0 + 1 \times 1$   
 $bin = 1$   
 $14 \dots 0$   
 $n = 1 \times 10$   
 $\dots 10$

```

int main(void)
{
 int num, base, result;
 char choice;
 printf("Enter 'b' for binary or 'o' for octal : ");
 scanf("%c", &choice);
 printf("Enter a number : ");
 scanf("%d", &num);
 base = (choice == 'b') ? 2 : 8;
 result = func(num, base);
 printf("Decimal number is %d\n", result);
 return 0;
}

int func(int n, int base)
{
 int rem, d, j=1, dec=0;
 while(n>0)
 {
 rem = n%10; /*extract last digit*/
 d = rem*j;
 dec+=d;
 j*=base;
 n/=10; /*skip last digit*/
 }
 return dec;
}

```

### Problem 8

Write a program to implement these formulae of permutations and combinations.

Number of permutations of  $n$  objects taken  $r$  at a time :  $p(n, r) = n! / (n-r)!$

Number of combinations of  $n$  objects taken  $r$  at a time :  $c(n, r) = n! / (r! * (n-r)!) = p(n, r) / r!$

```

/*P6.24*/
#include<stdio.h>
long factorial(int);
long perm(int, int);
long comb(int, int);
int main(void)
{
 int n, r;
 printf("Enter n and r : ");
 scanf("%d%d", &n, &r);
 printf("Total combinations are : %d\n", comb(n, r));
 printf("Total permutations are : %d\n", perm(n, r));
 return 0;
}

long comb(int n, int r)
{
 long c;
 c = perm(n, r) / factorial(r);
 return c;
}

long perm(int n, int r)
{
 long p;
 p = factorial(n) / factorial(n-r);
 return p;
}

long int factorial(int n)
{
 int i;

```

```

 long int fact=1;
 if(n==0)
 return 1;
 for(i=n; i>1; i--)
 fact*=i;
 return fact;
}

```

### Problem 9

Write a program to print Pascal's triangle.

```

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1

/*P6.25 Pascal's triangle*/
#include<stdio.h>
long factorial(int);
long comb(int, int);
int main(void)
{
 int i, j, k;
 printf("Enter number of rows for Pascal's triangle : ");
 scanf("%d", &k);
 for(i=0; i<k; i++)
 {
 for(j=0; j<=i; j++)
 printf("%5ld", comb(i, j));
 printf("\n");
 }
 return 0;
}

long comb(int n, int r)
{
 long c;
 c = factorial(n) / (factorial(r) * factorial(n-r));
 return c;
}

long int factorial(int n)
{
 int i;
 long int fact=1;
 if(n==0)
 return 1;
 for(i=n; i>1; i--)
 fact*=i;
 return fact;
}

```

### Problem 10

Write a program to convert a decimal number to roman number. The roman numbers corresponding to decimal numbers are as -

|        |        |          |
|--------|--------|----------|
| 1 - i  | 9 - ix | 100 - c  |
| 4 - iv | 10 - x | 500 - d  |
| 5 - v  | 50 - l | 1000 - m |

Some examples are-

|    |          |     |          |      |           |
|----|----------|-----|----------|------|-----------|
| 14 | xiv      | 123 | cxxiii   | 1009 | mix       |
| 48 | xxxxviii | 772 | dcclxxii | 2856 | mmdccclvi |

/\*P6.26 Program to convert a decimal number to roman number\*/

```
#include<stdio.h>
int roman(int, int, char);
int main(void)
{
 int num;
 printf("Enter a number : ");
 scanf("%d", &num);
 if(num==1000)
 num = roman(num, 1000, 'm');
 if(num>=500)
 num = roman(num, 500, 'd');
 if(num>=100)
 num = roman(num, 100, 'c');
 if(num>=50)
 num = roman(num, 50, 'l');
 if(num>=10)
 num = roman(num, 10, 'x');
 if(num>=5)
 num = roman(num, 5, 'v');
 if(num>=1)
 num = roman(num, 1, 'i');
 printf("%s", num);
 return 0;
}
```

int roman(int n, int k, char c)

```
{
 if(n==9)
 {
 printf("ix");
 return 0;
 }
 if(n==4)
 {
 printf("iv");
 return 0;
 }
 while(n>=k)
 {
 printf("%c", c);
 n=n-k;
 }
 return n;
}
```

### Problem 11

Write a program to find out the prime factors of a number. Some examples of prime factors are-

Prime factors of 56 - 2, 2, 2, 7

Prime factors of 98 - 2, 7, 7

Prime factors of 121 - 11, 11

/\*P6.27 Prime Factors\*/

```
#include<stdio.h>
void pfact(int, num);
int main(void)
{
 int num;
 printf("Enter a number : ");
}
```

```
scanf("%d", &num);
pfact(num);
printf("\n");
return 0;
}
```

void pfact(int num)

```
{
 int i;
 for(i=2; num!=1; i++)
 while(num%i==0)
 {
 printf("%d ", i);
 num=num/i;
 }
}
```

### Problem 12

Write a program to find out the sum of this series.

$$x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} \dots \dots \dots \frac{x^{(2*n-1)}}{(2*n-1)!}$$

The  $i$ th term of the series is  $x^{(2*i-1)} / (2*i-1)!$ . We'll take a loop from  $i=1$  to  $i=n$  and add all the terms. To calculate the numerator and denominator of a term we'll use the functions `power()` and `factorial()`. Since the terms have alternate positive and negative signs, we'll take a variable `sign`, initialize it with 1 and multiply it with -1 on each iteration of the loop.

/\*P6.28 Sum of series\*/

```
#include<stdio.h>
long int factorial(int n);
double power(float x, int n);
double series(float x, int n);
int main(void)
{
 float x;
 int n;
 printf("Enter x : ");
 scanf("%f", &x);
 printf("Enter number of terms : ");
 scanf("%d", &n);
 printf("%lf\n", series(x, n));
 return 0;
}
```

double series(float x, int n)

```
{
 int i, j, sign=1;
 float term, sum=0;
 for(i=1; i<=n; i++)
 {
 sign = (i%2==0)?-1:1;
 j=2*i-1;
 term=sign*power(x, j)/factorial(j);
 sum+=term;
 }
 return sum;
}
```

long int factorial(int n)

```
{
 int i;
 long int fact=1;
 if(n==0)
 return 1;
 for(i=n; i>1; i--)
```

```

 fact*=i;
 return fact;
}
double power(float x, int n)
{
 int i;
 float p=1;
 for(i=1; i=n; i++)
 p=p*x;
 return p;
}

```

## Exercise

Assume stdio.h is included in all programs.

```

1. void func(void);
 int main(void)
 {
 printf("Lucknow\n");
 goto ab;
 return 0;
 }
 void func(void)
 {
 ab:
 printf("Bareilly\n");
 }

```

```

3. int func(void);
 int main(void)
 {
 int x=10;
 x=func();
 printf("%d\n",x);
 return 0;
 }
 int func(void)
 {
 printf("Function\n");
 }

```

```

5. int main(void)
 {
 int s;
 s=func(1,2);
 printf("%d\n",s);
 s=func(1,2,3,4);
 printf("%d\n",s);
 return 0;
 }
 int func(int a,int b,int c)
 {
 return a+b+c;
 }

```

```

2. void func(int a,int b);
 int main(void)
 {
 int x;
 x=func(2,3);
 return 0;
 }
 void func(int a,int b)
 {
 int s;
 s=a+b;
 return;
 }

```

```

4. int add(int x,y,z)
 {
 return x+y+z;
 }
 int main(void)
 {
 int sum;
 sum=add(1,2,3);
 return 0;
 }

```

```

6. int sum(int x,int y);
 int multiply(int x,int y);
 int main(void)
 {
 int m=6,n=3;
 printf("%d\t",multiply(m,n));
 printf("%d\t",multiply(15,4));
 printf("%d\t",multiply(m+n,m-n));
 printf("%d\n",multiply(6,sum(m,n)));
 return 0;
 }
 int multiply(int x,int y)
 {
 return x*y;
 }
 int sum(int x,int y)
 {
 return x+y;
 }

```

```

7. int func(int x,int y);
 int main(void)
 {
 int p=func(5,6);
 printf("%d",p);
 return 0;
 }
 int func(int x,int y)
 {
 int x=2;
 return x*y;
 }

```

```

9. int max(int a,int b)
 {
 return a>b? a: b;
 }
 int main(void)
 {
 int a=2,b=8,c=3;
 printf("%d\n",max(a,max(b,c)));
 return 0;
 }

```

```

11. int diff(int x,int y)
 {
 return x-y;
 }
 int sum(int x,int y)
 {
 return x+y;
 }
 int main(void)
 {
 int a=20,b=5,c=2,d=6;
 printf("%d\t",a+diff(d,c));
 printf("%d\n",diff(a,sum(diff(b,c),d)));
 return 0;
 }

```

```

13. int func(int a,int b,int c);
 int main(void)
 {
 int x=1,y=2,z=3,result;
 result=func(x,y,(z=5,z+10));
 printf("x=%d, y=%d, z=%d",x,y,z);
 printf("result=%d\n",result);
 return 0;
 }
 func(int a,int b,int c)
 {
 return 2*(a+b+c);
 }

```

```

8. int min(int a,int b);
 int main(void)
 {
 int a=10,b=5;
 printf("%d\n",min(a,b));
 return 0;
 }
 int min(int a,int b)
 {
 a<b? return a: return b;
 }

```

```

10. void func(int x,int y);
 int main(void)
 {
 int x;
 x=func(5,6)+100;
 printf("%d",x);
 return 0;
 }
 void func(int x,int y)
 {
 int z;
 z=x+y;
 }

```

```

12. int sqr(int x);
 int cube(int x);
 int func(int n);
 int main(void)
 {
 int n=5;
 printf("%d\n",func(n));
 return 0;
 }
 int sqr(int x)
 {
 return x*x;
 }
 int cube(int x)
 {
 return x*x*x;
 }
 int func(int n)
 {
 return n+sqr(n-2)+cube(n-3);
 }

```

```

14. int sum(int a,int b);
 int main(void)
 {
 (void)sum(1,2);
 return 0;
 }
 int sum(int a,int b)
 {
 printf("Sum is %d\n",a+b);
 return a+b;
 }

```

```

15. int square1(int a);
 int square2(double a);
 double square3(int a);
 double square4(double a);
 int main(void)
 {
 double x=2.5,y;
 y=square1(x);
 printf("%lf\t",y);
 y=square2(x);
 printf("%lf\t",y);
 y=square3(x);
 printf("%lf\t",y);
 y=square4(x);
 printf("%lf\n",y);
 return 0;
 }

int square1(int a)
{
 return a*a;
}

int square2(double a)
{
 return a*a;
}

double square3(int a)
{
 return a*a;
}

double square4(double a)
{
 return a*a;
}

18. void func(void);
 int main(void)
 {
 int i=5;
 for(i=i+1; i<8; i++)
 func();
 return 0;
 }

void func(void)
{
 int j;
 for(j=1; j<3; j++)
 printf("%d\t",++j);
}

20. int func(int a,int b,int c);
 int main(void)
 {
 int x;
 x=func(2,3,4);
 printf("%d\n",x);
 return 0;
 }

int func(int a,int b,int c)
{
 return a,b,c;
}

```

```

16. int main(void)
 {
 int func(int a,int b)
 {
 return (a+b);
 }
 int c;
 c = func(3,5);
 printf("%d",c);
 return 0;
 }

17. void display(int,int);
 int main(void)
 {
 int x=15;
 float y=290.5;
 display(x,y);
 return 0;
 }

void display(int a,int b)
{
 printf("%d %d\n",a,b);
}

19. int func(int a,int b)
 int main(void)
 {
 int i=2,j=3;
 printf("%d\n",func(i,j));
 return 0;
 }

int func(int a,int b)
{
 a=a-5;
 b=b+5;
 return (a + --b);
}

21. void func(int a,int b);
 int main(void)
 {
 int i=5,j=10;
 func(i/2,j%3);
 return 0;
 }

void func(int a,int b)
{
 a/=2;
 b--;
 printf("%d\t",a+b);
}

```

```

22. int s=5;
 void func(void);
 int main(void)
 {
 func();
 printf("%d\n",a);
 return 0;
 }

void func(void)
{
 int a=2;
 printf("%d\t",a);
}

24. int func(int k);
 int main(void)
 {
 int i=0,k=3;
 i+=func(k);
 i+=func(k);
 i+=func(k);
 printf("%d\n",i);
 return 0;
 }

int func(int k)
{
 static int m=2;
 m=m+k;
 return m;
}

26. void func(int a,static int b);
 int main(void)
 {
 func(1,2);
 func(3,4);
 return 0;
 }

void func(int a,static int b)
{
 a++;
 b++;
 printf("%d %d\n",a,b);
}

28. int func(int n);
 int main(void)
 {
 printf("%d ",func(2));
 printf("%d ",func(5));
 printf("%d ",func(2));
 return 0;
 }

int func(int n)
{
 static int s=0;
 int i;
 for(i=1; i<=n; i++)
 s+=i;
 return s;
}

29. Rewrite the function func() without using else and variable x.
 int func(int a,int b);
 int f1(int x,int y);

```

```

23. int func(int x,int y);
 int main(void)
 {
 int a=2,b=5;
 a=func(a+b,a-b);
 printf("%d\n",a);
 return 0;
 }

int func(int x,int y)
{
 return x+y,x-y;
}

25. int main(void)
 {
 int i=9;
 if(i==9)
 {
 int i=25;
 }
 printf("i=%d\n",i);
 return 0;
 }

27. int func(void);
 int main(void)
 {
 int i;
 for(i=1; i<=6; i++)
 printf("%d ",func(i));
 return 0;
 }

int func(void)
{
 static int k=1;
 k*=2;
 return k;
}

```

```

int f2(int x,int y);
int main(void)
{
 int a,b;
 printf("Enter a and b : ");
 scanf("%d%d",&a,&b);
 printf("%d ",func(a,b));
 return 0;
}

int func(int a,int b)
{
 int x;
 if(a==b)
 x=f1(a,b);
 else
 x=f2(a,b);
 return x;
}

int f1(int x,int y)
{
 return x+y;
}

int f2(int x,int y)
{
 return x-y;
}

```

30. Rewrite the function func() without using the variable flag.

```

#include<math.h>
int func(int n);
int main(void)
{
 int n;
 printf("Enter a number : ");
 scanf("%d",&n);
 printf("%d ",func(n));
 return 0;
}

int func(int n)
{
 int i,flag=0;
 for(i=2; i<=sqrt(n); i++)
 {
 if(n%i==0)
 {
 flag=1;
 break;
 }
 }
 return flag;
}

```

Rewrite the functions in exercises from 31 to 35 using a single return statement.

```

31. int func(int n)
{
 if(n==39)
 {
 n+=5;
 return n;
 }
 else
 return n;
}

32. int func(int x,int b)
{
 if(x==5)
 return b+2;
 else if(x>5)
 return b+10;
 else
 return b;
}

```

```

33. int abs(int a)
{
 if(a<0)
 return -a;
 else
 return a;
}

a = a-b;
if(a!=0)
{
 a++;
 return a;
}
else
 return 0;
}

```

```

34. int func(int a,int b)
{
 a=a-b;
 if(a!=0)
 return a;
 else
 return 0;
}

```

Rewrite the functions in exercises 36,37,38,39 without using else.

```

36. int isPalindrome(int num)
{
 if(num==reverse(num))
 return 1;
 else
 return 0;
}

```

```

37. int func(int a,int b)
{
 a=a-b;
 if(a!=2)
 {
 a++;
 return a;
 }
 else
 {
 a=a+b;
 printf("%d ",a);
 return b;
 }
}

```

```

38. int func(int m,int n)
{
 if(m==5)
 return n-10;
 else if(m>5)
 return n+10;
 else
 return n;
}

```

```

39. char findGrade(int m1,int m2,int m3,int m4)
{
 float total,per;
 total = m1+m2+m3+m4;
 per = total/4;
 if(per>=85)
 return 'A';
 else if(per>=70)
 return 'B';
 else if(per>=55)
 return 'C';
 else if(per>=40)
 return 'D';
 else
 return 'E';
}

```

40. Write a function that inputs a number and prints the multiplication table of that number.

41. Write a function cubesum() that accepts an integer and returns the sum of the cubes of individual digits of that number. Use this function to make functions PrintArmstrong() and isArmstrong() to print Armstrong numbers and to find whether a number is an Armstrong number. (See program P5.25)

42. Write a function prodDigits() that inputs a number and returns the product of digits of that number.

43. If all digits of a number n are multiplied by each other repeating with the product, the one digit number obtained at last is called the multiplicative digital root of n. The number of times digits need to be multiplied to reach one digit is called the multiplicative persistence of n.

For example -  $86 \rightarrow 48 \rightarrow 32 \rightarrow 6$  (MDR 5, MPersistence 3)  
 $341 \rightarrow 12 \rightarrow 2$  (MDR 2, MPersistence 2)  
 $679 \rightarrow 378 \rightarrow 168 \rightarrow 48 \rightarrow 32 \rightarrow 6$  (MDR 6, MPersistence 5)

Using the function `prodDigits()` of previous exercise write functions `MDR()` and `MPersistence()` that input a number and return its multiplicative digital root and multiplicative persistence respectively.

44. Write a function `sumPdivisors()` that finds the sum of proper divisors of a number. Proper divisors of a number are those numbers by which the number is divisible, except the number itself. For example proper divisors of 36 are 1, 2, 3, 4, 6, 9, 18.

45. A number is called perfect if the sum of proper divisors of that number is equal to the number. For example 28 is a perfect number, since  $1+2+4+7+14=28$ . Write a program to print all the perfect numbers in a given range.

46. Two different numbers are called amicable numbers if the sum of the proper divisors of each is equal to the other number. For example 220 and 284 are amicable numbers.

Sum of proper divisors of 220 =  $1+2+4+5+10+11+20+22+44+55+110=284$

Sum of proper divisors of 284 =  $1+2+4+71+142=220$ .

Write a function to print pairs of amicable numbers in a range.

47. Write a single function to print both amicable pairs and perfect numbers.

48. Using the function `sumPrDivisors()` in Ex 44, write a single line function `isPrime()` containing only a return statement. This function should return 1 if the number is prime and 0 otherwise.

49. Write a function `convert()` that can convert inches to cms and vice versa. The first argument should be the length to be converted and the second argument should be a character('i' or 'c') denoting the measurement unit of the length given in first argument.

50. Write a function `Product()` that accepts two integers and returns their product. Don't use the \* operator. Similarly make two more functions `Quotient()` and `Remainder()` that accept two integers and return the quotient and remainder respectively, without the use of / and % operators.

51. Write a function to find whether a character is alphanumeric.

52. Write a function that accepts a character, if the character is a lower case alphabet its upper case equivalent is returned otherwise the unchanged character is returned.

53. Write a function to find the sum of this series upto n terms.

$$1 + 1/4 + 1/9 + 1/16 + \dots$$

54. Write a program to print twin primes less than 1000. If two consecutive odd numbers are both prime (e.g. 17, 19) then they are known as twin primes.

55. Write a function `isLeap()` which inputs a year and returns 1 if the year is leap otherwise 0.

56. Write a function `isValid()` to find whether a date is valid. (See Program P5.37)

57. Write a function `cmpDate()` to compare two dates, it should return 1 if Date1 is before Date2, -1 if Date 2 is before Date1 and 0 if dates are same.

58. Write a function `findJulian()` that returns Julian day of a date. (See Program P5.41)

59. Write a function `printDayOfWeek()` to print day of the week for a given date (See Program P5.41). Use functions `isValid()`, `isLeap()`, and `findJulian()` from previous exercises.

60. Write a function to find difference of two dates. (See Program P5.38). Use functions `isValid()`, `isLeap()`, and `findJulian()`, `cmpDate()` from previous exercises.

## Recursion

Recursion is a process in which a problem is defined in terms of itself. The problem is solved by repeatedly breaking it into smaller problems, which are similar in nature to the original problem. The smaller problems are solved and their solutions are applied to get the final solution of the original problem. To implement recursion technique in programming, a function should be capable of calling itself. A recursive function is a function that calls itself.

```
int main(void)
{

 rec();

 /*End of main()*/
}

void rec()
{

 rec(); /*recursive call*/

} /*End of rec()*/
```

Here the function `rec()` is calling itself inside its own function body, so `rec()` is a recursive function. When `main()` calls `rec()`, the code of `rec()` will be executed and since there is a call to `rec()` inside `rec()`, again `rec()` will be executed. It seems that this process will go on infinitely but in practice, a terminating condition is written inside the recursive function which ends this recursion. This terminating condition is also known as exit condition or the base case. This is the case when function will stop calling itself and will finally start returning.

Recursion proceeds by repeatedly breaking a problem into smaller versions of the same problem, till finally we get the smallest version of the problem which is simple enough to solve. The smallest version of problem can be solved without recursion and this is actually the base case.

### 7.1 Writing a recursive function

Recursion is actually a way of thinking about problems, so before writing a recursive function for a problem we should be able to define the solution of the problem in terms of a similar type of a smaller problem. The two main steps in writing a recursive function are-

1. Identification of the base case and its solution, i.e. the case where solution can be achieved without recursion. There may be more than one base case.
  2. Identification of the general case or the recursive case i.e. the case in which recursive call will be made.
- Identifying the base case is very important because without it the function will keep on calling itself resulting in infinite recursion.

We must ensure that each recursive call takes us closer to the base case i.e. the size of problem should be diminished at each recursive call. The recursive calls should be made in such a way that finally we arrive at the



base case. If we don't do so, we will have infinite recursion. So merely defining a base case will not help us avoid infinite recursion, we should implement the function such that the base case is finally reached.

## 7.2 Flow of control in Recursive functions

Before understanding the flow of control in recursive calls, first let us see how control is transferred in simple function calls.

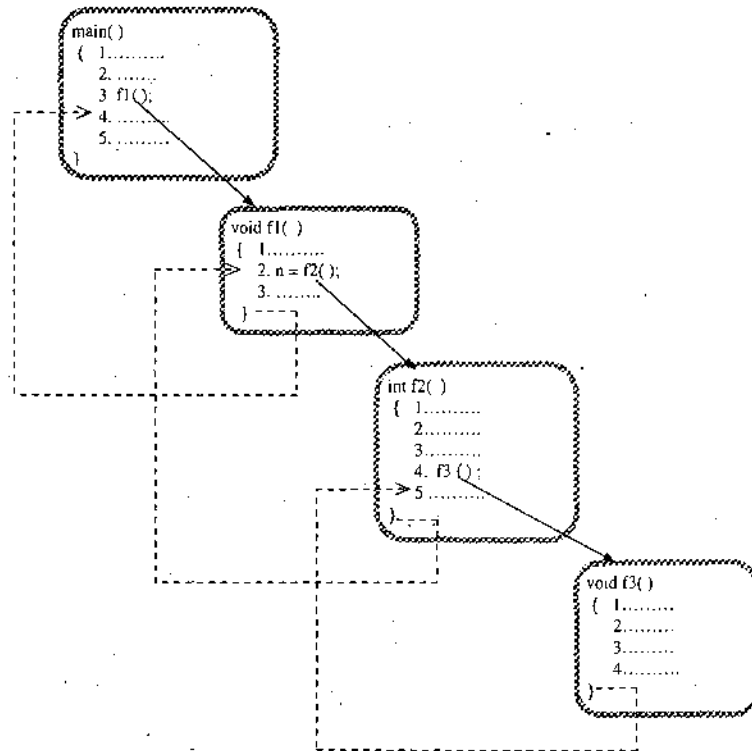


Figure 7.1 Flow of control in normal function calls

In the figure 7.1, the flow of control is-

- \* main() calls function f1() at line 3 so control is transferred to f1().
- \* Inside function f1(), the function f2() is called at line 2 so control is transferred to function f2().
- \* Inside function f2(), the function f3() is called at line 4 so control is transferred to function f3().
- \* Inside function f3() no function is called so control returns back to function f2() at line 5.
- \* After finishing function f2(), control is returned to function f1() at line 2. Here control returned to line 2 because the work of assigning the return value of f2() to variable n still remains.
- \* Now after finishing the execution of function f1() the control is returned to main() at line 4.

When the execution of a function is finished we return to the caller(parent) function at the place where we had left it. Recursive calls are no different and they behave in a similar manner. In figure 7.2, the function that is being called each time is the same. We will call the different calls of func() as different instances or different invocations of func().

Initially main() calls func(), so in first instance of func() value of formal parameter n is 5. Line 1 of this function is executed, after this the terminating condition is checked and since it is false we don't return. Now line 4 is executed and then at line 5, func() is called with argument 3.

Now control transfers to the second instance of func() and inside this instance, value of n is 3. Line 1 of this second instance of func() is executed, after this the terminating condition is checked and since it is false we don't return. Now line 4 is executed and then at line 5, func() is called with argument 1.

Now control transfers to the third instance of func() and inside this instance, value of n is 1. Line 1 of this third instance of func() is executed, after this the terminating condition is checked and since it is true, we return.

We return to second instance of func() at line 6. Note that now we are inside second instance of func() so value of n is 3. After executing lines 6 and 7 we return back to the line 6 of first instance.

Now we are inside first instance of func() so value of n is 5. After executing lines 6 and 7 of first instance we return to main() at line 4.

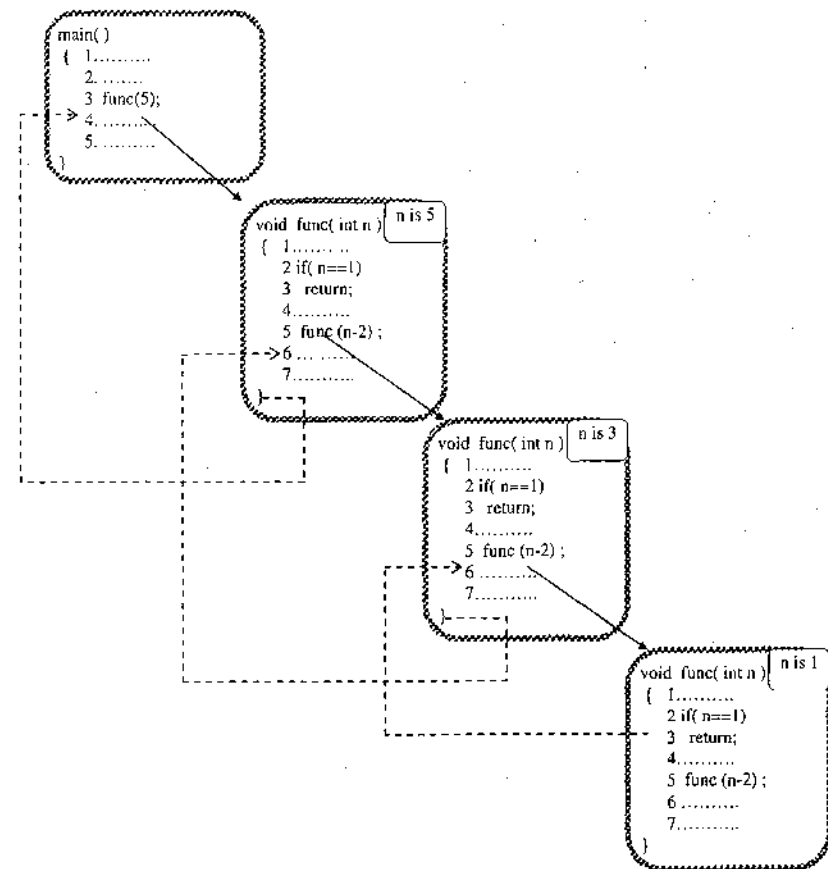


Figure 7.2 Flow of control in recursive function calls

So we can see that the recursive functions are called in a manner similar to that of regular functions, but here the same function is called each time. When execution of an instance of recursive function is finished, we return to the previous instance where we had left it.

We know each function has some local variables that exist only inside that function. The local variables of the called function are active while the local variables of the caller function are kept on hold or suspended. The same case occurs in recursion but here the caller function and called function are copies of the same function. When a function is called recursively, then for each instance a new set of formal parameters and local variables (except static) is created. Their names are same as declared in function but their memory locations are different and they contain different values. These values are remembered by the compiler till the end of function call, so that these values are available while returning. In the example of figure 7.2, we saw that there were three instances of `func()`, but each instance had its own copy of formal parameter `n`.

The number of times that a function calls itself is known as the recursive depth of that function. In the example of figure 7.2, the depth of recursion is 3.

### 7.3 Winding and unwinding phase

All recursive functions work in two phases- winding phase and unwinding phase. Winding phase begins when the recursive function is called for the first time, and each recursive call continues the winding phase. In this phase the function keeps on calling itself and no return statements are executed in this phase. This phase terminates when the terminating condition becomes true in a call. After this the unwinding phase begins and all the recursive function calls start returning in reverse order till the first instance of function returns. In unwinding phase the control returns through each instance of the function.

In some algorithms we need to perform some work while returning from recursive calls, in that case we put that particular code in the unwinding phase i.e. just after the recursive call.

We will try to understand the concept of recursion through numerous examples, although most of them may not be very efficient yet they are classic examples of learning how recursion works. Initially when learning recursion, we should trace the functions and see how the control is transferred to understand the behavior of recursive functions.

## 7.4 Examples of Recursion

### 7.4.1 Factorial

We know that the factorial of a positive integer  $n$  can be found out by multiplying all integers from 1 to  $n$ .  
 $n! = 1 * 2 * 3 * \dots * (n-1) * n$

This is the iterative definition of factorial, and we can easily write an iterative function for it using a loop. Now we will try to find out the solution of this factorial problem recursively.

We know that-  $4! = 4 * 3 * 2 * 1$   
 We can write it as-  $4! = 4 * 3!$  (since  $3! = 3 * 2 * 1$ )  
 Similarly we can write-  $3! = 3 * 2!$   
 $2! = 2 * 1!$   
 $1! = 1 * 0!$

So in general we can say that factorial of a positive integer  $n$  is the product of  $n$  and factorial of  $n-1$ .  
 $n! = n * (n-1)!$

Now problem of finding out factorial of  $(n-1)$  is similar to that of finding out factorial of  $n$ , but it is smaller in size. So we have defined the solution of factorial problem in terms of itself. We know that the factorial of 0 is 1. This can act as the terminating condition or the base case. So the recursive definition of factorial can be written as-

### Recursion

$$n! = \begin{cases} 1 & n = 0 \\ n * (n-1)! & n > 0 \end{cases}$$

Now we will write a program, which finds out the factorial using a recursive function.

**Ex7.1 Program to find the factorial of a number by recursive method\*/**

```
#include <stdio.h>
long int fact(int n);
int main(void)
{
 int num;

 printf("Enter a number : ");
 scanf("%d", &num);
 if(num < 0)
 printf("No factorial for negative number\n");
 else
 printf("Factorial of %d is %ld\n", num, fact(num));
 return 0;
}

long int fact(int n)
{
 if(n == 0)
 return 1;
 return (n * fact(n-1));
}
/*End of fact()*/
```

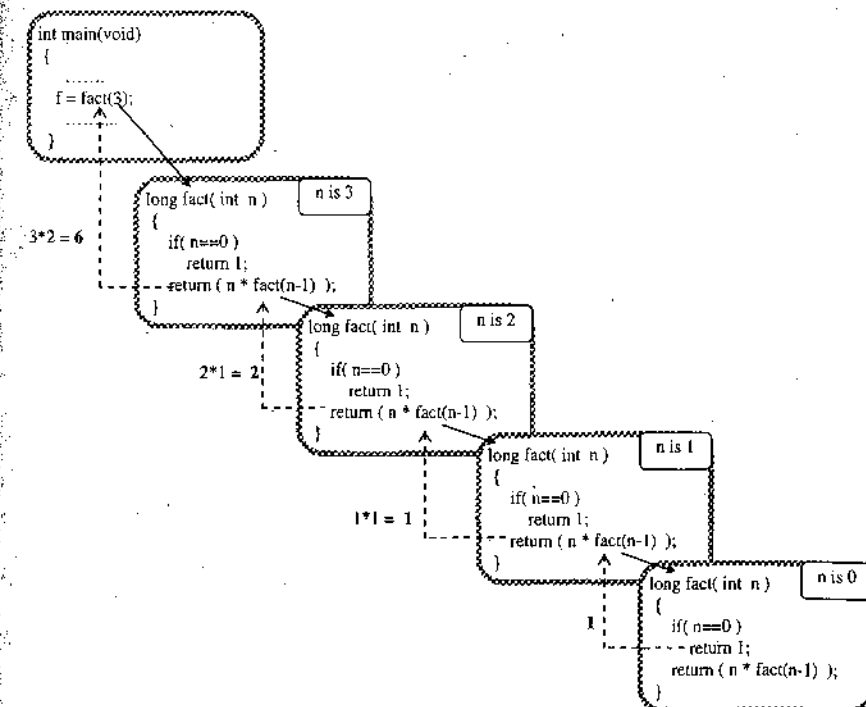


Figure 7.3 Finding Factorial of a number recursively

The function `fact()` returns 1 if the argument `n` is 0, otherwise it returns `n*fact(n-1)`. To return `n*fact(n-1)`, the value of `fact(n-1)` has to be calculated for which `fact()` has to be called again with an argument of `n-1`. This process of calling `fact()` continues till it is called with an argument of 0. Suppose we want to find out the factorial of 3.

|                                                                             |                 |
|-----------------------------------------------------------------------------|-----------------|
| Initially <code>main()</code> calls <code>fact(3)</code>                    | } winding phase |
| Since <code>3&gt;0</code> , <code>fact(3)</code> calls <code>fact(2)</code> |                 |
| Since <code>2&gt;0</code> , <code>fact(2)</code> calls <code>fact(1)</code> |                 |
| Since <code>1&gt;0</code> , <code>fact(1)</code> calls <code>fact(0)</code> |                 |

First time when `fact()` is called, its argument is the actual argument given in the `main()` which is 3. So in the first invocation of `fact()` the value of `n` is 3. Inside this first invocation, there is a call to `fact()` with argument `n-1`, so now `fact()` is invoked for the second time and this time the argument is 2. Now the second invocation calls third invocation of `fact()` and this time argument is 1. The third invocation of `fact()` calls the fourth invocation with an argument of 0.

When `fact()` is called with `n=0`, the condition inside `if` statement becomes true, i.e. we have reached the base case, so now the recursion stops and the statement `return 1` is executed. The winding phase terminates here and the unwinding phase begins and control starts returning towards the original call.

Now every invocation of `fact()` will return a value to the previous invocation of `fact()`. These values are returned in the reverse order of function calls.

|                                                                                                   |                   |
|---------------------------------------------------------------------------------------------------|-------------------|
| Value returned by <code>fact(0)</code> to <code>fact(1)</code> = 1                                | } Unwinding Phase |
| Value returned by <code>fact(1)</code> to <code>fact(2)</code> = $1 * \text{fact}(0) = 1 * 1 = 1$ |                   |
| Value returned by <code>fact(2)</code> to <code>fact(3)</code> = $2 * \text{fact}(1) = 2 * 1 = 2$ |                   |
| Value returned by <code>fact(3)</code> to <code>main()</code> = $3 * \text{fact}(2) = 3 * 2 = 6$  |                   |

The function `fact()` is called 4 times, each function call is different from another and the argument supplied is different each time.

In the program we have taken long `int` as the return value of `fact()`, as the value of factorial for even small value of `n` exceeds the range of `int` data type.

## 7.4.2 Summation of numbers from 1 to n

The next example of a recursive function that we are taking is a function that computes the sum of integers from 1 to `n`. This is similar to the factorial problem; there we had to find the product of numbers from 1 to `n`, while here we have to find the sum.

We can say that the sum of the numbers from 1 to `n` is equal to `n` plus the sum of numbers 1 to `n-1`.

`summation(n) = n + summation(n-1)`

`summation(4) = 4 + summation(3)`  
 $= 4 + 3 + \text{summation}(2)$   
 $= 4 + 3 + 2 + \text{summation}(1)$   
 $= 4 + 3 + 2 + 1 + \text{summation}(0)$

We know that `summation(0)` will be equal to 0, and this can act as the base case. So the base case occurs when the function is called with value of `n` equal to 0 and in this case the function should return 0.

```
int summation(int n)
{
 if(n==0)
 return 0;
 return (n + summation(n-1));
} /*End of summation()*/
```

## 7.4.3 Displaying numbers from 1 to n

In the previous two examples we had found out the product and sum of numbers from 1 to `n`, now we will write a recursive function to display these numbers. This function has to only display the numbers so it will not return any value and will be of type `void`.

We have written two functions `display1()` and `display2()`, these functions are traced in figures 7.4 and 7.5. Let us see which one gives us the desired output.

```
void display1(int n)
{
 if(n==0)
 return;
 printf("%d ",n);
 display1(n-1);
} /*End of display1()*/

void display2(int n)
{
 if(n==0)
 return;
 display2(n-1);
 printf("%d ",n);
} /*End of display2()*/
```

The function `display1()` is traced in figure 7.4. Initially the `printf()` inside the first invocation will be executed and in that invocation value of `n` is 3, so first 3 is displayed. After this the `printf()` inside second invocation is executed and 2 is displayed, and then `printf()` inside third invocation is executed and 1 is displayed. In the fourth invocation the value of `n` becomes 0, we have reached the base case and the recursion stopped. So this function displays the values from `n` to 1, but we wanted to display the values from 1 to `n`.

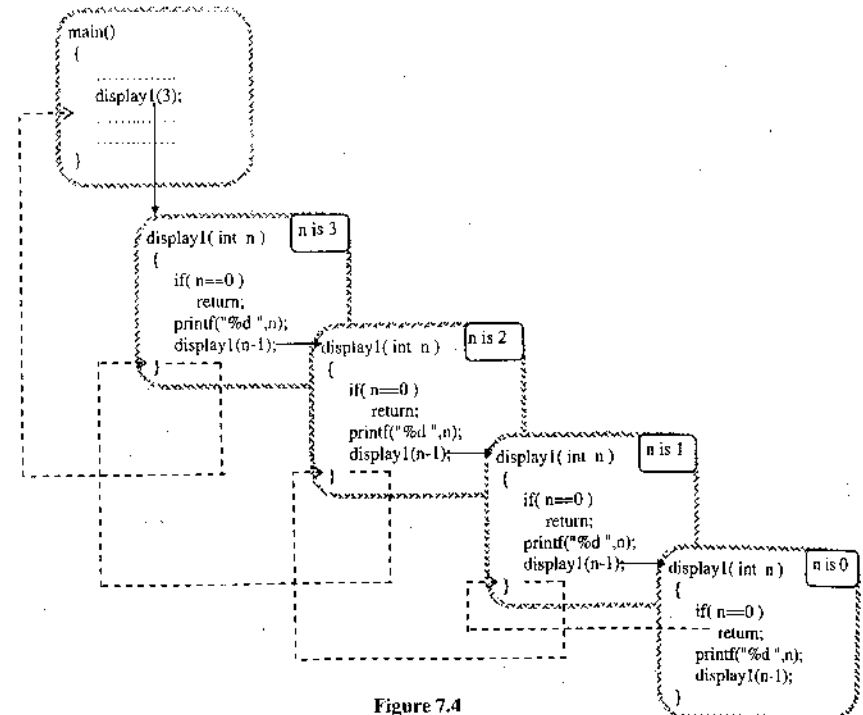


Figure 7.4

Now let us trace the function `display2()`.

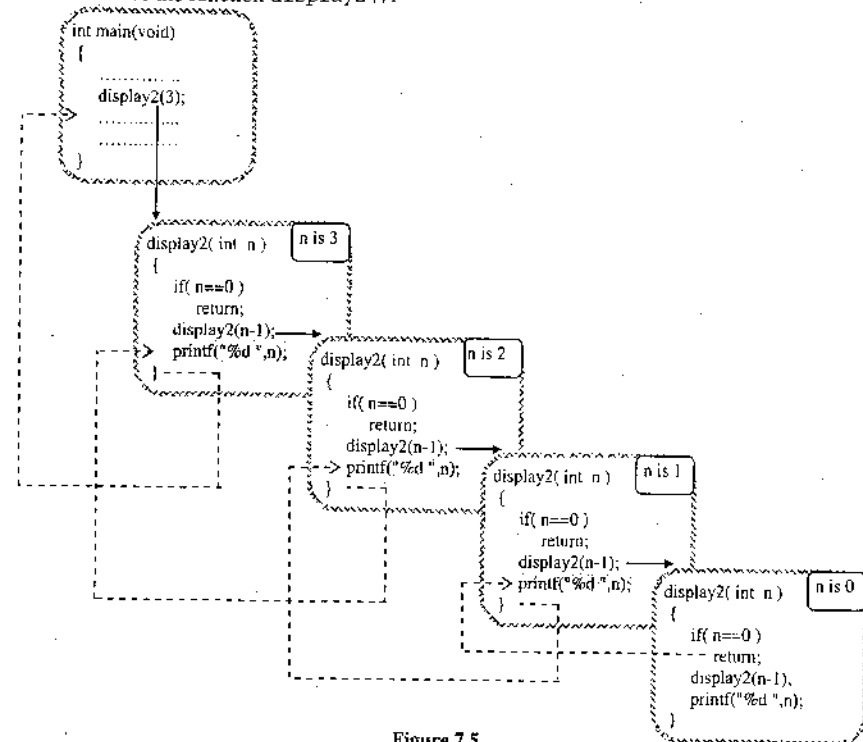


Figure 7.5

In `display2()`, the `printf()` is after the recursive call so the `printf()` functions are not executed in the winding phase, but they are executed in the unwinding phase. In unwinding phase, the calls return in reverse order, so first of all the `printf()` of third invocation is executed and in this invocation value of `n` is 1, so 1 is displayed. After this the `printf()` of second invocation is executed and in this invocation value of `n` is 2 so 2 is displayed and at last the `printf()` of first invocation is executed and 3 is displayed.

#### 7.4.4 Display and Summation of series

Let us take a simple series of numbers from 1 to `n`. We want to write a recursive function that displays this series and finds the sum of this series. For example if the number of terms is 5, then our function should give its output.

$$1 + 2 + 3 + 4 + 5 = 15$$

We have already seen how to display and find the sum of numbers from 1 to `n` so let us combine that logic to write a function for our series.

**P7.3 Program to display and find out the sum of series\*/**

```

/* Series : 1 + 2 + 3 + 4 + 5 + */
#include <stdio.h>
int rseries(int n);
int main(void)
{
 int n;
 printf("Enter number of terms : ");
 scanf("%d", &n);
}

```

```

printf("\b\b = %d\n\n", rseries(n)); /* \b to erase last + sign */
return 0;
/* End of main() */
int rseries(int n)
{
 int sum;
 if(n==0)
 return 0;
 return n + rseries(n-1);
 printf("%d + ", n);
}
/* End of rseries() */

```

Let us find out whether the function `rseries()` gives us the desired output or not. This function will return the sum of the series but will not display any term of the series. This is because in the unwinding phase when control returns to the previous invocation, the return statement (`return n + rseries(n-1)`) is executed and so the function returns without executing the `printf()` function. To make the function work correctly we can write it like this-

```

int rseries(int n)
{
 int sum;
 if(n==0)
 return 0;
 sum = n + rseries(n-1);
 printf("%d + ", n);
 return sum;
}
/* End of rseries() */

```

The `printf()` is before the return statement and after the recursive call, so it will always be executed in the unwinding phase.

#### 7.4.5 Sum of digits of an integer and displaying an integer as sequence of characters

The problem of finding sum of digits of a number can be defined recursively as-

`sumdigits(n)` = least significant digit of `n` + `sumdigits(n with least significant digit removed)`

The sum of digits of a single digit number is the number itself, and it can act as the base case.

If we have to find sum of digits of 45329, we can proceed as-

$$\begin{aligned}
 \text{sumdigits}(45329) &= 9 + \text{sumdigits}(4532) \\
 \text{sumdigits}(4532) &= 2 + \text{sumdigits}(453) \\
 \text{sumdigits}(453) &= 3 + \text{sumdigits}(45) \\
 \text{sumdigits}(45) &= 5 + \text{sumdigits}(4) \\
 \text{sumdigits}(4) &= 4
 \end{aligned}$$

The least significant digit of the integer `n` can be extracted by the expression `n%10`. The recursive call has to be made with the least significant digit removed and this is done by calling the function with `(n/10)`. The base case would be when the function is called with a one digit argument.

```

/* Finds the sum of digits of an integer */
int sumdigits(long int n)
{
 if(n/10==0) /* if n is a single digit number */
 return n;
 return n%10 + sumdigits(n/10);
}
/* End of sumdigits() */

/* Displays the digits of an integer */
void display(long int n)
{
 if(n/10==0)
 {

```

```
printf("%d",n);
return;
```

```
display(n/10);
printf("%d",n%10);
```

```
/*End of display()*/
```

```
/*Displays the digits of an integer in reverse order*/
void Rdisplay(long int n)
```

```
{
 if(n/10==0)
```

```
{
 printf("%d",n);
 return;
```

```
printf("%d",n%10);
Rdisplay(n/10);
```

```
/*End of Rdisplay()*/
```

## 7.4.6 Base conversion

Now we will write a recursive function which will convert a decimal number to binary, octal and hexadecimal base. To do this conversion we have to divide the decimal number repeatedly by the base till it is reduced to 0 and print the remainders in reverse order. If the base is hexadecimal then we have to print alphabets for remainder values greater than or equal to 10. We want to print the remainders in reverse order, so we can do this work in the unwinding phase.

```
/*P7.5 Program to convert a positive decimal number to Binary, Octal or Hexadecimal*/
```

```
#include<stdio.h>
```

```
void convert(int, int);
```

```
int main(void)
```

```
{
```

```
 int num;
 printf("Enter a positive decimal number : ");
 scanf("%d",&num);
 convert(num,2); printf("\n");
 convert(num,8); printf("\n");
 convert(num,16); printf("\n");
 return 0;
```

```
/*End of main()*/
```

```
void convert(int num,int base)
```

```
{
```

```
 int rem = num%base;
 if(num==0)
 return;
 convert(num/base,base);
 if(rem < 10)
 printf("%d",rem);
 else
 printf("%c",rem-10+'A');
/*End of convert()*/
```

## 7.4.7 Exponentiation of a float by a positive integer

The iterative definition for finding  $a^n$  is

$a^n = a * a * a * \dots * a$  n times

The recursive definition can be written as-

$$a^n = \begin{cases} 1 & n=0 \quad (\text{Base case}) \\ a * a^{n-1} & n>0 \quad (\text{Recursive case}) \end{cases}$$

```
/*P7.6 Program to raise a floating point number to a positive integer*/
```

```
#include<stdio.h>
```

```
float power(float a,int n);
```

```
int main(void)
```

```
{
```

```
 float a,p;
```

```
 int n;
```

```
 printf("Enter a and n : ");
```

```
 scanf("%f%d",&a,&n);
```

```
 p = power(a,n);
```

```
 printf("%f raised to power %d is %f\n",a,n,p);
```

```
 return 0;
```

```
/*End of main()*/
```

```
float power(float a,int n)
```

```
{
```

```
 if(n==0)
```

```
 return 1;
```

```
 else
```

```
 return(a * power(a, n-1));
```

```
/*End of power()*/
```

## 7.4.8 Prime Factorization

Prime factorization of a number means factoring a number into a product of prime numbers. For example prime factors of numbers 84 and 45 are-

$84 = 2 * 2 * 3 * 7$

$45 = 3 * 3 * 5$

To find prime factors of a number n, we check its divisibility by prime numbers 2,3,5,7,... till we get a divisor d. Now d becomes a prime factor and the problem reduces to finding prime factors of  $n/d$ . The base case occurs when problem reduces to finding prime factors of 1. Let us see how we can find prime factors of 84.

84 is divisible by 2, so 2 is a PF and now problem reduces to finding PFs of  $84/2 = 42$

42 is divisible by 2, so 2 is a PF and now problem reduces to finding PFs of  $42/2=21$

21 is not divisible by 2 so we check divisibility by next prime number which is 3,

21 is divisible by 3, so 3 is PF and now problem reduces to finding PFs of  $21/3=7$ .

7 is not divisible by 3 so we check divisibility by next prime number which is 5,

7 is not divisible by 5 so we check divisibility by next prime number which is 7,

7 is divisible by 7, so 7 is a PF and now problem reduces to finding PFs of  $7/7=1$

The recursive function for printing the prime factors of a number are-

```
void PFactors(int num)
```

```
{
```

```
 int i=2;
```

```
 if(num==1)
```

```
 return;
```

```
 while(num%i==0)
```

```
 i++;
```

```
 printf("%d ",i);
```

```
 PFactors(num/i);
```

```
/*End of PFactors()*/
```

For simplicity we have checked the divisibility by all numbers starting from 2(prime and non prime), but we will get only prime factors. For example 6 is non prime factor of 84 but it has already been removed as a factor of 2 and factor of 3.

## 7.4.9 Greatest Common Divisor

The greatest common divisor (or highest common factor) of two integers is the greatest integer that divides both of them without any remainder. It can be found out by using Euclid's remainder algorithm which states that-

$$\text{GCD}(a, b) = \begin{cases} a & b = 0 \quad (\text{Base case}) \\ \text{GCD}(b, a \% b) & \text{otherwise} \quad (\text{Recursive case}) \end{cases}$$

|             |                            |
|-------------|----------------------------|
| CD(35, 21)  | (a = 35, b = 21)           |
| GCD(21, 14) | (a = 21, b = 35 % 21 = 14) |
| GCD(14, 7)  | (a = 14, b = 21 % 14 = 7)  |
| GCD(7, 0)   | (a = 7, b = 14 % 7 = 0)    |
| 7           |                            |

```

D(12, 26) (a = 12, b = 26)
GCD(26, 12) (a = 26, b = 12 % 26 = 12)
GCD(12, 2) (a = 12, b = 26 % 12 = 2)
GCD(2, 0) (a = 2, b = 12 % 2 = 0)
2

```

A recursive function can be written as:-  
`GCD(int a, int b)`

```

 if(b==0)
 return a;
 return GCD(b, a%b);
} // End of GCD()

```

Here nothing is to be done in the unwinding phase. The value returned by the last recursive call just becomes return value of previous recursive calls, and finally it becomes the return value of the first recursive call.

### 1.10 Fibonacci Series

onacci series is a sequence of numbers in which the first two numbers are 1, and after that each number is sum of previous two numbers.

1, 2, 3, 5, 8, 13, 21, 34, 55, 89, .....

problem of finding the  $n^{\text{th}}$  Fibonacci number can be recursively defined as-

$$x(n) = \begin{cases} 1 & n=0 \text{ or } n=1 \quad (\text{Base case}) \\ \text{fib}(n-1) + \text{fib}(n-2) & n>1 \quad (\text{Recursive case}) \end{cases}$$

7.9 Program to generate fibonacci series\*/.

```

fib(int n);
main(void)

 int nterms, i;
 printf("Enter number of terms: ");
 scanf("%d", &nterms);

 for(i=0; i<nterms; i++)
 printf("%d ", fib(i));
 printf("\n");
 return 0;
}

// End of main() */
fib(int n)

 if(n==0 || n==1)
 return 1;
 return(fib(n-1) + fib(n-2));
}

```

```

}/*End of fib()*/

```

Here the function `fib()` is called two times inside its own function body. The following figure shows the recursive calls of function `fib()` when it is called with argument 5.

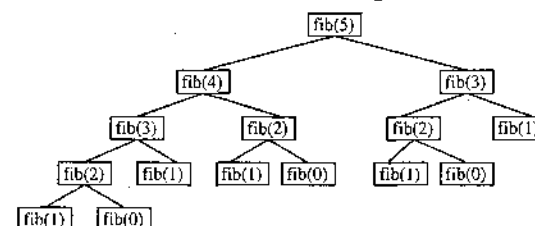


Figure 7.6

This implementation of Fibonacci is very inefficient because it performs same computations repeatedly, for example in the above example it computed the value of `fib(2)` 3 times.

### 7.4.11 Checking Divisibility by 9 and 11

We can easily check divisibility by any number using % operator but here we will develop functions for checking divisibility by 9 and 11 using the definitions that we have learnt in mathematics because these can help us improve our recursive thinking.

A number is divisible by 9 if and only if the sum of digits of the number is divisible by 9.

$$\text{test}(4968589) \rightarrow 4 + 9 + 6 + 9 + 8 + 5 + 8 + 9 = 58$$

```
test(58) -> 5 + 8 = 13
```

```
test(13) -> 1 + 3 = 4
```

```
test(4) -> not divisible by 9
```

```
test(1469358) -> 1 + 4 + 6 + 9 + 3 + 5 + 8 = 36
```

```
test(36) -> 3 + 6 = 9
```

```
test(9) -> divisible by 9
```

The function will be recursively called for the sum of digits of the number. The recursion will stop when the number reduces to one digit. If that digit is 9, number is divisible by 9. If that digit is less than 9 then number is not divisible by 9.

```
int divisibleBy9(long int n)
{
 int sumofDigits;
 if(n==9)
 return 1;
 if(n<9)
 return 0;
 sumofDigits=0;
 while(n>0)
 {
 sumofDigits+=n%10;
 n/=10;
 }
 return divisibleBy9(sumofDigits);
} /*End of divisibleBy9()*/
```

This function returns 1 if number is divisible by 9 otherwise it returns 0.

Now let us make a function that checks divisibility by 11. A number is divisible by 11 if and only if the difference of the sums of digits at odd positions and even positions is either zero or divisible by 11.

$$\text{test}(91628153) \rightarrow [28(9+6+8+5) - 7(1+2+1+3)] = 21$$

```
test(21) -> (2 - 1) = 1
```

```
test(1) -> Not divisible by 11
```

```
62938194 -> [32(6+9+8+9) - 10(2+3+1+4)] = 22
test(22) -> [2 ~ 2] = 0
test(0) -> divisible by 11
```

The function will be recursively called for the difference of digits in odd and even positions. The recursion will stop when the number reduces to one digit. If that digit is 0, then number is divisible by 11 otherwise it is not divisible by 11.

```
int divisibleBy11(long int n)
{
 int s1=0,s2=0,diff;
 if(n==0)
 return 1;
 if(n<10)
 return 0;
 while(n>0)
 {
 s1+=n%10;
 n/=10;
 s2+=n%10;
 n/=10;
 }
 diff = s1>s2 ? (s1-s2) : (s2-s1);
 return divisibleBy11(diff);
}
/*End of divisibleBy11()*/
```

This function returns 1 if number is divisible by 11 otherwise it returns 0.

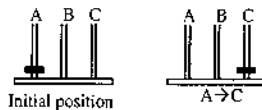
## 7.4.12 Tower of Hanoi

The problem of Tower of Hanoi is to move disks from one pillar to another using a temporary pillar. Suppose we have a source pillar A which has finite number of disks, and these disks are placed on it in a decreasing order i.e. largest disk is at the bottom and the smallest disk is at the top. Now we want to place all these disks on destination pillar C in the same order. We can use a temporary pillar B to place the disks temporarily whenever required. The conditions for this game are-

1. We can move only one disk from one pillar to another at a time.
2. Larger disk cannot be placed on smaller disk.

Suppose the number of disks on pillar A is  $n$ . First we will solve the problem for  $n=1$ ,  $n=2$ ,  $n=3$  and then we will develop a general procedure for the solution. Here A is the source pillar, C is the destination pillar and B is the temporary pillar.

For  $n=1$



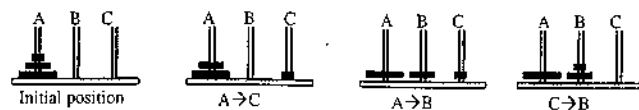
Move the disk from pillar A to pillar C. (A → C)

For  $n=2$



- (i) Move disk 1 from pillar A to B (A → B)
- (ii) Move disk 2 from A to C (A → C)
- (iii) Move disk 1 from pillar B to C (B → C)

For  $n=3$



## Recursion



- (i) Move disk 1 from pillar A to C (A → C)
- (ii) Move disk 2 from pillar A to B (A → B)
- (iii) Move disk 1 from pillar C to B (C → B)
- (iv) Move disk 3 from pillar A to C (A → C)
- (v) Move disk 1 from pillar B to A (B → A)
- (vi) Move disk 2 from pillar B to C (B → C)
- (vii) Move disk 1 from pillar A to C (A → C)

These were the solutions for  $n=1$ ,  $n=2$ ,  $n=3$ . From these solutions we can observe that first we move disks from source pillar (A) to temporary pillar (B) and then move the largest ( $n^{\text{th}}$ ) disk to the destination pillar (C). So the general solution for  $n$  disks can be written as-

1. Move upper  $n-1$  disks from A to B using C as the temporary pillar.
2. Move  $n^{\text{th}}$  disk from A to C.
3. Move  $n-1$  disks from B to C using A as the temporary pillar.

The base case would be when we have to move only one disk, in that case we can simply move the disk from source to destination pillar and return. The recursion tree for  $n=5$  is given below.

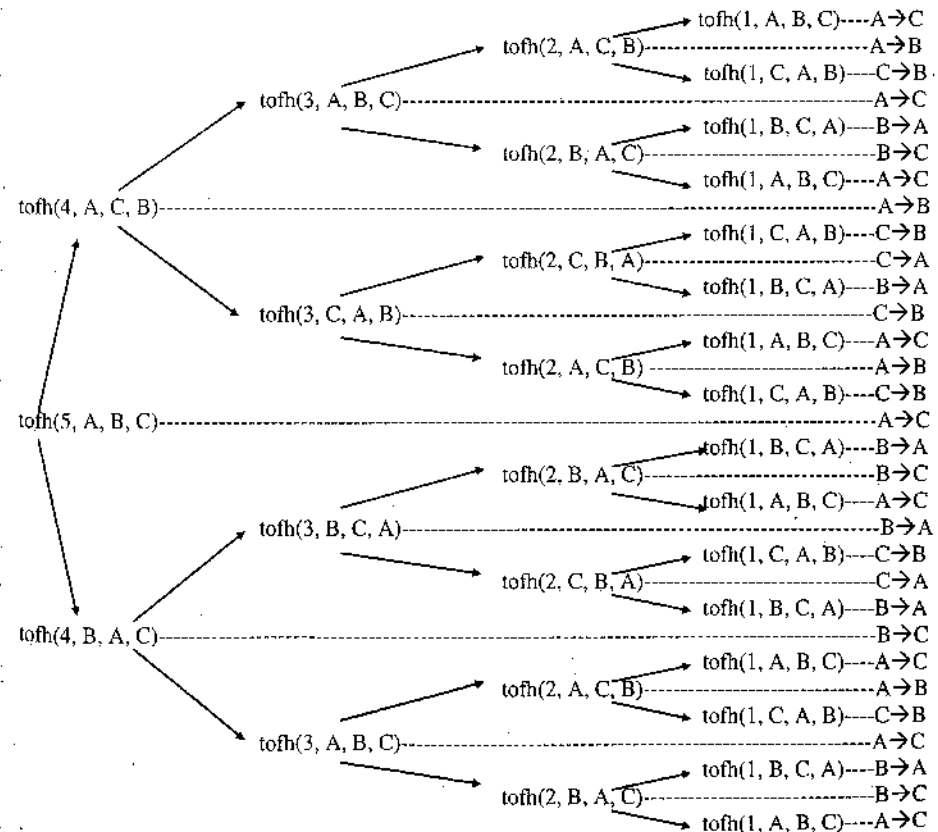


Figure 5.7 Tower of Hanoi

The recursive function for solving tower of Hanoi can be defined as-

$$\text{toh}(n, \text{source}, \text{temp}, \text{dest}) = \begin{cases} \text{Move disk 1 from source to dest} & n=1 \\ \text{toh}(n-1, \text{source}, \text{dest}, \text{temp}) \\ \text{Move } n^{\text{th}} \text{ disk from source to dest} & n>1 \\ \text{toh}(n-1, \text{temp}, \text{source}, \text{dest}) \end{cases}$$

/\*P7.11 Program to solve Tower of Hanoi problem using recursion\*/

```
#include<stdio.h>
void toh(int ndisk,char source,char temp,char dest);
int main(void)
{
 char source='A',temp='B',dest='C';
 int ndisk;
 printf("Enter the number of disks : ");
 scanf("%d", &ndisk);
 printf("Sequence is :\n");
 toh(ndisk,source,temp,dest);
 return 0;
}

void toh(int ndisk,char source,char temp,char dest)
{
 if(ndisk==1)
 {
 printf("Move Disk %d from %c-->%c\n",ndisk,source,dest);
 return;
 }
 toh(ndisk-1,source,dest,temp);
 printf("Move Disk %d from %c-->%c\n",ndisk,source,dest);
 toh(ndisk-1,temp,source,dest);
}
/*End of toh()*/
```

## 7.5 Implementation of Recursion

We have seen that recursive calls execute just like normal function calls, so there is no special technique for implementing them. All function calls whether recursive or non recursive are implemented through run time stack and in the previous chapter we had seen how it is done. Here we will take the example of function fact() called by main with argument 3, and see the changes in the run time stack as the function fact() is evaluated.

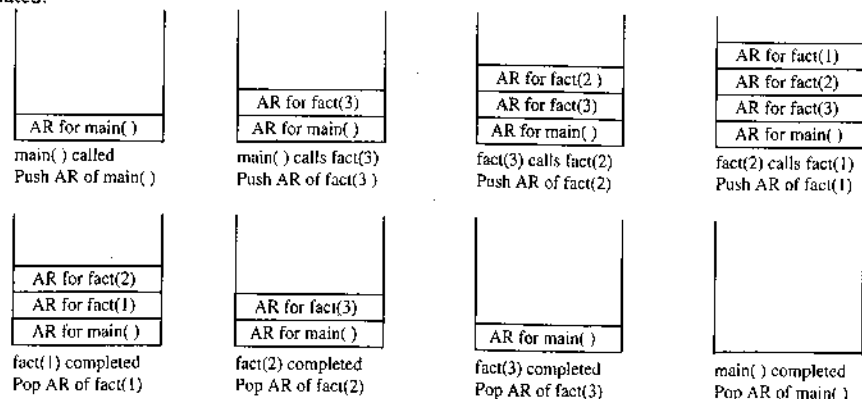


Figure 7.7

In the winding phase, the stack grows as new activation records are created and pushed for each invocation of the function. In the unwinding phase, the activation records are popped from the stack in LIFO manner till the original call returns.

## 7.6 Recursion vs. Iteration

All repetitive problems can be implemented either recursively or iteratively. First we will compare the workings of both approaches.

In loops, the same block of code is executed repeatedly, and repetition occurs when the block of code is finished or a continue statement is encountered. In recursion, the same block of code is repeatedly executed and repetition occurs when the function calls itself.

In loops, the variables inside the loop are modified using some update statement. In recursion, the new values are passed as parameters to the next recursive call.

For the loop to terminate, there is a terminating condition and the loop progresses in such a way that this condition is eventually hit. If this does not happen then we get an infinite loop. For the recursion to terminate, there is a terminating condition (base case) when function stops calling itself, and the recursion should proceed in such a way that we finally hit the base case. If this does not happen then we will have infinite recursion, and the function will keep on calling itself till the stack is exhausted and we get stack overflow error.

Recursion involves pushing and popping activation records of all the currently active recursive calls on the stack. So the recursive version of a problem is usually slower because of the time spent in pushing and popping these activation records. It is also expensive in terms of memory as it uses space in run time stack to store these activation records. If the recursion is too deep, then the stack may overflow and the program will crash. On the other hand the iterative versions do not have to pay for this function call overhead and so are faster and require less space.

Recursive solutions involve more execution overhead than their iterative counterparts, but their main advantage is that they simplify the code and make it more compact and elegant. Recursive algorithms are easier to understand because the code is shorter and clearer.

Recursion should be used when the underlying problem is recursive in nature or when the data structure on which we are operating is recursively defined like trees. Iteration should be used when the problem is not inherently recursive, or the stack space is limited.

For some problems which are very complex, iterative algorithms are harder to implement and it is easier to solve them recursively. In these cases, recursion offers a better way of writing our code which is both logical and easier to understand and maintain. So sometimes it may be worth sacrificing efficiency for code readability.

Recursion can be removed by maintaining our own stack or by using iterative version.

## 7.7 Tail recursion

Before studying about tail recursive functions, let us see what tail recursive calls are. A recursive call is tail recursive if it is the last statement to be executed inside the function.

```
void display1(int n)
{
 if(n==0)
 return;
 printf("%d ",n);
 display1(n-1); /*Tail recursive Call*/
}
/*End of display1()*/

void display2(int n)
```



```

if(n==0)
 return;
display2(n-1); /*Not a Tail recursive Call*/
printf("%d ",n);
}/*End of display2()*/

```

In non void functions, if the recursive call appears in return statement and that call is not part of an expression then the call is tail recursive.

```

int GCD(int a,int b)
{
 if(b==0)
 return a;
 return GCD(b,a%b); /*Tail recursive call*/
}/*End of GCD()*/

long fact(int n)
{
 if(n==0)
 return(1);
 return(n * fact(n-1)); /*Not a tail recursive call*/
}/*End of fact()*/

```

Here the call fact(n-1) appears in the return statement but it is not a tail recursive call because the call is part of an expression. Now let us see some functions which have more than one recursive calls.

```

void tofh(int ndisk,char source,char temp,char dest)
{
 if(ndisk==1)
 {
 printf("Move Disk %d from %c-->%c\n",ndisk,source,dest);
 return;
 }
 tofh(ndisk-1,source,dest,temp); /*Not tail recursive call*/
 printf("Move Disk %d from %c-->%c\n",ndisk,source,dest);
 tofh(ndisk-1,temp,source,dest); /*Tail recursive call*/
}/*End of tofh()*/

```

Here the first recursive call is not a tail recursive call while the second one is a tail recursive call. In the next function we have two recursive calls and both are tail recursive.

```

int binary_search(int arr[],int item, int low, int up)
{
 int mid;
 if(up<low)
 return -1;
 mid = (low+up)/2;
 if(item>arr[mid])
 return binary_search(arr,item,mid+1,up); /*tail recursive call*/
 else if(item<arr[mid])
 return binary_search(arr,item,low,mid-1); /*tail recursive call*/
 else
 return mid;
}/*End of binary_search()*/

```

Here only one recursive call will be executed in each invocation of the function, and that recursive call will be the last one to be executed inside the function. So both the calls in this function are tail recursive.

The two recursive calls in fibonacci() function(P7.9) are not tail recursive because these calls are part of an expression, and after returning from the call, the return value has to be added to the return value of other recursive call.

A function is tail recursive if all the recursive calls in it are tail recursive. In the examples given earlier the tail recursive functions are - display1(), GCD(), binary\_search(). The functions display2(), tofh() and fact() are not tail recursive functions.

Tail recursive functions can easily be written using loops, because as in a loop there is nothing to be done after an iteration of the loop finishes, in tail recursive functions there is nothing to be done after the current recursive call finishes execution. Some compilers automatically convert tail recursion to iteration for improving the performance.

In tail recursive functions, the last work that a function does is a recursive call, so no operation is left pending after the recursive call returns. In non void tail recursive functions (like GCD) the value returned by the last recursive call is the value of the function. Hence in tail recursive functions, there is nothing to be done in the unwinding phase.

Since there is nothing to be done in the unwinding phase, we can jump directly from the last recursive call to the place where recursive function was first called. So there is no need to store the return address of previous recursive calls and values of their local variables, parameters, return values etc. In other words there is no need of pushing new AR for all recursive calls.

Some modern compilers can detect tail recursion and perform tail recursion optimization. They do not push new activation record when a recursive call occurs; rather they overwrite the previous activation record by the current activation record, while retaining the original return address. So we have only one activation record in the stack at a time, and this is for the currently executing recursive call. This improves the performance by reducing the time and memory requirement. Now it doesn't matter how deep the recursion is, the space required will be always be constant.

Since tail recursion can be efficiently implemented by compilers, we should try to make our recursive functions tail recursive whenever possible.

A recursive function can be written as a tail recursive function using an auxiliary parameter. The result is accumulated in this parameter and this is done in such a way that there is no pending operation left after the recursive call. For example we can rewrite the factorial function that we have written earlier as a tail recursive function.

```

long TRfact(int n, int result)
{
 if(n==0)
 return result;
 return TRfact(n-1, n*result);
}/*End of TRfact()*/

```

This function should be called as TRfact(n,1). We can make a helper function to initialize the value of result to 1. The use of this helper function hides the auxiliary parameter.

```

tailRecursiveFact(int n)
{
 return TRfact(n,1);
}/*End of TailRecursiveFact()*/

```

Functions which are not tail recursive are called augmentive recursive functions and these types of function have to finish the pending work after the recursive call finishes.

## 7.8 Indirect and direct Recursion

If a function f1() calls f2() and the function f2() in turn calls f1(), then this is indirect recursion, because the function f1() is calling itself indirectly.

```

f1()
{
 f2();
}

```

```

}
f2()
{
 f1();
}

```

The chain of functions in indirect recursion may involve any number of functions, for example `f1()` calls `f2()`, `f2()` calls `f3()`, `f3()` calls `f4()`, `f4()` calls `f1()`. If a function calls itself directly i.e. `f1()` is called inside its own function body, then that recursion is direct recursion. All the examples that we have seen in this chapter use direct recursion. Indirect recursion is complex and is rarely used.

## Exercise

Find the output of programs from 1 to 16.

```

1. int main(void)
{
 printf("%d %d\n", func1(3,8), func2(3,8));
 return 0;
}

func1(int a, int b)
{
 if(a>b)
 return 0;
 return b + func1(a,b-1);
}

func2(int a, int b)
{
 if(a>b)
 return 0;
 return a + func2(a+1,b);
}

```

```

2. int main(void)
{
 printf("%d \n", func(3,8));
 return 0;
}

int func(int a, int b)
{
 if(a>b)
 return 1000;
 return a + func(a+1,b);
}

```

```

3. int main(void)
{
 printf("%d\n", func(6));
 printf("%d\n", func1(6));
 return 0;
}

int func(int a)
{
 if(a==10)
 return a;
 return a + func(a+1);
}

int func1(int a)
{
 if(a==0)
 return a;
 return a + func1(a+1);
}

```

```

4. int main(void)
{
 printf("%d\n", func(4,8));
 printf("%d\n", func(3,8));
 return 0;
}

int func(int a, int b)
{
 if(a==b)
 return a;
 return a + b + func(a+1,b-1);
}

```

```

5. int main(void)
{
 func1(10,18);
 printf("\n");
 func2(10,18);
 return 0;
}

void func1(int a, int b)
{
 if(a>b)
 return;
 printf("%d ", b);
 func1(a,b-1);
}

void func2(int a, int b)
{
 if(a>b)
 return;
 func2(a,b-1);
 printf("%d ", b);
}

```

```

6. main()
{
 func1(10,18);
 printf("\n");
 func2(10,18);
 return 0;
}

void func1(int a, int b)
{
 if(a>b)
 return;
 printf("%d ", a);
 func1(a+1,b);
}

void func2(int a, int b)
{
 if(a>b)
 return;
 func2(a+1,b);
 printf("%d ", a);
}

```

```

7. int main(void)
{
 printf("%d\n", func(3,8));
 printf("%d\n", func(3,0));
 printf("%d\n", func(0,3));
 return 0;
}

int func(int a, int b)
{
 if(b==0)
 return 0;
 if(b==1)
 return a;
 return a + func(a,b-1);
}

```

```

8. int main(void)
{
 printf("%d\n", count(17243));
 return 0;
}

int count(int n)
{
 if(n==0)
 return 0;
 return 1 + count(n/10);
}

```

```

9. int main(void)
{
 printf("%d\n", func(14837));
 return 0;
}

int func(int n)
{
 return (n)? n%10 + func(n/10) : 0;
}

```

```

10. int main(void)
{
 printf("%d\n", count(123212,2));
 return 0;
}

int count(long int n, int d)
{
 if(n==0)
 return 0;
 else if(n%10 == d)
 return 1 + count(n/10,d);
 else
 return count(n/10,d);
}

```

11. int main(void)

```
{
 char str[100], a;
 printf("Enter a string :");
 gets(str);
 printf("Enter a character :");
 scanf("%c", &a);
 printf("%d\n", f(str, a));
 return 0;
}

int f(char *s, char a)
{
 if(*s == '\0')
 return 0;
 if(*s == a)
 return 1 + f(s+1, a);
 return f(s+1, a);
}
```

12. int main(void)

```
{
 func1(4);
 func2(4);
 return 0;
}

void func1(int n)
{
 int i;
 if(n==0)
 return;
 for(i=1; i<=n; i++)
 printf("%*s",
 printf("\n");
 func1(n-1);
}

void func2(int n)
{
 int i;
 if(n==0)
 return;
 func2(n-1);
 for(i=1; i<=n; i++)
 printf("%*s",
 printf("\n");
}
```

13. Write a recursive function to input and add n numbers.

14. Write a recursive function to enter a line of text and display it in reverse order, without storing the text in an array.

15. Write a recursive function to count all the prime numbers between numbers a and b(both inclusive).

16. A positive proper divisor of n is a positive divisor of n, which is different from n. For example 1,3,5,9,15 are positive proper divisors 45, but 45 is not a proper divisor of 45. Write a recursive function that displays all the proper divisors of a number and returns their sum.

17. A number is perfect if the sum of all its positive proper divisors is equal to the number, for example 28 is a perfect number since  $28 = 1 + 2 + 4 + 7 + 14$ . Write a recursive function that finds whether a number is perfect or not.

18. Write a recursive function that displays a positive integer in words, for example if the integer is 2134 then it is displayed as - two one three four.

19. Write a recursive function that reverses an integer. For example if the input is 43287 then the function should return the integer 78234.

20. Write a recursive function to find remainder when a positive integer a is divided by positive integer b.

21. Write a recursive function to find quotient when a positive integer a is divided by positive integer b.

22. The computation of  $a^n$  can be made efficient if we apply the following procedure instead of multiplying n times -

$$\begin{array}{llll}
 a^8 = (a^2)^4 & a^{11} = a * (a^2)^5 & a^{19} = a * (a^2)^9 & a^{20} = (a^2)^{10} \\
 a^4 = (a^2)^2 & a^5 = a * (a^2)^2 & a^9 = a * (a^2)^4 & a^{10} = (a^2)^5 \\
 a^2 = (a^2)^1 & a^2 = (a^2)^1 & a^4 = (a^2)^2 & a^5 = a * (a^2)^2 \\
 a^1 = a * (a^2)^0 & a^1 = a * (a^2)^0 & a^2 = (a^2)^1 & a^2 = (a^2)^1 \\
 & & a^1 = a * (a^2)^0 & a^1 = a * (a^2)^0
 \end{array}$$

Write a recursive function to compute  $a^n$  using this procedure.

23. Write a recursive function to multiply two numbers by Russian peasant method. Russian peasant method multiplies any two positive numbers using multiplication by 2, division by 2 and addition. Here the first number is divided by 2 (integer division), and the second is multiplied by 2 repeatedly until the first number reduces to 1. Suppose we have to multiply 19 by 25, we write the result of division and multiplication by 2, in the two columns like this-

|    |     |     |
|----|-----|-----|
| 19 | 25  | Add |
| 9  | 50  | Add |
| 4  | 100 |     |
| 2  | 200 |     |
| 1  | 400 | Add |
|    | 475 |     |

Now to get the product we'll add those values of the right hand column, for which the corresponding left column values are odd. So 25, 50, 400 will be added to get 475, which is the product of 19 and 25.

24. Write recursive functions to find values of  $\lfloor \log_2 N \rfloor$  and  $\lfloor \log_3 N \rfloor$ .

25. Write a recursive function to find the Binomial coefficient  $C(n,k)$  which is defined as-

$$C(n,0)=1$$

$$C(n,n)=1$$

$$C(n,k) = C(n-1,k-1) + C(n-1,k)$$

26. Write a program to find out the sum of this series, both by iterative and recursive methods.

$$x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \dots - \frac{x^{(2*n-1)}}{(2*n-1)!}$$

27. Write a recursive function to print these pyramids of numbers.

```

1 1 2 3 4 4 3 2 1
1 2 1 2 3 3 2 1
1 2 3 1 2 2 1
1 2 3 4 1 1

```

28. A triangular number is the number of dots required to fill an equilateral triangle. The first 4 triangular numbers are 1, 3, 6, 10.

```

*
**


```

Write a recursive function that returns  $n^{\text{th}}$  triangular number.

The variables that we have used till now are capable of storing only one value at a time. Consider a situation when we want to store and display the age of 100 employees. For this we have to do the following-

1. Declare 100 different variables to store the age of employees.
2. Assign a value to each variable.
3. Display the value of each variable.

Although we can perform our task by the above three steps, it would be difficult to handle so many variables in the program and the program would become very lengthy. The concept of arrays is useful in these types of situations where we can group similar type of data items.

An array is a collection of similar type of data items and each data item is called an element of the array. The data type of the elements may be any valid data type like char, int or float. The elements of array share the same variable name but each element has a different index number known as subscript.

For the above problem we can take an array variable age of size 100 and of type int. The size of this array variable is 100 so it is capable of storing 100 integer values. The individual elements of this array are-

age[0], age[1], age[2], age[3], age[4], ..... age[98], age[99]

In C the subscripts start from zero, so age[0] is the first element, age[1] is the second element of array and so on.

Arrays can be single dimensional or multidimensional. The number of subscripts determines the dimension of array. A one-dimensional array has one subscript, two dimensional array has two subscripts and so on. The one-dimensional arrays are known as vectors and two-dimensional arrays are known as matrices. In this chapter first we will study about single dimensional arrays and then move on to multi dimensional arrays.

## 8.1 One Dimensional Array

### 8.1.1 Declaration of 1-D Array

Like other simple variables, arrays should also be declared before they are used in the program. The syntax for declaration of an array is-

```
data_type array_name[size];
```

Here array\_name denotes the name of the array and it can be any valid C identifier, data\_type is the data type of the elements of array. The size of the array specifies the number of elements that can be stored in the array. It may be a positive integer constant or constant integer expression. Here are some examples of array declarations-

```
int age[100];
float salary[15];
char grade[20];
```

Here age is an array of type int, which can store 100 elements of type int. The array salary is a float type array of size 15, can hold values of type float and third one is a character type array of size 20, can hold characters. The individual elements of the above arrays are-

#### Arrays

```
age[0], age[1], age[2], age[99]
salary[0], salary[1], salary[2], salary[14]
grade[0], grade[1], grade[2], grade[19]
```

When an array is declared, the compiler allocates space in memory sufficient to hold all the elements of the array, so the size of array should be known at the compile time. Hence we can't use variables for specifying the size of array in the declaration. The symbolic constants can be used to specify the size of array. For example-

```
#define SIZE 10
int main(void)

{
 int size = 15;
 float salary[SIZE]; /*Valid*/
 int marks[size]; /*Not valid*/

}
```

The use of symbolic constant to specify the size of array makes it convenient to modify the program if the size of array is to be changed later, because the size has to be changed only at one place, in the #define directive.

### 8.1.2 Accessing 1-D Array Elements

The elements of an array can be accessed by specifying the array name followed by subscript in brackets. In C, the array subscripts start from 0. Hence if there is an array of size 5, the valid subscripts will be from 0 to 4. The last valid subscript is one less than the size of the array. This last valid subscript is known as the upper bound of the array and 0 is known as the lower bound of the array. Let us take an array-

```
int arr[5]; /*Size of array arr is 5, can hold five integer elements*/
```

The elements of this array are-

```
arr[0], arr[1], arr[2], arr[3], arr[4]
```

Here 0 is the lower bound and 4 is the upper bound of the array.

The subscript can be any expression that yields an integer value. It can be any integer constant, integer variable, integer expression or return value(int) from a function call. For example, if i and j are integer variables then these are some valid subscripted array elements-

```
arr[3] arr[i] arr[i+j] arr[2*j] arr[i++]
```

A subscripted array element is treated as any other variable in the program. We can store values in them, print their values or perform any operation that is valid for any simple variable of the same data type. For example if arr and sal are two arrays of sizes 5 and 10 respectively, then these are valid statements-

```
int arr[5];
float sal[10];
int i;
scanf("%d",&arr[1]); /*input value into arr[1]*/
printf("%f",sal[3]); /*print value of sal[3]*/
arr[4]=25; /*assign a value to arr[4]*/
arr[4]++; /*increment the value of arr[4] by 1*/
sal[5] += 200; /*Add 200 to sal[5]*/
sum=arr[0]+arr[1]+arr[2]+arr[3]+arr[4]; /*Add all the values of array arr[5]*/
i++;
scanf("%f",&sal[i]); /*Input value into sal[2]*/
printf("%f",sal[i]); /*Print value of sal[2]*/
printf("%f",sal[i+1]); /*Print value of sal[2] and increment the value of i*/
```

There is no check on bounds of the array. For example, if we have an array arr of size 5, the valid subscripts are only 0, 1, 2, 3, 4 and if someone tries to access elements beyond these subscripts, like arr[5], arr[10], the compiler will not show any error message but this may lead to run time errors. So it is the responsibility of programmer to provide array bounds checking wherever needed.

### 8.1.3 Processing 1-D Arrays

For processing arrays we generally use a for loop and the loop variable is used at the place of subscript. The initial value of loop variable is taken 0 since array subscripts start from zero. The loop variable is increased by 1 each time so that we can access and process the next element in the array. The total number of passes in the loop will be equal to the number of elements in the array and in each pass we will process one element. Suppose arr is an array of type int-

(i) Reading values in arr

```
for(i=0; i<10; i++)
 scanf("%d",&arr[i]);
```

(ii) Displaying values of arr

```
for(i=0; i<10; i++)
 printf("%d ",arr[i]);
```

(iii) Adding all the elements of arr

```
sum=0;
for(i=0; i<10; i++)
 sum+=arr[i];
```

*/\*P8.1 Program to input values into an array and display them\*/*

```
#include<stdio.h>
int main(void)
{
 int arr[5],i;
 for(i=0; i<5; i++)
 {
 printf("Enter a value for arr[%d] : ",i);
 scanf("%d",&arr[i]);
 }
 printf("The array elements are : \n");
 for(i=0; i<5; i++)
 printf("%d\t",arr[i]);
 printf("\n");
 return 0;
}
```

Output:

```
Enter a value for arr[0]: 12
Enter a value for arr[1]: 45
Enter a value for arr[2]: 59
Enter a value for arr[3]: 98
Enter a value for arr[4]: 21
The array elements are:
12 45 59 98 21
```

*/\*P8.2 Program to add elements of an array\*/*

```
#include<stdio.h>
int main(void)
{
 int arr[10],i,sum=0;
 for(i=0; i<10; i++)
 {
 printf("Enter a value for arr[%d] : ",i);
 scanf("%d",&arr[i]);
 sum+=arr[i];
 }
 printf("Sum=%d\n",sum);
 return 0;
}
```

*/\*P8.3 Program to count even and odd numbers in an array\*/*

```
#include<stdio.h>
#define SIZE 10
int main(void)
{
```

```
int arr[SIZE],i,even=0,odd=0;
for(i=0; i<SIZE; i++)
{
 printf("Enter a value for arr[%d] : ",i);
 scanf("%d",&arr[i]);
 if(arr[i]%2 == 0)
 even++;
 else
 odd++;
}
printf("Even numbers=%d, Odd numbers=%d\n",even,odd);
return 0;
```

### 8.1.4 Initialization of 1-D Array

After declaration, the elements of a local array have garbage value while the elements of global and static array are automatically initialized to zero. We can explicitly initialize arrays at the time of declaration. The syntax for initialization of an array is-

```
data_type array_name[size]={value1, value2,.....,valueN};
```

Here array\_name is the name of the array variable, size is the size of the array and value1, value2,.....,valueN are the constant values known as initializers, which are assigned to the array element one after another. These values are separated by commas and there is a semicolon after the ending braces. For example-

```
int marks[5]={50,85,70,65,95};
```

The values of the array elements after this initialization are-

```
marks[0]:50, marks[1]:85, marks[2]:70, marks[3]:65, marks[4]:95
```

While initializing a 1-D array, it is optional to specify the size of the array. If the size is omitted during initialization then the compiler assumes the size of array equal to the number of initializers. For example-

```
int marks[]={99,78,50,45,67,89};
float sal[]={25.5,38.5,24.7};
```

Here the size of array marks is assumed to be 6 and that of sal is assumed to be 3.

If during initialization the number of initializers is less than the size of array, then all the remaining element of array are assigned value zero. For example-

```
int marks[5]={99, 78};
```

Here the size of array is 5 while there are only 2 initializers. After this initialization, the value of the element are-

```
marks[0]:99, marks[1]:78, marks[2]:0, marks[3]:0, marks[4]:0
```

So if we initialize an array like this-

```
int arr[100]={0};
```

then all the elements of arr will be initialized to zero.

If the number of initializers is more than the size given in brackets then compiler will show an error. For example-

```
int arr[5]={1,2,3,4,5,6,7,8}; /*Error*/
```

We can't copy all the elements of an array to another array by simply assigning it to the other array. For example if we have two arrays a and b, then-

```
int a[5]={1,2,3,4,5};
int b[5];
b=a; /*Not valid*/
```

We will have to copy all the elements of array one by one, using a for loop.

```
for(i=0; i<5; i++)
 b[i]=a[i];
```

In the following program we will find out the largest and smallest number in an integer array.

*/\*P8.4 Program to find the largest and smallest number in an array\*/*

```
#include<stdio.h>
int main(void)
{
 int i, arr[10]={2,5,4,1,8,9,11,6,3,7};
 int small, large;
 small=large=arr[0];
 for(i=1; i<10; i++)
 {
 if(arr[i] < small)
 small=arr[i];
 if(arr[i] > large)
 large=arr[i];
 }
 printf("Smallest=%d, Largest=%d\n", small, large);
 return 0;
}
```

We have taken the value of first element as the initial value of small and large. Inside the for loop, we will start comparing from second element onwards so this time we have started the loop from 1 instead of 0.

The following program will reverse the elements of an array.

*/\*P8.5 Program to reverse the elements of an array\*/*

```
#include<stdio.h>
int main(void)
{
 int i, j, temp, arr[10]={1,2,3,4,5,6,7,8,9,10};
 for(i=0, j=9; i<j; i++, j--)
 {
 temp=arr[i];
 arr[i]=arr[j];
 arr[j]=temp;
 }
 printf("After reversing, the array is : ");
 for(i=0; i<10; i++)
 printf("%d ", arr[i]);
 printf("\n");
 return 0;
}
```

In the for loop we have used comma operator and taken two variables i and j. The variable i is initialized with the lower bound and j is initialized with upper bound. After each pass of the loop, i is incremented while j is decremented. Inside the loop, arr[i] is exchanged with arr[j]. So arr[0] will be exchanged with arr[9], arr[1] with arr[8], arr[2] with arr[7] and so on.

The next program prints the binary equivalent of a decimal number. The process of obtaining a binary number from decimal number 29 is given below-

|      | Quotient | Remainder |            |
|------|----------|-----------|------------|
| 29/2 | 14       | 1         | a[0] MSB ↑ |
| 14/2 | 7        | 0         | a[1]       |
| 7/2  | 3        | 1         | a[2]       |
| 3/2  | 1        | 1         | a[3]       |
| 1/2  | 0        | 1         | a[4] LSB   |

We will store the remainders in an array, and then at last the array is printed in reverse order to get the binary number.

*/\*P8.6 Program to convert a decimal number to binary number\*/*

```
#include<stdio.h>
int main(void)
```

```
{
 int num, arr[15], i, j;
 printf("Enter a decimal number : ");
 scanf("%d", &num);
 i=0;
 while(num>0)
 {
 arr[i] = num%2; /*store the remainder in array*/
 num/=2;
 i++;
 }
 printf("Binary number is : ");
 for(j=i-1; j>=0; j--) /*print the array backwards*/
 printf("%d", arr[j]);
 printf("\n");
 return 0;
}
```

Output:

Enter a decimal number : 29

Binary number is : 11101

## 8.1.5 1-D Arrays and Functions

### 8.1.5.1 Passing Individual Array Elements to a Function

We know that an array element is treated as any other simple variable in the program. So like other simple variables, we can pass individual array elements as arguments to a function.

*/\*P8.7 Program to pass array elements to a function\*/*

```
#include<stdio.h>
void check(int num);
int main(void)
{
 int arr[10], i;
 printf("Enter the array elements : ");
 for(i=0; i<10; i++)
 {
 scanf("%d", &arr[i]);
 check(arr[i]);
 }
 return 0;
}

void check(int num)
{
 if(num%2==0)
 printf("%d is even\n", num);
 else
 printf("%d is odd\n", num);
}
```

### 8.1.5.2 Passing whole 1-D Array to a Function

We can pass whole array as an actual argument to a function. The corresponding formal argument should be declared as an array variable of the same data type.

```
int main(void)
{
 int arr[10]
 //...
 func(arr); /*In function call, array name is specified without brackets*/
 return 0;
}
```

```
func(int val[10])
```

It is optional to specify the size of the array in the formal argument, for example we may write the function definition as-

```
func(int val[])
```

We have studied that changes made in formal arguments do not affect the actual arguments, but this is not the case while passing an array to a function. The mechanism of passing an array to a function is quite different from that of passing a simple variable. We know that in the case of simple variables, the called function creates a copy of the variable and works on it, so any changes made in the function do not affect the original variable. When an array is passed as an actual argument, the called function actually gets access to the original array and works on it, so any changes made inside the function affect the original array. Here is a program in which an array is passed to a function.

*/\*P8.8 Program to understand the effect of passing an array to a function\*/*

```
#include<stdio.h>
void func(int val[]);
int main(void)
{
 int i, arr[6]={1,2,3,4,5,6};
 func(arr);
 printf("Contents of array are : ");
 for(i=0; i<6; i++)
 printf("%d ", arr[i]);
 printf("\n");
 return 0;
}

void func(int val[])
{
 int sum=0, i;
 for(i=0; i<6; i++)
 {
 val[i]=val[i]*val[i];
 sum+=val[i];
 }
 printf("Sum of squares=%d\n", sum);
}
```

Output :

Sum of squares=91  
Contents of array are : 1 4 9 16 25 36

Here we can see that the changes made to the array inside the called function are reflected in the calling function. The name of the formal argument is different but it refers to the original array.

Since it is not necessary to specify the size of array in the function definition, we can write general functions that can work on arrays of same type but different sizes. For example in the next program, the function add() is capable of adding the elements of any size of an integer array.

*/\*P8.9 Program that uses a general function which works on arrays of different sizes\*/*

```
#include<stdio.h>
int add(int arr[], int n);
int main(void)
{
 int a[5]={2,4,6,8,10};
 int b[8]={1,3,5,7,9,11,13,15};
 int c[10]={1,2,3,4,5,6,7,8,9,10};
```

```
printf("Sum of elements of array a : %d\n", add(a, 5));
printf("Sum of elements of array b : %d\n", add(b, 8));
printf("Sum of elements of array c : %d\n", add(c, 10));
return 0;
```

```
int add(int arr[], int n)
```

```
{
 int i, sum=0;
 for(i=0; i<n; i++)
 sum+=arr[i];
 return sum;
}
```

Output :

Sum of elements of array a : 30  
Sum of elements of array b : 64  
Sum of elements of array c : 55

## 8.2 Two Dimensional Arrays

### 8.2.1 Declaration and Accessing Individual Elements of a 2-D array

The syntax of declaration of a 2-D array is similar to that of 1-D arrays, but here we have two subscripts.

```
data_type array_name[rowsize][columnsize];
```

Here rowsize specifies the number of rows and columnsize represents the number of columns in the array. The total number of elements in the array are rowsize\*columnsize. For example suppose we have an array arr declared as-

```
int arr[4][5];
```

Here arr is a 2-D array with 4 rows and 5 columns. The individual elements of this array can be accessed by applying two subscripts, where the first subscript denotes the row number and the second subscript denotes the column number. The starting element of this array is arr[0][0] and the last element is arr[3][4]. The total number of elements in this array is 4\*5 = 20.

|       | Col 0     | Col 1     | Col 2     | Col 3     | Col 4     |
|-------|-----------|-----------|-----------|-----------|-----------|
| Row 0 | arr[0][0] | arr[0][1] | arr[0][2] | arr[0][3] | arr[0][4] |
| Row 1 | arr[1][0] | arr[1][1] | arr[1][2] | arr[1][3] | arr[1][4] |
| Row 2 | arr[2][0] | arr[2][1] | arr[2][2] | arr[2][3] | arr[2][4] |
| Row 3 | arr[3][0] | arr[3][1] | arr[3][2] | arr[3][3] | arr[3][4] |

### 8.2.2 Processing 2-D Arrays

For processing 2-D arrays, we use two nested for loops. The outer for loop corresponds to the row and the inner for loop corresponds to the column.

```
int arr[4][5];
```

(i) Reading values in arr

```
for(i=0; i<4; i++)
 for(j=0; j<5; j++)
 scanf("%d", &arr[i][j]);
```

(ii) Displaying values of arr

```
for(i=0; i<4; i++)
 for(j=0; j<5; j++)
 printf("%d ", arr[i][j]);
```

This will print all the elements in the same line. If we want to print the elements of different rows on different lines then we can write like this-

```
for(i=0; i<4; i++)
{
 for(j=0; j<5; j++)
 printf("%d ", arr[i][j]);
```

```
printf("\n");
```

Here the `printf("\n")` statement causes the next row to begin from a new line.

```
/*P8.10 Program to input and display a matrix*/
#define ROW 3
#define COL 4
#include<stdio.h>
int main(void)
{
 int mat[ROW][COL], i, j;
 printf("Enter the elements of the matrix(%dx%d) row-wise :\n", ROW, COL);
 for(i=0; i<ROW; i++)
 for(j=0; j<COL; j++)
 scanf("%d", &mat[i][j]);

 printf("The matrix that you have entered is :\n");
 for(i=0; i<ROW; i++)
 {
 for(j=0; j<COL; j++)
 printf("%5d", mat[i][j]);
 printf("\n");
 }
 printf("\n");
 return 0;
}
```

### 3.2.3 Initialization of 2-D Arrays

2-D arrays can be initialized in a way similar to that of 1-D arrays. For example-

```
int mat[4][3] = {11,12,13,14,15,16,17,18,19,20,21,22};
```

These values are assigned to the elements row-wise, so the values of elements after this initialization are-

|               |               |               |
|---------------|---------------|---------------|
| mat[0][0]: 11 | mat[0][1]: 12 | mat[0][2]: 13 |
| mat[1][0]: 14 | mat[1][1]: 15 | mat[1][2]: 16 |
| mat[2][0]: 17 | mat[2][1]: 18 | mat[2][2]: 19 |
| mat[3][0]: 20 | mat[3][1]: 21 | mat[3][2]: 22 |

While initializing we can group the elements row-wise using inner braces. For example-

```
int mat[4][3] = { {11,12,13}, {14,15,16}, {17,18,19}, {20,21,22} };
int mat[4][3] = {
```

```
 {11,12,13}, /*Row 0*/
 {14,15,16}, /*Row 1*/
 {17,18,19}, /*Row 2*/
 {20,21,22} /*Row 3*/
};
```

Here the values in the first inner braces will be the values of Row 0, values in the second inner braces will be values of Row 1 and so on. Now consider this array initialization-

```
int mat[4][3] = {
 {11}, /*Row 0*/
 {12,13}, /*Row 1*/
 {14,15,16}, /*Row 2*/
 {17} /*Row 3*/
};
```

The remaining elements in each row will be assigned values 0, so the values of elements will be-

|               |               |               |
|---------------|---------------|---------------|
| mat[0][0]: 11 | mat[0][1]: 0  | mat[0][2]: 0  |
| mat[1][0]: 12 | mat[1][1]: 13 | mat[1][2]: 0  |
| mat[2][0]: 14 | mat[2][1]: 15 | mat[2][2]: 16 |
| mat[3][0]: 17 | mat[3][1]: 0  | mat[3][2]: 0  |

In 2-D arrays, it is optional to specify the first dimension while initializing but the second dimension should always be present. For example-

```
int mat[][3] = {
 {1,10},
 {2,20,200},
 {3},
 {4,40,400}
};
```

Here first dimension is taken 4 since there are 4 rows in initialization list.

A 2-D array is also known as a matrix. The next program adds two matrices; the order of both the matrices should be same.

```
/*P8.11 Addition of two matrices*/
#define ROW 3
#define COL 4
#include<stdio.h>
int main(void)
{
 int i, j, mat1[ROW][COL], mat2[ROW][COL], mat3[ROW][COL];
 printf("Enter matrix mat1(%dx%d) row-wise :\n", ROW, COL);
 for(i=0; i<ROW; i++)
 for(j=0; j<COL; j++)
 scanf("%d", &mat1[i][j]);

 printf("Enter matrix mat2(%dx%d) row-wise :\n", ROW, COL);
 for(i=0; i<ROW; i++)
 for(j=0; j<COL; j++)
 scanf("%d", &mat2[i][j]);

 /*Addition*/
 for(i=0; i<ROW; i++)
 for(j=0; j<COL; j++)
 mat3[i][j] = mat1[i][j] + mat2[i][j];

 printf("The resultant matrix mat3 is :\n");
 for(i=0; i<ROW; i++)
 {
 for(j=0; j<COL; j++)
 printf("%5d", mat3[i][j]);
 printf("\n");
 }
 return 0;
}
```

Output:

```
Enter matrix mat1(3x4) row-wise:
1284
5678
3214
Enter matrix mat2(3x4) row-wise:
2542
1526
9472
The resultant matrix mat3 is:
3 7 12 6
6 11 9 14
12 6 8 6
```

Now we will write a program to multiply two matrices. Multiplication of matrices requires that the number of columns in first matrix should be equal to the number of rows in second matrix. Each row of first matrix is multiplied with the column of second matrix and then added to get the element of resultant matrix. If we multiply two matrices of order  $m \times n$  and  $n \times p$  then the multiplied matrix will be of order  $m \times p$ . For example-

$$A_{2 \times 2} = \begin{bmatrix} 4 & 5 \\ 3 & 2 \end{bmatrix} \quad B_{2 \times 3} = \begin{bmatrix} 2 & 6 & 3 \\ -3 & 2 & 4 \end{bmatrix}$$

$$C_{2 \times 3} = \begin{bmatrix} 4*2 + 5*(-3) & 4*6 + 5*2 & 4*3 + 5*4 \\ 3*2 + 2*(-3) & 3*6 + 2*2 & 3*3 + 2*4 \end{bmatrix} = \begin{bmatrix} -7 & 34 & 32 \\ 0 & 22 & 17 \end{bmatrix}$$



```

/*P8.12 Multiplication of two matrices*/
#include<stdio.h>
#define ROW1 3
#define COL1 4
#define ROW2 COL1
#define COL2 2
int main(void)
{
 int mat1[ROW1][COL1], mat2[ROW2][COL2], mat3[ROW1][COL2];
 int i, j, k;
 printf("Enter matrix mat1(%dx%d) row-wise : \n", ROW1, COL1);
 for(i=0; i<ROW1; i++)
 for(j=0; j<COL1; j++)
 scanf("%d", &mat1[i][j]);
 printf("Enter matrix mat2(%dx%d) row-wise : \n", ROW2, COL2);
 for(i=0; i<ROW2; i++)
 for(j=0; j<COL2; j++)
 scanf("%d", &mat2[i][j]);
 /*Multiplication*/
 for(i=0; i<ROW1; i++)
 for(j=0; j<COL2; j++)
 {
 mat3[i][j] = 0;
 for(k=0; k<COL1; k++)
 mat3[i][j] += mat1[i][k] * mat2[k][j];
 }
 printf("The Resultant matrix mat3 is : \n");
 for(i=0; i<ROW1; i++)
 {
 for(j=0; j<COL2; j++)
 printf("%5d", mat3[i][j]);
 printf("\n");
 }
 return 0;
}

```

Output:

```

Enter matrix mat1(3x4) row-wise :
2 1 4 3
5 2 7 1
3 1 4 2
Enter matrix mat2(4x2) row-wise :
1 2
3 4
2 5
6 2
The Resultant matrix mat3 is:
31 34
31 55
26 34

```

The next program finds out the transpose of a matrix. Transpose matrix is defined as the matrix that is obtained by interchanging the rows and columns of a matrix. If a matrix is of  $m \times n$  order then its transpose matrix will be of order  $n \times m$ .

```

/*P8.13 Tranpose of matrix*/
#include<stdio.h>
#define ROW 3
#define COL 4
int main(void)
{
 int mat1[ROW][COL], mat2[COL][ROW], i, j;
 printf("Enter matrix mat1(%dx%d) row-wise : \n", ROW, COL);
 for(i=0; i<ROW; i++)
 for(j=0; j<COL; j++)
 scanf("%d", &mat1[i][j]);

```

```

 for(i=0; i<COL; i++)
 for(j=0; j<ROW; j++)
 mat2[i][j] = mat1[j][i];

 printf("Transpose of matrix is: \n");
 for(i=0; i<COL; i++)
 {
 for(j=0; j<ROW; j++)
 printf("%5d", mat2[i][j]);
 printf("\n");
 }
 return 0;
}

```

Output:

```

Enter matrix mat1(3x4) row-wise :
2 1 4 3
5 2 7 1
3 1 4 2
Transpose of matrix is:
3 6 9
2 5 3
1 8 4
5 2 1

```

### 8.3 Arrays with more than two Dimensions

We will just give a brief overview of three-d arrays. We can think of a three-d array as an array of 2-D arrays. For example if we have an array-

```
int arr[2][4][3];
```

We can think of this as an array which consists of two 2-D arrays where each of those 2-D arrays has 4 rows and 3 columns.

$$[0] \begin{bmatrix} [0][0] & [0][1] & [0][2] \\ [1][0] & [1][1] & [1][2] \\ [2][0] & [2][1] & [2][2] \\ [3][0] & [3][1] & [3][2] \end{bmatrix} \quad [1] \begin{bmatrix} [0][0] & [0][1] & [0][2] \\ [1][0] & [1][1] & [1][2] \\ [2][0] & [2][1] & [2][2] \\ [3][0] & [3][1] & [3][2] \end{bmatrix}$$

The individual elements are-

```

arr[0][0][0], arr[0][0][1], arr[0][0][2], arr[0][1][0], arr[0][3][2]
arr[1][0][0], arr[1][0][1], arr[1][0][2], arr[1][1][0], arr[1][3][2]

```

Total number of elements in the above 3-d array are-

$$= 2 * 4 * 3 \\ = 24$$

This array can be initialized as-

```

arr[2][4][3] = {
 {
 {1, 2, 3}, /*Matrix 0, Row 0*/
 {4, 5}, /*Matrix 0, Row 1*/
 {6, 7, 8}, /*Matrix 0, Row 2*/
 {9}, /*Matrix 0, Row 3*/
 },
 {
 {10, 11}, /*Matrix 1, Row 0*/
 {12, 13, 14}, /*Matrix 1, Row 1*/
 {15, 16}, /*Matrix 1, Row 2*/
 {17, 18, 19} /*Matrix 1, Row 3*/
 }
};

```

The value of elements after this initialization are-

```
arr[0][0][0]: 1 arr[0][0][1]: 2 arr[0][0][2]: 3
arr[0][1][0]: 4 arr[0][1][1]: 5 arr[0][1][2]: 0
arr[0][2][0]: 6 arr[0][2][1]: 7 arr[0][2][2]: 8
arr[0][3][0]: 9 arr[0][3][1]: 0 arr[0][3][2]: 0

arr[1][0][0]: 10 arr[1][0][1]: 11 arr[1][0][2]: 0
arr[1][1][0]: 12 arr[1][1][1]: 13 arr[1][1][2]: 14
arr[1][2][0]: 15 arr[1][2][1]: 16 arr[1][2][2]: 0
arr[1][3][0]: 17 arr[1][3][1]: 18 arr[1][3][2]: 19
```

Remember that the rule of initialization of multidimensional arrays is that the last subscript varies most frequently and the first subscript varies least frequently.

### 8.3.1 Multidimensional Array and Functions

Multidimensional arrays can also be passed to functions like 1-D arrays. When passing multidimensional arrays, the first(leftmost) dimension may be omitted but all other dimensions have to be specified in the function definition. For example it would be invalid to write a function like this-

```
func(int a[],int b[][],int c[][][]) /*Invalid*/
```

```
{

}
```

In arrays b and c we can't omit all the dimensions. The correct form is-

```
func(int a[],int b[][4],int c[][3][5]) /*Valid*/
```

```
{

}
```

## 8.4 Introduction to Strings

We'll discuss strings in detail in a separate chapter; here is just a brief introduction to strings. In C, strings are treated as arrays of type char and are terminated by a null character('\0'). This null character has ASCII value zero.

These are the two forms of initialization of a string variable-

```
char str[10]={'t','n','d','i','a','\0'};
char str[10]="India"; /*Here the null character is automatically placed at the end*/
```

### 8.4.1 Input and output of strings

*/\*P8.14, Input and output of strings using scanf() and printf()\*/*

```
#include<stdio.h>
int main(void)
{
 char str[10]="Anpara";
 printf("String is : %s\n",str);
 printf("Enter new value for string : ");
 scanf("%s",str);
 printf("String is : %s\n",str);
 return 0;
}
```

Output :

```
String is : Anpara
Enter new value for string : Bareilly
String is : Bareilly
```

The next program uses the functions gets() and puts() for the input and output of strings.

*/\*P8.15 Program for input and output of strings using gets() and puts()\*/*  
#include<stdio.h>

```
int main(void)
```

```
char str[10];
printf("Enter a string : ");
gets(str);
printf("String is : ");
puts(str);
return 0;
```

Output :

```
Enter a string : New Delhi
String is : New Delhi
```

## Some Additional Problems

### Problem 1

Write a program using arrays, that reads a decimal number and converts it to (1) Binary (2) Octal or (3) Hexadecimal depending on user's choice.

*/\*P8.16 Program to convert a decimal number to Binary, octal or hexadecimal\*/*

```
#include<stdio.h>
void func(int num,int b);
int main(void)
{
 int num,ch;
 printf("Enter a decimal number : ");
 scanf("%d",&num);
 printf("1.Binary\n2.Octal\n3.Hexadecimal\n");
 printf("Enter your choice : ");
 scanf("%d",&ch);
 switch(ch)
 {
 case 1:
 printf("Binary equivalent is : ");
 func(num,2);
 break;
 case 2:
 printf("Octal equivalent is : ");
 func(num,8);
 break;
 case 3:
 printf("Hexadecimal equivalent is : ");
 func(num,16);
 break;
 }
 printf("\n");
 return 0;
}
```

```
void func(int num,int b)
```

```
{
 int i=0,j,rem;
 char arr[20];
 while(num>0)
 {
 rem=num%b;
 num/=b;
 if(rem>9 && rem<16)
 arr[i++] = rem-10+'A';
 else
 arr[i++] = rem+'0';
 }
 for(j=i-1; j>=0; j--)
```

```
printf("%c",arr[j]);
```

## Problem 2

Write a program to search for an element in an array using linear search.

Sequential search or linear search is performed in a linear way i.e. it starts from the beginning of the list and continues till we find the item or reach the end of list. The item to be searched is compared with each element of the list one by one, starting from the first element. For example, consider the array in figure 8.1, suppose we want to search for value 12 in this array. This value is compared with `arr[0]`, `arr[1]`, `arr[2]`, ..... `arr[10]`, `arr[11]`. Since the value is found at index 11, the search is successful.

| 0  | 1  | 2  | 3 | 4  | 5  | 6 | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
|----|----|----|---|----|----|---|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 96 | 19 | 85 | 9 | 16 | 29 | 2 | 36 | 41 | 67 | 53 | 12 | 66 | 6  | 75 | 82 | 89 | 23 | 93 | 45 |

Figure 8.1

Now suppose we have to find value 56 in the above array. This value is compared with `arr[0]`, `arr[1]`, `arr[2]`, ..... `arr[19]`. The value was not found even after examining all the elements of the array so the search is unsuccessful. The program for linear search is-

```
/*P8.17 Linear search in an array*/
#include <stdio.h>
#define MAX 50
int LinearSearch(int arr[],int n,int item);
int main(void)
{
 int i,n,item,arr[MAX],index;
 printf("Enter the number of elements : ");
 scanf("%d",&n);
 printf("Enter the elements : \n");
 for(i=0; i<n; i++)
 scanf("%d",&arr[i]);
 printf("Enter the item to be searched : ");
 scanf("%d",&item);
 index=LinearSearch(arr,n,item);
 if(index!=-1)
 printf("%d not found in array\n",item);
 else
 printf("%d found at position %d\n",item,index);
 return 0;
}

int LinearSearch(int arr[],int n,int item)
{
 int i=0;
 while(i<n && item!=arr[i])
 i++;
 if(i<n)
 return i;
 else
 return -1;
}
```

The function `LinearSearch()` returns location of the item if found, or -1 otherwise. The while loop can terminate in two cases, first is when value of `i` becomes equal to `n`; this is the case when whole array has been scanned and item is not found in the array. The second case which makes the while loop to terminate is when the value of item is found in the array i.e. `item==arr[i]`.

In the while loop of the `LinearSearch()` we are doing two comparisons in each iteration, the comparison `i<n` can be avoided by using a sentinel value. In the last location of the array i.e. `arr[n]`, we can temporarily

place the value of item that is being searched. This value terminates the search when the search item is present in the array.

| 0  | 1  | 2  | 3 | 4  | 5  | 6 | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|----|----|----|---|----|----|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 96 | 19 | 85 | 9 | 16 | 29 | 2 | 36 | 41 | 67 | 53 | 12 | 66 | 6  | 75 | 82 | 89 | 23 | 93 | 45 | 12 |

| 0  | 1  | 2  | 3 | 4  | 5  | 6 | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|----|----|----|---|----|----|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 96 | 19 | 85 | 9 | 16 | 29 | 2 | 36 | 41 | 67 | 53 | 12 | 66 | 6  | 75 | 82 | 89 | 23 | 93 | 45 | 56 |

The function for linear search using sentinel is-

```
int LinearSearch(int arr[],int n,int item)
{
 int i=0;
 while(item!=arr[i])
 i++;
 if(i<n)
 return i;
 else
 return -1;
}
```

The number of comparisons in an unsuccessful search can be reduced if the array is sorted. In this case we need not search for the item till the end of the list, we can terminate our search as soon as we find an element that is greater or equal to the search item (in ascending array). If the element is equal to the search item, the search is successful otherwise it is unsuccessful. The function for linear search in an ascending order array is-

```
int LinearSearch(int arr[],int n,int item)
{
 int i=0;
 while(i<n && arr[i]<item)
 i++;
 if(arr[i]==item)
 return i;
 else
 return -1;
}
```

## Problem 3

Write a program for searching an element in a sorted array through binary search.

The prerequisite for binary search is that the array should be sorted. Firstly we compare the item to be searched with the middle element of the array. If the item is found there, our search finishes successfully otherwise the array is divided into two halves, first half contains all elements to the left of the middle element and the other one consists of all the elements to the right side of the middle element. Since the array is sorted, all the elements in the left half will be smaller than the middle element and the elements in the right half will be greater than the middle element. If the item to be searched is less than the middle element, it is searched in left half otherwise it is searched in the right half.

Now search proceeds in the smaller portion of the array (subarray) and the item is compared with its middle element. If the item is same as the middle element, search finishes otherwise again the subarray is divided into two halves and the search is performed in one of these halves. This process of comparing the item with the middle element and dividing the array continues till we find the required item or get a portion which does not have any element.

To implement this procedure we will take 3 variables viz. `low`, `up` and `mid` that will keep track of the status of lower limit, upper limit and middle value of that portion of the array, in which we will search the element. If the array contains even number of elements, there will be two middle elements, we'll take first one as the middle element. The value of the `mid` can be calculated as-

```
mid = (low+up)/2;
```

The middle element of the array would be `arr[mid]`, the left half of the array would be `arr[low]....arr[mid-1]` and the right half of the array would be `arr[mid+1]....arr[up]`.

The item is compared with the mid element, if it is not found then the value of low or up is updated for selecting the left or right half. When low becomes greater than up, the search is unsuccessful as there is no portion left in which to search.

If `item > arr[mid]`  
 Search will resume in right half which is `arr[mid+1] ..... arr[up]`  
`low = mid + 1`, up remains same

If `item < arr[mid]`  
 Search will resume in left half which is `arr[low] ..... arr[mid-1]`  
`up = mid - 1`, low remains same

If `item == arr[mid]`, search is successful  
 Item found at mid position

If `low > up`, search is unsuccessful  
 Item not present in array

Let us take a sorted array of 20 elements and search for elements 41, 62, and 63 in this array one by one. The portion of array in which the element is searched is shown with a bold boundary in the figure.

Search 41

`low = 0, up = 19, mid = (0+19)/2 = 9`

|   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 6 | 9 | 15 | 19 | 23 | 29 | 32 | 36 | 41 | 53 | 62 | 66 | 72 | 75 | 82 | 89 | 90 | 93 | 96 |    |

41 < 47  
 Search in left half  
`up = mid - 1 = 8`

`low = 0, up = 8, mid = (0+8)/2 = 4`

|   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 6 | 9 | 15 | 19 | 23 | 29 | 32 | 36 | 41 | 47 | 53 | 62 | 66 | 72 | 75 | 82 | 89 | 90 | 93 | 96 |

41 > 23  
 Search in right half  
`low = mid + 1 = 5`

`low = 5, up = 8, mid = (5+8)/2 = 6`

|   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 6 | 9 | 15 | 19 | 23 | 29 | 32 | 36 | 41 | 47 | 53 | 62 | 66 | 72 | 75 | 82 | 89 | 90 | 93 | 96 |

41 > 32  
 Search in right half  
`low = mid + 1 = 7`

`low = 7, up = 8, mid = (7+8)/2 = 7`

|   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 6 | 9 | 15 | 19 | 23 | 29 | 32 | 36 | 41 | 47 | 53 | 62 | 66 | 72 | 75 | 82 | 89 | 90 | 93 | 96 |

41 > 36  
 Search in right half  
`low = mid + 1 = 8`

`low = 8, up = 8, mid = (8+8)/2 = 8`

|   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 6 | 9 | 15 | 19 | 23 | 29 | 32 | 36 | 41 | 47 | 53 | 62 | 66 | 72 | 75 | 82 | 89 | 90 | 93 | 96 |

41 found

Search 62

`low = 0, up = 19, mid = (0+19)/2 = 9`

|   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 6 | 9 | 15 | 19 | 23 | 29 | 32 | 36 | 41 | 53 | 62 | 66 | 72 | 75 | 82 | 89 | 90 | 93 | 96 |    |

62 > 47  
 Search in right half  
`low = mid + 1 = 10`

`low = 10, up = 19, mid = (10+19)/2 = 14`

|   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 6 | 9 | 15 | 19 | 23 | 29 | 32 | 36 | 41 | 47 | 53 | 62 | 66 | 72 | 75 | 82 | 89 | 90 | 93 | 96 |

62 < 75  
 Search in left half  
`up = mid - 1 = 13`

`low = 10, up = 13, mid = (10+13)/2 = 11`

|   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 6 | 9 | 15 | 19 | 23 | 29 | 32 | 36 | 41 | 47 | 53 | 62 | 66 | 72 | 75 | 82 | 89 | 90 | 93 | 96 |

62 found

Search 63

`low = 0, up = 19, mid = (0+19)/2 = 9`

|   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 6 | 9 | 15 | 19 | 23 | 29 | 32 | 36 | 41 | 53 | 62 | 66 | 72 | 75 | 82 | 89 | 90 | 93 | 96 |    |

63 > 47  
 Search in right half  
`low = mid + 1 = 10`

`low = 10, up = 19, mid = (10+19)/2 = 14`

|   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 6 | 9 | 15 | 19 | 23 | 29 | 32 | 36 | 41 | 47 | 53 | 62 | 66 | 72 | 75 | 82 | 89 | 90 | 93 | 96 |

63 < 75  
 Search in left half  
`up = mid - 1 = 13`

`low = 10, up = 13, mid = (10+13)/2 = 11`

|   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 6 | 9 | 15 | 19 | 23 | 29 | 32 | 36 | 41 | 47 | 53 | 62 | 66 | 72 | 75 | 82 | 89 | 90 | 93 | 96 |

63 > 62  
 Search in right half  
`low = mid + 1 = 12`

`low = 12, up = 13, mid = (12+13)/2 = 12`

|   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0 | 1 | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 6 | 9 | 15 | 19 | 23 | 29 | 32 | 36 | 41 | 47 | 53 | 62 | 66 | 72 | 75 | 82 | 89 | 90 | 93 | 96 |

63 < 66  
 Search in left half  
`up = mid - 1 = 11`

Now `low = 12` and `up = 11`, the value of low has exceeded the value of up so the search is unsuccessful. The program for binary search is-

```

/*P8.18 Binary search in an array*/
#include <stdio.h>
#define MAX 50
int BinarySearch(int arr[], int size, int item);
int main(void)
{
 int i, size, item, arr[MAX], index;
 printf("Enter the number of elements : ");
 scanf("%d", &size);

 printf("Enter the elements (in sorted order) : \n");
 for(i=0; i<size; i++)
 scanf("%d", &arr[i]);

 printf("Enter the item to be searched : ");
 scanf("%d", &item);
 index = BinarySearch(arr, size, item);

 if(index == -1)
 printf("%d not found in array\n", item);
 else
 printf("%d found at position %d\n", item, index);
 return 0;
}

int BinarySearch(int arr[], int size, int item)
{
 int low=0, up=size-1, mid;
 while(low<=up)
 {
 mid = (low+up)/2;
 if(item > arr[mid])
 low = mid+1; /*Search in right half*/
 else if(item < arr[mid])
 up = mid-1; /*Search in left half*/
 else
 return mid;
 }
 return -1;
}

```

The function `BinarySearch()` returns the location of the item if found, or -1 otherwise.

Binary search is more efficient than linear search. For an array of about 1000000 elements, the maximum number of comparisons required to find any element would be only 20.

## Problem 4

Write a procedure which finds the smallest element of the array and swaps it with the 0<sup>th</sup> element of the array.

```
/*P8.19*/
#include <stdio.h>
#define MAX 100
int main(void)
{
 int arr[MAX], i, j, n, temp, min;
 printf("Enter the number of elements : ");
 scanf("%d", &n);
 for(i=0; i<n; i++)
 {
 printf("Enter element %d : ", i+1);
 scanf("%d", &arr[i]);
 }
 /*Find the index of smallest element*/
 min=0;
 for(j=1; j<n; j++)
 {
 if(arr[min]>arr[j])
 min=j;
 }
 if(min!=0)
 {
 temp=arr[0];
 arr[0]=arr[min];
 arr[min]=temp;
 }
 for(i=0; i<n; i++)
 printf("%d ", arr[i]);
 printf("\n");
 return 0;
}
```

## Problem 5

Write a program to sort the elements of a 1-D array, in ascending order through selection sort.

Sorting is a procedure in which the given elements are arranged in ascending or descending order. For example if the elements of an array are-

5, 11, 15, 8, 7, 54, 63, 44

After sorting these elements in ascending order the elements would be-

5, 7, 8, 11, 15, 44, 54, 63

There are several methods for sorting an array, we'll discuss only three of them(selection, bubble, insertion). These are discussed in Problems 5, 7 and 10 respectively.

Suppose that you are given some numbers and asked to arrange them in ascending order. The most intuitive way to do this would be to find the smallest number and put it in the first place and then find the second smallest number and put it in the second place and so on. This is the simple technique on which selection sort is based. It is named so because in each pass it selects the smallest element and keeps it in its exact place.

Suppose we have  $n$  elements stored in an array  $arr$ . First we will search the smallest element from  $arr[0]$ ..... $arr[n-1]$  and exchange it with  $arr[0]$ . This will place the smallest element of list at 0<sup>th</sup> position of array. Now we will search smallest element from remaining elements  $arr[1]$ ..... $arr[n-1]$  and exchange it with  $arr[1]$ . This will place the second smallest element of the list at 1<sup>st</sup> position of the array. This process continues till the whole array is sorted. The whole process is as-

Pass 1 :

1. Search the smallest element from  $arr[0]$ ..... $arr[n-1]$ .

C in depth Arrays

2. Exchange this element with  $arr[0]$ .

Result :  $arr[0]$  is sorted.

Pass 2 :

1. Search the smallest element from  $arr[1]$ ..... $arr[n-1]$ .

2. Exchange this element with  $arr[1]$ .

Result :  $arr[0]$ ,  $arr[1]$  are sorted.

Pass  $n-1$  :

1. Search the smallest element from  $arr[n-2]$  and  $arr[n-1]$ .

2. Exchange this element with  $arr[n-2]$ .

Result :  $arr[0]$ ,  $arr[1]$ ,..... $arr[n-2]$  are sorted.

Now all the elements except the last one have been put in their proper place. The remaining last element  $arr[n-1]$  will definitely be the largest of all and so it is automatically at its proper place. Thus we need only  $n-1$  passes to sort the array. Let us take a list of elements in unsorted order and sort it by applying selection sort.

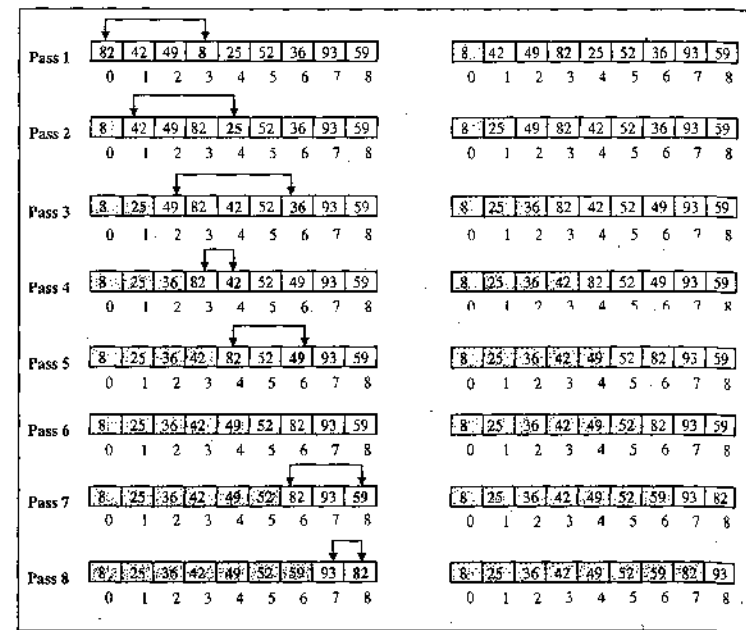


Figure 8.2 Selection Sort

In the first pass, 8 is the smallest element among  $arr[0]$ ..... $arr[8]$ , so it is exchanged with  $arr[0]$  i.e. 82. In the second pass, 25 is the smallest among  $arr[1]$ ..... $arr[8]$  so it is exchanged with  $arr[1]$  i.e. 42. The other passes proceed in a similar way. The shaded portion shows the elements that have been put in their final place.

/\*P8.20 Selection sort\*/

```
#include <stdio.h>
#define MAX 100
int main(void)
{
 int arr[MAX], i, j, n, temp, min;
 printf("Enter the number of elements : ");
 scanf("%d", &n);
```

```

for(i=0; i<n; i++)
{
 printf("Enter element %d : ", i+1);
 scanf("%d", &arr[i]);
}
/*Selection sort*/
for(i=0; i<n-1; i++)
{
 /*Find the index of smallest element*/
 min=i;
 for(j=i+1; j<n; j++)
 {
 if(arr[min]>arr[j])
 min=j;
 }
 if(i!=min)
 {
 temp=arr[i];
 arr[i]=arr[min];
 arr[min]=temp;
 }
 printf("Sorted list is : \n");
 for(i=0; i<n; i++)
 printf("%d ", arr[i]);
 printf("\n");
 return 0;
}

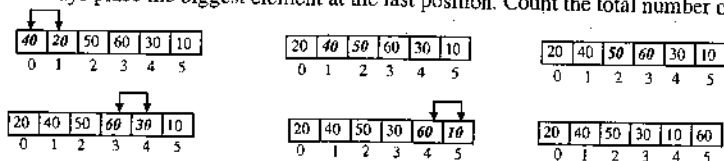
```

Each iteration of outer for loop corresponds to a single pass. In each iteration of outer for loop we have to compare  $arr[i]$  with the smallest element among  $arr[i+1] \dots arr[n-1]$ . The inner for loop is used to find the index of the smallest element and it is stored in  $min$ . Initially variable  $min$  is initialized with  $i$ . After this,  $arr[min]$  is compared with each of the elements  $arr[i+1], arr[i+2] \dots arr[n-1]$  and whenever we get a smaller element, its index is assigned to  $min$ .

After finding the smallest element, it is exchanged with  $arr[i]$ . We have preceded this swap operation with a condition to avoid swapping of an element with itself. This situation arises when an element is already in its proper place. In the pass 6 of figure 8.2,  $arr[5]$  has to be swapped with  $arr[5]$  which is obviously redundant.

#### Problem 6

Write a program that scans a one-dimensional array from left to right and compares all adjacent elements. Any two adjacent elements  $arr[j]$  and  $arr[j+1]$  should be exchanged if  $arr[j]$  is greater than  $arr[j+1]$ . This procedure will always place the biggest element at the last position. Count the total number of exchanges done.



```

21*/
#include <stdio.h>
#define MAX 100
main(void)
{
 int arr[MAX], i, j, temp, n, exchanges;
 printf("Enter the number of elements : ");
 scanf("%d", &n);
 for(i=0; i<n; i++)
 {

```

```

 printf("Enter element %d : ", i+1);
 scanf("%d", &arr[i]);
 }
 exchanges=0;
 for(j=0; j<n-1; j++)
 {
 if(arr[j] > arr[j+1])
 {
 temp = arr[j];
 arr[j] = arr[j+1];
 arr[j+1] = temp;
 exchanges++;
 }
 }
 for(i=0; i<n; i++)
 printf("%d ", arr[i]);
 printf("\nTotal exchanges = %d", exchanges);
 return 0;
}

```

#### Problem 7

Write a program to sort the elements of a 1-D array, in ascending order through bubble sort.

Bubble sort proceeds by scanning the list and exchanging the adjacent elements if they are out of order with respect to each other. It compares each element with its adjacent element and swaps them if they are not in order i.e.  $arr[j]$  will be compared with  $arr[j+1]$  and if  $arr[j] > arr[j+1]$  then they will be swapped.

In selection sort we searched for the smallest element and then performed the swap, while here swap will be performed as soon as we find two adjacent elements out of order. So in selection sort there was only one swap in a pass while in bubble sort there may be many swaps in a single pass.

After first pass the largest element will be at its proper position in the array i.e.  $(n-1)^{th}$  position, after second pass the second largest element will be placed at its proper position i.e.  $(n-2)^{th}$  position. Similarly after each pass the next larger elements will be moved to the end of the list and placed at their proper positions. If there are  $n$  elements, then only  $n-1$  passes are required to sort the array and the procedure is as-

##### Pass 1 :

Compare  $arr[0]$  and  $arr[1]$ . If  $arr[0] > arr[1]$  then exchange them,  
Compare  $arr[1]$  and  $arr[2]$ . If  $arr[1] > arr[2]$  then exchange them,  
Compare  $arr[2]$  and  $arr[3]$ . If  $arr[2] > arr[3]$  then exchange them,

Compare  $arr[n-2]$  and  $arr[n-1]$ . If  $arr[n-2] > arr[n-1]$  then exchange them.  
Result : Largest element is placed at  $(n-1)^{th}$  position  
 $arr[n-1]$  is sorted.

##### Pass 2 :

Compare  $arr[0]$  and  $arr[1]$ . If  $arr[0] > arr[1]$  then exchange them,  
Compare  $arr[1]$  and  $arr[2]$ . If  $arr[1] > arr[2]$  then exchange them,  
Compare  $arr[2]$  and  $arr[3]$ . If  $arr[2] > arr[3]$  then exchange them,

Compare  $arr[n-3]$  and  $arr[n-2]$ . If  $arr[n-3] > arr[n-2]$  then exchange them.  
Result : Second largest element is placed at  $(n-2)^{th}$  position  
 $arr[n-2], arr[n-1]$  are sorted.

##### Pass n-2 :

Compare  $arr[0]$  and  $arr[1]$ . If  $arr[0] > arr[1]$  then exchange them,  
Compare  $arr[1]$  and  $arr[2]$ . If  $arr[1] > arr[2]$  then exchange them.

Result : arr[2],.....arr[n-2], arr[n-1] are sorted.

Pass n-1 :

Compare arr[0] and arr[1]. If arr[0] > arr[1] then exchange them.

Result : arr[1], arr[2],.....arr[n-2], arr[n-1] are sorted.

In the second pass, comparisons will be done only up to (n-2)<sup>th</sup> position i.e. the last comparison is done between arr[n-3] and arr[n-2], because the largest element has already been placed at its proper position i.e. position (n-1). In the third pass, the last comparison is done between arr[n-4] and arr[n-3], because the second largest element has already been placed at its proper position i.e. position (n-2). In the last pass there is only one comparison which is in between arr[0] and arr[1]. The remaining element arr[0] will definitely be the smallest element, so the whole list is sorted after n-1 passes.

Let us take an array in unsorted order and sort it by applying bubble sort.

40 20 50 60 30 10

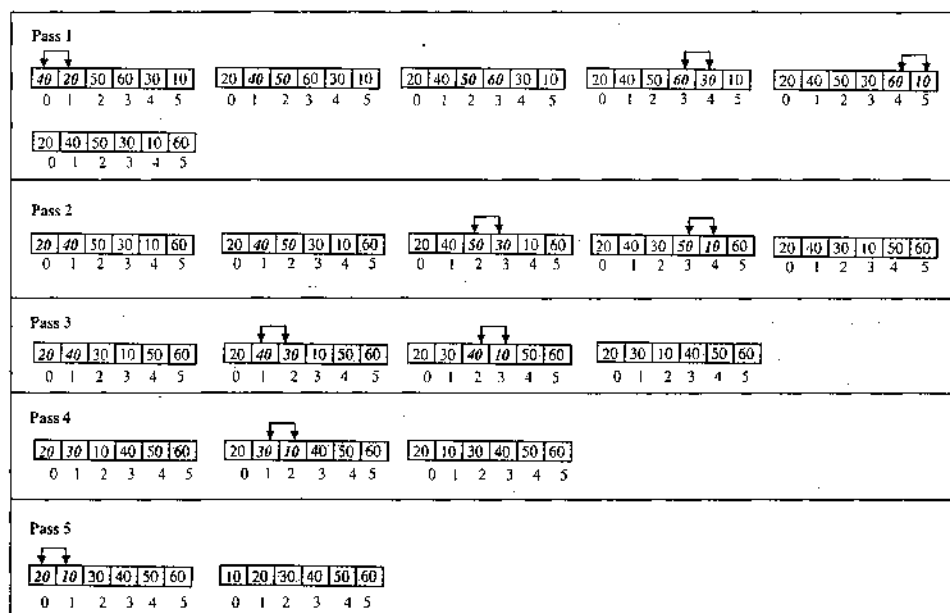


Figure 8.3 Bubble sort

Let us see what happens in the first pass. First arr[0] is compared with arr[1], since 40 > 20 they are swapped. Now arr[1] is compared with arr[2], since 40 < 50 they are not swapped. Now arr[2] is compared with arr[3], since 50 < 60 they are not swapped. Now arr[3] is compared with arr[4], since 50 > 30 they are swapped. Now arr[4] is compared with arr[5], since 60 > 10 they are swapped. At the end of this pass the largest element 60 is placed at the last position. The elements which are being compared are shown in italics and the elements which have been placed in proper place are shaded.

Sometimes it is possible that a list of n elements becomes sorted in less than n-1 passes. For example consider this list - 40 20 10 30 60 50

after first pass the list is - 20 10 30 40 50 60

after second pass the list is - 10 20 30 40 50 60

The list of 6 elements becomes sorted in only 2 passes. Hence other passes are unnecessary and there is no need to proceed further. Now the question is how we will be able to know that the list has become sorted. If no swaps occur in a pass it means that the list is sorted. For example in the above case there will be no swaps in the

third pass. We can take a variable that keeps record of the number of swaps in a pass and if no swaps occur then we can terminate our procedure.

```

/*P8.22 Bubble sort*/
#include <stdio.h>
#define MAX 100
int main(void)
{
 int arr[MAX], i, j, temp, n, xchanges;
 printf("Enter the number of elements : ");
 scanf("%d", &n);
 for(i=0; i<n; i++)
 {
 printf("Enter element %d : ", i+1);
 scanf("%d", &arr[i]);
 }
 /*Bubble sort*/
 for(i=0; i<n-1; i++)
 {
 xchanges = 0;
 for(j=0; j<n-1-i; j++)
 {
 if(arr[j] > arr[j+1])
 {
 temp = arr[j];
 arr[j] = arr[j+1];
 arr[j+1] = temp;
 xchanges++;
 }
 }
 if(xchanges==0) /*If list is sorted*/
 break;
 }
 printf("Sorted list is :\n");
 for(i=0; i<n; i++)
 printf("%d ", arr[i]);
 printf("\n");
 return 0;
}
/*End of main()*/

```

The figure 8.4 shows the procedure of bubble sort along with the values of i and j. Each iteration of outer for loop corresponds to a single pass.

| Pass 1 |     |     |     |     |    | Pass 2 |     |     |     | Pass 3 |     |     | Pass 4 |     | Pass 5 |
|--------|-----|-----|-----|-----|----|--------|-----|-----|-----|--------|-----|-----|--------|-----|--------|
| i=0    |     |     |     |     |    | i=1    |     |     |     | i=2    |     |     | i=3    |     | i=4    |
| j=0    | j=1 | j=2 | j=3 | j=4 |    | j=0    | j=1 | j=2 | j=3 | j=0    | j=1 | j=2 | j=0    | j=1 | j=0    |
| 0      | 40  | 20  | 20  | 20  | 20 | 20     | 20  | 20  | 20  | 20     | 20  | 20  | 20     | 20  | 20     |
| 1      | 20  | 40  | 40  | 40  | 40 | 40     | 40  | 40  | 40  | 40     | 40  | 40  | 40     | 40  | 40     |
| 2      | 50  | 50  | 50  | 50  | 50 | 50     | 50  | 50  | 50  | 50     | 50  | 50  | 50     | 50  | 50     |
| 3      | 60  | 60  | 60  | 60  | 30 | 30     | 30  | 30  | 50  | 10     | 10  | 10  | 40     | 40  | 40     |
| 4      | 30  | 30  | 30  | 30  | 60 | 10     | 10  | 10  | 10  | 50     | 50  | 50  | 50     | 50  | 50     |
| 5      | 10  | 10  | 10  | 10  | 10 | 60     | 60  | 60  | 60  | 60     | 60  | 60  | 60     | 60  | 60     |
| Ex     |     |     |     |     |    | Ex     |     |     |     | Ex     |     |     | Ex     |     | Ex     |

Figure 8.4 Bubble Sort

### Problem 8

Write a program that inserts an element in an array at a specified index by moving elements to the right.

Insert 90 at index 2

|    |    |    |    |    |   |
|----|----|----|----|----|---|
| 20 | 40 | 50 | 60 | 30 |   |
| 0  | 1  | 2  | 3  | 4  | 5 |

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 20 | 40 | 90 | 50 | 60 | 30 |
| 0  | 1  | 2  | 3  | 4  | 5  |

**\*P8.23 Program to insert an item in an array at a specified index by moving other elements to the right\***

```

#include<stdio.h>
#define SIZE 10
int main(void)
{
 int arr[SIZE];
 int i, item, index;
 printf("Enter elements of the array : \n");
 for(i=0; i<SIZE-1; i++) /*rightmost space in the array should be empty*/
 scanf("%d", &arr[i]);
 printf("Enter the item to be inserted : ");
 scanf("%d", &item);
 printf("Enter the index where item is to be inserted : ");
 scanf("%d", &index);

 for(i=SIZE-2; i>=index; i--)
 arr[i+1]=arr[i]; /*Shift elements to the right*/
 arr[i+1]=item; /*Insert item at the proper place*/

 for(i=0; i<SIZE; i++)
 printf("%d ", arr[i]);
 printf("\n");
 return 0;
}

```

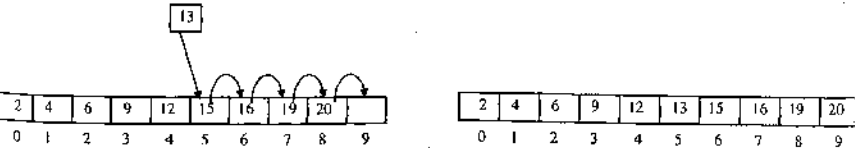
**Problem 9**

Write a program to insert an element in a sorted array at proper place, so that the array remains sorted after insertion also.

Suppose we have an array of size 10 and there are nine elements in it which are in ascending order, one of the space is empty.

|   |   |   |   |    |    |    |    |    |   |
|---|---|---|---|----|----|----|----|----|---|
| 2 | 4 | 6 | 9 | 12 | 15 | 16 | 19 | 20 |   |
| 0 | 1 | 2 | 3 | 4  | 5  | 6  | 7  | 8  | 9 |

To insert an item, we'll compare it with the elements of array from the right side and keep on shifting them to the right. As soon as we get an element less than the item we'll stop this process and insert the item. Suppose we have to insert the number 13 in the array-



|   |   |   |   |    |    |    |    |    |    |
|---|---|---|---|----|----|----|----|----|----|
| 2 | 4 | 6 | 9 | 12 | 13 | 15 | 16 | 19 | 20 |
| 0 | 1 | 2 | 3 | 4  | 5  | 6  | 7  | 8  | 9  |

**\*P8.24 Program to insert an item in a sorted array at the proper place by shifting other elements to the right\***

```

#include<stdio.h>
#define SIZE 10
int main(void)
{
 int arr[SIZE];
 int i, item;
 printf("Enter elements of the array(in sorted order) : \n");
 for(i=0; i<SIZE-1; i++) /*rightmost space in the array should be empty*/
 scanf("%d", &arr[i]);
 printf("Enter the item to be inserted : ");
 scanf("%d", &item);
 for(i=SIZE-2; item<arr[i] && i>=0; i--)
 arr[i+1]=arr[i]; /*Shift elements to the right*/
}

```

```

arr[i+1]=item; /*Insert item at the proper place*/

for(i=0; i<SIZE; i++)
 printf("%d ", arr[i]);
printf("\n");
return 0;

```

The condition  $i \geq 0$  in the loop is there because if the item is to be inserted at 0<sup>th</sup> place, then value of  $i$  will become  $-1$ , but a  $[-1]$  is not a valid element.

### Problem 10

Write a program to sort the elements of a 1-D array, in ascending order through insertion sort.

To sort an array through insertion sort we'll use the insertion procedure described in the previous program. Here we will place each element of array at proper place in the previously sorted element list.

The insertion sort proceeds by inserting each element at the proper place in a sorted list. This is the same technique used by card players for arranging cards. When they receive a card, they place it in the appropriate place among the cards that they have already arranged.

We will consider our list to be divided into two parts - sorted and unsorted. Initially the sorted part contains only the first element of the list and unsorted part contains the rest of the elements. In each pass, the first element from the unsorted part is taken and inserted into the sorted part at appropriate place. If there are  $n$  elements in the list, then after  $n-1$  passes the unsorted part disappears and our whole list becomes sorted. The process of inserting each element in proper place is-

#### Pass 1 :

Sorted part : arr[0]

Unsorted part : arr[1], arr[2], arr[3], ..... , arr[n-1]

arr[1] is inserted before or after arr[0].

Result : arr[0] and arr[1] are sorted.

#### Pass 2 :

Sorted part : arr[0], arr[1]

Unsorted part : arr[2], arr[3], ..... , arr[n-1]

arr[2] is inserted before arr[0], or in between arr[0] and arr[1] or after arr[1].

Result : arr[0], arr[1] and arr[2] are sorted.

#### Pass 3 :

Sorted part : arr[0], arr[1], arr[2]

Unsorted part : arr[3], arr[4], ..... , arr[n-1]

arr[3] is inserted at its proper place among arr[0], arr[1], arr[2]

Result : arr[0], arr[1], arr[2], arr[3] are sorted.

#### Pass n-1 :

Sorted part : arr[0], arr[1], ..... , arr[n-2]

Unsorted part : arr[n-1]

arr[n-1] is inserted at its proper place among arr[0], arr[1], ..... , arr[n-2]

Result : arr[0], arr[1], arr[3], ..... , arr[n-1] are sorted.

To insert an element in the sorted part we need a vacant position, and this space is created by moving all the larger elements one position to the right. Now let us take a list of elements in unsorted order and sort them by applying insertion sort.

82 42 49 8 25 52 36 93 59

The shaded portion in figure 8.5 shows the sorted part of the list and the arrows show the movement of elements. To insert an element we have to scan the sorted part and search for the appropriate place where this



element can be inserted. We will use sequential search for this purpose. We also need a vacant position for inserting this element, and for this all the larger elements are moved one position right. For example in pass 6 the elements 42, 49, 52, 82 are moved one position right to make space for insertion of 36.

If we start our searching from the end of the sorted part instead of its beginning then we can simultaneously shift the elements also.

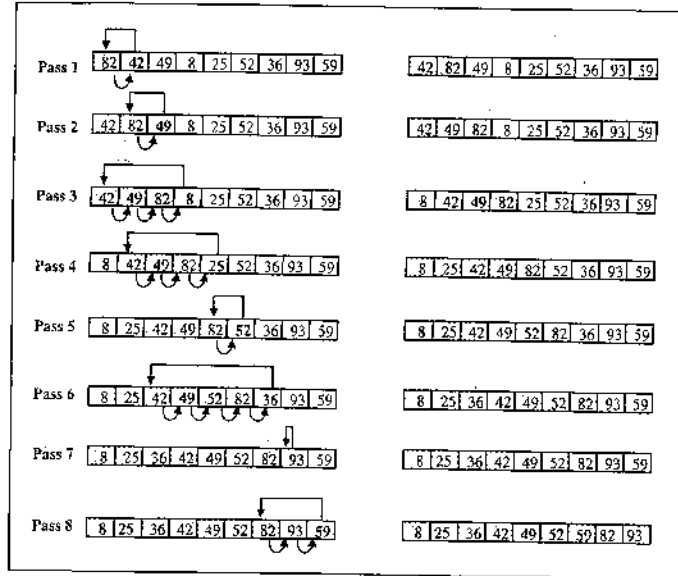


Figure 8.5 Insertion Sort

/\*P8.25 Program of sorting using insertion sort\*/

```
#include<stdio.h>
#define MAX 100
int main(void)
{
 int arr[MAX], i, j, k, n;
 printf("Enter the number of elements : ");
 scanf("%d", &n);
 for(i=0; i<n; i++)
 {
 printf("Enter element %d : ", i+1);
 scanf("%d", &arr[i]);
 }
 /*Insertion sort*/
 for(i=1; i<n; i++)
 {
 k=arr[i]; /*k is to be inserted at proper place*/
 for(j=i-1; j>=0 && k<arr[j]; j--)
 arr[j+1]=arr[j];
 arr[j+1]=k;
 }
 printf("Sorted list is :\n");
 for(i=0; i<n; i++)
 printf("%d ", arr[i]);
 printf("\n");
 return 0;
}
/*End of main()*/
```

In each iteration of for loop, first element of the unsorted part is inserted into the sorted part. The element to be inserted (arr[i]) is stored in the variable k. In the inner for loop, the sorted part is scanned to find the exact location for the insertion of the element arr[i]. The search starts from the end of the sorted part so variable j is initialized to i-1. The search stops when we either reach the beginning of the sorted part or we get an element less than k. Inside the inner for loop, the elements are moved right one position, and obviously these are elements which are greater than k. At the end k is inserted at its proper place.

### Problem 11

Write a program for merging two sorted arrays into a third sorted array

If there are two sorted arrays, then process of combining these sorted arrays into another sorted array is called merging. A simple approach could be to place the second array after the first and then sort the whole by applying any sorting technique, but by doing this we are not utilizing the fact that both the arrays are sorted. Since both arrays are sorted, we can merge them efficiently by making only one pass through each array. Let us take two arrays arr1 and arr2 in sorted order, we will combine them into a third sorted array arr3.

arr1- 5 8 9 28 34 arr2- 4 22 25 30 33 40 42

We will take one element from each array, compare them and then take the smaller one in third array. This process will continue until the elements of one array are finished. Then the remaining elements of unfinished array are put in the third array. The whole process for merging is shown in figure 8.5 arr3 is the merged array, i, j and k are variables used for subscripts of arr1, arr2 and arr3 respectively.

Initially i, j and k point to the beginning of the three arrays so i=0, j=0, k=0. The elements arr1[i] and arr2[j] are compared, and the smaller one is copied to the third array arr3 at position k. The variable k is incremented and one of the variables i or j is incremented.

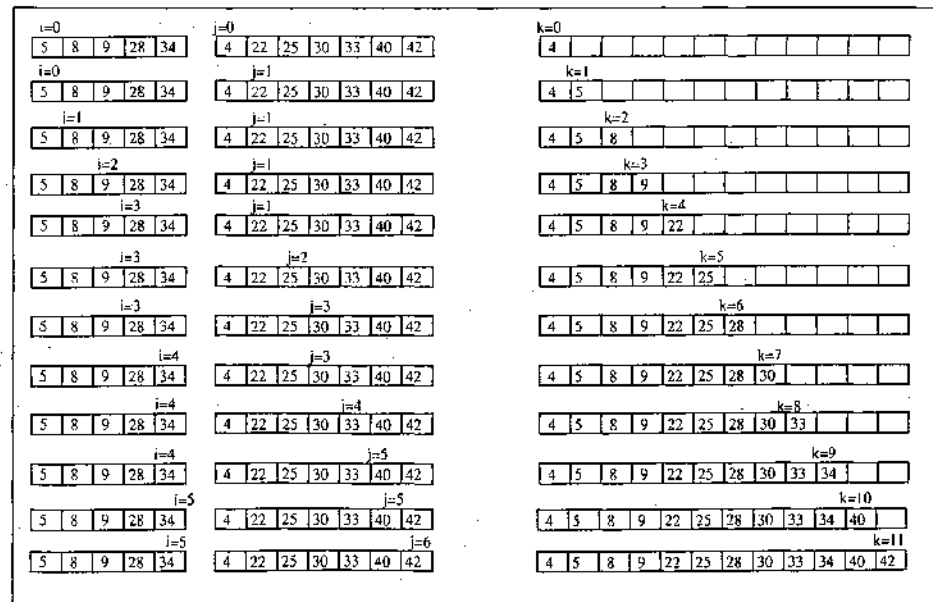


Figure 8.5 Merging

/\*P8.26 Program of merging two sorted arrays into a third sorted array\*/

```
#include<stdio.h>
```

```
#define MAX 100
void merge(int arr1[], int arr2[], int arr3[], int n1, int n2);
int main(void)
{
 int arr1[MAX], arr2[MAX], arr3[2*MAX], n1, n2, i;
 printf("Enter the number of elements in array 1 : ");
 scanf("%d", &n1);
 printf("Enter all the elements in sorted order :\n");
 for(i=0; i<n1; i++)
 {
 printf("Enter element %d : ", i+1);
 scanf("%d", &arr1[i]);
 }
 printf("Enter the number of elements in array 2 : ");
 scanf("%d", &n2);
 printf("Enter all the elements in sorted order :\n");
 for(i=0; i<n2; i++)
 {
 printf("Enter element %d : ", i+1);
 scanf("%d", &arr2[i]);
 }
 merge(arr1, arr2, arr3, n1, n2);
 printf("\nMerged list : ");
 for(i=0; i<n1+n2; i++)
 printf("%d ", arr3[i]);
 printf("\n");
 return 0;
}

void merge(int arr1[], int arr2[], int arr3[], int n1, int n2)
{
 int i, j, k;
 i=0; /*Index for first array*/
 j=0; /*Index for second array*/
 k=0; /*Index for merged array*/
 while((i<n1-1)&&(j<n2-1))
 {
 if(arr1[i]<arr2[j])
 arr3[k++] = arr1[i++];
 else
 arr3[k++] = arr2[j++];
 }
 while(i<n1-1) /*Put remaining elements of arr1 into arr3*/
 arr3[k++] = arr1[i++];
 while(j<n2-1) /*Put remaining elements of arr2 into arr3*/
 arr3[k++] = arr2[j++];
}

/*End of merge()*/
```

## Problem 12

Write a program to print the Pascal triangle using a 2-D array.

```
1
2 1
3 3 1
4 6 4 1
5 10 10 5 1
6 15 20 15 6 1
```

We'll calculate and store all the elements of Pascal triangle in a 2-D array. From the above figure we can observe that-

- All the elements of Column 0 are 1.
- All the elements for which Row and Column are same, are 1.
- Any other element can be obtained by adding two elements of previous row as-  
 $a[i][j] = a[i-1][j-1] + a[i-1][j]$ ;  
 where  $i$  and  $j$  represent row and column number.

### 8.27 Pascal's triangle\*/

```
#include<stdio.h>
#define MAX 15
main(void)
{
 int a[MAX][MAX];
 int i, j, n;
 printf("Enter n : ");
 scanf("%d", &n);
 for(i=0; i<n; i++)
 {
 for(j=0; j<=i; j++)
 if(j==0 || i==j)
 a[i][j]=1;
 else
 a[i][j]=a[i-1][j-1]+a[i-1][j];
 }
 for(i=0; i<n; i++)
 {
 for(j=0; j<=i; j++)
 printf("%5d", a[i][j]);
 printf("\n");
 }
 return 0;
}
```

## Problem 13

Write a program to print the magic matrix.

- Magic matrix is a square matrix of order  $n \times n$ , i.e. number of rows is equal to number of columns.
  - A magic matrix exists only for odd values of  $n$ .
  - The numbers in matrix will be  $1, 2, 3, 4, \dots, n^2$  and each number can occur in the matrix only once.
  - The sums of elements of every row, column, and diagonal are equal. This sum is always equal to  $n(n^2+1)/2$ .
- The magic matrices for  $n = 3$ ,  $n = 5$ ,  $n = 7$  are shown below-

|   |   |   |
|---|---|---|
| 2 | 9 | 4 |
| 7 | 5 | 3 |
| 6 | 1 | 8 |

$n=3$   
Sum = 15

|    |    |    |    |    |
|----|----|----|----|----|
| 9  | 2  | 25 | 18 | 11 |
| 3  | 21 | 19 | 12 | 10 |
| 22 | 20 | 13 | 6  | 4  |
| 16 | 14 | 7  | 5  | 23 |
| 15 | 8  | 1  | 24 | 17 |

$n=5$   
Sum = 65

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| 20 | 11 | 2  | 49 | 40 | 31 | 22 |
| 12 | 3  | 43 | 41 | 32 | 23 | 21 |
| 4  | 44 | 42 | 33 | 24 | 15 | 13 |
| 45 | 36 | 34 | 25 | 16 | 14 | 5  |
| 37 | 35 | 26 | 17 | 8  | 6  | 46 |
| 29 | 27 | 18 | 9  | 7  | 47 | 38 |
| 28 | 19 | 10 | 1  | 48 | 39 | 30 |

$n=7$   
Sum = 175

The procedure for creating a magic matrix is-

Start filling numbers from the centre column of bottom row, so initially place the number 1 in the centre column of bottom row. Keep on placing the numbers by moving one row down and one column left (down-left) till you reach one of the following situations-

- If you reach at the bottom left of the matrix, or a square that is already filled then move one row up in the same column. This situation will arise when previously placed number was divisible by  $n$ .



```

 printf("\n");
 for(i=0; i<n; i++)
 {
 for(j=0; j<n; j++)
 printf("%4d", a[i][j]);
 printf("\n");
 }
 printf("\n\n");
 for(start=0, end=n-1; start<=end; start++, end--)
 {
 for(i=start; i<=end; i++)
 printf("%d ", a[start][i]);
 for(i=start+1; i<=end; i++)
 printf("%d ", a[i][end]);
 for(i=end-1; i>=start; i--)
 printf("%d ", a[end][i]);
 for(i=end-1; i>=start+1; i--)
 printf("%d ", a[i][start]);
 }
 return 0;

```

## Exercise

```
int main(void)
```

```

{
 int i, size=5, arr[size];
 for(i=0; i<size; i++)
 scanf("%d", &arr[i]);
 for(i=0; i<size; i++)
 printf("%d ", arr[i]);
 return 0;
}

```

```
2. int main(void)
```

```

{
 int arr[4]={2,4,8,16}, i=3, j=0;
 while(i)
 {
 j+=arr[i];
 i--;
 }
 printf("j=%d\n", j);
 return 0;
}

```

```
int main(void)
```

```

{
 int i=0, j=0, arr[6]={4,2,6,0,5,10};
 while(arr[i])
 {
 j+=arr[i];
 i++;
 }
 printf("j=%d\n", j);
 return 0;
}

```

```
4. void func(int arr[]);
```

```

int main(void)
{
 int arr[5]={5,10,15,20,25};
 func(arr);
 return 0;
}

void func(int arr[])
{
 int i=5, sum=0;
 while(i>2)
 sum=sum+arr[--i];
 printf("sum=%d\n", sum);
}

```

```
int main(void)
```

```

{
 int x[10], y[3][4], z[2][3][5];
 printf("%u\t%u\t%u\n", sizeof(x), sizeof(y), sizeof(z));
 return 0;
}

```

```

void swapvar(int a, int b);
void swaparr(int arr1[5], int arr2[5]);
int main(void)

```

```

{
 int a=4, b=6;
 int arr1[5]={1,2,3,4,5};
 int arr2[5]={6,7,8,9,10};
 swapvar(a, b);
 swaparr(arr1, arr2);
 printf("a=%d, b=%d\n", a, b);
 printf("arr1[0]=%d, arr1[4]=%d\n", arr1[0], arr1[4]);
 printf("arr2[0]=%d, arr2[4]=%d\n", arr2[0], arr2[4]);
 return 0;
}

```

```
void swapvar(int a, int b)
```

```

{
 int temp;
 temp=a, a=b, b=temp;
}

```

```
void swaparr(int arr1[5], int arr2[5])
```

```

{
 int i, temp;
 for(i=0; i<5; i++)
 { temp=arr1[i], arr1[i]=arr2[i], arr2[i]=temp; }
}

```

```
7. int main(void)
```

```

{
 int i, j, arr[3][4]={ {1,2,3,4}, {5,6,7,8}, {9,10,11,12} };
 for(i=0; i<4; i++)
 {
 for(j=0; j<3; j++)
 printf("%3d", arr[j][i]);
 printf("\n");
 }
 return 0;
}

```

```
8. int main(void)
```

```

{
 int a[10]={2,-3,4,-5,6,7,1,9,-10,-11};
 int j, x, k=0;
 for(i=0; i<10; i++)
 {
 x=a[k];
 if(x<0)
 {
 for(j=k; j<10; j++)
 a[j]=a[j+1];
 a[9]=x;
 }
 else
 k++;
 }
 for(i=0; i<10; i++)
 printf("%d ", a[i]);
 return 0;
}

```

```
9. #define N 10
```

```

int main(void)
{
 int i, a[N];
 a[0]=a[1]=1;
 for(i=2; i<N; i++)
 a[i]=a[i-1]+a[i-2];
 for(i=0; i<N; i++)
 printf("%d ", a[i]);
 return 0;
}

```

```
10. #define N 6
 int main(void)
 {
 int i, j, a[N] = {1, 2, 3, 4, 5, 6};
 for(i=0; i<N; i++)
 for(j=0; j<i; j++)
 a[i] += a[j];
 for(i=0; i<N; i++)
 printf("%d ", a[i]);
 return 0;
 }
```

```
12. #define N 10
 int main(void)
 {
 int i, j, a[N] = {1};
 for(i=0; i<N; i++)
 for(j=0; j<i; j++)
 a[i] += a[j];
 for(i=0; i<N; i++)
 printf("%d ", a[i]);
 return 0;
 }
```

```
14. void func(int arr[], int i, int j);
 int main(void)
 {
 int arr[10] = {1, 6, 2, 7, 3, 4, 8, 10, 9, 5};
 func(arr, 2, 7);
 return 0;
 }

 void func(int arr[], int i, int j)
 {
 if(i > j)
 return;
 func(arr, i+1, j);
 printf("%d ", arr[i]);
 }
```

```
16. int func(int arr[], int size);
 int main(void)
 {
 int arr[10] = {2, 3, 1, 4, 6, 34};
 printf("%d\n", func(arr, 6));
 return 0;
 }

 int func(int arr[], int size)
 {
 int m;
 if(size == 1)
 return arr[0];
 m = func(arr, size-1);
 if(arr[size-1] < m)
 return arr[size-1];
 else
 return m;
 }
```

```
11. #define N 6
 int main(void)
 {
 int i, j, a[N] = {1, 2, 3, 4, 5, 6};
 for(i=0; i<N; i++)
 for(j=i+1; j<N; j++)
 a[i] += a[j];
 for(i=0; i<N; i++)
 printf("%d ", a[i]);
 return 0;
 }
```

```
13. int main(void)
 {
 int arr[10] = {1, 2, 3, 4, 8, 10};
 printf("%d\n", func(arr, 5));
 return 0;
 }

 int func(int arr[], int size)
 {
 if(size == 0)
 return 0;
 else if(arr[size-1] % 2 == 0)
 return 1 + func(arr, size-1);
 else
 return func(arr, size-1);
 }
```

```
15. int main(void)
 {
 int arr[10] = {1, 2, 3, 4, 8, 10};
 printf("%d\n", func(arr, 6));
 return 0;
 }

 int func(int arr[], int size)
 {
 if(size == 0)
 return 0;
 return arr[size-1] + func(arr, size-1);
 }
```

```
17. int func(int arr[], int low, int high);
 int main(void)
 {
 int arr[10] = {3, 4, 2, 11, 8, 10};
 printf("%d\n", func(arr, 0, 5));
 return 0;
 }

 int func(int arr[], int low, int high)
 {
 int mid, left, right;
 if(low == high)
 return arr[low];
 mid = (low+high)/2;
 left = func(arr, low, mid);
 right = func(arr, mid+1, high);
 if(left < right)
 return left;
 else
 return right;
 }
```

18. Write a program to exchange adjacent elements of an array.

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 20 | 40 | 50 | 60 | 30 | 80 |
| 0  | 1  | 2  | 3  | 4  | 5  |

→

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 40 | 20 | 60 | 50 | 80 | 30 |
| 0  | 1  | 2  | 3  | 4  | 5  |

19. Write a program to find the difference of the largest and smallest elements of an array.

20. Write a program to find the largest and second largest elements of an array.

21. Write a function to reverse only first n elements of an array.

22. Write a program to sort an array and display its contents in ascending order and descending order.

23. Write a program to sort numbers of a 1-d array in descending order using

(i) selection sort (ii) bubble sort (iii) insertion sort

24. Write a program to check whether all the elements of an array are distinct.

25. Write a program to remove duplicate elements from an unsorted array.

26. Write a program to remove duplicate elements from a sorted array.

27. Two elements  $a[i]$  and  $a[j]$  of an array form an inversion if  $a[i] > a[j]$  and  $i < j$ . For example if the array is 4, 9, 2, 8, 7 there are 5 inversions formed by the following pairs of elements - (4, 2), (9, 2), (9, 8), (9, 7), (8, 7). The number of inversions is minimum (zero) if an array is sorted in ascending order and is maximum  $(size * (size-1)/2)$  if array is sorted in descending order. Write a program to count the number of inversions in an array.

28. Write a program to find the most frequent element in a sorted array.

29. An element in an array is called a leader if it is greater than all elements to the right of it in that array. The last element is always a leader. For example, in the array {2, 4, 9, 6, 3, 5, 2}, there are 4 leaders which are 9, 6, 5 and 2. If an array is in ascending order, it has only one leader and if it is in descending order then each element is a leader. Write a program to find leaders in an array.

30. Write functions FirstOcc() and LastOcc() that find the first and last occurrence of an element in a sorted array using binary search.

31. Write a program to sort the elements of a matrix (i) Row-wise (ii) Column-wise

|         |   |                     |         |   |                         |
|---------|---|---------------------|---------|---|-------------------------|
| 5 8 2 1 | → | 1 2 5 8             | 5 8 2 1 | → | 1 3 2 1                 |
| 3 6 9 4 |   | 3 4 6 9             | 3 6 9 4 |   | 3 6 2 4                 |
| 1 7 2 8 |   | 1 2 7 8             | 1 7 2 8 |   | 5 7 5 7                 |
| 8 3 5 7 |   | 3 5 7 8             | 8 3 5 7 |   | 8 8 9 8                 |
| Matrix  |   | (i) Sorted Row-wise | Matrix  |   | (ii) Sorted Column-wise |

32. Write a program to find an element in a matrix, where the matrix is sorted row-wise and column-wise.

33. Write a program to interchange first and last rows of a matrix.

34. Write a program to reverse the rows of a matrix.

|             |   |             |
|-------------|---|-------------|
| 1 2 3 4     | → | 13 14 15 16 |
| 5 6 7 8     |   | 9 10 11 12  |
| 9 10 11 12  |   | 5 6 7 8     |
| 13 14 15 16 |   | 1 2 3 4     |

35. Write a program to interchange first and last columns of a matrix.

36. Write a program to reverse the columns of a matrix

|             |   |             |
|-------------|---|-------------|
| 1 2 3 4     | → | 4 3 2 1     |
| 5 6 7 8     |   | 8 7 6 5     |
| 9 10 11 12  |   | 12 11 10 9  |
| 13 14 15 16 |   | 16 15 14 13 |

37. Write a program to find out whether a square matrix is symmetric or not. A square matrix is symmetric if transpose of the matrix is equal to the matrix.

38. Write a recursive function to find the sum of all even numbers in an array.

39. Write a recursive function that finds the sum of all elements of an array by repeatedly partitioning it into two almost equal parts.

40. Write a recursive function to reverse the elements of an array.

41. Write a recursive function to find whether the elements of an array are in strict ascending order or not.

42. Write a program to find the sum of rows and columns of a 2-d array and store the sums in the same array.

|   |   |   |   |   |    |    |    |   |    |    |
|---|---|---|---|---|----|----|----|---|----|----|
| 1 | 2 | 2 | 1 | 4 | 1  | 2  | 2  | 1 | 4  | 10 |
| 5 | 4 | 3 | 2 | 5 | 5  | 4  | 3  | 2 | 5  | 19 |
| 6 | 3 | 2 | 1 | 4 | 6  | 3  | 2  | 1 | 4  | 16 |
| 3 | 5 | 4 | 2 | 3 | 3  | 5  | 4  | 2 | 3  | 17 |
|   |   |   |   |   | 15 | 14 | 11 | 6 | 16 | 62 |

Write a program to print a rectangular matrix spirally.

Write a program to print Spiral Matrix. A spiral matrix is  $n \times n$  square matrix formed by placing the numbers 1, 2, 3, 4, ...,  $n^2$  in spiral form starting from the leftmost column and topmost row. Spiral matrices can exist for both even and odd values of  $n$ . The spiral matrices for  $n=3$ ,  $n=4$ ,  $n=7$  are shown below-

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 | 9 | 4 |
| 7 | 6 | 5 |

n=3

|    |    |    |   |
|----|----|----|---|
| 1  | 2  | 3  | 4 |
| 12 | 13 | 14 | 5 |
| 11 | 16 | 15 | 6 |
| 10 | 9  | 8  | 7 |

n=4

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| 24 | 25 | 26 | 27 | 28 | 29 | 8  |
| 23 | 40 | 41 | 42 | 43 | 30 | 9  |
| 22 | 39 | 48 | 49 | 44 | 31 | 10 |
| 21 | 38 | 47 | 46 | 45 | 32 | 11 |
| 20 | 37 | 36 | 35 | 34 | 33 | 12 |
| 19 | 18 | 17 | 16 | 15 | 14 | 13 |

n=7

Write a program to partition an array such that all the negative numbers are on the left side of array and positive numbers on the right side.

Write a program to create an array next\_ge for an unsorted array arr containing  $n$  elements such that next\_ge[i] = next greater element of arr[i] in array arr i.e. first greater element on the right of arr[i], -1 if no such element exists.

Example-

arr: 3 1 6 2 5 8 9 4 7  
next\_ge: 6 6 8 5 8 9 -1 7 -1

Write a program to find the  $k$ th smallest element in an array.

Write a program to reverse a portion of an array.

Write a program to modify the elements of an array such that the first element becomes the last element of the array and other elements are shifted towards left.

3 4 5 6 7 8 9 → 2 3 4 5 6 7 8 9 1

In the previous problem, we have rotated the array to the left by one element. Now modify the previous program so that can rotate the array by any number of elements. For example when we rotate the array by 3 elements the result would be-  
3 4 5 6 7 8 9 → 4 5 6 7 8 9 1 2 3

C is a very powerful language and the real power of C lies in pointers. The use of pointers makes the code more efficient and compact. Some of the uses of pointers are-

- (i) Simulating call by reference
- (ii) Returning more than one value from a function.
- (iii) Accessing dynamically allocated memory
- (iv) Implementing data structures like linked lists, trees and graphs.
- (v) Improving efficiency

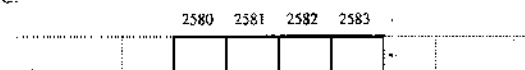
## 9.1 About Memory

Before studying pointers it is important to understand how memory is organized in a computer. The memory in a computer is made up of bytes arranged in a sequential manner. Each byte is identified by an index number, which is called the address of that byte. The address of these bytes start from zero and the address of last byte is one less than the size of memory.

We have studied that it is necessary to declare a variable before using it, since compiler has to reserve space for it. The data type of the variable also has to be mentioned so that the compiler knows how much space needs to be reserved. Now let us see what happens when we declare a variable. Suppose we declare a variable age of type int-

```
int age;
```

The compiler reserves 4 consecutive bytes from memory for this variable and associates the name age with it. The address of leftmost byte(rightmost on some machines) from the 4 allocated bytes is known as the address of variable age.



Suppose compiler has reserved bytes numbered 2580, 2581, 2582 and 2583 for the storage of variable age, the address of variable age will be 2580. Let us assign some value to this variable-

```
age=20;
```

Now this value will be stored in these 4 bytes in binary form. The number of bytes allocated depends on the size of data type of variable. For example 8 bytes would have been allocated for a double variable. Now we will see how to find out the address of a variable.

## 9.2 Address Operator

C provides an address operator '&', which returns the address of a variable when placed before it. This operator can be read as "the address of", so &age means address of variable age, similarly &salary means address of variable salary. The following program prints the address of variables using address operator.

```
/*P9.1 Program to print address of variables using address operator*/
```

```
#include<stdio.h>
int main(void)
{
```

```
 int age=30;
 float salary=1500.50;
 printf("Address of age=%p\n", &age);
 printf("Address of salary=%p\n", &salary);
 return 0;
}
```

Output:  
Address of age=0012FED4  
Address of salary=0012FEC8

The format specifier %p is used to print addresses. These addresses may be different each time you run your program, it depends on which part of memory is allocated by operating system for this program.

The address operator cannot be used with constants, expressions or register variables.

```
&j; /*Valid, used with a variable*/
&arr[1]; /*Valid, used with an array element*/
&289; /*Invalid, used with a constant*/
&(j+k); /*Invalid, used with an expression*/
&rvar; /*Invalid if rvar is a register variable*/
```

This address operator is not new for us, we have already used it in scanf() function. The address of variable is provided to scanf(), so that it knows where to write the input value. So now you can understand why '&' was placed before the variable names in scanf().

## 9.3 Pointer Variables

Finally after this brief introduction, it is time to introduce pointers. A pointer is a variable that stores memory address. Like all other variables it also has a name, has to be declared and occupies some space in memory. It is called pointer because it points to a particular location in memory by storing the address of that location.

### 9.3.1 Declaration of Pointer Variables

Like other variables, pointer variables should also be declared before being used. The general syntax of declaration is-

```
data_type *pname;
```

Here pname is the name of pointer variable, which should be a valid C identifier. The asterisk '\*' preceding this name informs the compiler that the variable is declared as a pointer. Here data\_type is known as the base type of pointer. Let us take some pointer declarations-

```
int *iptr;
float *fptr;
char *cptr, ch1, ch2;
```

Here iptr is a pointer that should point to variables of type int, similarly fptr and cptr should point to variables of float and char type respectively. Here type of variable iptr is 'pointer to int' or (int \*), or we can say that base type of iptr is int. We can also combine the declaration of simple variables and pointer variables as we have done in the third declaration statement where ch1 and ch2 are declared as variables of type char.

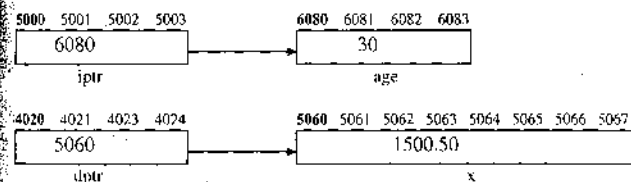
Pointers are variables so compiler will reserve space for them and they will also have some address. All pointers irrespective of their base type generally occupy same space in memory. Now the question arises that when all the pointers contain addresses only and each one occupies same space, then why do we have to mention the data type in the declaration statement. We will come to know about this when we study about indirection operator and pointer arithmetic.

### 9.3.2 Assigning Address to Pointer Variables

Global and local static pointers are automatically initialized to NULL but when we declare an automatic pointer variable it contains garbage value i.e. it may be pointing anywhere in the memory. So we should always assign an address before using it in the program. The use of an unassigned pointer may give unpredictable results and even cause the program to crash. Pointers can be assigned the address of a variable using assignment statement. For example-

```
int *iptr, age=30;
double *dptr, x=1500.50;
iptr=&age;
dptr=&x;
```

Now iptr contains the address of variable age i.e. it points to variable age, similarly dptr points to variable x. Since iptr is declared as a pointer of type int, we should assign address of only integer variables to it. If we assign address of some other data type then compiler will not show any error but the results will be incorrect.



Here the value of pointer iptr is 6080 which is the address of variable age, and the value of pointer dptr is 5060 which is the address of variable x.

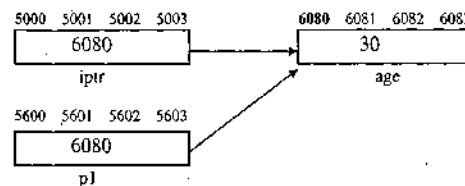
We can initialize the pointer at the time of declaration, but in this case the variable should be declared before the pointer. For example-

```
int age=30, *iptr=&age;
double x=1500.50, *dptr=&x;
```

It is also possible to assign the value of one pointer variable to the other, provided their base type is same. For example if we have an integer pointer p1 then we can assign the value of iptr to it as-

```
p1=iptr;
```

Now both pointer variables iptr and p1 contain the address of variable age and point to the same variable age.



We can assign constant zero or NULL to a pointer of any type. NULL is a symbolic constant defined in stdio.h, stddef.h and many other header files. The assignment of NULL to a pointer guarantees that it does not point to any valid memory location. This can be done as-

```
ptr=NULL;
```

### 9.3.4 Dereferencing Pointer Variables

Till now we have done a lot of programming and in all our programs we have used the name of a variable for accessing it. We can also access a variable indirectly using pointers. For this we will use the indirection opera-

tor (\*). By placing the indirection operator before a pointer variable, we can access the variable whose address is stored in the pointer. Let us take some examples-

```
int a=87;
float b=4.5;
int *p1=&a;
float *p2=&b;
```

In our program, if we place '\*' before p1 then we can access the variable whose address is stored in p1. Since p1 contains the address of variable a, we can access the variable a by writing \*p1. Similarly we can access variable b by writing \*p2. So we can use \*p1 and \*p2 in place of variable names a and b anywhere in our program. Let us see some examples-

|                            |                  |                        |
|----------------------------|------------------|------------------------|
| *p1=9;                     | is equivalent to | a=9;                   |
| (*p1)++;                   | is equivalent to | a++;                   |
| x = *p2+10;                | is equivalent to | x = b+10;              |
| printf("%d %f", *p1, *p2); | is equivalent to | printf("%d %f", a, b); |
| scanf("%d%f", p1, p2);     | is equivalent to | scanf("%d%f", &a, &b); |

The indirection operator can be read as 'value at the address'. For example \*p1 can be read as 'value at the address p1'. This indirection operator (\*) is different from the asterisk that was used while declaring the pointer variable.

Now let us see what is meant by the term \*(&age), where age is a variable. Since &age is an address, dereferencing it with \* operator will give the variable at that address and the variable at that address is age. Hence \*(&age) is same as writing age.

*/\*P9.2 Dereferencing pointer variables\*/*

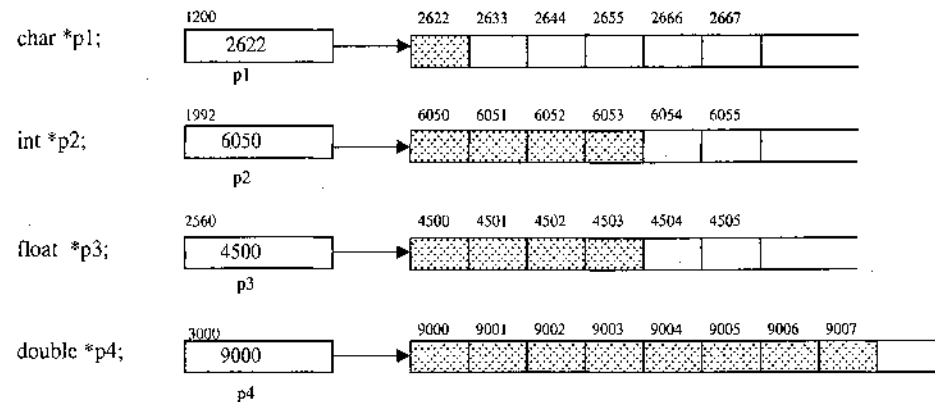
```
#include<stdio.h>
int main(void)
{
 int a=87;
 float b=4.5;
 int *p1=&a;
 float *p2=&b;
 printf("Value of p1 = Address of a = %p\n", p1);
 printf("Value of p2 = Address of b = %p\n", p2);
 printf("Address of p1 = %p\n", &p1);
 printf("Address of p2 = %p\n", &p2);
 printf("Value of a = %d %d %d\n", a, *p1, *(&a));
 printf("Value of b = %.1f %.1f %.1f\n", b, *p2, *(&b));
 return 0;
}
```

Output:

```
/value of p1 = Address of a = 0012FED4
/value of p2 = Address of b = 0012FEC8
/address of p1 = 0012FEB8
/address of p2 = 0012FEB0
/value of a = 87 87 87
/value of b = 4.5 4.5 4.5
```

If a pointer that contains NULL is dereferenced, the results are implementation dependent.

We have seen that while declaring a pointer variable, we have to mention the data type. The reason is that when we use the indirection operator, the number of bytes retrieved will be different for different data types. The value of the pointer only tells the address of starting byte. For example suppose we have a pointer ptr which contains the address 2000 and when we write \*ptr the compiler knows that it has to access the information starting at address 2000. So the compiler will look at the base type of the pointer and will retrieve the information depending on that base type. For example if base type is int then 4 bytes information will be retrieved and if base type is double, 8 bytes information will be retrieved and so on. The following figures illustrate this fact. The shaded portion shows the number of bytes retrieved.



The size of pointer variable is same for all type of pointers but the memory that will be accessed while dereferencing is different.

*/\*P9.3 Program to print size of pointer variables and size of values dereferenced by them\*/*

```
#include<stdio.h>
int main(void)
{
 char a='x', *p1=&a;
 int b=12, *p2=&b;
 float c=12.4, *p3=&c;
 double d=18.34, *p4=&d;
 printf("sizeof(p1)=%u, sizeof(*p1)=%u\n", sizeof(p1), sizeof(*p1));
 printf("sizeof(p2)=%u, sizeof(*p2)=%u\n", sizeof(p2), sizeof(*p2));
 printf("sizeof(p3)=%u, sizeof(*p3)=%u\n", sizeof(p3), sizeof(*p3));
 printf("sizeof(p4)=%u, sizeof(*p4)=%u\n", sizeof(p4), sizeof(*p4));
 return 0;
}
```

Output:

```
sizeof(p1)=4, sizeof(*p1)=1
sizeof(p2)=4, sizeof(*p2)=4
sizeof(p3)=4, sizeof(*p3)=4
sizeof(p4)=4, sizeof(*p4)=8
```

## 9.4 Pointer Arithmetic

All types of arithmetic operations are not possible with pointers. The only valid operations that can be performed are-

- (1) Addition of an integer to a pointer and increment operation.
- (2) Subtraction of an integer from a pointer and decrement operation
- (3) Subtraction of a pointer from another pointer of same type.

Pointer arithmetic is somewhat different from ordinary arithmetic. Here all arithmetic is performed relative to the size of base type of pointer (size of data types is machine dependent so the results may vary on different machines). For example if we have an integer pointer p1 which contains address 1000 then on incrementing we get 1004 instead of 1001. This is because the size of int data type is 4. Similarly on decrementing p1, we will get 996 instead of 999. The expression (p1+3) will represent the address 1012. If we add or subtract an integer from a pointer, the result is another pointer of the same type. Let us see some pointer arithmetic for char, int, float and double pointers.

```
char a='x', *pc=&a;
int b=5, *pi=&b;
```



```
double c=4.564,*pd=&c;
```

Suppose the address of variables *a*, *b* and *c* are 1000, 4000, 5000 respectively, so initially values of *pc*, *pi* and *pd* will be 1000, 4000 and 5000.

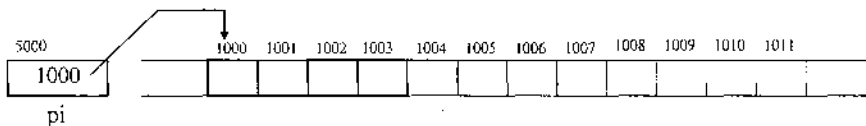
```
pc++; or ++pc; pc=1000+1=1001 (Since char is of 1 byte)
pc=pc-3; pc=1001-3=998
pc=pc+5; pc=998+5=1003
pc--; or --pc; pc=1003-1=1002
```

```
pi++; or ++pi; pi=4000+4=4004 (Since int is of 4 bytes)
pi=pi-3; pi=4004-3=3992
pi=pi+5; pi=3992+5=4012
pi--; or --pi; pi=4012-4=4008
```

```
pd++; or ++pd; pd=5000+8=5008 (Since double is of 8 bytes)
pd=pd-3; pd=5008-3*8=4984
pd=pd+5; pd=4984+5*8=5024
pd--; or --pd; pd=5024-8=5016
```

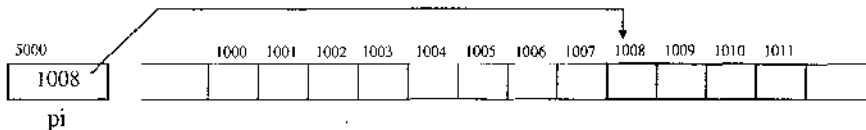
The compiler scales all this arithmetic automatically since it knows the base type of pointer. The arithmetic in the case of char pointer seems to be like ordinary arithmetic because the size of a char is 1 byte. The addresses of variables *a*, *b* and *c* are not affected by these operations, only the pointer moves ahead or backwards.

Suppose *pi* is an integer pointer that contains address 1000 initially.



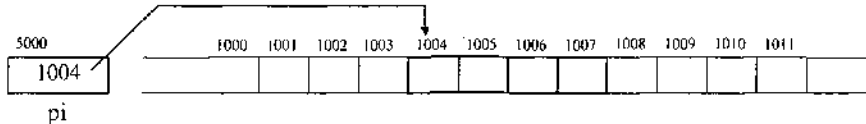
(a) *pi*-*pi*+2;

Now *pi* contains address 1008, and if dereferenced it will return information stored in bytes 1008, 1009, 1010 and 1011.



(b) *pi*--;

Now *pi* contains address 1004, and if dereferenced it will return information stored in bytes 1004, 1005, 1006 and 1007.



(c) `printf("%d", *(pi+2));`

The integer stored in bytes 1012, 1013, 1014, 1015 will be printed. The pointer *pi* will not move to address 1012 since we have not assigned any new value to it.

It is important to note that when we move a pointer somewhere else in memory by incrementing/decrementing or adding/subtracting integers, it not necessary that the pointer still points to a variable of same data type or a valid memory location. The task of allocating memory locations to variables is done by the compiler. We don't know where and in what order it has stored them. We should move the pointers in such a

way that they always point to valid memory locations. In case of arrays, elements are stored in consecutive order, so this arithmetic is generally applied in arrays.

#### Ex 9.4 Pointer arithmetic\*/

```
#include<stdio.h>
int main(void)
{
 int a=5,*pi=&a;
 char b='x',*pc=&b;
 float c=5.5,*pf=&c;
 printf("Value of pi=Address of a=%p\n",pi);
 printf("Value of pc=Address of b=%p\n",pc);
 printf("Value of pf=Address of c=%p\n",pf);
 pi++;
 pc++;
 pf++;
 printf("Now value of pi=%p\n",pi);
 printf("Now value of pc=%p\n",pc);
 printf("Now value of pf=%p\n",pf);
 return 0;
}
```

Output:

```
Value of pi=Address of a=0012FED4
Value of pc=Address of b=0012FF8F
Value of pf=Address of c=0012FEA4
Now value of pi=0012FED8
Now value of pc=0012FF90
Now value of pf=0012FEA8
```

#### Ex 9.5 Postfix/prefix increment/decrement in a pointer variable of base type int\*/

```
#include<stdio.h>
int main(void)
{
 int a=5;
 int *p;
 p=&a;
 printf("Value of p = Address of a = %p\n",p);
 printf("Value of p = %p\n",++p);
 printf("Value of p = %p\n",p++);
 printf("Value of p = %p\n",p);
 printf("Value of p = %p\n",--p);
 printf("Value of p = %p\n",p--);
 printf("Value of p = %p\n",p);
 return 0;
}
```

Output:

```
Value of p = Address of a = 0012FED4
Value of p = 0012FED8
Value of p = 0012FED8
Value of p = 0012FEDC
Value of p = 0012FED8
Value of p = 0012FED8
Value of p = 0012FED4
```

In first `printf` the address of *a* is printed. In second `printf`, first the pointer is incremented then its value is printed. Since base type of pointer is `int`, it is incremented by 4. Similarly other `printf` statements are executed.

Subtraction of a pointer from another pointer of same base type returns an integer, which denotes the number of elements of that type between the two pointers. If we have two `int` pointers *ptr1* and *ptr2*, containing addresses 3000 and 3020 respectively then *ptr2*-*ptr1* will give 5 (since size of `int` is 4). This operation is generally performed when both pointer variables point to the elements of same array.

The arithmetic operations that cannot be performed on pointers are-

1. Addition, multiplication, division of two pointers.
2. Multiplication between pointer and any number.
3. Division of a pointer by any number.
4. Addition of float or double values to pointers.

## 9.5 Precedence of Dereferencing Operator and Increment/Decrement Operators

The precedence level of \* operator and increment/decrement operators is same and their associativity is from right to left. There can be confusion while combining these operators in pointer expressions, so we should use them carefully.

Suppose ptr is an integer pointer and x is an integer variable. Now we'll see how the pointer expressions given below are interpreted.

```
x=*ptr++;
x=**++ptr;
x=++*ptr;
x=(*ptr)++;
```

(i) `x=*ptr++;`

The expression `*ptr++` is equivalent to `*(ptr++)` as these operators associate from right to left. Thus the increment operator will be applied to ptr, and not to \*ptr. The increment operator is postfix, so first the value of ptr will be used in the expression and then it will be incremented. Here first the integer pointed to by ptr will be assigned to x and then ptr will be incremented. This is same as-

```
x=*ptr;
ptr=ptr+1;
```

(ii) `x=**++ptr;`

The expression `**++ptr` is equivalent to `*(++*ptr)`. Here also the increment operator is applied to ptr. The increment operator is prefix, so first ptr will be incremented and then its new value will be used in the expression. Hence firstly the value of ptr is incremented, then value at the new address is assigned to x. This is same as-

```
ptr=ptr+1;
x=*ptr;
```

(iii) `x=++*ptr;`

The expression `++*ptr` is equivalent to `++(*ptr)`. Here the increment operator is applied to \*ptr and not ptr. So here the value of pointer will not change but the value pointed to by the pointer will change i.e. \*ptr will be incremented. Since the increment operator is prefix, first the value of \*ptr will increment and then this value will be assigned to x. This is same as-

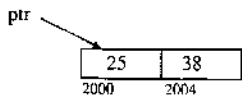
```
*ptr=*ptr+1;
x=*ptr;
```

(iv) `x=(*ptr)++;`

Here also the increment operator is applied to \*ptr and since it is postfix increment, first the value of \*ptr will be assigned to x and then it will be incremented. This is same as-

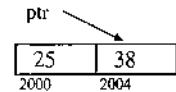
```
x=*ptr;
*ptr=*ptr+1;
```

Let us take an example and understand how these expressions are interpreted. Suppose value at address 2000 is 25, value at address 2004 is 38, ptr is an integer pointer that contains address 2000, thus value of \*ptr is 25. Now let us see what will be the results in the above four cases, if this is the initial condition in all cases.



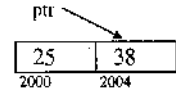
(i) `x=*ptr++;`

Value of x=25, Address contained in ptr=2004, \*ptr=38



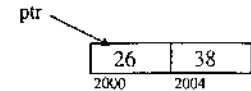
(ii) `x=**++ptr;`

Value of x=38, Address contained in ptr=2004, \*ptr=38



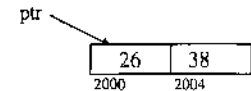
(iii) `x=++*ptr;`

Value of x=26, Address contained in ptr=2000, \*ptr=26



(iv) `x=(*ptr)++;`

Value of x=25, Address contained in ptr=2000, \*ptr=26



From the above four forms of pointer expressions, first one is generally used since it is used to access the elements in an array. In the above statements we have taken increment operator, the same rules apply for decrement operator also.

## 9.6 Pointer Comparisons

The relational operators `==`, `!=`, `<`, `<=`, `>`, `>=` can be used with pointers. The operators `==` and `!=` are used to compare two pointers for finding whether they contain same address or not. They will be equal only if both are NULL or they contain address of same variable. The use of these operators is valid between pointers of same type or between NULL pointer and any other pointer, or between void pointer and any other pointer.

The relational operators `<`, `>`, `<=`, `>=` are valid between pointers of same type. These operations make sense only when both the pointers point to elements of the same array.

## 9.7 Pointer to Pointer

We know that pointer is a variable that can contain memory address. This pointer variable takes some space in memory and hence it also has an address. We can store the address of a pointer variable in some other variable, which is known as a pointer to pointer variable. Similarly we can have a pointer to pointer to pointer variable and this concept can be extended to any limit, but in practice only pointer to pointer is used. Pointer to pointer is generally used while passing pointer variables to functions.

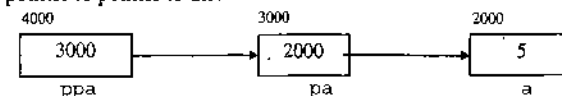
The syntax of declaring a pointer to pointer is-

```
data_type **pptr;
```

Here variable pptr is a pointer to pointer and it can point to a pointer pointing to a variable of type data\_type. The double asterisk used in the declaration informs the compiler that a pointer to pointer is being declared. Now let us take an example-

```
int a=5;
int *pa=&a;
int **ppa=&pa;
```

Here type of variable a is int, type of variable pa is (int \*) or pointer to int, and type of variable ppa is (int \*\*) or pointer to pointer to int.



Here `pa` is a pointer variable which contains the address of the variable `a`, and `ppa` is a pointer to pointer variable, which contains the address of the pointer variable `pa`.

We know that `*pa` gives value of `a`, similarly `*ppa` will give the value of `pa`. Now let us see what value will be given by `**ppa`.

`**ppa`  
 $\rightarrow$  `*(**ppa)`  
 $\rightarrow$  `*pa` (Since `*ppa` gives `pa`)  
 $\rightarrow$  `a` (Since `*pa` gives `a`)

Thus we can see that `**ppa` will give the value of `a`. So to access the value indirectly pointed to by a pointer to pointer, we can use double indirection operator. The table given below will make this concept clear.

|                |    |     |       |      |
|----------------|----|-----|-------|------|
| Value of a     | a  | *pa | **ppa | 5    |
| Address of a   | &a | pa  | *ppa  | 2000 |
| Value of pa    | &a | pa  | *ppa  | 2000 |
| Address of pa  |    | &pa | ppa   | 3000 |
| Value of ppa   |    | &pa | ppa   | 3000 |
| Address of ppa |    |     | &ppa  | 4000 |

```
/*P9.6 Pointer to pointer*/
#include<stdio.h>
int main(void)
```

```
{
 int a=5;
 int *pa;
 int **ppa;
 pa=&a;
 ppa=&pa;
 printf("Address of a=%p\n",&a);
 printf("Value of pa=Address of a=%p\n",pa);
 printf("Value of *pa=Value of a=%d\n",*pa);
 printf("Address of pa=%p\n",&pa);
 printf("Value of ppa=Address of pa=%p\n",ppa);
 printf("Value of *ppa=Value of pa=%p\n",*ppa);
 printf("Value of **ppa=Value of a=%d\n",**ppa);
 printf("Address of ppa=%p\n",&ppa);
 return 0;
}
```

Output:

```
Address of a=0012FED4
Value of pa=Address of a=0012FED4
Value of *pa=Value of a=5
Address of pa=0012FEC8
Value of ppa=Address of pa=0012FEC8
Value of *ppa=Value of pa=0012FED4
Value of **ppa=Value of a=5
Address of ppa=0012FEC8
```

## 9.8 Pointers and One Dimensional Arrays

The elements of an array are stored in contiguous memory locations. Suppose we have an array `arr` of type `int` and size 5.

```
int arr[5]={1,2,3,4,5};
```

It is stored in memory as-

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| 5000   | 5004   | 5008   | 5012   | 5016   |
| 1      | 2      | 3      | 4      | 5      |
| arr[0] | arr[1] | arr[2] | arr[3] | arr[4] |

Here 5000 is the address of first element, and since each element (type `int`) takes 4 bytes, address of next element is 5004, and so on. The address of first element of the array is also known as the base address of the array.

When the name of an array is used in any expression, it is implicitly converted to a pointer to the first element of the array. The type of this pointer depends on the type of the array, if we have an array of `int` type, then its type is 'constant pointer to `int`'. The two exceptions to this rule are when the array name is used as an operand of `sizeof` and `&` operators. When array name is operand of `sizeof` operator, the value returned is size of whole array and when array name is operand of `&` operator the value is a pointer to the whole array(not to the first element).

In C language, pointers and arrays are closely related. We can access the array elements using pointer expressions. Actually the compiler accesses the array elements by converting subscript notation to pointer notation. Following are the main points for understanding the relationship of pointers with arrays.

1. Elements of an array are stored in consecutive memory locations.
2. When the name of an array is used in any expression, the value of the name is a constant pointer that denotes the address of the first element of the array (eg. array name `arr` in an expression is equivalent to `&arr[0]`).
3. According to pointer arithmetic, when a pointer variable is incremented, it points to the next location of its base type.

Let us take an example-

```
int arr[5]={5,10,15,20,25};
```

Here `arr` is an array that has 5 elements each of type `int`.

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| 2000   | 2004   | 2008   | 2012   | 2016   |
| 5      | 10     | 15     | 20     | 25     |
| arr[0] | arr[1] | arr[2] | arr[3] | arr[4] |

We can get the address of an element of array by applying `&` operator in front of subscripted variable name. Hence `&arr[0]` gives address of 0<sup>th</sup> element, `&arr[1]` gives the address of 1<sup>st</sup> element and so on. Since array subscripts start from 0, we'll refer to the first element of array as 0<sup>th</sup> element and so on.

The following program shows that the elements of an array are stored in consecutive memory locations.

```
/*P9.7 Program to print the value and address of the elements of an array.*/
```

```
#include<stdio.h>
int main(void)
{
 int arr[5]={5,10,15,20,25};
 int i;
 for(i=0; i<5; i++)
 {
 printf("Value of arr[%d] = %d\n",i,arr[i]);
 printf("Address of arr[%d] = %p\n",i,&arr[i]);
 }
 return 0;
}
```

Output:

```
Value of arr[0] = 5 Address of arr[0] = 0012FEC4
Value of arr[1] = 10 Address of arr[1] = 0012FEC8
Value of arr[2] = 15 Address of arr[2] = 0012FECC
Value of arr[3] = 20 Address of arr[3] = 0012FED0
Value of arr[4] = 25 Address of arr[4] = 0012FED4
```

The name of the array `arr` denotes the address of 0<sup>th</sup> element of array. The address of 0<sup>th</sup> element can be given by `&arr[0]` also so `arr` and `&arr[0]` represent the same address. The name of an array is a constant pointer, and according to pointer arithmetic when an integer is added to a pointer then we get the address of next element of same base type. Hence `arr+1` will denote the address of the next element `arr[1]`. Similarly `arr+2` denotes the address of `arr[2]` and so on. In other words we can say that the pointer expression `arr+i` points to *i*<sup>th</sup> element of array, `arr+2` points to 2<sup>nd</sup> element of array and so on.

|                    |   |                                   |   |                          |   |      |
|--------------------|---|-----------------------------------|---|--------------------------|---|------|
| <code>arr</code>   | → | Points to 0 <sup>th</sup> element | → | <code>&amp;arr[0]</code> | → | 2000 |
| <code>arr+1</code> | → | Points to 1 <sup>st</sup> element | → | <code>&amp;arr[1]</code> | → | 2004 |
| <code>arr+2</code> | → | Points to 2 <sup>nd</sup> element | → | <code>&amp;arr[2]</code> | → | 2008 |
| <code>arr+3</code> | → | Points to 3 <sup>rd</sup> element | → | <code>&amp;arr[3]</code> | → | 2012 |
| <code>arr+4</code> | → | Points to 4 <sup>th</sup> element | → | <code>&amp;arr[4]</code> | → | 2016 |

In general we can write-

The pointer expression `arr+i` denotes the same address as `&arr[i]`.

Now if we dereference `arr`, then we get the 0<sup>th</sup> element of array, i.e. expression `*arr` or `*(arr-0)` represent 0<sup>th</sup> element of array. Similarly on dereferencing `(arr+1)` we get the 1<sup>st</sup> element and so on.

|                       |   |                                  |   |                     |   |    |
|-----------------------|---|----------------------------------|---|---------------------|---|----|
| <code>*arr</code>     | → | Value of 0 <sup>th</sup> element | → | <code>arr[0]</code> | → | 5  |
| <code>*(arr+1)</code> | → | Value of 1 <sup>st</sup> element | → | <code>arr[1]</code> | → | 10 |
| <code>*(arr+2)</code> | → | Value of 2 <sup>nd</sup> element | → | <code>arr[2]</code> | → | 15 |
| <code>*(arr+3)</code> | → | Value of 3 <sup>rd</sup> element | → | <code>arr[3]</code> | → | 20 |
| <code>*(arr+4)</code> | → | Value of 4 <sup>th</sup> element | → | <code>arr[4]</code> | → | 25 |

In general we can write-

`*(arr+i) → arr[i]`

So in subscript notation the address of an array element is `&arr[i]` and its value is `arr[i]`, while in pointer notation the address is `(arr+i)` and the value is `*(arr+i)`. Sometimes the pointer notation is more efficient than the subscript notation.

*/\*P9.8 Program to print the value and address of elements of an array using pointer notation\*/*

```
#include<stdio.h>
int main(void)
{
 int arr[5]={5,10,15,20,25};
 int i;
 for(i=0; i<5; i++)
 {
 printf("Value of arr[%d] = %d\t",i,*(arr+i));
 printf("Address of arr[%d] = %p\n",i,&arr[i]);
 }
 return 0;
}
```

The output of this program is similar to that of P9.7.

Array subscripting is commutative, i.e. `arr[i]` is same as `i[arr]`. We had seen earlier that-

`arr[i]` is equivalent to `*(arr+i)`

Now `*(arr+i)` is same as `*(i+arr)`, so we can write the above statement as-

`arr[i]` is equivalent to `*(i+arr)`

Now `*(i+arr)` can be written as `i[arr]` in subscript notation, so

`arr[i]` is equivalent to `i[arr]`

The form `i[arr]` is legal and is equivalent to using `arr[i]` but should never be used as it makes the code harder to read.

*/\*P9.9 Program to print the value of array elements using pointer and subscript notation\*/*

```
#include<stdio.h>
int main(void)
{
 int arr[5]={5,10,15,20,25};
 int i=0;
 for(i=0; i<5; i++)
 {
 printf("Value of arr[%d]= ",i);
 printf("%d\t",arr[i]);
 printf("%d\t",*(arr+i));
 printf("%d\t",*(i+arr));
 printf("%d\n",i[arr]);
 printf("Address of arr[%d] = %p\n",i,&arr[i]);
 }
 return 0;
}
```

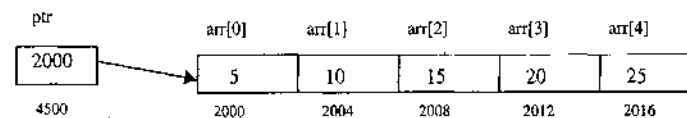
Output:

```
Value of arr[0]= 5 5 5 5
Address of arr[0]= 0012FEC4
Value of arr[1]= 10 10 10 10
Address of arr[1]= 0012FEC8
Value of arr[2]= 15 15 15 15
Address of arr[2]= 0012FECC
Value of arr[3]= 20 20 20 20
Address of arr[3]= 0012FED0
Value of arr[4]= 25 25 25 25
Address of arr[4]= 0012FED4
```

## 9.9 Subscripting Pointer Variables

Suppose we take a pointer variable `ptr`, and initialize it with the address of the 0<sup>th</sup> element an array `arr`.

```
int *ptr;
ptr=arr; /*We can also write ptr=&arr[0]*/
```



On applying pointer arithmetic and dereferencing we can see that the expression `ptr+i` denotes the address of *i*<sup>th</sup> element of array and the expression `*(ptr+i)` denotes the value of *i*<sup>th</sup> element of array. According to the equivalence of pointer and subscript notations, `*(ptr+i)` can be written as `ptr[i]`. So if we have a pointer variable pointing to the 0<sup>th</sup> element of array, then we can access the elements of array by subscripting that pointer variable. This equivalence of pointer and subscript notations is used in dynamic arrays and while sending arrays to functions.

Now let us see the difference between the name of an array and a pointer variable. The name of an array is a constant pointer and so it will always point to the 0<sup>th</sup> element of the array. It is not a variable so we can't assign some other address to it neither can we move it by incrementing or decrementing.

```
arr=# /*Illegal*/
arr++; /*Illegal*/
arr=arr-1; /*Illegal*/
```

But since `ptr` is a pointer variable, all these operations are valid for it.

```
ptr=# /*Now ptr points to variable num*/
ptr++; /*ptr points to next location*/
ptr=ptr-1; /*ptr points to previous location*/
```

When an array is declared, space is allocated for the specified number of elements but when a pointer variable is declared space is allocated only for the pointer variable.

```

/*P9.10 Program to print the value and address of array elements by subscripting a pointer variable*/
#include<stdio.h>
int main(void)
{
 int arr[5]={5,10,15,20,25};
 int i,*p;
 p=arr;
 for(i=0; i<5; i++)
 {
 printf("Address of arr[%d]= %p %p %p %p\n",i,&arr[i],arr+i,p+i,&p[i]);
 printf("Value of arr[%d]= %d %d %d %d\n",i,arr[i].*(arr+i).*(p+i),p[i]);
 }
 return 0;
}

```

Output:

```

Address of arr[0]= 0012FEC4 0012FEC4 0012FEC4 0012FEC4
Value of arr[0]= 5 5 5 5
Address of arr[1]= 0012FEC8 0012FEC8 0012FEC8 0012FEC8
Value of arr[1]= 10 10 10 10
Address of arr[2]= 0012FEC0 0012FEC0 0012FEC0 0012FEC0
Value of arr[2]= 15 15 15 15
Address of arr[3]= 0012FED0 0012FED0 0012FED0 0012FED0
Value of arr[3]= 20 20 20 20
Address of arr[4]= 0012FED4 0012FED4 0012FED4 0012FED4
Value of arr[4]= 25 25 25 25

```

## 9.10 Pointer to an Array(Array pointer)

In the previous section, we had a pointer that pointed to the 0<sup>th</sup> element of array. We can also declare a pointer that can point to the whole array instead of only one element of array. This pointer is useful when talking about multidimensional arrays. Let us see how we can declare a pointer to an array.

```
int (*ptr)[10];
```

Here ptr is pointer that can point to an array of 10 integers. Since subscripts have higher precedence than indirection, it is necessary to enclose the indirection operator and pointer name inside parentheses. Here the type of ptr is 'pointer to an array of 10 integers'. Note that the pointer that points to the 0<sup>th</sup> element of array and the pointer that points to the whole array are totally different. The following program shows this-

```

/*P9.11 Program to understand difference between pointer to an integer and pointer to an array of integers*/
#include<stdio.h>
int main(void)
{

```

```

 int *p; /*Can point to an integer*/
 int (*ptr)[5]; /*Can point to an array of 5 integers*/
 int arr[5];
 p=arr; /*Points to 0th element of arr*/
 ptr=&arr; /*Points to the whole array arr*/
 printf("p=%p,ptr=%p\n",p,ptr);
 p++;
 ptr++;
 printf("p=%p,ptr=%p\n",p,ptr);
 return 0;
}

```

Output:

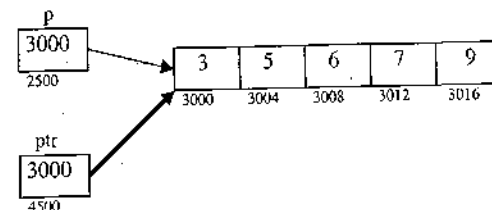
```

p=0012FEAC,ptr=0012FEAC
p=0012FEB0,ptr=0012FEC0

```

Here p is pointer that points to 0<sup>th</sup> element of array arr, while ptr is a pointer that points to the whole array arr. The base type of p is int while base type of ptr is 'an array of 5 integers'. We know that the pointer arithmetic is performed relative to the base size, so if we write ptr++, then the pointer ptr will be shifted forward

ward by 20 bytes. The following figure shows the pointers p and ptr, in this chapter we will use a darker arrow to denote pointer to an array.



On dereferencing a pointer expression we get a value pointed to by that pointer expression. Pointer to an array points to an array, so on dereferencing it we should get the array, and the name of array denotes the base address. So whenever a pointer to an array is dereferenced, we get the base address of the array to which it points.

```

/*P9.12 Program to dereference a pointer to an array*/
#include<stdio.h>
int main(void)
{

```

```

 int arr[5] = {3,5,6,7,9};
 int *p=arr;
 int (*ptr)[5]=&arr;
 printf("p=%p, ptr=%p\n",p,ptr);
 printf("*p=%d, *ptr=%p\n",*p,*ptr);
 printf("sizeof(p)=%u,sizeof(*p)=%u\n",sizeof(p),sizeof(*p));
 printf("sizeof(ptr)=%u,sizeof(*ptr)=%u\n",sizeof(ptr),sizeof(*ptr));
 return 0;
}

```

Output:

```

p=0012FEC4, ptr=0012FEC4
*p=3, *ptr=0012FEC4
sizeof(p)=4,sizeof(*p)=4
sizeof(ptr)=4,sizeof(*ptr)=20

```

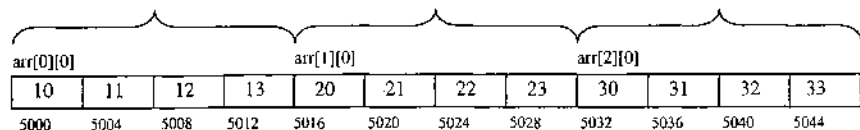
## 9.11 Pointers and Two Dimensional Arrays

In a two dimensional array, we can access each element by using two subscripts, where first subscript represents row number and second subscript represents the column number. The elements of 2-D array can be accessed with the help of pointer notation also. Suppose arr is a 2-D array, we can access any element arr[i][j] of this array using the pointer expression `*(arr+i)+j`. Now we'll see how this expression can be derived. Let us take a two dimension array arr [3] [4]-

```
int arr[3][4]={ {10,11,12,13}, {20,21,22,23}, {30,31,32,33}};
```

|       | Column 0 | Column 1 | Column 2 | Column 3 |
|-------|----------|----------|----------|----------|
| Row 0 | 10       | 11       | 12       | 13       |
| Row 1 | 20       | 21       | 22       | 23       |
| Row 2 | 30       | 31       | 32       | 33       |

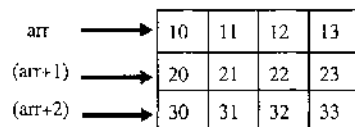
We have been talking about 2-D arrays in terms of rows and columns, but since memory in computer is organized linearly it is not possible to store the 2-D array in rows and columns. The concept of rows and columns is only theoretical, actually a 2-D array is stored in row major order i.e. rows are placed next to each other. The following figure shows how the above 2-D array will be stored in memory.



Each row can be considered as a 1-D array, so a two-dimensional array can be considered as a collection of one-dimensional arrays that are placed one after another. In other words we can say that a 2-D array is an array of arrays. So here `arr` is an array of 3 elements where each element is a 1-D array of 4 integers.

We know that the name of an array is a constant pointer that points to the 0<sup>th</sup> element of array. In the case of 1-D arrays, 0<sup>th</sup> element is a 1-D array, so the name of a 2-D array represents a pointer to a 1-D array. For example in the above case, `arr` is a pointer to 0<sup>th</sup> 1-D array and contains address 5000. Since `arr` is a 'pointer to an array of 4 integers', according to pointer arithmetic the expression `arr+1` will represent the address 5016 and expression `arr+2` will represent address 5032.

So we can say that `arr` points to the 0<sup>th</sup> 1-D array, `arr+1` points to the 1<sup>st</sup> 1-D array and `arr+2` points to the 2<sup>nd</sup> 1-D array.



|                    |                                                         |                                       |        |
|--------------------|---------------------------------------------------------|---------------------------------------|--------|
| <code>arr</code>   | - Points to 0 <sup>th</sup> element of <code>arr</code> | - Points to 0 <sup>th</sup> 1-D array | - 5000 |
| <code>arr+1</code> | - Points to 1 <sup>st</sup> element of <code>arr</code> | - Points to 1 <sup>st</sup> 1-D array | - 5016 |
| <code>arr+2</code> | - Points to 2 <sup>nd</sup> element of <code>arr</code> | - Points to 2 <sup>nd</sup> 1-D array | - 5032 |

In general we can write-

`arr+i` Points to  $i^{\text{th}}$  element of `arr` → Points to  $i^{\text{th}}$  1-D array

Since `arr+i` points to  $i^{\text{th}}$  element of `arr`, on dereferencing it we'll get  $i^{\text{th}}$  element of `arr` which is of course a 1-D array. Thus the expression `*(arr+i)` gives us the base address of  $i^{\text{th}}$  1-D array.

We have proved earlier that the pointer expression `*(arr+i)` is equivalent to the subscript expression `arr[i]`. So `*(arr+i)` which is same as `arr[i]` gives the base address of  $i^{\text{th}}$  1-D array.

`*(arr+0) - arr[0]` - Base address of 0<sup>th</sup> 1-D array - Points to 0<sup>th</sup> element of 0<sup>th</sup> 1-D array - 5000  
`*(arr+1) - arr[1]` - Base address of 1<sup>st</sup> 1-D array - Points to 0<sup>th</sup> element of 1<sup>st</sup> 1-D array - 5016  
`*(arr+2) - arr[2]` - Base address of 2<sup>nd</sup> 1-D array - Points to 0<sup>th</sup> element of 2<sup>nd</sup> 1-D array - 5032

In general we can write-

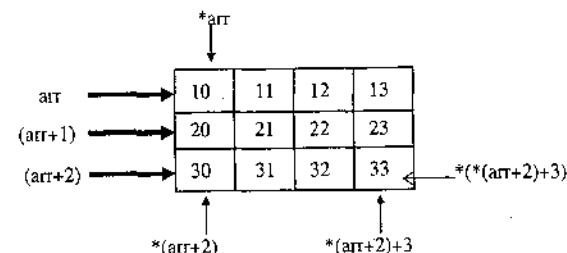
`*(arr+i) - arr[i]` - Base address of  $i^{\text{th}}$  1-D array - Points to 0<sup>th</sup> element of  $i^{\text{th}}$  1-D array

Note that both the expressions `(arr+i)` and `*(arr+i)` are pointers, but their base type is different. The base type of `(arr+i)` is 'an array of 4 ints' while the base type of `*(arr+i)` or `arr[i]` is `int`.

To access an individual element of our 2-D array, we should be able to access any  $j^{\text{th}}$  element of the  $i^{\text{th}}$  1-D array. Since the base type of `*(arr+i)` is `int` and it contains the address of 0<sup>th</sup> element of  $i^{\text{th}}$  1-D array, we can get the addresses of subsequent elements in the  $i^{\text{th}}$  1-D array by adding integer values to `*(arr+i)`. For example `*(arr+i)+1` will represent the address of 1<sup>st</sup> element of  $i^{\text{th}}$  1-D array and `*(arr+i)+2` will represent the address of 2<sup>nd</sup> element of  $i^{\text{th}}$  1-D array. Similarly `*(arr+i)+j` will represent the address of  $j^{\text{th}}$  element of  $i^{\text{th}}$  1-D array. On dereferencing this expression we can get the  $j^{\text{th}}$  element of the  $i^{\text{th}}$  1-D array.

`arr` Points to 0<sup>th</sup> 1-D array  
`*arr` Points to 0<sup>th</sup> element of 0<sup>th</sup> 1-D array  
`(arr+i)` Points to  $i^{\text{th}}$  1-D array

`*(arr+i)` Points to 0<sup>th</sup> element of  $i^{\text{th}}$  1-D array  
`*(arr+i)+j` Points to  $j^{\text{th}}$  element of  $i^{\text{th}}$  1-D array  
`*(*(arr+i)+j)` Represents the value of  $j^{\text{th}}$  element of  $i^{\text{th}}$  1-D array



**Program to print the values and address of elements of a 2-D array\*/**  

```
#include<stdio.h>
int main(void)
```

```
int arr[3][4] = {
 {10,11,12,13},
 {20,21,22,23},
 {30,31,32,33}
};

int i,j;
for(i=0; i<3; i++)
{
 printf("Address of %dth array = %p %p\n",i,arr[i],*(arr+i));
 for(j=0; j<4; j++)
 printf("%d %d ",arr[i][j],*(*(arr+i)+j));
 printf("\n");
}
return 0;
```

Output:

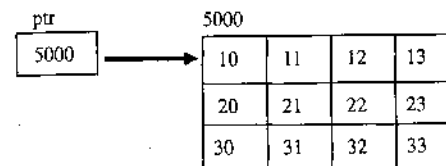
```
Address of 0th array = 0012FEA8 0012FEA8
10 10 11 11 12 12 13 13
Address of 1th array = 0012FEB8 0012FEB8
20 20 21 21 22 22 23 23
Address of 2th array = 0012FEC8 0012FEC8
30 30 31 31 32 32 33 33
```

## 9.12 Subscripting Pointer to an Array

In section 9.9, we had seen how to subscript a pointer variable that contains the base address of a 1-D array. Now we'll see how to subscript a pointer to an array that contains the base address of a 2-D array.

Suppose `arr` is a 2-D array with 3 rows and 4 columns and `ptr` is a pointer to an array of 4 integers, and `ptr` contains the base address of array `arr`.

```
int arr[3][4] = {{10,11,12,13},{20,21,22,23},{30,31,32,33}};
int (*ptr)[4];
ptr=arr;
```



Since `ptr` is a pointer to an array of 4 integers, `ptr+i` will point to  $i^{\text{th}}$  row. On dereferencing `ptr+i`, we get base address of  $i^{\text{th}}$  row. To access the address of  $j^{\text{th}}$  element of  $i^{\text{th}}$  row we can add `j` to the pointer expression `*(ptr+i)`. So the pointer expression `*(ptr+i)+j` gives the address of  $j^{\text{th}}$  element of  $i^{\text{th}}$  row and the pointer expression `*(*(ptr+i)+j)` gives the value of the  $j^{\text{th}}$  element of  $i^{\text{th}}$  row.

We have studied that the pointer expression `*(*(ptr+i)+j)` is equivalent to subscript expression `ptr[i][j]`. So if we have a pointer variable containing the base address of 2-D array, then we can access the elements of array by double subscripting that pointer variable.

*/\*P9.14 Program to print elements of a 2-D array by subscripting a pointer to an array variable\*/*

```
#include<stdio.h>
int main(void)
{
 int arr[3][4] = { {10,11,12,13}, {20,21,22,23}, {30,31,32,33} };
 int (*ptr)[4];
 ptr = arr;
 printf("%p %p %p\n", ptr, ptr+1, ptr+2);
 printf("%p %p %p\n", *ptr, *(ptr+1), *(ptr+2));
 printf("%d %d %d\n", **ptr, ***(ptr+1)+2, ***(ptr+2)+3);
 printf("%d %d %d\n", ptr[0][0], ptr[1][2], ptr[2][3]);
 return 0;
}
```

Output:

```
0012FEA8 0012FEB8 0012FEC8
0012FEA8 0012FEB8 0012FEC8
10 22 33
10 22 33
```

### 9.13 Pointers and Three Dimensional Arrays

In a three dimensional array we can access each element by using three subscripts. Let us take a 3-D array-

```
int arr[2][3][2] = { { {5,10}, {6,11}, {7,12} }, { {20,30}, {21,31}, {22,32} } };
```

We can consider a three dimensional array to be an array of 2-D arrays i.e. each element of a 3-D array is considered to be a 2-D array. The 3-D array `arr` can be considered as an array consisting of two elements where each element is a 2-D array. The name of the array `arr` is a pointer to the 0<sup>th</sup> element of the array, so `arr` points to the 0<sup>th</sup> 2-D array.

Now let us see how we can access any element of a 3-d array using pointer notation.

|                                 |                                                                                                                                                                                                                                               |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>arr</code>                | Points to 0 <sup>th</sup> 2-D array                                                                                                                                                                                                           |
| <code>arr+i</code>              | Points to $i^{\text{th}}$ 2-D array                                                                                                                                                                                                           |
| <code>*(arr+i)</code>           | Gives base address of $i^{\text{th}}$ 2-D array,<br>so points to 0 <sup>th</sup> element of $i^{\text{th}}$ 2-D array,<br>each element of 2-D array is a 1-D array,<br>so it points to 0 <sup>th</sup> 1-D array of $i^{\text{th}}$ 2-D array |
| <code>*(arr+i)+j</code>         | Points to $j^{\text{th}}$ 1-D array of $i^{\text{th}}$ 2-D array                                                                                                                                                                              |
| <code>*(*(arr+i)+j)</code>      | Gives base address of $j^{\text{th}}$ 1-D array of $i^{\text{th}}$ 2-D array,<br>so it points to 0 <sup>th</sup> element of $j^{\text{th}}$ 1-D array of $i^{\text{th}}$ 2-D array                                                            |
| <code>*(*(arr+i)+j)+k</code>    | Points to $k^{\text{th}}$ element of $j^{\text{th}}$ 1-D array of $i^{\text{th}}$ 2-D array                                                                                                                                                   |
| <code>*(*(*(arr+i)+j)+k)</code> | Gives the value of $k^{\text{th}}$ element of $j^{\text{th}}$ 1-D array of $i^{\text{th}}$ 2-D array                                                                                                                                          |

Thus the pointer expression `*(*(*(arr+i)+j)+k)` is equivalent to the subscript expression `arr[i][j][k]`.

Earlier we had seen that the expression `*(arr+i)` is equivalent to `arr[i]` and the expression `*(*(arr+i)+j)` is equivalent to `arr[i][j]`. So we can say that `arr[i]` represents the base address of  $i^{\text{th}}$  2-D array and `arr[i][j]` represents the base address of the  $j^{\text{th}}$  1-D array of  $i^{\text{th}}$  2-D array.

*/\*P9.15 Program to print the elements of 3-D array using pointer notation\*/*

```
#include<stdio.h>
int main(void)
{
 int arr[2][3][2] = {
 { {5,10}, {6,11}, {7,12} },
 { {20,30}, {21,31}, {22,32} }
 };

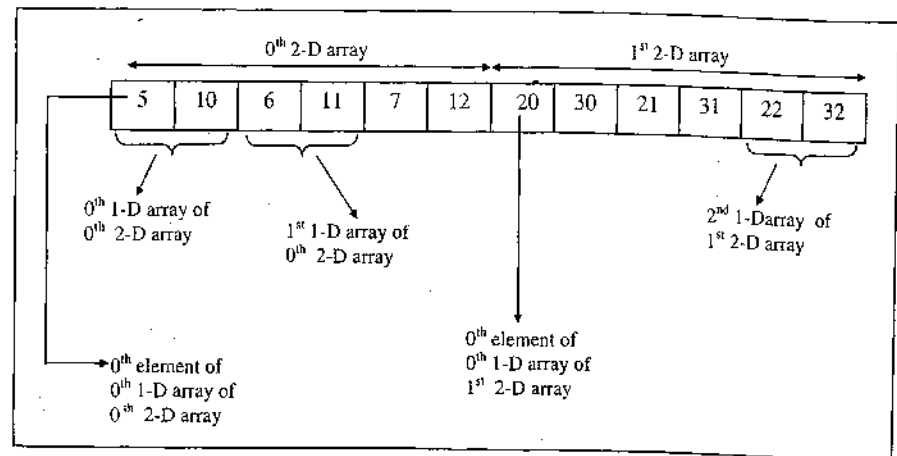
 int i,j,k;
 for(i=0; i<2; i++)
 for(j=0; j<3; j++)
 for(k=0; k<2; k++)
 printf("%d\t", *(*(arr+i)+j)+k);
 printf("\n");

 return 0;
}
```

Output:

```
5 10
6 11
7 12
20 30
21 31
22 32
```

The following figure shows how the 3-D array used in the above program is stored in memory.



### 9.14 Pointers and Functions

Arguments can be passed to functions in two ways-

- Call by value
- Call by reference

In call by value, only the values of arguments are sent to the function while in call by reference, addresses of arguments are sent to the function. In call by value method, any changes made to the formal parameters do not change the actual argument. In call by reference method, any changes made to the formal parameters change the actual arguments. C uses only call by value when passing arguments to a function, but we can simulate call by reference using pointers. All the functions that we had written so far used call by value method. Here is another simple program that uses call by value-

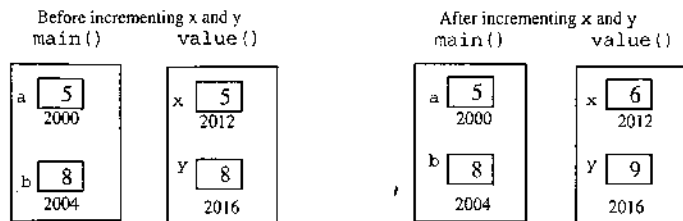
```
/*P9.16 Call by value*/
#include<stdio.h>
void value(int x,int y);
int main(void)
{
 int a=5,b=8;
 printf("a=%d,b=%d\n",a,b);
 value(a,b);
 printf("a=%d,b=%d\n",a,b);
 return 0;
}

void value(int x,int y)
{
 x++;
 y++;
 printf("x=%d,y=%d\n",x,y);
}

Output:
a=5,b=8
x=6,y=9
a=5,b=8
```

Here *a* and *b* are variables declared in the function *main()*, while *x* and *y* are declared in the function *value()*. These variables reside at different addresses in memory. Whenever the function *value()* is called, two variables named *x* and *y* are created and are initialized with the values of variables *a* and *b*. This type of parameter passing is called call by value since we are only supplying the values of actual arguments to the calling function. Any operation performed on variables *x* and *y* in the function *value()*, will not affect variables *a* and *b*.

Before calling the function *value()*, the value of *a*=5 and *b*=8. The values of *a* and *b* are copied into *x* and *y*. Since the memory locations of *x*, *y* and *a*, *b* are different, when the values of *x* and *y* are incremented there will not be any effect on the values of *a* and *b*. Therefore after calling the function, *a* and *b* are same as before calling the function and have the value 5 and 8.



Although C does not have call by reference, we can simulate it by passing addresses of variables as arguments to the function. To accept the addresses inside the function, we will need pointer variables. Here is a program that simulates call by reference by passing addresses of variables *a* and *b*.

```
/*P9.17 Call by reference*/
#include<stdio.h>
void ref(int *p,int *q);
int main(void)
{

```

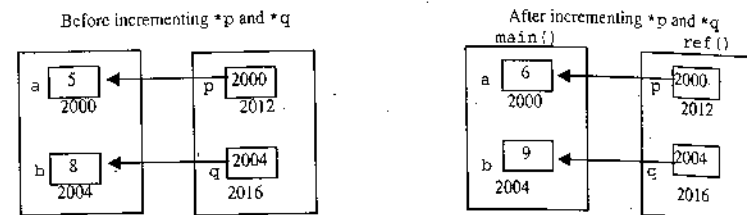
```
 int a=5,b=8;
 printf("a=%d,b=%d\n",a,b);
 ref(&a,&b);
 printf("a=%d,b=%d\n",a,b);
 return 0;
}

void ref(int *p,int *q)
{
 (*p)++;
 (*q)++;
 printf("p=%d, *q=%d\n",*p,*q);
}
```

Output:  
a=5,b=8  
p=6,\*q=9  
a=6,b=9

Here we are passing addresses of variables *a* and *b* in the function call. So the receiving formal parameters in the function declaration should be declared of pointer type. Whenever function *ref()* is called, two pointer to *int* variables, named *p* and *q* are created and they are initialized with the addresses of *a* and *b*. Now if we dereference pointers *p* and *q*, we will be able to access variables *a* and *b* from function *ref()*.

*main()* accesses the memory locations occupied by variables *a* and *b* by writing their names, while *ref()* accesses the same memory locations indirectly by writing *\*p*, *\*q*.



Before calling the function *ref()*, the value of *a*=5 and *b*=8. The value of actual arguments are copied into pointer variables *p* and *q*, and here the actual arguments are addresses of variables *a* and *b*. Since *p* contains address of variable *a*, we can access variable *a* inside *ref()* by writing *\*p*, similarly variable *b* can be accessed by writing *\*q*.

Now *(\*p)++* means value at address 2000 (which is 5) is incremented. Similarly *(\*q)++* means value at address 2004 (which is 8) is incremented. Now the value of *\*p*=6 and *\*q*=9. When we come back to *main()*, we see that the values of variables *a* and *b* have changed. This is because the function *ref()* knew the addresses of *a* and *b*, so it was able to access them indirectly. In this way we could simulate call by reference by passing addresses as arguments.

Call by reference can be used when the called function has to return more than one values to the function. It is also useful when the caller function needs to modify its parameters or when passing large data objects. If we pass a large structure to a function using call by value, a copy of that structure will be made, but if we want to avoid the overhead of making a copy we can pass a pointer to the structure instead of the structure.

## 9.15 Returning more than one value from a Function

We have studied that we can return only one value from a function through *return* statement. This limitation can be overcome using call by reference. Let us take a simple example to understand this concept. Suppose we want a function to return the sum, difference and product of two numbers passed to it. If we use *return* statement, we will have to make three different functions with one *return* statement in each. The following program shows how we can return all these values from a single function.

```
/*P9.18 Returning more than one value from a function using call by reference*/
```



```
#include<stdio.h>
void func(int x,int y,int *ps,int *pd,int *pp);
int main(void)
{
 int a,b,sum,diff,prod;
 a=6;
 b=4;
 func(a,b,&sum,&diff,&prod);
 printf("Sum=%d, Difference=%d, Product=%d\n",sum,diff,prod);
 return 0;
}

void func(int x,int y,int *ps,int *pd,int *pp)
{
 *ps=x+y;
 *pd=x-y;
 *pp=x*y;
}
```

Output:  
Sum=10, Difference=2, Product=24

In func(), variables a and b are passed by value while variables sum, diff, prod are passed by reference. The function func() knows the addresses of variables sum, diff and prod, so it accesses these variables indirectly using pointers and assigns appropriate values to them.

## 9.16 Function Returning Pointer

We can have a function that returns a pointer. The syntax of declaration of such type of function is-type \*func(type1,type 2,...);

For example-

```
float *fun(int, char); /*This function returns a pointer to float*/
int *func(int, int); /*This function returns a pointer to int*/
```

While returning a pointer, make sure that the memory address returned by the pointer will exist even after the termination of function. For example a function of this form is wrong.

```
int main(void)
{
 int *ptr;
 ptr=func();

}

int *func()
{
 int x=5;
 int *p=&x;

 return p;
}
```

Here we are returning a pointer which points to a local variable. We know that a local variable exists only inside the function. Suppose the variable x is stored at address 2500, the value of p will be 2500 and this value will be returned by the function func(). As soon as func() terminates, the local variables x will cease to exist. The address returned by func() is assigned to pointer variable ptr inside main(), so now ptr will contain address 2500. When we dereference ptr, we are trying to access the value of a variable that no longer exists. So never return a pointer that points to a local variable. Now we will take a program that uses a function returning pointer.

```
/*P9.19 Function returning pointer*/
#include<stdio.h>
int *fun(int *p,int n);
int main(void)
{
 int n=5,arr[10]={1,2,3,4,5,6,7,8,9,10};
```

```
 int *ptr;
 ptr=fun(arr,n);
 printf("Value of arr=%p, Value of ptr=%p, value of *ptr=%d\n",arr,ptr,*ptr);
 return 0;
}
```

```
int *fun(int *p,int n)
```

```
 p=p+n;
 return p;
}
```

Output:  
Value of arr=0012FPA4, Value of ptr=0012FEB8, value of \*ptr=6

## 9.17 Passing a 1-D Array to a Function

In the previous chapter we had studied that when an array is passed to a function, the changes made inside the function affect the original array. This is because the function gets access to the original array. Here is a simple program that verifies this fact.

/\*P9.20 Passing 1-D array to a function\*/

```
#include<stdio.h>
void func(int a[]);
int main(void)
{
 int i,arr[5]={3,6,2,7,1};
 func(arr);
 printf("Inside main(): ");
 for(i=0; i<5; i++)
 printf("%d ",arr[i]);
 printf("\n");
 return 0;
}
```

```
void func(int a[])
{
 int i;
 printf("Inside func(): ");
 for(i=0; i<5; i++)
 {
 a[i]=a[i]+2;
 printf("%d ",a[i]);
 }
 printf("\n");
}
```

Output:  
Inside func() : 5 8 4 9 3  
Inside main() : 5 8 4 9 3

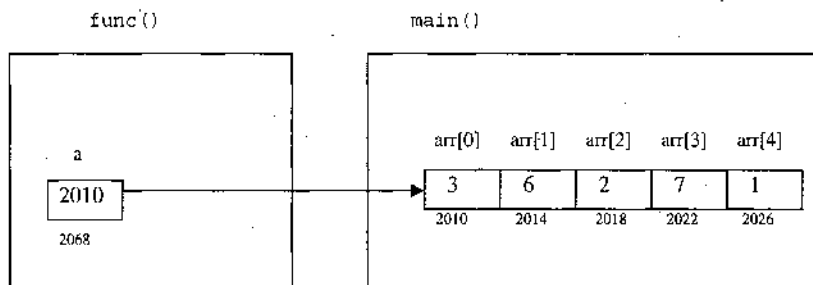
Now after studying about pointers we are in a position to understand what actually happens when an array is passed to a function. Actually a whole array is never passed to a function, only a pointer to the first element of the array is passed. There are three ways of declaring a formal parameter which has to receive the array. We can declare it as an unsized or sized array or we can declare it as a pointer.

```
func(int a[])
```

```
func(int a[5])
```

```
func(int *a)
```

In all the three cases the compiler reserves space only for a pointer variable inside the function. In the function call, the array name is passed without any subscript or address operator. Since array name represents the address of first element of array, this address is assigned to the pointer variable in the function. So inside the function we have a pointer that contains the base address of the array. In the above program, the argument `a` is declared as a pointer variable whose base type is `int`, and it is initialized with the base address of array `arr`. We have studied that if we have a pointer variable containing the base address of an array, then we can access any array element either by pointer notation or subscript notation. So inside the function, we can access any  $i^{\text{th}}$  element of `arr` by writing `*(a+i)` or `a[i]`. Since we are directly accessing the original array, all the changes made to the array in the called function are reflected in the calling function.



The following program will illustrate the point that we have discussed.

*/\*P9.21 Program to verify the fact that when an array is passed to a function, the receiving parameter is declared as a pointer\*/*

```
#include<stdio.h>
void func(double f[],int *i,char c[5]);
int main(void)
{
 double d_arr[5] = {1.4,2.5,3.7,4.1,5.9};
 int i_arr[5] = {1,2,3,4,5};
 char c_arr[5] = {'a','b','c','d','e'};
 printf("Inside main() : ");
 printf("sizeof(d_arr)=%u\t",sizeof(d_arr));
 printf("sizeof(i_arr)=%u\t",sizeof(i_arr));
 printf("sizeof(c_arr)=%u\n",sizeof(c_arr));
 printf("%p %p %p\n",d_arr,i_arr,c_arr);
 func(d_arr,i_arr,c_arr);
 return 0;
}
void func(double d[],int *i,char c[5])
{
 printf("Inside func() : ");
 printf("sizeof(d)=%u\t",sizeof(d));
 printf("sizeof(i)=%u\t",sizeof(i));
 printf("sizeof(c)=%u\n",sizeof(c));
 printf("%p %p %p\n",d,i,c);
}
```

Output:  
 Inside main() : sizeof(d\_arr)=40    sizeof(i\_arr)=20    sizeof(c\_arr)=5  
 0012FEB0 0012FE94 0012FE84  
 Inside func() : sizeof(d)=4    sizeof(i)=4    sizeof(c)=4  
 0012FEB0 0012FE94 0012FE84

Inside the function `func()`, `d`, `i`, `c` are declared as pointers, and this is evident by the fact that the size of each one of them is 4 bytes. Pointers to arrays are passed to functions instead of whole arrays because it is more efficient to copy a pointer instead of copying the whole array.

## 9.18 Passing a 2-D Array to a Function

We have studied in the previous chapter that whenever a multidimensional array is passed to a function, it is optional to specify the leftmost dimension but all other dimensions must be specified. So if we have a 2-D array with 3 rows and 4 columns, then the definition of a function that accepts it can be written in these two ways-

```
func(int a[3][4])
```

```
func(int a[][4])
```

Any changes made to the array in the function will be reflected in the calling function. Whenever a 2-D array is passed to a function, the function actually receives a pointer to a 1-D array, where the size of 1-D array is equal to the number of columns. For example in the above case the function receives a pointer to an array of 4 integers. So we may write the function definition in this form also-

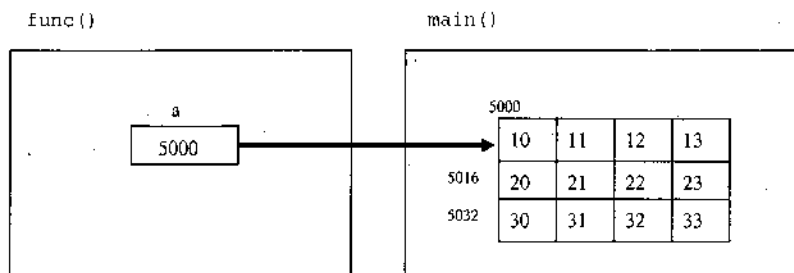
```
func(int (*a)[4])
```

Here `a` is declared as a pointer to an array of 4 integers, and it is initialized with the base address of the original 2-D array. Now inside the function we can use either pointer notation or subscript notation to access the elements. The following program will make these points clear.

*/\*P9.22 Passing a 2-D array to a function\*/*

```
#include<stdio.h>
void func(int (*a)[4]);
int main(void)
{
 int i,j,arr[3][4] = {
 {10,11,12,13},
 {20,21,22,23},
 {30,31,32,33}
 };
 printf("Inside main() : sizeof(arr) = %u\n",sizeof(arr));
 func(arr);
 printf("Contents of array after calling func() are :\n");
 for(i=0; i<3; i++)
 {
 for(j=0; j<4; j++)
 printf("%d ",arr[i][j]);
 printf("\n");
 }
 return 0;
}
void func(int (*a)[4])
{
 int i,j;
 printf("Inside func() : sizeof(a) = %u\n",sizeof(a));
 printf("Inside func() : sizeof(*a) = %u\n",sizeof(*a));
 for(i=0; i<3; i++)
 for(j=0; j<4; j++)
 a[i][j] = a[i][j]*2;
}
```

Output:  
 Inside main() : sizeof(arr) = 48  
 Inside func() : sizeof(a) = 4  
 Inside func() : sizeof(\*a) = 16  
 Contents of array after calling func() are :  
 22 22 22 22  
 42 42 42 42  
 62 62 62 62



## 9.19 Array of Pointers(Pointer Array)

We can declare an array that contains pointers as its elements. Every element of this array is a pointer variable that can hold address of any variable of appropriate type. The syntax of declaring an array of pointers is similar to that of declaring arrays except that an asterisk is placed before the array name.

datatype \*arrayname[size];

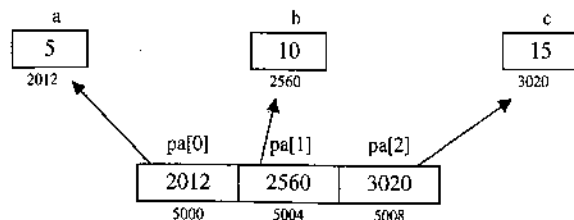
For example to declare an array of size 10 that contains integer pointers we can write-

```
int *arrp[10];
/*P9.23 Array of pointers*/
#include<stdio.h>
int main(void)
{
 int *pa[3];
 int i,a=5,b=10,c=15;
 pa[0]=&a; pa[1]=&b; pa[2]=&c;

 for(i=0; i<3; i++)
 {
 printf("pa[%d]=%p\t",i,pa[i]);
 printf("pa[%d]=%d\n",i,*pa[i]);
 }
 return 0;
}
```

Output:

```
pa[0]=0012FEB4 *pa[0]=5
pa[1]=0012FEA8 *pa[1]=10
pa[2]=0012FE9C *pa[2]=15
```



Here `pa` is declared as an array of pointers. Every element of this array is a pointer to an integer. `pa[i]` gives the value of the  $i^{\text{th}}$  element of `pa` which is address of an `int` variable and `*pa[i]` gives the value of the  $i^{\text{th}}$  `int` variable.

The array of pointers can also contain addresses of elements of another array.

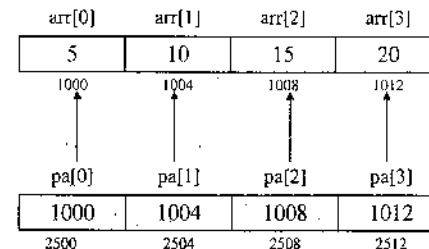
/\*P9.24 Program for understanding array of pointers\*/

```
#include<stdio.h>
int main(void)
{
 int i,arr[4] = {5,10,15,20};
 int *pa[4];
 for(i=0; i<4; i++)
 pa[i] = &arr[i];
 for(i=0; i<4; i++)
 {
 printf("pa[%d] = %p\t",i,pa[i]);
 printf("pa[%d] = %d\n",i,*pa[i]);
 }
 return 0;
}
```

Output:

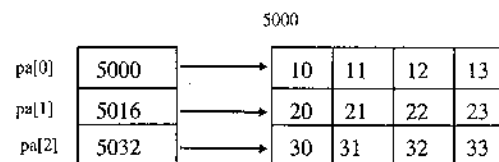
```
pa[0]=0012FEB4 *pa[0]=5
pa[1]=0012FEC0 *pa[1]=10
pa[2]=0012FEC4 *pa[2]=15
pa[3]=0012FEC8 *pa[3]=20
```

Here `pa` is declared as array of pointers. Each element of this array contains the address of each element of array `arr`.



Now we will take a 2-D array `arr` with 3 rows and 4 columns. An array of pointers of size 3 is declared and each pointer in this array is assigned the address of  $0^{\text{th}}$  element of each row of the 2-D array, i.e.  $i^{\text{th}}$  element of `pa` is a pointer to  $0^{\text{th}}$  element of  $i^{\text{th}}$  row of a 2-D array. This can be done as-

```
int arr[3][4]={{10,11,12,13},{20,21,22,23},{30,31,32,33}};
int *pa[3];
for(i=0; i<3; i++)
 pa[i]=arr[i];
```



Now let us see how we can access the elements of the 2-D array `arr` using the array of pointers `pa`. `pa[0]` is a pointer to the  $0^{\text{th}}$  element of  $0^{\text{th}}$  1-D array of the array `arr`, similarly `pa[i]` will be a pointer to  $0^{\text{th}}$  element of  $i^{\text{th}}$  1-D array of `arr`.

If we want the address of  $j^{\text{th}}$  element of  $i^{\text{th}}$  1-D array, we can add `j` to the expression `pa[i]`. Thus the expression `pa[i]+j` will give the address of  $j^{\text{th}}$  element of  $i^{\text{th}}$  1-D array and the expression `*(pa[i]+j)` will give the value of the  $j^{\text{th}}$  element of  $i^{\text{th}}$  1-D array.

We know that `pa[i]` is equivalent to `*(pa+i)`, so the above expression can be written as `*(*(pa+i)+j)`, and we know that this expression is equivalent to `pa[i][j]`. Thus we can access the  $j^{\text{th}}$  element of  $i^{\text{th}}$  1-D array by writing `pa[i][j]`.

### \*P9.25 Array of pointers\*

```
#include<stdio.h>
int main(void)
{
 int i,j,arr[3][4]={{10,11,12,13},{20,21,22,23},{30,31,32,33}};
 int *pa[3];
 for(i=0; i<3; i++)
 pa[i]=arr[i];

 for(i=0; i<3; i++)
 {
 for(j=0; j<4; j++)
 printf("%d ",pa[i][j]);
 printf("\n");
 }
 return 0;
}
```

Output:

```
11 12 13
21 22 23
31 32 33
```

Generally these pointer arrays contain pointers to dynamically allocated memory.

## 9.20 void Pointers

We have studied that a pointer should be assigned an address of the same type as mentioned in pointer declaration. For example if we have a pointer to `int`, then it would be incorrect to assign the address of a float variable to it. These mixed conversions can be performed using explicit casts but it leads to undefined behaviour. An exception to this rule is a pointer to `void`. A pointer to `void` is a generic pointer that can point to any data type. We can assign address of any data type to a void pointer and a void pointer can be assigned to a pointer of any data type without any explicit casts. The syntax of declaration of a void pointer is-

```
id *vpt;

// Here void is a keyword and vpt is declared as a pointer of void type. Let us take some examples-
```

```
int i=2,*ip=&i;
float f=2.3,*fp=&f;
double d;
id *vp;

// =fp; /*Incorrect*/
// =ip; /*Correct*/
// =fp; /*Correct*/
// =&d; /*Correct*/
```

A void pointer can't be dereferenced simply by using indirection operator. Before dereferencing, it should be cast to appropriate pointer data type. For example if `vp` is a void pointer and it holds the address of an integer variable then we can't dereference it just by writing `*vp`. We'll have to write `*(int *)vp`, where left asterisk is the indirection operator and `(int *)` is used for typecasting. Similarly pointer arithmetic can't be performed on void pointers without typecasting.

### \*9.26 Dereferencing a void pointer\*

```
#include<stdio.h>
int main(void)
{
 int a=3;
 float b=3.4,*fp=&b;
 void *vp;
 vp=&a;
 printf("Value of a = %d\n",*(int *)vp);
 *(int *)vp=12;
 printf("Value of a = %d\n",*(int *)vp);
 vp=fp;
}
```

```
printf("Value of b = %f\n",*(float *)vp);
return 0;
}
```

Output:

```
Value of a = 3
Value of a = 12
Value of b = 3.400000
```

### \*9.27 Pointer arithmetic in void pointers\*

```
#include<stdio.h>
int main(void)
{
 int i;
 float a[4]={1.2,2.5,3.6,4.9};
 void *vp;
 vp=a;
 for(i=0; i<4; i++)
 {
 printf("%.1f\t", *(float *)vp);
 (float *)vp=(float *)vp+1; /*Can't write vp=vp+1*/
 }
 printf("\n");
 return 0;
}
```

Output:

```
1.2 2.5 3.6 4.6
```

void pointers are generally used to pass pointers to functions which have to perform same operations on different data types.

## 9.21 Dynamic Memory Allocation

The memory allocation that we have done till now was static memory allocation. The memory that could be used by the program was fixed i.e. we could not allocate or deallocate memory during the execution of program. In many applications it is not possible to predict how much memory would be needed by the program at run time. For example suppose we declare an array of integers of size 200.

```
int emp_no[200];
```

In an array, it is must to specify the size of array while declaring, so the size of this array will be fixed during runtime. Now two types of problems may occur. The first case is that the number of values to be stored is less than the size of array, so there will be wastage of memory. For example if we have to store only 50 values in the above array, then space for 150 values (600 bytes) is wasted. In second case our program fails if we want to store more values than the size of array, for example if there is need to store 205 values in the above array.

To overcome these problems we should be able to allocate and deallocate memory at run time. The process of allocating memory at the time of execution is called dynamic memory allocation. The allocation and release of this memory space can be done using library functions that are declared in `stdlib.h`. These functions allocate memory from a memory area called heap and deallocate this memory whenever not required, so that it can be used again for some other purpose.

Pointers play an important role in dynamic memory allocation because we can access the dynamically allocated memory only through pointers.

### 9.21.1 malloc()

Declaration : `void *malloc(size_t size);`

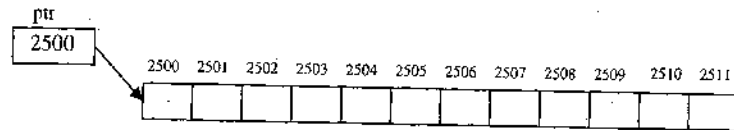
This function is used to allocate memory dynamically. The argument `size` specifies the number of bytes to be allocated. The type `size_t` is defined in `stdlib.h` and other headers as unsigned `int`. On success, `malloc()` returns a pointer to the first byte of allocated memory. The returned pointer is of type `void`, which can be type cast to appropriate type of pointer. It is generally used as-

```
ptr=(datatype *)malloc(specified size);
```

Here `ptr` is a pointer of type `datatype`, and specified size is the size in bytes required to allocated. The expression `(datatype *)` is used to typecast the pointer returned by `malloc()`. For example-

```
int *ptr;
ptr=(int *)malloc(12);
```

This allocates 12 contiguous bytes of memory space and the address of first byte is stored in the pointer variable `ptr`.



This space can hold 3 integers. The allocated memory contains garbage value. We can use `sizeof` operator to make the program portable and more readable.

```
ptr=(int *)malloc(3*sizeof(int));
```

This allocates the memory space to hold three integer values.

If there is not sufficient memory available in heap then `malloc()` returns `NULL`. So we should always check the value returned by `malloc()`.

```
ptr=(float *)malloc(10*sizeof(float));
if(ptr==NULL)
 printf("Sufficient memory not available");
```

Unlike memory allocated for variables and arrays, dynamically allocated memory has no name associated with it. So it can be accessed only through pointers. We have a pointer which points to the first byte of the allocated memory and we can access the subsequent bytes using pointer arithmetic.

*/\*P9.28 Program to understand dynamic allocation of memory\*/*

```
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 int *p,n,i;
 printf("Enter the number of integers to be entered : ");
 scanf("%d",&n);
 p=(int *)malloc(n*sizeof(int));
 if(p==NULL)
 {
 printf("Memory not available\n");
 exit(1);
 }
 for(i=0; i<n; i++)
 {
 printf("Enter an integer : ");
 scanf("%d",&p[i]);
 }
 for(i=0; i<n; i++)
 printf("%d\t",*(p+i));
 return 0;
}
```

The function `malloc()` returns a void pointer and we have studied that a void pointer can be assigned to any type of pointer without typecasting. But we have used typecasting because it is a good practice to do so and moreover it ensures compatibility with C++.

## 9.21.2 calloc()

Declaration : `void *calloc(size_t n, size_t size);`

The `calloc()` function is used to allocate multiple blocks of memory. It is similar to `malloc()` function except for two differences. The first one is that it takes two arguments. The first argument specifies the number of blocks and the second one specifies the size of each block. For example-

```
ptr=(int *)calloc(5,sizeof(int));
```

This allocates 5 blocks of memory, each block contains 4 bytes and the starting address is stored in the pointer variable `ptr`, which is of type `int`. An equivalent `malloc()` call would be-

```
ptr=(int *)malloc(5*sizeof(int));
```

The other difference between `calloc()` and `malloc()` is that the memory allocated by `malloc()` contains garbage value while the memory allocated by `calloc()` is initialized to zero. But this initialization by `calloc()` is not very reliable, so it is better to explicitly initialize the elements whenever there is need to do so.

Like `malloc()`, `calloc()` also returns `NULL` if there is not sufficient memory available in the heap.

## 9.21.3 realloc()

Declaration : `void *realloc(void *ptr, size_t newsize)`

We may want to increase or decrease the memory allocated by `malloc()` or `calloc()`. The function `realloc()` is used to change the size of the memory block. It alters the size of the memory block without losing the old data. This is known as reallocation of memory. This function takes two arguments, first is a pointer to the block of memory that was previously allocated by `malloc()` or `calloc()` and second one is the new size for that block. For example-

```
ptr=(int *)malloc(size);
```

This statement allocates the memory of the specified size and the starting address of this memory block is stored in the pointer variable `ptr`. If we want to change the size of this memory block, then we can use `realloc()` as-

```
ptr=(int *)realloc(ptr,newsize);
```

This statement allocates the memory space of `newsize` bytes, and the starting address of this memory block is stored in the pointer variable `ptr`. The `newsize` may be smaller or larger than the old size. If the `newsize` is larger, then the old data is not lost and the newly allocated bytes are uninitialized. The starting address contained in `ptr` may change if there is not sufficient memory at the old address to store all the bytes consecutively. This function moves the contents of old block into the new block and the data of the old block is not lost. On failure, `realloc()` returns `NULL` and in this case the old memory is not deallocated and it remains unchanged.

If `ptr` is a null pointer, `realloc()` behaves like `malloc()` function. If `ptr` is not a pointer returned by `malloc()`, `calloc()` or `realloc()`, the behaviour is undefined.

*/\*P9.29 realloc() Function\*/*

```
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 int i,*ptr;
 ptr=(int *)malloc(5*sizeof(int));
 if(ptr==NULL)
 {
 printf("Memory not available\n");
 exit(1);
 }
 for(i=0; i<5; i++)
 *(ptr+i)=i*2;
 ptr=(int *)realloc(ptr,9*sizeof(int)); /*Allocate memory for 4 more integers*/
 if(ptr==NULL)
 {
 printf("Memory not available\n");
 exit(1);
 }
 for(i=5; i<9; i++)
 *(ptr+i)=i*2;
 return 0;
}
```

```

 printf("Memory not available\n");
 exit(1);
}
for(i=5; i<9; i++)
 *(ptr+i)=i*10;

for(i=0; i<9; i++)
 printf("%d ", *(ptr+i));
return 0;
}

```

Output:  
0 2 4 6 8 50 60 70 80

### 9.21.4 free()

Declaration : void free(void \*p)

The dynamically allocated memory is not automatically released; it will exist till the end of program. If we have finished working with the memory allocated dynamically, it is our responsibility to release that memory so that it can be reused. The function free() is used to release the memory space allocated dynamically. The memory released by free() is made available to the heap again and can be used for some other purpose.

free(ptr);

Here ptr is a pointer variable that contains the base address of a memory block created by malloc() or calloc(). Once a memory location is freed it should not be used. We should not try to free any memory location that was not allocated by malloc(), calloc() or realloc().

When the program terminates all the memory is released automatically by the operating system but it is a good practice to free whatever has been allocated dynamically. We won't get any errors if we don't free the dynamically allocated memory, but this would lead to memory leak i.e. memory is slowly leaking away and can be reused only after the termination of program. For example consider this function-

```

void func()
{
 int *ptr;
 ptr = (int*)malloc(10*sizeof(int));
}

```

Here we have allocated memory for 10 integers through malloc(), so each time this function is called, space for 10 integers would be reserved. We know that the local variables vanish when the function terminates, and since ptr is a local pointer variable, it will be finished automatically at the termination of function. But the space allocated dynamically is not deallocated automatically, so that space remains there and can't be used, leading to memory leaks. We should free the memory space by putting a call to free() at the end of the function.

Since the memory space allocated dynamically is not released after the termination of function, it is valid to return a pointer to dynamically allocated memory. For example-

```

int *func()
{
 int *ptr;
 ptr=(int*)malloc(10*sizeof(int));
 return ptr;
}

```

Here we have allocated memory through malloc() in func(), and returned a pointer to this memory. Now the calling function receives the starting address of this memory, so it can use this memory. Note that now the call to function free() should be placed in the calling function when it has finished working with this memory. Here func() is declared as a function returning pointer. Recall that it is not valid to return address of a local variable since it vanishes after the termination of function.

## 9.22 Dynamic Arrays

The memory allocated by malloc(), calloc() and realloc() is always made up of contiguous bytes. In C there is equivalence between pointer notation and subscript notation i.e. we can apply subscripts to a pointer variable. So we can access the dynamically allocated memory through subscript notation also. We can utilize these features to create dynamic arrays whose size can vary during run time. Now we'll rewrite the program P9.28 using subscript notation.

```

/*P9.30 Program to access dynamically allocated memory as a 1d array*/
#include<stdio.h>
#include<stdlib.h>
int main(void)

```

```

{
 int *p,n,i;
 printf("Enter the number of integers to be entered : ");
 scanf("%d",&n);
 p=(int *)malloc(n*sizeof(int));
 if(p==NULL)
 {
 printf("Memory not available\n");
 exit(1);
 }
 for(i=0; i<n; i++)
 {
 printf("Enter an integer : ");
 scanf("%d",&p[i]);
 }
 for(i=0; i<n; i++)
 printf("%d\t",p[i]);
 return 0;
}

```

In this way we can simulate a 1-D array for which size is entered at execution time. Now we'll see how to create a dynamically allocated 2-D array. In the next program we have used a pointer to an array to dynamically allocate a 2-D array. Here the number of columns is fixed while the number of rows can be entered at run time.

```

/*P9.31 Program to dynamically allocate a 2-D array using pointer to an array*/
#include<stdio.h>
#include<stdlib.h>
int main(void)

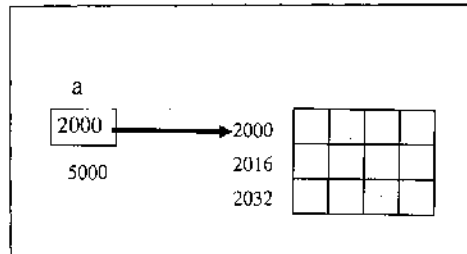
```

```

{
 int i,j,rows;
 int (*a)[4];
 printf("Enter number of rows : ");
 scanf("%d",&rows);
 a=(int(*)[4])malloc(rows*4*sizeof(int));
 for(i=0; i<rows; i++)
 for(j=0; j<4; j++)
 {
 printf("Enter a[%d][%d] : ",i,j);
 scanf("%d",&a[i][j]);
 }
 printf("The matrix is :\n");
 for(i=0; i<rows; i++)
 {
 for(j=0; j<4; j++)
 printf("%5d",a[i][j]);
 printf("\n");
 }
 free(a);
 return 0;
}

```

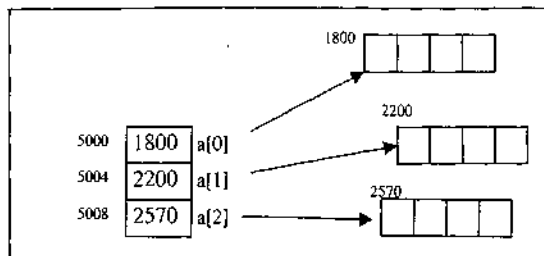
Suppose the number of rows entered is 3. The following figure shows how the dynamically allocated memory is accessed using pointer to an array. Since there are 3 rows and 4 columns, we'll allocate 48 bytes through `malloc()`, and the address returned by `malloc()` is assigned to `a`. The return value of `malloc()` is cast appropriately.



Now we'll allocate a 2-D array using array of pointers. Here the number of rows is fixed while the number of columns can be entered at run time.

```
/*P9.32 Program to dynamically allocate a 2-D array using array of pointers*/
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 int *a[3], i, j, cols;
 printf("Enter number of columns : ");
 scanf("%d", &cols);
 /*Initialize each pointer in array by address of dynamically allocated memory*/
 for(i=0; i<3; i++)
 a[i] = (int *)malloc(cols*sizeof(int));
 for(i=0; i<3; i++)
 for(j=0; j<cols; j++)
 {
 printf("Enter value for a[%d][%d] : ", i, j);
 scanf("%d", &a[i][j]);
 }
 printf("The matrix is :\n");
 for(i=0; i<3; i++)
 {
 for(j=0; j<cols; j++)
 printf("%5d", a[i][j]);
 printf("\n");
 }
 for(i=0; i<3; i++)
 free(a[i]);
 return 0;
}
```

Suppose the number of columns entered is 4. This figure shows how to dynamically allocate a 2-D array using array of pointers. In this case the rows are not allocated consecutively.



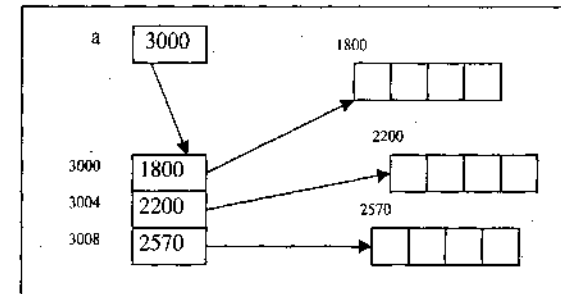
If we want to enter both the number of rows and number of columns at run time, then we can dynamically allocate the array of pointers also.

```
/*P9.33 Program to dynamically allocate a 2-D array*/
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 int **a, i, j, rows, cols;
 printf("Enter number of rows and columns : ");
 scanf("%d%d", &rows, &cols);
 /*Allocate a one dimensional array of int pointers*/
 a = (int **)malloc(rows*sizeof(int *));

 /*Allocate a one dimensional array of integers for each row pointer */
 for(i=0; i<rows; i++)
 a[i] = (int *)malloc(cols * sizeof(int));

 for(i=0; i<rows; i++)
 for(j=0; j<cols; j++)
 {
 printf("Enter a[%d][%d] : ", i, j);
 scanf("%d", &a[i][j]);
 }
 printf("The matrix is :\n");
 for(i=0; i<rows; i++)
 {
 for(j=0; j<cols; j++)
 printf("%5d", a[i][j]);
 printf("\n");
 }
 for(i=0; i<rows; i++)
 free(a[i]);
 free(a);
 return 0;
}
```

Suppose the number of rows entered is 3 and the number of columns entered is 4, then the figure for this program would be-



Here the rows are not contiguous, but we can access each element with subscripting. The size of any row can easily be increased or decreased by `realloc()`.

## 9.23 Pointers to Functions

The code of a function resides in memory and so every function has an address like all other variables in the program. We can get the address of a function by just writing the function's name without parentheses.

```
/*P9.34 Program to illustrate that every function has an address and how to access that address*/
#include<stdio.h>
```

```

void func1(),
int main(void)
{
 printf("Address of function main() is : %p\n",main);
 printf("Address of function func1() is : %p\n",func1);
 return 0;
}

void func1()
{
 printf("C in depth\n");
}

Output:
Address of function main() is : 004113C0
Address of function func1() is : 00411104

```

### 9.23.1 Declaring a Pointer to a Function

We have seen that functions have addresses, so we can have pointers that can contain these addresses and hence point to them. The syntax for declaration of a pointer to a function is-

return type (\*ptr\_name) (type1, type2, .....);

For example-

```

float (*fp)(int);
char (*func_p)(float, char);

```

Here fp is a pointer that can point to any function that returns a float value and accepts an int value as argument. Similarly func\_p is a pointer that can point to functions returning char and accepting float and char as arguments.

We can see that this declaration is similar to the declaration of a function, except that the pointer name is preceded by a \* and is enclosed in parentheses. Use of \* is obvious since we did this while declaring pointers to variables also, but why is the pointer name enclosed in parentheses. Let us remove the parentheses and see.

```
float *fp(int);
```

How would you declare a function returning a pointer to float and taking an int value? Well exactly in the same manner as above. So in this declaration fp is declared to be a function rather than a pointer, this happened because parentheses have higher precedence than \* operator. This is the reason for enclosing the pointer name inside parentheses.

Now we have learnt how to declare a pointer to a function, the next step is to assign a function's address to it.

```

float (*fp)(int, int); /*Declaring a function pointer*/
float func(int, int); /*Declaring a function*/
fp=func; /*Assign address of function func() to pointer fp*/

```

After the above assignment fp contains the address of function func(). Declaring a function is necessary before using its address anywhere in the program because without declaration the compiler will not know about this function and will generate an error.

### 9.23.2 Calling a Function through Function Pointer

Now let us see how to invoke a function using a function pointer.

```

r=func(a,b); /*Calling a function in usual way*/
r=(*fp)(a,b); /*Calling a function via function pointer*/
r=fp(a,b); /*Calling a function via function pointer, indirection operator can be omitted*/

```

The effect and result of calling a function by its name or by a function pointer is exactly the same. While using the function pointer we can omit the indirection operator as we have done in the third statement, but the second statement makes it clear to the user that a function pointer is used.

```

/*P9.35 Program to invoke a function using function pointer*/
#include<stdio.h>

```

```

float add(int, float), result;
int main(void)
{
 float (*fp)(int, float);
 float result;

 fp=add; /*Assign address of function add() to pointer fp*/

 /*Invoking a function directly using function's name*/
 result=add(5,6.6);
 printf("%f\n", result);

 /*Invoking a function indirectly by dereferencing function pointer*/
 result=(*fp)(5,6.6);
 printf("%f\n", result);
 return 0;
}

float add(int a, float b)
{
 return a+b;
}

```

```

Output:
11.600000
11.600000

```

### 9.23.3 Passing a function's address as an argument to other function

We can send the function's address as an argument to other function in the same way as we send other arguments. To receive the function's address as an argument, a formal parameter of the appropriate type should be declared. We can then invoke the function sent as an argument by dereferencing the formal pointer variable. The following program will make this point clear.

```

/*P9.36 Program to send a function's address as an argument to other function*/
#include<stdio.h>
void func(char, void(*fp)(float));
void fun1(float);
int main(void)
{
 printf("Function main() called\n");
 func('a', fun1);
 return 0;
}

void func(char b, void(*fp)(float))
{
 printf("Function func() called\n");
 (*fp)(8.5);
}

void fun1(float f)
{
 printf("Function fun1() called\n");
}

```

```

Output:
Function main() called
Function func() called
Function fun1() called

```

Here func() is a function which accepts two arguments, a char and a function pointer. This function pointer can point to any function that accepts a float and returns nothing.

fun1() is a function that accepts a float and returns nothing so we can send its address as second argument to the function func(). The function main() calls function func() while the function func() calls function fun1() indirectly using function pointer. Now we will write the same program and this time we will send a pointer that contains the address of function fun1().



```
/*P9.37 Program to pass a pointer containing function's address as an argument*/
#include<stdio.h>
void func(char,void(*fp)(float));
void fun1(float);
int main(void)
```

```
{
 void (*p)(float);
 p=fun1;

 printf("Function main() called\n");
 func('a',p);
 return 0;
}

void func(char b,void(*fp)(float)) /*Value of p stored in fp*/
{
 printf("Function func() called\n");
 (*fp)(8.5); /*Calling fun1 indirectly using pointer*/
}

void fun1(float t)
{
 printf("Function fun1() called \n");
}
```

Output:  
Function main() called  
Function func() called  
Function fun1() called

### 9.23.4 Arrays of function pointers

Function pointer can be used in applications where we don't know in advance which function will be called. In that case we can take the addresses of different functions in an array and then call the appropriate function depending on some index number.

Let us take a program and understand this concept. In this program we'll add, subtract, multiply or divide two numbers depending on user's choice. For this we will make four different functions and the addresses of these functions will be stored in an array of function pointers.

```
float add(float,int); /*Declaration of functions*/
float sub(float,int);
float mul(float,int);
float div(float,int);

float (*fp[4])(float,int); /*Declare an array of function pointers*/

fp[0]=add; /*Assigning address to elements of the array of function pointer*/
fp[1]=sub;
fp[2]=mul;
fp[3]=div;
```

Instead of the above assignment statements, we could have initialized the array as-

```
float (*fp[]) (float,int)={add,sub,mul,div};
```

Now we can see that-

```
(*fp)[0](a,b); is equivalent to add(a,b);
(*fp)[1](a,b); is equivalent to sub(a,b);
(*fp)[2](a,b); is equivalent to mul(a,b);
(*fp)[3](a,b); is equivalent to div(a,b);
```

```
/*P9.38 Array of function pointers*/
#include<stdio.h>
float add(float,int);
float sub(float,int);
float mul(float,int);
```

```
float div(float,int);
int main(void)
{
 int i,b;
 float a;
 float (*fp[]) (float,int)={add,sub,mul,div};
 char *operation[]={"Add","Subtract","Multiply","Divide"};

 printf("Enter a float and a integer : ");
 scanf("%f%d",&a,&b);

 for(i=0;i<4;i++)
 printf("%s: %f\n",operation[i],(*fp[i])(a,b));

 return 0;
}

float add(float a,int b)
{
 return a+b;
}

float sub(float a,int b)
{
 return a-b;
}

float mul(float a,int b)
{
 return a*b;
}

float div(float a,int b)
{
 return a/b;
}
```

Function pointers are generally used in compilers, interpreters and database programs.

Here is a review of the pointer declarations used in this chapter-

```
int *p; /*Pointer to int*/
int **p; /*Pointer to pointer to int*/
int p[20]; /*Array of 20 int pointers*/
int (*p)[20]; /*Pointer to an array of 20 integers*/
int *f(void); /*Function that returns an int pointer*/
int (*fp)(void); /*Pointer to a function, function returns int*/
int (*fp[4])(void); /*An array of 4 pointers to functions, each function returns int*/
int *(*fp)(void); /*Pointer to a function, function returns an int pointer*/
float *(*fp)(int,float); /*Pointer to a function, function takes two arguments of int
and float type and returns a float pointer*/
float *(*fp[4])(int,float); /*An array of 4 pointers to functions, each function takes two
arguments of int and float type and returns a float pointer*/
```

### Exercise

Find the output of the following programs. Assume stdio.h is included in all programs.

```
1. int main(void)
{
 int a=5,*ptr;
 ptr=&a;
 printf("Input a number : ");
 scanf("%d",&ptr); /*Suppose the input number is 16*/
 printf("%d %d\n",a,*ptr);
 return 0;
}
```

```

2. int main(void)
{
 int *ptr;
 printf("Enter a number : ");
 scanf("%d", ptr);
 printf("%d\n", *ptr);
 return 0;
}

3. int main(void)
{
 int arr[5], i;
 for(i=0; i<5; i++)
 printf("%p ", arr+i);
 printf("\nEnter 5 numbers : ");
 for(i=0; i<5; i++)
 scanf("%d", arr+i);
 for(i=0; i<5; i++)
 printf("%d ", *(arr+i));
 return 0;
}

4. int main(void)
{
 int i, arr[5]={25, 30, 35, 40, 45}, *p;
 p=arr;
 for(i=0; i<5; i++)
 printf("%d\t%d\t", *(p+i), p[i]);
 return 0;
}

5. int main(void)
{
 int i, arr[5]={25, 30, 35, 40, 45}, *p;
 p = &arr[4];
 for(i=0; i<5; i++)
 printf("%d\t%d\t", *(p-i), p[-i]);
 return 0;
}

6. int main(void)
{
 int i, arr[5]={25, 30, 35, 40, 55}, *p;
 for(i=0; i<5; i++)
 {
 printf("%d ", *arr);
 arr++;
 }
 return 0;
}

7. int main(void)
{
 int i, arr[5]={25, 30, 35, 40, 45}, *p=arr;
 for(i=0; i<5; i++)
 {
 (*p)++;
 printf("%d ", *p);
 p++;
 }
 return 0;
}

```

```

8. int main(void)
{
 int i, arr[5]={25, 40, 55, 70, 85}, *p=arr;
 for(i=0; i<5; i++)
 printf("%d ", *p++);
 printf("\n");
 for(i=0; i<5; i++)
 printf("%d ", *--p);
 printf("\n");
 return 0;
}

9. int main(void)
{
 int i, arr[5]={25, 40, 55, 70, 85}, *p=arr;
 for(i=0; i<8; i++)
 printf("%d ", ++*p);
 printf("\n");
 for(i=0; i<7; i++)
 printf("%d ", (*p)++);
 printf("\n");
 return 0;
}

10. int main(void)
{
 int arr[10]={25, 30, 35, 40, 55, 60, 65, 70, 85, 90}, *p;
 for(p=&arr[0]; p<arr+10; p++)
 printf("%d ", *p);
 return 0;
}

11. int main(void)
{
 int arr[10]={25, 30, 35, 40, 55, 60, 65, 70, 85, 90}, *p;
 for(p=arr+2; p<arr+8; p=p+2)
 printf("%d ", *p);
 return 0;
}

12. int main(void)
{
 int i, arr[10]={25, 30, 35, 40, 55, 60, 65, 70, 85, 90};
 int *p=arr+9;
 for(i=0; i<10; i++)
 printf("%d ", *p--);
 return 0;
}

13. int main(void)
{
 int arr[10]={25, 30, 35, 40, 55, 60, 65, 70, 85, 90}, *p;
 for(p=arr+9; p>=arr; p--)
 printf("%d ", *p);
 return 0;
}

14. int main(void)
{
 int arr[4]={10, 20, 30, 40};
 int x=100, *ptr=arr;
 printf("%p\t%d\t%d\n", ptr, *ptr, x);
}

```

```

 x=*ptr++;
 printf("%p %d %d\n",ptr,*ptr,x);
 x=**++ptr;
 printf("%p %d %d\n",ptr,*ptr,x);
 x=++*ptr;
 printf("%p %d %d\n",ptr,*ptr,x);
 x=(*ptr)++;
 printf("%p %d %d\n",ptr,*ptr,x);
 return 0;
}

15. int main(void)
{
 int x,arr[8]={11,22,33,44,55,66,77,88};
 x=(arr+2)[3];
 printf("%d\n",x);
 return 0;
}

16. int main(void)
{
 int arr[8]={11,22,33,44,55,66,77,88};
 int *p,*q;
 q=arr/2;
 p=q*2;
 printf("%d %d",*p,*q);
 return 0;
}

17. int main(void)
{
 int arr[6]={1,2,3,4,5,6};
 int *p=arr;
 printf("Size of p=%u,Size of arr=%u\n",sizeof(p),sizeof(arr));
 return 0;
}

18. int main(void)
{
 float a=5,*p,**pp;
 p=&a;
 pp=&p;
 printf("a=%f, p=%p, pp=%p\n",a,p,pp);
 a=a+1;
 p=p+1;
 pp=pp+1;
 printf("a=%f, p=%p, pp=%p\n",a,p,pp);
 return 0;
}

19. int a=5,b=10;
void change1(int *p)
{ p=&a; }
void change2(int **pp)
{ *pp=&b; }
int main(void)
{
 int x=20,*ptr=&x;
 printf("%d ",*ptr);
 change1(ptr);
 printf("%d ",*ptr);
 change2(&ptr);
}

```

```

 printf("%d\n",*ptr);
 return 0;
}

20. void func(int x,int *y);
int main(void)
{
 int a=2,b=6;
 func(a,&b);
 printf("a=%d, b=%d\n",a,b);
 return 0;
}
void func(int x,int *y)
{
 int temp;
 temp=x;
 x=*y;
 *y=temp;
}

21. void func(int **pp);
int main(void)
{
 int *ptr;
 func(&ptr);
 printf("%d\n",*ptr);
 return 0;
}
void func(int **pp)
{
 int num=10;
 *pp=#
}

22. void func(int x,int y);
int main(void)
{
 int a=5,b=8;
 func(a,b);
 printf("a=%d,b=%d\n",a,b);
 return 0;
}
void func(int x,int y)
{
 int temp;
 temp=*&x, *(&x)=*&y, *(&y)=temp;
}

23. void func1(int *ptr);
void func2(int **pptr);
int main(void)
{
 int arr[5]={1,2,3,4,5};
 int *p = arr;
 printf("p = %p,\t",p);
 func1(p);
 printf("p = %p,\t",p);
 func2(&p);
 printf("p = %p\n",p);
 return 0;
}
void func1(int *ptr)
{
}

```

```

 ptr++;
}
void func2(int **pptr)
{
 (*pptr)++;
}

```

24. void func(int a[10]);  
int main(void)

```

{
 int arr[10];
 func(arr);
 return 0;
}

void func(int a[10])
{
 int b[10], x=5;
 a=&x;
 b=&x;
}

```

25. int main(void)

```

{
 int arr[3][4];
 printf("%p\t", arr);
 printf("%p\t", arr[0]);
 printf("%p\n", &arr[0][0]);
 printf("%u\t", sizeof(arr));
 printf("%u\t", sizeof(arr[0]));
 printf("%u\n", sizeof(arr[0][0]));
 return 0;
}

```

26. int main(void)

```

{
 int arr[3][4][5];
 printf("%p\t", arr);
 printf("%p\t", arr[0]);
 printf("%p\t", arr[0][0]);
 printf("%p\n", &arr[0][0][0]);
 printf("%u\t", sizeof(arr));
 printf("%u\t", sizeof(arr[0]));
 printf("%u\t", sizeof(arr[0][0]));
 printf("%u\n", sizeof(arr[0][0][0]));
 return 0;
}

```

27. void func(int a[]);

```

int main(void)
{
 int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
 func(arr+3);
 return 0;
}

```

void func(int a[])

```

{
 int i;
 for(i=0; a[i]!=8; i++)
 printf("%d ", a[i]);
}

```

28. void swap(int \*b, int \*c);

int main(void)

```

{
 int i, j;
 int arr[10] = {3, 2, 4, 1, 5, 9, 8, 10, 7, 6};
 for(i=0; i<10; i++)
 for(j=0; j<10-i-1; j++)
 if(*(arr+j) > *(arr+j+1))
 swap(arr+j, arr+j+1);
 for(i=0; i<10; i++)
 printf("%d\t", arr[i]);
 printf("\n");
 return 0;
}

```

void swap(int \*b, int \*c)

```

{
 int temp;
 temp=*b, *b=*c, *c=temp;
}

```

29. int main(void)

```

{
 int i, arr[3][4] = {{10, 11, 12, 13}, {20, 21, 22, 23}, {30, 31, 32, 33}};
 int *pa[3];
 int (*p)[4];
 p=arr;
 for(i=0; i<3; i++)
 pa[i] = arr[i];
 printf("%d %d %d\n", pa[0][0], pa[0][1], pa[2][13]);
 printf("%d %d %d\n", p[0][0], p[0][1], p[2][3]);
 return 0;
}

```

30. #include<stdlib.h>

```

int *func1(void);
int *func2(void);
int main(void)
{
 int *ptr1, *ptr2;
 ptr1=func1();
 ptr2=func2();
 printf("%d %d\n", *ptr1, *ptr2);
 free(ptr2);
 return 0;
}

```

int \*func1(void)

```

{
 int a=8, *p=&a;
 return p;
}

```

int \*func2(void)

```

{
 int *p;
 p=(int *)malloc(sizeof(int));
 *p=9;
 return p;
}

```

31. int main(void)

```

{
 int i, arr[3][4] = {{10, 11, 12, 13}, {20, 21, 22, 23}, {30, 31, 32, 33}};
 int *p=&arr[0][0];
 for(i=0; i<12; i++)
 printf("%d ", p[i]);
}

```

```
printf("\n");
return 0;
}
```

```
32. int main(void)
{
 int a[2][3];
 a[1][2]=9;
 printf("%d\n",a[1,2]);
 return 0;
}
```

```
33. int main(void)
{
 int a[5]={1},b[5]={1};
 if(a==b)
 printf("Same\n");
 else
 printf("Different\n");
 return 0;
}
```

There is no separate data type for strings in C, they are treated as arrays of type `char`. A character array is a string if it ends with a null character (`'\0'`). This null character is an escape sequence with ASCII value 0, and is different from the ASCII character `'0'`. Strings are generally used to store and manipulate data in text form like words or sentences.

### 10.1 String Constant (String Literal)

A string constant is a sequence of characters enclosed in double quotes, and is also called a literal. The double quotes are not a part of the string. Some examples of string constants are-

```
"y"
"Taj Mahal"
"2345"
"Subhash Chandra Bose was a great leader"
"" (Null string, contains only '\0')
"My age is %d and height is %f\n" (Control string used in printf)
```

Whenever a string constant is written anywhere in a program, it is stored somewhere in memory as an array of characters terminated by a null character (`'\0'`). The string constant itself becomes a pointer to the first character in the array. For example the string "Taj Mahal" will be stored in memory as-

| 1000 | 1001 | 1002 | 1003 | 1004 | 1005 | 1006 | 1007 | 1008 | 1009 |
|------|------|------|------|------|------|------|------|------|------|
| T    | a    | j    |      | M    | a    | h    | a    | l    | \0   |

Each character occupies one byte and compiler automatically inserts the null character at the end. The string constant "Taj Mahal" is actually a pointer to the character `'T'`. So whenever a string constant is used in the program it is replaced by a pointer pointing to the string.

If we have a pointer variable of type `char *`, we can assign the address of this string constant to it as-

```
char *p="Taj mahal";
```

Similarly when we write-

```
printf("Subhash Chandra Bose\n");
```

then actually a pointer to character (`char *`) is passed to the `printf()` function.

Identical string constants used in a program need not be distinct. It depends on the compiler whether they are stored separately or as same string i.e. they can be at same address or at different addresses.

Now let us see what the expression `"software" [4]` represents. According to the equivalence of pointer and subscript notations, this expression is equivalent to `*("software"+4)`. We know that `"software"` represents the address of first character i.e. `'s'`. On adding 4 to this address we get the address of character `'w'`, and on dereferencing it we get the character `'w'`.

In some compilers we can change the string constant by storing its address in a pointer, but it is not a good to do so and the behaviour of such a program is undefined. So we should never attempt to alter the string constants.

We have studied that a string constant gives the address of first character in it, but there is an exception to this rule; when the string constant is used as an initializer for a character array then it does not represent any address and it is not stored anywhere in memory. For example-

```
char arr[]="Deep";
```

Here the string constant "Deep" is not stored in memory and thus does not represent any address. This declaration statement reserves space for array arr and is equivalent to-

```
char arr[]={ 'D', 'e', 'e', 'p', '\0' };
```

If we have a statement like this -

```
char *ptr="Deep";
```

Space is allocated for pointer variable ptr, string constant "Deep" is stored separately in memory and the pointer variable ptr is initialized with the starting address of "Deep".

Note that the 'b' and "b" are different. 'b' is a character constant which represents the ASCII value of character 'b' while "b" is a string constant which consists of character 'b' and null character '\0'.

Inside a string constant, the backslash is considered as an escape character, so if there is a need to include backslash character within a string constant then it should be preceded by another backslash. If we want to include double quotes inside string constant, then it should be preceded by a backslash. For example consider these two printf statements-

```
printf("good\\bad");
printf("I love \"C\" programming");
```

The output of these 2 statements would be-

```
good\bad
I love "C" programming
```

The length of a string is not limited to a line only, it can be continued by adding a backslash at the end of the line.

If the string constants are placed adjacent to each other then the compiler concatenates them and places a single null character at the end of the concatenated string. For example these two string constants will be concatenated by the compiler as-

```
'Red "Fort" → "Red Fort"
```

## 10.2 String Variables

A string variable is a 1-d array of ASCII characters terminated by a null character. To create a string variable we need to declare a character array with size sufficient to hold all the characters of the string including null character.

```
char str[]={ 'N', 'e', 'w', ' ', 'D', 'e', 'l', 'h', 'i', '\0' };
```

We may also initialize it as-

```
char str[]="New Delhi";
```

This initialization is same as the previous one and in this case the compiler automatically inserts the null character at the end. Note that here the string constant does not represent an address. The array str will be stored in memory as-

|        |        |        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| 4500   | 4501   | 4502   | 4503   | 4504   | 4505   | 4506   | 4507   | 4508   | 4509   |
| N      | e      | w      |        | D      | e      | l      | h      | i      | \0     |
| str[0] | str[1] | str[2] | str[3] | str[4] | str[5] | str[6] | str[7] | str[8] | str[9] |

Here we have not specified the size of array, but if we specify it then we should take care that array should be large enough to hold all the characters including the null character.

```
/*P10.1 Program to print characters of a string and address of each character*/
#include<stdio.h>
int main(void)
```

```
char str[]="India";
int i;
for(i=0; str[i]!='\0'; i++)
{
 printf("Character = %c\t",str[i]);
 printf("Address = %p\n",&str[i]);
}
return 0;
```

Output:

```
Character=i Address=0012FED0
Character=n Address=0012FED1
Character=d Address=0012FED2
Character=i Address=0012FED3
Character=a Address=0012FED4
```

```
/*P10.2 Program to print the address and characters of a string using pointer*/
```

```
#include<stdio.h>
int main(void)
{
 char str[]="India";
 char *p;
 p=str;
 while(*p!='\0')
 {
 printf("Character = %c\t",*p);
 printf("Address = %p\n",p);
 p++;
 }
 return 0;
```

The output of this program is same as that of previous program.

Here p is a pointer variable which holds the base address of array str[]. Incrementing this pointer by 1 gives the address of next element of character array str[], so by continually incrementing and dereferencing this pointer we can print all the elements of the string. This procedure is similar to that applied in other arrays, except that here the loop terminates when the character '\0' is encountered which signifies the end of string.

We can enter and print strings using %s specification in the control string of printf() and scanf().

```
/*P10.3 Input and output strings using scanf() and printf()*/
```

```
#include<stdio.h>
int main(void)
{
 char name[40];
 printf("Enter a name : ");
 scanf("%s",name);
 printf("%s %s\n",name,"Srivastava");
 return 0;
```

```
1st run-
Enter a name: Deepali
Deepali Srivastava
```

```
2nd run-
Enter a name : Suresh Kumar
Suresh Srivastava
```

The printf() takes the base address of string and continues to display the characters until it encounters the character '\0'.

When we enter a string using %s, the null character is automatically stored at the end of array. We haven't used & sign in the scanf() since the name of the array is address of the array. In the 2nd run when we entered a string with space we could not get the required result. This is because scanf() stops reading as soon as it

encounters a whitespace. For entering strings with whitespaces we can use the function `gets()`. It stops reading only when it encounters a newline and replaces this newline by the null character. We have another function `puts()` which can output a string and replaces the null character by a newline.

*/\*P10.4 Input and output strings using gets() and puts().\*/*

```
#include<stdio.h>
int main(void)
{
 char name[20];
 printf("Enter a name : ");
 gets(name);
 printf("Entered name is : ");
 puts(name);
 return 0;
}
```

Output:

```
Enter a name : Suresh Kumar
Entered name is : Suresh Kumar
```

The function `gets()` can't check the buffer space, if more input is provided than the size of the array then it will overwrite locations after the end of the array. To avoid this we can use the function `fgets()` which is discussed in the chapter on files.

```
fgets(name, sizeof(name), stdin);
```

## 10.3 String Library Functions

There are several library functions used to manipulate strings and the prototypes for these functions are in header file `string.h`. In this section we will discuss how to use some of the most commonly used string functions like `strlen()`, `strcpy()`, `strcmp()`, `strcat()` and see how these functions are created.

### 10.3.1 strlen()

Declaration : `size_t strlen(char const *string);`

This function returns the length of the string i.e. the number of characters in the string excluding the terminating null character. The type `size_t` is defined in `stddef.h` and is an unsigned integer type. It accepts a single argument, which is pointer to the first character of the string. For example `strlen("suresh")` returns the value 6. Similarly if `s1` is an array that contains the name "deepali" then `strlen(s1)` returns the value 7.

*/\*P10.5 strlen() function\*/*

```
#include<stdio.h>
#include<string.h>
int main(void)
{
 char str[50];
 printf("Enter a string : ");
 gets(str);
 printf("Length of the string is : %u\n", strlen(str));
 return 0;
}
```

Output:

```
Enter a string : CinDepth
Length of the string is : 8
```

Let us see how we can create this function.

Array version-

```
unsigned int astrlen(char str[])
{
 int i=0;
 while(str[i]!='\0')
 i++;
 return i;
}
```

Pointer version-

```
unsigned int pstrlen(char *str)
{
 char *p=str;
 while(*p!='\0')
 p++;
 return p-str;
}
```

We can write the while loops as `while(*str[i])` and `while(*p)` because the ASCII value of `'\0'` is 0 which is considered false in C.

### 10.3.2 strcmp()

Declaration : `int strcmp(const char *s1, const char *s2);`

This function is used for lexicographic comparison of two strings. If the two strings match, `strcmp()` returns a value 0, otherwise it returns a non-zero value. It compares the strings character by character, and the comparison stops when either the end of string is reached or the corresponding characters in the two strings are not same.

`strcmp(s1, s2)` returns a value-

```
<0 when s1 < s2
=0 when s1 == s2
>0 when s1 > s2
```

The exact value returned in case of dissimilar strings is not defined. We only know that if `s1 < s2` then a negative value will be returned and if `s1 > s2` then a positive value will be returned.

*/\*P10.6 Program to understand the work of strcmp() function\*/*

```
#include<stdio.h>
#include<string.h>
int main(void)
{
 char str1[10], str2[10];
 printf("Enter two strings : ");
 gets(str1);
 gets(str2);
 if(strcmp(str1, str2)==0)
 printf("Strings are same\n");
 else
 printf("Strings are not same\n");
 return 0;
}
```

Output:

```
Enter two strings : Bangalore
Mangalore
Strings are not same
```

Let us see how we can create this function.

Array version-

```
int astrcmp(char str1[], char str2[])
{
 int i;
 for(i=0; str1[i]==str2[i]; i++)
 if(str1[i]!='\0')
 return 0;
 return (str1[i]-str2[i]);
}
```

Pointer version-

```
int pstrcmp(char *str1, char *str2)
```

```

while(*str1 == *str2)
{
 if(*str1 == '\0')
 return 0;
 str1++;
 str2++;
}
return(*str1 - *str2);

```

### 10.3.3 strcpy()

Declaration : char \*strcpy(char \*s1, const char s2);

This function is used for copying one string to another string. The call strcpy(str1, str2) copies str2 to str1 including the null character. Here str2 is the source string and str1 is the destination string. The old contents of the destination string str1 are lost. The function returns a pointer to destination string str1.

The destination string should be a character array or a char pointer initialized to a character array or a char pointer initialized to dynamically allocated memory. It can't be a string constant because it has to be modified. The function calls like strcpy("New", str1) or strcpy("New", "York") are wrong because "New" is a string constant and so we can't overwrite it.

The programmer should ensure that the destination array has enough space to hold the second string. If the length of the source string is more than the size of destination array, all the extra characters will be written in the memory locations following the array.

/\*P10.7 strcpy() function\*/

```

#include<stdio.h>
#include<string.h>
int main(void)
{
 char str1[10], str2[10];
 printf("Enter a string : ");
 scanf("%s", str2);
 strcpy(str1, str2);
 printf("First string : %s \t Second string : %s\n", str1, str2);
 strcpy(str1, "Delhi");
 strcpy(str2, "Bangalore");
 printf("First string : %s \t Second string : %s\n", str1, str2);
 return 0;
}

```

Output:

```

Enter a string : Chennai
First string : Chennai Second string : Chennai
First string : Delhi Second string : Bangalore

```

The result is undefined if the two string arguments overlap.

Array version-

```

char *aststrcpy(char str1[], char str2[])
{
 int i=0;
 while((str1[i]=str2[i])!='\0')
 i++;
 return str1;
}

```

This function can be written as-

```

char *aststrcpy(char str1[], char str2[])
{
 int i=0;
 while((str1[i++] = str2[i]))
 ;
 return str1;
}

```

Pointer version-

```

char *pstrcpy(char *str1, char *str2)
{
 while((*str1 = *str2) != '\0')
 {
 str1++;
 str2++;
 }
 return str1;
}

```

The pointers str1 and str2 can be incremented in the while condition and there is no need for mentioning '\0' in the condition as its value is 0. So the function can be written concisely as-

```

char *pstrcpy(char *str1, char *str2)
{
 while(*str1++ = *str2++)
 ;
 return str1;
}

```

### 10.3.4 strcat()

Declaration : char \*strcat(char \*str1, const char \*str2);

This function is used to append a copy of a string at the end of other string. If first string is "Purva" and second string is "Belmont" then after using this function the first string becomes "PurvaBelmont". The null character from str1 is removed and str2 is added at the end of str1. The second string str2 remains unaffected. A pointer to the first string str1 is returned by the function. The result is undefined if both the strings overlap. Like strcpy(), here also the programmer needs to make sure that there is enough space in the first string.

/\*P10.8 strcat() function\*/

```

#include<stdio.h>
#include<string.h>
int main(void)
{
 char str1[20], str2[20];
 printf("Enter two strings : ");
 gets(str1);
 gets(str2);
 strcat(str1, str2);
 printf("First string : %s \t Second string : %s\n", str1, str2);
 strcat(str1, "_one");
 printf("Now first string is : %s \n", str1);
 return 0;
}

```

Output:

```

Enter two strings : data
base
First string : database Second string : base
Now first string is : database_one

```

### Creation of this function-

Array version-

```

char *astrcat(char str1[], char str2[])
{
 int i=0, j=0;
 while(str1[i]!='\0')
 i++;
 while(str1[i++] = str2[j++]) /*go to the end of first string*/
 ;
 return str1;
}

```



Pointer version-

```
char *pstrcat(char *str1, char *str2)
{
 char *p=str1;
 while(*p!='\0')
 p++;
 while(*p++ = *str2++)
 ;
 return str1;
}
```

The functions `strcat()` and `strcpy()` return a pointer to the first string so they can be nested.

```
/*P10.9 Nesting of strcat() and strcpy() functions */
#include<stdio.h>
#include<string.h>
int main(void)
{
 char str1[30]="Subhash ";
 char str2[10]="Chandra ";
 char str3[20];
 strcat(strcat(str1, str2), "Bose");
 printf("str1 - %s\n", str1);
 strcpy(str3, "Dev");
 strcat(strcat(str3, "anshi"));
 printf("str3 - %s\n", str3);
 return 0;
}
```

Output:  
str1 - Subhash Chandra Bose  
str3 - Devanshi

## 10.4 String Pointers

We can take a char pointer and initialize it with a string constant. For example-

```
char *ptr="Chennai";
```

Here `ptr` is a char pointer which points to the first character of the string constant "Chennai" i.e. `ptr` contains the base address of this string constant.

Let us compare the strings defined as arrays and strings defined as pointers.

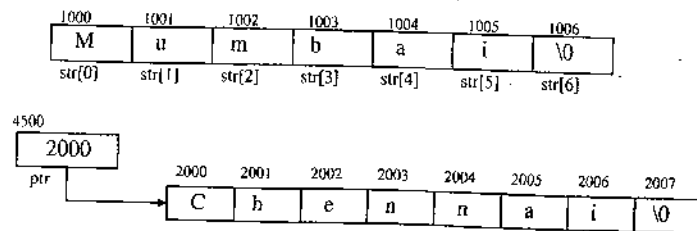
```
char str1="Mumbai";
char *ptr="Chennai";
```

In the array form, initialization is a short form for-

```
char str1={'M','u','m','b','a','i','\0'};
```

while in pointer form, address of string constant is assigned to the pointer variable.

Now let us see how they are represented in memory.



Here `str` is an array of characters and 7 bytes are reserved for it. Since `str` is the name of an array, it is a constant pointer which will always point to the first element of array. The elements of array are initialized with

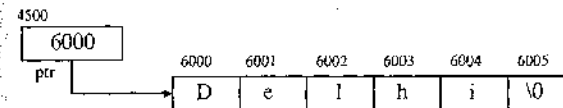
the characters of the string. Note that we had mentioned before that when a string constant appears as array initializer, it does not return an address.

In the second case, the string constant "Chennai" is stored somewhere in memory with 8 consecutive bytes reserved for it. The string constant returns the address of the first character of the string that is assigned to the pointer variable `ptr`. So in this case total 12 bytes are reserved, 4 bytes for the pointer variable and 8 bytes for the string.

The main difference is that `str` is a constant pointer and will always contain address 1000 while `ptr` is a pointer variable and may contain any other address. So string assignments are valid for pointers while they are invalid for strings defined as arrays.

```
str="Bombay"; /*Invalid*/
ptr="Delhi"; /*Valid*/
```

We can assign string of any length to `ptr`. That string constant will be stored somewhere and its address will be assigned to `ptr`.



We can assign a different string to `str` by `scanf()`, `gets()`, `fgets()`, `strcpy()` or by assigning characters.

```
strcpy(str, "Bombay");
scanf("%s", str);
gets(str);
str[0]='B'; str[1]='o'; str[1]='m'; str[1]='b'; str[1]='a'; str[1]='y'; str[1]='\0';
```

We have studied earlier that trying to change a string constant leads to undefined behaviour. So the following types of statements should not be written.

```
char *ptr="Bareilly";
ptr[0]='D'; /*Wrong*/
scanf("%d", ptr); /*Wrong*/
strcpy(ptr, "Bareilly"); /*Wrong*/
```

We can't use `scanf()`, `gets()` or `fgets()` with an uninitialized pointer since it contains garbage value and does not point to any valid location..

```
char *ptr;
scanf("%d", ptr); /*Wrong*/
gets(ptr); /*Wrong*/
```

First we should allocate memory through `malloc()` and make this pointer point to that region of memory.

```
ptr=(char *)malloc(20);
scanf("%s", ptr); /*Valid*/
```

```
/*P10.10*/
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 char *str;
 str=(char *)malloc(10);
 printf("Enter a string: ");
 scanf("%s", str);
 printf("String is: %s\n", str);
 return 0;
}
```

Output:  
Enter a string: Carmel  
String is: Carmel

## 10.5 Array of Strings or Two Dimensional Array of Characters

Strings are character arrays so array of strings means array of character type arrays i.e. a two dimensional array of characters. Suppose we declare and initialize a two-dimensional array of characters as-

```
char arr[5][10]={
 "white",
 "red",
 "green",
 "yellow",
 "blue"
};
```

This initialization is equivalent to

```
char arr[5][10]={
 {'w','h','i','t','e','\0'},
 {'r','e','d','\0'},
 {'g','r','e','e','n','\0'},
 {'y','e','l','l','o','w','\0'},
 {'b','l','u','e','\0'}
};
```

Here first subscript of array denotes number of strings in the array and second subscript denotes the maximum length that each string can have. The space reserved for this 2-d array is 50 bytes.

Here the name of 2-D array is arr and it gives the base address of the array i.e. it gives the address of first string.

```
arr[0] represents 0th string, points to 0th character of 0th string
arr[1] represents 1st string, points to 0th character of 1st string
arr[i] represents ith string, points to 0th character of ith string
arr[i][j] represents jth character in ith string
```

So we see that if we want to access individual characters in the string we use two subscripts and if we want to access the strings we use a single subscript.

|      |   |   |   |    |    |    |    |  |  |  |      |
|------|---|---|---|----|----|----|----|--|--|--|------|
| 2000 | w | h | i | t  | e  | \0 |    |  |  |  | 2009 |
| 2010 | r | e | d | \0 |    |    |    |  |  |  | 2019 |
| 2020 | g | r | e | n  | \0 |    |    |  |  |  | 2029 |
| 2030 | y | e | l | l  | o  | w  | \0 |  |  |  | 2039 |
| 2040 | b | l | u | e  | \0 |    |    |  |  |  | 2049 |

This is the internal storage representation of array of strings. 2000 is the base address of the first string. Similarly, 2010 is the base address of the second string. Here 10 bytes are reserved in memory for each string. We can see that first string takes only 6 bytes, so 4 bytes are wasted. Similarly 2<sup>nd</sup> string takes only 4 bytes and 6 bytes are wasted. The total number of bytes occupied by the array is 50 while the strings use only 28 bytes so 22 bytes of memory is wasted. Now we will see through a program how to print these strings.

```
/*P10.11 Program to print the strings of the two-dimensional character array*/
#include<stdio.h>
#define N 5
#define LEN 10
int main(void)
{
 char arr[N][LEN]={
 "white",
 "red",
 "green",
 "yellow",
 "blue"
 };
}
```

```
 "blue"
};

int i;
for(i=0; i<N; i++)
{
 printf("String = %s\t", arr[i]);
 printf("Address of string = %p\n", arr[i]);
}

return 0;
```

Output:

```
String = white Address of string = 0012FEA4
String = red Address of string = 0012FEAE
String = green Address of string = 0012FEB8
String = yellow Address of string = 0012FEC2
String = blue Address of string = 0012FECC
```

In the above program we have initialized the 2-D array with strings. Suppose we don't initialize it at the time of declaration and try to assign strings to it afterwards like this-

```
arr[0]="white"; /*Invalid*/
arr[1]="red"; /*Invalid*/
```

This will give an error 'lvalue required'. Similarly we cannot write

```
arr[0]=arr[1]; /*Invalid*/
```

If we want to assign values to strings we'll have to use strcpy() or scanf() function.

```
strcpy(arr[0], "white"); /*Valid*/
scanf("%s", arr[0]); /*Valid*/
strcpy(arr[0], arr[1]); /*Valid*/
```

Now we will take a program to sort strings alphabetically using the selection sort technique.

/\*P10.12 Program to sort array of strings\*/

```
#include<stdio.h>
#include<string.h>
#define N 5
#define LEN 10
int main(void)
{
 char arr[N][LEN]={
 "white",
 "red",
 "green",
 "yellow",
 "blue"
 };

 char temp[10];
 int i, j;
 printf("Before sorting :\n");
 for(i=0; i<N; i++)
 printf("%s\t", arr[i]);
 printf("\n");
 for(i=0; i<N; i++)
 for(j=i+1; j<N; j++)
 if(strcmp(arr[i], arr[j]) > 0)
 {
 strcpy(temp, arr[i]);
 strcpy(arr[i], arr[j]);
 strcpy(arr[j], temp);
 }

 printf("After sorting :\n");
 for(i=0; i<N; i++)
 printf("%s\t", arr[i]);
 return 0;
```

```

}
Output:
Before sorting :
white red green yellow blue
After sorting :
blue green red white yellow

```

The internal representation of strings after sorting is-

|      |   |   |   |    |    |    |    |  |  |      |
|------|---|---|---|----|----|----|----|--|--|------|
| 2000 | b | l | u | e  | \0 |    |    |  |  | 2009 |
| 2010 | g | r | e | e  | n  | \0 |    |  |  | 2019 |
| 2020 | r | e | d | \0 |    |    |    |  |  | 2029 |
| 2030 | w | h | i | t  | e  | \0 |    |  |  | 2039 |
| 2040 | y | e | l | l  | o  | w  | \0 |  |  | 2049 |

Internal representation after sorting

Here the sorting is not very efficient since each time the whole string is being copied.

## 10.6 Array of Pointers to Strings

We have already studied about array of pointers in the previous chapter. Array of pointers to strings is an array of char pointers in which each pointer points to the first character of a string i.e. each element of this array contains the base address of a string. Let us take an example and see how this array can be declared and initialized.

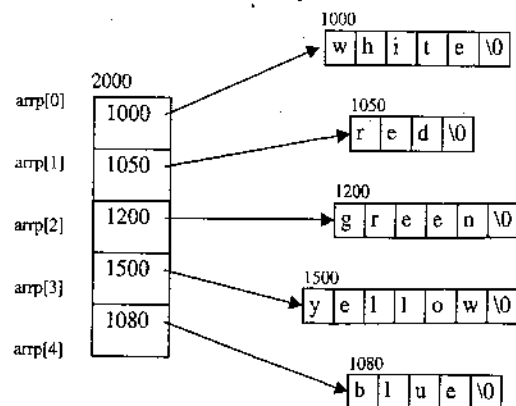
```

char *arrp[]={
 "white",
 "red",
 "green",
 "yellow",
 "blue"
};

```

Here `arrp` is an array of pointers to strings. We have not specified the size of array, so the size is determined by the number of initializers, which are string constants here. `arrp[0]` contains the base address of string "white", similarly `arrp[1]` contains the base address of string "red".

Now let us see how these strings are stored in memory-



Here all strings occupy 28 bytes and 20 bytes are occupied by the array of pointers. So the total bytes occupied are 48 bytes. Now let us take a program and see how to print these strings.

```

//10.13 Array of pointers to strings*/
#include<stdio.h>
int main(void)
{
 int i;
 char *arrp[]={
 "white",
 "red",
 "green",
 "yellow",
 "blue"
 };
 for(i=0; i<5; i++)
 {
 printf("String : %s\n", arrp[i]);
 printf("Address of string : %p\n", arrp[i]);
 printf("Address of string is stored at : %p\n", arrp+i);
 }
 return 0;
}

```

Output:

```

String : white
Address of string : 00424098
Address of string is stored at : 0012FEB8
String : red
Address of string : 00424094
Address of string is stored at : 0012FEB8
String : green
Address of string : 0042408C
Address of string is stored at : 0012FEC0
String : yellow
Address of string : 00424084
Address of string is stored at : 0012FEC4
String : blue
Address of string : 0042407C
Address of string is stored at : 0012FEC8

```

Instead of initializing we could have assigned strings after declaration as-

```

arrp[0]="white"; /*Valid*/
arrp[1]="red"; /*Valid*/

```

Remember this type of assignment was invalid when we used 2-D array. Here it is valid because `arrp[0]` is a pointer variable and it can be assigned address of any string.

Now let us compare array of strings and array of pointers to strings. Earlier we had studied the differences between strings declared as pointers and strings declared as arrays. Here also we have same differences that are illustrated by this program.

//10.14 Program to show the differences between array of strings and array of pointers to strings\*/

```

#include<stdio.h>
#include<string.h>
int main(void)
{
 char arr[5][10];
 char *arrp[5];

 arr[0]="January"; /*Invalid*/
 arrp[0]="January"; /*Valid*/

 strcpy(arr[1], "February"); /*Valid*/
 strcpy(arrp[1], "February"); /*Invalid, arrp[1] not initialized*/
}

```

```

scanf("%s",arr[2]); /*Valid*/
scanf("%s",arrp[2]); /*Invalid, arrp[2] not initialized*/

arrp[3]=(char *)malloc(10);
strcpy(arrp[3],"March"); /*Valid*/

arrp[4]=(char *)malloc(10);
scanf("%s",arrp[4]); /*Valid*/
return 0;

```

We have seen that we can't input strings directly in an array of pointers using `scanf()`. We have to first allocate memory through `malloc()`. Here is a program in which we first enter the string and then allocate memory required for that string. This allocated memory is pointed by an element of array of pointers.

```

/*P10.15*/
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main(void)
{
 char *arrp[10],str[20];
 int i;
 for(i=0; i<10; i++)
 {
 printf("Enter string %d : ",i+1);
 gets(str);
 /*Now allocate memory sufficient to hold the string*/
 arrp[i]=(char *)malloc(strlen(str)+1);
 strcpy(arrp[i],str);
 }
 for(i=0; i<10; i++)
 printf("%s\t",arrp[i]);
 printf("\n");
 for(i=0; i<10; i++)
 free(arrp[i]);
 return 0;
}

```

If we want to enter the number of strings at run time, we can allocate the array of pointers dynamically as we have done in program P9.33.

Array of pointers is also useful in grouping together logically related data items. For example we can take the names of months in an array and then access them by appropriate subscript.

```

/*P10.16 Program to input a date and print the month*/
#include<stdio.h>
int main(void)
{
 int d,m,y;
 char *months[]={ "January", "February", "March", "April", "May", "June",
 "July", "August", "September", "October", "November",
 "December" };
 printf("Enter date (dd/mm/yyyy): ");
 scanf("%d/%d/%d",&d,&m,&y);
 printf("%d %s %d\n",d,months[m-1],y);
 return 0;
}

```

Output:  
Enter date : 2/5/2002  
2 May 2002

We could do this by `switch` statement also but using array of pointers makes the code concise. The next program sorts the strings represented by array of pointers using selection sort technique.

/\*P10.17 Program to sort strings represented by array of pointers\*/

```

#include<stdio.h>
#include<string.h>
#define N 5
int main(void)
{
 char *arrp[N]={"white","red","green","yellow","blue"};
 int i,j;
 char *temp;

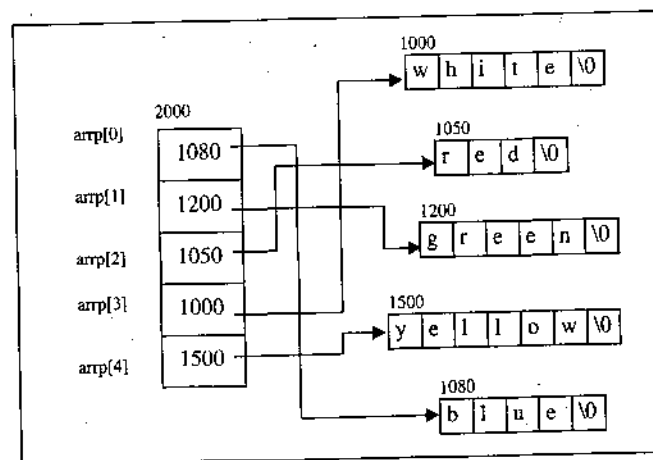
 printf("Before sorting : \n");
 for(i=0; i<N; i++)
 printf("%s\t",arrp[i]);
 printf("\n");

 for(i=0; i<N; i++)
 for(j=i+1; j<N; j++)
 if(strcmp(arrp[i],arrp[j])>0)
 {
 temp=arrp[i];
 arrp[i]=arrp[j];
 arrp[j]=temp;
 }

 printf("After sorting : \n");
 for(i=0; i<N; i++)
 printf("%s\t",arrp[i]);
 printf("\n");
 return 0;
}

```

Output:  
Before sorting :  
white red green yellow blue  
After sorting :  
blue green red white yellow



Here sorting is efficient since only pointers are being exchanged.

## 10.7 String Library Functions

### 10.7.1 `strncpy()`

Declaration : `char *strncpy(char *s1, const char *s2, size_t n);`

This function copies exactly `n` characters from source string `s2` to the destination string `s1`. It returns the value of `s1`. If length of source string `s2` is less than `n`, then null characters are added to the destination string till all `n` characters have been written. For example if `n` is 10 and length of source string `s2` is 6 then 4 null characters are written at the end. If length of source string is more than or equal to `n`, then `n` characters will be written to destination string and the null character will not be added at the end. In this case the resultant destination string is not null terminated so it can't be used with `printf()`, `strlen()` and other string functions. The behavior of this function is undefined if source and destination strings overlap.

```
/*P10.18*/
#include<stdio.h>
#include<string.h>
int main(void)
{
 char str1[10];
 strncpy(str1, "Departmental", 6);
 str1[6]='\0';
 printf("%s\n", str1);
 strncpy(str1, "Dep", 6);
 printf("%s\n", str1);
 return 0;
}
```

Output:  
Depart  
Dep

### 10.7.2 strncat()

Declaration : `char *strncat(char *str1, const char *str2, size_t n);`

This function is same as `strcat()` but it concatenates only a portion of a string to another string. The null character from `str1` is removed and `n` characters of `str2` are appended to `str1`, and a null character is added at the end of `str1`. This function returns the value of `str1`.

```
/*P10.19*/
#include<stdio.h>
#include<string.h>
int main(void)
{
 char str1[15]="ABC";
 strncat(str1, "DEFGHIJ", 4);
 printf("%s\n", str1);
 return 0;
}
```

Output:  
ABCDEFG

### 10.7.3 strncmp()

Declaration : `int *strncmp(const char *arr1, const char *arr2, size_t length);`

This function is similar to `strcmp()` but it compares characters of strings for a specified length.

```
/*P10.20*/
#include<stdio.h>
#include<string.h>
int main(void)
{
 if (strncmp("Deepali", "DeePanjali", 4)==0)
 printf("Same\n");
 else
 printf("Different\n");
}
```

return 0;

Output:  
Same

### 10.7.4 strchr() and strrchr()

Declaration : `char *strchr(const char *str, int ch);`  
Declaration : `char *strrchr(const char *str, int ch);`

`strchr()` returns a pointer to the first(leftmost) occurrence of the character `ch` in the string `str`. If the character is not present in the string then it returns NULL.

`strrchr()` returns a pointer to the last(rightmost) occurrence of the character `ch` in the string pointed to by `str`. If the character is not present in the string then it returns NULL.

```
/*P10.21*/
#include<stdio.h>
#include<string.h>
int main(void)
{
 char *p;
 p=strchr("Multinational", 'n');
 printf("%s\n", p);
 p=strrchr("Multinational", 'n');
 printf("%s\n", p);
 return 0;
}
```

Output:  
national  
nal

### 10.7.5 strpbrk()

Declaration : `char *strpbrk(const char *str1, const char *str2);`

This function returns a pointer to first character of string `str1`, which matches with any character of `str2`. It returns NULL if there are no common characters in the two strings.

```
/*P10.22*/
#include<stdio.h>
#include<string.h>
int main(void)
{
 char *p1, *p2, *p3;
 p1=strpbrk("abcmnop", "lmn");
 p2=strpbrk("abcmnop", "ln");
 p3=strpbrk("1234ABCD", "COT");
 printf("%s\t%s\t%s\n", p1, p2, p3);
 return 0;
}
```

Output:  
mnop CD

### 10.7.6 strspn() and strcspn()

Declaration : `size_t strspn(const char *str1, const char *str2);`  
Declaration : `size_t strcspn(const char *str1, const char *str2);`

`strspn()` returns the number of characters in the beginning of `str1` that match any of the characters in `str2`.

For example-

`strspn("c in depth", "abcd");` will return 1  
`strspn("2012 Purva Belmont", "0123456789");` will return 4

```
strspn(" main", " \t"); will return 1
```

**strcspn()** returns the number of characters in the beginning of `str1` that don't match any of the characters in `str2`.

For example-

```
strcspn("abcmnop", "lmn"); will return 3
strcspn("J P Nagar 5 phase", "0123456789"); will return 10
strcspn("structure", "aeiou"); will return 3
```

### 10.7.7 strstr()

**Declaration** : `char *strstr(const char *s1, const char *s2);`

This function is used to locate the first occurrence of a substring in another string. The second argument is a pointer to the substring and the first argument is a pointer to the string in which the substring is to be searched. This function returns a pointer to the beginning of the first occurrence of the substring in another string. If the substring is not present then it returns `NULL` and if length of string `s2` is zero, then it returns `s1`.

```
/*P10.23*/
#include<stdio.h>
#include<string.h>
int main(void)
{
 char *ptr;
 ptr=strstr("placement section", "cement");
 printf("%s\n", ptr);
 return 0;
}
```

Output:  
cement section

### 10.7.8 strtok()

**Declaration** : `char *strtok(char *s1, const char *s2);`

This function is used to extract tokens from a string based on some separator characters. A sequence of calls to `strtok()` breaks the string `s1` into tokens, where each token is delimited by a character from the string `s2`.

The first call to `strtok()` has a non-`NULL` first argument, while the subsequent calls have a `NULL` first argument. The second argument which is a string containing the separator characters can differ from call to call. Each call to `strtok()` finds a token in string `s1` and appends a `NULL` character to the token and returns a pointer to it. It returns `NULL` when there are no tokens left in `s1`. `strtok()` saves a pointer to the character following the token found, from where the next search for a token starts in subsequent calls.

```
/*P10.24*/
#include<stdio.h>
#include<string.h>
main ()
{
 char str[]="...why?but;not?oh!, where when";
 char seps[]="?!;., \t";
 char *t;
 t=strtok(str, seps);
 while(t!=NULL)
 {
 printf("%s ", t);
 t=strtok(NULL, seps);
 }
 return 0;
}
```

Output:  
why but not oh where when

### 10.7.9 sprintf()

**Declaration** : `int sprintf(char *str, const char *controlstring[, argument1, argument2, .....]);`  
**Header file** : `stdio.h`

This function is similar to `printf()` function except that instead of sending the formatted output to the screen, it stores the formatted output in a character array. Thus with the help of this function we can convert variables of any data type to string. A null character is appended at the end automatically. This function returns the number of characters output to the string excluding the null character. It is the responsibility of the programmer to make the size of string large enough to store all the arguments so that no overflow occurs. In the following program we will convert integer and float values to strings using `sprintf()`.

*/\*P10.25 Program to convert integer and float values to strings using sprintf() function\*/*

```
#include<stdio.h>
int main(void)
{
 char str1[10];
 char str2[10];
 int x=1348;
 float y=234.56;
 sprintf(str1, "%d", x);
 sprintf(str2, "%.2f", y);
 printf("str1=%s, str2=%s\n", str1, str2);
 return 0;
}
```

Output:  
str1=1348, str2=234.56

*/\*P10.26 sprintf() function\*/*

```
#include<stdio.h>
int main(void)
{
 char str[50];
 char name[10]="Suresh";
 int m1=89, m2=78, m3=80;
 float per=(m1+m2+m3)/3.0;
 char gr='A';
 sprintf(str, "Result-%s %d %d %d %.2f %c\n", name, m1, m2, m3, per, gr);
 printf("The string str is : %s\n", str);
 return 0;
}
```

Output:  
The string str is : Result-Suresh 89 78 80 82.33 A

### 10.7.10 sscanf()

**Declaration** : `int sscanf(const char *str, const char *controlstring[, address1, address2, .....]);`  
**Header file** : `stdio.h`

This function is same as the `scanf()` function except that data is read from a string rather than the standard input. We can read the formatted text from a string and convert it into variables of different data types. In the following program, we have two strings that are converted to integer and float values using `sscanf()`.

*/\*P10.27 Program to convert strings to integer and float values\*/*

```
#include<stdio.h>
int main(void)
{
 char str1[10]="1348";
 char str2[10]="234.56";
 int x;
 float y;
 sscanf(str1, "%d", &x);
 sscanf(str2, "%f", &y);
 printf("Value of x=%d, Value of y=%.2f\n", x, y);
}
```

```
return 0;
```

Output:  
Value of x=1348, Value of y=234.56

```
/*P10.28 sscanf() function*/
#include<stdio.h>
int main(void)
{
 char name[10];
 int age;
 float sal;
 char str[30]="Shravani 23 28000.0";
 sscanf(str, "%s%d%f", name, &age, &sal);
 printf("Name : %s\n", name);
 printf("Age : %d\n", age);
 printf("Salary : %.2f\n", sal);
 return 0;
}
```

Output:  
Name : Shravani  
Age : 23  
Salary : 28000.00

sscanf() can be used for validating input data. We know how to put simple validity checks on data such as whether it is in specified range or not, but what if the user starts entering a string where an integer is required. This type of error is difficult to detect and correct. Here we can use sscanf() to check whether a valid integer value is entered or not. For this we take the input in a string instead of an integer variable and then check each character of that string, and if any character in the string is other than a digit, '+' or '-', then we can print the message that input is not a valid integer. If all the characters input in the string are valid then we can convert that string to an integer variable with the help of sscanf() and then use that integer variable in our program.

## Some Additional Problems

### Problem 1

Write a program to enter a string and print it in reverse order.

```
/*P10.29 Program to enter a string and print it in reverse order*/
#include<stdio.h>
#include<string.h>
int main(void)
{
 char str[50];
 int len;
 printf("Enter a string : ");
 gets(str);
 for(len=strlen(str)-1; len>=0; len--)
 printf("%c", str[len]);
 return 0;
}
```

### Problem 2

Write a program to test whether a string is palindrome or not. A string is palindrome if it remains the same when reversed. For example – "radar", "madam".

```
/*P10.30 Program to test whether a string is palindrome or not*/
#include<stdio.h>
#include<string.h>
int main(void)
```

```
char str[50];
int i, j;
printf("Enter a string : ");
gets(str);
for(i=0, j=strlen(str)-1; i<=j; i++, j--)
 if(str[i]!=str[j])
 break;
if(i>j)
 printf("String is a palindrome\n");
else
 printf("String is not a palindrome\n");
return 0;
```

### Problem 3

Write a function reverseStr() that takes a string as argument and reverses it. Write another function reverse() that reverses only a portion of the string.

```
/*P10.31*/
#include<stdio.h>
#include<string.h>
void reverseStr(char str[]);
void reverse(char str[], int start, int end);
int main(void)
```

```
char str[50];
strcpy(str, "I have many books");
reverseStr(str);
puts(str);
reverse(str, 6, 9);
puts(str);
return 0;
```

```
void reverseStr(char str[])
```

```
{
 int i, j;
 char tmp;
 for(i=0, j=strlen(str)-1; i<=j; i++, j--)
 {
 tmp=str[i];
 str[i]=str[j];
 str[j]=tmp;
 }
}
```

```
void reverse(char str[], int start, int end)
```

```
{
 char tmp;
 while(start<=end)
 {
 tmp=str[start];
 str[start]=str[end];
 str[end]=tmp;
 start++;
 end--;
 }
}
```

### Problem 4

Write a function reverse\_letters() to reverse the letters in the words of a string. For example if the string "I have many books", after reversing the letters of words it becomes "I evah ynam skoob".

```

/*P10.32*/
#include<stdio.h>
#include<string.h>
void reverse(char str[], int start, int end);
void reverse_letters(char str[], int start, int end);
int main(void)
{
 char str[50];
 strcpy(str, "I have many books");
 reverse_letters(str, 0, strlen(str)-1);
 puts(str);
 return 0;
}

void reverse_letters(char str[], int start, int end)
{
 int w_start, w_end;
 for(w_start=w_end=start; w_end<end; w_end++)
 {
 if(str[w_end]==' ')
 continue;
 w_start=w_end;
 while(str[w_end]!=' ' && w_end<end)
 w_end++;
 w_end--;
 reverse(str, w_start, w_end); /*Reverse the letters of the word*/
 }
}

void reverse(char str[], int start, int end)
{
 char tmp;
 while(start<=end)
 {
 tmp=str[start];
 str[start]=str[end];
 str[end]=tmp;
 start++;
 end--;
 }
}

```

### Problem 5

Write a function to reverse the words in a string. For example if the string is "I have many books", after reversing the words it becomes "books many have I".

"I have many books"  $\xrightarrow{\text{Reverse whole string}}$  "skoob ynam evah I"  $\xrightarrow{\text{Reverse letters of words}}$  "books many have I"

```

/*P10.33*/
#include<stdio.h>
#include<string.h>
void reverse_words(char str[], int start, int end);
void reverse_letters(char str[], int start, int end);
void reverse(char str[], int start, int end);
int main(void)
{
 char str[50];
 strcpy(str, "I have many books");
 reverse_words(str, 0, strlen(str)-1);
 puts(str);
 return 0;
}

```

```

void reverse_words(char str[], int start, int end)
{
 reverse(str, start, end);
 reverse_letters(str, start, end);
}

void reverse_letters(char str[], int start, int end)
{
 int w_start, w_end;
 for(w_start=w_end=start; w_end<end; w_end++)
 {
 if(str[w_end]==' ')
 continue;
 w_start=w_end;
 while(str[w_end]!=' ' && w_end<end)
 w_end++;
 w_end--;
 reverse(str, w_start, w_end); /*Reverse the letters of the word*/
 }
}

void reverse(char str[], int start, int end)
{
 char tmp;
 while(start<=end)
 {
 tmp=str[start];
 str[start]=str[end];
 str[end]=tmp;
 start++;
 end--;
 }
}

```

### Problem 6

Write a function count() that searches for a character in the string and returns the number of occurrences of that character in the string. It should take two arguments, first a string and then a character. Write another function replace() which replaces all the occurrences of a character from a string with another character. It should take three arguments, a string and two characters.

```

/*P10.34*/
#include<stdio.h>
#include<string.h>
int count(char *str, char ch);
void replace(char *str, char ch1, char ch2);
int main(void)
{
 char str[30];
 strcpy(str, "Programming in C language");
 printf("%d\n", count(str, 'a'));
 replace(str, 'a', 'z');
 puts(str);
 return 0;
}

int count(char *str, char ch)
{
 int cnt=0;
 while(*str++!='\0')
 if(*str==ch)
 cnt++;
 return cnt;
}

void replace(char *str, char ch1, char ch2)
{

```



```

while(*str++!='\0')
 if(*str==ch1)
 *str=ch2;
}

```

### Problem 7

Write a function which deletes all the occurrences of a character from a string. It should take two arguments, a string and a character.

```

/*P10.35*/
#include<stdio.h>
#include<string.h>
void del_char(char *str, char ch);
int main(void)
{
 char str[50]="Data Structures through c in depth";
 del_char(str, 'u');
 puts(str);
 return 0;
}

void del_char(char *str, char ch)
{
 int i, j;
 for(i=0, j=0; i<=strlen(str)-1; i++)
 if(str[i]!=ch)
 str[j++] = str[i];
 str[j] = '\0';
}

```

### Problem 8

Write a function to count the number of words in a string and print each word on a separate line.

```

/*P10.36*/
#include<stdio.h>
#include<ctype.h>
int count_words(char *str);
int main(void)
{
 char str[50]=" blue green red ";
 printf("Number of words in string=%d\n", count_words(str));
 return 0;
}

int count_words(char *str)
{
 int count=0;
 while(*str!='\0')
 {
 while(isspace(*str)) /*Pass over white spaces*/
 str++;
 if(*str=='\0')
 return count;
 while(!isspace(*str) && *str!='\0')
 {
 putchar(*str);
 str++;
 }
 count++;
 printf("\n");
 }
 return count;
}

```

### Problem 9

Write a program to read in a string and output the frequency of each character in that string.

```

/*P10.37*/
#include<stdio.h>
#include<string.h>
void count(char *str);
int main(void)
{
 char str[50];
 printf("Enter a string : ");
 gets(str);
 count(str);
 return 0;
}

void count(char *str1)
{
 char str[50], ch;
 int i, j, count, n;
 strcpy(str, str1);
 n=strlen(str);

 for(i=0; i<n; i++)
 {
 if(str[i]!='\0')
 {
 ch=str[i];
 count=0;
 for(j=0; j<n; j++)
 if(str[j]==ch)
 {
 count++;
 str[j]=0;
 }
 printf("%c occurs %d times\n", ch, count);
 }
 }
}

```

### Problem 10

Write a function that returns the index of first non repeated character in a string. If all characters are repeated then the function should return -1.

find\_firstNR("Suresh Kumar Srivastava") returns 3  
 find\_firstNR("amita mital") returns 5  
 find\_firstNR("anant") returns -1

```

/*P10.38*/
#include<stdio.h>
#include<string.h>
int find_firstNR(char str[]);
int main(void)
{
 char str[50]="Suresh Kumar Srivastava";
 printf("%d\n", find_firstNR(str));
 return 0;
}

int find_firstNR(char str[])
{
 int i, j;
 int end=strlen(str)-1;
}

```

```

for(i=0; i<end; i++)
{
 for(j=0; j<=end; j++)
 if(str[i]==str[j] && i!=j)
 break;
 if(j==end+1)
 return i;
}
return -1;
}

```

### Problem 11

Write functions `str_to_i()` and `str_to_d()` that convert string to integer and double respectively.

```

/*P10.39*/
#include<stdio.h>
#include<string.h>
#include<ctype.h>
#include<math.h>
int str_to_i(char str[]);
double str_to_d(char str[]);
int main(void)
{
 char str[20];
 printf("Enter a string : ");
 scanf("%s",str);
 printf("%d\n",str_to_i(str));

 printf("Enter a string : ");
 scanf("%s",str);
 printf("%lf\n",str_to_d(str));
 return 0;
}

int str_to_i(char str[])
{
 int i,num=0,sign;
 i=0;

 while(!isspace(str[i]))/*Skip Initial white spaces*/
 i++;
 sign=(str[i]=='-') ? -1 : 1;

 if(str[i]!='-' || str[i]!='+')
 i++;
 while(isdigit(str[i]))
 num=num*10+(str[i++]-'0');
 return sign*num;
}

double str_to_d(char str[])
{
 int i=0,j,sign;
 double num=0;

 while(!isspace(str[i]))/*Skip Initial white spaces*/
 i++;
 sign=(str[i]=='-') ? -1 : 1;
 if(str[i]!='-' || str[i]!='+')
 i++;
 while(isdigit(str[i]))
 num=num*10+(str[i++]-'0');
 if(str[i]!='.')
 i++;
 j=i;
}

```

```

while(isdigit(str[i]))
 num=num*10+(str[i++]-'0');
return sign*num/pow(10,i-j);
}

```

### Problem 12

Write functions `i_to_str()` and `d_to_str()` to convert integer and double values to string.

```

/*P10.40*/
#include<stdio.h>
#include<string.h>
int i_to_str(int num,char str[],int base);
double d_to_str(double num,char str[]);
int main(void)
{
 char str1[30];
 i_to_str(45,str1,16);puts(str1);
 i_to_str(45,str1,10);puts(str1);
 i_to_str(45,str1,8);puts(str1);
 i_to_str(45,str1,2);puts(str1);
 i_to_str(-45,str1,10);puts(str1);
 d_to_str(27.647611,str1); puts(str1);
 d_to_str(-12.345622,str1); puts(str1);
 return 0;
}

int i_to_str(int num,char str[],int b)
{
 int i=0,temp,rem,j,sign;

 sign=num<0? -1 : 1;
 if(sign==-1)
 num=-num;
 while(num>0)
 {
 rem=num%b;
 num/=b;
 if(rem>9 && rem<16)
 str[i++]='A'+rem-10;
 else
 str[i++]='0'+rem;
 }
 if(sign==-1)
 str[i++]='-';
 str[i]='\0';

 for(i=0,j=strlen(str)-1; i<j; i++,j--) /*Reverse the string*/
 {
 temp=str[i];
 str[i]=str[j];
 str[j]=temp;
 }

 d_to_str(double num,char str[])
 {
 int i,k;
 double d;
 i=num;
 i_to_str(i,str,10);
 str[strlen(str)+1]='\0';
 str[strlen(str)]='.';
 d=num-i;
 k=d*1000000;
 if(k<0)

```

```

 k=-k;
 i_to_str(k, str+strlen(str), 10);
}

```

### Problem 13

Write functions to encrypt text and to decrypt the encrypted text.

(i) Perform the encryption so that every character is replaced by its next character. For example replace a by b, b by c and so on, replace z by a.

Plain text : program

Encrypted text : qspbsbn

Decrypted text : program

(ii) Perform the encoding according to these replacements

```

a b c d e f g h i j k l m n o p q r s t u v w x y z
m n k g h d t a b w v u p r q c z j x i e y f l o s

```

Plain text : program

Encoded text : cqjtmjp

Decoded text : program

```

/*P10.41*/
#include<stdio.h>
#include<string.h>
void encrypt1(char *str);
void decrypt1(char *str);
void encrypt2(char *str1, char *str2);
void decrypt2(char *str1, char *str2);
int strchr(char *str, char ch);

int main(void)
{
 char str[100], str2[30];
 strcpy(str, "program");
 puts(str);
 encrypt1(str);
 puts(str);
 decrypt1(str);
 puts(str);
 strcpy(str2, "m n k g h d t a b w v u p r q c z j x i e y f l o s");
 strcpy(str, "program");
 puts(str);
 encrypt2(str, str2);
 puts(str);
 decrypt2(str, str2);
 puts(str);
 return 0;
}

```

```

void encrypt1(char *str)
{
 for(; *str!='\0'; str++)
 {
 if(*str<97 || *str>122)
 continue;
 if(*str=='z')
 *str='a';
 else
 *str=*str+1;
 }
}

```

```

void decrypt1(char *str)
{
 for(; *str!='\0'; str++)

```

```

{
 if(*str<97 || *str>122)
 continue;
 if(*str=='a')
 *str='z';
 else
 *str=*str-1;
}

void encrypt2(char *str1, char *str2)
{
 for(; *str1!='\0'; str1++)
 {
 if(*str1<97 || *str1>122)
 continue;
 *str1=str2[*str1-97];
 }
}

void decrypt2(char *str1, char *str2)
{
 int i;
 for(; *str1!='\0'; str1++)
 {
 if(*str1<97 || *str1>122)
 continue;
 i=strch(str2, *str1);
 *str1=i+97;
 }
}

int strch(char *str, char ch)
{
 int i;
 for(i=0; str[i]!='\0'; i++)
 if(str[i]==ch)
 return i;
}

```

### Problem 14

Write a program to enter a positive 4 digit number and display it in words.

```

/*P10.42*/
#include<stdio.h>
int main(void)
{
 int n, num, d=0, dig[4];
 char *ones[]={"", "One", "Two", "Three", "Four", "Five", "Six",
 "Seven", "Eight", "Nine", "Ten"};
 char *el[]={"Ten", "Eleven", "Twelve", "Thirteen", "Fourteen",
 "Fifteen", "Sixteen", "Seventeen", "Eighteen", "Nineteen"};
 char *tens[]={"", "", "Twenty", "Thirty", "Forty", "Fifty",
 "Sixty", "Seventy", "Eighty", "Ninety"};
 printf("Enter a 4 digit number : ");
 scanf("%d", &num);
 n=num;
 do
 {
 dig[d]=n%10;
 n/=10;
 d++;
 }while(n>0);
}

```

```

if(d==4)
 printf("%s Thousand", ones[dig[3]]);
if(d>=3 && dig[2]!=0)
 printf(" %s Hundred ", ones[dig[2]]);
if(d>=2)
{
 if(dig[1]==0)
 printf("%s\n",ones[dig[0]]);
 else if(dig[1]==1)
 printf("%s\n",el[dig[0]]);
 else
 printf("%s %s\n",tens[dig[1]],ones[dig[0]]);
}
if(d==1 && num!=0)
 printf("%s\n",ones[dig[0]]);
if(num==0)
 printf("Zero\n");
return 0;

```

### Problem 15

Write recursive functions for finding length of a string and displaying the string in front and reverse order.

A string can be defined recursively as-

. A string may be an empty string.

. A string may be a character followed by a smaller string (one character less).

The string "leaf" is character 'l' followed by string "eaf", similarly string "eaf" is character 'e' followed by string "af", string "af" is character 'a' followed by string "f", string "f" is character 'f' followed by empty string. So while defining recursive operations on strings, case of empty string will serve as the base case for terminating recursion.

The length of an empty string is 0 and this will be our terminating condition. In general case, the function is recursively called for a smaller string, which does not contain the first character of the string.

```

int length(char *str)
{
 if(*str == '\0')
 return 0;
 return (1 + length(str+1));
}

```

We can print a string by printing the first character of the string followed by printing of the smaller string, if the string is empty there is nothing to be printed so we will do nothing and return (base case).

```

void display(char *str)
{
 if(*str == '\0')
 return;
 putchar(*str);
 display(str+1);
}

```

By changing the order of printing statement and recursive call we can get the function for displaying string in reverse order.

```

void Rdisplay(char *str)
{
 if(*str == '\0')
 return;
 Rdisplay(str+1);
 putchar(*str);
}

```

```

putchar(*str);
}

```

/\*P10.43\*/

```

#include<stdio.h>
int length(char *str);
void display(char *str);
void Rdisplay(char *str);
int main(void)
{
 char str[50]="Devanashi";
 printf("%d\n",length(str));
 display(str);
 printf("\n");
 Rdisplay(str);
 return 0;
}

```

```

int length(char *str)
{
 if(*str == '\0')
 return 0;
 return (1 + length(str+1));
}

```

```

void display(char *str)
{
 if(*str == '\0')
 return;
 putchar(*str);
 display(str+1);
}

```

```

void Rdisplay(char *str)
{
 if(*str == '\0')
 return;
 Rdisplay(str+1);
 putchar(*str);
}

```

### Exercise

What is the output of exercises from 1 to 30. Assume that files `stdio.h` and `string.h` are included wherever required.

```

1. int main(void)
{
 int i=0;
 char name[10]={'M','o','h','i','n','i','\0'};
 while(name[i])
 putchar(name[i++]);
 return 0;
}

```

```

2. int main(void)
{
 char *str;
 printf("Enter a string : ");
 gets(str);
 printf("String is %s\n",str);
 return 0;
}

```

```

3. int main(void)
{
 char *str;
 printf("Enter a string : ");
 gets(str);
 printf("String is %s\n",str);
 return 0;
}

```

```

{
 char *str1="Good", *str2="Morning";
 strcat(str1, str2);
 printf("%s\n", str1);
 return 0;
}

4. int main(void)
{
 char str[10]="How";
 strcat(str, '?');
 printf("%s\n", str);
 return 0;
}

5. int main(void)
{
 char str[]="Vijaynagar";
 str=str+5;
 printf("%s\n", str);
 return 0;
}

6. void func(char str[]);
int main(void)
{
 char str[]="Vijaynagar";
 func(str);
 return 0;
}

void func(char str[])
{
 str=str+5;
 printf("%s\n", str);
}

7. int main(void)
{
 char str[]={70, 97, 105, 116, 104, 0};
 printf("%s\n", str);
 return 0;
}

8. int main(void)
{
 char *p="Devanshi";
 char arr[]="Devanshi";
 *(p+2)='h';
 arr[2]='h';
 puts(p);
 puts(arr);
 return 0;
}

9. int main(void)
{
 char str[]="painstaking";
 char *p=str+5;
 printf("%c\t", *p);
 printf("%s\n", p);
 return 0;
}

```

```

10. int main(void)
{
 printf("%c\t", "Determination"[2]);
 printf("%c\t", *("Determination"+2));
 printf("%s\t", "Determination"+2);
 printf("Determination"+2);
 printf("\t");
 printf("Determination"+strlen("Deepali"));
 printf("\t");
 printf("Determination"+sizeof("Deepali"));
 printf("\n");
 return 0;
}

11. int main(void)
{
 char str[]="Lucknow";
 char *p=str;
 p=p+3;
 p[3]='t';
 printf("%s\t%s\n", str, p);
 return 0;
}

12. int main(void)
{
 char *p[]={"Orange", "Yellow", "Sky", "Blue", "Black"};
 char arr[10];
 printf("%s %s %s\n", p[1], p[2], p[3]);
 strcpy(arr, "Lucknow");
 printf("%s\n", arr);
 return 0;
}

13. int main(void)
{
 char str1[15]="Good ";
 char str2[]="Evening";
 strcpy(str1+strlen(str1), str2);
 printf("%s\n", str1);
 return 0;
}

14. int main(void)
{
 char str1[]="Parul";
 char str2[10];
 strcpy(str2, str1);
 if(str1==str2)
 printf("Same\n");
 else
 printf("Different\n");
 return 0;
}

15. int main(void)
{
 char x[]="Shilpee";
 char y[20];
 y="Anjali";
 printf("%s %s\n", x, y);
}

```

```

 return 0;
}

6. int main(void)
{
 char str1[]="deep";
 char str2[]={'d','e','e','p'};
 if(strcmp(str1,str2)==0)
 printf("Same\n");
 else
 printf("Different\n");
 return 0;
}

7. int main(void)
{
 char str1[]="Parul",str2[]="Devanshi";
 if(strlen(str1)-strlen(str2)>=0)
 puts(str1);
 else
 puts(str2);
 return 0;
}

8. int main(void)
{
 char name[15]="Vikramaditya";
 int i=0;
 while(name[i])
 {
 printf("%c ",name[i]);
 i=i+3;
 }
 return 0;
}

9. int main(void)
{
 char str[10][20];
 int i;
 for(i=0; i<10; i++)
 scanf("%s",str[i]);
 for(i=0; i<10; i++)
 printf("%s ",str[i]);
 return 0;
}

10. int main(void)
{
 char *str[10];
 int i;
 for(i=0; i<10; i++)
 scanf("%s",str[i]);
 for(i=0; i<10; i++)
 printf("%s",str[i]);
 return 0;
}

11. char *combine(char *arr1,char *arr2);
int main(void)
{
 char str1[20],str2[20];
 char *p;
 strcpy(str1,"Suresh ");

```

```

 strcpy(str2,"Kumar");
 p=combine(str1,str2);
 puts(p);
 return 0;
}

char *combine(char *arr1,char *arr2)
{
 char str[80];
 strcpy(str,arr1);
 strcat(str,arr2);
 return str;
}

22. int main(void)
{
 char *str="Deepali Srivastava";
 int i=0;
 while(str[i+i]);
 printf("%d\n",i);
 return 0;
}

23. int main(void)
{
 int d1,m1,y1;
 char date[11]="24/05/1973";
 date[2]=date[5]='\0';
 sscanf(date,"%d",&d1);
 sscanf(date+3,"%d",&m1);
 sscanf(date+6,"%d",&y1);
 date[2]=date[5]='/';
 printf("d1=%d,m1=%d,y1=%d\n",d1,m1,y1);
 printf("date=%s\n",date);
 return 0;
}

24. void func(char *p);
int main(void)
{
 char *str="doubtful";
 func(str);
 return 0;
}

void func(char *p)
{
 if(*p!='f')
 {
 printf("%c",*p);
 func(++p);
 }
}

25. void func(char *p);
int main(void)
{
 char *str="tap";
 func(str);
 return 0;
}

void func(char *p)
{
 if(*p)
 {

```

```
func(p+1);
printf("%c", *p++);
}
```

26. int main(void)

```
{
 char *ptr;
 ptr="My name is %s and age is %d\n";
 printf(ptr, "Ranju", 30);
 return 0;
}
```

27. void func1(char x[]);

void func2(char x[]);

int main(void)

```
{
 char arr[5];
 puts(arr);
 func1(arr);
 puts(arr);
 func2(arr);
 puts(arr);
 return 0;
}
```

void func1(char x[])

```
{
 x="Jack";
 puts(x);
}
```

void func2(char x[])

```
{
 x[0]='J', x[1]='i', x[2]='l', x[3]='l', x[4]='\0';
 puts(x);
}
```

28. int main(void)

```
{
 char *ptr;
 ptr="Every saint has a past,\n
 Every sinner has a future.\n";
 printf("Giving " "is " "living." "\n");
 printf(ptr);
 return 0;
}
```

29. int main(void)

```
{
 int marks;
 char name[50];
 printf("Enter marks : ");
 scanf("%d", &marks);
 printf("Enter name : ");
 gets(name);
 puts(name);
 return 0;
}
```

30. int main(void)

```
{
 char *p, str[100]=" ";
 strcpy(str, str+strlen(str, " \t"));
}
```

```
puts(str);
return 0;
}
```

31. Write a function to convert a string into uppercase.

32. Write a function for performing case insensitive string comparison.

33. Unlike gets(), the function fgets() doesn't delete the newline character entered at the end but retains it. What should we do if we don't want the newline character in the string.

34. Write a function to remove all leading and trailing blanks from a string.

35. Input a string and change it so that the characters are placed in alphabetical order. For example the string "Devanshi" should be changed to "aDehinsv".

36. Write a program to abbreviate input text. For example if the input is "World Health Organization", then the output should be WHO.

37. Write a function to extract a substring from a string. Assume that the substring starts at the i<sup>th</sup> character (start counting from 0<sup>th</sup> character) and is n characters long.

38. Write a function to replace adjacent multiples spaces in a string by a single space.

39. Write your own pointer versions of the library functions strcpy(), strcmp(), strcat().

40. Write a function strstr\_r() which takes two strings as arguments and returns a pointer to the beginning of the last occurrence of the second string in the first string.

41. Write a function find\_indexF() which takes two strings as arguments and returns index of first occurrence of the second string in first string. Write a similar function find\_indexP() that returns the index of last occurrence of the second string in first string.

42. Write a function str\_start() which takes two strings as arguments and returns 1 if second string is present at the start of first string, otherwise returns 0. Write another function str\_end() that returns 1 if second string is present at the end of first string otherwise returns 0.

43. Write a program to replace all occurrences of word "Bangalore" by "Bengaluru" in a string.

44. Write a program to find the number of occurrences of a particular word in a string.

45. Write a program to input two strings and remove all the common characters from both the strings. Display the resultant strings.

46. Rewrite program P5.41, so that the day is printed using array of pointers instead of switch.

47. Write a recursive function to count the number of vowels in a string.

48. Write a recursive function to replace each occurrence of a character by another character in a string.

49. Write a recursive function to reverse a string.

50. Write a recursive function to return the index of first occurrence of a character in a string.

51. Write a recursive function to return the index of last occurrence of a character in a string.

52. Write a recursive function to find whether a string is palindrome or not. A palindrome is a string that is read the same way forward and backward for example "radar", "hannah", "madam".

53. In the program of previous problem, make changes so that spaces, punctuation marks, uppercase and lowercase differences are ignored. The strings "A man, a plan, a canal - Panama!", "Live Evil" should be recognized as palindromes.

54. Write a recursive function to convert a string of numbers to an integer.

55. Write a recursive function to print all possible permutations of a string. For example if the string is "abc" then all possible permutations are abc, acb, bac, bca, cba, cab.

# Structure and Union

Array is a collection of same type of elements but in many real life applications we may need to group different types of logically related data. For example if we want to create a record of a person that contains name, age and height of that person, we can't use array because all the three data elements are of different types. To store these related fields of different data types we can use a structure, which is capable of storing heterogeneous data. Data of different types can be grouped together under a single name using structures. The data elements of a structure are referred to as members.

## 11.1 Declaring a Structure type

Declaration of a structure creates a template or format that describes the characteristics of its members. All the variables that would be declared of this structure type, will take the form of this template. The general syntax of a structure declaration is-

```
struct tagname
{
 datatype member1;
 datatype member2;

 datatype memberN;
};
```

Here `struct` is a keyword, `member1`, `member2`, ....., `memberN` are members of the structure and are declared inside curly braces. There should be a semicolon at the end of the curly braces. These members can be of any data type like `int`, `char`, `float`, `array`, `pointers` or another structure type. `tagname` is the name of the structure and it is used further in the program to declare variables of this structure type.

Declaration of a structure provides one more data type in addition to the built in data types. We can declare variables of this new data type that will have the format of the declared structure. It is important to note that declaration of a structure template does not reserve any space in memory for the members; space is reserved only when variables of this structure type are declared. Although the syntax of declaration of members inside the template is identical to the syntax we use in declaring variables, these members are not variables, they don't have any existence until they are attached with a structure variable. The member names inside a structure should be different from one another but these names can be similar to any other variable name declared outside the structure. The member names inside two different structures can also be same.

Let us take an example of declaring a structure template.

```
struct student
{
 char name[20];
 int rollno;
 float marks;
};
```

Here `student` is the structure tag and there are three members of this structure viz. `name`, `rollno` and `marks`. Structure template can be declared globally or locally i.e. it can be placed before all functions in the

structure and Union

program or it can be locally present in a function. If the template is global then it can be used by all functions while if it is local then only the function containing it can use it.

## 11.2 Declaring Structure Variables

By declaring a structure we have only created a format, the actual use of structures will be when we declare variables based on this format. We can declare structure variables in two ways-

- With structure declaration
- Using the structure tag

### With Structure Declaration

```
struct student
{
 char name[20];
 int rollno;
 float marks;
} stu1, stu2, stu3;
```

Here `stu1`, `stu2` and `stu3` are variables of type `struct student`. When we declare a variable while declaring the structure template, the tagname is optional. So we can also declare them as-

```
struct
{
 char name[20];
 int rollno;
 float marks;
} stu1, stu2, stu3;
```

If we declare variables in this way, then we will not be able to declare other variables of this structure type anywhere else in the program nor can we send these structure variables to functions. If a need arises to declare a variable of this type in the program then we will have to write the whole template again. So although the tagname is optional, it is always better to specify a tagname for the structure.

### Using Structure Tag

We can also declare structure variables using structure tag. This can be written as-

```
struct student
{
 char name[20];
 int rollno;
 float marks;
} struct student stu1, stu2;
struct student stu3;
```

Here `stu1`, `stu2` and `stu3` are structure variables that are declared using the structure tag `student`. Declaring a structure variable reserves space in memory. Each structure variable declared to be of type `struct student` has three members viz. `name`, `rollno` and `marks`. The compiler will reserve space for each variable sufficient to hold all the members. For example each variable of type `struct student` will occupy 28 (20+4+4) bytes (this size can vary see section 11.6).

## 11.3 Initialization of Structure Variables

The syntax of initializing structure variables is similar to that of arrays. All the values are given in curly braces and the number, order and type of these values should be same as in the structure template definition. The initializing values can only be constant expressions.

```
struct student
```



```
char name[20];
int rollno;
float marks;
}stu1={"Mary",25,98};
struct student stu2={"John",24,67.5};
```

Here value of members of stu1 will be "Mary" for name, 25 for rollno, 98 for marks. The values of members of stu2 will be "John" for name, 24 for rollno, 67.5 for marks.

We cannot initialize members while defining the structure.

```
struct student
{
 char name[20];
 int rollno;
 float marks=99; /*Invalid*/
}stu;
```

This is invalid because there is no variable named marks, and no memory is allocated while declaration of structure type. If the number of initializers is less than the number of members then the remaining members are initialized with zero. For example if we have this initialization-

```
struct student stu1={"Mary"};
```

Here the members rollno and marks of stu1 will be initialized to zero. This is equivalent to the initialization-

```
struct student stu1={"Mary",0,0};
```

## 11.4 Accessing Members of a Structure

For accessing any member of a structure variable, we use the dot (.) operator which is also known as the period or membership operator. The format for accessing a structure member is-

```
structvariable.member
```

Here on the left side of the dot there should be a variable of structure type and on right hand side there should be the name of a member of that structure. For example consider the following structure-

```
struct student
{
 char name[20];
 int rollno;
 float marks;
};
struct student stu1, stu2;
```

name of stu1 is given by - stu1.name

rollno of stu1 is given by - stu1.rollno

marks of stu1 is given by - stu1.marks

name of stu2 is given by - stu2.name

rollno of stu2 is given by - stu2.rollno

marks of stu2 is given by - stu2.marks

We can use stu1.name, stu1.marks, stu2.marks etc like any other ordinary variables in the program. They can be read, displayed, processed, assigned values or can be send to functions as arguments. We can't use student.name or student.rollno because student is not a structure variable, it is a structure tag.

```
/*P11.1 Program to display the values of structure members*/
```

```
#include<stdio.h>
#include<string.h>
struct student
{
 char name[20];
 int rollno;
 float marks;
};
```

```
int main(void)
{
 struct student stu1={"Mary",25,68};
 struct student stu2,stu3;
 strcpy(stu2.name,"John");
 stu2.rollno=26;
 stu2.marks=98;
 printf("Enter name, rollno and marks for stu3 : ");
 scanf("%s %d %f", stu3.name, &stu3.rollno, &stu3.marks);
 printf("stu1 : %s %d %.2f\n",stu1.name,stu1.rollno,stu1.marks);
 printf("stu2 : %s %d %.2f\n",stu2.name,stu2.rollno,stu2.marks);
 printf("stu3 : %s %d %.2f\n",stu3.name,stu3.rollno,stu3.marks);
 return 0;
}
```

Output.

Enter name, rollno and marks for stu3 : Tom 27 79.5

stu1 : Mary 25 68.00

stu2 : John 26 98.00

stu3 : Tom 27 79.50

In this program we have declared three variables of type struct student. The first variable stu1 has been initialized, the members of second variable stu2 are given values using separate statements and the values for third variable stu3 are input by the user. Note that since stu2.name is an array, we can't assign a string to it using assignment operator, hence we have used the strcpy() function.

The dot operator is one of the highest precedence operators, its associativity is from left to right. It will take precedence over all other unary, relational, logical, arithmetic and assignment operators. For example the expression ++stu.marks is equivalent to ++(stu.marks).

## 11.5 Assignment of Structure Variables

We can assign values of a structure variable to another structure variable, if both variables are defined of the same structure type. For example-

```
/*P11.2 Program to assign a structure variable to another structure variable*/
```

```
struct student
{
 char name[20];
 int rollno;
 float marks;
};
int main(void)
{
 struct student stu1={"Oliver",12,98};
 struct student stu2;
 stu2=stu1;
 printf("stu1:%s %d %.2f\n",stu1.name,stu1.rollno,stu1.marks);
 printf("stu2:%s %d %.2f\n",stu2.name,stu2.rollno,stu2.marks);
 return 0;
}
```

Output :

stu1 : Oliver 12 98.00

stu2 : Oliver 12 98.00

Unary, relational, arithmetic, bitwise operators are not allowed with structure variables. We can use these operators with the members provided the member is not a structure itself.

## 11.6 Storage allocation and Size of Structure

We may need to find out the size of structure in some situations like reading or writing to files. To find out the size of a structure by sizeof operator, we can either use the structure variable name or the tagname with the struct keyword. For example-

```
sizeof(struct student)
sizeof(stu1)
sizeof(stu2)
```

Here if `stu1` and `stu2` are variables of type `struct student`, then all the three expressions will give the same result.

Size of structures may be different on different machines. This is because of certain memory alignment restrictions on some computers. For example some machines may store integers only at addresses which are multiple of 4. Consider this structure-

```
struct
{
 char ch;
 int num;
}var;
```

The memory required for one `char` and one `int` variable is 5 bytes, but size of this structure on most machines would be 8 bytes and there will be a hole of 3 bytes in between. Sometimes there are unused bytes after the last member(padding) to satisfy the alignment requirements. Due to these reasons, size of whole structure may not be equal to the sum of sizes of its members. So it is always better to find the size of structure variable by using `sizeof` operator rather than using the sum of sizes of its members.

There may be unused bytes in the structure, and due to this even if two structures have same values for all members, these unused bytes will have different values, so comparing of structures is not allowed in C.

Memory for members is allocated in the order given in the declaration of the structure template.

## 11.7 Array of Structures

We know that array is a collection of elements of same datatype. We can declare array of structures where each element of array is of same structure type. Array of structures can be declared as-

```
struct student stu[10];
```

Here `stu` is an array of 10 elements, each of which is a structure of type `struct student`. Each element of `stu` has 3 members, which are `name`, `rollno` and `marks`. These structures can be accessed through subscript notation. To access the individual members of these structures we will use the dot operator as usual.

```
stu[0].name stu[0].rollno stu[0].marks
stu[1].name stu[1].rollno stu[1].marks
stu[2].name stu[2].rollno stu[2].marks
.....
stu[9].name stu[9].rollno stu[9].marks
```

All the structures of an array are stored in consecutive memory locations.

*/\*P11.3 Array of structures\*/*

```
#include<stdio.h>
struct student
{
 char name[20];
 int rollno;
 float marks;
};

int main(void)
{
 int i;
 struct student stuarr[10];
 for(i=0; i<10; i++)
 {
 printf("Enter name,rollno and marks : ");
 scanf("%s%d%f", stuarr[i].name, &stuarr[i].rollno, &stuarr[i].marks);
 }
 for(i=0; i<10; i++)
 printf("%s %d %f\n", stuarr[i].name, stuarr[i].rollno, stuarr[i].marks);
}
```

```
return 0;
```

The array of structures may be initialized using the same syntax as in arrays. For example-

```
struct student stuarr[3]={
 {"Mary", 12, 98.5},
 {"John", 11, 97},
 {"Tom", 12, 89.5}
};
```

The inner pairs of braces are optional if all the initializers are present in the list.

## 11.8 Arrays within Structures

We can have an array as a member of structure. In structure `student`, we have taken the member name as an array of characters. Now we will declare another array inside the structure `student`.

```
struct student
{
 char name[20];
 int rollno;
 int submarks[4];
};
```

The array `submarks` denotes the marks of students in 4 subjects.

If `stu` is a variable of type `struct student` then-

`stu.submarks[0]` - Denotes the marks of the student in first subject

`stu.submarks[1]` - Denotes the marks in second subject.

`stu.name[0]` - Denotes the first character of the name member.

`stu.name[4]` - Denotes the fifth character of the name member.

If `stuarr[10]` is an array of type `struct student` then-

`stuarr[0].submarks[0]` - Denotes the marks of first student in first subject

`stuarr[4].submarks[3]` - Denotes the marks of fifth student in fourth subject.

`stuarr[0].name[0]` - Denotes the first character of name member of first student

`stuarr[5].name[7]` - Denotes the eighth character of name member of sixth student

*/\*P11.4 Program to understand arrays within structures\*/*

```
#include<stdio.h>
struct student
{
 char name[20];
 int rollno;
 int submarks[4];
};

main(void)
{
 int i,j;
 struct student stuarr[3];
 for(i=0; i<3; i++)
 {
 printf("Enter data for student %d\n", i+1);
 printf("Enter name : ");
 scanf("%s", stuarr[i].name);
 printf("Enter roll number : ");
 scanf("%d", &stuarr[i].rollno);
 for(j=0; j<4; j++)
 {
 printf("Enter marks for subject %d : ", j+1);
 scanf("%d", &stuarr[i].submarks[j]);
 }
 }
}
```

```

for(i=0; i<3; i++)
{
 printf("Data of student %d\n", i+1);
 printf("Name: %s, Roll number: %d\nMarks:", stuarr[i].name, stuarr[i].rollno);
 for(j=0; j<4; j++)
 printf("%d ", stuarr[i].submarks[j]);
 printf("\n");
}
return 0;

```

### 11.9 Nested Structures (Structure within Structure)

The members of a structure can be of any data type including another structure type i.e. we can include a structure within another structure. A structure variable can be a member of another structure. This is called nesting of structures.

```

struct tag1
{
 member1;
 member2;

 struct tag2
 {
 member1;
 member2;

 member m;
 }var1;

 member n;
}var2;

```

For accessing member1 of inner structure we will write-  
var2.var1.member1

Here is an example of nested structures-

```

struct student
{
 char name[20];
 int rollno;
 struct date
 {
 int day;
 int month;
 int year;
 }birthdate;
 float marks;
}stu1, stu2;

```

Here we have defined a structure date inside the structure student. This structure date has three members day, month and year, and birthdate is a variable of type struct date. We can access the members of inner structure as-

```

stu1.birthdate.day → day of birthdate of stu1
stu1.birthdate.month → month of birthdate of stu1
stu1.birthdate.year → year of birthdate of stu1
stu2.birthdate.day → day of birthdate of stu2

```

Here we have defined the template of structure date inside the structure student, we could have defined it outside and declared its variables inside the structure student using the tag. But remember if we define the

inner structure outside, then this definition should always be before the definition of outer structure. Here in this case the date structure should be defined before the student structure.

```

struct date
{
 int day;
 int month;
 int year;
}

struct student
{
 char name[20];
 int rollno;
 float marks;
 struct date birthdate;
}stu1, stu2;

```

The advantage of defining date structure outside is that we can declare variables of date type anywhere else also. Suppose we define a structure teacher, then we can declare variables of date structure inside it.

```

struct teacher
{
 char name[20];
 int age;
 float salary;
 struct date birthdate, joindate;
}t1, t2;

```

The nested structures may also be initialized at the time of declaration. For example-

```

struct teacher t1={"Sam", 34, 9000, {8, 12, 1970}, {1, 7, 1995}};

```

Nesting of a structure within itself is not valid. For example the following structure definition is invalid-

```

struct person{
 char name[20];
 int age;
 float height;
 struct person father; /*Invalid*/
}emp;

```

The nesting of structures can be extended to any level. The following example shows nesting at level three i.e. first structure is nested inside a second structure and second structure is nested inside a third structure.

```

struct time
{
 int hr;
 int min;
 int sec;
}

struct date
{
 int day;
 int month;
 int year;
 struct time t;
}

struct student
{
 char name[20];
 struct date dob; /*Date of birth*/
}stu1, stu2;

```

To access hour of date of birth of student stu1 we can write-  
stu1.dob.t.hr

### 11.10 Pointers to Structures

We have studied that pointer is a variable which holds the starting address of another variable of any data type like int, float or char. Similarly we can have pointers to structures, which can point to the starting address of a structure variable. These pointers are called structure pointers and can be declared as-

```
struct student
{
 char name[20];
 int rollno;
 int marks;
};
struct student stu, *ptr;
```

Here ptr is a pointer variable that can point to a variable of type struct student. We will use the & operator to access the starting address of a structure variable, so ptr can point to stu by-

```
ptr=&stu;
```

There are two ways of accessing the members of a structure through structure pointer.

We know ptr is a pointer to a structure, so by dereferencing it we can get the contents of structure variable. Hence \*ptr will give the contents of stu. So to access members of a structure variable stu we can write-

```
(*ptr).name (*ptr).rollno (*ptr).marks
```

Here parentheses are necessary because dot operator has higher precedence than the \* operator. There is an alternate way of accessing structure members through pointers using the arrow operator (->) which is formed by hyphen symbol and greater than symbol. So we can access the members as-

```
ptr->name ptr->rollno ptr->marks
```

The arrow operator has same precedence as that of dot operator and it also associates from left to right.

*/\*P11.5 Program to understand pointers to structures\*/*

```
#include<stdio.h>
struct student
{
 char name[20];
 int rollno;
 int marks;
};
int main(void)
{
 struct student stu1={"Mary",25,68};
 struct student *ptr=&stu1;
 printf("Name - %s\t", ptr->name);
 printf("Rollno - %d\t", ptr->rollno);
 printf("Marks - %d\n", ptr->marks);
 return 0;
}
```

We can also have pointers that point to individual members of a structure variable. For example-

```
int *p=&stu.rollno;
float *ptr=&stu.marks;
```

The expression &stu.rollno is equivalent to &(stu.rollno) because the precedence of dot operator is more than that of address operator.

## 11.11 Pointers within Structures

A pointer can also be used as a member of structure. For example we can define a structure like this-

```
struct student
{
 char name[20];
 int *ptrmem;
};
struct student stu, *stuptr=&stu;
```

Here ptrmem is pointer to int and is a member of the structure student.

To access the value of ptrmem, we'll write

```
stu.ptrmem or stuptr->ptrmem
```

To access the value pointed to by stu.ptrmem, we'll write

```
(stu.ptrmem or *stuptr->ptrmem)
```

Since the priority of dot and arrow operators is more than that of dereference operator, the expression stu.ptrmem is equivalent to \*(stu.ptrmem), and the expression \*stuptr->ptrmem is equivalent to (stuptr->ptrmem).

## 11.12 Structures and Functions

Structures can be passed as arguments to function in different ways. We can pass individual members, whole structure variable or structure pointers to the function. Similarly a function can return either a structure member or whole structure variable or a pointer to structure.

### 11.12.1 Passing Structure Members as Arguments

We can pass individual structure members as arguments to functions like any other ordinary variable.

*P11.6 Program to understand how structure members are sent to a function\*/*

```
#include<stdio.h>
#include<string.h>
struct student
{
 char name[20];
 int rollno;
 int marks;
};
void display(char name[],int rollno,int marks);
int main(void)
{
 struct student stu1={"John",12,87};
 struct student stu2;
 strcpy(stu2.name,"Mary");
 stu2.rollno=18;
 stu2.marks=90;
 display(stu1.name,stu1.rollno,stu1.marks);
 display(stu2.name,stu2.rollno,stu2.marks);
 return 0;
}
void display(char name[],int rollno,int marks)
{
 printf("Name - %s\t",name);
 printf("Rollno - %d\t",rollno);
 printf("Marks - %d\n",marks);
}
```

Output:

```
Name - John Rollno - 12 Marks - 87
Name - Mary Rollno - 18 Marks - 90
```

Here we have passed members of the variables stu1 and stu2 to the function display(). The names of the formal arguments can be similar to the names of the members. We can pass the arguments using call by reference also so that the changes made in the called function will be reflected in the calling function. In that case we'll have to send the addresses of the members. It is also possible to return a single member from a function.

### 11.12.2 Passing Structure Variable as Argument

Passing individual members to function becomes cumbersome when there are many members and the relationship between the members is also lost. We can pass the whole structure as an argument.

*/\*P11.7 Program to understand how a structure variable is sent to a function\*/*

```
#include<stdio.h>
struct student
{
 char name[20];
 int rollno;
 int marks;
};
void display(struct student);
int main(void)
{
 struct student stu1={"John",12,87};
 struct student stu2={"Mary",18,90};
 display(stu1);
 display(stu2);
 return 0;
}
void display(struct student stu)
{
 printf("Name - %s\t",stu.name);
 printf("Rollno - %d\t",stu.rollno);
 printf("Marks - %d\n",stu.marks);
}
```

Output:  
Name - John Rollno - 12 Marks - 87  
Name - Mary Rollno - 18 Marks - 90

Here it is necessary to define the structure template globally because it is used by both functions to declare variables.

The name of a structure variable is not a pointer unlike arrays, so when we send a structure variable as an argument to a function, a copy of the whole structure is made inside the called function and all the work is done on that copy. Any changes made inside the called function are not visible in the calling function since we are working only on a copy of the structure variable, not on the actual structure variable.

### 112.3 Passing Pointers to Structures as Arguments

If the size of a structure is very large, then it is not efficient to pass whole structure to the function since a copy of it has to be made inside the called function. In this case it is better to send address of the structure.

We can access the members of the structure variable inside the calling function using arrow operator. In this case any changes made to the structure variable inside the called function, will be visible in the calling function since we are actually working on the original structure variable.

*/\*P11.8 Program to understand how a pointer to structure variable is sent to a function\*/*

```
#include<stdio.h>
struct student
{
 char name[20];
 int rollno;
 int marks;
};
void display(struct student *);
void inc_marks(struct student *);
int main(void)
{
 struct student stu1={"John",12,87};
 struct student stu2={"Mary",18,90};
 inc_marks(&stu1);
 inc_marks(&stu2);
 display(&stu1);
 display(&stu2);
}
```

```
return 0;
void inc_marks(struct student *stuptr)
{
 (stuptr->marks)++;
}
void display(struct student *stuptr)
{
 printf("Name - %s\t",stuptr->name);
 printf("Rollno - %d\t",stuptr->rollno);
 printf("Marks - %d\n",stuptr->marks);
}
```

Output:  
Name - John Rollno - 12 Marks - 88  
Name - Mary Rollno - 18 Marks - 91

### 11.2.4 Returning a Structure Variable from Function

Structure variables can be returned from functions as any other variable. The returned value can be assigned to a structure of the appropriate type.

*/\*P11.9 Program to understand how a structure variable is returned from a function\*/*

```
#include<stdio.h>
struct student
{
 char name[20];
 int rollno;
 int marks;
};
void display(struct student);
struct student change(struct student stu);
int main(void)
{
 struct student stu1={"John",12,87};
 struct student stu2={"Mary",18,90};
 stu1=change(stu1);
 stu2=change(stu2);
 display(stu1);
 display(stu2);
 return 0;
}
struct student change(struct student stu)
{
 stu.marks=stu.marks + 5;
 stu.rollno=stu.rollno-10;
 return stu;
}
void display(struct student stu)
{
 printf("Name - %s\t", stu.name);
 printf("Rollno - %d\t", stu.rollno);
 printf("Marks - %d\n", stu.marks);
}
```

Output:  
Name - John Rollno - 2 Marks - 92  
Name - Mary Rollno - 8 Marks - 95

### 11.2.5 Returning a Pointer to Structure from a Function

Pointers to structures can also be returned from functions. In the following program, the function func() returns a pointer to structure.

*/\*P11.10 Program to understand how a pointer to structure is returned from a function\*/*

```
#include<stdio.h>
```

```
#include<string.h>
struct student
{
 char name[20];
 int rollno;
 int marks;
};

void display(struct student *);
struct student *func(struct student *p1, struct student *p2);
int main(void)
{
 struct student *stuptr;
 struct student stu1={"Dev",23,78}, stu2={"Ved",12,89};
 stuptr=func(&stu1, &stu2);
 display(stuptr);
 return 0;
}

struct student *func(struct student *p1, struct student *p2)
{
 if(p1->marks > p2->marks)
 return p1;
 else
 return p2;
}

void display(struct student *stuptr)
{
 printf("Name - %s\t", stuptr->name);
 printf("Rollno - %d\t", stuptr->rollno);
 printf("Marks - %d\n", stuptr->marks);
}

Output:
Ved 12 89
```

### 11.2.6 Passing Array of Structures as Argument

We can pass an array of structures to a function, where each element of array is of structure type.

*/\*P11.11 Program to understand how an array of structures is sent to a function\*/*

```
#include<stdio.h>
struct student
{
 char name[20];
 int rollno;
 int marks;
};

void display(struct student);
void dec_marks(struct student stuarr[]);
int main(void)
{
 int i;
 struct student stuarr[3]={
 {"Mary",12,98},
 {"John",11,97},
 {"Tom",13,89}
 };

 dec_marks(stuarr);
 for(i=0; i<3; i++)
 display(stuarr[i]);
 return 0;
}

void dec_marks(struct student stuarr[])
{
 int i;
 for(i=0; i<3; i++)
```

```
stuarr[i].marks=stuarr[i].marks 10;
```

```
void display(struct student stu)
```

```
printf("Name - %s\t", stu.name);
printf("Rollno - %d\t", stu.rollno);
printf("Marks - %d\n", stu.marks);
```

Output:

```
Name - Mary Rollno - 12 Marks - 88
Name - John Rollno - 11 Marks - 87
Name - Tom Rollno - 13 Marks - 79
```

All the changes made in the array of structures inside the called function will be visible in the calling function. The following program calculates the total marks and grade of students, and then sorts the names of students on the basis of marks and prints out the sorted records.

*/\*P11.12 Program to print the sorted records of students\*/*

```
#include<stdio.h>
#define N 5
struct student
{
 char name[20];
 int rollno;
 int marks[6];
 int total;
 char grade;
};

void display(struct student arr);
void calculate(struct student arr[]);
void sort(struct student arr[]);
int main(void)
{
 struct student stu[N],temp;
 int i,j;
 for(i=0; i<N; i++)
 {
 printf("Enter name : ");
 scanf("%s",stu[i].name);
 printf("Enter rollno : ");
 scanf("%d",&stu[i].rollno);
 stu[i].total=0;
 printf("Enter marks in 6 subjects : ");
 for(j=0; j<6; j++)
 scanf("%d",&stu[i].marks[j]);
 }

 calculate(stu);
 sort(stu);
 for(i=0; i<N; i++)
 display(stu[i]);
 return 0;
}

void calculate(struct student stu[])
{
 int i,j;
 for(i=0; i<N; i++)
 {
 for(j=0; j<6; j++)
 stu[i].total += stu[i].marks[j];
 if(stu[i].total > 500)
 stu[i].grade='A';
 else if(stu[i].total > 400)
 stu[i].grade='B';
 else if(stu[i].total > 250)
 stu[i].grade='C';
 }
```

```

else
 stu[i].grade='D';
}
}

void sort(struct student stu[])
{
 int i,j;
 struct student temp;
 for(i=0; i<N-1; i++)
 for(j=i+1; j<N; j++)
 if(stu[i].total < stu[j].total)
 {
 temp=stu[i];
 stu[i]=stu[j];
 stu[j]=temp;
 }
}

void display(struct student stu)
{
 printf("Rollno - %d\t", stu.rollno);
 printf("Name - %s\n", stu.name);
 printf("Total - %d\t", stu.total);
 printf("Grade -%c\n\n", stu.grade);
}

```

In the following program the records are sorted on different keys depending on the choice of the user.

```

/*P11.13 Program to sort the records on different keys*/
#include<stdio.h>
#define N 5
struct date
{
 int day;
 int month;
 int year;
};
struct employee
{
 char name[20];
 struct date dob;
 struct date doj;
 int salary;
};
void sort_name(struct employee emp[]);
void sort_dob(struct employee emp[]);
void sort_doj(struct employee emp[]);
void sort_salary(struct employee emp[]);
void display(struct employee emp[]);
int datecmp(struct date date1, struct date date2);
int main(void)
{
 struct employee emp[N];
 int i, choice;
 for(i=0; i<N; i++)
 {
 printf("Enter name :");
 scanf("%s", emp[i].name);
 printf("Enter date of birth(dd/mm/yy): ");
 scanf("%d/%d/%d", &emp[i].dob.day, &emp[i].dob.month, &emp[i].dob.year);
 printf("Enter date of joining(dd/mm/yy): ");
 scanf("%d/%d/%d", &emp[i].doj.day, &emp[i].doj.month, &emp[i].doj.year);
 printf("Enter salary : ");
 scanf("%d", &emp[i].salary);
 printf("\n");
 }
 while(1)

```

```

{
 printf("1.Sort by name\n");
 printf("2.Sort by date of birth\n");
 printf("3.Sort by date of joining\n");
 printf("4.Sort by salary\n");
 printf("5.Exit\n");
 printf("6.Enter your choice :");
 scanf("%d", &choice);
 switch(choice)
 {
 case 1:
 sort_name(emp);
 break;
 case 2:
 sort_dob(emp);
 break;
 case 3:
 sort_doj(emp);
 break;
 case 4:
 sort_salary(emp);
 break;
 case 5:
 exit(1);
 default:
 printf("Wrong choice\n");
 }
 printf("\n");
 display(emp);
}

return 0;

void display(struct employee emp[])
{
 int i;
 printf("NAME\t\tDOB\t\tDOJ\t\tSALARY\n");
 for(i=0; i<N; i++)
 {
 printf("%s\t\t", emp[i].name);
 printf("%d/%d/%d\t\t", emp[i].dob.day, emp[i].dob.month, emp[i].dob.year);
 printf("%d/%d/%d\t\t", emp[i].doj.day, emp[i].doj.month, emp[i].doj.year);
 printf("%d\n", emp[i].salary);
 }
 printf("\n");
}

void sort_name(struct employee emp[])
{
 struct employee temp;
 int i,j;
 for(i=0; i<N-1; i++)
 for(j=i+1; j<N; j++)
 if(strcmp(emp[i].name, emp[j].name) > 0)
 {
 temp=emp[i];
 emp[i]=emp[j];
 emp[j]=temp;
 }
}

void sort_dob(struct employee emp[])
{
 struct employee temp;
 int i,j;
 for(i=0; i<N-1; i++)
 for(j=i+1; j<N; j++)
 if(datecmp(emp[i].dob, emp[j].dob) > 0)

```

```

 temp=emp[i];
 emp[i]=emp[j];
 emp[j]=temp;
}

void sort_doj(struct employee emp[])
{
 struct employee temp;
 int i,j;
 for(i=0; i<N-1; i++)
 for(j=i+1; j<N; j++)
 if(datecmp(emp[i].doj,emp[j].doj)> 0)
 {
 temp=emp[i];
 emp[i]=emp[j];
 emp[j]=temp;
 }
}

void sort_salary(struct employee emp[])
{
 struct employee temp;
 int i,j;
 for(i=0; i<N-1; i++)
 for(j=i+1; j<N; j++)
 if(emp[i].salary < emp[j].salary)
 {
 temp=emp[i];
 emp[i]=emp[j];
 emp[j]=temp;
 }
}

/*Returns 1 if date1 < date2, returns -1 if date1 > date2, return 0 if equal*/
int datecmp(struct date date1, struct date date2)
{
 if(date1.year < date2.year)
 return 1;
 if(date1.year > date2.year)
 return -1;
 if(date1.month < date2.month)
 return 1;
 if(date1.month > date2.month)
 return -1;
 if(date1.day < date2.day)
 return 1;
 if(date1.day > date2.day)
 return -1;
 return 0;
}

```

### 1.3 Self Referential Structures

A structure that contains pointers to structures of its own type is known as self referential structure. For example-

```

struct tag
{
 datatype member1;
 datatype member2;

 struct tag *ptr1;
 struct tag *ptr2;
}

```

Here ptr1 and ptr2 are structure pointers that can point to structure variables of type struct tag, so struct tag is a self referential structure. These types of structures are helpful in implementing data structures like linked lists and trees. We will discuss the data structure linked list in this chapter.

### 11.4 Linked List

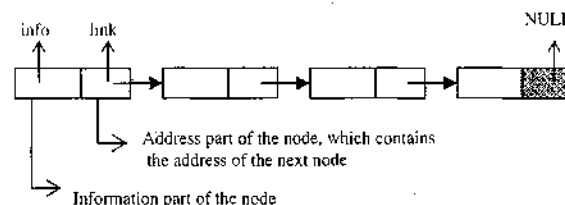
List is a collection of elements. There are two ways of maintaining a list in computer memory. The first way is to take an array for storing the elements of the list, but arrays have some disadvantages. Insertion and deletion of an element from an array requires more processing. If the number of elements in the list is less than the size of array then memory will be wasted and if the number of elements exceeds the size of array then also we will have problems. The second way of implementing a list in memory is by using a self referential structure. These types of lists are known as linked lists. A linked list is made up of nodes where each node has two parts, first part contains the information and second part contains the address of the next node. The address part of the last node of linked list will be NULL. The general form of a node of linked list is-

```

struct node
{
 type member1;
 type member2;

 struct node *link; /*Pointer to next node of the list*/
}

```



The nodes of a linked list are not stored contiguously in memory. In array list we could perform all the operations using the array name. In the case of linked list, we'll perform all the operations with the help of a pointer that points to the first node of the linked list. This pointer is generally named start and it is the only source through which we can access our linked list. The list will be considered empty if the pointer start contains NULL value.

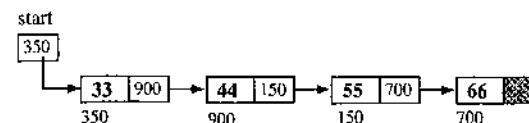
Now we will take a linked list that contains an integer value in the information part. The structure for the nodes of this list will be defined as-

```

struct node
{
 int info;
 struct node *link;
}

```

Here is a linked list that has 4 nodes-



We can clearly see that the address of each node is stored in the link part of the previous node, and the address of first node is stored in the pointer variable start. The link part of the last node contains NULL. The pointer variable start will be declared and initialized to NULL.



```
struct node *start=NULL;
```

Now we will discuss the following operations on a linked list.

- (i) Traversal of a linked list
- (ii) Searching an element
- (iii) Insertion of an element
- (iv) Deletion of an element
- (v) Creation of a linked list
- (vi) Reversal of a linked list

The main() function and declarations are given here, and the code of other functions are given with the explanation.

```
/*P11.14 Program of linked list*/
#include<stdio.h>
#include<stdlib.h>
struct node
{
 int info;
 struct node *link;
};
struct node *create_list(struct node *start);
void display(struct node *start);
void count(struct node *start);
void search(struct node *start, int data);
struct node *addatbeg(struct node *start, int data);
struct node *addatend(struct node *start, int data);
struct node *addafter(struct node *start, int data, int item);
struct node *addbefore(struct node *start, int data, int item);
struct node *addatpos(struct node *start, int data, int pos);
struct node *del(struct node *start, int data);
struct node *reverse(struct node *start);

int main(void)
{
 struct node *start=NULL;
 int choice, data, item, pos;
 while(1)
 {
 printf("1.Create List\n");
 printf("2.Display\n");
 printf("3.Count\n");
 printf("4.Search\n");
 printf("5.Add to empty list / Add at beginning\n");
 printf("6.Add at end\n");
 printf("7.Add after node\n");
 printf("8.Add before node\n");
 printf("9.Add at position\n");
 printf("10.Delete\n");
 printf("11.Reverse\n");
 printf("12.Quit\n");
 printf("Enter your choice : ");
 scanf("%d", &choice);

 switch(choice)
 {
 case 1:
 start=create_list(start);
 break;
 case 2:
 display(start);
 break;
 case 3:
 count(start);
```

```
 break;
 case 4:
 printf("Enter the element to be searched : ");
 scanf("%d", &data);
 search(start, data);
 break;
 case 5:
 printf("Enter the element to be inserted : ");
 scanf("%d", &data);
 start=addatbeg(start, data);
 break;
 case 6:
 printf("Enter the element to be inserted : ");
 scanf("%d", &data);
 start=addatend(start, data);
 break;
 case 7:
 printf("Enter the element to be inserted : ");
 scanf("%d", &data);
 printf("Enter the element after which to insert : ");
 scanf("%d", &item);
 start=addafter(start, data, item);
 break;
 case 8:
 printf("Enter the element to be inserted : ");
 scanf("%d", &data);
 printf("Enter the element before which to insert : ");
 scanf("%d", &item);
 start=addbefore(start, data, item);
 break;
 case 9:
 printf("Enter the element to be inserted : ");
 scanf("%d", &data);
 printf("Enter the position at which to insert : ");
 scanf("%d", &pos);
 start=addatpos(start, data, pos);
 break;
 case 10:
 printf("Enter the element to be deleted : ");
 scanf("%d", &data);
 start=del(start, data);
 break;
 case 11:
 start=reverse(start);
 break;
 case 12:
 exit(1);
 default:
 printf("Wrong choice\n");
 }
 }
 /*End of switch*/
 /*End of while*/
 return 0;
}
/*End of main()*/
```

In the function main(), we have taken an infinite loop and inside the loop we have written a switch statement. In the different cases of this switch statement, we have implemented different operations of linked list. To come out of the infinite loop and exit the program we have used the function exit().

The structure pointer start is declared in main() and initialized to NULL. We send this pointer to all functions because start is the only way of accessing linked list. Functions like those of insertion, deletion, reversal will change our linked list and value of start might change so these functions return the value of start. The functions like display(), count(), search() do not change the linked list so their return type is void.

### 1.4.1 Traversing a Linked List

Traversal means visiting each node, starting from the first node till we reach the last node. For this we will take a structure pointer `p` which will point to the node that is currently being visited. Initially we have to visit the first node so `p` is assigned the value of `start`.

```
p=start;
```

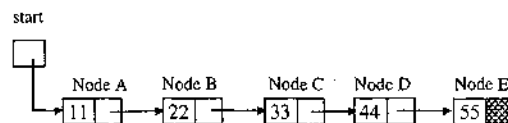
Now `p` points to the first node of linked list. We can access the info part of first node by writing `p->info`. Now we have to shift the pointer `p` forward so that it points to the next node. This can be done by assigning the address of the next node to `p` as-

```
p=p->link;
```

Now `p` has address of the next node. Similarly we can visit each node of linked list through this assignment until `p` has `NULL` value, which is link part value of last element. So the linked list can be traversed as-

```
printf("%d ",p->info);
p=p->link;
```

Let us take an example to understand how the assignment `p=p->link` makes the pointer `p` move forward. From now onwards we will not show the addresses, we will show only the info part of the list in the figures.



In figure 3.4, node A is the first node so `start` points to it, node E is the last node so its link is `NULL`.

Initially `p` points to node A, `p->info` gives 11 and `p->link` points to node B

After the statement `p=p->link`;

`p` points to node B, `p->info` gives 22 and `p->link` points to node C

After the statement `p=p->link`;

`p` points to node C, `p->info` gives 33 and `p->link` points to node D

After the statement `p=p->link`;

`p` points to node D, `p->info` gives 44 and `p->link` points to node E

After the statement `p=p->link`;

`p` points to node E, `p->info` gives 55 and `p->link` is `NULL`

After the statement `p=p->link`;

`p` becomes `NULL`, i.e. we have reached the end of the list so we come out of the loop.

The following function `display()` displays the contents of the linked list.

```
void display(struct node *start)
```

```
{
 struct node *p;
 if(start == NULL)
 {
 printf("List is empty\n");
 return;
 }
 p=start;
 printf("List is :\n");
 while(p != NULL)
 {
 printf("%d ",p->info);
 p=p->link;
 }
 printf("\n\n");
}
```

```
/*End of display() */
```

Don't think of using `start` for moving forward. If we use `start=start->link`, instead of `p=p->link` then we will lose `start` and that is the only means of accessing our list. The following function `count()` finds out the number of elements of the linked list.

```
int count(struct node *start)
```

```
{
 struct node *p;
 int cnt=0;
 p=start;
 while(p!=NULL)
 {
 p=p->link;
 cnt++;
 }
 printf("Number of elements are %d\n",cnt);
}
/*End of count() */
```

### 1.4.2 Searching in a Linked List

For searching an element, we traverse the linked list and while traversing we compare the info part of each element with the given element to be searched. In the function given below, `item` is the element which we want to search.

```
int search(struct node *start,int item)
```

```
{
 struct node *p=start;
 int pos=1;
 while(p!=NULL)
 {
 if(p->info == item)
 {
 printf("Item %d found at position %d\n",item,pos);
 return;
 }
 p=p->link;
 pos++;
 }
 printf("Item %d not found in list\n",item);
}
/*End of search() */
```

### 1.4.3 Insertion in a Linked List

There can be four cases while inserting a node in a linked list.

1. Insertion at the beginning.

2. Insertion in an empty list.

3. Insertion at the end.

4. Insertion in between the list nodes.

To insert a node, initially we will dynamically allocate space for that node using `malloc()`. Suppose `tmp` is a pointer that points to this dynamically allocated node. In the info part of the node we will put the data value.

```
tmp=(struct node *)malloc(sizeof(struct node));
```

```
tmp->info=data;
```

The link part of the node contains garbage value; we will assign address to it separately in the different cases. In our explanation we will refer to this new node as node T.

#### 1.4.3.1 Insertion at the beginning of the list

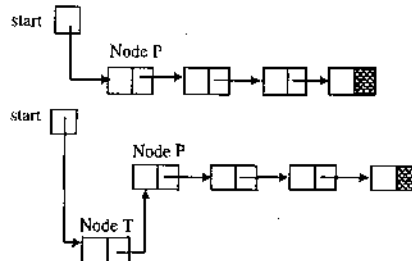
We have to insert node T at the beginning of the list. Suppose the first node of list is node P, so the new node T should be inserted before it.

#### Before insertion

Node P is the first node  
start points to node P

#### After insertion

Node T is first node  
Node P is second node  
start points to node T  
Link of node T points to node P



Insertion at the beginning of the list

(i) Link of node T should contain the address of node P, and we know that start has address of node P so we should write-

```
tmp->link=start;
```

After this statement, link of node T will point to node P.

(ii) We want to make node T the first node; hence we should update start so that now it points to node T.

```
start=tmp;
```

The order of the above two statements is important. First we should make link of T equal to start and after that only we should update start. Let's see what happens if the order of these two statements is reversed.

```
start=tmp;
```

start points to tmp and we lost the address of node P.

```
tmp->link=start;
```

Link of tmp will point to itself because start has address of tmp. So if we reverse the order, then link of node T will point to itself and we will be stuck in an infinite loop when the list is processed.

The following function `addatbeg()` adds a node at the beginning of the list.

```
struct node *addatbeg(struct node *start, int data)
{
 struct node *tmp;
 tmp=(struct node *)malloc(sizeof(struct node));
 tmp->info=data;
 tmp->link=start;
 start=tmp;
 return start;
}/*End of addatbeg()*/
```

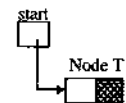
### 11.4.3.2 Insertion in an empty list

**Before insertion**  
start is NULL



#### After insertion

T is the only node  
start points to T  
link of T is NULL



Insertion in an empty list

When the list is empty, value of start will be NULL. The new node that we are adding will be the only node in the list. Since it is the first node, start should point to this node and it is also the last node so its link should be NULL.

```
tmp->link=NULL;
start=tmp;
```

Since initially start was NULL, we can write start instead of NULL in the first statement, so now these two statements can be written as-

```
tmp->link=start;
start=tmp;
```

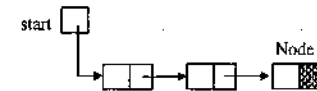
These two statements are same as in the previous case (11.4.3.1), so we can see that this case reduces to the previous case of insertion in the beginning and the same code can be written for both the cases.

### 11.4.3.3 Insertion at the end of the list

We have to insert a new node T at the end of the list. Suppose the last node of list is node P, so node T should be inserted after node P.

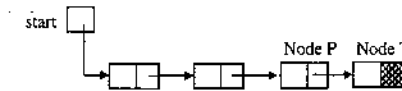
#### Before insertion

Node P is the last node  
Link of node P is NULL



#### After insertion

Node T is last node  
Node P is second last node  
Link of node T is NULL  
Link of P points to node T



Insertion at the end of the list

Suppose we have a pointer p pointing to the node P. These are the two statements that should be written for this insertion-

```
p->link=tmp;
tmp->link=NULL;
```

So in this case we should have a pointer p pointing to the last node of the list. The only information about the linked list that we have is the pointer start. So we will traverse the list till the end to get the pointer p and then do the insertion. This is how we can obtain the pointer p.

```
p=start;
while(p->link!=NULL)
 p=p->link;
```

In traversal of list(11.4.1) our terminating condition was  $(p!=NULL)$ , because there we wanted the loop to terminate when p becomes NULL. Here we want the loop to terminate when p is pointing to the last node so the terminating condition is  $(p->link!=NULL)$ .

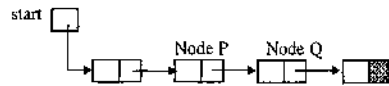
The following function `addatend()` inserts a node at the end of the list.

```
struct node *addatend(struct node *start, int data)
{
 struct node *p,*tmp;
 tmp=(struct node *)malloc(sizeof(struct node));
 tmp->info=data;
 p=start;
 while(p->link!=NULL)
 p=p->link;
 p->link=tmp;
 tmp->link=NULL;
 return start;
}/*End of addatend()*/
```

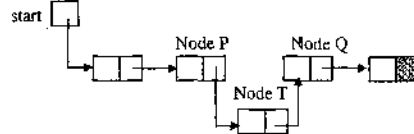
### 3.4 Insertion in between the list nodes

ave to insert a node T between nodes P and Q.

**insertion**  
Q is after node P  
if P points to node Q



**insertion**  
T is between nodes P and Q  
if node T points to node Q  
if node P points to node T



Insertion in between the list nodes

Suppose we have two pointers *p* and *q* pointing to nodes P and Q respectively. The two statements that should be written for insertion of node T are-

```
p->link=q;
link=tmp;
```

Before insertion address of node Q is in *p->link*, so instead of pointer *q* we can write *p->link*. Now the statements for insertion can be written as-

```
p->link=p->link;
link=tmp;
```

The order of these two statements is important, if you write them in the reverse order then you will lose links. The address of node Q is in *p->link*, suppose we write the statement *link=tmp;* first then we will lose the address of node Q, there is no way to reach node Q and our list is broken. So first we should assign the address of node Q to the link of node T by writing *p->link=p->link*. Now we have stored the address of node Q, so we are free to change *p->link*.

we will see three cases of insertion in between the nodes-

Insertion after a node

Insertion before a node

Insertion at a given position

The two statements of insertion (*tmp->link=p->link; p->link=tmp;*) will be written in all the three cases, but the way of finding the pointer *p* will be different.

#### 3.4.1 Insertion after a node

In this case we are given a value from the list, and we have to insert the new node after the node that contains this value. Suppose the node P contains the given value, and node Q is its successor. We have to insert the new node T after node P, i.e. T is to be inserted between nodes P and Q. In the function given below, *data* is the new value to be inserted and *item* is the value contained in node P. For writing the two statements of insertion (*tmp->link=p->link; p->link=tmp;*), we need to find the pointer *p* which points to the node that contains *item*. The procedure is same as we have seen in searching of an element in the linked list.

```
struct node *addafter(struct node *start,int data,int item)
```

```

{
 struct node *tmp,*p;
 p=start;
 while(p!=NULL)
 {
 if(p->info == item)
 {
 tmp=(struct node *)malloc(sizeof(struct node));

```

```

 tmp->info=data;
 tmp->link=p->link;
 p->link=tmp;
 return start;
 }
 }

```

```
p=p->link;
```

```

}
printf("%d not present in the list\n",item);
return start;
/*End of addafter()*/

```

Let us see what happens if *item* is present in the last node and we have to insert after the last node. In this case *p* points to last node and its link is NULL, so *tmp->link* is automatically assigned NULL and we don't have any need for a special case of insertion at the end. This function will work correctly even if we insert after the last node.

#### 3.4.3.2 Insertion before a node

In this case we are given a value from the list, and we have to insert the new node before the node that contains this value. Suppose the node Q contains the given value, and node P is its predecessor. We have to insert the new node T before node Q, i.e. T is to be inserted between nodes P and Q. In the function given below, *data* is the new value to be inserted and *item* is the value contained in node Q. For writing the two statements of insertion (*tmp->link=p->link; p->link=tmp;*) we need to find the pointer *p* which points to the predecessor of the node that contains *item*. Since *item* is present in Q and we have to find pointer to node P, here the condition for searching would be *if(p->link->info == item)* and terminating condition of the loop would be *(p->link!=NULL)*.

```
struct node *addbefore(struct node *start,int data,int item)
```

```

{
 struct node *tmp,*p;
 if(start == NULL)
 {
 printf("List is empty\n");
 return start;
 }
 /*If data to be inserted before first node*/
 if(item == start->info)
 {
 tmp=(struct node *)malloc(sizeof(struct node));
 tmp->info=data;
 tmp->link=start;
 start=tmp;
 return start;
 }
 p=start;
 while(p->link!=NULL)
 {
 if(p->link->info == item)
 {
 tmp=(struct node *)malloc(sizeof(struct node));
 tmp->info=data;
 tmp->link=p->link;
 p->link=tmp;
 return start;
 }
 p=p->link;
 }
 printf("%d not present in the list\n",item);
 return start;
/*End of addbefore()*/
}

```

If the node is to be inserted before the first node, then that case has to be handled separately because we have to update start in this case. If the list is empty, start will be NULL and the term start->info will create problems so before checking the condition if (item == start->info) we should check for empty list.

#### 11.4.3.4.3 Insertion at a given position

In this case we have to insert the new node at a given position. The two insertion statements are same as in the previous cases (tmp->link=p->link; p->link=tmp;). The way of finding pointer p is different. If we have to insert at the first position we will have to update start so that case is handled separately.

```
struct node *addatpos(struct node *start, int data, int pos)
{
 struct node *tmp, *p;
 int i;
 p = start;
 for(i=1; i<pos-1 && p!=NULL; i++)
 p=p->link;
 if(p==NULL)
 printf("There are less than %d elements\n", pos);
 else
 {
 tmp=(struct node *)malloc(sizeof(struct node));
 tmp->info=data;
 if(pos==1)
 {
 tmp->link=start;
 start=tmp;
 }
 else
 {
 tmp->link=p->link;
 p->link=tmp;
 }
 }
 return start;
} /*End of addatpos()*/
```

### 11.4.4 Creation of a Linked List

A list can be created using the insertion operations. First time we will have to insert into an empty list, and then we will keep on inserting nodes at the end of the list. The case of insertion in an empty list reduces to the case of insertion in the beginning, so for inserting the first node we will call addatbeg(), and then for insertion of all other nodes we will call addatend().

```
struct node *create_list(struct node *start)
{
 int i, n, data;
 printf("Enter the number of nodes : ");
 scanf("%d", &n);
 start=NULL;
 if(n==0)
 return start;
 printf("Enter the element to be inserted : ");
 scanf("%d", &data);
 start=addatbeg(start, data);
 for(i=2; i<=n; i++)
 {
 printf("Enter the element to be inserted : ");
 scanf("%d", &data);
 start=addatend(start, data);
 }
 return start;
}
```

```
/*End of create_list()*/
```

### 11.4.5 Deletion in a Linked List

For deletion of any node, the pointers are rearranged so that this node is logically removed from the list. To physically remove the node and return the memory occupied by it to the pool of available memory we will use the function free(). We will take a pointer variable tmp which will point to the node being deleted so that after the pointers have been altered we will still have address of that node in tmp to free it. There can be four cases while deleting an element from a linked list.

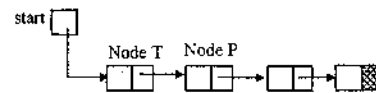
1. Deletion of first node.
2. Deletion of the only node.
3. Deletion in between the list.
4. Deletion at the end.

In all the cases, at the end we should call free(tmp) to physically remove node T from the memory.

#### 11.4.5.1 Deletion of first node

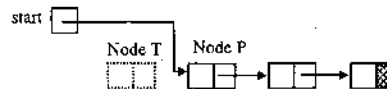
Before deletion

Node T is the first node  
start points to node T  
Link of node T points to node P



After deletion

Node P is the first node  
start points to node P



Deletion of first node

Since the node to be deleted is the first node, tmp will be assigned the address of first node.

tmp=start;

So now tmp points to the first node, which has to be deleted. Since start points to the first node of linked list, start->link will point to the second node of linked list. After deletion of first node, the second node (node P) would become the first one, so start should be assigned the address of the node P as-

start=start->link;

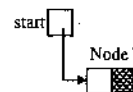
After this statement, start points to node P so now it is the first node of the list.

#### 11.4.5.2 Deletion of the only node

If there is only one node in the list and we have to delete it, then after deletion the list would become empty and start would have NULL value.

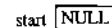
Before deletion

T is the only node  
start points to T  
link of T is NULL



After deletion

start is NULL



Deletion of the only node

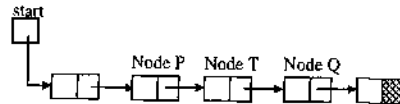
```
tmp=start;
start=NULL;
```

In the second statement, we can write `start->link` instead of `NULL`. So this case reduces to the first case (11.4.5.1).

### 11.4.5.3 Deletion in between the list nodes

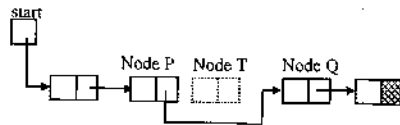
**Before deletion**

link of P points to node T  
link of T points to node Q



**After deletion**

Node Q is after node P  
link of P points to node Q



Deletion in between the list nodes

Suppose node T is to be deleted and pointer `tmp` points to it and we have pointers `p` and `q` which point to nodes P and Q respectively. For deletion of node T we will just link the predecessor of T (node P) to successor of T (node Q).

```
p->link=q;
```

The address of `q` is stored in `tmp->link` so instead of `q` we can write `tmp->link` in the above statement.

```
p->link=tmp->link;
```

The value to be deleted is in node T and we need a pointer to its predecessor which is node P, so as in `ddbefore()` our condition for searching will be `if(p->link->info == data)`. Here `data` is the element to be deleted.

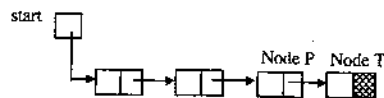
```
=start;
while(p->link!= NULL)
```

```
if(p->link->info == data)
{
 tmp=p->link;
 p->link=tmp->link;
 free(tmp);
 return start;
}
p=p->link;
```

### 11.4.5.4 Deletion at the end of the list

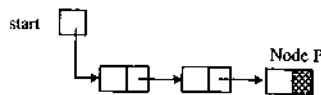
**Before deletion**

Node T is the last node  
link of P points to T  
link of node T is NULL



**After deletion**

Node P is last node  
link of node P is NULL



Deletion at the end of the list

`p` is a pointer to node P, then node T can be deleted by writing the following statement.

```
p->link=NULL;
```

Since link of node T is `NULL`, in the above statement instead of `NULL` we can write `tmp->link`. Hence we can write this statement as-

```
p->link=tmp->link;
```

So we can see that this case reduces to the previous case (11.4.5.3).

Now we will write a function for deletion of an element from the list. We can't delete from an empty list so firstly we will check if the list is empty. If the element to be deleted is the first element of the list then it is deleted accordingly and we return from the function.

Now the element can either be in between the list nodes or at end of the list. These two cases can be handled in the same way. If we reach the end of list and node containing the value is not found, then a message is displayed.

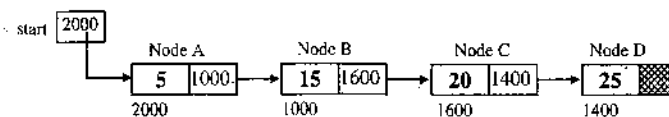
```
struct node *del(struct node *start, int data)
{
 struct node *tmp, *p;
 if(start == NULL)
 {
 printf("List is empty\n");
 return start;
 }
 if(start->info == data) /*Deletion of first node*/
 {
 tmp=start;
 start=start->link;
 free(tmp);
 return start;
 }
 p=start; /*Deletion in between or at the end*/
 while(p->link!=NULL)
 {
 if(p->link->info == data)
 {
 tmp=p->link;
 p->link=tmp->link;
 free(tmp);
 return start;
 }
 p=p->link;
 }
 printf("Element %d not found\n", data);
 return start;
} /*End of del()*/
```

### 11.4.6 Reversing a Linked List

The following changes need to be done in a linked list for reversing it.

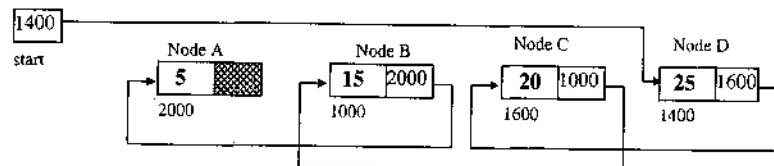
1. First node should become the last node of linked list.
2. Last node should become the first node of linked list and now `start` should point to it.
3. Link of 2<sup>nd</sup> node should point to 1<sup>st</sup> node, link of 3<sup>rd</sup> node should point to 2<sup>nd</sup> node and so on.

Let us take a linked list and reverse it.



- (i) Node A is the first node so start points to it.
- (ii) Node D is the last node so its link is NULL.
- (iii) Link of A points to B, link of B points to C and link of C points to D.

After reversing, the linked list would be-



- (i) Node D is the first node so start points to it.
- (ii) Node A is the last node so its link is NULL.
- (iii) Link of D points to C, link of C points to B and link of B points to A.

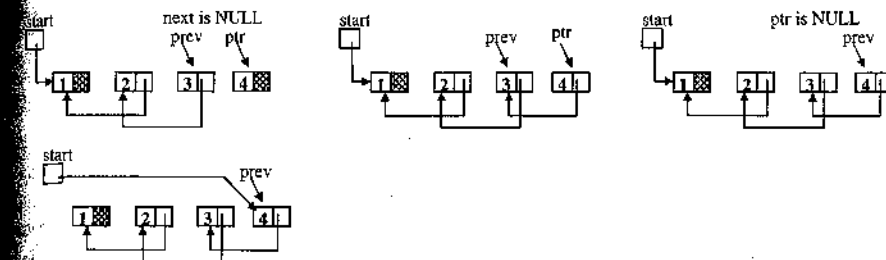
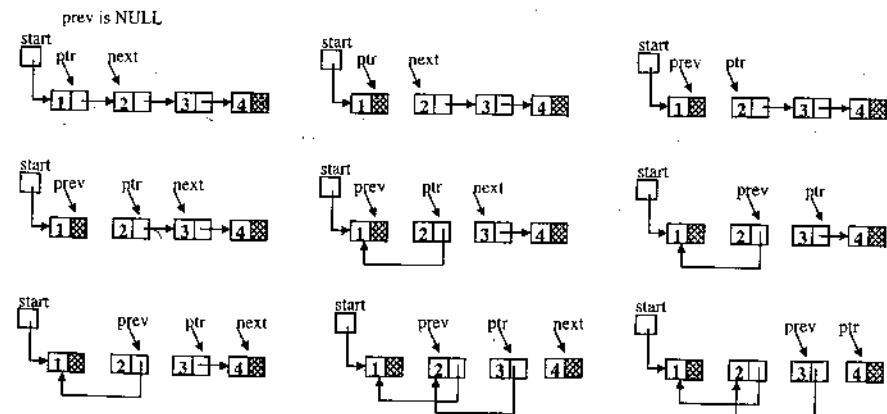
Now let us see how we can make a function for the reversal of a linked list. We will take three pointers prev, ptr and next. Initially the pointer ptr will point to start and prev will be NULL. In each pass, first the link of pointer ptr is stored in pointer next and after that the link of ptr is changed so that it points to its previous node. The pointers prev and ptr are moved forward. Here is the function reverse() that reverses a linked list.

```

struct node *reverse(struct node *start)
{
 struct node *prev, *ptr, *next;
 prev=NULL;
 ptr=start;
 while(ptr!=NULL)
 {
 next=ptr->link;
 ptr->link=prev;
 prev=ptr;
 ptr=next;
 }
 start=prev;
 return start;
}/*End of reverse()*/

```

The following example shows all the steps of reversing a linked list.



## 1.5 union

Union is a derived data type like structure and it can also contain members of different data types. The syntax used for declaration of a union type, declaration of union variables and for accessing members is similar to that used in structures, but here keyword union is used instead of struct. The main difference between union and structure is in the way memory is allocated for the members. In a structure each member has its own memory location, whereas members of union share the same memory location. When a variable of type union is declared, compiler allocates sufficient memory to hold the largest member in the union. Since all members share the same memory location, we can use only one member at a time. Thus union is used for saving memory. The concept of union is useful when it is not necessary to use all members of the union at a time. The syntax of declaration of a union is-

```
union union_name
```

```

 datatype member1;
 datatype member2;

```

Like structure variables, union variables can be declared along with the declaration or separately. For example-

```
union union_name
```

```

 datatype member1;
 datatype member2;

```

```
variable_name;
```

```
union union_name variable_name;
```

We can access the union members using the same syntax used for structures. If we have a union variable then the members can be accessed using dot(.) operator, and if we have a pointer to union then the members can be accessed using the arrow (->) operator.

**Ex 11.15 Program for accessing union members \*/**

```

#include<stdio.h>
main(void)
{
 union result
 {
 int marks;
 char grade;
 float per;
 }res;
 res.marks=90;
 printf("Marks : %d\n",res.marks);
 res.grade='A';
 printf("Grade : %c\n",res.grade);

```

```
res.per=85.5;
printf("Percentage : %f\n",res.per);
```

```
Output:
marks:90
grade:A
percentage:85.500000
```

Before the first printf, value 90 is assigned to the union member marks, after first printf, the value 'A' is assigned to the union member grade. Only one member of union can hold value at a time, all members can't be used simultaneously. Thus a union variable of type result can be treated as either an int variable or char variable or a float variable. It is the responsibility of the programmer to keep track of member that currently holds the value, if the programmer tries to extract the value of any other member then the result is implementation defined.

Union variables can also be initialized, but there is a limitation. We know that due to sharing of memory, all members can't hold values simultaneously. So during initialization also only one member can be given an initial value, and this privilege is given to the first member. Hence only the first member of a union can be given an initial value. The type of the initializer should match with the type of the first member. For example, we can initialize the above union variable as-

```
union result res = {78};
```

Now we will take a program and compare the memory allocated for a union and structure variable.

**P11.16 Program to compare the memory allocated for a union and structure variable\*/**

```
#include<stdio.h>
struct stag
```

```
{
char c;
int i;
float f;
```

```
};
union utag
```

```
{
char c;
int i;
float f;
```

```
};
int main(void)
```

```
{
 union utag uvar;
 struct stag svar;
 printf("Size of svar = %u\n", sizeof(svar));
 printf("Address of svar : %p\n", &svar);
 printf("Address of members : %p %p %p\n", &svar.c, &svar.i, &svar.f);
 printf("Size of uvar = %u\n", sizeof(uvar));
 printf("Address of uvar : %p\n", &uvar);
 printf("Address of members : %p %p %p\n", &uvar.c, &uvar.i, &uvar.f);
 return 0;
}
```

```
Output:
Size of svar = 12
Address of svar : 0012FEC0
Address of members : 0012FEC0 0012FEC4 0012FEC8
Size of uvar = 4
Address of uvar : 0012FED4
Address of members : 0012FED4 0012FED4 0012FED4
```

Addresses of members of a union are same while the addresses of members of a structure are different. The difference in the sizes of variables svar and uvar also indicates that union is very economical in the use of memory. The sizes of both union and structure variables may be affected by padding as discussed previously.

Structure may be a member of union or a union may be a member of structure. For example-

```
int marks;
```

```
char grade;
```

```
};
struct res
```

```
{
char name[15];
int age;
union result performance;
};
```

Here data is the structure variable of type struct res. It has three members, an array of characters name, an integer age and a union member performance. Union will take only one value at a time, either an integer value marks or a character value grade. This can also be written as-

```
struct res
{
char name[15];
int age;
union result
{
int marks;
char grade;
}performance;
};
```

If we want to use the member grade then we can write-

```
data.performance.grade
```

Similarly to use the member marks we can write-

```
data.performance.marks
```

Some other features of union are-

- 1) Arrays of unions can be declared.
- 2) Functions can take union variable as arguments and can return union variables.
- 3) Pointers to unions can be declared.
- 4) Unions can be nested.
- 5) Unions can be self referential like structures.

Besides saving memory space, unions can be helpful in other situations also. We can declare an array of unions where each element of array can be used to hold values of different data type. The program below shows this-

**P11.17 Program that uses an array of union to hold values of different data types\*/**

```
#include<stdio.h>
union num
```

```
{
int i;
long int l;
float f;
double d;
```

```
};
int main(void)
```

```
{
 union num arr[10]; /*Size of this array will be 10*sizeof(double)*/
 arr[0].i=12;
 arr[1].l=400000;
 arr[2].f=11.12;
 arr[3].d=23.34;
 printf("%d\n",arr[0].i);
 printf("%ld\n",arr[1].l);
 printf("%f\n",arr[2].f);
 printf("%f\n",arr[3].d);
}
```

```
Output:
12
400000
11.120000
23.340000
```



## 11.6 typedef

The type definition facility allows us to define a new name for an existing data type. The general syntax is-

```
typedef data_type new_name;
```

Here typedef is a keyword, data\_type is any existing data type that may be a standard data type or a user-defined type, new\_name is an identifier, which is a new name for this data type. Note that we are not creating any new data type but we are only creating a new name for the existing data type. For example we can define a new name for int type by writing-

```
typedef int marks;
```

Now marks is a synonym for int and we can use marks instead of int anywhere in the program, for example

```
marks sub1, sub2;
```

Here sub1, sub2 are actually int variables and are similar to any variable declared using int keyword. The above declaration is equivalent to-

```
int sub1, sub2;
```

Some more examples are-

```
typedef unsigned long int ulint;
typedef float real;
```

Here ulint is another name for type unsigned long int, and real is another name for float. The typedef declaration can be written wherever other declarations are allowed. We can give more than one name to a single data type using only one typedef statement. For example-

```
typedef int age, marks, units;
```

In the above typedef statement, we have defined three names for the data type int.

Since typedef is syntactically considered as a storage class, we can't include a storage class in typedef statement. For example statements of these types are invalid-

```
typedef static char schar;
typedef extern int marks;
```

Now we'll see how typedef can be used to define new names for pointers, arrays, functions and structures.

### 1. Pointers

```
typedef float *fptr;
```

After this statement, fptr is a synonym for float \* or pointer to float. Now consider this declaration-

```
fptr p, q, *r;
```

Here p and q are declared as pointer to float, while r is declared as a pointer to pointer to float.

### 2. Array

```
typedef int intarr[10];
```

After this statement intarr is another name for integer arrays of size 10. Now consider this declaration statement-

```
intarr a, b, c[15]; (Equivalent to → int a[10], b[10], c[15][10];)
```

Here a, b are declared as 1-D arrays of size 10, and c is declared as 2-D array of size 15x10.

### 3. Functions

```
typedef float funct(float, int);
```

Here funct is any function that takes two values, one float and one int and returns a float value.

Now consider this declaration statement-

```
funct add, sub, mul, div;
```

Here add, sub, mul, div are declared as functions that take a float and int value as arguments and return float value. The above statement is equivalent to the following declaration statements-

```
float add(float, int);
float sub(float, int);
float mul(float, int);
float div(float, int);
```

### Structures-

We can use typedef for defining a new name for structures. Suppose we have this structure definition-

```
struct studentrec{
 char name[20];
 int marks;
};
```

Now whenever we want to use this structure we have to write struct studentrec. We can give a short and meaningful name to this structure by typedef.

```
typedef struct studentrec Student;
```

Now we can declare variable like this-

```
Student stu1, stu2; (Equivalent to → struct studentrec stu1, stu2;)
```

We can also combine typedef and structure definition. The syntax is as-

```
typedef struct tagname
```

```
datatype member1;
```

```
.....
newname;
```

Here tagname can be same as the newname. We can omit the tagname if the structure is not self referential.

```
typedef struct
```

```
 char name[20];
 int age;
} person;
person student, teacher, emp;
```

Here person is a new name for this structure and we have defined three structure variables, which have the format of the above definition.

In our linked list structure definition, we can write-

```
typedef struct node
{
 int item;
 struct node *link;
} Node;
```

Now we can use Node instead of struct node anywhere in our program-

```
Node *start, *ptr;
```

Here new name is same as that of tagname. Here we can't omit the tagname since we have to define a pointer to the same structure inside the structure.

Advantages of using typedef-

1. It makes our programs more readable and understandable since we can document our program by giving meaningful and descriptive names for existing types.

2. In structures it is important since we can give a single name to the structure, we need not write struct keyword repeatedly.

3. It makes our programs more portable. When program is run on a different machine on which standard data types have different size, only typedef statement has to be changed.

The typedef declarations may seem similar to #define directives, but they are different. The difference is that the typedef declarations are interpreted by the compiler while #define directives are processed by the preprocessor. In #define directive we can define an expansion for any text while in typedef we can only define new names for data types only. The following program shows the difference between #define directive and typedef declaration.

*/\*P11.18 Program to understand the difference between #define and typedef\*/*

```
#include<stdio.h>
#define dp double *
int main(void)
{
 dp p1,p2,p3;
 typedef double *dptr;
 dptr ptr1,ptr2,ptr3;
 printf("%u %u %u\n",sizeof(p1),sizeof(p2),sizeof(p3));
 printf("%u %u %u\n",sizeof(ptr1),sizeof(ptr2),sizeof(ptr3));
}
```

Output:  
4 8 8  
4 4 4

From the output we can see that p1 is declared as a pointer to double while p2 and p3 are declared as double variables. Actually the preprocessor expanded the declaration as-  
double \*p1, p2, p3;

So using #define we could not successfully define a new name for pointer to double type. The variables ptr1, ptr2, ptr3 are all declared as pointers to double, so typedef successfully defined a new name for the type pointer to double.

## Exercise

What is the output of the following programs; assume that stdio.h is included in all programs.

```
1. int main(void)
{
 struct result
 {
 int marks;
 char grade;
 };
 struct result A1, B1;
 A1.marks=80; A1.grade='A';
 B1=A1;
 printf("Marks=%d\t",B1.marks);
 printf("Grade=%c\n",B1.grade);
 return 0;
}

2. int main(void)
{
 struct rec
 {
 char *name;
 int age;
 }*ptr;
 char name[10]="Somalika";
 ptr=(struct rec *)malloc(sizeof(struct rec));
 ptr->name=name;
 ptr->age=93;
 printf("%s\t",ptr->name);
}
```

```
printf("%d\n",ptr->age);
return 0;
```

```
struct student {char name[20]; int age;};
int main(void)
{
 struct student stu1={"Anita", 10},stu2={"Anita",12};
 if(stu1 == stu2)
 printf("Same\n");
 else
 printf("Not same\n");
 return 0;
}
```

```
void func(struct tag v);
int main(void)
```

```
struct tag
{
 int i;
 char c;
};
struct tag var={2,'s'};
func(var);
return 0;
```

```
void func(struct tag v)
{
 printf("%d %c\n",v.i,v.c);
}
```

```
void func(struct {int i; char c;} v);
int main(void)
{
 struct {int i; char c;}var={2,'s'};
 func(var);
 return 0;
}
```

```
void func(struct {int i; char c;} v)
{
 printf("%d %c\n",v.i,v.c);
}
```

```
struct tag{ int i; char c;};
void func(struct tag);
int main(void)
{
 struct tag var={12,'c'};
 func(var);
 printf("%d\n",var.i);
 return 0;
}
```

```
void func(struct tag var)
{
 var.i++;
}
```

```
struct tag{ int i; char c;};
void func(struct tag *);
int main(void)
{
}
```

```

 struct tag var={12,'c'};
 func(&var);
 printf("%d\n",var.i);
 return 0;
}
void func(struct tag *ptr)
{
 ptr->i++;
}

```

```

8. #include<string.h>
int main(void)
{
 union tag
 {
 char name[15];
 int age;
 }rec;
 strcpy(rec.name,"Somalika");
 rec.age=23;
 printf("Name=%s\n",rec.name);
 return 0;
}

```

```

9. struct
{
 char a[20];
 int b;
 union
 {
 double c;
 struct
 {
 char d[15];
 float e;
 }x;
 }y;
}z;
int main(void)
{
 printf("%u %u %u\n", sizeof(z.y.x), sizeof(z.y), sizeof(z));
 return 0;
}

```

```

10. int main(void)
{
 typedef short int s_int;
 unsigned s_int var=3;
 printf("%u", var);
 return 0;
}

```

```

11. typedef struct tag{int i; char c;}tag;
int main(void)
{
 struct tag v1={1,'a'};
 tag v2={2,'b'};
 printf("%d %c %d %c\n",v1.i,v1.c,v2.i,v2.c);
 return 0;
}

```

```

12. typedef struct{char name[20]; int age;}stu;

```

```

typedef struct{int data; node *link;}node;
int main(void)
{
 stu *p=malloc(sizeof(stu));
 node *ptr= malloc(sizeof(node));
 p->age=30;
 ptr->data=3;
 printf("%d %d\n",p->age,ptr->data);
 return 0;
}

```

Write a program to accept name, age and city of five people and display the name and city of eldest one.

Write a program to accept name and arrival time of five trains and display the name with Railway time format.(2PM is written as 14.00 )

Write a program to accept 10 records with the structure-

```

struct {
 char name[20];
 int age;
 float salary;
};

```

Display the records before and after sorting where sorting is based on name.

Write a function to count the number of occurrences of an element in a single linked list.

Write a function to find the smallest and largest element of a single linked list.

Write a function to create a copy of a single linked list.

Write a function to move the largest element to the end of a single linked list.

Write a function to move the smallest element to the beginning of a single linked list.

Write a program to remove first node of the list and insert it at the end, without changing info part of any node.

Write a program to remove the last node of the list and insert it in the beginning, without changing info part of any node.

Write recursive functions for the following operations on a linked list.

- (i) Find length of the list
- (ii) Find sum of all elements in the list
- (iii) Display the linked list
- (iv) Display the list in reverse order
- (v) Reverse the linked list.
- (vi) Insert a node at the end of the list
- (vii) Delete the last node from the list
- (viii) Search for an element in the list

The input and output operations that we have performed so far were done through keyboard and screen only. The data was input through keyboard and was output on the screen. After termination of program, all the entered data is lost because primary memory is volatile. If the data has to be used later, then it becomes necessary to keep it in permanent storage device. There are many I/O library functions through which data can be stored on the disk or any secondary storage device. The stored data can be read whenever required. A data file is a collection of related data placed on a secondary storage device.

File handling in C can be broadly categorized in two types-

High-level Files (standard files or stream oriented files)

Low-level Files(system oriented files)

High level file handling is managed by library functions while low level file handling is managed by system calls. The high level file handling is commonly used since it is easier to manage and hides most of the details from the programmer. In this chapter we will discuss high-level file handling only.

The header file `stdio.h` should be included in the program to make use of the I/O functions. We have already performed I/O with screen and keyboard using functions like `scanf()`, `printf()`, `gets()`, `puts()`, `getchar()` and `putchar()`. The advantage of using stream oriented file I/O is that the I/O in files is similar to screen, keyboard I/O. For example we have functions like `fscanf()`, `fprintf()`, `fgets()`, `fputs()` which are equivalent to the functions that we have studied earlier.

## 2.1 Text and Binary Streams

Input and output in C is performed with the help of logical data streams, which can be connected to different input-output devices. The concept of streams makes input and output on different devices uniform. The programmer is not concerned about the different properties of these devices, all of them can be handled in a similar way. File management is done by the execution environment, and streams are channels through which data can flow from program to environment and from environment to program. A stream is associated with a file by opening the file and is disassociated with the file by closing the file. There are two types of streams - text stream and binary streams. Text stream is a stream of characters and binary stream is a stream of processed bytes.

A text stream is a sequence of characters, arranged in lines where each line consists of zero or more characters and is terminated by a newline character. In binary streams data is represented as it is coded internally in the computer's main memory, without any modifications. Unix system does not make any distinction between text file and binary files.

Text files are in human readable form and they can be created and read using any text editor, while binary files are not in human readable form and they can be created and read only by specific programs written for the purpose. The binary data stored in the file can't be read using a text editor. The integer 167985 will take only 4 bytes in a binary file while it will occupy 6 bytes in a text file because in binary file it is stored as an integer value while in text file it is stored as a sequence of 6 characters i.e. '1', '6', '7', '9', '8', '5'.

Both text files and binary files keep record of the length of the file, and identify the end of file when this length is reached. In text files, there is one more way to detect the end of file. The character with ASCII value 26 is considered to be the end of file character in text files. All input functions stop reading from a text file when this character is encountered and return an end of file signal to the program. C does not insert this character in the file, it can be entered through the keyboard by Ctrl+Z(or Ctrl + D on some systems). In binary files no such character represents the end of file.

In binary format, data is stored in the same way as it is represented in memory so no conversions have to take place while transferring of data between memory and file. In text format some conversions have to take place while transferring data between memory and file. For example while writing to a text file newline("\n") has to be converted to underlying system's end of line representation and while reading from a text file this end of line representation is converted to '\n'. All these conversions are done by the library functions.

The input and output operations in binary files take less time as compared to that of text files because in binary files no conversions have to take place. However the data written using binary format is not very portable since the size of data types and byte order may be different on different machines. In text format, these problems do not arise so it is considered more portable.

## 12.2 Buffer

Buffer is an area in memory where the data is temporarily stored before being written to the file. When we open a file, a buffer is automatically associated with its file pointer. Whatever data we send to the file is not immediately written to the file. First it is sent to the buffer and when the buffer is full, its contents are written to the file. When the file is closed, all the contents of buffer are automatically written to the file even if the buffer is not full. This is called flushing the buffer, we can also explicitly flush the buffer by a function `fflush()` described later.

The concept of buffer is used to increase efficiency. Had there been no buffer we would have to access the disk each time for writing even single byte of data. This would have taken lot of time because each time the disk is accessed, the read/write head has to be repositioned. When buffering is done, the data is collected in the buffer and data equal to the size of buffer is written to the file at a time, so the number of times disk is accessed decreases, which improves the efficiency.

There can be three types of buffering - fully buffered, line buffered and unbuffered. In fully buffered streams, the data is transferred only when the buffer is full. In line buffered streams the data is transferred either when the buffer is full or when a newline character is written to the buffer. In unbuffered streams the data is transferred as quickly as possible. When a file is opened, a fully buffered stream is attached to it. The buffering can be changed after opening the file and before performing any operation on it using the functions `setbuf()` or `setvbuf()`.

The steps for file operations in C programming are-

1. Open a file
2. Read the file or write data in the file
3. Close the file

The functions `fopen()` and `fclose()` are used for opening and closing the files respectively.

## 12.3 Opening a File

A file must be opened before any I/O operations can be performed on that file. The process of establishing a connection between the program and file is called opening the file. When a file is opened, a stream is associated with that file, a buffer is created in the main memory for transferring data to and from the file.

A structure named `FILE` is defined in the header file `stdio.h` that contains all information about the file like name, status(read, write etc), buffer location, current position in the buffer, end of file status, error status etc. All these details are hidden from the programmer and the operating system takes care of all these things.

A file pointer (stream pointer) is a pointer to a structure of type `FILE`. Whenever a file is opened, a structure of type `FILE` is associated with it, and a file pointer that points to this structure identifies this file. The function `fopen()` is used to open a file.

Declaration : `FILE *fopen(const char *filename, const char *mode);`

`fopen()` function takes two strings as arguments, the first one is the name of the file to be opened and the second one is the mode that decides which operations (read, write, append etc) are to be performed on the file. On success, `fopen()` returns a pointer of type `FILE` and on error it returns `NULL`. The return value of `fopen()` is assigned to a `FILE` pointer declared previously. For example-

```
FILE *fp1, *fp2;
fp1=fopen("myfile.txt", "w");
fp2=fopen("yourfile", "r");
```

The name of a file is limited to `FILENAME_MAX` characters. After opening the file with `fopen()`, the name of file is not used in the program for any operation on it. Whenever we have to perform any operation on the file, we will use the file pointer returned by `fopen()` function. So the name of file is sometimes known as external name, while the file pointer associated with it is known as its internal name. The second argument represents the mode in which the file is to be opened. The possible values of mode are-

#### 1. "w" (write)

If the file doesn't exist then this mode creates a new file for writing, and if the file already exists then the previous data is erased and the new data entered is written to the file.

#### 2. "a" (append)

If the file doesn't exist then this mode creates a new file, and if the file already exists then the new data entered is appended at the end of existing data. In this mode, the data existing in the file is not erased as in "w" mode.

#### 3. "r" (read)

This mode is used for opening an existing file for reading purpose only. The file to be opened must exist and the previous data of file is not erased.

#### 4. "w+" (write+read)

This mode is same as "w" mode but in this mode we can also read and modify the data. If the file doesn't exist then a new file is created and if the file exists then previous data is erased.

#### 5. "r+" (read+write)

This mode is same as "r" mode but in this mode we can also write and modify existing data. The file to be opened must exist and the previous data of file is not erased. Since we can add new data and modify existing data, this mode is also called update mode.

#### 6. "a+" (append+read)

This mode is same as the "a" mode but in this mode we can also read the data stored in the file. If the file doesn't exist, a new file is created and if the file already exists then new data is appended at the end of existing data. We cannot modify existing data in this mode.

To open a file in binary mode we can append 'b' to the mode, and to open the file in text mode 't' can be appended to the mode. Since text mode is the default mode, 't' is generally omitted while opening files in text mode. For example-

|                |                                   |
|----------------|-----------------------------------|
| "wb"           | Binary file opened in write mode  |
| "ab+" or "a+b" | Binary file opened in append mode |
| "rt+" or "r+t" | Text file opened in update mode   |
| "w"            | Text file opened in write mode    |

## 12.4 Errors in Opening Files

If an error occurs in opening a file, then `fopen()` returns `NULL`. So we can check for any errors in opening by checking the return value of `fopen()`.

```
FILE *fp;
fp=fopen("text.dat", "w");
if (fp==NULL)
{
 printf("Error in opening file");
 exit(1);
}
```

Errors in opening a file may occur due to various reasons, for example-

1. If we try to open a file in read or update mode, and the file doesn't exist or we do not have read permission on that file.
2. If we try to create a file but there is no space on the disk or we don't have write permission.
3. If we try to create a file that already exists.
4. Operating system limits the number of files that can be opened at a time and we are trying to open more files than that number.

We can give full pathname to open a file. Suppose we want to open a file whose path is "E:\book\bookdir\names.dat", then we'll have to write as-

```
fp=fopen("E:\\book\\bookdir\\names.dat", "r");
```

Here we have used double backslash because single backslash inside string is considered as an escape character, 'b' and 'n' will be regarded as escape sequences if we use single backslash. In Unix, a single forward slash can be used.

Never give the mode in single quotes, as it is a string and not a character constant.

```
fp=fopen("file.dat", 'w'); /*Error*/
```

## 12.5 Closing a File

The file that was opened using `fopen()` function must be closed when no more operations are to be performed on it.

Declaration : `int fclose(FILE *fptr);`

After closing the file, connection between file pointer and the file is broken. Now the file pointer `fptr` is free to be connected to some other file. On closing the file, the buffer associated with it is flushed i.e. all the data that is in the buffer is written to the file. The buffers allocated by the system for the file are freed after the file is closed, so that these buffers can be available for other files. Some systems have a limitation on the number of files that can be opened at a time, so we should close the files that are not currently in use so that we can open other files.

Although all the files are closed automatically when the program terminates normally, sometimes it may be necessary to close the file by `fclose()` e.g. when we have to reopen the file in some other mode or when we exceed the number of opened files permitted by the system. Moreover it is a good practice to close files explicitly by `fclose()` when no more operations are to be performed on the file because it becomes clear to the reader of the program that the file has no use now.

`fclose()` returns `EOF` on error and `0` on success ( `EOF` is a constant defined in `stdio.h` and its value is generally `-1`). An error in `fclose()` may occur when there is not sufficient space on the disk or when the disk has been taken out of the drive.

If more than one files are opened, then we can close all the files by calling `fclose()` for each file.

```
fclose(fp1);
fclose(fp2);
```

We can also close multiple files by calling a single function `fcloseall()`. It closes all the opened files.

Declaration: `int fcloseall(void);`

On error, `fcloseall()` returns `EOF` and on success it returns the number of files closed. We can use it as-

```

fcloseall();
if (n==30F)
 printf("Could not close all open files\n");
else
 printf("%d files successfully closed\n",n);

```

## 12.6 End of File

The file reading functions need to know the end of file so that they can stop reading. When the end of file is reached, the operating system sends an end-of-file signal to the program. When the program receives this signal, the file reading functions return EOF, which is a constant defined in the file `stdio.h` and its value is -1. EOF is an integer value, so make sure that the return value of the function is assigned to an integer variable. Note that the value EOF is not present at the end of the file, it is returned by the file reading functions when end of file is reached.

## 12.7 Structure of a File Program

```

int main(void)
{
 FILE *fp;
 fp=fopen("filename", "mode");

 fclose(fp);
 return 0;
}

```

## 12.8 Standard streams

When the program starts executing, the operating system opens three standard text streams automatically and provides constant file pointers for them.

| File pointer | Stream          | Buffering     |
|--------------|-----------------|---------------|
| stdin        | Standard input  | Line-buffered |
| stdout       | Standard output | Line-buffered |
| stderr       | Standard error  | Unbuffered    |

The standard input is generally associated with the keyboard while the standard output and standard error streams are associated with the screen. The associations may be changed by calling the `freopen()` function.

The functions used for file I/O are-

|               |                                                                                           |
|---------------|-------------------------------------------------------------------------------------------|
| Character I/O | - <code>fgetc()</code> , <code>fputc()</code> , <code>getc()</code> , <code>putc()</code> |
| String I/O    | - <code>fgets()</code> , <code>fputs()</code>                                             |
| Formatted I/O | - <code>fscanf()</code> , <code>fprintf()</code>                                          |
| Record I/O    | - <code>fread()</code> , <code>fwrite()</code>                                            |

## 12.9 Character I/O

### 12.9.1 fputc ()

Declaration: `int fputc(int c, FILE *fptr);`

This function writes a character to the specified file at the current file position and then increments the file position indicator. On success it returns an integer representing the character written, and on error it returns EOF.

```

/*P12.1 Program to understand the use of fputc() function*/
#include<stdio.h>
#include<stdlib.h>

```

```

int main(void)
{
 FILE *fptr;
 int ch;
 if ((fptr=fopen("myfile", "w"))==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 printf("Enter text : \n");
 /*Press Ctrl+z/Ctrl+d to stop reading characters*/
 while ((ch=getchar())!=EOF)
 fputc(ch, fptr);
 fclose(fptr);
 return 0;
}

```

Output:

```

Enter text:
The woods are lovely, dark and deep
But I have miles to go before I sleep.
^Z

```

After the execution of this program, this text along with the ^Z character will be written to the file `myfile`.

### 12.9.2 fgetc()

Declaration : `int fgetc(FILE *fptr);`

This function reads a single character from a given file and increments the file position indicator. On success it returns the character after converting it to an `int` without sign extension. On end of file or error it returns EOF. In the next program we will read the file `myfile` that we have created in the previous program.

```

/*P12.2 Program to understand the use of fgetc()*/
#include<stdio.h>
#include<stdlib.h>
int main(void)

```

```

{
 FILE *p;
 int ch;
 if ((p=fopen("myfile", "r"))==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 while ((ch=fgetc(p))!=EOF)
 printf("%c", ch);
 fclose(p);
 return 0;
}

```

Output:

```

The woods are lovely, dark and deep
But I have miles to go before I sleep.

```

The while loop that we have written in the program is equivalent to this code-

```

while (fgetc(p);
 ch!=EOF)
 printf("%c", ch);
 ch=fgetc(p);

```

The value returned by `fputc()` and `fgetc()` is not of type `char` but is of type `int`. This is because these functions return an integer value EOF(-1) on end of file or error. The variable `ch` that is used to store the character read from the file, is also declared to be of `int` type for this reason only.

The next program copies a file to another file.

```
/*P12.3 Copy a file to another file*/
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 FILE *sptr,*dptr;
 int ch;
 if((sptr=fopen("source.txt","r"))==NULL)
 {
 printf("Error in opening source file\n");
 exit(1);
 }
 if((dptr=fopen("destination.txt","w"))==NULL)
 {
 printf("Error in opening destination file\n");
 exit(1);
 }
 while((ch=fgetc(sptr))!=EOF)
 fputc(ch,dptr);
 fclose(sptr);
 fclose(dptr);
 return 0;
}
```

### 12.9.3 getc() and putc()

The operations of `getc()` and `putc()` are similar to that of `fgetc()` and `fputc()`, the only difference is that the former two are defined as macros while the latter two are functions.

## 12.10 String I/O

### 12.10.1 fputs()

Declaration : `int fputs(const char *str, FILE *fptr);`

This function writes the null terminated string pointed to by `str` to a file associated with file pointer `fptr`. The null character that marks the end of string is not written to the file. On success it returns a nonnegative value and on error it returns EOF.

```
/*P12.4 Program to understand the use of fputs()*/
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 FILE *fptr;
 char str[80];
 if((fptr=fopen("test","w"))==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 printf("Enter the text\n");
 printf("To stop entering, press Ctrl+d/Ctrl+z\n");
 while(gets(str)!=NULL)
 fputs(str,fptr);
 fclose(fptr);
 return 0;
}
```

Suppose we enter this text after running the program-  
Yesterday is history  
Tomorrow is mystery

Today is a gift  
That's why it is called Present  
Z

When the first line of text is entered and Enter key is pressed, the function `gets()` converts the newline character to the null character and the array `str` contains "Yesterday is history" (20 characters +1 null character). Now `str` is written to the file `test` using `fputs()`. The null character is not written to the file, so only 20 characters are written.

In previous chapters we had studied about the function `puts()` that prints the string on the screen. The difference between these two functions is that `puts()` translates null character to a newline, but `fputs()` does not. `fputs()` will write a newline character to the file only if it is contained in the string. So after the first line of text, newline character is not entered in the file. Immediately after the first line of text, the second one is written.

In the following program, a newline will also be written at the end of each line in the file.

```
/*P12.5 Program to understand the use of fputs()*/
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 FILE *fptr;
 char str[80];
 if((fptr=fopen("test1","w"))==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 printf("Enter the text\n");
 printf("To stop entering, press Ctrl+d/Ctrl+z\n");
 while(gets(str)!=NULL)
 {
 strcat(str,"\n");
 fputs(str,fptr);
 }
 fclose(fptr);
 return 0;
}
```

### 12.10.2 fgets()

Declaration : `char *fgets(char *str, int n, FILE *fptr);`

This function is used to read characters from a file and these characters are stored in the string pointed to by `str`. It reads at most `n-1` characters from the file where `n` is the second argument. `fptr` is a file pointer which is associated with the file from which characters are read. This function returns the string pointed to by `str` on success, and on error or end of file it returns NULL.

This function reads characters from the file until either a newline or an end of file is encountered or till `n-1` characters have been read. It appends a null character ('\0') after the last character read to terminate the string. The following program reads characters from the file `test` that we have created in P12.4.

```
/*P12.6 Program to understand the use of fgets()*/
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 FILE *fptr;
 char str[80];
 if((fptr=fopen("test","r"))==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
}
```

```
while (fgets(str, 80, fptr) != NULL)
 puts(str);
fclose(fptr);
return 0;
```

Output:  
'esterday is historyTomorrow is mysteryToday is a giftThat's why it is called P  
esent.

Suppose we call the `fgets()` function with second argument as 20, then the output would be:

'esterday is histor  
Tomorrow is Myster  
Today is a giftTha  
s why it is calle  
Present

When `fgets()` was called with second argument as 80, it read 79 characters from the file and then stored them in array `str` and appended a null character. Now `str` was passed to `puts()`, so the string `str` was displayed on the screen. The function `puts()` converts the null character of string to a newline, so after displaying the string a newline is displayed. In the second iteration of the loop, the function `fgets()` encounters end of file after reading 6 characters only so it stops reading, stores these characters in `str`, appends null character and displays `str` on the screen using `puts()`. When `fgets()` was called with second argument as 20, it read 19 characters from the file.

`gets()` reads characters from the standard input while `fgets()` reads from a file. The difference between `gets()` and `fgets()` is that `fgets()` does not replace the newline character read by the null character, while `gets()` does. If `fgets()` reads a newline, both newline and null character will be present in the final string.

In `gets()` it may be possible that input is more than the size of array, since C does not check for array bounds, so an overflow may occur but in `fgets()` we can limit the size of input with the help of second argument.

## 12.11 Formatted I/O

We have studied about functions which can output and input characters and strings from files. The next two functions that we are going to study now, can input and output a combination of different data types in a formatted way. Formatting in files is generally used when there is a need to display data on terminal or print data in some format.

### 12.11.1 fprintf()

This function is same as the `printf()` function but it writes formatted data into the file instead of the standard output(screen). This function has same parameters as in `printf()` but it has one additional parameter which is a pointer of FILE type, that points to the file to which the output is to be written. It returns the number of characters output to the file on success, and EOF on error.

Declaration : `fprintf(FILE *fptr, const char *format[, argument, ...]);`

\*P12.7 Program to understand the use of `fprintf()`\*/

```
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 FILE *fp;
 char name[20];
 int age;
 if ((fp=fopen("rec", "w"))==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
}
```

```
 printf("Enter your name and age : ");
 scanf("%s%d", name, &age);
 fprintf(fp, "My name is %s and age is %d ", name, age);
 fclose(fp);
 return 0;
}
```

\*P12.8 Program to understand the use of `fprintf()`\*/

```
#include<stdio.h>
#include<stdlib.h>
struct student
{
 char name[20];
 float marks;
} stu;
int main(void)
{
 FILE *fp;
 int i, n;
 if ((fp=fopen("students", "w"))==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 printf("Enter number of records : ");
 scanf("%d", &n);
 for(i=1; i<=n; i++)
 {
 printf("Enter name and marks : ");
 scanf("%s%f", stu.name, &stu.marks);
 fprintf(fp, "%s %f", stu.name, stu.marks);
 }
 return 0;
}
```

### 12.11.2 fscanf()

Declaration : `fscanf(FILE *fptr, const char *format[, address, ...]);`

This function is similar to the `scanf()` function but it reads data from file instead of standard input, so it has one more parameter which is a pointer of FILE type which points to the file from which data will be read. It returns the number of arguments that were assigned some values on success, and EOF at the end of file.

In the next program we'll read the file `students` that we have created using `fprintf()` in P12.8. Note that the format string used in `fscanf()` should be same as the format string used in `fprintf()` while writing to the file.

\*P12.9 Program to understand the use of `fscanf()`\*/

```
#include<stdio.h>
#include<stdlib.h>
struct student
{
 char name[20];
 float marks;
} stu;
int main(void)
{
 FILE *fp;
 if ((fp=fopen("students", "r"))==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 printf("NAME\tMARKS\n");
}
```



```
while(fscanf(fp,"%s%f",stu.name,&stu.marks)!=EOF)
 printf("%s\t%f\n",stu.name,stu.marks);
fclose(fp);
return 0;
}
```

We had mentioned earlier that the file pointers `stdout` and `stdin` are automatically opened. If we use these file pointers in the functions `fprintf()` and `fscanf()`, then these function calls become equivalent to `printf()` and `scanf()`.

If we replace the file pointer `fp` by `stdout` then-

`fprintf(stdout,"My age is %s",age);` is equivalent to `printf("My age is %s",age);`

If we replace the file pointer `fp` by `stdin` then-

`fscanf(stdin,"%s%d",name,&age);` is equivalent to `scanf("%s%d",name,&age);`

## 12.12 Block Read/Write

The functions `fread()` and `fwrite()` are used to read and write blocks of data into files. Each block has some fixed size, it may be a structure or an array. Although we can read or write any type of data varying from a single character to arrays and structures through these functions, these are mainly used to read and write structures and arrays. For using these functions, the file is generally opened in binary mode (e.g. "wb", "rb").

### 12.12.1 fwrite()

This function is used for writing an entire block to a given file.

Declaration : `size_t fwrite(const void *ptr, size_t size, size_t n, FILE *fptr);`

`ptr` is a pointer which points to the block of memory that contains the information to be written to the file, `size` denotes the length of an item in bytes, `n` is the number of items to be written to the file, and `fptr` is a FILE pointer which points to the file to which the data is written.

If successful, `fwrite()` will write `n` items i.e. `n*size` bytes to the file and will return `n`. On error or end of file it will return a number less than `n`.

To write a single float value contained in variable `fval` to the file

```
fwrite(&fval,sizeof(float),1,fp);
```

To write an array of integers `arr[10]` to the file.

```
fwrite(arr,sizeof(arr),1,fp);
```

To write only first 5 elements from the above array to the file

```
fwrite(arr,sizeof(int),5,fp);
```

Note that here in second argument we will send size of integer, because here the items that we are writing are integers not array.

To write a structure variable which is defined as-

```
struct record
{
 char name[20];
 int roll;
 float marks;
} student;
fwrite(&student,sizeof(student),1,fp);
```

This will write a single structure variable to the file.

To write an array of structures

Suppose `stu_arr[200]` is an array of structure defined above-

```
fwrite(stu_arr,sizeof(stu_arr),1,fp);
```

Here it will write all the 200 elements of array `stu_arr` to the file.

write only a part of an array of structure

Suppose in the above array `stu[200]` we have entered records of only 100 students then it is no use writing whole array to the file (garbage value will be written). We can write 100 records as-

```
fwrite(stu_arr,sizeof(struct record),100,fp);
```

12.10 Program to understand the use of `fwrite()` \*/

```
#include<stdio.h>
#include<stdlib.h>
struct record
{
 char name[20];
 int roll;
 int marks;
} student;
main(void)
{
 int i,n;
 FILE *fp;
 if((fp=fopen("stu","wb"))==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 printf("Enter number of records : ");
 scanf("%d",&n);

 for(i=0; i<n; i++)
 {
 printf("Enter name : ");
 scanf("%s",student.name);
 printf("Enter roll no : ");
 scanf("%d",&student.roll);
 printf("Enter marks : ");
 scanf("%d",&student.marks);
 fwrite(&student,sizeof(student),1,fp);
 }
 fclose(fp);
 return 0;
}
```

We have used `sizeof` operator instead of sending the size directly, so that our program becomes portable because the size of data types may vary on different computers. Moreover if new elements are added to our structure, we need not recalculate and change the size in our program.

### 12.12.2 fread()

This function is used to read an entire block from a given file.

Declaration : `size_t fread(void *ptr, size_t size, size_t n, FILE *fptr);`

Here `ptr` is a pointer that points to the block of memory which receives the data read from the file, `size` is the length of each item in bytes, `n` is the number of items to be read from the file and `fptr` is a file pointer which points to the file from which data is read.

On success it reads `n` items from the file and returns the value of `n`, if error or end of file occurs then it returns a value less than `n`. We can use `feof()` and `ferror()` to check these conditions.

To read a single float value from the file and store it in a variable `fval`.

```
fread(&fval,sizeof(float),1,fp);
```

To read array of integers from file and store them in array `arr`.

```
fread(arr,sizeof(arr),1,fp);
```

To read 5 integers from the file and store them in first five elements of array `arr`

```
fread(arr,sizeof(int),5,fp);
```

To read a structure variable that is defined as-

```
struct record
```

```
{
 char name[20];
 int roll;
 float marks;
}student;

fread(&student, sizeof(student), 1, fp);
```

This will read a single structure variable from the file and store it in variable student.

To read an array of structures

Suppose stu\_arr[200] is an array of structure.

```
fread(stu_arr, sizeof(stu_arr), 1, fp);
```

Here it will read an array of structures from file and store it in stu\_arr.

To read 100 records from file and store them in first 100 structures of stu\_arr[200].

```
fread(stu_arr, sizeof(struct record), 100, fp);
```

The following program reads the file stu created in P12.10.

*/\*P12.11 Program to understand the use of fread()\*/*

```
#include<stdio.h>
#include<stdlib.h>
struct record
{
 char name[20];
 int roll;
 int marks;
}student;
int main(void)
{
 FILE *fp;
 fp=fopen("stu", "rb");
 if(fp==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 printf("\nNAME\tROLLNO\tMARKS\n");
 while(fread(&student, sizeof(student), 1, fp)==1)
 {
 printf("%s\t", student.name);
 printf("%d\t", student.roll);
 printf("%d\n", student.marks);
 }
 fclose(fp);
 return 0;
}
```

The fread() function returns the number of records successfully read, so it will return 1 till there are records in the file and will return a number less than 1 when there will be no records in the file. So we have used this condition in our while loop to stop reading when end of file is reached.

## 12.13 Random Access to File

We can access the data stored in the file in two ways, sequentially or randomly. So far we have used only sequential access in our programs. For example if we want to access the forty-fourth record then first forty-three records should be read sequentially to reach the forty-fourth record. In random access, data can be accessed and processed randomly i.e. in this case the forty-fourth record can be accessed directly. There is no need to read each record sequentially for accessing a particular record. Random access takes less time than the sequential access.

supports these functions for random access file processing-

```
fseek()
ftell()
rewind()
fsetpos()
fgetpos()
```

Before studying these functions it is necessary to understand the concept of file position indicator. File position indicator points to a particular byte in the file and all read and write operations on the file take place at this byte. This indicator automatically moves forward when a read or write operation takes place. To access the data randomly, we will have to take control of this position indicator.

### 12.13.1 fseek()

This function is used for setting the file position indicator at the specified byte. Any read or write operation on the file will be done at the new position.

Declaration : int fseek(FILE \*fp, long displacement, int origin);

Here fp is a file pointer, displacement is a long integer which can be positive or negative and it denotes the number of bytes which are skipped backward (if negative) or forward (if positive) from the position specified in the third argument. This argument is declared as long integer so that it is possible to move in large files.

The third argument named origin is the position relative to which the displacement takes place. It can take one of these three values.

| Constant     | Value | Position          |
|--------------|-------|-------------------|
| SEEK_SET     | 0     | Beginning of file |
| SEEK_CURRENT | 1     | Current position  |
| SEEK_END     | 2     | End of file       |

These three constants are defined in stdio.h, so we can either use these names or their values. Some examples of usage of fseek() function are-

```
fseek(p, 10L, 0);
```

Origin is 0, which means that displacement will be relative to beginning of file so position indicator is moved 10 bytes forward from the beginning of the file. Since the second argument is a long integer, L is attached to it.

```
fseek(p, 5L, 1);
```

Position indicator is moved 5 bytes forward from the current position.

```
fseek(p, 8L, SEEK_SET);
```

Position indicator is moved 8 bytes forward from the beginning of file.

```
fseek(p, -5L, 1);
```

Position indicator is moved 5 bytes backward from the current position.

```
fseek(p, 0L, 2);
```

Origin is 2 which represents end of file and displacement is 0 which means that 0 bytes are moved from end of file. So after this statement position indicator will point to the end of file.

```
fseek(p, -6L, SEEK_END);
```

Position indicator is moved 6 bytes backward from the end of file.

```
fseek(p, 0L, 0);
```

This means 0 bytes are moved from the beginning of file. After this statement position indicator points to the beginning of file.

On success fseek() returns the value 0, and on failure it returns a non-zero value. Consider the file stu that we had created earlier in program P12.10. In the next program we'll be able to read a particular number of record randomly from the file with the help of fseek() function.

*12.12 Program to understand the use of fseek()\*/*  
#include<stdio.h>

```
#include<stdlib.h>
struct record
{
 char name[20];
 int roll;
 int marks;
}student;
int main(void)
{
 int n;
 FILE *fp;
 fp=fopen("stu", "rb");
 if(fp==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 printf("Enter the record number to be read : ");
 scanf("%d", &n);
 fseek(fp, (n-1)*sizeof(student), 0); /*skip n-1 records*/
 fread(&student, sizeof(student), 1, fp); /*Read the nth record*/
 printf("%s\t", student.name);
 printf("%d\t", student.roll);
 printf("%d\t", student.marks);
 fclose(fp);
 return 0;
}
```

Suppose we want to read the 5<sup>th</sup> record from the file, i.e.  $n=5$ . We'll skip first 4 records and place our position indicator at the beginning of the 5<sup>th</sup> record, so now `fread()` will read the 5<sup>th</sup> record. In general if we want to read the  $n^{\text{th}}$  record we'll skip  $n-1$  records with `fseek()` and place the position indicator at the beginning of the  $n^{\text{th}}$  record.

### 12.13.2 ftell()

Declaration : `long ftell(FILE *fp);`

This function returns the current position of the file position indicator. The value is counted from the beginning of the file.

*/\*P12.13 Program to understand the use of ftell()\*/*

```
#include<stdio.h>
#include<stdlib.h>
struct record
{
 char name[20];
 int roll;
 int marks;
}student;
int main(void)
{
 FILE *fp;
 fp=fopen("stu", "rb");
 if(fp==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 printf("Position indicator in the beginning -> %ld\n", ftell(fp));
 while(fread(&student, sizeof(student), 1, fp)==1)
 {
 printf("Position indicator -> %ld\n", ftell(fp));
 printf("%s\t", student.name);
 printf("%d\t", student.roll);
 }
}
```

```
printf("%d\n", student.marks);
}
printf("%d\n", ftell(fp));
fclose(fp);
```

On error `ftell()` returns `-1L` and sets `errno` to a positive value.

The functions `fsetpos()` and `fgetpos()` are like `fseek()` and `ftell()` but they can be used for huge files, while `fseek()` and `ftell()` can use offsets limited to the range of `long int`.

### 12.13.3 rewind()

Declaration : `void rewind(FILE *fp);`

This function is used to move the file position indicator to the beginning of the file. The function `rewind()` is useful when we open a file for update.

Using `rewind(fp)` is equivalent to  
`fseek(fp, 0L, 0);`  
`clearerr(fp);`

*/\*P12.14 Program to understand the use of rewind()\*/*

```
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 FILE *fp;
 fp=fopen("stu", "rb+");
 if(fp==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 printf("Position indicator -> %ld\n", ftell(fp));
 fseek(fp, 0, 2);
 printf("Position indicator -> %ld\n", ftell(fp));
 rewind(fp);
 printf("Position indicator -> %ld\n", ftell(fp));
 fclose(fp);
 return 0;
}
```

Now we will write some programs to perform various operations on the file `stu`.

*/\*P12.15 Program to append records to a file\*/*

```
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 struct record
 {
 char name[20];
 int roll;
 int marks;
 }student;
 FILE *fp;
 int choice=1;
 fp=fopen("stu", "ab"); /*opened in append mode*/
 if(fp==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 while(choice==1)
```

```

printf("Enter name : ");
scanf("%s", student.name);
printf("Enter roll no : ");
scanf("%d", &student.roll);
printf("Enter marks : ");
scanf("%d", &student.marks);
fwrite(&student, sizeof(student), 1, fp);
printf("Want to enter more ? (1 for yes / 0 for no) : ");
scanf("%d", &choice);
}
fclose(fp);
return 0;

```

**12.16 Program to read records from a file and calculate grade of each student display it**

= A if marks >= 80  
 = B if marks >= 60 and < 80  
 = C if marks < 60

```

#include<stdio.h>
#include<stdlib.h>
main(void)
{
 struct record
 {
 char name[20];
 int roll;
 int marks;
 } student;
 FILE *fp;
 fp=fopen("stu", "rb"); /*opened in read mode*/
 if(fp==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 printf("\nNAME\t\tMARKS\t\tGRADE\n\n");
 while(fread(&student, sizeof(student), 1, fp)==1)
 {
 printf("%s\t\t", student.name);
 printf("%d\t\t", student.marks);
 if(student.marks >= 80)
 printf("A\n");
 else if(student.marks >= 60)
 printf("B\n");
 else
 printf("C\n");
 }
 fclose(fp);
 return 0;
}

```

**12.17 Program to modify records in a file**

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
main(void)
{
 struct record
 {
 char name[20];
 int roll;
 int marks;
 } student;
 FILE *fp;
 char name[20];
 unsigned flag=0;
 fp=fopen("stu", "rb+");
 if(fp==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 printf("Enter name of student whose record is to be modified : ");
 scanf("%s", name);

 while(fread(&student, sizeof(student), 1, fp)==1)
 {
 if(strcmp(student.name, name)==0)
 {
 printf("Enter new data :>\n");
 printf("Enter name : ");
 scanf("%s", student.name);
 printf("Enter roll no : ");
 scanf("%d", &student.roll);
 printf("Enter marks : ");
 scanf("%d", &student.marks);
 fseek(fp, -size, 1);
 fwrite(&student, sizeof(student), 1, fp);
 flag=1;
 break;
 }
 }
 if(flag==0)
 printf("Name not found in file\n");
 else
 printf("Record Modified.....\n");
 fclose(fp);
 return 0;
}

```

For modifying records we have opened the file stu in update (rb+) mode. We will read all the records one by one with the help of `fread()`, and when the name of the student matches with the name we have entered for modification, we will enter new data for the student, and write that data to the file. Before `fwrite()` we have used `fseek()` to skip one record backwards because while reading we've reached the end of the record which is to be modified, i.e. position indicator is at the beginning of next record, so if we write new data without using `fseek()` then it will be written over the next record. After modifying the data we'll stop reading and come out of the while loop with the help of break statement.

**12.18 Program to delete a record from the file**

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
main(void)
{
 struct record
 {
 char name[20];
 int roll;
 int marks;
 } student;
 FILE *fp, *fptmp;
 char name[20];
 unsigned flag=0;
 fp=fopen("stu", "rb+");
 if(fp==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 printf("Enter name of student whose record is to be modified : ");
 scanf("%s", name);

 while(fread(&student, sizeof(student), 1, fp)==1)
 {
 if(strcmp(student.name, name)==0)
 {
 printf("Enter new data :>\n");
 printf("Enter name : ");
 scanf("%s", student.name);
 printf("Enter roll no : ");
 scanf("%d", &student.roll);
 printf("Enter marks : ");
 scanf("%d", &student.marks);
 fseek(fp, -size, 1);
 fwrite(&student, sizeof(student), 1, fp);
 flag=1;
 break;
 }
 }
 if(flag==0)
 printf("Name not found in file\n");
 else
 printf("Record Modified.....\n");
 fclose(fp);
 return 0;
}

```

```

 exit(1);
}
printf("Enter the name to be deleted : ");
scanf("%s", name);
fptmp=fopen("tempfile", "wb");
while(fread(&student, sizeof(student), 1, fp)==1)
{
 if(strcmp(name, student.name)!=0)
 fwrite(&student, sizeof(student), 1, fptmp);
 else
 flag=1;
}
fclose(fp);
fclose(fptmp);
remove("stu");
rename("tempfile", "stu");
if(flag==0)
 printf("Name not found in file\n");
else
 printf("Record deleted.....\n");
return 0;
}

```

For deleting a record from stu we'll make a temporary file tempfile and copy all the records to that file except the record to be deleted, and then we'll delete the original file stu and rename tempfile to stu. The macro remove() is used for deleting a file and function rename() is used to rename a file.

*/\*P12.19 Program to display the records in sorted order, sorting is performed in ascending order w.r.t. name\*/*

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
int main(void)
{
 struct record
 {
 char name[20];
 int roll;
 int marks;
 } student, temp, stu[50];
 FILE *fp;
 int i, j, k=0;
 fp=fopen("stu", "rb"); /*opened in read mode*/
 if(fp==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }
 while(fread(&student, sizeof(student), 1, fp)==1)
 stu[k++]=student;
 /*Bubble sort*/
 for(i=0; i<k; i++)
 {
 for(j=0; j<k-1-i; j++)
 {
 if(strcmp(stu[j].name, stu[j+1].name)>0)
 {
 temp=stu[j];
 stu[j]=stu[j+1];
 stu[j+1]=temp;
 }
 }
 }
 printf("\nNAME\t\tROLLNO\t\tMARKS\n\n");
 for(i=0; i<k; i++)

```

```

 {
 printf("%s\t\t", stu[i].name);
 printf("%d\t\t", stu[i].roll);
 printf("%d\n", stu[i].marks);
 }
 fclose(fp);
 return 0;
}

```

Here we have read all the records from the file into an array and then we have sorted that array by bubble sort. After sorting we have displayed the sorted array on the screen. Note that the records in the file stu are in their original unsorted form. If we want the records in the file to become sorted, then we should rewind the file and then write the sorted array to the file with the help of fwrite() like this-

```

rewind(fp);
write(stu, sizeof(student), k, fp);

```

In this case file stu should be opened in update mode. We may also write the sorted records to another file. Now we'll see how to sort on more than one key. Suppose name is primary key and marks is taken as secondary key, and sorting is to be done in ascending order w.r.t. name and in descending order w.r.t. marks.

```

for(i=0; i<k; i++)
{
 for(j=0; j<k-1-i; j++)
 {
 if(strcmp(stu[j].name, stu[j+1].name)>0)
 {
 temp=stu[j];
 stu[j]=stu[j+1];
 stu[j+1]=temp;
 }
 else if(strcmp(stu[j].name, stu[j+1].name)==0)
 if(stu[j].marks < stu[j+1].marks)
 {
 temp=stu[j];
 stu[j]=stu[j+1];
 stu[j+1]=temp;
 }
 }
}
/*End of inner for loop*/
/*End of outer for loop*/

```

Now we'll write a program to merge two files that are sorted on the same key, such that the merged file is also sorted on the same key. The logic is somewhat similar to the merging of arrays that we had done in chapter 7.

Create two files named sectionA and sectionB using program P12.10. The records in these two files should be in descending order w.r.t marks. So either you can enter the records in sorted order or you may sort the file after entering the records. The program given next merges these two sorted files into a third sorted file. The merged file is named merged and it contains all the records of sectionA and sectionB in descending order w.r.t marks. This file can be read using program P12.11.

*/\*P12.20 Program to merge two files\*/*

```

#include<stdio.h>
#include<stdlib.h>
struct record
{
 char name[20];
 int roll;
 int marks;
} stu1, stu2;
int main(void)
{
 FILE *fp1, *fp2, *fp3;
 int i, j;
 if((fp1=fopen("sectionA", "rb")) == NULL)

```

```

 printf("Error in opening file\n");
 exit(1);
}
if((fp2=fopen("sectionB", "rb"))==NULL)
{
 printf("Error in opening file\n");
 exit(1);
}
if((fp3=fopen("merged", "wb"))==NULL)
{
 printf("Error in opening file\n");
 exit(1);
}
i=fread(&stu1, sizeof(stu1), 1, fp1);
j=fread(&stu2, sizeof(stu2), 1, fp2);

while((i==1) && (j==1))
{
 if(stu1.marks > stu2.marks)
 {
 fwrite(&stu1, sizeof(stu1), 1, fp3);
 i=fread(&stu1, sizeof(stu1), 1, fp1);
 }
 else
 {
 fwrite(&stu2, sizeof(stu2), 1, fp3);
 j=fread(&stu2, sizeof(stu2), 1, fp2);
 }
}
while(i==1) /*Write remaining records of sectionA into merged*/
{
 fwrite(&stu1, sizeof(stu1), 1, fp3);
 i=fread(&stu1, sizeof(stu1), 1, fp1);
}
while(j==1) /*Write remaining records of sectionB into merged*/
{
 fwrite(&stu1, sizeof(stu1), 1, fp3);
 j=fread(&stu2, sizeof(stu2), 1, fp2);
}
fclose(fp1);
fclose(fp2);
fclose(fp3);
return 0;

```

next program is a menu driven program in which we manage a database of books. All the operations that we studied till now have been combined in this single program.

2.21 Write a program to manage a database of books\*/

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
insert(FILE *fp);
del(FILE *fp);
modify(FILE *fp);
booksold(FILE *fp);
search(FILE *fp, char *name);
display(FILE *fp);
list(FILE *fp);

```

```

char name[50];
int ncopies;
float cost;

```

```

book;
int main(void)
{
 int choice;
 FILE *fp;
 fp=fopen("books", "rb+");
 if(fp==NULL)
 {
 fp=fopen("books", "wb+");
 if(fp==NULL)
 {
 puts("Error in opening file\n");
 exit(1);
 }
 }
 while(1)
 {
 printf("1.Insert a new record\n");
 printf("2.Delete a record\n");
 printf("3.Display record of a book\n");
 printf("4.Modify an existing record\n");
 printf("5.List all records\n");
 printf("6 Book sold\n");
 printf("7.Exit\n");
 printf("Enter your choice : ");
 scanf("%d", &choice);

 switch(choice)
 {
 case 1:
 insert(fp);
 break;
 case 2:
 del(fp);
 break;
 case 3:
 display(fp);
 break;
 case 4:
 modify(fp);
 break;
 case 5:
 list(fp);
 break;
 case 6:
 booksold(fp);
 break;
 case 7:
 fclose(fp);
 exit(1);
 default:
 printf("Wrong choice\n");
 }
 } /*End of while */
 return 0;
} /*End of main() */

void insert(FILE *fp)
{
 fseek(fp, 0, 2);
 printf("Enter book name : ");
 scanf("%s", book.name);
 printf("Enter number of copies : ");
 scanf("%d", &book.ncopies);
 printf("Enter cost of book : ");

```

```

scanf("%f",&book.cost);
fwrite(&book,sizeof(book),1,fp);
/*End of insert()*/

id del(FILE *fp)
{
 FILE *fptmp;
 char name[20];
 printf("Enter the name of book to be deleted from database : ");
 scanf("%s",name);
 if(search(fp,name)==0)
 return;
 fptmp=fopen("tempfile","wb");
 rewind(fp);
 while(fread(&book, sizeof(book),1,fp) != 1)
 {
 if(strcmp(name,book.name)!=0)
 fwrite(&book,sizeof(book),1,fptmp);
 }
 fclose(fp);
 fclose(fptmp);
 remove("books");
 rename("tempfile","books");
 printf("\nRecord deleted.....\n\n");
 fp=fopen("books", "rb+");
/*End of del()*/
id modify(FILE *fp)
{
 char name[50];
 long size=sizeof(book);
 printf("Enter the name of the book to be modified : ");
 scanf("%s",name);
 if(search(fp,name) == 1)
 {
 printf("Enter new data-->\n\n");
 printf("Enter book name : ");
 scanf("%s",book.name);
 printf("Enter number of copies : ");
 scanf("%d",&book.ncopies);
 printf("Enter cost of book : ");
 scanf("%f",&book.cost);
 fseek(fp,-size,1);
 fwrite(&book,sizeof(book),1,fp);
 printf("\nRecord successfully modified\n\n");
 }
/*End of modify()*/
id booksold(FILE *fp)
{
 char name[50];
 long size=sizeof(book);
 printf("Enter the name of the book to be sold : ");
 scanf("%s", name);
 if(search(fp,name)==1)
 {
 if(book.ncopies > 0)
 {
 book.ncopies--;
 fseek(fp,-size,1);
 fwrite(&book, sizeof(book),1,fp);
 printf("One book sold\n");
 printf("Now number of copies=%d\n",book.ncopies);
 }
 else
 printf("Book is out of stock\n\n");
 }
}

```

```

}
/*End of booksold()*/

id display(FILE *fp)
{
 char name[50];
 printf("Enter the name of the book : ");
 scanf("%s",name);
 if(search(fp,name)==1)
 {
 printf("\nName\t%s\n",book.name);
 printf("Copies\t%d\n",book.ncopies);
 printf("Cost\t%f\n",book.cost);
 }
/*End of display()*/

search(FILE *fp,char *name)
{
 unsigned flag=0;
 rewind(fp);
 while(fread(&book, sizeof(book),1,fp)==1)
 {
 if(strcmp(name,book.name)==0)
 {
 flag=1;
 break;
 }
 }
 if(flag == 0)
 printf("\nName not found in file\n\n");
 return flag;
/*End of search()*/

id list(FILE *fp)
{
 rewind(fp);
 printf("\nNAME\tCOPIES\tCOST\n\n");
 while(fread(&book, sizeof(book),1,fp)==1)
 {
 printf("%s\t",book.name);
 printf("%d\t",book.ncopies);
 printf("%f\n",book.cost);
 }
 printf("\n");
/*End of list()*/
}

```

We have opened the file books in update(rb+) mode. If the file doesn't exist then fopen() will return NULL, so we'll create the file by opening in "wb+" mode. So first time when we'll run this program, file will be opened in "wb+" mode. The operations permitted in "wb+" mode and "rb+" mode are same but the problem with "rb+" mode is that it can't create a file and with "wb+" mode is that it erases the contents of the file each time file is opened.

The function insert() is simple, we place the position indicator at the end of file by fseek() and then insert a new record there.

The function search() searches for a record in the file, and if the record does not exist then it displays a message and returns 0. If the record exists then the position indicator is placed at the end of the record, which was searched and the function returns 1. Before starting our search we have called rewind() function so that search starts from the beginning of the file. This function search() is used in del(), modify(), booksold() and display() functions.

The del() function first calls search(), and if the record doesn't exist we return from the function. Otherwise the record is deleted using a temporary file as we have seen earlier. At the end of del() function the

file books is opened in "rb+" mode because this file was closed in this function, and it is the responsibility of this function to open the file in "rb+" before returning to main menu and performing any other operation.

The function booksold() searches for the book to be sold and it reduces the number of copies by 1. If the number of copies is 0 then it displays the message that book is out of stock. For reducing number of copies we have written book.ncopies--. We could write this because book is a global variable and after calling search() it contains the record of the book to be sold.

The function display() searches for a particular record and displays its details on the screen while the function list() displays the details of the all the books on the screen.

## 12.14 Other File Functions

### 12.14.1 feof()

So far we have used the return value of the function to detect the end of file while reading. For example fgetc(), getc(), getw(), fprintf() return EOF and fgets() returns NULL on end of file. But these functions return the same value both on end of file and on error. So if a function stops reading we can't make out whether the end of file was reached or an error was encountered. We can check separately for these two conditions with the help of ferror() and feof().

Declaration : int feof(FILE \*fptr);

The macro feof() is used to check the end of file condition. It returns a nonzero value if end of file has been reached otherwise it returns zero.

In the program P12.11 we could have used feof() function to read the file.

```
fread(&student, sizeof(student), 1, fp);
while(!feof(fp))
{
 printf("%s\t", student.name);
 printf("%d\t", student.roll);
 printf("%f\n", student.marks);
 fread(&student, sizeof(student), 1, fp);
}
```

Let us see what would happen if we write the above code like this-

```
while(!feof(fp))
{
 fread(&student, sizeof(student), 1, fp);
 printf("%s\t", student.name);
 printf("%d\t", student.roll);
 printf("%f\n", student.marks);
}
```

In this case, the last record of the program may be processed twice, since in C end of file is indicated only after an input function has made an attempt to read at the end of file.

### 12.14.2 ferror()

Declaration : int ferror(FILE \*fptr);

The macro ferror() is used for detecting any error occurred during read or write operations on a file. It returns a nonzero value if an error occurs i.e. if the error indicator for that file is set, otherwise it returns zero.

```
/*P12.22 Program to understand the use of ferror()*/
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 FILE *fptr;
 int ch;
 fptr=fopen("test", "w");
```

```
 ch=getc(fptr);
 if(ferror(fptr))
 {
 printf("Error in read operation\n");
 exit(1);
 }
 else
 printf("%c", ch);
 fclose(fptr);
 return 0;
```

Output:  
Error in read operation

Here the file is opened in write mode and an attempt is made to read from the file, this gives an error which is detected by ferror().

Generally the return value of the input function is checked to stop reading from the file. Almost all input functions return same value on end of file and on error, so to distinguish between these two cases, the functions feof() and ferror() are used.

P12.23 Program to understand the use of feof() and ferror()\*/

```
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 FILE *fptr;
 int ch;
 if((fptr=fopen("myfile", "r"))==NULL)
 {
 printf("File doesn't exist\n");
 exit(1);
 }
 while((ch=getc(fptr))!=EOF)
 printf("%c", ch);
 if(feof(fptr))
 printf("End of file\n");
 if(ferror(fptr))
 printf("Error\n");
 fclose(fptr);
 return 0;
```

### 12.14.3 clearerr()

Declaration : void clearerr(FILE \*fptr);

This function is used to set the end of file and error indicators to 0.

P12.24 Function to understand the use of clearerr()\*/

```
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 FILE *fptr;
 int ch;
 fptr=fopen("test", "w");
 ch=getc(fptr);
 if(ferror(fptr))
 {
 printf("Error in read operation\n");
 clearerr(fptr);
 }
 fclose(fptr);
```



```
return 0;
```

### 12.14.4 perror()

Declaration : `void perror(const char *str);`

This function is used to display a message supplied by the user along with the error message generated by the system. It prints the argument `str`, and a colon, then the message related to the current value of `errno`, and then a newline. `errno` is a global variable of type `int` declared in file `errno.h`, and it is set to a particular value whenever an error occurs. The use of this function can be understood by reading the next program P12.25.

### 12.14.5 rename()

Declaration : `int rename(const char *oldname, const char *newname);`

This function is used to rename a file. We can give full path also in the argument, but the drives should be the same although directories may differ. On success it returns 0 and on error it returns -1 and `errno` is set to one of these values-

|         |                           |
|---------|---------------------------|
| ENOENT  | No such file or directory |
| EACCES  | Permission denied         |
| ENOTSAM | Not same device           |

/\*P12.25 Program to understand the use of rename()\*/

```
#include<stdio.h>
int main(void)
{
 char old_name[80], new_name[80];
 printf("Enter the name of file to be renamed : ");
 gets(old_name);
 printf("Enter a new name for the file : ");
 gets(new_name);
 if(rename(old_name, new_name)==0)
 printf("File is renamed to %s\n", old_name, new_name);
 else
 perror("File not renamed");
 return 0;
}
```

Suppose we give a filename which does not exist then value of `errno` will be set to `ENOENT` and the message displayed by `perror()` will be-

File not renamed : No such file or directory

### 12.14.6 remove()

Declaration : `int remove(const char *filename);`  
`remove` is a macro that deletes a file.

### 12.14.7 fflush()

Declaration : `int fflush(FILE *fptr);`

This function writes any buffered unwritten output to the file associated with `fptr`. The file is not closed after call to `fflush()`. On success it returns 0 and on error it returns EOF. If we call this function with `stderr`, then all the unwritten buffered error messages will be written.

```
fflush(stderr);
```

To flush all output streams we can write-

```
fflush(NULL);
```

### 12.14.8 tmpfile()

Declaration : `FILE *tmpfile(void);`

`tmpfile()` creates a temporary binary file and opens it in "wb+" mode. On success it returns a pointer of type `FILE` that points to the temporary file and on error it returns `NULL`. The file created by `tmpfile()` is temporary because it is automatically deleted when closed or when the program terminates.

### 12.14.9 tmpnam()

Declaration : `char *tmpnam(char str[L_tmpnam]);`

This function is used to generate a unique name for a file. The argument `str` can be `NULL` or an array of minimum `L_tmpnam` characters. If `str` is `NULL` then `tmpnam()` stores the temporary file name in an internal static array and returns a pointer to that array. If `str` is not `NULL`, then `tmpnam()` stores the temporary file name in `str` and returns `str`. The number of different file names that can be generated by `tmpnam()` during the execution of program is specified by `TMP_MAX`, which is defined in `stdio.h` file. Note that `tmpnam()` only creates a file name, it does not create the file. A file with the name generated by `tmpnam()` can be opened using `fopen()`, and it should be closed using `fclose()`.

### 12.14.10 freopen()

Declaration : `FILE *freopen(const char *filename, const char *mode, FILE *fptr);`

This function is used to associate a new file with a file pointer. The file associated with `fptr` is closed as by `fclose()`, after that the filename that is specified in the first argument is opened in the specified mode as by `fopen()`, and this `fptr` is now associated with this file. On success `freopen()` returns `fptr`, and on error it returns `NULL`.

This function is generally used to change the files associated with file pointers `stdin`, `stdout` and `stderr`. For example the file pointer `stderr` is associated with the screen, if we want to store the error messages in a file instead of displaying them on the screen, then we can use `freopen()` as-

```
freopen("errorfile", "a", stderr);
```

## 12.15 Command Line Arguments

The function `main()` can accept two parameters. The definition of `main()` when it accepts parameters is-

```
int main(int argc, char *argv[])
```

The first parameter is of type `int` and the second one is an array of pointers to strings. These parameters are conventionally named `argc` (argument counter) and `argv` (argument vector), and are used to access the arguments supplied at the command prompt. Let us see how this is done.

The program is compiled and an executable file is created. In DOS, name of the executable file is same as the name of the program and it has a `.exe` extension. In UNIX, the executable file is named as `a.out`. We can give any other name to the executable file in UNIX using `-o` option as-

```
cc -o myprog myprog.c
```

Now the program is executed at the command prompt, and arguments are supplied to it. The first argument is always the name of executable file. The parameters `argc` and `argv` can be used to access the command line

arguments. The parameter `argc` represents the number of arguments on command line. All the arguments supplied at the command line are stored internally as strings and their addresses are stored in the array of pointers named `argv`.

```
/*P12.26 Program to understand command line arguments*/
#include<stdio.h>
int main(int argc, char *argv[])
{
 int i;
 printf("argc=%d\n",argc);
 for(i=0; i<argc; i++)
 printf("argv[%d]=%s\n",i,argv[i]);
 return 0;
}
```

Suppose the name of the program is `myprog.c` and it is executed on the command prompt as `myprog you r 2 good`

The variable `argc` will have value 5, since total five arguments are supplied at the command prompt. The first argument is the name of the program, so `argv[0]` points to the base address of string "myprog" (it might contain the whole path of the file). Similarly `argv[1]` points to the base address of string "you", `argv[2]` points to the base address of string "r", `argv[3]` points to the base address of string "2", `argv[4]` points to the base address of string "good".

The output will be-

```
argc=5
gv[0]=myprog
gv[1]=you
gv[2]=r
gv[3]=2
gv[4]=good
```

Note that each argument is stored as a string, so `argv[3]` does not represent the integer 2 but it is a pointer to string "2". If we want to use it as an integer in the program then we will have to convert it to an integer using function `atoi()`.

## Some Additional Problems

### Problem 1

Write a program that copies contents of a file to another file. The names of two files should be sent as command line arguments.

```
/*P12.27 Program to copy a file to another*/
#include<stdio.h>
#include<stdlib.h>
int main(int argc, char *argv[])
{
 FILE *source,*dest;
 int c;
 if(argc!=3)
 {
 printf("Wrong number of arguments\n");
 exit(1);
 }
 if((source=fopen(argv[1],"r"))==NULL)
 {
 printf("Can't open source file\n");
 exit(1);
 }
 if((dest=fopen(argv[2],"w"))==NULL)
```

```
{
 printf("Can't open destination file\n");
 exit(1);
}
while((c=fgetc(source))!=EOF)
 fputc(c,dest);
fclose(source);
fclose(dest);
return 0;
}
```

In this program, first we've checked the number of command line arguments using `argc`, it is a good practice to do so.

### Problem 2

Write a program to remove all comment lines from a syntax error free C source program.

```
/*P12.28*/
#include<stdio.h>
#include<stdlib.h>
int main(void)
{
 FILE *fp1,*fp2;
 char name[50];
 int c1,c2,found='n';
 printf("Enter the file name : ");
 scanf("%s",name);

 if((fp1=fopen(name,"r"))==NULL)
 {
 printf("Error in opening file\n");
 exit(1);
 }

 fp2=fopen("c:\\new.c","w");
 c1=fgetc(fp1);
 c2=fgetc(fp1);

 while(c2!=EOF)
 {
 if(c1=='/' && c2=='*')
 found='y';
 if(found=='n')
 fputc(c1,fp2);
 if(c1=='*' && c2=='/')
 {
 found='n';
 c2=fgetc(fp1);
 }
 c1=c2;
 c2=fgetc(fp1);
 }
 fclose(fp1);
 fclose(fp2);
 return 0;
}
```

### Problem 3

Write a program to count the number of words in a text file. Assume that a word ends with a space, tab, newline, comma, fullstop, semicolon, colon, hyphen.

```

/*P12.29 Program to count the number of words*/
#include<stdio.h>
#include<stdlib.h>
int is_end(int ch);
int main(void)
{
 char line[81];
 int i, count=0;
 FILE *fptr;
 if((fptr=fopen("test.txt", "r")) == NULL)
 {
 printf("File doesn't exist\n");
 exit(1);
 }
 while((fgets(line, 81, fptr)) != NULL)
 {
 for(i=0; line[i]!='\0'; i++)
 if(is_end(line[i]))
 count++;
 }
 printf("Number of words in the file=%d\n", count);
 fclose(fptr);
 return 0;
}

int is_end(int ch)
{
 switch(ch)
 {
 case '\n': case '\t': case ' ': case ',': case '.': case ':': case ';': case '!':
 case '-':
 return 1;
 }
 return 0;
}

```

This is a simple program that counts the words in a text file, by counting the characters that terminate a word. This program may give incorrect output if the file contains two or more adjacent terminating characters. For example there are two or more adjacent spaces or a full stop followed by a space or a newline. In this case we can modify the if condition as-

```
if(is_end(line[i]) && !is_end(line[i-1]))
```

In this case the initial value of i in for loop will be 1.

#### Problem 4

Write a program to count the number of occurrences of a given word in a text file. No line of text file contains more than 80 characters(including newline character). Suppose the name of file is test.txt and the word to be counted is "that" and the file contains the following data.

This is the house that Jack built, that is the house that John built.

This is better than that.

His name is John.

She is not sure whether that is his book.

The output should reproduce the lines containing the word along with the number of occurrences of given word and at the end it should give the frequency of the given word in the file as-

Number of times the given word occurs in the file is 5.

```

/*P12.30*/
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

```

```

display(char line[], char wordtext[]);
is_end(int ch);
main(void)

char line[81];
int total=0;
FILE *fptr;

if((fptr=fopen("test.txt", "r"))==NULL)
{
 printf("File doesn't exist\n");
 exit(1);
}
while((fgets(line, 81, fptr))!=NULL)
 total=total + display(line, "that");
printf("Number of times the given word occurs in file is %d\n", total);
fclose(fptr);
return 0;

```

```

display(char line[], char wordtext[])
{
 int i, j, k, len;
 char str[80];
 int count=0;
 len=strlen(wordtext);
 for(i=0; line[i]!='\0'; i++)
 {
 k=0;
 if(is_end(line[i-1]) && is_end(line[i+len]))
 {
 for(k=0, j=i; k<len; j++, k++)
 str[k]=line[j];
 str[k]='\0';
 if(strcmp(str, wordtext) == 0)
 count++;
 }
 }
 if(count>0)
 {
 printf("%s:", line);
 printf("count=%d\n", count);
 }
 return count;
}

```

```

is_end(int ch)
{
 switch(ch)
 {
 case '\n': case '\t': case ' ': case ',': case '.': case ':': case ';': case '!':
 case '-':
 return 1;
 }
 return 0;
}

```

Output :

This is the house that Jack built, that is the house that John built.

count=3

This is better than that.

count=1

He is not sure whether that is his book.

count=1

Number of times the given word occurs in the file is 5.

In the function `display()`, first we find the length of the given word using `strlen()` function. Suppose length is 4, then we will extract the 4 letter words in the string `str` one by one, and this string `str` is compared with the given word using `strcmp()`.

This program would not count the word if it appears in the beginning of the line. For example if there is a line in the file like this-

his book.

Here the word "that" will not be counted. The reason for this is that in our logic we have assumed that before a word, there is a word-ending character (eg space, newline, comma etc).

But this is the first character of the array, so this condition fails and the word is not counted. To get the correct output we can modify the if condition as-

```
(is_end(line[i-1]) || i==0) && is_end(line[i+len]))
```

If you want to count those occurrences of the word also, when it is embedded inside another word, then remove the above if condition totally. For example if the word to be searched is "rot", then its occurrence in words like **parrot**, **rotten**, **carrot**, **rotate**, **rotary** will be counted.

The search performed by our program is case sensitive, since the comparison performed by `strcmp()` is case sensitive. If we want to perform case insensitive search then we'll have to make a function of our own that performs case insensitive search of two strings.

```
cmp_in(char *str1, char *str2)
```

```
int i;
for(i=0; str1[i] != '\0'; i++)
if (toupper(str1[i]) != toupper(str2[i]))
return 1;
return 0;
```

This function compares two strings of equal lengths and returns 1 if they are not same, otherwise 0. The declaration for the function `toupper` is in the header file `ctype.h`.

## Problem 5

Write a program to read a text file `wrong.txt` and copy to another file `right.txt` after making corrections in the file as follows:

Replace the first character of the word to uppercase, if the previous word was terminated by a fullstop or a newline.

Replace the word 'a' with the word 'an' if the next word starts with a vowel.

Phonetic a, e, i, o, u or A, E, I, O, U are vowels)

```
12.31*/
#include<stdio.h>
#include<ctype.h>
#include<stdlib.h>
is_vowel(int ch);
main(void)
```

```
char wrong[100],right[150];
int i,j;
FILE *fptr1,*fptr2;

if((fptr1=fopen("wrong.txt","r")) == NULL)
{
 printf("Error in opening file\n");
 exit(1);
}
if((fptr2=fopen("right.txt","w")) == NULL)
```

```
{
 printf("Error in opening file\n");
 exit(1);
}
while((fgets(wrong,100,fptr1)) != NULL)
{
 i=j=0;
 while(wrong[i]!='\0')
 {
 if(islower(wrong[i]) && (i==0 || wrong[i-1]!='.'))
 right[j++]=toupper(wrong[i++]);
 else if(wrong[i]==' ' && wrong[i-1]!='a' && is_vowel(wrong[i+1]))
 {
 right[j++]='n';
 right[j++]=wrong[i++];
 }
 else
 right[j++]=wrong[i++];
 }
 right[j]='\0';
 fputs(right, fptr2);
}
```

```
int is_vowel(int ch)
```

```
switch(ch)
{
 case 'a': case 'A':
 case 'e': case 'E':
 case 'i': case 'I':
 case 'o': case 'O':
 case 'u': case 'U':
 return 1;
}
return 0;
```

## Exercise

Find the output of the following programs; assume `stdio.h` is included in all programs.

```
1.int main(void)
```

```
{
 FILE *fptr;
 unsigned char ch;
 fptr=fopen("myfile.txt","r");
 while((ch=fgetc(fptr))!=EOF)
 putchar(ch);
 fclose(fptr);
 return 0;
}
```

```
2. int main(void)
```

```
{
 FILE *fp;
 int ch;
 fp=fopen("myfile.txt", "w");
 fprintf(fp,"If equal love there cannot be..");
 fputc(26,fp);
 fprintf(fp,"..let the more loving one be me\n");
 fclose(fp);
 fp=fopen("myfile.txt","r");
```

```

while((ch=fgetc(fp))!=EOF)
 putchar(ch);
return 0;
}

```

3. int main(void)

```

{
 FILE *fptr1, *fptr2;
 char fname[20];
 printf("Enter the path of first file : ");
 scanf("%s", fname);
 fptr1=fopen(fname, "r");
 if(fptr1==NULL)
 {
 printf("Error in opening first file\n");
 exit(1);
 }
 fptr2=fopen("c:\\mydir\\names.txt", "r");
 if(fptr2==NULL)
 {
 printf("Error in opening second file\n");
 exit(1);
 }
 fclose(fptr1);
 fclose(fptr2);
 return 0;
}

```

Suppose the path of first file is entered as- c:\\mydir\\myfile.txt

4. int main(void)

```

{
 FILE *fptr;
 int ch;
 fptr=fopen("names.txt", "r");
 while((ch=fgetc(fptr))!=EOF)
 putchar(ch);
 fclose(fptr);
 return 0;
}

```

5. int main(void)

```

{
 char name[50];
 int empid;
 fprintf(stdout, "Enter your name : ");
 fgets(name, 50, stdin);
 fprintf(stdout, "Enter your empid : ");
 fscanf(stdin, "%d", &empid);
 fprintf(stdout, "Your empid is : %d", empid);
 fputc('\n', stdout);
 fprintf(stdout, "Your name is : ");
 fputs(name, stdout);
 return 0;
}

```

6. int main(void)

```

{
 FILE *fptr;
 char str[80];
 fptr=fopen("test.txt", "r");
 while(fgets(str, 80, fptr)!=EOF)
 puts(str);
 return 0;
}

```

```

int main(int argc, char *argv[])
{

```

```

 int i, sum=0;
 for(i=1; i<argc; i++)
 sum=sum+atoi(argv[i]);
 printf("%d\n", sum);
 return 0;
}

```

Suppose the executable file is C:\Add.exe and on the command line it is invoked as-  
Add.exe 2 4 6 8

1. Write a program to copy a file to another file such that blank lines are not written to the new file.
2. Write a program to convert all the lower case characters of a file to upper case.
3. Write a program to display the total number of alphabets and numeric characters in a file.
4. Write a program to insert line numbers and page numbers in a file.
5. Write a program to remove all comments from the file. Assume that a comment starts with double slash ( // ) and continues till the end of the line.
6. Write a program that copies data from different source files to a destination file. The names of destination file and source files should be passed through command line arguments.
7. Consider this structure-

```

struct employee
{
 char name;
 int age;
 int sal;
}

```

Write a program to store records of this structure in a file and sort them on the basis of name and age, where name is the primary key and age is secondary key.

## 13

## The C Preprocessor

The C preprocessor scans and modifies the source code before compilation. The main functions performed by the preprocessor are-

- Simple Macro Substitution
- Macros with parameters
- Conditional Compilation
- File Inclusion
- Error generation and pragma directives.

- Some advantages of using preprocessor are-
- Enhances readability of the program.
  - Program modification becomes easy.
  - Program becomes portable and efficient.
  - Can be used for testing and debugging.

The lines starting with the symbol # are known as preprocessor directives. When the preprocessor finds a line starting with the symbol #, it considers that line as a command for itself and works accordingly. All the directives are executed by the preprocessor, and the compiler does not receive any line starting with # symbol.

Some features of preprocessor directives are-

- Each preprocessor directive starts with a # symbol.
- There can be only one directive on a line.
- To continue a directive on next line, we should place a backslash at the end of the line.
- Directives don't end with a semicolon.
- The preprocessor directives can be placed anywhere in a program (inside or outside functions) but they are usually written at the beginning of a program.
- A directive is active from the point of its appearance till the end of the program.

The preprocessor directives that perform these functions are given below-

|        |          |        |        |       |         |
|--------|----------|--------|--------|-------|---------|
| #if    | #include | #if    | #else  | #elif | #endif  |
| #ifdef | #ifndef  | #error | #undef | #line | #pragma |

There are three operators that are used with these directives-

- Defined operator - defined()
- Stringizing operator - #
- Token pasting operator - ##

Apart from the facilities provided through directives, another important task that preprocessor performs is that it replaces each comment by a single space.

A preprocessor directive consisting only of the symbol # is known as the null directive and it has no effect.

## 13.1 #define

We have already used this directive to define symbolic constants. The general syntax is-

```
#define macro_name macro_expansion
```

Here macro\_name is any valid C identifier, and it is generally in capital letters to distinguish it from other variables. The macro\_expansion can be any text. A space is necessary between the macro\_name and macro\_expansion. The C preprocessor replaces all the occurrences of macro\_name with the macro\_expansion. Some examples of #define directive are-

```
#define TRUE 1
#define FALSE 0
#define PI 3.14159265
#define MAX 100
#define RETIRE_AGE 58
```

The C preprocessor searches for macro\_name in the C source code and replaces it with the macro\_expansion. For example wherever the macro name TRUE appears in the code, it is replaced by 1. These types of constants are known as symbolic constants. These constants increase the readability of the program, for example it is better to use the name PI instead of its value. Moreover if we decide to change the value of a constant, then the change has to be made only at one place. For example if we decide to increase the RETIRE\_AGE from 58 to 60, then we have to change the #define directive only. If we had not defined this constant, we would have to replace each occurrence of 58 by 60. We have already utilized this feature to define the size of an array.

We can also define any string constant in the place of macro\_expansion.

```
13.1 Program to show macro expansion*/
#include <stdio.h>
#define MSG "I understand the use of #define\n"
int main(void)
{
 printf(MSG);
 return 0;
}
```

Output:  
I understand the use of #define

The C preprocessor searches for the macro name MSG and substitutes the message "I understand the use of #define\n" in the printf() call.

We can have macro expansions of any sort. Here are some more examples-

```
#define AND &&
#define OR ||
#define BEGIN int main(void){
#define END }
#define INFINITE while(1)
#define NEW_LINE printf("\n");
#define ERROR printf("An error has occurred\n");
```

There is a semicolon in the last two directives. This semicolon is just a part of the macro\_expansion. Now whenever we write NEW\_LINE or ERROR in the program, they will be replaced by printf calls and the terminating semicolon.

Remember that the C preprocessor will simply replace the macro\_name with the macro\_expansion and the macro\_expansion can be any text. So it is our responsibility to see that after replacement the resulting code is valid in C. For example suppose we define a macro MAX like this-

```
#define MAX 5;
```

Now consider this declaration statement-

```
int arr[MAX];
```

This will be translated by the preprocessor as-

```
int arr[5];
```

and this is not valid in C.

If the macro\_name appears inside a character constant, string constant or a comment then it is not replaced and is left as it is. For example-

```
#define SUN 2
#define WINDOWS 3
printf("Rays of SUN are coming through the WINDOWS");
```

Here the replacement will not take place inside the string.

It is also possible to define a macro without any macro expansion. For example-

```
#define TEST
```

This is used in compilation directives where the preprocessor only needs to know whether the macro has been defined or not.

### 13.2 Parameterized Macros

The #define directive can also be used to define macros with parameters. The general syntax is-

```
#define macro_name(par1, par2,) macro_expansion
```

Here par1, par2, ..... are the formal parameters. The macro\_name is replaced with the macro\_expansion and the formal parameters are replaced by the corresponding actual arguments supplied in the macro call. So the macro\_expansion will be different for different actual arguments. For example suppose we define these two macros-

```
#define SUM(x,y) ((x)+(y))
#define PROD(x,y) ((x)*(y))
```

Now suppose we have these two statements in our program-

```
s=SUM(5,6);
p=PROD(m,n);
```

After passing through the preprocessor these statements would be expanded as-

```
s=({5}+{6});
p=({m}*{n});
```

Since this is just replacement of text, the arguments can be of any data type. For example we may use the macro SUM to find out the sum of long or float types also.

The following statement-

```
s=SUM(2.3,4.8);
```

would be expanded as-

```
s=({2.3}+{4.8});
```

*/\*P13.2 Program to understand macros with arguments\*/*

```
#include<stdio.h>
#define SUM(x,y) ((x)+(y))
#define PROD(x,y) ((x)*(y))
int main(void)
{
 int l,m,i,j,a=5,b=3;
 float p,q;
 l=SUM(a,b);
 m=PROD(a,b);
 i=SUM(4,6);
 j=PROD(4,6);
 p=SUM(2.2,3.4);
 q=PROD(2.2,3.4);
 printf("l=%d, m=%d, i=%d, j=%d, p=%f, q=%f\n",l,m,i,j,p,q);
 return 0;
}
```

Output:

```
l=8, m=15, i=10, j=24, p=5.6, q=7.5
```

Let us see some more examples of parameterized macros.

```
#define SQUARE(x) ((x)*(x))
#define MAX(x,y) ((x)>(y) ? (x) : (y))
```

```
#define MIN(x,y) ((x)<(y) ? (x) : (y))
#define ISLOWER(c) (c>=97 && c<=122)
#define ISUPPER(c) (c>=65 && c<=90)
#define TOUPPER(c) ((c)+'A'-'a')
#define TSLRAP(y) ((y%100!=0 && y%4==0) || (y%400==0))
#define BLANK_LINES(n) { int i; for(i=0; i<n; i++) printf("\n"); }
#define CIRCLE_AREA(rad) (3.14*(rad)*(rad))
```

Note that there should be no space between the macro\_name and left parenthesis, otherwise the macro expansion is considered to start from the left parenthesis. For example if we write a macro like this-

```
#define SQUARE (x) ((x)*(x))
```

Now any call like SQUARE(5) would be expanded as-

```
(x) ((x)*(x)) (5)
```

Here SQUARE is considered as a macro without parameters and the text (x) ((x)\*(x)) is regarded as macro expansion. So SQUARE is replaced by the macro expansion and (5) is written as it is.

### 13.3 Nesting in Macros

One macro name can also be used for defining another macro name i.e. the macro expansion can also contain the name of another macro. For example-

```
#define PI 3.14
#define PISQUARE PI*PI
```

Now if we have an expression-

```
t=PISQUARE;
```

First time the preprocessor will expand it as-

```
t=PI*PI;
```

The preprocessor rescans this expanded expression, and if any macros are found then it replaces them. This process of rescanning continues till no macros are left in the expression. So finally the expanded expression would be-

```
t=3.14*3.14;
```

Here is another example of nesting of macros-

```
#define ISLOWER(c) (c>=97 && c<=122)
#define TOUPPER(c) ISLOWER(c) ? ((c)+'A'-'a') : c
```

Here the macro TOUPPER uses the macro ISLOWER in its expansion. If the character is lowercase then only it will be changed to uppercase otherwise it will remain unchanged.

The following program illustrates the use of nested macros-

*Fig.3 Program to understand nesting in macros\*/*

```
#include<stdio.h>
#define ISLOWER(c) (c>=97 && c<=122)
#define ISUPPER(c) (c>=65 && c<=90)
#define ISALPHA(c) ISLOWER(c) || ISUPPER(c)
#define ISNUM(c) (c>=48 && c<=57)
#define ISALPHANUM(c) ISALPHA(c) || ISNUM(c)
int main(void)
{
 char ch;
 printf("Enter a character : \n");
 scanf("%c",&ch);
 if(ISALPHANUM(ch))
 printf("%c is an alphanumeric character\n",ch);
 else
 printf("%c is not an alphanumeric character\n",ch);
 return 0;
}
```

If the macro name appears inside its own expansion, then it is not expanded. So don't try to write recursive macros like this one-

```
#define FACTORIAL(k) k==0 ? 1 : k*FACTORIAL(k-1) /*Incorrect*/
```

### 13.4 Problems with Macros

You must be wondering why so many parentheses are used in the macro expansion. Since the preprocessor simply replaces the formal parameter with the actual argument, we may not get the desired result in some cases. Let us see what sort of problems can occur while using macros and how they can be avoided.

In the following program we have defined a macro PROD without any parentheses in the macro expansion

```
/*P13.4*/
#include<stdio.h>
#define PROD(x,y) x*y
int main(void)
{
 printf("%d\t", PROD(2,4));
 printf("%d\n", PROD(3+4, 5+1));
 return 0;
}
```

Output:  
8      24

Here the first printf() gives the correct result 8, but the second printf() prints 24 while we were expecting the result to be a product of 7 and 6 i.e. 42. Let us see how the preprocessor expanded these macros.

|                |                |         |
|----------------|----------------|---------|
| PROD(2,4)      | is expanded as | 2*4     |
| PROD(3+4, 5+1) | is expanded as | 3+4*5+1 |

The precedence of multiplication operator is more than addition operator so the calculation went like this,  $3+20+1=24$  and hence we got the wrong result. This problem can be solved by enclosing each formal parameter inside parentheses. So now we rewrite our program like this-

```
/*P13.5*/
#include<stdio.h>
#define PROD(x,y) (x)*(y)
int main(void)
{
 printf("%d\t", PROD(2,4));
 printf("%d\t", PROD(3+4, 5+1));
 printf("%d\n", 60/PROD(2,3));
 return 0;
}
```

Output:  
8      42      90

Now the first two printf() give us the desired result. In the third printf() we had tried to divide 60 by the product of 2 and 3 and the expected result was 10 but we got 90, so still there is a problem in our macro. Again let us see how preprocessor has done the expansions.

|                |                |             |
|----------------|----------------|-------------|
| PROD(2,4)      | is expanded as | (2)*(4)     |
| PROD(3+4, 5+1) | is expanded as | (3+4)*(5+1) |
| 60/PROD(2,3)   | is expanded as | 60/(2)*(3)  |

Here we can see that in the last case, first 60 is divided by 2 and then the result is multiplied by 3 because and \* operators associate from left to right. So the result was not the expected one. To solve this sort of problem we should enclose whole macro expansion inside parentheses. So the correct way of defining macro is-

```
#define PROD(x,y) ((x)*(y))
```

Now  $60/\text{PROD}(2,3)$  would be expanded as  $60/((2)*(3))$  and thus 60 is divided by the product of 2 and 3, and we can get the desired result.

So if your macro is accepting arguments, it is better to put parentheses around the entire macro expansion and also around each argument. Now consider this program and we will find another sort of problem.

```
/*P13.6*/
#include<stdio.h>
#define SQUARE(x) ((x)*(x))
int main(void)
{
 int k=5, s;
 s=SQUARE(k++);
 printf("s=%d, k=%d\n", s, k);
 return 0;
}
```

Output:  
s=25, k=7

The statement  $s=\text{SQUARE}(k++)$ ; is expanded as  $s=((k++)*(k++))$ ;

The value of k is incremented twice while we intended to assign the square of k to variable s, and then increase the value of k only once. So in this case it is better if we use a function instead of macro.

```
/*P13.7*/
#include<stdio.h>
int square(int x)
{
 return x*x;
}
```

```
int main(void)
{
 int k=5, s;
 s=square(k++);
 printf("s=%d, k=%d\n", s, k);
 return 0;
}
```

Output:  
s=25, k=6

Now we will see a different type of problem. Consider this program, in which we have written a macro to swap the value of 2 variables of any datatype.

```
/*P13.8*/
#include<stdio.h>
#define SWAP(dtype,x,y) {dtype t; t=x, x=y, y=t;}
int main(void)
{
 int a=2, b=5;
 SWAP(int, a, b);
 printf("a=%d, b=%d\n", a, b);
 return 0;
}
```

Output:  
a=5, b=2

There is no problem in this program. It worked according to our expectations and swapped the values of a and b. The macro was expanded in this way-

$\text{SWAP}(\text{int}, a, b)$  is expanded to  $\{ \text{int } t; t=a, a=b, b=t; \}$

After macro expansion, the main() function would look like this-

```
int main(void)
{
 int a=2, b=5;
 { int t; t=a, a=b, b=t; }
 printf("a=%d, b=%d\n", a, b);
 return 0;
}
```



Here `t` is a variable that is local to the block of code in which it is defined.

Now consider the next program, it is similar to the above program except that the names of the variables to be swapped are `s` and `t`.

```
/*P13.9*/
#include<stdio.h>
#define SWAP(dtype,x,y) { dtype t; t=x; x=y; y=t; }
int main(void)
{
 int s=2,t=5;
 SWAP(int,s,t);
 printf("s=%d, t=%d\n",s,t);
 return 0;
}
```

Output:  
s=2, t=5

The values of variables `s` and `t` were not swapped. After macro expansion, the function `main()` would look like this:

```
int main(void)
{
 int s=2,t=5;
 { int t; t=s; s=t; t=t; }
 printf("s=%d, t=%d\n",s,t);
 return 0;
}
```

So now you know why things went wrong. When the macro was expanded, there was a conflict between the variable `t` declared inside the block, and the variable `t` declared outside the block in `main()`. To avoid such type of problems, we can use some naming convention for local variables declared in macros. For example we may decide to write them in capitals.

Here is another problem that can cause the program to give unexpected results.

```
/*P13.10*/
#include<stdio.h>
#define MACRO(x) if(x==0) printf("Out for a Duck\n")
int main(void)
{
 int runs=12;
 if(runs<100)
 MACRO(runs);
 else
 printf("Scored a century\n");
 return 0;
}
```

Output:  
Scored a century

The runs were only 12, so the output is wrong. Let's see what happened after the code was expanded.

The expanded code is-

```
if(runs<100)
 if(runs==0) printf("Out for a duck");
else
 printf("Scored above century\n");
```

The `else` part was matched with the `if` part of the macro. To avoid this problem either we can enclose the whole macro inside parentheses or we may use a conditional operator.

```
#define MACRO(x) { if (x==0) printf("Out for a Duck\n") }
#define MACRO(x) x==0 ? printf("Out for a duck\n") : printf("\n")
```

## 13.5 Macros vs. Functions

We have seen that macros with arguments can perform tasks similar to functions. In this topic we'll compare the advantages and disadvantages of functions and macros.

A macro is expanded into inline code so the text of macro is inserted into the code for each macro call. Hence macros make the code lengthy and the compilation time increases. On the other hand the code of a function is written only at one place, regardless of the number of times it is called so the use of functions makes the code smaller.

In functions, the passing of arguments and returning a value takes some time and hence the execution of the program becomes slow while in the case of macros this time is saved and they make the program faster. So functions are slow but take less memory while macros are fast but occupy more memory due to duplicity of code. If the macro is small it is good but if it is large and is called many times then it is better to change it into a function since it may increase the size of the program considerably. But sometimes macros can prove very useful and also improve the execution speed. For example in the file `stdio.h`, `getchar()` and `putchar()` are defined as macros. If they were defined as functions then there would be a function call for processing each character, which would increase the run time.

Preprocessor just replaces or substitutes the text without any sort of checking. So macros can be used with arguments of different types, as we have seen earlier in the case of macro `SUM` in program P13.2. Functions perform type checking so separate functions have to be written for each data type. But remember that lack of type checking facility in macros makes them more error prone. Macros don't have addresses, while functions have addresses and they can be called using a function pointer. So whether to use a macro or a function depends on the memory available, your requirement and the nature of the task to be performed.

## 13.6 Generic Functions

We can define macros that can be used to generate functions for different data types. These types of macros are called generic functions. These macros generally take the function name and data type as arguments and the macro call is replaced by a function definition. Let us take a program to understand this.

```
/*P13.11 Program to understand generic functions*/
#include<stdio.h>
#define MAX(FNAME,DTYPE) \
 DTYPE FNAME(DTYPE X, DTYPE Y) \
 { \
 return X>Y ? X : Y; \
 }
int main(void)
{
 (max_int,int)
 (max_float,float)
 (max_double,double)
 (main(void)
 {
 int p;
 float q;
 double r;
 p=max_int(3,9);
 q=max_float(7.4,5.7);
 r=max_double(12.34,13.54);
 printf("p=%d,q=%f,r=%f\n",p,q,r);
 return 0;
 })
}
```

Output:  
p=7.40, r=13.54

The three macro calls written just before `main()` are expanded as-

```
MAX(max_int, int) → int max_int(int X, int Y)
{
 return X>Y ? X : Y;
}
```

```

MAX(max_float, float) → float max_float(float X, float Y)
{
 return X>Y ? X : Y;
}

MAX(max_double, double) → double max_double(double X, double Y)
{
 return X>Y ? X : Y;
}

```

So we can see that the three macro calls written before `main()` are expanded into function definitions. In this way we can generate function definitions for different data types.

### 13.7 #undef

The definition of a macro will exist from the `#define` directive till the end of the program. If we want to undefine this macro we can use the `#undef` directive.

Syntax-

```
#undef macro_name
```

After this directive if the `macro_name` is encountered in the program then it will not be replaced by the macro expansion. In other words now the scope of the macro is limited to the code between `#define` and `#undef` directives. This is used when we want the macro definition to be valid for only a part of the program. This directive is generally useful with other conditional compilation directives.

### 13.8 Stringizing Operator (#)

If in the definition of macro, a formal parameter occurs inside a string in the macro expansion then it is not replaced by the actual argument. For example if we have a macro-

```
#define SHOW(var) printf("var=%d", var)
```

Then a call `SHOW(x);` will be expanded as `printf("var=%d", x);`

Here the formal parameter `var` outside the string was replaced by the actual argument `x`, but inside the string this replacement did not take place.

To solve this sort of problem we can use the stringizing operator. This operator is used within the definition of macro. It causes the argument it precedes to be turned into a string or in other words it stringizes a macro argument. So if in the macro expansion, we have a formal parameter preceded by `#`, then both this operator and parameter are replaced by the actual argument surrounded within double quotes. We can write the above macro using stringizing operator as-

```
#define SHOW(var) printf("#var" "%d", var)
```

Now a call `SHOW(x);` will be expanded as `printf("#x" "%d", x);`

We know that adjacent strings are concatenated so after string concatenation this becomes-  
`printf("x=%d", x);`

We can make the above macro more general so that it can display variable of any type.

```
/*P13.12 Program to understand the use of stringizing operator*/
```

```
#include<stdio.h>
```

```
#define SHOW(var, format) printf("#var" "%s" format "\n", var);
```

```
int main(void)
```

```
{
 int x=9;
 float y=2.5;
 char z='$';
 SHOW(x,d);
 SHOW(y,0.2f);
 SHOW(z,c);
 SHOW(x*y,0.2f);
 return 0;
}
```

```
Input:
```

```
9
```

```
2.50
```

```
22.50
```

The macros in the above program were expanded as-

```

SHOW(x,d); → printf("x" "%d" "\n", x);
SHOW(y,0.2f); → printf("y" "%0.2f" "\n", y);
SHOW(z,c); → printf("z" "%c" "\n", z);
SHOW(x*y,0.2f); → printf("x*y" "%0.2f" "\n", x*y);

After string concatenation the above statements look like this-
printf("x=%d\n", x);
printf("y=%0.2f\n", y);
printf("z=%c\n", z);
printf("x*y=%0.2f\n", x*y);

```

### 13.9 Token Pasting Operator (##)

Token pasting operator `##` is used in a macro definition to concatenate two tokens into a single token. As the name implies, this operator pastes the two tokens into one.

```
/*P13.13 Program to understand the use of token pasting operator*/
```

```
#include<stdio.h>
```

```
#define PASTE(a,b) a##b
```

```
#define MARKS(subject) marks_##subject
```

```
int main(void)
```

```
{
 int k2=14, k3=25;
 int marks_chem=89, marks_maths=98;
 printf("%d %d", PASTE(k,2), PASTE(k,3));
 printf("%d %d\n", MARKS(chem), MARKS(maths));
 return 0;
}
```

```
Input:
```

```
25 89 98
```

The token pasting operator converts the above statements into-

```

printf("%d %d", k2, k3);
printf("%d %d", marks_chem, marks_maths);

```

### 13.10 Including Files

The preprocessor directive `#include` is used to include a file into the source code. We have already used this directive to include header files in our programs. The filename should be within angle brackets or double quotes. The syntax is-

```

#include "filename"
#include <filename>

```

The preprocessor replaces the `#include` directive by the contents of the specified file. After including the file, the entire contents of file can be used in the program. If the filename is in double quotes, first it is searched in the current directory (where the source file is present), if not found there then it is searched in the standard include directory. If the filename is within angle brackets, then the file is searched in the standard include directory only. The specification of standard include directory is implementation defined.

Generally angled brackets are used to include standard header files while double quotes are used to include header files related to a particular program. We can also specify the whole path instead of the path name. For example-

```
#include "C:\mydir\myfile.h"
```

Note that here the path is not a string constant so there is no need to double the backslashes.

#include directive should not be used to include the contents of other '.c' files. It is generally used to include header files only. Header files are given '.h' extension to distinguish them from other C files, although this is not necessary. Header files generally contain macro definitions, declarations of enum, structure, union, typedef, external functions and global variables. The function definitions or global variable definitions should not be put in the header files.

Include files can be nested i.e. an included file can contain another #include directive.

## 13.11 Conditional Compilation

There may be situations when we want to compile some parts of the code based on a condition. We know that before compilation the source code passes through the preprocessor. So we can direct the preprocessor to supply only some parts of the code to the compiler for compilation.

Conditional compilation means compilation of a part of code based on some condition. These conditions are checked during the preprocessing phase. The directives used in conditional compilation are-

#ifdef, #ifndef, #if, #else, #elif, #endif

Every #if directive should end with a #endif directive. The working of these directives is somewhat similar to that of if...else construct.

### 13.11.1 #if and #endif

An expression which is followed by the #if is evaluated, if result is non-zero then the statements between #if and #endif are compiled, otherwise they are skipped. This is written as-

```
#if constant-expression
.....
statements
.....
#endif
```

The constant-expression should be an integral expression and it should not contain enum constants, sizeof operator, cast operator or any keyword or variables. It can contain arithmetic, logical, relational operators. If any undefined identifier appears in the expression, it is treated as having the value zero.

```
/*P13.14 Program to understand the use of #if directive*/
#include<stdio.h>
#define FLAG 8
int main(void)
{
 int a=20,b=4;
 #if FLAG>=3
 printf("Value of FLAG is greater than or equal to 3\n");
 a=a+b;
 b=a*b;
 printf("Values of variables a and b have been changed\n");
 #endif
 printf("a=%d,b=%d\n",a,b);
 printf("Program completed\n");
 return 0;
}
```

Output:  
Value of FLAG is greater than or equal to 3  
Values of variables a and b have been changed  
a=24, b=96  
Program completed

In this program FLAG is defined with the value 8. First the constant expression FLAG>=3 is evaluated, since it is true(non zero), all the statements between #if and #endif are compiled. Suppose the value of FLAG

changed to 2, now the constant expression FLAG>=3 would evaluate to false(zero), hence the statements between #if and #endif will not be compiled. In this case the output of the program would be-

```
a=20, b=4
Program completed
```

### 13.11.2 #else and #elif

#else is used with the #if preprocessor directive. It is analogous to if...else control structure. The syntax is

```
#if constant-expression
.....
statements
.....
#else
.....
statements
.....
#endif
```

If the constant expression evaluates to non-zero then the statements between #if and #else are compiled, otherwise the statements between #else and #endif are compiled.

/\*P13.15 Program to understand the use of #else directive\*/

```
#include<stdio.h>
#define FLAG 8
int main(void)
{
 int a=20,b=4;
 #if FLAG>=3
 printf("Value of FLAG is greater than or equal to 3\n");
 a=a+b;
 b=a*b;
 #else
 printf("Value of FLAG is less than 3\n");
 a=a-b;
 b=a/b;
 #endif
 printf("a=%d,b=%d\n",a,b);
 printf("Program completed\n");
 return 0;
}
```

Output:  
Value of FLAG is greater than or equal to 3  
a=24, b=96  
Program completed

Here the value of FLAG is 8, so the constant expression FLAG>=3 is evaluated to true and hence the statements between #if and #else are compiled. If the value of FLAG is changed to 2, then the constant expression FLAG>=3 would evaluate to zero, so now the statements between #else and #endif will be compiled and the output of the program would be-

```
Value of FLAG is less than 3
a=16, b=4
Program completed
```

Nesting of #if and #else is also possible and this can be done using #elif. This is analogous to the else...if ladder that we had studied in control statements. Every #elif has one constant expression, first the expression is evaluated if the value is true then that part of code is compiled and all other #elif expressions are skipped, otherwise the next #elif expression is evaluated.

Syntax-

```
#if constant-expression1
.....
#elif constant-expression2
.....
#elif constant-expression3
.....
```

#else

#endif

The above code can be written using #if and #else directives as-

#if constant-expression1

#else

#if constant-expression2

#else

#if constant-expression3

#else

#endif

#endif

#endif

Here for each #if there should be a #endif, while when using #elif there is only one #endif. Let us take a program that uses #elif directive-

*\*P13.16 Program to understand the use of #elif directive\*/*

#include&lt;stdio.h&gt;

define FLAG 1

int main(void)

int a=20,b=4;

#if FLAG==0

printf("Value of FLAG is zero\n");

a++;

b++;

#elif FLAG==1

printf("Value of FLAG is one\n");

a--;

b--;

#elif FLAG==2

printf("Value of FLAG is two\n");

a=a-3;

b=b-3;

#else

printf("Value of FLAG is greater than two or less than zero\n");

a=a+b;

b=a-b;

#endif

printf("a=%d,b=%d\n",a,b);

printf("Program completed\n");

return 0;

Output:

Value of FLAG is one

a=19, b=3

Program completed

Here the expression FLAG==1 is true, hence the statements corresponding to this #elif are compiled.

### 13.11.3 defined Operator

Syntax : defined(macro\_name)

This operator is used only with #if and #elif directives. It evaluates to 1 if the macro\_name has been defined using #define, otherwise it evaluates to zero. For example-

#if defined(FLAG)

If the macro FLAG has been defined using #define, then the value of expression defined(FLAG) would be 1, otherwise its value would be 0.

### 13.11.4 #ifdef and #ifndef

The directives #ifdef and #ifndef provide an alternative short form for combining #if with defined operator.

#if defined(macro\_name) is equivalent to #ifdef macro\_name

#if !defined(macro\_name) is equivalent to #ifndef macro\_name

Syntax of #ifdef-

#ifdef macro\_name

#endif

If the macro\_name has been defined with the #define directive, then the statements between #ifdef and #endif will be compiled. If the macro\_name has not been defined or was undefined using #undef then these statements are not compiled.

Syntax of #ifndef-

#ifndef macro\_name

#endif

If the macro\_name has not been defined using #define or was undefined using #undef, then the statements between #ifndef and #endif are compiled. If the macro\_name has been defined, then these statements are not compiled.

Let us take a program-

*\*P13.17 Program to understand the use of #ifdef directive\*/*

#include&lt;stdio.h&gt;

define FLAG

int main(void)

int a=20,b=4;

#ifdef FLAG

printf("FLAG is defined\n");

a++;

b++;

#endif

printf("a=%d,b=%d\n",a,b);

printf("Program completed\n");

return 0;

Output:

FLAG is defined

a=21, b=5

Program completed

The macro FLAG has been defined, so the statements between #ifdef and #endif are compiled. If we delete the definition of macro FLAG from the program, then these statements won't be compiled and the output of the program would be-

a=20, b=4

Program completed

*\*P13.18 Program to understand the use of #undef and #ifdef directives\*/*

#include&lt;stdio.h&gt;

define FLAG

int main(void)

int a=20,b=4;

#ifdef FLAG

```

printf("FLAG is defined\n");
a++;
b++;
#endif
#undef FLAG
#ifdef FLAG
printf("Preprocessor\n");
a++;
b++;
#endif
printf("a=%d,b=%d\n",a,b);
printf("Program completed\n");
return 0;
}

```

Output:  
FLAG is defined  
a=21,b=5  
Program completed

Here we have undefined the macro FLAG in the middle of program. So the macro FLAG is defined only for the part of the program that is before the #undef directive, and it is undefined for the rest of the program.

*/\*P13.19 Program to understand the use of #ifndef directive\*/*  
#include<stdio.h>  
int main(void)

```

{
 int a=20,b=4;
 #ifndef MAX
 printf("MAX is not defined\n");
 a--;
 b--;
 #endif
 printf("a=%d,b=%d\n",a,b);
 printf("Program completed\n");
 return 0;
}

```

Output:  
MAX is not defined  
a=19,b=3  
Program completed

Here the macro MAX has not been defined, so the statements between #ifndef and #endif are compiled. The existence of defined operator is important in some cases. For example we can't use the short forms(#ifdef and #ifndef) here-

(i) When we need to check the existence of more than one macro, for example-

```
#if defined(ABC) && defined(PQR) && !defined(XYZ)
```

(ii) When the existence of macro is to be checked in #elif

```

#if defined(ABC)
.....
#elif defined(PQR)
.....
#elif defined(XYZ)
.....
#endif

```

There are lot of situations where conditional compilation can prove useful e.g. writing portable code, debugging, commenting. We'll discuss all these cases one by one.

### 13.11.5 Writing Portable Code

Suppose we want to write a program that can be run on different systems. In this case some lines of the code would be system dependent. These lines can be written using conditional compilation directives. Suppose we

have made different header files corresponding to different machines. We can write code to include the appropriate header file while working on a specific machine.

```

#ifdef MACHINE==HP
 #define hfile abc.h
#elif MACHINE==IBM
 #define hfile pqr.h
#elif MACHINE==UNISYS
 #define hfile xyz.h
else
 #define hfile newfile.h
#endif
#include"hfile"

```

Now when we want to work on HP machine, we can define the macro MACHINE as-

```
#define MACHINE HP
```

So the file abc.h will be included.

If we want to work on UNISYS machine, we can define MACHINE as-

```
#define MACHINE UNISYS
```

Now the file xyz.h will be included.

Similarly if we have some code in the program that is different for different machines, we can use the above structure.

### 13.11.6 Debugging

The compiler can detect syntax errors but it is unable to detect run time errors, which result in incorrect output. To trace these types of errors in the program the debugging has to be done by the programmer. In a large program where many functions and variables are involved this can be a difficult task. For this debugging purpose we can insert some printf statements inside the code that tell us about the intermediate results. After the debugging is complete we don't need these statements and we can delete them. But suppose after few days we again need to debug the program, then we'll have to insert all those statements once again. We have another better alternative using preprocessor directives. Using these directives we can make these debugging statements active or inactive depending on our needs. The following program shows this-

```

#include<stdio.h>
#define DEBUG
int main(void)
{
 int x;
 #ifdef DEBUG
 printf("Starting main()\n");
 #endif

 func();

 #ifdef DEBUG
 printf("Now value of x=%d and y=%d\n",x,y);
 #endif

 func()
 {
 #ifdef DEBUG
 printf("Inside func()\n");
 #endif

 }
}

```

When the debugging is complete we can undefine or delete/comment the macro DEBUG. This will make the #ifdef condition false and all the debugging statements will not be compiled. So in this way we can switch

Here `dec_const` is any decimal constant and `string_const` is any string constant. This directive assigns `dec_const` and `string_const` to the macros `__LINE__` and `__FILE__` respectively. If the `string_const` is not specified then the macro `__FILE__` remains unchanged.

*\*P13.21 Program to understand the use of #line\**

```
#include<stdio.h>
int main(void)
{
 printf("C in depth\n");
 printf("%d %s\n", __LINE__, __FILE__);
 #line 25 "myprog.c"
 printf("%d %s\n", __LINE__, __FILE__);
 return 0;
}
```

Output:  
C in depth  
C:\CinDepth\Programs\prog.c  
5 myprog.c

### 3.12.2 #error

This preprocessor directive is used for debugging purpose. `#error` directive stops compilation and displays a fatal error message attached with this.

Syntax-  
#error message

For example-

```
#ifndef MACRO
error MACRO is not defined
#endif
```

```
#ifdef __TINY__
error This program will not run in the tiny model.
#endif
```

```
#if defined(_Windows) && !defined(_BUILDRTLDDLL)
error Timer not available for Windows
#endif
```

Suppose a program has been written using ANSI C standard, then this line can be inserted at the beginning of the code.

```
#ifndef __STDC__
error This program should be run using ANSI C
#endif
```

Suppose there are two files and both of them can't be included at a time, then we can use `#error` directive to stop inclusion of both files at a time. For example suppose the files `stdarg.h` and `varargs.h` can't be included at a time, then we can insert `#error` directive inside the files like this-

```
/*Structure of file stdarg.h*/
#ifndef __STDARG_H
#define __STDARG_H
#ifdef __VARARGS_H
#error Can't include both stdargs.h and varargs.h
#endif
/*contents of file.....*/
#endif
```

```
/*Structure of file varargs.h*/
#ifndef __VARARGS_H
#define __VARARGS_H
#ifdef __STDARG_H
#error Can't include both stdarg.h and varargs.h
#endif
#endif
```

```
#endif
/*contents of file.....*/
#endif
```

### 3.12.3 #pragma

This is an implementation defined directive that allows various instructions to be given to the compiler.

Syntax-  
#pragma name

Here name is the name of the pragma we want. The pragmas may be different for different compilers. You should check your compiler's manual for the pragmas available and their details. Any unrecognised pragma directive is just ignored, without showing any error or warning.

### Exercise

What is the output of the following programs; assume `stdio.h` is included in all programs.

```
#define MAX 5;
int main(void)
{
 printf(" %d ",MAX);
 return 0;
}
```

```
#define MSSG printf("If you lapse,don't collapse\n");
int main(void)
{
 MSSG
 return 0;
}
```

```
#define PROD (x,y) ((x)*(y))
int main(void)
{
 int a=3,b=4;
 printf("Product of a and b=%d",PROD(a,b));
 return 0;
}
```

```
#define A 50
#define B A+100
int main(void)
{
 int i,j;
 i=B/20;
 j=500-B;
 printf("i=%d,j=%d\n",i,j);
 return 0;
}
```

```
#define NEW_LINE printf("\n");
#define BLANK_LINES(n) { int i; for(i=0; i<n; i++) printf("\n"); }
int main(void)
{
 printf("When you have a chance");
 NEW_LINE
 printf("to embrace an opportunity");
 BLANK_LINES(3)
 printf("Give it a big hug");
}
```

```

NEW_LINE
return 0;
}

6. #define INFINITE while(1)
#define CHECK(a) if(a==0) break
int main(void)
{
 int x=5;
 INFINITE
 {
 printf("%d ",x--);
 CHECK(x);
 }
 return 0;
}

7. #define ABS(x) ((x)<0? -(x) : (x))
int main(void)
{
 int array[4]={1,-2,3, 4};
 int *p=array+3;
 while(p>=array)
 {
 printf("%d ",ABS(*p));
 p--;
 }
 return 0;
}

8. #define ;
int main(void)
{
 printf("If the lift to success is broken. ");
 printf("Try the stairs.");
 return 0;
}

9. #define CUBE(x) (x*x*x)
int main(void)
{
 printf("%d\n",CUBE(1+2));
 return 0;
}

10. #define CUBE(x) ((x)*(x)*(x))
int main(void)
{
 int i=1;
 while(i<=8)
 printf("%d\t",CUBE(i++));
 return 0;
}

11. #define SWAP(dtype,x,y) { dtype t; t=x+y, x=t-x, y=t-y; }
int main(void)
{
 int a=1, b=2, x=3, y=4, s=25, t=26;
 SWAP(int,a,b);
 SWAP(int,x,y);
 SWAP(int,s,t);
 printf("a=%d, b=%d, x=%d, y=%d, s=%d, t=%d\n",a,b,x,y,s,t);
}

```

```

return 0;
}

2. #define INC(dtype,x,i) x=x+i
int main(void)
{
 int arr[5]={20,34,56,12,96},*ptr=arr;
 INC(int,arr[2],3);
 INC(int*,ptr,2);
 printf("ptr=%d\n",*ptr);
 return 0;
}

3. #define INT int
int main(void)
{
 INT a=2,*p=&a;
 printf("%d %d\n",a,*p);
 return 0;
}

4. #define Y 10
int main(void)
{
 #if X || Y && Z
 printf("Sea in Depth\n");
 #else
 printf("See in depth\n");
 #endif
 return 0;
}

15. int main(void)
{
 int x=3,y=4,z;
 z=x+y;
 #include<string.h>
 printf("%d\n",z);
 return 0;
}

16. #define DIFF(FNAME, DTYPE, RTYPE) \
 RTYPE FNAME(DTYPE X, DTYPE Y){ return X-Y;}
DIFF(diff_int, int, int)
DIFF(diff_iptr, int*, int)
DIFF(diff_float, float, float);
DIFF(diff_fptr, float*, int);
int main(void)
{
 int iarr[5]={1,2,3,4,5},a,p,q;
 float farr[7]={1.2,2.3,3.4,4.5,5.6,6.7,7.8},b;
 a=diff_int(iarr[4],iarr[1]);
 b=diff_float(farr[6],farr[2]);
 p=diff_iptr(&iarr[4],&iarr[1]);
 q=diff_fptr(&farr[4],&farr[1]);
 printf("a=%d, b=%.1f, p=%d, q=%d\n",a,b,p,q);
 return 0;
}

17. #define MAX 3
int main(void)
{

```

```

printf("Value of MAX is %d\n",MAX);
#undef MAX
#ifdef MAX
 printf("Have a good day");
#endif
return 0;
}

#define PRINT1(message) printf(message);
#define PRINT2(message) printf("message");
#define PRINT3(message) printf("#message");
int main(void)
{
 PRINT1("If we rest, we rust.\n");
 PRINT2("Attack life, its going to kill you anyways.\n");
 PRINT3("Well done is better than well said.\n");
 return 0;
}

#define show(value) printf("#value"="%d\n",value);
int main(void)
{
 int a=10,b=5,c=4;
 show(a/b*c);
 return 0;
}

#define MACRO(a) if(a<=5) printf("#a"="%d\n",a);
int main(void)
{
 int x=6, y=15;
 if(x<=y)
 MACRO(x);
 else
 MACRO(y);
 return 0;
}

int main(void)
{
 #line 100 "system.c"
 printf("%d %s\n", __LINE__, __FILE__);
 return 0;
}

#include<stdio.h>
#define toupper(c)\
 ((c)>='a' && (c)<='z' ? (c) + ('A'-'a') : (c))
int main(void)
{
 char str[]="Devanshi",*p;
 p=str;
 while(*p!='\0')
 printf("%c",*p++);
 printf("\n");
 p=str;
 while(*p!='\0')
 printf("%c",toupper(*p++));
 return 0;
}

```

# 14

## Operations on Bits

We know that inside the computer, data is represented in binary digits called bits (0 and 1). Till now we were able to access and manipulate bytes only. But some applications, such as system programming require manipulation of individual bits within a byte. In most high-level languages this facility is not available, but C has the special feature to manipulate individual bits of a byte. This feature is implemented through bitwise operators that support bitwise operations. These bitwise operators are-

| Operator Symbol | Operation                      |
|-----------------|--------------------------------|
| &               | Bitwise AND                    |
|                 | Bitwise OR                     |
| ^               | Bitwise exclusive OR(XOR)      |
| ~               | Bitwise NOT (One's complement) |
| <<              | Bitwise left shift             |
| >>              | Bitwise right shift            |

Here all the operators are binary, except the complement operator, which is unary. These bitwise operators work on data of integral types only i.e. char, int, short, long including both signed and unsigned types. When these operators are applied to signed types, the result may be implementation dependent, because different implementations represent signed data in different ways, so it is better to use unsigned integer types while doing bit manipulation.

In our discussion, we will take 8-bit bytes and 4-byte words unless otherwise specified i.e. we will take size of an integer to be 32 bits. For portable programs we can use the expression `sizeof(int)*CHAR_BIT` instead of 32. `CHAR_BIT` is defined in file `limits.h` and it denotes the number of bits in a byte, and `sizeof(int)` gives the size of an integer in bytes.

The operands for bitwise operators can be constants or variables. These operators operate on each bit of the operand, so while using these operators we will consider the binary representation (bit pattern) of the operand. For example if we have an integer variable `x=0x1254F`, the bit pattern of this integer will be-

```
00000000 00000001 00100101 01001111
```

We have placed a space after each group of 8 bits for clarity. In the bit pattern, the numbering of bits starts from 0 and they are numbered from right to left.

```

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 1 0 1 0 0 1 1 1 1 1

```

Here the rightmost(0<sup>th</sup>) bit is the least significant bit(low-order bit), while the leftmost(31<sup>st</sup>) bit is the most significant bit(high-order bit). A bit is on(or set) if it has value 1, while it is off(or clear) it has value 0.

We will use a function `displayBits()` in our programs, that will take an integer as input and print the 31 bit binary pattern of that integer. To make the output readable we have a space after each group of 8 bits. The definition of this function is given in program P14.4. In our examples, we'll represent the integers in hexadecimal, because it is easier to convert hexadecimal to binary and vice versa. To enter and display the values in hexadecimal, we can use `%x`.



All bitwise operators except the complement operator can be combined with the assignment operator to form the compound assignment operators-  
&= |= <=> <<= >>=

### 14.1 Logical bitwise operators (&, |, ^, ~)

Bitwise AND, bitwise OR and bitwise XOR are binary operators and require two operands. These operands are compared bitwise i.e. all the corresponding bits in both operands are compared.  
In bitwise AND, the resulting bit is 1 only when the bits in both operands are 1, otherwise it is 0.  
In bitwise OR, the resulting bit is 1 only when the bits in both operands are 1, otherwise it is 1.  
In bitwise XOR, the resulting bit is 1, if bits of both operands have different value, otherwise it is 0.  
The bitwise complement operator is a unary operator and requires only one operand. It flips(inverts) the value of all the bits in the operand, i.e. all 1-bits are changed to 0-bits and all 0-bits are changed to 1-bits.

| Bitwise AND      |                 |                 |                  |
|------------------|-----------------|-----------------|------------------|
| Bit of Resulting | Bit of Operand2 | Bit of Operand1 | Bit of Resulting |
| 0                | 0               | 0               | 0                |
| 0                | 0               | 1               | 0                |
| 0                | 1               | 0               | 0                |
| 0                | 1               | 1               | 0                |
| 1                | 0               | 0               | 0                |
| 1                | 0               | 1               | 0                |
| 1                | 1               | 0               | 0                |
| 1                | 1               | 1               | 1                |

| Bitwise OR       |                 |                 |                  |
|------------------|-----------------|-----------------|------------------|
| Bit of Resulting | Bit of Operand2 | Bit of Operand1 | Bit of Resulting |
| 0                | 0               | 0               | 0                |
| 0                | 0               | 1               | 1                |
| 0                | 1               | 0               | 1                |
| 0                | 1               | 1               | 1                |
| 1                | 0               | 0               | 1                |
| 1                | 0               | 1               | 1                |
| 1                | 1               | 0               | 1                |
| 1                | 1               | 1               | 1                |

| Bitwise XOR      |                 |                 |                  |
|------------------|-----------------|-----------------|------------------|
| Bit of Resulting | Bit of Operand2 | Bit of Operand1 | Bit of Resulting |
| 0                | 0               | 0               | 0                |
| 0                | 0               | 1               | 1                |
| 0                | 1               | 0               | 1                |
| 0                | 1               | 1               | 0                |
| 1                | 0               | 0               | 1                |
| 1                | 0               | 1               | 0                |
| 1                | 1               | 0               | 1                |
| 1                | 1               | 1               | 0                |

Let us take two integer variables a and b. The binary representations of these variables and the results after performing bitwise operations are shown below-

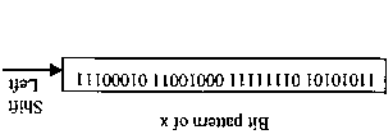
|            |                                   |           |                                     |              |                                     |
|------------|-----------------------------------|-----------|-------------------------------------|--------------|-------------------------------------|
| a=0xd214ad | 0001101 1100010 00010100 10101101 | b=0x124f3 | 00000000 00000001 00100100 11110011 | a&b=0x4a1    | 00000000 00000000 00000100 10100001 |
| a=0xd214ad | 0001101 1100010 00010100 10101101 | b=0x124f3 | 00000000 00000001 00100100 11110011 | a b=0xd34ff  | 0001101 1100011 00110100 11111111   |
| a=0xd214ad | 0001101 1100010 00010100 10101101 | b=0x124f3 | 00000000 00000001 00100100 11110011 | a^b=0xd34ff  | 0001101 1100011 00110100 11111111   |
| a=0xd214ad | 0001101 1100010 00010100 10101101 | b=0x124f3 | 00000000 00000001 00100100 11110011 | a~b=0xd214ad | 0001101 1100010 00010100 10101101   |
| a=0xd214ad | 0001101 1100010 00010100 10101101 | b=0x124f3 | 00000000 00000001 00100100 11110011 | a~b=0xd214ad | 0001101 1100010 00010100 10101101   |
| a=0xd214ad | 0001101 1100010 00010100 10101101 | b=0x124f3 | 00000000 00000001 00100100 11110011 | a~b=0xd214ad | 0001101 1100010 00010100 10101101   |

When the bitwise operators &, |, ^ operate on two operands of different sizes, the size of smaller operand is increased to match the size of larger operand. For example if there are two operands of sizes 16 and 32 bits then the 16-bit operand will be converted to 32 bits. The extra bits are added to the left of the smaller operand. If the smaller operand is unsigned then all these extra bits are filled with zeros, and if it is signed then these bits are filled with the sign bit.

```
int a,b;
printf("Enter values for a and b in hexadecimal: ");
scanf("%x%x",&a,&b);
printf("a=%X\n",a);
displayBits(a);
printf("b=%X\n",b);
displayBits(b);
printf("a&b=%X\n",a&b);
displayBits(a&b);
```

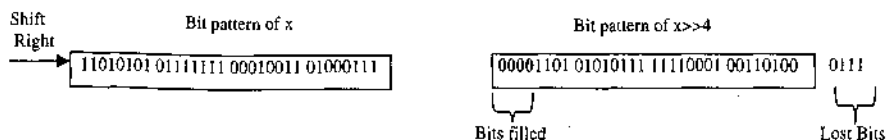
### 14.2 Shift Operators

The order of precedence of these operators from highest to lowest is - bitwise complement, bitwise AND, bitwise XOR, bitwise OR. The complement operator associates from right to left and the other three associate from left to right. The operators == and != have higher precedence than the three binary bitwise operators, so we need to place parentheses if we want bitwise operation to take place before the comparison.  
Bitwise left shift operator(<<) is used for shifting the bits left. It requires two operands; the left operand is the operand whose bits are shifted and the right operand indicates the number of bits to be shifted. On shifting the bits left, an equal number of bit positions on the right are vacated. These positions are filled in with 0 bits. Let us take an integer variable x=0x57F1347 and find x<<4.



When the complement operator is applied to an operand twice, the result is the original operand i.e. -(~a) is equal to a. This feature of complement operator can be used for encrypting and decrypting data. To encrypt the data, we can apply the complement operator to it, and to decrypt the data i.e. to get back the original data we can apply the complement operator to the encrypted data. For example-  
Original data : 00000000 00000000 00001111 01010011  
Encrypted data : 11111111 11111111 1110000 10101100 (By applying ~ to original data)  
Decrypted data : 00000000 00000000 00001111 01010011 (By applying ~ to encrypted data)  
The order of precedence of these operators from highest to lowest is - bitwise complement, bitwise AND, bitwise XOR, bitwise OR. The complement operator associates from right to left and the other three associate from left to right. The operators == and != have higher precedence than the three binary bitwise operators, so we need to place parentheses if we want bitwise operation to take place before the comparison.

```
Output:
Enter values for a and b in hexadecimal: 123FA5 A3C42
a=123FA5
b=A3C42
a^b=23C00
b^a=23C00
00000000 00000001 00111100 01000000
00000000 00010101 00111111 11100111
b=1803B7
a~b=FFDC05A
11111111 1110101 1100000 01011010
b~b=FFFC3BD
11111111 1110101 11000011 10111101
```



Right shift in an unsigned integer

In right shift, if the first operand is a signed integer, some machines perform logical shift while others perform arithmetic shift. In logical shift, the vacated bit positions are always filled with zeros while in arithmetic shift, the vacated bit positions are filled with the sign bit. Sign bit is the leftmost bit (high-order bit) and is 1 for negative integers and 0 for nonnegative integers.

So if the left operand is an unsigned integer or signed nonnegative integer, the vacated positions are filled with zeros. If the left operand is a signed negative integer, the behaviour is undefined and the result is machine dependent. Thus to avoid non portable code, we should use unsigned integers while right shifting.

We know that expressions like  $x/2$  or  $x*5$  do not change the value of  $x$ , similarly  $x \ll 4$  or  $x \gg 3$  will not change the value of  $x$ . If we write  $x = x \ll 4$ , then only the value of  $x$  will be changed.

While using left shift and right shift operators, the result is undefined if the right operand is negative or it is more than or equal to the number of bits used to represent the left operand. Shift operators can be used in generating masks which we will study in section 14.3. Both the shift operators have equal precedence and left to right associativity.

#### 14.2.1 Multiplication and Division by 2 using shift operators

The effect of shifting one bit right is equivalent to integer division by 2, and the effect of shifting one bit left is equivalent to multiplication by 2. So we can use shift operators to multiply and divide integers by power of 2. For example to multiply an integer by  $2^2$ , we'll shift it left by 2 bits, to multiply by  $2^3$  we'll shift it left by 3 bits. Similarly to divide an integer by  $2^2$ , we'll shift it right by 2 bits, to divide by  $2^3$  we'll shift it right by 3 bits. Let us take an unsigned integer  $a$  and see the effect of shifting on it.

| Statement             | Bit Pattern of $a$                  | Decimal value of $a$            |
|-----------------------|-------------------------------------|---------------------------------|
| (i) $a$               | 00000000 00000000 00000000 00101101 | 45                              |
| (ii) $a = a \ll 1$    | 00000000 00000000 00000000 01011010 | $90 (= 45 * 2^1)$               |
| (iii) $a = a \ll 2$   | 00000000 00000000 00000001 01101000 | $360 (= 90 * 2^2)$              |
| (iv) $a = a \ll 5$    | 00000000 00000000 00101101 00000000 | $11520 (= 360 * 2^5)$           |
| (v) $a = a \gg 2$     | 00000000 00000000 00001011 01000000 | $2880 (= 11520 / 2^2)$          |
| (vi) $a = a \gg 7$    | 00000000 00000000 00000000 00010110 | $22 (= 2880 / 2^7)$             |
| (vii) $a = a \ll 10$  | 00000000 00000000 01011000 00000000 | $22528 (= 22 * 2^{10})$         |
| (viii) $a = a \ll 10$ | 00000001 01100000 00000000 00000000 | $23068672 (= 22528 * 2^{10})$   |
| (ix) $a = a \ll 8$    | 01100000 00000000 00000000 00000000 | $1610612736 (= 23068672 * 2^8)$ |

In statement (ix), the effect of multiplication by  $2^8$  is not seen, because a bit with a value of 1 has been dropped from the left. Thus in the case of left shift, if a bit with value of 1 is shifted and lost, then the effect of multiplication by powers of 2 is not seen.

```

/*P14.2*/
#include <stdio.h>
void displayBits(int x);
int main(void)
{
 int a=7;
 printf("a=%d\t", a); displayBits(a);
 a=a<<2;
 printf("a=%d\t", a); displayBits(a);
 a=a<<3;
 printf("a=%d\t", a); displayBits(a);
 a=a>>1;

```

```

printf("a=%d\t", a); displayBits(a);
a=a>>3;
printf("a=%d\t", a); displayBits(a);
return 0;

```

```

input:
7 00000000 00000000 00000000 00000111
28 00000000 00000000 00000000 00011100
224 00000000 00000000 00000000 11100000
112 00000000 00000000 00000000 01110000
14 00000000 00000000 00000000 00001110

```

### 14.3 Masking

Masking is an operation in which we can selectively mask or filter the bits of a variable, such that some bits are changed according to our needs while the others remain unchanged. Through masking, we can manipulate bits in a bit pattern and perform operations such as testing a bit, inverting a bit, switching on or off a bit. Masking is performed with the help of bitwise operators. The bit pattern to be masked is taken as the first operand and the second operand is called mask. The mask is selected according to our requirement.

In further discussion, we will take an arbitrary bit pattern of 32 bits and show different types of masking on it.

$b_{31}b_{30}b_{29}b_{28}b_{27}b_{26}b_{25}b_{24} \quad b_{23}b_{22}b_{21}b_{20}b_{19}b_{18}b_{17}b_{16} \quad b_{15}b_{14}b_{13}b_{12}b_{11}b_{10}b_9b_8 \quad b_7b_6b_5b_4b_3b_2b_1b_0$

Here  $b_{31}, b_{30}, \dots, b_2, b_1, b_0$  are bits and they may be 0 or 1.

#### 14.3.1 Masking Using Bitwise AND

```

b31b30b29b28b27b26b25b24 b23b22b21b20b19b18b17b16 b15b14b13b12b11b10b9b8 b7b6b5b4b3b2b1b0 (Value of x)
0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 (Mask)
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 (New Value)

```

Here we can see that whenever there is a 0 in the mask, new bit value becomes 0 while the new bit value remains unchanged when there is a 1 in the mask.

We can clear any bit using the & operator. For example if we want to clear the last 4 bits in a bit pattern, we can choose the mask such that last four bits are 0, and all other bits are 1.

```

b31b30b29b28b27b26b25b24 b23b22b21b20b19b18b17b16 b15b14b13b12b11b10b9b8 b7b6b5b4b3b2b1b0 (Value of x)
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 (Mask)
b31b30b29b28b27b26b25b24 b23b22b21b20b19b18b17b16 b15b14b13b12b11b10b9b8 b7b6b5b40 0 0 0 0 (New Value)

```

We can use either of these two statements to switch off the last 4 bits in an integer.

```

x = x & 0xFFFFFFF0;
x = x & ~0xF;

```

In first statement, the mask is  $0xFFFFFFF0$  and in second the mask is  $\sim 0xF$ . The second form is portable as it is independent of the word length of the machine. If we take the first form, the mask for 16-bit integer would be  $0xFFFF0$  and for 32-bit integer it would be  $0xFFFFFFF0$ . So the second form is preferable as it makes the code portable.

To clear the 2<sup>nd</sup>, 5<sup>th</sup> and 8<sup>th</sup> bits, the mask would be  $\sim 0x124$  (11111111 11111111 11111110 11011011)

To clear the 5<sup>th</sup> bit, the mask would be  $\sim 0x20$  (11111111 11111111 11111111 11011111)

Bitwise AND operator can be used to test whether a particular bit is set or clear. Suppose we want to test the 5<sup>th</sup> bit, we will select the mask with only 5<sup>th</sup> bit set-

```

b31b30b29b28b27b26b25b24 b23b22b21b20b19b18b17b16 b15b14b13b12b11b10b9b8 b7b6b5b4b3b2b1b0 (Value of x)
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 (Mask)
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 (New Value)

```

If 5<sup>th</sup> bit ( $b_5$ ) in the bit pattern of  $x$  is 0, then the new value will be zero, and if 5<sup>th</sup> bit in original bit pattern is 1, then the new value will be non zero(32).

```

mask=0x20;
if ((x&mask)==0)
 else printf("5th bit is off");
 printf("5th bit is on");

```

So by choosing an appropriate mask and using bitwise AND operator, we can test whether a bit is on or off. We can retrieve bit value by using if...else, conditional operator or shift operator.

```

1. if ((x&mask)==0)
 bit=0;
 else
 bit=1;
2. bit=(x&mask)>0 ? 1 : 0;
3. bit=(x&mask)>>5;

```

In the last case, we have made the extracted fifth bit the rightmost bit by using shift operator.

### 14.3.2 Masking Using Bitwise OR

Whenever there is a 1 in the mask, new bit value becomes 1 while the new bit value remains unchanged when there is a 0 in the mask. We can use bitwise OR operator to set a particular bit or bits. The mask is chosen in such a way that the bits to be set should be 1 and rest of the bits should be 0. Suppose we want to set the 5<sup>th</sup> bit.

|                                                                                                                                 |                                                                                                                                 |                                                                                                                               |                                                                                                                         |              |
|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|--------------|
| b <sub>31</sub> b <sub>30</sub> b <sub>29</sub> b <sub>28</sub> b <sub>27</sub> b <sub>26</sub> b <sub>25</sub> b <sub>24</sub> | b <sub>23</sub> b <sub>22</sub> b <sub>21</sub> b <sub>20</sub> b <sub>19</sub> b <sub>18</sub> b <sub>17</sub> b <sub>16</sub> | b <sub>15</sub> b <sub>14</sub> b <sub>13</sub> b <sub>12</sub> b <sub>11</sub> b <sub>10</sub> b <sub>9</sub> b <sub>8</sub> | b <sub>7</sub> b <sub>6</sub> b <sub>5</sub> b <sub>4</sub> b <sub>3</sub> b <sub>2</sub> b <sub>1</sub> b <sub>0</sub> | (Value of x) |
| 1 1 1 1 1 1 1 1                                                                                                                 | 0 0 0 0 0 0 0 0                                                                                                                 | 1 1 1 1 1 1 1 1                                                                                                               | 0 0 0 0 0 0 0 0                                                                                                         | (Mask)       |
| 1 1 1 1 1 1 1 1                                                                                                                 | 1 1 1 1 1 1 1 1                                                                                                                 | 1 1 1 1 1 1 1 1                                                                                                               | 1 1 1 1 1 1 1 1                                                                                                         | (New Value)  |

### 14.3.3 Masking Using Bitwise XOR

Whenever there is a 1 in the mask the corresponding bit value is inverted while the new bit value remains unchanged when there is a 0 in the mask. We know that complement operator (~) complements all the bits in a number. But if we want to complement some particular bits in a number, we can use bitwise XOR operator. The mask should be chosen in such a way that the bits to be complemented should be 1, rest of the bits should be 0. Now suppose we want to invert the value of 5<sup>th</sup> bit.

|                                                                                                                                 |                                                                                                                                 |                                                                                                                               |                                                                                                                         |              |
|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|--------------|
| b <sub>31</sub> b <sub>30</sub> b <sub>29</sub> b <sub>28</sub> b <sub>27</sub> b <sub>26</sub> b <sub>25</sub> b <sub>24</sub> | b <sub>23</sub> b <sub>22</sub> b <sub>21</sub> b <sub>20</sub> b <sub>19</sub> b <sub>18</sub> b <sub>17</sub> b <sub>16</sub> | b <sub>15</sub> b <sub>14</sub> b <sub>13</sub> b <sub>12</sub> b <sub>11</sub> b <sub>10</sub> b <sub>9</sub> b <sub>8</sub> | b <sub>7</sub> b <sub>6</sub> b <sub>5</sub> b <sub>4</sub> b <sub>3</sub> b <sub>2</sub> b <sub>1</sub> b <sub>0</sub> | (Value of x) |
| 0 0 0 0 0 0 0 0                                                                                                                 | 0 0 0 0 0 0 0 0                                                                                                                 | 0 0 0 0 0 0 0 0                                                                                                               | 0 0 1 0 0 0 0 0                                                                                                         | (Mask)       |
| 0 0 0 0 0 0 0 0                                                                                                                 | 0 0 0 0 0 0 0 0                                                                                                                 | 0 0 0 0 0 0 0 0                                                                                                               | 0 0 1 0 0 0 0 0                                                                                                         | (New Value)  |

Thus the bitwise XOR operator can be used to toggle bits between values 0 and 1. For example if we have an integer variable x, and we want to toggle its 5<sup>th</sup> bit. Suppose initially 5<sup>th</sup> bit in variable x is 1.

(5<sup>th</sup> bit changes to 0, and all other bits unchanged)  
 (5<sup>th</sup> bit changes to 1, and all other bits unchanged)

a^mask; (5<sup>th</sup> bit changes to 0, and all other bits unchanged)

From the above operations we can also see that when an operand is XORed with a value twice, result is same as the original operand. This feature can be used for encryption.

For example, if initial value of a variable var=0xF13A

var=var^0x1F; (Encryption)

var=var^0x1F; (Decryption)

After first statement the value of var becomes 0xF125, while after second statement we get back original value var(0xF13A).

When a value is XORed with itself the result is zero, this is obvious since all corresponding bits will be same. For example value of expressions like var^var will be zero. This feature of xor operator can be utilized to compare two values for equality.

Now let us summarize the different manipulations on 5<sup>th</sup> bit in an integer variable x-

|                                            |            |
|--------------------------------------------|------------|
| Test 5 <sup>th</sup> bit in variable x :   | x&mask     |
| Set 5 <sup>th</sup> bit in variable x :    | x=x mask;  |
| Clear 5 <sup>th</sup> bit in variable x :  | x=x&~mask; |
| Invert 5 <sup>th</sup> bit in variable x : | x=x^mask;  |

Similarly we can manipulate any bit by these operations, we just have to choose the mask appropriately.

|                                                             |            |                                       |
|-------------------------------------------------------------|------------|---------------------------------------|
| For manipulating 6 <sup>th</sup> bit :                      | mask=0x40  | (00000000 00000000 00000000 01000000) |
| For manipulating 9 <sup>th</sup> bit :                      | mask=0x200 | (00000000 00000000 00000010 00000000) |
| For manipulating 3 <sup>rd</sup> and 6 <sup>th</sup> bits : | mask=0x48  | (00000000 00000000 00000000 01001000) |
| For manipulating the low order byte:                        | mask=0xFF  | (00000000 00000000 00000000 11111111) |

We can calculate the mask for a particular bit position by left shifting integer 1 by position of bit.

mask=1<<bitposition;

For manipulating 4<sup>th</sup> bit : mask=1<<4 (00000000 00000000 00000000 00010000)

For manipulating 6<sup>th</sup> bit : mask=1<<6 (00000000 00000000 00000000 01000000)

14.3.4/

```

#include<stdio.h>
void displayBits(int x);
int main(void)

```

```

 int a,bit,mask,bitposition;
 printf("Enter an integer : ");
 scanf("%d",&a);
 printf("Enter the bit position : ");
 scanf("%d",&bitposition);
 mask=1<<bitposition;
 printf("a=%d\n",a);
 displayBits(a);
 if ((a&mask)==0)
 bit=0;
 else
 bit=1;
 printf("The bit at position %d is %d\n",bitposition,bit);
 return 0;

```

If we want to manipulate all bits one by one, we can use a loop, for example the following program will test and print all the bits in an integer variable. This requires testing of each bit in the integer. The bit numbering is from right to left, but we will have to print the bits from left to right so firstly 31<sup>st</sup> bit will be printed, then 30<sup>th</sup> and so on till 0<sup>th</sup> bit. So we will start testing bits from the 31<sup>st</sup> bit onwards. To test 31<sup>st</sup> bit, mask should be 1<<31. Similarly to test 30<sup>th</sup> bit, mask should be 1<<30 and so on.

/\*P14.4 Printing the binary pattern of an integer\*/

```
#include<stdio.h>
void displayBits(int x);
int main(void)
{
 int x;
 printf("Enter an integer : ");
 scanf("%d", &x);
 displayBits(x);
 return 0;
}

void displayBits(int x)
{
 int i, mask;
 for(i=31; i>=0; i--)
 {
 mask=1<<i;
 putchar((x&mask)?'1':'0'); /*Test and print ith bit*/
 if(i%8==0)
 putchar(' '); /*Space after 8 bits*/
 }
 printf("\n");
}
```

In 1<sup>st</sup> iteration mask is : 10000000 00000000 00000000 00000000      31<sup>st</sup> bit is tested and printed  
 In 2<sup>nd</sup> iteration mask is : 01000000 00000000 00000000 00000000      30<sup>th</sup> bit is tested and printed  
 In 3<sup>rd</sup> iteration mask is : 00100000 00000000 00000000 00000000      29<sup>th</sup> bit is tested and printed

In last iteration mask is : 00000000 00000000 00000000 00000001      0<sup>th</sup> bit is tested and printed

To make this program portable, we can modify the for loop as-  
 for(i=sizeof(int)\*CHAR\_BIT-1; i>=0; i--)

Another way of finding masks is to take an array of masks which stores masks for manipulating bit positions.  
 unsigned int arr\_mask[]={ 0x1, 0x2, 0x4, 0x8, 0x10, 0x20, 0x40, 0x80, 0x100, 0x200,  
 0x400, 0x800, 0x1000, 0x2000, 0x4000, 0x8000, 0x10000, 0x20000, 0x40000, 0x80000,  
 0x100000, 0x200000, 0x400000, 0x800000, 0x1000000, 0x2000000, 0x4000000, 0x8000000,  
 0x10000000, 0x20000000, 0x40000000, 0x80000000};

Now the mask for manipulating any bit can be calculated as-  
 mask=arr\_mask[bitposition];  
 For example to switch on the 4<sup>th</sup> bit we can write  
 x=x|arr\_mask[4];

## Some Additional Problems

### Problem 1

Write a program to find whether a number is even or odd, using bitwise operators.

A number will be odd if its least significant bit(rightmost) is 1 and it will be even if its least significant bit is 0. This means we just have to test whether the least significant bit is 1 or 0. We can use bitwise AND operator for this and take a mask in which the rightmost bit is 1 and all other bits are 0.

```
/*P14.5 Program to find whether a number is even or odd*/
#include<stdio.h>
int main(void)
{
 int num;
 int mask=0x1;
 printf("Enter a number : ");
 scanf("%d", &num);
```

```
if((num & mask)==0)
 printf("Number is even\n");
else
 printf("Number is odd\n");
return 0;
```

### Problem 2

How does bit pattern of n change after the statement n=n&(n-1);

Let us first see with the help of some examples what happens to bit pattern of n when 1 is subtracted from it.

```
n=2000 00000000 00000000 00000111 11010000
n=1999 00000000 00000000 00000111 11001111
n=12 00000000 00000000 00000000 00001100
n=11 00000000 00000000 00000000 00001011
n=133 00000000 00000000 00000000 10000101
n=132 00000000 00000000 00000000 10000100
```

We can see that when one is subtracted from a number, all the bits upto and including the rightmost 1-bit starting from the leftmost bit are inverted, and other bits remain unchanged. In the first example, 4<sup>th</sup> bit is the rightmost 1-bit, so 0<sup>th</sup>, 1<sup>st</sup>, 2<sup>nd</sup>, 3<sup>rd</sup>, 4<sup>th</sup> bits are inverted. If we take bitwise AND of n and (n-1) then we get a number whose rightmost 1 bit is cleared and other bits remain unchanged. So the statement n=n&(n-1); clears the rightmost 1 bit of an integer.

```
n 00000000 00000000 00000111 11010000 (Rightmost 1 bit is 4th bit)
n-1 00000000 00000000 00000111 11001111
n&(n-1) 00000000 00000000 00000111 11000000 (4th bit cleared)
```

/\*P14.6 Clear rightmost 1 bit\*/

```
#include<stdio.h>
void displayBits(int x);
int main(void)
{
 unsigned n=140;
 displayBits(n);
 n=n&(n-1);
 displayBits(n);
 return 0;
```

Output:  
 00000000 00000000 00000000 10001100  
 00000000 00000000 00000000 10001000

### Problem 3

Write a program to find whether a number is an exact power of 2.

If a number is exact power of 2, it will have only one 1-bit in its binary pattern. For example-

```
16 (00000000 00000000 00000000 00010000)
256 (00000000 00000000 00000001 00000000)
```

If a number is not a power of 2, it will definitely have more than one 1-bits.

In the previous problem we have seen that the expression n&(n-1) gives us a number which is same as the original except that the rightmost set bit is cleared. In a number which is power of 2, there is only one set bit which is also the rightmost bit. This rightmost 1-bit will be cleared and since all other bits are also 0, the value of expression n&(n-1) will be zero.

Thus if the value of expression n&(n-1) is zero, it means that n is a power of 2.

```

n=1024(power of 2)
n 00000000 00000000 00000100 00000000
n-1 00000000 00000000 00000011 11111111
n & (n-1) 00000000 00000000 00000000 00000000

```

```

/*P14.7*/
#include<stdio.h>
int main(void)
{
 unsigned n;
 printf("Enter n : ");
 scanf("%d",&n);
 if((n&(n-1))==0) /*parentheses around (n&(n-1)) are necessary*/
 printf("%d is power of 2\n",n);
 else
 printf("%d is not power of 2\n",n);
 return 0;
}

```

This program considers 0 as a power of 2. The condition can be modified so that 0 is not considered a power of 2.

```

if(n && !(n&(n-1)))
 printf("%d is power of 2\n",n);
else
 printf("%d is not power of 2\n",n);

```

#### Problem 4

Write a program to count the number of set bits in an integer.

We can test each bit by using & operator and appropriate masks, and count the 1-bits.

```

int count_setbits1(unsigned x)
{
 int count=0,mask,i;
 for(i=0; i<32; i++)
 {
 mask=1<<i;
 if((x&mask)!=0) /*Check ith bit*/
 count++;
 }
 return count;
}

```

This loop will always iterate 32 times. We can reduce the number of iterations if there are leading 0-bits in the number.

```

int count_setbits2(unsigned x)
{
 int count=0;
 while(x!=0)
 {
 if((x&1)!=0) /*IF lowest order bit is 1*/
 count++;
 x>>=1; /*Shift x right*/
 }
 return count;
}

```

We keep shifting the value right so that each bit becomes LSB and is checked. The mask 0x1 is used to test the LSB. We can stop when value becomes 0 because there are no more 1-bits to count. So this function is more efficient than previous one because in previous one, the iterations were equal to the number of bits, but here the

iterations stop as soon as we get the last 1-bit(leftmost 1-bit). (This function should be used with unsigned integers only to avoid undefined behaviour while right shifting)

We can further reduce the number of iterations by making use of the expression  $x = x \& (x-1)$ .

```

int count_setbits3(unsigned x)
{
 int count=0;
 while(x!=0)
 {
 count++;
 x=x&(x-1); /*Clear the rightmost 1-bit*/
 }
 return count;
}

```

Here we keep on clearing the rightmost 1-bit till all the 1-bits are cleared. In this method number of iterations is equal to the number of 1-bits.

The last method that we give is very efficient and uses a lookuptable. We can create a lookuptable (array) whose each entry denotes the number of set bits in the corresponding index number. For example if we create an array of size 16, then the lookuptable would be-

|   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |
|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|
| 0 | 1 | 1 | 2 | 1 | 2 | 2 | 3 | 1 | 2 | 2  | 3  | 2  | 3  | 3  | 4  |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |

In 15(1111) there are four 1-bits so there is 4 at 15<sup>th</sup> place, In 14(1110) there are three 1-bits so there is 3 at 14<sup>th</sup> place.

Now we can count set bits in groups of 4.

```

/*P14.8 Count Set bits*/
#include<stdio.h>
void displayBits(int x);
int main(void)
{
 unsigned x=1124;
 displayBits(x);
 printf("%d \n",count_setbits4(x));
 return 0;
}

int table[]={0,1,1,2,1,2,2,3,1,2,2,3,2,3,3,4};
int count_setbits4(unsigned x)
{
 int count=0;
 while(x!=0)
 {
 count=count+table[x&0xF];
 x=x>>4;
 }
 return count;
}

```

If we want to process bits in group of 8 we will need a lookup table of size 256 that stores the count of 1-bits for each number from 1 to 255.

```

int table[]
{0,1,1,2,2,3,1,2,2,3,2,3,3,4,1,2,2,3,2,3,3,4,2,3,3,4,3,4,4,5,1,2,2,3,2,3,3,4,2,3,3,4,3,4,
4,5,2,3,3,4,3,4,4,5,3,4,4,5,4,5,5,6,1,2,2,3,2,3,3,4,2,3,3,4,3,4,4,5,3,4,4,5,
5,5,6,2,3,3,4,3,4,4,5,3,4,4,5,4,5,5,6,3,4,4,5,4,5,5,6,4,5,5,6,5,6,6,7,1,2,2,3,2,3,3,4,2,3,3,
4,3,4,4,5,2,3,3,4,3,4,4,5,3,4,4,5,4,5,5,6,2,3,3,4,3,4,4,5,3,4,4,5,4,5,5,6,3,4,4,5,4,5,5,6,4,
5,5,6,5,6,6,7,2,3,3,4,3,4,4,5,3,4,4,5,4,5,5,6,3,4,4,5,4,5,5,6,4,5,5,6,5,6,6,7,3,4,4,5,4,5,5,6,
4,5,5,6,5,6,6,7,4,4,5,5,5,6,6,7,5,6,6,7,6,7,7,8};

```

In this case the while loop would be-

```
while(x!=0)

count=count+table{x&0xFF};
x=x>>8;
```

### Problem 5

Write a program to manipulate  $n$  bits starting at position  $p$  in a number  $x$ . If  $p=7$  and  $n=4$  then the bits to be manipulated are 7<sup>th</sup>, 6<sup>th</sup>, 5<sup>th</sup> and 4<sup>th</sup> bits.

We will need to create a mask in which all the bit positions to be manipulated are 1 and others are 0. If  $p=7$  and  $n=4$  then the mask should be 00000000 00000000 00000000 11110000. This mask is given by  $(\sim 0 < n) < p - n + 1$ .

```
0 11111111 11111111 11111111
0<n 11111111 11111111 11110000
(~0<n) 00000000 00000000 00000000 00001111
(~0<n)<p-n+1 00000000 00000000 00000000 11110000
```

\*P14.9 To set, clear, invert  $n$  bits starting from position  $p$  in a number  $x$ \*/

```
include<stdio.h>
void displayBits(int x);
int main(void)

int mask,p,n,x;
p=7,n=4;
mask=~(0<n)<p-n+1;
x=10345;
displayBits(x);
x=x|mask; /*Set*/
displayBits(x);

x=10345;
displayBits(x);
x=x&~mask; /*Clear*/
displayBits(x);

x=10345;
displayBits(x);
x=x^mask; /*Invert*/
displayBits(x);
return 0;
```

### Problem 6

Given two numbers  $x$  and  $y$ , write a function that returns  $x$  such that the  $n$  bits starting at position  $p$  in  $x$  are replaced by the  $n$  bits starting at position  $p$  of  $y$ .

In the previous problem, the mask is  $\sim(0 < n) < p - n + 1$ .

Suppose  $p=7$ ,  $n=4$ ,  $x=0x23173b4$ ,  $y=0x557$ ;

Clear the 7<sup>th</sup>, 6<sup>th</sup>, 5<sup>th</sup> and 4<sup>th</sup> bits of  $x$ .

```
x 00000010 00110001 01110011 10110100
~mask 11111111 11111111 11111111 00001111
x & ~mask 00000010 00110001 01110011 00000100
```

Extract the 7<sup>th</sup>, 6<sup>th</sup>, 5<sup>th</sup> and 4<sup>th</sup> bits of  $y$

```
y 00000000 00000000 00000101 01010111
mask 00000000 00000000 00000000 11110000
y & mask 00000000 00000000 00000000 01010000
```

Place the 7<sup>th</sup>, 6<sup>th</sup>, 5<sup>th</sup> and 4<sup>th</sup> bits of  $y$  in the corresponding positions of  $x$

```
x & ~mask | y & mask 00000010 00110001 01110011 01010100
```

\*P14.10\*/

```
include<stdio.h>
void displayBits(int x);
signed func(unsigned x, unsigned y, int p, int n);
int main(void)

int p=7,n=4;
unsigned x=0x23173b4, y=0x557;
displayBits(x);
displayBits(y);
x=func(x,y,p,n);
displayBits(x);
return 0;

signed func(unsigned x, unsigned y, int p, int n)

unsigned mask=~(0<n)<p-n+1;
return (x&~mask)|(y&mask);
```

Output:

```
00000010 00110001 01110011 10110100
00000000 00000000 00000101 01010111
00000010 00110001 01110011 01010100
```

### Problem 7

Modify the program in previous problem so that now in  $x$  the  $n$  bits starting at position  $p$  are replaced by the rightmost  $n$  bits of  $y$ .

Suppose  $p=7$ ,  $n=4$ ,  $x=0x23173b4$ ,  $y=0x557$ ;

Clear the 7<sup>th</sup>, 6<sup>th</sup>, 5<sup>th</sup> and 4<sup>th</sup> bits of  $x$ .

```
x 00000010 00110001 01110011 10110100
~mask 11111111 11111111 11111111 00001111
x & ~mask 00000010 00110001 01110011 00000100
```

Right shift  $y$  so that rightmost 4 digits come at 7<sup>th</sup>, 6<sup>th</sup>, 5<sup>th</sup> and 4<sup>th</sup> positions

```
y 00000000 00000000 00000101 01010111
y<<p-n+1 00000000 00000000 01010101 01110000
```

Extract the 7<sup>th</sup>, 6<sup>th</sup>, 5<sup>th</sup> and 4<sup>th</sup> bits of  $y$

```
mask 00000000 00000000 00000000 11110000
y<<p-n+1 & mask 00000000 00000000 00000000 01110000
```

Place the 7<sup>th</sup>, 6<sup>th</sup>, 5<sup>th</sup> and 4<sup>th</sup> bits of  $y$  in the corresponding positions of  $x$

```
(x & ~mask) | (y<<p-n+1 & mask) 00000010 00110001 01110011 01110100
```

\*P14.11\*/

```
include<stdio.h>
void displayBits(int x);
signed func(unsigned x, unsigned y, int p, int n);
int main(void)

int p=7,n=4;
unsigned x=0x23173b4, y=0x557;
displayBits(x);
```

```

displayBits(y);
x=func(x,y,p,n);
displayBits(x);
return 0;
}

unsigned func(unsigned x,unsigned y,int p,int n)
{
 unsigned mask=~(~0<<n)<<p-n+1;
 return (x&~mask)|(y<<p-n+1 & mask);
}

Output:
00000010 00110001 01110011 10110100
00000000 00000000 00000101 01010111
00000010 00110001 01110011 10110100

```

### Problem 8

Given two numbers, write a function to insert a number y in number x. The number y should occupy n bits in x starting at position p. Assume that number y can fit in these n bits.

Suppose p=13, n=6, x=0x23173b4, y=0x2F

```

x 00000010 00110001 01110011 10110100
~mask 11111111 11111111 11000000 11111111
x & ~mask 00000010 00110001 01000000 10110100

y 00000000 00000000 00000000 00101111
y<<p-n+1 00000000 00000000 00101111 00000000

(x&~mask)|(y<<p-n+1) 00000010 00110001 01101111 10110100

```

```

/*P4.12*/
#include<stdio.h>
void displayBits(int x);
unsigned func(unsigned x,unsigned y,int p,int n);
int main(void)
{
 int p=13,n=6;
 unsigned x=0x23173b4,y=0x2F;
 displayBits(x);
 displayBits(y);
 x=func(x,y,p,n);
 displayBits(x);
 return 0;
}

unsigned func(unsigned x,unsigned y,int p,int n)
{
 unsigned mask=~(~0<<n)<<p-n+1;
 return (x&~mask)|(y<<p-n+1);
}

```

### Problem 9

Write a program to manipulate bits from position i to j in a number x. If i=2 and j=7 then the bits to be manipulated are 2<sup>nd</sup>, 3<sup>rd</sup>, 4<sup>th</sup>, 5<sup>th</sup>, 6<sup>th</sup>, 7<sup>th</sup>.

The mask for manipulating these bits is given by  $\sim(\sim 0 < j-i+1) < i$ .

If i=2, j=7, the mask will be 00000000 00000000 00000000 11111100

```

~0 11111111 11111111 11111111 11111111
~0<<j-i+1 11111111 11111111 11111111 11000000
~(~0<<j-i+1) 00000000 00000000 00000000 00111111
~(~0<<j-i+1)<i 00000000 00000000 00000000 11111100

```

```

/*P14.13*/
#include<stdio.h>
void displayBits(int x);
int main(void)
{
 int mask,i=2,j=7,x;

 mask=~(~0<<j-i+1)<i;

 x=103145;
 displayBits(x);
 x=x|mask; /*Set*/
 displayBits(x);

 x=103145;
 displayBits(x);
 x=x&~mask; /*Clear*/
 displayBits(x);

 x=103145;
 displayBits(x);
 x=x^mask; /*Invert*/
 displayBits(x);
 return 0;
}

```

### Problem 10

Write a program to swap the values of two variables using bitwise XOR.

```

/*P14.14 Swapping values without using a temporary variable through bitwise XOR*/
#include<stdio.h>
int main(void)
{
 int x,y;
 printf("Enter values for x and y : ");
 scanf("%d%d",&x,&y);
 printf("x=%d,y=%d\n",x,y);
 x=x^y;
 y=x^y;
 x=x^y;
 printf("x=%d,y=%d\n",x,y);
 return 0;
}

```

### Problem 11

Write a program to print the bit pattern of 2's complement of a number.

We can find out the 2's complement by adding 1 to the one's complement. Another way to obtain the 2's complement is to scan the bit pattern from right to left, and invert all the bits after the first appearance of a bit with value 1. For example-

Bit pattern : 00000000 00000000 00010001 0110 0000  
 2's complement : 11111111 11111111 11101110 1010 0000

/\*P14.15 Program to print the two's complement of a number\*/

```
#include<stdio.h>
void displayBits(int x);
int main(void)
{
 int num,i,mask;
 printf("Enter a number : ");
 scanf("%d",&num);
 displayBits(num);
 printf("Two's complement is : \n");
 displayBits(~num+1);

 for(i=0; i<=31; i++)
 {
 mask=1<<i;
 if((num&mask)!=0) /*Find a bit with value 1*/
 break;
 }
 for(i=i+1; i<=31; i++)
 {
 mask=1<<i;
 num=num^mask; /*Invert the bit*/
 }
 printf("Two's complement is : \n");
 displayBits(num);
 return 0;
}
```

Output:  
 Enter a number : 68  
 00000000 00000000 00000000 01000100  
 Two's complement is :  
 11111111 11111111 11111111 10111100  
 Two's complement is :  
 11111111 11111111 11111111 10111100

### Problem 12

Write a function nextHighestPow2() that takes an integer n as argument and returns the smallest power of two that is greater than or equal to n.

nextHighestPow2(100) returns 128  
 nextHighestPow2(20) returns 32  
 nextHighestPow2(256) returns 256

If the bit position of leftmost 1-bit is i, then the next highest power is given by  $1 \ll (i+1)$ . For example bit pattern of 100 is 00000000 00000000 00000000 01100100. In this pattern, the position of leftmost 1-bit is 6. So the next power of 2 is given by  $1 \ll 7$  which is 00000000 00000000 00000000 10000000 (128).

```
/*P14.16*/
#include<stdio.h>
unsigned nextHighestPow2(unsigned int n);
void displayBits(int x);
int main(void)
{
 unsigned n=600;
 printf("%d\n", (n));
 displayBits(n);
 displayBits(nextHighestPow2(n));
 printf("%d\n", nextHighestPow2(n));
}
```

return 0;

signed nextHighestPow2(unsigned int n)

```
int i;
if(n&&!(n&(n-1))) /*if n is exact power of 2, see P14.7*/
 return n;
i=-1;
while(n!=0)
{
 n>>=1;
 i++;
}
return 1<<i+1;
```

### Problem 13

left or right shifting, the bits that fall off the end are lost. Write a function for implementing circular right shift in which the bits that fall off the right end are placed in the left i.e. the bits are rotated. The first argument should be the number whose bits will be rotated and the second argument should denote number of rotations. Write a similar function for circular left shift.

111111 00100101 11110110 11110101  $\xrightarrow{\text{Rotate Right}}$  10101011 11110001 00101111 10110111  
 111111 00100101 11110110 11110101  $\xrightarrow{\text{Rotate Left}}$  11100100 10111110 11011110 10101111

P14.17\*/

```
#include<stdio.h>
void displayBits(int x);
rotate_right1(int num, int n);
rotate_right2(int num, int n);
rotate_left1(int num, int n);
rotate_left2(int num, int n);
main(void)
```

```
int num,n,i,bit;
printf("Enter the number to be rotated : ");
scanf("%d", &num);
printf("Enter the number of rotations : ");
scanf("%d", &n);
n%=32;
displayBits(num);
printf("\n");
displayBits(rotate_right1(num,n));
displayBits(rotate_right2(num,n));
displayBits(rotate_left1(num,n));
displayBits(rotate_left2(num,n));
return 0;
```

rotate\_right1(int num,int n)

```
int i;
for(i=1; i<=n; i++)
{
 if(num&1) /*Test LSB*/
 num=(num>>1) | (1<<31); /*Switch on MSB*/
 else
 num=(num>>1) & ~(1<<31); /*Switch off MSB*/
}
return num;
```



```

int rotate_left1(int num,int n)
{
 int i;
 for(i=1; i<=n; i++)
 {
 if(num&(1<<31)) /*Test MSB*/
 num=(num<<1) | 1; /*Switch on LSB*/
 else
 num=(num<<1) & ~1; /*Switch off LSB*/
 }
 return num;
}

int rotate_right2(int num,int n)
{
 return (num>>n) |(num<<(32-n));
}

int rotate_left2(int num,int n)
{
 return (num<<n) |(num>>(32-n));
}

```

#### Problem 14

Write a function `mult(x,n,err)` that uses the shift operator `<<` to compute the value of  $(x * 2^n)$ . Assume that both `x` and `n` are unsigned positive integers and are 32 bits in size. The 'err' parameter is set by the function to zero if calculation is successful and 1 if an overflow occurs. An overflow occurs when the resultant value doesn't fit in a 32 bit unsigned integer.

```

/*P14.18*/
#include<stdio.h>
unsigned mult(unsigned int num,unsigned int n,int *err);
int main(void)
{
 unsigned int num,n,err=0,result;

 printf("Enter the number and power of 2 : ");
 scanf("%u%u",&num,&n);

 result=mult(num,n,&err);
 if(err==1)
 printf("Overflow\n");
 else
 printf("Result=%u\n",result);
 return 0;
}

unsigned mult(unsigned int num,unsigned int n,int *err)
{
 unsigned int result=num;
 while(n>0)
 {
 num=num<<1; /*Multiply by 2*/
 n--;
 if(num<result) /*unsigned integer overflow wraps around*/
 {
 *err=1;
 break;
 }
 result=num;
 }
 return result;
}

```

#### Problem 15

Write a function that reverses the bits in an integer.

```

/*P14.19*/
#include<stdio.h>
void displayBits(int x);
unsigned int reverse(unsigned int num);
int main(void)
{
 unsigned int x=234242156;
 displayBits(x);
 displayBits(reverse(x));
 return 0;
}

unsigned int reverse(unsigned int num)
{
 unsigned int i,rev=0;
 for(i=0; i<32; i++)
 if((num&(1<<i))) /*test ith bit of num*/
 rev=rev|(1<<(31-i)); /*set (31-i)th bit of rev equal to ith bit of num*/
 return rev;
}

```

Some other ways of reversing bits are given in Exercise.

#### Problem 16

Write a program that inputs a binary pattern less than or equal to 32 bits and converts it to an integer.

```

/*P14.20 Program to convert a binary pattern to an integer*/
#include<stdio.h>
int main(void)
{
 char bit;
 int i, num=0;

 printf("Enter any bit pattern less than or equal to 32 bits :\n");
 for(i=0; i<=31; i++)
 {
 bit=getchar();
 if(bit=='0')
 num=num<<1;
 else if(bit=='1')
 num=(num<<1)+1;
 else
 break;
 }
 printf("Hexadecimal : %x\n", num);
 printf("Decimal : %d\n", num);
 return 0;
}

```

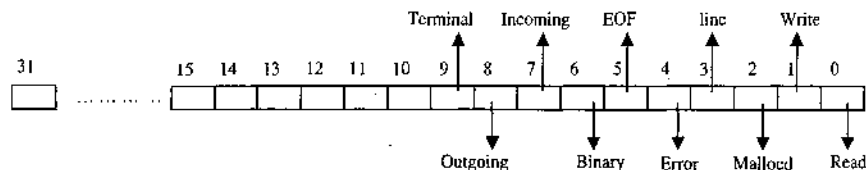
Now we know very well how to manipulate individual bits within a byte. This bit manipulation is useful in maintaining boolean flags. For example we can use a character variable to hold 8 flags or an integer variable to hold 32 flags. Each bit can be used to represent a flag. This will definitely save memory when many flags are to be used in a program and also it is easy to maintain a single variable for several flags. We can test, switch on or off, toggle any individual flag by bitwise operators. Different mask is taken for manipulating each flag. To increase readability, generally each mask corresponding to a flag is given a name using `#define`. Let us take an example and understand how this is done. We'll declare a variable of unsigned integer type, and then we'll use the individual bits to represent different flags associated with the attributes of a file. The variable is declared as-

```

unsigned int flags;

```

Now we'll use 10 bits of this variable to represent 10 different flags.



To increase readability, we'll give a name to each mask corresponding to these bits.

```
#define _F_RDWR 0x3 /* Read/write flag 0th and 1st bit */
#define _F_READ 0x1 /* Read only file 0th bit */
#define _F_WRITE 0x2 /* Write only file 1st bit */
#define _F_BUF 0x4 /* Malloced Buffer data 2nd bit */
#define _F_LBUF 0x8 /* line-buffered file 3rd bit */
#define _F_ERR 0x10 /* Error indicator 4th bit */
#define _F_EOF 0x20 /* EOF indicator 5th bit */
#define _F_BIN 0x40 /* Binary file indicator 6th bit */
#define _F_IN 0x80 /* Data is incoming 7th bit */
#define _F_OUT 0x100 /* Data is outgoing 8th bit */
#define _F_TERM 0x200 /* File is a terminal 9th bit */
```

Now we can perform masking operations to manipulate the different flags like this-

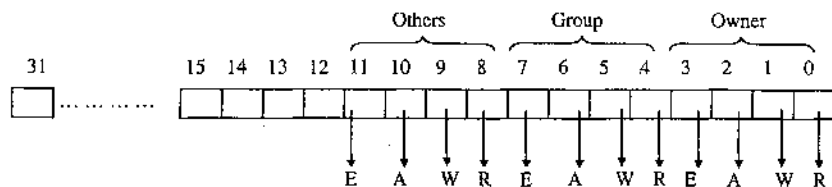
```
/*To switch on error flag*/
flags=flags|_F_ERR;

/*To switch off write flag*/
flags=flags&~_F_WRITE;

/*To toggle incoming flag*/
flags=flags^_F_IN;

/*To test EOF flag*/
if((flags & _F_EOF)==0)
 printf("EOF flag is not set");
else
 printf("EOF flag is set");
```

Now we'll take another example in which a variable named permissions will contain information about the different access permissions given to different class of users. There are three types of users viz. owner, group, others, and there are four types of permissions that can be given to these users viz. read, write, append, execute.



The masks corresponding to these bits can be named as-

```
#define R_OWNER 0x1 /* 0th bit */
#define W_OWNER 0x2 /* 1st bit */
#define A_OWNER 0x4 /* 2nd bit */
#define E_OWNER 0x8 /* 3rd bit */
#define R_GROUP 0x10 /* 4th bit */
#define W_GROUP 0x20 /* 5th bit */
#define A_GROUP 0x40 /* 6th bit */
#define E_GROUP 0x80 /* 7th bit */
```

```
#define R_OTHERS 0x100 /* 8th bit */
#define W_OTHERS 0x200 /* 9th bit */
#define A_OTHERS 0x400 /* 10th bit */
#define E_OTHERS 0x800 /* 11th bit */
signed int permission;
permission=0x19F;
```

After this statement owners will get all permissions, group will get read and execute permission, and others will get only read permission.

```
/*To grant execute permission to others we can write-
permission=permission|E_OTHERS; /*Switch on 11th bit*/
/*To take away read permission from group we can write-
permission=permission&~R_GROUP; /*Switch off 4th bit*/
```

## 4.4 Bit Fields

We have seen how to access and manipulate individual bits or group of bits using bitwise operators. Bit fields provide an alternative method of accessing bits. A bit field is a collection of adjacent bits. It is defined inside a structure but is different from other members because its size is specified in terms of bits. The data type of a bit field can be int, signed int or unsigned int.

Let us take an example that shows the syntax of defining bit fields.

```
struct tag
{
 unsigned a : 2;
 unsigned b : 5;
 unsigned c : 1;
 unsigned d : 3;
};
```

Here the structure has four bit fields a, b, c and d. The sizes of a, b, c and d are 2, 5, 1 and 3 bits respectively. The bit fields can be accessed like other members of structure using the dot operator. Whenever they appear inside expressions, they are treated like integers (of smaller range). These are some valid expressions using bit fields.

```
struct tag var;
var.a=2;
printf("%d",var.b);
var.a+=var.b;
if(var.c==1)
 printf("Flag is on\n");
```

If the size of a bit field is n, then the range of values that the bit field can take is from 0 to  $2^n-1$ .

| Bit field | Size in bits | Range of values        |
|-----------|--------------|------------------------|
|           | 2            | 0 to $2^2-1$ (0 to 3)  |
|           | 5            | 0 to $2^5-1$ (0 to 31) |
|           | 1            | 0 to $2^1-1$ (0 and 1) |
|           | 3            | 0 to $2^3-1$ (0 to 7)  |

It would be invalid to assign any value to a bitfield outside its range, for example-

```
var.b=54; /*Invalid*/
```

We can't apply sizeof and address operators to bit fields. So we can't use scanf() to input values in a bit field.

```
scanf("%d",&var.a); /*Invalid*/
```

We may input the value into a temporary variable and then assign it to the bit field.

```
scanf("%d",&temp);
var.a=temp;
```

If we have a pointer to structure then arrow operator( $\rightarrow$ ) can be used to access the bit fields. Code using bitfields is easier to understand than the equivalent masking operations, but bitfields are considered non-portable as most of the issues related with them are implementation dependent.

We had mentioned that the data type of bit fields can be `int`, `signed int` or `unsigned int`. A plain `int` field may be treated as signed by some compilers while as unsigned by others. So for portability, it is better to clearly specify signed or unsigned in the declaration of bit fields. If a bitfield is defined as signed, then it should be at least 2 bits long because one bit is used for sign.

The direction of allocation of bit fields within an integer is also implementation dependent. Some C compilers allocate the bit fields from right to left, while others may allocate them from left to right. If the bit fields are assigned from right to left, then the first field occupies the least significant bit. If the bit fields are assigned from left to right, then the first field occupies the most significant bit.

The other implementation dependent issue about bit fields is that whether they can cross integer boundaries or not. For example consider this structure-

```
struct tag
{
 unsigned a : 10;
 unsigned b : 7;
 unsigned c : 17;
};
```

The first two fields occupy only 17 bits in a 32 bit integer, so 15 bits can still be used for another bit field. But the bits needed for next bit field is more than 15. Some implementations may start the next field(c) from a new integer, while others may just place the next field in 17 adjacent bits i.e. 15 unused from the previous integer and 2 bits from the next integer.

We can define unnamed bitfields for controlling the alignment of bitfields within an integer. The size of unnamed bitfield provides padding within the integer.

```
struct tag{
 unsigned a : 10;
 unsigned b : 7;
 unsigned : 15; /*padding within first integer*/
 unsigned c : 17;
};
```

Here the unnamed bitfield fills out the 15 unused bits of first integer, so the bitfield `c` starts with the second integer. Since these 15 bits don't have any name, they can't be accessed from the program.

We can also take the size of the unnamed bitfield zero, so we have no need to provide the padding and next bitfield will start with second integer.

```
struct tag
{
 unsigned a : 10;
 unsigned b : 7;
 unsigned : 0;
 unsigned c : 17; /*this field starts from next integer*/
};
```

Now we'll take the example of file permissions that we had seen earlier using bitwise operators.

```
struct permission
{
 unsigned r_owner : 1;
 unsigned w_owner : 1;
 unsigned a_owner : 1;
 unsigned e_owner : 1;
 unsigned r_group : 1;
 unsigned w_group : 1;
 unsigned a_group : 1;
 unsigned e_group : 1;
 unsigned r_others : 1;
```

```
unsigned w_others : 1;
unsigned a_others : 1;
unsigned e_others : 1;
```

```
struct permissions perm={1,1,1,1,1,0,0,1,1,0,0,0};
```

After this statement owners will get all permissions, group will get read and execute permission, and others will get only read permission.

Now we can write statements to grant and take permissions like this-

```
perm.e_others=1; /*grant execute permission to others*/
perm.r_group=0; /*take away read permission from group*/
```

## Exercise

What is the output of the following programs; assume `stdio.h` is included in all programs.

```
int main(void)
{
 int x=7,y=19;
 printf("%d %d ",x&y,x&y);
 printf("%d %d ",x|y,x|y);
 printf("%d\n",x^y);
 return 0;
}
```

```
int main(void)
{
 unsigned int x,y,z;
 displayBits(0xFFFF);
 x=y=z=0xFFFF;
 x=(x>>5)<<5; displayBits(x);
 y=(y>>3)<<3; displayBits(y);
 z=(z>>2)<<2; displayBits(z);
 return 0;
}
```

```
int main(void)
{
 int k;
 k=((3<<4)^(96>>1));
 printf("%d\n",k);
 return 0;
}
```

```
int main(void)
{
 int x=0x1F;
 x<<2;
 printf("%X ",x);
 x>>2;
 printf("%X\n",x);
 return 0;
}
```

```
int main(void)
{
 unsigned int arr_mask[]={ 0x1, 0x2, 0x4, 0x8, 0x10, 0x20, 0x40, 0x80, 0x100, 0x200,
 0x400, 0x800, 0x1000, 0x2000, 0x4000, 0x8000, 0x10000, 0x20000, 0x40000, 0x80000,
 0x100000, 0x200000, 0x400000, 0x800000, 0x1000000, 0x2000000, 0x4000000, 0x8000000,
 0x10000000, 0x20000000, 0x40000000, 0x80000000};
```

```

int i,x=0x54038F;
for(i=31; i>=0; i--)
 x & arr_mask[i] ? putchar('1') : putchar('0');
return 0;
}

```

```

6. int main(void)
{
 unsigned int num=0xA01D,pos=3,bit;
 unsigned int mask=1<<pos;
 bit=(num&mask) >> pos;
 printf("%u\n", bit);
 return 0;
}

```

```

7. int main(void)
{
 unsigned int num=0x1F,pos=3,bit;
 bit=(num>>pos) & 1;
 printf("%u\n", bit);
 return 0;
}

```

```

8. int main(void)
{
 int i,num=0xA0DF;
 for(i=31; i>=0; i--)
 printf("%d", (num>>i)&1);
 return 0;
}

```

```

9. int main(void)
{
 int i,num=0x1A3B;
 unsigned int mask=1<<31;
 for(i=31; i>=0; i--)
 {
 (num&mask) ? printf("1") : printf("0");
 mask=mask>>1;
 }
 return 0;
}

```

```

10. void func(int x);
int main(void)
{
 func(0x1AE3);
}
void func(int x)
{
 int i,mask;
 mask=1<<31;
 for(i=1; i<=32; i++)
 {
 putchar((x&mask)?'1':'0');
 x<<=1;
 if(i%8==0)
 putchar(' ');
 }
 printf("\n");
 return 0;
}

```

```

11. int func(unsigned int x);
int main(void)
{
 printf("%d\n", func(0x1AE3));
 return 0;
}
int func(unsigned x)
{
 int count=0,mask=1,i;
 for(i=0; i<32; i++)
 {
 if((x&mask)!=0) /*Check ith bit*/
 count++;
 mask<<=1;
 }
 return count;
}

```

```

12. unsigned int func(unsigned int x);
int main(void)
{
 unsigned int x=0x1AE3;
 displayBits(x);
 x=func(x);
 printf("X\n",x);
 displayBits(x);
 return 0;
}
unsigned int func(unsigned int num)
{
 unsigned int i,r=0;
 for(i=0; num!=0; i++)
 {
 r=(r<<1)|num&1;
 num>>=1;
 }
 r<<=32-i;
 return r;
}

```

```

13. unsigned int func(unsigned int x);
int main(void)
{
 unsigned int x=0x1AE3;
 displayBits(x);
 x=func(x);
 printf("X\n",x);
 displayBits(x);
 return 0;
}
unsigned int func(unsigned int num)
{
 unsigned int lmask,rmask,mask;
 lmask=1<<31;
 rmask=1;
 while(lmask>rmask)
 {
 mask=lmask|rmask;
 if((num&mask)!=0 && (num&mask)!=mask)
 num^=mask;
 lmask>>=1;
 rmask<<=1;
 }
 return num;
}

```

14. Write one line statements for performing the following on an unsigned integer x.

- (i) Set most significant bit
- (ii) Clear most significant bit
- (iii) Invert all bits
- (iv) Set all bits
- (v) Invert least significant byte
- (vi) Invert all bits at even positions 0,2,4,6,8,....
- (vii) Invert all bits at odd positions 1,3,5,7,9,....
- (viii) Clear all bits at even positions 0,2,4,6,8,....
- (ix) Clear all bits at odd positions 1,3,5,7,9,....
- (x) Insert 3 trailing zeros
- (xi) Find if every bit is set
- (xii) Multiply x by 7 using bitwise operators
- (xiii) Multiply x by 9 using bitwise operators
- (xiv) Multiply x by 3.5 using bitwise operators

15. Write a function to compute word length of machine, i.e. number of bits in an integer.

16. Write a program to find remainder when a number x is divided by a number y, where y is power of 2. Don't use % operator, use bitwise operators.

17. Write a function isMultiple() using bitwise operators, that takes two arguments x and i and returns 1 if x is a multiple of 2 to the power of i, and 0 otherwise.

18. Write a program to find the number of bits that need to be changed to convert a number a to another number b.

19. What do the following loops count.

```
count=0;
for(z=x^y; z!=0; z=z&(z-1))
 count++;
count=0;
for(z=x^y; z!=0; z>>=1)
 count+=z&1;
```

20. Write a program to clear n bits starting at position p in a number x. If n is negative then n bits to the left of n should be cleared, otherwise n bits to the right should be cleared. If p=7 and n=4 then the bits to be cleared are 7<sup>th</sup>, 6<sup>th</sup>, 5<sup>th</sup> and 4<sup>th</sup> bits. If p=7 and n=-4 then the bits to be cleared are 7<sup>th</sup>, 8<sup>th</sup>, 9<sup>th</sup>, 10<sup>th</sup> bits.

21. Write a function that takes integer arguments x, p and n, and returns x with n bits starting at position p (towards right) placed at the positions 0 to n-1, and all other bits are cleared.

Example: x=103145, p=7, n=5  
 00000000 00000001 10010010 11101001  
 00000000 00000000 00000000 00011101

22. Write a function to merge bits from two numbers and store the result in another number. Given a bit position i, all bits in resultant number from 0 to i should be equal to corresponding bits of x, and from i+1 to last bit should be equal to corresponding bits of y.

Example: i=9  
 x - 00000000 00000000 00000001 00100011  
 y - 00000000 11111111 11111111 11111111  
 r - 00000000 11111111 11111101 00100011

23. Which bits will be cleared by the following function clear().

```
unsigned clear(unsigned x, int i, int j)
{
 unsigned mask = -0 << (j+1) | (1 << i) - 1;
 return x & mask;
}
```

24. Which bits will be set by the following function set().

```
unsigned set(unsigned int x, int i, int j)
{
 int mask = 0;
 for(p=i; p<j; p++)
 mask = mask | 1 << p;
```

```
return x | mask;
```

25. How can we find whether x is of the form  $2^n - 1$  (15, 31, 63, ....).

26. Write functions increment() and decrement() to add and subtract 1 from an integer using bitwise operators.

27. What will be the output of this program on a 2's complement machine.

```
int main(void)
{
 int x=245;
 x = -(-x);
 printf("%d\t", x);
 x = -(-x);
 printf("%d\n", x);
}
```

28. Write a program to isolate the rightmost (lowest order) 1 bit i.e. clear all bits except the rightmost 1 bit.

```
00000010 00100000 00001011 11010000 -> 00000000 00000000 00000000 00010000
00000100 00000101 01101011 11001001 -> 00000000 00000000 00000000 00000001
```

29. Write a function to find log base 2 of a number that is a power of 2. What will your function return if input is not a power of 2.

30. A number has odd parity if it has odd number of set bits, and even parity if it has even number of set bits. Write a function to find parity of an unsigned integer, the function should return 1 for odd parity and 0 for even parity.

31. Write Russian peasant multiply method given in Chapter 5, without using /, \*, % operators.

32. What do the following functions count.

```
int count1(unsigned x)
```

```
{
 int count = 0;
 while(x != 0)
 {
 count++;
 x = x & (x - 1);
 }
 return count;
}
```

```
int count2(int x)
```

```
{
 return 32 - count1(x);
}
```

```
int count3(int x)
```

```
{
 return count1(-x);
}
```

```
int count4(int x)
```

```
{
 int c = 0;
 for(x = ~x; x != 0; x = x & (x - 1))
 c++;
 return c;
}
```

33. The following program sorts an array and uses a function swap() that swaps values of two integers.

```
#include <stdio.h>
void swap(int *a, int *b);
int main(void)
{
 int x=2, y=2;
 int arr[5] = {1, 2, 5, 3, 4}, i, j, n=5, min;
 for(i=0; i<n-1; i++)
 {
 min=i;
 for(j=i+1; j<n; j++)
 {
 if(arr[min] > arr[j])
 min=j;
 }
 swap(&arr[i], &arr[min]);
 }
```

```

 }
 printf("Sorted list is : \n");
 for(i=0; i<n; i++)
 printf("%d ", arr[i]);
 printf("\n");
 return 0;
}

void swap(int *a, int *b)
{
 int temp;
 temp=*a;
 *a=*b;
 *b=temp;
}

```

Will there be any change in the output of the above program if swap1 () is used instead of swap ().

```

void swap1(int *x, int *y)
{
 *x = *x ^ *y;
 *y = *x ^ *y;
 *x = *x ^ *y;
}

```

34. Write a function to right-propagate the rightmost 1-bit, i.e. set all the bits after the rightmost 1 bit.  
00000000 00000001 11100010 01000000 -> 00000000 00000001 11100010 01111111
35. Write a function to right propagate the leftmost 1-bit i.e. set all the bits after the leftmost 1-bit.  
00000000 00001000 00101001 00010000 -> 00000000 00001111 11111111 11111111
36. When we right propagate the leftmost 1-bit of a number x the resulting number that we get is 1 less than the next highest power of x. So to get the next highest power of x we can right propagate the leftmost 1 bit and add 1 to it. This will give power of two that is greater than n. Write a function nextHighestPow2 () by propagating the leftmost 1-bit.
37. Write a function to count number of leading zeros in the bit pattern of an integer.
38. Write a function to set the rightmost 0-bit.
39. Write a program to swap adjacent bits of an unsigned integer i.e. bits present at even positions should be swapped with bits present at odd positions.
40. Write a program to swap adjacent groups of 4 bits of an unsigned integer.  
00000001 01011111 10010011 10100111  
00010000 11110101 00111001 01111010
41. Write a program to swap adjacent bytes of an unsigned integer.
42. Write a program to reverse bytes of a 32-bit unsigned integer.  
1111010 00010010 11001101 00000100 -> 00000100 11001101 00010010 11110101
43. What is the output of this program.

```

int main(void)
{
 unsigned x=0xFA2E4;
 displayBits(x);
 displayBits(func(x));
 return 0;
}

```

unsigned func(int x)

```

x= ((x&0x55555555)<<1) | ((x&0xaaaaaaaa)>>1);
x= ((x&0x33333333)<<2) | ((x&0xcccccccc)>>2);
x= ((x&0x0f0f0f0f)<<4) | ((x&0xf0f0f0f0)>>4);
x= ((x&0x00ff00ff)<<8) | ((x&0xffff0000)>>8);
x= ((x&0x0000ffff)<<16) | ((x&0xffff0000)>>16);
return x;
}

```

44. What is the output of this program.

```

int main(void)

```

```

{
 unsigned x=12345678;
}

```

```

displayBits(x);
displayBits(func(x));
printf("%d\n", func(x));
return 0;
}

```

unsigned func(unsigned x)

```

x= (x&0x55555555) + ((x&0xaaaaaaaa)>>1);
x= (x&0x33333333) + ((x&0xcccccccc)>>2);
x= (x&0x0f0f0f0f) + ((x&0xf0f0f0f0)>>4);
x= (x&0x00ff00ff) + ((x&0xffff0000)>>8);
x= (x&0x0000ffff) + ((x&0xffff0000)>>16);
return x;
}

```

45. Write a program for packing and unpacking this data in a 32 bit integer.

```

Empid - 1 to 4000
Obid - 1 to 100
Job status - T(0) or P(1)
Gender - F(0) or M(1)
Age - 20 to 65
Marital status - S(0), M(1), D(2), W(3)

```

46. In Binary Coded Decimal(BCD) representation, a decimal digit is represented by 4 bits. For example-

BCD representation of 12345678 is - 00010010 00110100 01010110 01111000

Binary representation of 12345678 is - 00000000 10111100 01100001 01001110

In BCD, we can represent only those numbers which have 8-digits or less in a 32 bit integer.

Write a program for converting BCD to binary representation and vice versa.

47. Write a program for packing and unpacking 4 characters in a 32 bit integer.

## Miscellaneous Features

## 15.1 Enumeration

Sometimes the replacement of integer constants by some meaningful and descriptive names, enhances the readability of the code and makes it self documenting. For example suppose we are making a date related program then it would be better if we could use names like Jan, Feb, Mar, Apr instead of the numbers 1, 2, 3, 4.

An enumeration type is a user defined data type, which can take values only from a user defined list of named integer constants called enumerators. The syntax of defining an enumeration data type is same as that of structure or union. The general format of definition is-

```
enum tag
{
 member1;
 member2;

};
```

Here enum is a keyword, tag is an identifier that specifies the name of the new enumeration type being defined, and member1, member2 are identifiers which represent integer constants and are called enumerator constants or enumerators. The list of these enumerators is called enumerator list. Note that the members inside the braces are **not** variables, they are named integer constants.

After the definition, we can declare variables of this new data type as-

```
enum tag var1, var2, var3;
```

Here var1, var2, var3 are variables of type enum tag and they can take values from the enumerator list. The variables can also be declared with the definition as-

```
enum tag
{
 member1;
 member2;

} var1, var2, var3;
```

Here the tag is optional. Let us take an example -

```
enum month{Jan, Feb, Mar, Apr, May, Jun};
```

Here a new data type month is defined and the enumerator list contains six enumerators.

Internally the compiler treats these enumerators as integer constants. These are automatically assigned integer values beginning from 0, 1, 2... till the last member of the enumeration. In the above example, these enumerators will take following values-

```
Jan-0, Feb-1, Mar-2, Apr-3, May-4, Jun-5
```

These are the default values assigned to the enumerators. It is also possible to explicitly assign any value to the enumerators but in this case, the successive unassigned enumerators will take values one greater than the value of the previous enumerator. For example-

## Miscellaneous Features

```
enum month{Jan, Feb=4, Mar, Apr, May=11, Jun};
```

Now the enumerators will take following values-  
Jan-0, Feb-4, Mar-5, Apr-6, May-11, Jun-12

We can assign any signed integer value to enumerators, provided the value is within the range of int. It is also possible to assign same value to more than one enumerator.

```
enum month{Jan, January=0, Feb=1, February=1, Mar=2, March=2};
```

The enumerated variables can be processed like other integer variables. We can assign values to them from the enumerator list or they can be compared to other variables and values of the same type. For example-

```
enum month{Jan, Feb, Mar, Apr, May, Jun }m1,m2;
m1=Mar;
m2=May;
```

Now m1 has integer value 2 and m2 will take the value 4.

Any variable of type enum month can take values only from the 6 enumerators specified in the list. For example this is invalid-

```
m1=Dec; /*Invalid*/
```

Some other examples of enum data type definitions are-

```
enum suit{Spades, Hearts, Clubs, Diamonds};
enum position{Ace=1, King, Queen, Jack, Ten, Nine, Eight, Seven, Six, Five, Three, Two};
enum month{Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec};
enum day{Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday};
enum color{white, black, red, green, blue, yellow, pink, brown};
enum boolean{true, false};
enum switch{off, on};
enum subject{Hindi, English, Maths, Physics, Chemistry, Biology, History};
enum base{Binary=2, Octal=8, Decimal=10, Hexadecimal=16};
```

Here are some examples of code using these enum definitions-

```
enum color walls, floor;
if(walls==pink)
 floor=blue;
else
 floor=white;
```

```
enum day today;
day=Monday;
if(today==Sunday)
 printf("Holiday\n");
else if(today==Saturday)
 printf("Half working day\n");
else
 printf("Full working Day\n");
```

```
enum subject s;
int passmarks;
switch(s)
{
 case hindi:
 case english:
 passmarks=25;
 break;
 case maths:
 passmarks=40;
 break;
 case physics:
 case chemistry:
 passmarks= 35;
 break;
 default:
```

```
passmarks=33;
```

Suppose we need a function that returns number of days in a month. It can be written as-

```
enum month m;
if(m==Apr || m==Jun || m==Sep || m==Nov)
 return 30;
if(m==Jan || m==Mar || m==May || m==Jul || m==Aug || m==Oct || m==Dec)
 return 31;
if(m==Feb)
 return 28;
```

*\*P15.1 Program to print the value of enum variables\*/*  

```
#include<stdio.h>
int main(void)
```

```
enum month{Jan, Feb, Mar, Apr, May, Jun}m1, m2;
m1=Mar;
printf("m1=%d\n", m1);
printf("Enter value for m2 : ");
scanf("%d", &m2);
printf("m2=%d\n", m2);
return 0;
```

Output:  
1) 2  
Enter value for m2 : 5  
2) 5

It is not possible to perform input and output in terms of enumerator names. The input and output is only in the form of their integer values. You may be tempted to use %s to output and input enumerated variables but this is invalid since enumerators are not strings.

```
printf("%s", m1); /*Invalid*/
```

Since enumerators are identifiers, their names should be different from other identifiers in the same scope. The following code is wrong because the identifier chemistry has been used at two different places.

```
enum group1{physics, chemistry, maths};
enum group2{zoology, botany, chemistry}; /*Invalid*/
float maths; /*Invalid*/
```

We can use typedef in the definition of enum, for example-

```
typedef enum{false, true} boolean;
```

Now we can define variables like this-

```
boolean flag=true;
```

The definition of an enumeration type may not include a tag. For example-

```
enum {false, true, off=1, on, stop=0, start}
```

This type of enum definition which does not have a tag can be written if we don't want to declare any variables of enum type but just want to define constants. This is a better option than defining a long list of constants using #define.

Let us compare the constants defined by #define preprocessor directive and enum.

- i) By defining an enum, we define a new type whose variables can be declared but using #define we can only give name to some constant values.
- ii) enum is a part of C language(32 keywords) but #define is not.
- iii) All #define directives are global, while enum obeys the scope rules.
- iv) In #define we have to explicitly assign the values to all constants, while in enum the values are assigned by the compiler automatically.

We can always write our programs by using integer variables instead of enumerated variables, but use of enum in complicated programs makes the program more understandable.

## 2 const

Any variable is declared with const qualifier, then the value of that variable can't be changed by the program. The const qualifier can occur in the declaration before or after the data type. For example-

```
const int x=9;
int const x=9;
```

Both these declarations are equivalent and declare x as a const variable, the first one is used more common. Any attempt to change the value of this variable in the program will result in an error. For example these statements are invalid-

```
x=10; /*Invalid*/
func(); /*Invalid*/
x++; /*Invalid*/
```

A const variable can be given a value only through initialization or by some hardware devices. Note that the value of a const variable can't be modified by the program, but any external event outside the program can change its value.

If an array, structure or union is declared as const then each member in it becomes constant. For example-

```
const int arr[5]={10,11,12,13,14};
const struct {char x; int y; float z;}var={'A',12,29.5};
arr[2]=22; /*Invalid*/
var.x='B'; /*Invalid*/
```

Const variables are not true compile time constants in C, so we can't use const variables where constant expressions are required e.g. in array dimensions or case labels. For example the following code is invalid in C.

```
const int size=10;
char arr[size];
```

The const qualifier can be useful while passing array arguments to functions. We know that when an array is passed as an argument, the function gets access to the original array and can modify it. If we don't want the function to make any changes in the array then we can declare the array as const in the formal parameter list.

```
void func(const char arr[])
```

Now we will see how to use const in pointer declarations. We can declare three types of pointers using the qualifier const.

1) Pointer to const data

2) const pointer

3) const pointer to const data

Consider these declarations-

```
const int a=2, b=6;
const int *p1=&a; /*or int const *p1=&a;*/
```

Here p1 is declared as a pointer to const integer. We can change the pointer p1 but we can't change the variable pointed to by p1.

```
p1=9; /*Invalid*/
p1=&b; /*Valid since p1 is not a constant itself*/
```

Now consider these declarations-

```
const int a=2, b=6;
const *p2=&a;
```

Here p2 is declared as a const pointer. We can't change the pointer variable p2, but we can change the variable pointed to by p2.



```
*p2=9; /*Valid*/
p2=&b; /*Invalid since p2 is a constant*/
```

Now consider these declarations-

```
const int a=2, b=6;
const int const *p3=&a;
```

Here p3 is declared as a const pointer to const integer. We can neither change the pointer variable p3 nor the variable pointed to by it.

```
*p3=9; /*Invalid*/
p3=&b; /*Invalid*/
```

So the three different types of declarations of pointers using const are-

```
const int *ptr; /*pointer to const integer*/
int *const ptr; /*const pointer to an integer*/
const int *const ptr; /*const pointer to a const integer*/
```

The following example will further clarify this concept.

```
char str1[]="wealthy";
char str2[]="strong";

const char *pc=str1; /*pointer to const*/
char *const cp=str1; /*const pointer*/
const char *const cpc=str1; /*const pointer to const*/
```

```
pc=str2; /*Valid*/
cp=str2; /*Invalid*/
cpc=str2; /*Invalid*/
```

```
*pc='h'; /*Invalid*/
*cp='h'; /*Valid*/
*cpc='h'; /*Invalid*/
```

If we want to assign the address of a const variable to a non const pointer, then we'll have to use cast operator.

For example-

```
const int a=10;
int *ptr;
ptr=(int *)&a;
```

Now if we try to change the value of variable a indirectly through ptr then the result is undefined and there can be a run time error.

```
*ptr=12; /*undefined*/
```

## 15.3 volatile

Whenever a variable is encountered in the program, compiler reads the value of that variable from the memory. But sometimes for optimization purposes the compiler stores the value of the currently used variable in an unused register. Now when that variable is encountered next time in the program and the compiler sees that the program has not changed value of the variable, it reads the value from the register instead of the memory. This process saves time since accessing a register is faster than accessing memory.

This sort of automatic optimization by the compiler may sometimes lead to incorrect results. This generally happens when the value of a variable can be modified by some external process outside the program. These types of situations arise when we have memory mapped I/O, variables shared among multiple processes or variables that can be modified by interrupt routines.

For example suppose we have a variable time that represents current system time and its value is controlled by the system clock. Now consider this loop-

```
while(time!= T)
; /*Do nothing till value of variable time equals T*/
```

The optimizing phase of the compiler may observe that the value of variable time is not changing inside the loop, so it may decide to access its value once from the memory and then cache this value in a register. Now for

each iteration of the loop, compiler reads this cached value from the register and hence the loop will never terminate, but this is not what we wanted. It is our responsibility to inform the compiler that the value of a particular variable may change through some external process also.

To solve these types of problems we can use volatile qualifier. If a variable is declared with the qualifier volatile, then we are instructing the compiler to turn off the optimization process for that variable i.e. we are telling the compiler to read the value of that variable from memory only, each time it is encountered in the program. The optimization phase of compiler will never try to cache the value of a volatile variable in a register.

If we use the qualifier const along with the qualifier volatile, then the value of volatile variable can't be changed from inside the program. For example-

```
const volatile int x;
```

If an array, structure or union is declared as volatile then each member becomes volatile.

## 15.4 lvalue and rvalue

Sometimes we get compiler errors that use the terms lvalue and rvalue. An expression that represents a memory location and can occur on the left hand side of assignment operator is known as lvalue and an expression that represents a value and can occur on right hand side of assignment operator is known as rvalue. For example consider the following statement-

```
y=10;
```

Here y=10 is an rvalue as it represents a value and x is an lvalue as it represents a memory location where the value can be stored.

Some examples of lvalues are-

```
var (var is an integer variable)
ptr (ptr is a pointer variable)
ptr (ptr is a pointer variable)
emp (emp is a structure variable)
emp->salary (here emp is a structure variable and salary is its member)
arr[2] (arr is a 1-d array)
```

All these expressions can occur both on the left side and right side of assignment operator i.e. we can use their values in the program and their value can be modified.

Variables declared with const type qualifier or array types are lvalues as they have a memory location but they can't be modified. A modifiable lvalue is an lvalue that is not declared with keyword const or array type and thus can occur as left operand of assignment operator or as an operand of increment or decrement operator.

Some examples of rvalues are-

```
monday*z (here monday is an enum constant)
x (here x is an integer variable)
```

The value of these expressions can be used in the program, but they can't be modified, we can't write them on the left side of assignment operator. We will get error 'lvalue required' if we try to write statements like this-

```
x;
y*z=12;
emp;
```

## 15.5 Scope

Scope of an identifier is a region of program text where the identifier is visible i.e. region where it can be used.

The scope of an identifier is determined by the location where it is declared. There are four kinds of scopes - block scope, function scope, file scope and prototype scope.

### 5.5.1 Block Scope

The identifiers declared at the beginning of a block or in the list of parameter declarations in a function definition have block scope. These identifiers are visible in the whole block and their scope terminates at the end of the block. Block scope variables are also known as local variables.

### 5.5.2 File Scope

The identifiers declared outside any function have file scope. An identifier which has file scope is visible in all functions from its point of declaration to the end of source file in which it is declared. Global variables and function definitions have file scope. Function prototypes which are placed outside any function also have file scope.

### 5.5.3 Prototype Scope

As we have studied in chapter 6, a function prototype is a declaration of function that declares its return type and types of its parameters. It is optional to write the parameter names in the prototype, but if they are written these names have prototype scope and this scope is only limited to the prototype, i.e. it terminates with the end of the prototype. These names can be used anywhere else in the program.

### 5.5.4 Function Scope

The only type of identifier that has function scope is the label name. Label is an identifier that is followed by a colon, and is used in goto statements and switch statements. Function scope means that a label can be used anywhere in the function in which it appears, and so all label names in a function should be unique.

If the scopes overlap, the entity declared in the outer scope is not visible in the inner scope. In the case of nested blocks, if there is an identifier which is declared both in inner block and outer block, then the identifier of the inner block is hidden in the inner block. If there is an identifier which is declared both outside all blocks and inside a block, then inside the block the identifier that was declared outside is hidden.

Let us take an example for understanding the different scopes.

```
int x,y;
float func1(int num1, int num2);
int func2(int z);
int main(void)
{
 int p,q,r;

}

float func1(int n1, int n2);
{
 int a;

 {
 int a,b;

 }
 xyz:

 {
 int x;

 }
}

int func2(int z)
{

}
```

The variables `x` and `y` declared at line 1 and function names `func1()` and `func2()` have file scope. The identifiers `num1`, `num2` and `z` declared as parameter names in lines 2 and 3 have prototype scope. The variables `p,q,r` declared at line 6, `a` declared at line 12, `a,b` declared at line 16, `x` declared at line 22 have block scope.

The parameter names `n1` and `n2` declared at line 10 and `z` at line 26 also have block scope. The label name `xyz` declared at line 19 has function scope.

The variable `a` declared at line 12 can't be used in lines 15-18, since there is another variable with the same name in this block. Similarly the variable `x` declared at line 1 is hidden in the block 21-24.

## 15.6 Storage Classes

In addition to data type, each variable has one more attribute known as storage class. The proper use of storage classes makes our program efficient and fast. In larger multifile programs the knowledge of storage classes is indispensable. We can specify a storage class while declaring a variable. The general syntax is-

```
storage_class datatype variable_name;
```

There are four types of storage classes-

1. Automatic
2. External
3. Static
4. Register

The keywords `auto`, `static`, `register`, `extern` are used for these storage classes. So we may write declaration statements like this-

```
auto int x,y;
static float d;
register int z;
```

When the storage class specifier is not present in the declaration, compiler assumes a default storage class based on the place of declaration.

A storage class decides about these four aspects of a variable-

- (1) Lifetime - Time between the creation and destruction of a variable.
- (2) Scope - Locations where the variable is available for use.
- (3) Initial value - Default value taken by an uninitialized variable.
- (4) Place of storage - Place in memory where the storage is allocated for the variable (static memory, runtime stack or registers).

Now we'll discuss all these storage classes one by one in detail.

### 15.6.1 Automatic

All the variables declared inside a block/function without any storage class specifier are automatic variables. We may also use the keyword `auto` to declare automatic variables, although this is generally not done. The following two declaration statements are equivalent and both declare `a` and `b` to be automatic variables of type `int`.

```
void func(void)
```

```
 int a,b;
```

```
void func(void)
```

```
 auto int a,b;
```

The uninitialized automatic variables initially contain garbage value. The scope of these variables is inside the function or block in which they are declared and they can't be used in any other function/block. They are named automatic since storage for them is reserved automatically each time when the control enters the function/block and are released automatically when the function/block terminates. All automatic variables are stored on the runtime stack.

/\*P15.2 Automatic variables\*/

```
#include<stdio.h>
void func(void);
int main(void)
{
 func();
 func();
 func();
 return 0;
}
void func(void)
{
 int x=2,y=5;
 printf("x=%d,y=%d\n",x,y);
 x++;
 y++;
}
```

Output:

```
x=2, y=5
x=2, y=5
x=2, y=5
```

Here when the function `func()` is called first time, the variables `x` and `y` are created and initialized, and when the control returns to `main()`, these variables are destroyed. When the function `func()` is called for the second time, again these variables are created and initialized, and are destroyed after execution of the function. So automatic variables come into existence each time the function is executed and are destroyed when the function terminates.

Since automatic variables are known inside a function or block only, we can have variables of same name in different functions or blocks without any conflict. For example in the following program the variable `x` is used in different blocks (here blocks consist of function body) without any conflict.

/\*P15.3\*/

```
#include<stdio.h>
void func(void);
int main(void)
{
 int x=5;
 printf("x=%d\n",x);
 func();
 return 0;
}
void func(void)
{
 int x=15;
 printf("x=%d\n",x);
}
```

Output:

```
x=5 x=15
```

Here the variable `x` declared inside `main()` is different from the variable `x` declared inside the function `func()`. In the next program, there are different blocks inside `main()`, and the variable `x` declared in different blocks is different.

/\*P15.4\*/

```
#include<stdio.h>
```

```
int main(void)
{
 int x=3;
 printf("%d\n",x);
 {
 int x=10;
 printf("%d\n",x);
 }
 {
 int x=26;
 printf("%d\n",x);
 }
 printf("%d\n",x);
 return 0;
}
```

Output:

```
3
10
26
```

## 15.6.2 Register

Register storage class can be applied only to automatic variables. The scope, lifetime and initial value of register variables are same as that of automatic variables. The only difference between the two is in the place where they are stored. Automatic variables are stored in memory while register variables are stored in CPU registers. Registers are small storage units present in the processor. The variables stored in registers can be accessed much faster than the variables stored in memory, so the variables that are frequently used can be assigned register storage class for faster processing. For example the variables used as loop counters may be declared as register variables since they are frequently used.

/\*P15.5\*/

```
#include<stdio.h>
int main(void)
{
 register int i;
 for(i=0; i<20000; i++)
 printf("%d\n",i);
 return 0;
}
```

Register variables don't have memory addresses so we can't apply address operator(&) to them. There are limited number of registers in the processor hence we can declare only few variables as register. If many variables are declared as register and the CPU registers are not available then compiler will treat them as automatic variables.

We can specify register storage class only for `int`, `char` or pointer types. If a variable other than these types is declared as register variable then compiler treats it as an automatic variable. The register storage class specifier can be applied to formal arguments of a function while the other three storage class specifiers can't be used in this way.

## 15.6.3 External

The variables that have to be used by many functions and in different files can be declared as external variables. The initial value of an uninitialized external variable is zero.

Before studying external variables, let us first understand the difference between their definition and declaration. The declaration of an external variable declares the type and name of the variable, while the definition reserves storage for the variable as well as behaves as a declaration. The keyword `extern` is specified in declaration but not in definition. For example the definition of an external variable `salary` will be written as-

```
float salary;
```

Its declaration will be written as-

```
ern float salary;
```

following points will clarify the concept of definition and declaration of an external variable.

Definition of an external variable-

Definition creates the variable, so memory is allocated at the time of definition.

There can be only one definition.

The variable can be initialized with the definition and initializer should be constant.

The keyword `extern` is not specified in the definition. Some compilers may allow the use of `extern` but it is better to omit this keyword in the definition.

The definition can be written only outside functions.

Declaration of an external variable-

Declaration does not create a variable; it only refers to a variable that has already been created somewhere.

Memory is not allocated at the time of declaration.

There can be many declarations.

The variable cannot be initialized at the time of declaration.

The keyword `extern` is always specified in the declaration.

The declaration can be placed inside functions also.

Consider this program-

```
#include <stdio.h>
int x=8;
main(void)
```

```
.....
func1(void)
```

```
.....
func2(void)
```

In this program the variable `x` will be available to all functions, since an external variable is active from the start of its definition till the end of a program. Now if we change the place of definition of variable `x` like this-

```
#include <stdio.h>
main(void)
```

```
.....
func1(void)
```

```
.....
int x=8;
func2(void)
```

Now `x` is defined after `main()` and `func1()`, so it can't be used by these functions, it is available only to `func2()`. Suppose we want to use this variable in function `main()` then we can place a declaration in `main()` like this-

```
#include <stdio.h>
main(void)
```

```
.....
extern int x;
```

```
.....
void func1(void)
```

```
.....
int x=8;
void func2(void)
```

Now `x` will be available to functions `func2()` and `main()`.

Till now we had written our program in a single file. When the program is large it is written in different files and these files are compiled separately and linked together afterwards to form an executable program. Now we'll consider a multifile program which is written in three files viz. `first.c`, `second.c` and `third.c`.

```
first.c
int x=8;
int main(void)
{
.....
}
void func1()
{
.....
}
```

```
second.c
void func2(void)
{
.....
}
void func3(void)
{
.....
}
```

```
third.c
void func4(void)
{
.....
}
void func5(void)
{
.....
}
```

Here in the file `first.c`, an external variable `x` is defined and initialized. This variable can be used both in `main()` and `func1()` but it is not accessible to other files. Suppose file `second.c` wants to access this variable then we can put the declaration in this file as-

```
first.c
int x=8;
int main(void)
{
.....
}
void func1(void)
{
.....
}
```

```
second.c
extern int x;
void func2(void)
{
.....
}
void func3(void)
{
.....
}
```

```
third.c
void func4(void)
{
.....
}
void func5(void)
{
.....
}
```

Now this variable can be accessed and modified in files `first.c` and `second.c` and any changes made to it will be visible in both files. If this variable is needed by only one function in the file `second.c`, then the `extern` declaration can be put inside that function. So the declaration of an external variable using `extern` keyword is used to extend the scope of that variable.

Suppose our program consists of many files and in file `first.c`, we have defined many variables that may be needed by other files also. We can put an `extern` declaration for each variable in every file that needs it. Another better and practical approach is to collect all `extern` declarations in a header file and include that header file in the files, which require access to those variables.

### 15.6.4 Static

There are two types of static variables-

(1) Local static variables

## (2) Global static variables

## 15.6.4.1 Local Static Variables

When the storage specifier `static` is applied to variables declared inside blocks, their storage class changes from automatic to static. The scope of a local static variable is same as that of an automatic variable i.e. it can be used only inside the function or block in which it is defined. The lifetime of a static variable is more than that of an automatic variable. A static variable is created at the compilation time and it remains alive till the end of a program. It is not created and destroyed each time the control enters a function/block. Thus a static variable is created only once and its value is retained between function calls. If it has been initialized, then the initialization value is placed in it only once at the time of creation. It is not initialized each time the function is called.

A static variable can be initialized only by constants or constant expressions. Unlike automatic variables, we can't use the values of previously initialized variables to initialize static variables. If a static variable is not explicitly initialized then by default it takes initial value zero.

```
int x=8;
int y=x; /*Valid*/
static int z=x; /*Invalid, initializer should be constant*/
```

Let us take a program to understand the use of local static variables. This program is similar to P15.2 but here `x` and `y` are declared as static variables.

*/\*P15.6 Program to understand the use of local static variables\*/*

```
#include<stdio.h>
void func(void);
int main(void)
{
 func();
 func();
 func();
 return 0;
}
void func(void)
{
 static int x=2,y=5;
 printf("x=%d, y=%d\n",x,y);
 x++;
 y++;
}
```

Output:

```
x=2, y=5
x=3, y=6
x=4, y=7
```

Note that the effect of initialization is seen only in the first call. In subsequent calls initialization is not performed and variables `x` and `y` contain values left by the previous function call. The next program also illustrates the use of local static variables.

*/\*P15.7 Program to understand the use of local static variable\*/*

```
#include<stdio.h>
void func(int n);
int main(void)
{
 int n=1;
 printf("Enter a number : ");
 scanf("%d",&n);
 for(i=1; i<=10; i++)
 func(n);
 printf("\n");
 return 0;
}
void func(int n)
```

## Miscellaneous Features

```
static int step; /*Automatically initialized to 0*/
step=step+n;
printf("%d\t",step);
```

Output :

Enter the number : 4

```
4 8 12 16 20 24 28 32 36 40
```

The formal parameters in a function definition can't be made static.

## 15.6.4.2 Global Static Variables

If a local variable is declared as static then it remains alive throughout the program. In the case of global variables, the static specifier is not used to extend the lifetime since global variables already have a lifetime equal to the life of program. Here the static specifier is used for information hiding. If an external variable is defined as static, then it can't be used by other files of the program. So we can make an external variable private to a file by making it static.

first.c

```
int x=8;
static int y=10;
int main(void)
{

}
void func1(void)
{

}
```

second.c

```
extern int x;
void func2(void)
{

}
void func3(void)
{

}
```

third.c

```
void func4(void)
{

}
void func5(void)
{

}
```

Here the variable `y` is defined as a static external variable, so it can be used only in the file `first.c`. We can't use it in other files by putting an extern declaration for it.

## 15.7 Storage Class Specifiers in Functions

The storage class specifiers `extern` and `static` can be used with function definitions. The definition of a function without any storage specifier is equivalent to its definition with the keyword `extern` i.e. by default the definition of a function is considered external. If a function is external then it can be used by all the files of the program and if it is static then it can be used only in the file where it is defined.

If an external function is to be used in another file, then that file should contain function declaration and it is a good practice to specify `extern` in that declaration.

first.c

```
int main(void)
{

}
float func1(int x)
{

}
```

second.c

```
extern float func1(int);
func2(void)
{

}
static int func3(int x)
{

}
```

third.c

```
void func4(void)
{

}
void func5(void)
{

}
```

Here the function `func1()` is defined in file `first.c`. Its declaration is put in file `second.c`, so it can be used in this file also. The function `func3()` in file `second.c` is defined as static so it can't be used by any other file. Generally declarations of all functions are collected in a header file and that header file is included in other files.

## 15.8 Linkage

In multifile programs, if an identifier is present in more than one source file, then its linkage determines whether the identifier refers to the same entity. There are three types of linkages in C-

- (a) External linkage
- (b) Internal linkage
- (c) No linkage

The linkage of an identifier is determined by the place of declaration and storage specifier. Local variables, function parameters, typedef names, label names, structure tags have no linkage, so their scope is only within the block where they are declared. Global variables and functions have external linkage, so they can be used in any file of the program. Static global variables and static functions have internal linkage, so their scope is only in the file where they are declared.

The following table summarizes all the features of storage classes-

| Keyword         | Place of declaration                                          | Lifetime        | Scope                                                                               | Initial Value | Place of storage   | Linkage  | Initializer          |
|-----------------|---------------------------------------------------------------|-----------------|-------------------------------------------------------------------------------------|---------------|--------------------|----------|----------------------|
| auto (or none)  | Inside Function/Block                                         | Function /Block | Function/Block                                                                      | Garbage       | Memory (stack)     | None     | Constant/ Variable   |
| register        | Inside Function/Block                                         | Function /Block | Function/Block                                                                      | Garbage       | Registers          | None     | Constant/ Variable   |
| static (local)  | Inside Function/Block                                         | Program         | Function/Block                                                                      | Zero          | Memory (data area) | None     | Constant             |
| (none)          | Outside function (Definition of external variable)            | Program         | Definition to end of file (can be shared in other files using extern declaration)   | Zero          | Memory (data area) | External | Constant             |
| extern          | Outside or inside function (Declaration of external variable) | Program         | Declaration to end of file/Function                                                 | -             | Memory (data area) | External | Can't be initialized |
| static (global) | Outside function                                              | Program         | Definition to end of file (can't be shared in other files using extern declaration) | Zero          | Memory (data area) | Internal | Constant             |

## 15.9 Memory During Program Execution

The register variables are stored in CPU registers and rest of the variables are stored in memory. Now let us see how memory is organized when a C program is run.

**Code Area** - This area is used to store executable code of the program. The size of this area does not change during run time.

**Data Area** - This area stores static and global variables. It is further subdivided into two areas viz. initialized data area and uninitialized data area. The initialized data area stores all the initialized static and global variables while the uninitialized data area stores all the uninitialized static and global variables. The reason for division of this area is that all uninitialized variables can be collectively assigned value zero. The size of data area is also fixed and does not change during run time.

| Code               | Code of the program                                              |
|--------------------|------------------------------------------------------------------|
| Initialized Data   | Initialized static variables<br>Initialized global variables     |
| Uninitialized Data | Uninitialized static variables<br>Uninitialized global variables |
| Heap               | Dynamically allocated memory                                     |
| ↓                  |                                                                  |
| Stack              | Initialized and uninitialized automatic variables                |

Memory during execution of the program

**Heap** - This area is used for dynamically allocated memory. It is the responsibility of the programmer to allocate memory from the heap. The size of this area is dynamic i.e. it may change during run time.

**Stack** - Automatic variables are stored in this area. The size of this area keeps on changing during run time.

## 15.10 Functions with Variable Number of Arguments

In the functions that we have created till now, the number and data type of arguments was fixed at the time of function definition. In some situations we may need functions that can accept variable number of arguments of different types. The library functions `printf()` and `scanf()` are example of these type of functions. We have already used these functions many times with different number and type of arguments. We can also create our own functions that can accept variable number of arguments. These types of functions are also known as variadic functions.

The header file `stdarg.h` provides the facilities needed to define functions with variable number of arguments. This file defines a new type called `va_list` and three macros `va_start`, `va_arg`, `va_end` that can operate on variables of this new type.

### Type

`va_list` - Used to declare argument pointer variables.

### Macros

`va_start` - Initializes the argument pointer variable.

`va_arg` - Retrieves the current argument and increments the argument pointer.

`va_end` - Assigns NULL to argument pointer.

A function that accepts variable number of arguments should be defined with `ellipsis(...)` at the end of argument list. The `ellipsis` should occur only at the end of argument list and there should be at least one fixed argument. For example -

```
void func(char *str, int num,...)
{
 ...
}
```

Here `func()` takes two fixed arguments viz. `str`, `num` and after that it can accept any number of arguments. For example all these calls of `func()` are valid-

```
func("Chennai", 40, 67, 89, 'P', "Madras", 23, 67);
func("Lucknow", 35, 66);
func("Bareilly", 30, 'x', 20, 39, 12, 5);
```

In all these calls first two arguments are always a string and an integer, rest of the arguments can be of any type. Now inside the function definition we can access the fixed arguments using their name but the remaining arguments don't have a name so they are accessed using the macros defined in `stdarg.h` file. This is why fixed arguments are known as named arguments and variable number of arguments are known as unnamed arguments. Now we'll see how we can access these unnamed arguments inside the function body.

Initially we'll declare a variable of type `va_list`.

```
va_list ap;
```

This variable is conventionally named `ap` and is known as argument pointer, it will be used to point to the unnamed arguments. The macro `va_start` initializes `ap` and makes it point to the first unnamed argument passed to the function. This macro takes two arguments, first one is the argument pointer `ap` and second one is the name of the last fixed argument passed to the function (i.e. argument which is just before the ellipsis). For example in the previous function `func()`, `va_start` will be called as-

```
va_start(ap, num);
```

Now we can access individual variable arguments sequentially by using `va_arg`. This macro takes `ap` and the data type of the current argument.

```
arg=va_arg(ap, datatype);
```

It returns the value of the current argument and increments the pointer `ap` so that it points to the next argument. After calling `va_start`, the first call of `va_arg` returns first unnamed argument, second call returns second unnamed argument and so on. If datatype of the current unnamed argument in the function call does not match with the datatype in `va_arg` then the behaviour is undefined.

The macro `va_end` should be called before exiting from the function. This macro sets the argument pointer to `NULL`.

```
va_end(ap);
```

The unnamed arguments can't be used after calling `va_end`. If we want to use those arguments then once again we'll have to initialize `ap` with `va_start`.

The whole procedure is summarized in these steps-

- (i) Include the header file `stdarg.h`.
- (ii) The function header should contain ellipsis to denote the variable argument list.
- (iii) Declare a variable of type `va_list`.
- (iv) Initialize this argument pointer using `va_start`, so that it points to the first unnamed argument.
- (v) Use `va_arg` to retrieve the value of arguments.
- (vi) Call the macro `va_end` when you have finished working with these arguments.

There is no facility to count how many arguments were passed in the function call and what the type of each argument was. It is programmer's responsibility to pass this information to the function through fixed arguments. For example we may decide to take the first fixed argument as an integer that represents the total number of unnamed arguments. Generally a format string is passed as a fixed argument, which contains information about the type of each argument. For example `printf()` uses the format string that contains conversion specifications which denote the data type of each unnamed argument.

Now we'll take an example program and apply all these concepts. In this program we will make a function `sum()` which returns the sum of integers passed to it.

**Ex 15.8 Program to find out sum of integers\***

```
#include <stdio.h>
#include <stdarg.h>

sum(int num, ...)
{
 main(void)
 {
 printf("Total=%d\n", sum(2, 99, 68));
 printf("Total=%d\n", sum(3, 11, 79, 32));
 printf("Total=%d\n", sum(5, 23, 34, 45, 56, 78));
 return 0;
 }

 sum(int num, ...)
 {
 int i;
 va_list ap;
 int arg, total=0;
 va_start(ap, num);
 for(i=0; i<num; i++)
 {
 arg=va_arg(ap, int);
 printf("%d ", arg);
 total+=arg;
 }
 va_end(ap);
 return total;
 }
}
```

Output :

```
99 68 Total=167
11 79 32 Total=122
23 34 45 56 78 Total=236
```

Here we have called the function `sum()` with different number of arguments. We have taken only one fixed argument `num` that represents the number of unnamed arguments passed to the function. It is used in the loop to step through the unnamed arguments. Inside the function, firstly a variable of `va_list` type is declared. Then `va_start` is called and `num` is passed to it since it is the last fixed argument. After that each variable argument is obtained by calling `va_arg` repeatedly inside a loop.

In the above program we have taken variable number of arguments but all of them were of the same type. Now we'll take another program in which the unnamed arguments are of different types. The working of this function is somewhat similar to that of `printf()` function. In this function we'll send only one fixed argument and that will be a format string. Inside the function we'll scan this format string to know the data type of the unnamed argument. The text in the format string is printed through `putchar()`. We have printed character values and string values using `putchar()` and `fputs()` respectively. The integer and float values are first converted to strings and then printed using `fputs()`. We have used `i_to_str()` function that converts an integer to a string and `d_to_str()` function that converts a double to a string. The definitions of these functions are given in chapter on Strings. This `print()` function can print the binary equivalent of an integer.

**Ex 15.9 Program that uses a function similar to `printf()`\***

```
#include <stdarg.h>
#include <stdio.h>
#include <string.h>

void i_to_str(int num, char str[], int b);
void d_to_str(double num, char str[]);
void print(char *format, ...);

int main(void)
{
 int a=125;
 float b=563.66;
 char c='Q';
 char name[10]="Ranju";
 print("Value of a in binary=%b\n", a);
 print("Value of a in octal=%o\n", a);
}
```



```

print("Value of a in decimal=%d\n",a);
print("Value of a in hexadecimal=%x\n",a);
print("Value of b=%f\n",b);
print("Value of c=%c\n",c);
print("Value of name=%s\n",name);
return 0;
}

```

```

void print(char *format,...)
{

```

```

 char *p, str[50], *ptr;
 int k;
 float f;
 va_list ap;
 va_start(ap, format);

```

```

 for(p=format; *p!='\0'; p++)
 {

```

```

 if((*p)=='%')
 {

```

```

 p++;
 switch(*p)
 {

```

```

 case 'b':

```

```

 k=va_arg(ap, int);
 i_to_str(k, str, 2);
 fputs(str, stdout);
 break;

```

```

 case 'd':

```

```

 k=va_arg(ap, int);
 i_to_str(k, str, 10);
 fputs(str, stdout);
 break;

```

```

 case 'o':

```

```

 k=va_arg(ap, int);
 i_to_str(k, str, 8);
 fputs(str, stdout);
 break;

```

```

 case 'x':

```

```

 k=va_arg(ap, int);
 i_to_str(k, str, 16);
 fputs(str, stdout);
 break;

```

```

 case 'c':

```

```

 k=va_arg(ap, int);
 putchar(k);
 break;

```

```

 case 's':

```

```

 ptr=va_arg(ap, char *);
 fputs(ptr, stdout);
 break;

```

```

 case 'f':

```

```

 f=va_arg(ap, double);
 d_to_str(f, str);
 fputs(str, stdout);
 break;

```

```

 case '%':

```

```

 putchar('%');
 break;

```

```

 }
 }
 }
 else

```

```

 putchar(*p);
 }
}

```

```

void i_to_str(int num, char str[], int b)

```

```

int i=0, temp, rem, j, sign;

```

```

sign=num<0? -1: 1;

```

```

if(sign==1)

```

```

 num=-num;

```

```

while(num>0)

```

```

{
 rem=num%b;

```

```

 num/=b;

```

```

 if(rem>9 && rem<16)

```

```

 str[i++] = rem-10+'A';

```

```

 else

```

```

 str[i++] = rem+'0';
 }

```

```

if(sign==1)

```

```

 str[i++] = '-';

```

```

str[i] = '\0';

```

```

for(i=0, j=strlen(str)-1; i<j; i++, j--) /*Reverse the string*/

```

```

{
 temp=str[i];

```

```

 str[i]=str[j];

```

```

 str[j]=temp;
}

```

```

void d_to_str(double num, char str[])

```

```

{
 int i, k;

```

```

 double d;

```

```

 i=num;

```

```

 i_to_str(i, str, 10);

```

```

 str[strlen(str)+1] = '\0';

```

```

 str[strlen(str)] = '.';

```

```

 d=num-i;

```

```

 k=d*1000000;

```

```

 if(k<0)

```

```

 k=-k;

```

```

 i_to_str(k, str+strlen(str), 10);
}

```

Here we have used `int` in `va_arg` to extract a char value and `double` to extract a float value. This is because in the case of variable number of arguments the default argument promotions are applied i.e. arguments of type `char` and `short int` are promoted to `int` and arguments of type `float` are converted to `double`. So use `int` for extracting a char, `short int` or `int` and use `double` to extract a float or double.

We have taken the function `print()` to be of `void` type, if you want you can return the number of characters output as in `printf()`.

### 15.10.1 Passing variable number of arguments to another function

Suppose we write a function `func1()` that takes variable number of arguments and we want to pass all these unnamed arguments to another function `func2()`. For this the function `func2()` should be declared such that it can accept a variable of `va_list` type.

```

void func1(int num, ...)

```

```

{
 va_list ap;

```

```

 va_start(ap, num);

```

```

 func2(num, ap);
}

```



```
void func2(int num, va_list ap)
```

Inside func2() there is no need to declare a variable of va\_list type and call to macro va\_start is also not needed. The variable ap can be used in the macro va\_arg to retrieve the arguments. But in func2() we can use the unnamed arguments only once, it is not possible to call va\_start again and retrace the variable argument list. Now we'll take a simple example and see how this can be done. In the program P15.2 we had made a function sum() that displays the integers and returns their total. Now we'll make a separate function display() for displaying the numbers and will send the initialized argument pointer to this function.

*/\*P15.10 Program in which the variable length list is passed to another function:\*/*

```
#include<stdio.h>
#include<stdarg.h>
int sum(int, ...);
void display(int, va_list);
int main(void)
{
 printf("Total= %d\n", sum(2, 99, 68));
 printf("Total =%d\n", sum(3, 11, 79, 32));
 printf("Total= %d\n", sum(5, 23, 34, 45, 56, 78));
 return 0;
}
```

```
int sum(int num, ...)
{
 int i;
 va_list ap;
 int arg, total=0;
 va_start(ap, num);
 display(num, ap);
 for(i=0; i<num; i++)
 {
 arg=va_arg(ap, int);
 total+=arg;
 }
 va_end(ap);
 return total;
}
```

```
void display(int num, va_list ap)
{
 int i, arg;
 for(i=0; i<num; i++)
 {
 arg=va_arg(ap, int);
 printf("%d ", arg);
 }
}
```

The library function vprintf is one such function that accepts a variable of type va\_list. It is just like printf function except that it takes a variable of va\_list type instead of variable argument list. The prototypes of printf and vprintf are-

```
int printf(const char *format [, argument, ...]);
int vprintf(const char *format, va_list arglist);
```

Similarly vfprintf and vsprintf are analogous to fprintf and sprintf, except for this single difference.

```
int fprintf(FILE *stream, const char *format [, argument, ...]);
int vfprintf(FILE *stream, const char *format, va_list arglist);
int sprintf(char *buffer, const char *format [, argument, ...]);
int vsprintf(char *buffer, const char *format, va_list arglist);
```

Now we'll write a debugging function error() that will display the error message along with file name, function name and line number where the error occurred. This function won't be much useful if it takes only a fixed number of arguments of specific types because at different times the error message would have to display values of different variables. So we will define it as a variadic function. The first four arguments are fixed in which the first three represent filename, function name and line number respectively. The fourth argument is a format string which represents the error message and is similar to the format string of printf() function. After this we can have variable number of arguments which will be written along with the error message, according to the conversion specifications(%d, %s etc) present in the format string. The definition of error() function can be written as-

```
void error(char *file, char *fname, int line, char *format, ...)
```

```
{
 va_list ap;
 va_start(ap, format);
 printf("ERROR: ");
 printf("File - %s", file);
 printf("Function - %s", fname);
 printf("Line - %d\n", line);
 vprintf(format, ap);
 printf("\n");
}
```

Here we have used vprintf() to print the error message along with the variable number of arguments. Now suppose this function is invoked in function myfunc() of file myfile.c, at line 44 like this-

```
void myfunc(void)
{
 error(__FILE__, "myfunc", __LINE__, "Value of a must be less than %d and
 greater than %d\n", 100, 5);
}
```

The output will be this-

ERROR: File - c:\dir\myfile.c, Function - myfunc, Line - 44

Value of a must be less than 100 and greater than 5

If we want to send the output to the stream stderr, then we can use fprintf() and vfprintf() instead of printf() and vprintf() in the definition of error() function.

## Exercise

Find the output of the following programs, assume stdio.h is included in all programs.

```
enum month{jan, feb, mar, apr, may};
int main(void)
{
 enum month m;
 m=++feb;
 printf("%d\n", m);
 return 0;
}
```

```
enum day{sun=1, mon, tue, wed};
int main(void)
{
 enum day d1;
 printf("%d\t", mon);
 d1=mon+2;
 printf("%d\n", d1);
 return 0;
}
```

```

3. struct tag
{
 auto int x;
 static int y;
};
int main(void)
{
 struct tag s;
 s.x=4;
 s.y=5;
}

4. int var=6;
int main(void)
{
 int var=18;
 printf("%d\n",var);
}

5. int main(void)
{
 int i,sum=0;
 for(i=0; i<5; i++)
 {
 int i=10;
 sum=sum + i++;
 }
 printf("sum=%d\n",sum);
 return 0;
}

6. int main(void)
{
 int i,sum=0;
 for(i=0; i<5; i++)
 {
 static int i=10;
 sum=sum + i++;
 }
 printf("sum=%d\n",sum);
 return 0;
}

7. int x=89;
void func1(int x);
void func2(void);
int main(void)
{
 func1(x);
 printf("%d\t",x);
 func2();
 printf("%d\n",x);
 return 0;
}
void func1(int x)
{
 x++;
}
void func2(void)
{
 x++;
}

```

```

void func(void);
int x=2;
static int y=5;
int main(void)
{
 int x=3;
 func();
 func();
 printf("Inside main() : x=%d, y=%d\n",x,y);
 return 0;
}

void func(void)
{
 static int x;
 x=x+2;
 printf("Inside func(): x=%d, y=%d\n",x,y);
}

9. void func1(void);
void func2(void);
int main(void)
{
 func1();
 func2();
 return 0;
}
void func1(void)
{
 extern int x;
 x++;
 printf("%d\t",x);
}
int x=89;
void func2(void)
{
 x++;
 printf("%d\n",x);
}

10. #include<stdlib.h>
int *func(void);
int main(void)
{
 const int *ptr=func();
 *ptr=7;
 printf("**ptr=%d",*ptr);
 return 0;
}
int *func(void)
{
 int *p=malloc(sizeof(int));
 return p;
}

11. int main(void)
{
 char str1[]="hockey";
 char str2[]="Cricket";
 char const *p=str1;
 *p='j';
 p=str2;
 return 0;
}

```

```

12. int main(void)
{
 char str1[]="hockey";
 char str2[]="Cricket";
 char *const p=str1;
 *p='j';
 p=str2;
 return 0;
}

13. void func(int a[],const int b[],int c[]);
 int main(void)
 {
 int a[]={1,2,3,4};
 int b[]={5,6,7,8};
 int c[]={9,10,11,12};
 func(a,b,c);
 }
 void func(int a[],const int b[],int c[])
 {
 a=c;
 a[0]++;
 b=c;
 b[0]++;
 }

14. int thrice(int i);
 int main(void)
 {
 const int i=23;
 const int j=thrice(i);
 printf("j=%d\n",j);
 }
 int thrice(int i)
 {
 return 3*i;
 }

15. int func(void);
 int main(void)
 {
 int i=3;
 const int j=i;
 const int k=func();
 int *ptr=&k;
 const int m=*ptr;
 printf("%d\t%d\t%d\t%d\n",i,j,k,m);
 return 0;
 }
 int func(void)
 {
 return 4;
 }

16. #include<stdlib.h>
 int *func(int *psize);
 int main(void)
 {
 int i,size;
 const int *arr=func(&size);
 for(i=0; i<size; i++)
 {
 printf("Enter a[%d] : ", i);
 scanf("%d",&arr[i]);
 }
 }

```

```

 for(i=0; i<size; i++)
 printf("%d\t",arr[i]);
 return 0;
 }
 int *func(int *psize)
 {
 int *p;
 printf("Enter size : ");
 scanf("%d",psize);
 p=(int *)malloc(*psize * sizeof(int));
 return p;
 }

17. #include<stdarg.h>
 void func(int n,...);
 int main(void)
 {
 int count=4;
 func(count,2,3,4,5);
 return 0;
 }
 void func(int n,...)
 {
 va_list ap;
 int a,i;
 for(i=0; i<n; i++)
 {
 a=va_arg(ap,int);
 printf("%d ",a);
 }
 }

18. #include<stdarg.h>
 void func(int a,...,int b);
 int main(void)
 {
 int a=2,b=3,c=4,d=5;
 func(4,2,3,8,5);
 return 0;
 }
 func(int a,...,int b)
 {
 va_list *ap;
 va_start(ap,a);
 for(i=0; i<b; i++)
 printf("%d",va_arg(ap,int));
 va_end;
 }

19. int main(void)
 {
 int x=6;
 ++x++;
 printf("%d\n",x);
 return 0;
 }

```

## Building Project and Creation of Library

We have studied about C language, now we will come to the real life problems and see how we can solve them step by step. Here we will use our C program to develop one real life project with whole software life cycle of project. This will give you an idea of developing a full project in C. We will develop this project in Turbo C as well as in Unix. We will also see how to develop a library and use the functions of this library in different programs.

The project that we are going to make is related with date. Date related operations are of great importance in many applications. Most of the application projects require date related manipulations. So our aim is to develop a project in C, which will perform all date related manipulations. Now we will go step by step in different phases of software development life cycle for developing this project. These phases are-

1. Requirement analysis
2. Top level design
3. Detail design
4. Coding
5. Testing

### 16.1 Requirement Analysis

First step is requirement analysis. Here we think about requirement of our project by analyzing different manipulation of dates. We know what operations are needed with date, as well as different attributes of date. So after analyzing we came up with a document that gives all the requirement of the project. This is called as SRS(Software Requirement Specification).

1. Given date is valid or invalid.
2. Day of week from a given date.
3. Comparing two dates.
4. Difference of two dates in days.
5. Difference of two dates in years, months and days.
6. Add years to a given date.
7. Subtract years from a given date.
8. Add months to a given date.
9. Subtract months from a given date.
10. Add days to a given date.
11. Subtract days from a given date.

### 16.2 Top Level Design

#### Building Project and Creation of Library

Now we will go to the second step that is top-level design. Here we will divide the whole project in different modules. So whenever any change is needed, it will affect only one particular module and not others. Now we will write different modules in different files. All the modules will be implemented in different files as-

##### 1. Datefmt.c

This file will contain modules to convert a date from string form to integer variables and vice versa.

##### 2. Valid.c

This file will contain module to check whether a given date is valid or not.

##### 3. Leap.c

This file will contain module to check whether a given year is leap or not.

##### 4. Julian.c

This file will contain modules for getting julian number and converting julian number to months and days from a given date.

##### 5. Weekday.c

This file will contain module for getting day of week for a given date.

##### 6. Cmpdate.c

This file will contain module for comparing two dates.

##### 7. Diffymd.c

This file will contain module to find out difference of two dates in years, months and days.

##### 8. Diffdays.c

This file will contain module to find out difference of two dates in days.

##### 9. Addyears.c

This file will contain module for adding years to a given date.

##### 10. Subyears.c

This file will contain module for subtracting years from a given date.

##### 11. Addmonths.c

This file will contain module for adding months to a given date.

##### 12. Submonths.c

This file will contain module for subtracting months from a given date.

##### 13. Adddays.c

This file will contain module for adding days to a given date.

##### 14. Subdays.c

This file will contain module for subtracting days from a given date.

### 16.3 Detail Design

Now we will go to the next step that is detail design. Here we analyze all modules one by one for detail design. We analyze what functions will be required, what would be the input and return value of that function in each module and come up with one document for detail design, which contains the information of function prototype for each module. Let us see the function required in each module-

Note that date will be always in dd/mm/yyyy format.

##### 1. Datefmt.c

```
void splitDate(char *date, int *y, int *m, int *d)
```

|         |      |                                           |
|---------|------|-------------------------------------------|
| Input - | date | string in dd/mm/yyyy format               |
|         | y    | integer pointer, will get value of year.  |
|         | m    | integer pointer, will get value of month. |

Return value - d integer pointer, will get value of day  
None

void formDate(char \*date, int y, int m, int d)

Input - date pointer to character for getting date in string form  
y Integer that represents the year of given date  
m Integer that represents the month of given date  
d Integer that represents the day of given date

Return value - None

## 2. Valid.c

int isValid(char \*date)

Input- date  
Return Value 0 Date is not valid  
1 Date is valid

## 3. Leap.c

int isLeap(int year)

Input- year  
Return Value- 0 Not a leap year  
1 Leap year

## 4. Julian.c

int julian(int d, int m, int y)

Input- d day from given date  
m month from given date  
y year from given date  
Return Value- integer value containing julian number.

void revJulian(int j, int y, int \*d, int \*m)

Input- j contains julian no. for given year from given date  
y year from given date  
d integer value, will get day from julian number  
m integer value, will get month from julian number  
Return Value- None

## 5. Weekday.c

void weekDay(char \*date, char \*dayWeek)

Input- date Given date in dd/mm/yyyy format  
dayWeek pointer to character for getting day of week  
Return value- None

## 6. Cmpdate.c

int cmpDate(char \*date1, char \*date2)

Input- date1 First date in dd/mm/yyyy format  
date2 Second date in dd/mm/yyyy format

Return value- -1 date1 > date2  
0 date1 == date2  
1 date1 < date2

## 7. Diffymd.c

void diffYMD(char \*date1, char \*date2, int \*y, int \*m, int \*d)

Input- date1 First date in dd/mm/yyyy format  
date2 Second date in dd/mm/yyyy format  
y integer pointer, will get years  
m integer pointer, will get months  
d integer pointer, will get days

Return Value- None

## 8. Diffdays.c

int diffDays(char \*date1, char \*date2)

Input- date1 First date in dd/mm/yyyy format  
date2 Second date in dd/mm/yyyy format  
Return Value- integer value that contains difference of two dates in days

## 9. Addyears.c

void addYears(char \*date, int iyear, char \*newDate)

Input- date Given date  
iyear Number of years to be added  
newDate Character pointer, will get new date after addition  
Return Value- None

## 10. Subyears.c

void subYears(char \*date, int dyear, char \*newDate)

Input- date Given date  
dyear Number of years to be subtracted  
newDate Character pointer, will get new date after subtraction  
Return Value- None

## 11. Addmonths.c

void addMonths(char \*date, int imonth, char \*newDate)

Input- date Given date  
imonth Number of months to be added  
newDate Character pointer, will get new date after addition  
Return Value- None

## 12. Submonths.c

void subMonths(char \*date, int dmonth, char \*newDate)

Input- date Given date  
dmonth Number of months to be subtracted  
newDate Character pointer, will get new date after subtraction  
Return Value- None

## 13. Adddays.c

void addDays(char \*date, int days, char \*newDate)

Input- date Given date  
days Number of days to be added  
newDate Character pointer, will get new date after addition

Return Value- None

#### 4. Subdays.c

void subDays(char \*date, int days, char \*newDate);  
 Input- date Given date  
 days Number of days to be subtracted  
 newDate Character pointer, will get new date after subtraction

Return Value- None

### 16.4 Coding

Now we will go to the next step that is coding. Here we write logic for each function given in module, and finally write the code in C language. The code for all functions is given next. We maintain one header file that contains all function prototypes and will be included in the file wherever that function is used.

#### 16.4.1 dtmanip.h

```
void formDate(char *date, int y, int m, int d);
void splitDate(char *date, int *y, int *m, int *d);
int isValid(char *date);
int isLeap(int y);
void addDays(char *date, int days, char *newDate);
void addMonths(char *date, int imonth, char *newDate);
void addYears(char *date, int iyear, char *newDate);
int cmpDate(char *date1, char *date2);
int diffDays(char *date1, char *date2, int *y, int *m, int *d);
void diffYMD(char *date1, char *date2, int *y, int *m, int *d);
void subDays(char *date, int days, char *newDate);
void subMonths(char *date, int dmonth, char *newDate);
void subYears(char *date, int dyear, char *newDate);
void weekday(char *date, char *dayWeek);
void revJulian(int j, int y, int *d, int *m);
int julian(int d, int m, int y);
```

#### 6.4.2 Datefmt.c

```
void splitDate(char *date, int *y, int *m, int *d)
{
 date[2] = date[5] = '\\0';
 *d = atoi(date);
 *m = atoi(date+3);
 *y = atoi(date+6);
 date[2] = date[5] = '/';
} /*End of splitDate()*/

void formDate(char *date, int y, int m, int d)
{
 char arr[9][3] = {"01", "02", "03", "04", "05", "06", "07", "08", "09"};
 if (d < 10)
 strcpy(date, arr[d-1]);
 else
 sprintf(date, "%d", d);
 if (m < 10)
 strcpy(date+3, arr[m-1]);
 else
 sprintf(date+3, "%d", m);
 sprintf(date+6, "%d", y);
 date[2] = date[5] = '/';
} /*End of formDate()*/
```

The function splitDate() is used to get the values of day, month and year in different integer variables. We have used the library function atoi() that converts a string to an integer. The first argument to splitDate() is a string that contains the date in the format dd/mm/yyyy.

The function formDate() is used to form a date in format dd/mm/yyyy, from the values of day, month, year. Here we have used the library function sprintf() to convert an integers to strings.

#### 16.4.3 Valid.c

```
int isValid(char *date)
{
 int d, m, y;

 if (date[2] != '/' || date[5] != '/' || strlen(date) != 10)
 return 0;

 splitDate(date, &y, &m, &d);

 if (d == 0 || m == 0 || y == 0)
 return 0;
 if (d < 1 || d > 31 || m < 1 || m > 12)
 return 0;
 if (m == 2) /*check for number of days in February*/
 {
 if (d == 30 || d == 31 || (d == 29 && !isLeap(y)))
 return 0;
 }
 else if (m == 4 || m == 6 || m == 9 || m == 11) /*Check for days in April, June, Sept, Nov*/
 {
 if (d == 31)
 return 0;
 }
 return 1;
} /*End of isValid()*/
```

This function returns 1 if the date is valid, otherwise it returns zero. In this function, initially we check the format of the date. If the date is not in the format dd/mm/yyyy, then it is invalid. After this we call function splitDate() to get the values of variables y, m, d. Inside splitDate() we have used atoi() which returns 0 if it can't successfully convert string to an integer. For example suppose the user enters the date as : 09/11/1988, here the value of variable m will be zero since atoi() would not work successfully. So if any of the variables y, m, d has value zero, then the date is invalid.

#### 16.4.4 Leap.c

```
int isLeap(int year)
{
 if (year % 100 != 0 && year % 4 == 0 || year % 400 == 0)
 return 1;
 else
 return 0;
} /*End of isLeap()*/
```

#### 16.4.5 Julian.c

```
/*Function to calculate julian days*/
int julian(int d, int m, int y)
{
 int j = d;
 switch (m-1)
 {
```

```

case 11: j+=30;
case 10: j+=31;
case 9: j+=30;
case 8: j+=31;
case 7: j+=31;
case 6: j+=30;
case 5: j+=31;
case 4: j+=30;
case 3: j+=31;
case 2: j+=28;
case 1: j+=31;
}
if(!isLeap(y) && m>2)
 j=j+1;
return j;
} /*End of julian()*/

```

/\*Function to get the value of day and month from julian days\*/  
void revJulian(int j, int y, int \*d, int \*m)

```

{
 int i;
 int month[13]={0,31,28,31,30,31,30,31,31,30,31,30,31};
 if(isLeap(y))
 month[2]=29;
 for(i=1; i<=12; i++)
 {
 if(j<=month[i])
 break;
 j=j-month[i];
 }
 *d=j;
 *m=i;
} /*End of revJulian()*/

```

The logic of finding out the julian day is given in program P5.41 of Chapter 5. The function revJulian() finds out the value of day and month when the value of julian day and year is known. The array month stores the number of days in each month.

## 16.4.6 Weekday.c

/\*To find day of week from a given date\*/

#include <string.h>

void weekday(char \*date, char \*dayWeek)

```

{
 int d,m,y,j,f,h,fn,day;
 splitDate(date,&y,&m,&d);
 j=julian(d,m,y);
 f=(y-1)/4;
 h=(y-1)/100;
 fh=(y-1)/400;
 day=(y+j+f-h+fh)%7;
 switch(day)
 {
 case 0: strcpy(dayWeek,"Saturday"); break;
 case 1: strcpy(dayWeek,"Sunday"); break;
 case 2: strcpy(dayWeek,"Monday"); break;
 case 3: strcpy(dayWeek,"Tuesday"); break;
 case 4: strcpy(dayWeek,"Wednesday"); break;
 case 5: strcpy(dayWeek,"Thursday"); break;
 case 6: strcpy(dayWeek,"Friday"); break;
 }
}

```

/\*End of weekday()\*/

The logic of this function is similar to that given in P5.41.

## 16.4.7 Cmpdate.c

/\*To compare two dates\*/

int cmpDate(char \*date1, char \*date2)

```

{
 int d1,m1,y1,d2,m2,y2;
 splitDate(date1,&y1,&m1,&d1);
 splitDate(date2,&y2,&m2,&d2);
}

```

```

if(y1<y2)
 return 1;
if(y1>y2)
 return -1;
if(m1<m2)
 return 1;
if(m1>m2)
 return -1;
if(d1<d2)
 return 1;
if(d1>d2)
 return -1;
return 0;
} /*End of cmpDate()*/

```

This function returns 1 if date1 falls before date2,  
returns -1 if date2 falls before date1  
returns 0 if both dates are same.

## 16.4.8 Diffymd.c

/\*To get difference of two dates in years, months and days\*/  
void diffYMD(char \*date1, char \*date2, int \*y, int \*m, int \*d)

```

{
 int d1,m1,y1,d2,m2,y2;
 splitDate(date1,&y1,&m1,&d1);
 splitDate(date2,&y2,&m2,&d2);
 if(d2<d1)
 {
 if(m2==3)
 {
 if(isLeap(y2))
 d2=d2+29;
 else
 d2=d2+28;
 }
 else if(m2==5 || m2==7 || m2==10 || m2==12)
 d2=d2+30;
 else
 d2=d2+31;
 m2=m2-1;
 }
 if(m2<m1)
 {
 y2=y2-1;
 m2=m2+12;
 }
 *y=y2-y1;
 *m=m2-m1;
 *d=d2-d1;
} /*End of diffYMD()*/

```

The logic of this function is similar to that given in program P5.38.

### 6.4.9 Diffdays.c

*\*To get difference of two dates in days\**  
 int diffDays(char \*date1, char \*date2)

```
int d1, m1, y1, d2, m2, y2, j1, j2;
int y, d, i, days;
d=0;
splitDate(date1, &y1, &m1, &d1);
splitDate(date2, &y2, &m2, &d2);
j1=julian(d1, m1, y1);
j2=julian(d2, m2, y2);
if (y1==y2)
 return j2-j1;
for (i=y1+1; i<=y2-1; i++)
```

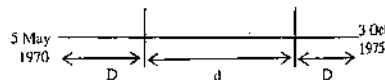
```
{
 if (isLeap(i))
 d=d+366;
 else
 d=d+365;
}
```

```
if (isLeap(y1))
 days=(366-j1)+d+j2;
else
 days=(365-j1)+d+j2;
return days;
```

*/\*End of diffDays()\*/*

Suppose we want to find out the difference in dates 5 May 1970 and 3 Oct 1975. The difference in days will be sum of three components D1, d, D2.

$j1 = 365 - \text{julian day of 5 May 1970} = 365 - 125 = 240$   
 $= \text{Total days in years from 1971 to 1974} = 365 + 366 + 365 + 365 = 1461$   
 $j2 = \text{julian day of 3 Oct 1975} = 276$



o total difference in days =  $240 + 1461 + 276 = 1977$

if both the dates are in the same year then we can find out the difference simply by subtracting the julian days.

### 6.4.10 Addyears.c

*\*To add years to a date\**  
 void addYears(char \*date, int iyear, char \*newDate)

```
int d, m, y;
splitDate(date, &y, &m, &d);
y=y+iyear;
if (d==29 && m==2 && !isLeap(y))
 d=28;
formDate(newDate, y, m, d);
```

*/\* End of addYears()\*/*

### 6.4.11 Subyears.c

*\*To subtract years from a date\**  
 void subYears(char \*date, int dyear, char \*newDate)

```
int d, m, y;
splitDate(date, &y, &m, &d);
y=y-dyear;
if (d==29 && m==2 && !isLeap(y))
 d=28;
formDate(newDate, y, m, d);
```

*/\*End of subYears()\*/*

### 16.4.12 Addmonths.c

*\*To add months to a date\**  
 void addMonths(char \*date, int imonth, char \*newDate)

```
int d, m, y, quot, rem;
splitDate(date, &y, &m, &d);
quot=imonth/12;
rem=imonth%12;
y=y+quot;
m=m+rem;
if (m>12)
{
 y=y+1;
 m=m-12;
}
if (m==2 && d>=29)
{
 if (!isLeap(y))
 d=28;
 if (isLeap(y))
 d=29;
}
if ((m==4 || m==6 || m==9 || m==11) && d==31)
 d=30;
formDate(newDate, y, m, d);
```

*/\*End of addMonths()\*/*

### 16.4.13 Submonths.c

*\*To subtract months from a date\**  
 void subMonths(char \*date, int dmonth, char \*newDate)

```
int d, m, y, quot, rem;
splitDate(date, &y, &m, &d);
quot=dmonth/12;
rem=dmonth%12;
y=y-quot;
m=m-rem;
if (m<=0)
{
 y=y-1;
 m=m+12;
}
if (m==2 && d>=29)
{
 if (!isLeap(y))
 d=28;
 if (isLeap(y))
 d=29;
}
if ((m==4 || m==6 || m==9 || m==11) && d==31)
 d=30;
formDate(newDate, y, m, d);
```

*End of subMonths()\*/*

### 16.4.14 Adddays.c

*\*add days to a date\**  
 void addDays(char \*date, int days, char \*newDate)

```
int d1, m1, y1, d2, m2, y2;
int j1, x, j2, k;
```



```

splitDate(date, &y1, &m1, &d1);

j1=julian(d1,m1,y1);
x = isLeap(y1) ? 366 : 365 - j1;
if(days<=x)
{
 j2=j1+days;
 y2=y1;
}
else
{
 days=days-x;
 y2=y1+1;
 k = isLeap(y2) ? 366 : 365;
 while(days>=k)
 {
 if(isLeap(y2))
 days=days-366;
 else
 days=days-365;
 y2++;
 k=isLeap(y2)?366:365;
 }/*End of while*/
 j2=days;
}/*End of else*/
revJulian(j2,y2,&d2,&m2);
formDate(newDate,y2,m2,d2);
}/* End of addDays()*/

```

Suppose we want to add days to the date 5 May 1970. Initially we'll find out the julian day(j1) of this date which is 125. Now we'll try to find out the year and julian day(j2) of the new date, and after that we'll use function revJulian() to find out the exact day and month of the new date.

First of all we'll find out the remaining days of the year 1970 by subtracting j1 from 365. This value comes out 240 and is stored in variable x. Now if the days to be added are less than or equal to x, then year of the new date will remain same and the julian day of the new date can be found out by writing j2=j1+days. For example if we have to add 200 days to 5 May 1970, then value of j2 will be 125 + 200 = 325, so after calling revJulian() and formDate() we get the new date as 21/11/1970.

Suppose we have to add 1900 days to 5 May 1970. First we'll subtract x=240 from 1900(comes out 1660) after this we'll reach the next year i.e. 1971. So now we have to add 1660 days to 1 Jan 1971. Now keep on subtracting the days of years from 1660 till you get a value less than 365(or 366 in case of leap year).

1660 - 365 = 1295, we've reached 1 Jan 1972

1294 - 366 = 929, we've reached 1 Jan 1973 (subtracted 366 since 1972 is leap)

929 - 365 = 564, we've reached 1 Jan 1974

564 - 365 = 199, we've reached 1 Jan 1975 (199 < 365)

Now the year y2 is 1975 and the julian day j2 is 199, so on calling revJulian() and formDate() we'll get the exact date which is 18/07/1975

### 16.4.15 Subdays.c

```

void subDays(char *date,int days,char *newDate)
{
 int d1,m1,y1,d2,m2,y2;
 int j1,j2,x,k;
 splitDate(date, &y1, &m1, &d1);
 j1=julian(d1,m1,y1);
 if(days<j1)
 {
 j2=j1-days;
 y2=y1;
 }
}

```

```

}
else
{
 days=days-j1; /*Now subtract days from 1st Jan y1*/
 y2=y1-1;
 k = isLeap(y2) ? 366 : 365;
 while(days>=k)
 {
 if(isLeap(y2))
 days=days-366;
 else
 days=days-365;
 y2--;
 k = isLeap(y2) ? 366 : 365;
 }
 j2=isLeap(y2) ? 366-days : 365-days;
 revJulian(j2,y2,&d2,&m2);
 formDate(newDate,y2,m2,d2);
}/*End of subDays()*/

```

The logic of this function is similar to that of addDays().

### 16.4.16 Main.c

```

#include<stdio.h>
#include<stdlib.h>
#include"dtmanip.h"
int()

int choice;
char date[11],date1[11],date2[11];
char dayWeek[10],newDate[11];
int iyear,imonth,dyear,dmonth,days;
int y,m,d;

while(1)
{
 printf("1. Date validation\n");
 printf("2. Compare dates\n");
 printf("3. Difference of Dates in days\n");
 printf("4. Difference of Dates in years,months,days\n");
 printf("5. Day of week\n");
 printf("6. Add years\n");
 printf("7. Add months\n");
 printf("8. Add days\n");
 printf("9. Subtract years\n");
 printf("10. Subtract months\n");
 printf("11. Subtract days\n");
 printf("12. Exit\n");
 printf("Enter your choice : ");
 scanf("%d",&choice);

 switch(choice)
 {
 case 1:
 printf("Enter date(dd/mm/yyyy) : ");
 scanf("%s",date);
 if(isValid(date))
 printf("Valid date\n");
 else
 printf("Not a Valid Date\n");
 break;
 case 2:

```

```

printf("Enter first date(dd/mm/yyyy) : ");
scanf("%s",date1);
printf("Enter second date(dd/mm/yyyy) : ");
scanf("%s",date2);
if(!isValid(date1) || !isValid(date2))
{
 printf("Enter only valid dates\n");
 break;
}
if(cmpDate(date1,date2)==0)
 printf("Date : %s and Date : %s are same.\n",date1,date2);
else if(cmpDate(date1,date2)==1)
 printf("Date : %s < Date : %s \n",date1,date2);
else if(cmpDate(date1,date2)==-1)
 printf("Date : %s > Date : %s \n",date1,date2);
break;
case 3:
printf("Enter first date(dd/mm/yyyy) : ");
scanf("%s",date1);
printf("Enter second date(dd/mm/yyyy) : ");
scanf("%s",date2);
if(!isValid(date1) || !isValid(date2))
{
 printf("Enter only valid dates\n");
 break;
}
if(cmpDate(date1,date2)==1)
 days=diffDays(date1,date2);
else
 days=diffDays(date2,date1);
printf("Difference in days = %d\n",days);
break;
case 4:
printf("Enter first date(dd/mm/yyyy) : ");
scanf("%s",date1);
printf("Enter second date(dd/mm/yyyy) : ");
scanf("%s",date2);
if(!isValid(date1) || !isValid(date2))
{
 printf("Enter only valid dates\n");
 break;
}
if(cmpDate(date1,date2)==1)
 diffYMD(date1,date2,&y,&m,&d);
else
 diffYMD(date2,date1,&y,&m,&d);
printf("Difference of the two dates is : ");
printf("%d years %d months %d days\n",y,m,d);
break;
case 5:
printf("Enter date(dd/mm/yyyy) : ");
scanf("%s",date);
weekDay(date,dayWeek);
printf("Day of week is %s\n",dayWeek);
break;
case 6:
printf("Enter date(dd/mm/yyyy) : ");
scanf("%s",date);
if(!isValid(date))
{
 printf("Enter a valid date\n");
 break;
}
printf("Enter the number of years to be added : ");
scanf("%d",&iyear);

```

```

 addYears(date,iyear,newDate);
 printf("New date is %s\n",newDate);
 break;
case 7:
printf("Enter date(dd/mm/yyyy) : ");
scanf("%s",date);
if(!isValid(date))
{
 printf("Enter a valid date\n");
 break;
}
printf("Enter the number of months to be added : ");
scanf("%d",&imonth);
addMonths(date,imonth,newDate);
printf("New date is %s\n",newDate);
break;
case 8:
printf("Enter date(dd/mm/yyyy) : ");
scanf("%s",date);
if(!isValid(date))
{
 printf("Enter a valid date\n");
 break;
}
printf("Enter the number of days to be added : ");
scanf("%d",&days);
addDays(date,days,newDate);
printf("New date is %s\n",newDate);
break;
case 9:
printf("Enter date(dd/mm/yyyy) : ");
scanf("%s",date);
if(!isValid(date))
{
 printf("Enter a valid date\n");
 break;
}
printf("Enter the number of years to be subtracted : ");
scanf("%d",&dyear);
subYears(date,dyear,newDate);
printf("New date is %s\n",newDate);
break;
case 10:
printf("Enter date(dd/mm/yyyy) : ");
scanf("%s",date);
if(!isValid(date))
{
 printf("Enter a valid date\n");
 break;
}
printf("Enter the number of months to be subtracted : ");
scanf("%d",&dmonth);
subMonths(date,dmonth,newDate);
printf("New date is %s\n",newDate);
break;
case 11:
printf("Enter date(dd/mm/yyyy) : ");
scanf("%s",date);
if(!isValid(date))
{
 printf("Enter a valid date\n");
 break;
}
printf("Enter the number of days to be subtracted : ");
scanf("%d",&ddays);

```

```

 subDays(date, days, newDate);
 printf("New date is %s\n", newDate);
 break;
 case 12:
 exit(1);
 default:
 printf("Wrong choice\n");
 /*End of switch*/
 }
 /*End of while*/
}
/*End of main()*/

```

## 16.5 Building Project in Turbo C

1. First go to Project → Open Project  
Create a project file DT.prj
2. Now a window will open that will display all files in the project. Right now we don't have any file added in project.
3. Now go to menu option Project → Add Items and add a file in the project. The added file will come in project window.
4. Similarly add all files one by one in the project. Now you can see all the files in project window.
5. Now you can open any file by double clicking on that item in project window. Any file can be deleted from the project by selecting it and pressing Delete or from the menu option delete item.
6. Now go to Compile → Build All, it will compile all the files and build the project. All the errors will come in window for each file. If there is no error then it will show success.
7. Now go to Run → Run and program will run.
8. Now close all windows including project item window.
9. Now go to Project → Close Project for closing the project.

## 16.6 Testing

Now we go for next and last step that is testing. Here we write different test cases for all modules and see whether the modules give correct results for each case. We perform testing for each individual module which is called module testing, as well as we do integration testing which tests that after integration also all modules are working fine or not or we find out which module is giving problem after integration.

In our project we will test functionality after integration because here module testing and integration testing is more or less same. So we will check functionality with different test cases, if any errors are coming then we will modify that particular part of coding and do the testing again.

## 16.7 Creation of Library and using it in your Program in Turbo C

To add a module in library, first we will produce .obj file by compilation then we will add the functions of .obj file to library with the help of tlib utility. The whole process is-

1. Produce the .obj file by compiling file as-  
d:\> tcc -c xyz.c (ENTER)  
Now xyz.obj will be created.  
We can also make the .obj file by Alt + F9.

2. Now we will add all the functions of xyz.obj in new library as-  
d:\> tlib ABC.lib+xyz.obj (ENTER)

Now new library ABC.lib will be created which will contain all the functions of XYZ.obj.

## Building Project and Creation of Library

3. Create a header file named ABC.h which contains prototype of all the functions available in library ABC.lib.
4. Now you can call any function of library ABC.lib by including header file ABC.h.
5. Suppose we want to add some more modules. First we will get the .obj file of all modules then they can be added as-  
d:\> tlib ABC.lib + XYZ1.obj + XYZ2.obj + XYZ3.obj (ENTER)

Now we will create library dtmanip.lib for our Date project and will call the functions in one sample file.

1. First we will produce .obj files for all modules as-  
d:\> tcc -c Datefmt.c Valid.c Leap.c Julian.c Weekday.c Cmpdate.c Diffymd.c Diffdays.c Addyears.c Subyears.c Addmonths.c Submonths.c Adddays.c Subdays.c
- This will produce .obj files for all modules.
2. Now we will create one new library dtmanip.lib for date manipulation by adding these modules.  
d:\> tlib dtmanip.lib+Datefmt.obj+Valid.obj+Leap.obj+Weekday.obj+Cmpdate.obj+Diffdays.obj+Diffymd.obj+Addyears.obj+Addmonths.obj+Adddays.obj+Subyears.obj+Submonths.obj+Subdays.obj+ Julian.obj

Now new library dtmanip.lib will be created, which will contain all the functions of above modules.

3. We will create one header file dtmanip.h, which is already given before.
4. Now we will create one sample file like main.c of Date project and call the functions available in dtmanip.lib.
5. Create new project testlib and add dtmanip.h, main.c and dtmanip.lib in this project.
6. Now we can build and run this project.

So now we have date manipulation library that can be used anywhere else also whenever date related manipulation is required.

## 16.8 Deletion of a Module from Library

We can delete any module from library as-

```
d:\> tlib dtmanip-Diffdays
```

This will delete the module of Diffdays from dtmanip library.

## 16.9 Getting Modules from Library

We can get module from library as-

```
d:\> tlib dtmanip*Valid
```

Now we will get the Valid module from dtmanip.lib. This can be added in different library as-

```
d:\> tlib newdt.lib+Valid.obj
```

This will add Valid module which we got from dtmanip.lib, into newdt.lib.

## 16.10 Changing Version of Module in Library

This can be done by deleting and then adding a particular module from library as-

> tlib dtmanip - Adddays

is will delete the module Adddays from dtmanip.lib library.

> tlib dtmanip + Adddays.obj

is will add the new version of module Adddays in dtmanip.lib library.

can do the same thing in one step also as-

> tlib dtmanip → Adddays

is will delete the Adddays module from dtmanip.lib and add the new version of Adddays in dtmanip.lib.

## 6.11 Building Project on Unix

ere is makefile concept for building project on unix platform. Unix provides a utility make to build project, s utility takes care of dependencies.

e know how to compile files with unix cc compiler and get the object and executable file. Suppose we want to mpile a single file Main.c.

-o Date Main.c

will compile Main.c and give the executable Date.

ppose we have more than one file then we can compile them as-

-o Date Main.c Datefmt.c Valid.c Leap.c Weekday.c Cmpdate.c Diffdays.c Diffymd.c Addyears.c dmonths.c Adddays.c Subyears.c Submonths.c Subdays.c Julian.c

ow it will compile all the files and give the executable Date as output.

ppose we are compiling all these files first time then it looks fine, but if we change source code of some file d again compile then it will be a time taking process. We can provide .o compiled object file of those files hich are not changed. Suppose only Main.c is changed and rest of them are same then we can use .o object file rest of them as-

-o Date Main.c Datefmt.o Valid.o Leap.o Weekday.o Cmpdate.o Diffdays.o Diffymd.o Addyears.o dmonths.o Adddays.o Subyears.o Submonths.o Subdays.o Julian.o

e can create only object file with -c option as-

-c file.c

o now we know that we have no need to compile all the files again if their source is not changed, we can use ur object file itself. But this is always very difficult to remember which file is changed and which one needs ot to recompile again. Even compiling these files again and again on command line is also a big problem. The olution is make utility. This utility takes care of all these problems and it even provides some more facility to anage project in a better way.

## 6.12 Writing Makefile

ake utility uses makefile for building project. This makefile contains commands that tell make utility how to ild your project with given source files. The makefile contains the dependency rules and construction rules.

ft side : right side

cc right side (compilation by using options for linking library)

Here in first line, left side is the target and right side is the source files on which target is dependent. Second line is for the construction. If any source file is changed then make will go for construction rule otherwise it will not do anything. We have to understand what exactly make utility does with makefile.

1. It reads makefile and comes to know which object file and library is needed for target and then it checks what source files are needed for each object.

2. It checks time and date for each object file against dependent source, if any source is of later date than its object file, then it will apply construction rule for that particular object.

3. Now it will check time and date of target against dependent object file if any object file is of later date than its target then it will apply construction rule for target.

Now let us see how will be our makefile for Date project then we will see what is the meaning of each line of our makefile.

Makefile 1-

```
#-----
Date project Makefile
#-----
```

```
#-----
```

TARGET=Dt

OBJS=Datefmt.o Valid.o Leap.o Julian.o Cmpdate.o Adddays.o Addmonths.o Addyears.o \
Subdays.o Submonths.o Subyears.o Diffymd.o Diffdays.o Weekday.o Main.o

```
$(TARGET):$(OBJS)
 cc -o $(TARGET) $(OBJS)
```

Datefmt.o:Datefmt.c

```
 cc -c Datefmt.c
```

Valid.o:Valid.c

```
 cc -c Valid.c
```

Leap.o:Leap.c

```
 cc -c Leap.c
```

Julian.o:Julian.c

```
 cc -c Julian.c
```

Cmpdate.o:Cmpdate.c

```
 cc -c Cmpdate.c
```

Adddays.o:Adddays.c

```
 cc -c Adddays.c
```

Addmonths.o:Addmonths.c

```
 cc -c Addmonths.c
```

Addyears.o:Addyears.c

```
 cc -c Addyears.c
```

Subdays.o:Subdays.c

```
 cc -c Subdays.c
```

Submonths.o:Submonths.c

```
 cc -c Submonths.c
```

Subyears.o:Subyears.c

```
$cc -c Datefmt.c Valid.c Leap.c Julian.c Cmpdate.c Adddays.c Addmonths.c Addyears.c \
Subdays.c Submonths.c Subyears.c Diffymd.c Diffdays.c Weekday.c
```

This will produce .o file for all source .c files.

2. Now we will assign all .o files to one variable.

```
$OBSJ= "Datefmt.o Valid.o Leap.o Julian.o Cmpdate.o Adddays.o Addmonths.o Addyears.o \
Subdays.o Submonths.o Subyears.o Diffymd.o Diffdays.o Weekday.o"
```

3. Library dt will be created with archiver ar as-  
ar rrv libdt.a \$OBSJ

4. Now we will write file Main.c and call the functions of dt library in it.

5. Now Main.c will be compiled as-  
cc -o Dt Main.c -L. -ldt

```
cc -c Subyears.c
iffymd.o:Diffymd.c
cc -c Diffymd.c
iffdays.o:Diffdays.c
cc -c Diffdays.c
'ckday.o:Weekday.c
cc -c Weekday.c
tain.o:Main.c
cc -c Main.c
```

he makefile can also be written as-

akefile2-

-----  
Date project Makefile  
-----

```
TARGET=Dt
OBSJ=Datefmt.o Valid.o Leap.o Julian.o Cmpdate.o Adddays.o Addmonths.o Addyears.o \
Subdays.o Submonths.o Subyears.o Diffymd.o Diffdays.o Weekday.o Main.o

ll:$(TARGET)
$(TARGET):$(OBSJ)
cc $(OBSJ) -o $@
```

### 16.13 Building Project with Make

We can simply execute make as-

\$make (ENTER)

It will search for file makefile in current directory and will build the project. If we have many makefiles for different projects, then we can use some naming convention to name makefile for projects. For example we can name our project makefile as project\_makefile. Here our project is Date so we can name our makefile as Date\_makefile.

We can execute this as-

\$make -f Date\_makefile (ENTER)

You should check the unix man page of make as this can be very useful for build work, which also takes a slice of project time. Generally one person in team is devoted for handling build work of project.

### 16.14 Creation of Library and Using in your Program in UNIX

Now we will see the process of creation of library and how to call functions of this library in another file.

1. First we will compile all the files for creating .o for source files.

When we think about optimization, it looks like we are talking only about making our code shorter. But it's not sometimes shorter code takes more execution time compared to large code. So here comes performance, it depends on whether you want shorter code that is good for maintenance but less in performance, or larger code that is bad for maintenance but good in performance. Now we will come to resource optimization. Actually resources are shared between different processes. Suppose your program takes more resources, then ultimately it will affect performance of other processes that need same resources. So we have a need to write and optimize our program keeping in mind resources e.g. processor's time. You will be surprised to know that most of the time, increasing of hardware like processors comes only because of our bad programming like unnecessary variables, bad searching and sorting algorithms. This is because we think only about running our program properly, and consider about resources only after getting some problem. Companies are spending more than \$50 million on resources, adding more than 12 processors in single server, only because they don't consider other things in programs and are helpless to find this individual problem. So our work is not only to write a program but also to develop the habit of writing optimized programs considering the overall view of the program. That's why optimization is a technique because it considers many other things in program not only code. You will see more things later in each topic.

## 1 Optimization with Tool

The first tool with our code is compiler, we can do lot of optimization with our compiler itself. It provides lot of options. So firstly we should understand our compiler better. Compiler manual can help you in understanding different options. We should know what our compiler can do for us and what it cannot do. So we do whatever it can do for us and we will not do whatever it does for us. We have some different tools, which can help us in writing our code in a better way. For example we can use tool purify to know where memory leak is coming in our code. Also we can use purecov to know how much coverage is done by our code, so that we can write code that has better coverage.

## 2 Optimization with Loop

### 2.1 Loop Unrolling

We can unroll small loops. Let us take an example-

```
a=5;
for(i=0; i<4; i++)
{
 value[i]=a;
 a=a+5;
}
```

It can be simply written like this-

```
value[]={5,10,15,20};
```

At initialization time. Alternatively it can be written in between program like this-

## Code Optimization in C

```
value[0]=5;
value[1]=10;
value[2]=15;
value[3]=20;
```

This is obviously better than previous one but most of the times compiler does this automatically, but we can expect little bit advantage and even it will be better to make a habit of writing optimized code. You can get this advantage in matrix program.

### 17.2.2 Avoiding Calculations in Loops

We should avoid any calculation, which is more or less constant in value within the loop. The same thing is true for avoiding calculation or anything, which can be removed from inner loop.

```
for(i=0; i<10; i++)
{
 value[i]=b*(20-b/3*c%2)*1/2;
}
```

We can avoid this calculation and it can be written as-

```
a=b*(20-b/3*c%2)/2;
for(i=0; i<10; i++)
{
 value[i]=a*i;
}
```

Always try to minimize things in inner loop because this can improve efficiency.

## 17.3 Fast Mathematics

### 17.3.1 Avoid Unnecessary Integer Division

We know that division operation is slow so we should try to minimize the division operations. Let us take an example-

```
int a,b,c,d;
d=a/b/c;
```

This division operation can be eliminated as-

```
d=a/(b*c);
```

Multiplication operation is more effective than division operation. So we should try to convert division operation to multiplication.

### 17.3.2 Multiplication and Division by Power of 2

We know that base of computer understanding is binary (0 or 1) i.e. base of 2. So we always need to do multiplication and division operations for numbers which are power of 2. One way is to do simple division or multiplication with the number, which is power of 2 is as-

```
a=a*8(8 is 23)
a=a/8(8 is 23)
```

The second way to perform the same operation is by using left shift << for multiplication operation and right shift >> for division operation. This can be done as-

```
a<<3(or a=a*23)
a>>3(or a=a/23)
```

So,  $a \ll b$  is equivalent to  $a \cdot 2^b$  and

$a \gg b$  is equivalent to  $a/2^b$

These bit operations will be much faster than multiplication and division operations. For simple operations, the compiler may automatically optimize the code but in case of complex expressions it is always advised to use bit operations, which is more effective optimization.

Suppose we want to multiply by 6 then it can be written as-

```
a = a<<1 + a<<2;
```

Compiler can not optimize this code so we should always make habit of writing optimized code without thinking that it can be optimized by compiler or not.

### 17.3.3 Simplifying Expressions

Sometimes we can reduce some operations by simplifying expressions. Let us see a simple example-

```
x*y + y*4
```

This can be simplified as -

```
(x+4)*y
```

This will reduce one multiplication operation.

### 17.4 Declare prototypes for Functions

We should always declare prototype for functions. This will tell the compiler about functions and it can be helpful for compiler to optimize the code.

### 17.5 Better Way of Calling Function

We know that calling a function means a stack will be setup with parameters of functions, so we should always try to reduce the parameter of functions, which can improve the stack mechanism. It will be better to use pointer instead of passing structure or big array, which can be overhead for stack and can decrease the efficiency of program.

### 17.6 Prefer int to char or short

We should always prefer int to char because C performs all operations of char with integer. In all operations like passing char to a function or any arithmetic operation, first char will be converted into integer and after completion of operation it will again be converted into char. For a single char this may not affect efficiency but suppose same operation is performed 100 times in a loop then it can decrease the efficiency of program.

### 17.7 Use of Register Variables

Register variables are stored in CPU registers. If you are declaring a variable as register then it will be accessed from register instead of memory which will always be faster. We know that registers are limited so we should declare only those variables as register, which are heavily used. If we have nested loops, then we should take the inner loop variables as register, because they will be used more times than outside loop variables.

### 17.8 Optimization with Switch Statement

Compilers translate switch statements in different ways. If case labels are small contiguous integer values, then it creates jump table. This is very fast and doesn't depend on number of case labels also. If case labels are longer and not contiguous then it creates comparison tree i.e. if...else statements. So in this case we should keep the most frequent label first and least frequent label should be at last. Let us take an example-

```
switch(no_of_day)
{
case 31:
```

#### Code Optimization in C

```
break;
case 30:
.....
break;
case 28:
case 29:
.....
break;
default:
printf("Incorrect value\n");
break;
}/*End of switch*/
```

Sometimes we see lot of repeated code written in all cases except one or two statements. Let us see an example-

```
switch(expression)
{
case a:
.....
break;
case b:
.....
break;
case c:
common statements;
different statements;
common statements;
break;
case d:
common statements;
different statements;
common statements;
break;
case e:
common statements;
different statements;
common statements;
break;
case f:
common statements;
different statements;
common statements;
break;
default:
break;
}
```

We can take all cases together and can write common statements only once and different statements in related cases using another switch. Here we will take cases c, d, e, f together and write common statements, then we can use another switch and write different statements in case c, d, e, f. Then after this switch we can again write common statements.

```
switch(expression)
{
case a:
.....
break;
case b:
.....
break;
```

```

e.c:
e.d:
e.e:
e.f:
common statements;
switch(expression);
{
case c:
different statements;
break;
case d:
different statements;
break;
case e:
different statements;
break;
case f:
different statements;
break;
}/*End of switch*/
common statements;
break;
ault:
break;
End of switch*/

```

## 9 Avoid Pointer Dereference

should try to pointer dereferencing in loop or while passing it as parameter. It creates lot of trouble in memory. Suppose it is used in loop then it will affect the efficiency. So better assign it to some temporary variable and then use that temporary variable in loop.

## 10 Prefer Pre Increment/Decrement to Post Increment/Decrement

should prefer pre increment/decrement to post increment/decrement wherever both of them have been used same work. In pre increment, it first increments the value and just copies to the variable location but in post increment it first copies the value to temporary variable, increments it and then copies the value to variable location. For a one time operation may be it will not affect but suppose this operation is in 1000 time loop then will sure effect the efficiency. So it will always be better to use pre increment/decrement to get efficiency.

## 11 Prefer Array to Memory Allocation

Wherever possible we should use array instead of pointer with memory allocation. Accessing with array index is always fast compared to dynamic storage. Basically it depends on the requirement also, but if we are sure that we are not going to use storage more than particular size then we can prefer array for that particular size. Memory allocation and deallocation has some burden and in some circumstances it can be avoided with array. We can take example of creating stack or queue, which will be better with array instead of implementing with linked list when we know the size of stack and queue will not be more than particular size.

## 12 Expression Order Understanding

We use lot of expressions in control statements and we also join more than one expressions by || or && operator. Let us take an example of || operator-

A || B

Here first A will be evaluated, if it's true then there is no need of evaluation of expression B and condition will become true. But if A is false then it will check for B, now if B is true then whole condition will become true otherwise it will be false. Suppose probability of expression A becoming true is very less and B becoming

true is high then most of the time after evaluating expression A, expression B will be evaluated. So most of the time we are evaluating A unnecessarily. In this case, it would be better to put the expression B first in condition, which has higher probability of becoming true.

B || A (preferred)

Now it will first check B and if it's true then condition will be true and it will skip most of the time evaluation of A and will increase the performance. Now let us check one example of && operator-

C && D

Here first expression C will be evaluated and if it is false then there is no need of evaluation of expression D and condition will become false. If C is true then it will check for D, if D is also true then whole condition will become true otherwise it will become false. Here we will check which expression has more probability of becoming false, that expression should be placed first expression in condition.

Now suppose probability of becoming true or false is equal in all expressions which are joined with || or && operator then the expression which can be evaluated faster than others should be first in condition.

## 17.13 Declaration of Local Function

We should always declare all the functions used in a program and if the function is used in that file only then it should always be declared as static, that means scope of that function will be limited to that file only. If it is not declared as static then it will be interpreted as extern and compiler will take unnecessary burden for external linkage of this function because it will assume that this can be used outside file also.

## 17.14 Breaking Loop with Parallel Coding

We can write parallel coding by writing instructions in pipeline. We can write one instruction of loop into parts, which can help to reduce the number of cycles and definitely increase the efficiency. Let us take an example-

```

total=0;
for(i=1; i<=100; ++i)
total+=i;

```

Here we are simply evaluating the total of numbers from 1 to 100. The same thing can be done as-

```

total1=0;
total2=0;
total3=0;
total4=0;
total=0;
for(i=1; i<=100; i=i+4)
{
total1=total1+i;
total2=total2+i+1;
total3=total3+i+2;
total4=total4+i+3;
}
total=total1 + total2 + total3 + total4;

```

It doesn't seem very attractive optimization but it can improve efficiency where big calculation is required like in mathematics or graphics.

## 17.15 Trick to use Common Expression

We can extract common expression, which is used at many places in the program. Let us take an example-

```

a*10/b;
c*5/b;

```

We can write the same as-



```
temp=5/b;
x=a*2*temp;
y=c*temp;
```

Here we are avoiding evaluation of an expression by using temp variable and multiplication. Here the expression is very small, but suppose it is a big one and used at lot of places then it will be better choice. These things are to be done manually by properly reviewing the code, because compiler will not be able to do optimization for these types of tricks.

### 17.16 Declaring Local Variables Based On Size

The local variables should be declared based on their sizes. We should prefer to declare larger to smaller datatype size variables. Compiler allocates variables in the same sequence in which they are declared. Declaring larger to smaller datatype variables will help the compiler in alignment and padding. Let us take an example-

```
int a;
float b;
long c;
double d,e[2];
```

This should be declared as-

```
double e[2];
double d;
long c;
float b;
int a;
```

### 17.17 Prefer Integer Comparison

We should always prefer integer comparison in conditions because it is faster as compared to any other data type.

### 17.18 Avoid String Comparison

We should try to avoid strings comparisons as much as possible. String comparison is very slow as it compares each character. Generally we use strcmp( ) function for comparing two strings. But this is very slow and can be very inefficient if we have lot of data in strings for comparing. We know we can't avoid comparing strings, but we can use some trick for the comparison-

1. Compare the index of '\0' of first string with the same index of second string.
2. Firstly compare first character of both strings; if they are same then only compare the full strings.

C can interact with assembly and this makes it more useful. It's good that we can interact with other language but the question arises why it is needed. We know that assembly is a low level language and it can interact directly with hardware. It does manipulation in registers so it is always fast compared to other languages. There are many things that can be done with assembly language only. We can use these features with C very easily. We can write inline assembly routine in our C code, and our C module can also interact with assembly module.

### 18.1 Inline Assembly Language

We can write assembly statement in C language by using asm keyword before each assembly statement in our program. For example-

```
asm mov AX, 5
```

Let us take a C program to print a character by using assembly statement.

```
main()
```

```
asm mov dl, 65
asm mov ah, 2
asm int 21h
```

Here we are using INT 21 with assembly. Let us see its description-

INT 21 H Service 2 – Character Output on Input-

DL= ASCII character  
AH=2

For writing same thing in MASM we have to use keyword \_asm and all the assembly statements will be in block starting with '{' and ending with '}'.

```
main()
```

```
_asm {
 mov dl, 65
 mov ah, 2

 int 21h
}
```

Now we can compile and run this program like other C programs. The C compiler parses assembly code from the program, creates an assembly file and then assembles it with the assembler (TASM or MASM), when it creates object file this object file will be linked to your C executable.

Now the question arises this can be done very easily in C language program, then why inline assembly is needed. Remember interaction through hardware can be done best through assembly language only. There are

any things which cannot be done with C, that can be done only through assembly language and there is no alternative C function available.

Suppose we want to get the scan code of any pressed key of keyboard. Every key has scan code including combination of special keys (Ctrl, Alt, Shift) and normal key. There is no C function available for getting scan code of pressed key. But we can use INT 16 with assembly for getting scan code. Let us see how INT 16 works.

INT 16H, service 0 - Read key from keyboard

Input- AH=0

Output- AH=Scan code AL=ASCII code

main()

```
asm mov ah,0
asm int 16h
```

After these two asm statements, scan code will be available in AH register which we can use in another part program.

Now we want to take the data in C statement and use it in asm statements. Suppose we want to print a string then we will need to use 9<sup>th</sup> routine of INT 21. See the description below-

INT 21 H Service 9 - String Print

Input-

DX=9

DS:DX points to a string that ends in '\$'

Service 9 of INT 21 is used to print the string which ends with '\$'. Let us take an example program.

main()

```
char * ptr="C with Assembly $";
asm mov dx,ptr
asm mov ah,9
asm int 21h
```

Here INT 21 Service 9 will print-  
C with Assembly

Now we want to write assembly statements in user defined function. This can be written as-

func();

func()

```
{
asm mov dl,65
asm mov ah,2
asm int 21h
}
```

Here we have written asm statements for printing character 'A' in function func(). Suppose we want to take this character in C statements then it can be as-

char ch='A';

main()

```
{
func();
}
```

func()

```
{
asm mov dl,ch
asm mov ah,2
asm int 21h
}
```

## C and Assembly Interaction

Here we have written a function which is not passing any value and does not return any value.

Suppose we want to write a function which passes some value and we want to use these parameter values in asm statement and want to return the value from function, then we will have to do it in some different way.

In C, when we pass parameters they go to the stack. It pushes the value of parameter instead of address, and for returning we use assembly instruction RET. It pushes the parameter in reverse order.

Let us take a program where we don't pass any parameter-

```
main()
{
func();
}
func()
{
asm mov dl,65
asm mov ah,2
asm int 21h
}
```

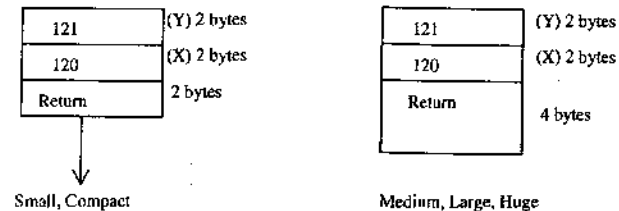
Here we are not passing any parameter, so the func() will be set up in stack as-



Here in large, medium and huge return address is of 4 bytes because these memory models can have more than 1 code segment.

Suppose we are passing parameters in our function. Let us take an example-

```
main()
{
display('x','y');
}
display(char x,char y)
{
asm
.....
.....
}
```



We know C pushes the parameters in reverse order(compiler dependent), so first y will be pushed and then x will be pushed in stack(undefined).

Now the question is how to get these parameters from stack to use in asm statement. SP is the stack pointer which points to the current word in stack. We can use Base pointer BP to store the position of SP, so that with the help of BP we can get the parameter value from stack and then it can be used in our asm statement.

### 3.2 Linking of Two Assembly Files

Now we will see how we can write two assembly files with one file having a procedure that will be called in another file. Similar things we will do with other C and assembly files where we will call assembly procedure in program and C functions in assembly program. But first we will learn how to link two assembly programs. Let us take a simple program that prints a single string.

```
CODE_SEG SEGMENT
 ASSUME CS:CODE_SEG, DS:CODE_SEG, ES:CODE_SEG, SS:CODE_SEG
 ORG 100H
 START: JMP DISPLAY
 STR DB "Suresh Srivastava.$"
 SUB_DISPLAY PROC NEAR
 MOV DX, OFFSET STR
 MOV AH, 9
 INT 21H
 INT 20H
 SUB_DISPLAY ENDP
 END START
```

Our program has procedure to print the string. Let us take another program that calls procedure, which is used to display the string.

```
CODE_SEG SEGMENT
 ASSUME CS:CODE_SEG, DS:CODE_SEG, ES:CODE_SEG, SS:CODE_SEG
 ORG 100H
 START: DISPLAY PROC NEAR
 CALL SUB_DISPLAY
 INT 20H
 SUB_DISPLAY ENDP
 SUB_DISPLAY PROC NEAR
 JMP GO
 ALL_OK DB "Suresh Srivastava.$"
 MOV DX, OFFSET ALL_OK
 MOV AH, 9
 INT 21H
 RET
 SUB_DISPLAY ENDP
CODE_SEG ENDS
 END START
```

Now think about modular programming, suppose we want to write all the procedures in different files and these procedures will be called in the main module. So we will have to link all the files. Let us take an example program that has two files, one file has procedure which is called in another file.

```
MAIN.ASM
CODE_SEG SEGMENT
 ASSUME CS:CODE_SEG, DS:CODE_SEG, ES:CODE_SEG, SS:CODE_SEG
 ORG 100H
 START:
 SUB_DISPLAY PROC NEAR
 CALL SUB_DISPLAY
```

```
 INT 20H
 DISPLAY ENDP
CODE_SEG ENDS
 END START

EXPROC.ASM

CODE_SEG SEGMENT
 ASSUME CS:CODE_SEG, DS:CODE_SEG, ES:CODE_SEG, SS:CODE_SEG
 SUB_DISPLAY PROC NEAR
 JMP GO
 ALL_OK DB "Suresh Srivastava.$"
 GO: MOV DX, OFFSET ALL_OK
 MOV AH, 9
 INT 21H
 RET
 SUB_DISPLAY ENDP
CODE_SEG ENDS
 END
```

We can see here both the files are in code segment. END START and ORG statements are only in MAIN.ASM. Because it can have only one entry and we want both files one after another in memory.

### 18.3 Memory Models

We have different memory models – tiny, small, medium, compact, huge which set the limit of code and data area. We select the memory model depending on our program requirements. Different models and their limit for code, data and array are given below-

|         | Code Segment        | Data Segment | Array  |
|---------|---------------------|--------------|--------|
| Tiny    | All in same segment |              |        |
| Small   | < 64 K              | < 64 K       | < 64 K |
| Medium  | > 64 K              | < 64 K       | < 64 K |
| Compact | < 64 K              | > 64 K       | < 64 K |
| Large   | > 64 K              | > 64 K       | < 64 K |
| Huge    | > 64 K              | > 64 K       | > 64 K |

### 18.4 C and Segments in Library

We can link assembly program with C program only when both of them have same segment name. C standard library uses different segment names based on their memory models. To overcome this problem, we have simplified segment directive which is supported by turbo c as well as Microsoft C. We have following keywords for declaration.

|          |   |                                                   |
|----------|---|---------------------------------------------------|
| CODE     | - | code                                              |
| DATA     | - | Initialized data                                  |
| DATA?    | - | Uninitialized data                                |
| FARDATA  | - | Initialized non-Dgroup data(Compact/Large/Huge)   |
| FARDATA? | - | Uninitialized non-Dgroup data(compact/Large/Huge) |

Here we have no need to write CODE\_SEG, when we will use the memory model then correct segment name will be placed. Let us see a program-

```
MAIN.ASM
MODEL SMALL
CODE
 ORG 100H
```

```

START:
DISPLAY PROC NEAR
 CALL SUB_DISPLAY
 INT 20H
DISPLAY ENDP
END START

```

EXPROC.ASM

```

.MODEL SMALL
.CODE
SUB_DISPLAY PROC NEAR
 JMP GO
 ALL_OK DB "Suresh Srivastava.$"
GO: MOV DX, OFFSET ALL_OK

 MOV AH, 9
 INT 21H
 RET
SUB_DISPLAY ENDP
END

```

Here model is declared as SMALL, so the segment names will be automatically placed based on memory model. Here code segment is automatically ended with the end of file. If data segment is available then code segment will end when the data segment starts. We can see that we don't have to worry for maintaining different segment names, with simplified directive, segment names will be placed based on memory model. For using segment address, we will have to use with @ prefix eg @code, @data.

Now we have to link both the files, so procedure will be declared as EXTRN in MAIN.ASM and it will be declared PUBLIC when its definition is available. See the program below for linking.

```

MAIN.ASM
EXTRN SUB_DISPLAY:NEAR
.MODEL SMALL
.CODE
 ORG 100H
START:
DISPLAY PROC NEAR
 CALL SUB_DISPLAY
 INT 20H
DISPLAY ENDP
END START

```

EXPROC.ASM

```

PUBLIC SUB_DISPLAY
.MODEL SMALL
.CODE
SUB_DISPLAY PROC NEAR
 JMP GO
 ALL_OK DB "Suresh Srivastava.$"
GO: MOV DX, OFFSET ALL_OK
 MOV AH, 9
 INT 21H
 RET
SUB_DISPLAY ENDP
END

```

Here we can see that SUB\_DISPLAY is declared as PUBLIC in file EXPROC.ASM, so that it can be called in another file. It is declared as EXTRN in MAIN.ASM, so it can be called in file MAIN.ASM.

```

d:\>TASM MAIN.ASM
d:\>TASM EXPROC.ASM
d:\>TLINK MAIN+EXPROC
d:\>EXE2BIN MAIN MAIN.COM
d:\>MAIN
Suresh Srivastava

```

## 18.5 Linking Assembly Procedure in C Program

Now we will see how we can call one assembly procedure in C program. We will write main program in C, which will call assembly procedure from asm program for getting the scan code of any key of keyboard.

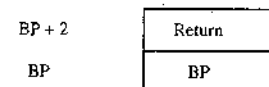
Main.c

```

extern int getscancode();
main()
{
 printf("Scan code of pressed key = %x\n", getscancode());
}

```

getscancode() function will be defined in asm program, so here it will be declared as extern to call this asm procedure in C program. But this procedure name will be with ( \_ ) underscore because for linking procedure in C this will be needed. Let us assume the memory model is small then our stack will be as-



Since we are not passing any parameter, only return value of function will be available in stack frame. See the asm procedure below-

GET.ASM

```

.MODEL SMALL
.CODE
 PUBLIC _getscancode
_getscancode PROC
 PUSH BP
 MOV BP, SP
 MOV AH, 0
 INT 16H
 POP BP
 RET
_getscancode ENDP
END

```

Here this procedure will get the scan code of pressed key and it will return this value. Here procedure name is in small letters because C is case sensitive and it will search for label in small letters. But assembler makes all public variables in capital letters, so we should use -mx option of assembler, so the same case of public variable will be used. Now we can link the assembly procedure with C program as-

```

d:\>tasm -mx GET.ASM
d:\>tec Main.c GET.obj

```

We have seen how to call assembly procedure in C program, but there was no parameter in function. Now we will see how to pass parameter in C function and how it can be used in assembly procedure. Suppose we are calling one function ADDNUM in C program as-

```

MAIN.C
extern int ADDNUM(int,int);
main()
{
 int total,a=5,b=10;
 total=ADDNUM(a,b);
 printf("Total=%d\n",total);
}

```

Now we will see how it will be in memory.

|        |      |
|--------|------|
| 5      | BP+6 |
| 10     | BP+4 |
| Return | BP+2 |
| Old BP | BP   |

Small, Compact

|        |      |
|--------|------|
| 5      | BP+8 |
| 10     | BP+6 |
| Return |      |
| Old BP | BP   |

Medium, Large, Huge

Suppose we are taking small memory model then we can see position of a is BP+6 and that of b is BP+4 in stack. Now we can use this in assembly procedure. The assembly procedure for this will be as-

```

ADD.ASM
MODEL SMALL
.CODE
 PUBLIC _ADDNUM
_ADDNUM PROC
 PUSH BP
 MOV BP, SP
 MOV AX, [BP+4]
 MOV BX, [BP+6]
 ADD AX, BX
 POP BP
 RET
_ADDNUM ENDP
END

```

Now we can link assembly procedure with C program as-

```

d:\>tasm -mx ADDNUM.ASM
d:\>tcc MAIN.c ADDNUM.obj

```

## Library Functions

### 19.1 Mathematical Functions <math.h>

#### pow()

Declaration : double pow(double x, double y);

This function returns the value of  $x^y$ .

#### sqrt()

Declaration : double sqrt(double x);

This function returns the square root of x, where  $x \geq 0$

#### ceil()

Declaration : double ceil(double x);

This function finds the smallest integer not less than x and returns it as a double. For example-

x = 2.4    Return value 3.0  
x = -2.4    Return value -2.0

#### floor()

Declaration : double floor(double x);

This function finds the largest integer not greater than x and returns it as a double. For example-

x = 2.4    Return value 2.0  
x = -2.4    Return value -3.0

#### fmod()

Declaration : double fmod(double x, double y);

It returns the floating point remainder of x/y. The sign of the result is the same as that of x. If the value of y is zero, the result is implementation defined.

x=7.0, y=4.0    Return value 3.0  
x=4.6, y=2.2    Return value 0.2

#### log()

Declaration : double log(double arg);

This function returns the natural logarithm (base e) of argument, arg > 0.

#### log10()

Declaration : double log10(double arg);

This function returns the base 10 logarithm of argument, arg > 0

**exp()**  
Declaration : double exp(double x);  
This function returns the value of  $e^x$ .

**ldexp()**  
Declaration : double ldexp(double arg, int exp);  
This function returns the value of the expression  $\text{arg} * 2^{\text{exp}}$ .

**frexp()**  
Declaration : double frexp(double arg, int \*exptr);  
This function splits the first argument into mantissa and exponent such that  $\text{arg} = \text{mantissa} * 2^{\text{exponent}}$ . It returns the value of mantissa whose range is [0.5, 1), and the value of exponent is stored in \*exptr. If the value of arg is 0 then the function returns 0 and the value stored in \*exptr is also 0.

**modf()**  
Declaration : double modf(double arg, double \*ptr);  
This function splits the first argument arg into integer and fractional part. Both parts have the same sign as arg. The fractional part is returned by the function, and the integer part is stored in \*ptr.

**sin()**  
Declaration : double sin(double arg);  
This function returns the trigonometric sine of the argument, where the argument is in radians.

**cos()**  
Declaration : double cos(double x);  
This function returns the trigonometric cosine of x.

**tan()**  
Declaration : double tan(double x);  
This function returns the trigonometric tangent of x, where the argument is in radians.

**asin()**  
Declaration : double asin(double x);  
This function returns the arc sine of x i.e.  $\sin^{-1}x$ . The argument should be in the range -1 to 1 and the return value lies in the range  $-\pi/2$  to  $\pi/2$ .

**acos()**  
Declaration : double acos(double x);  
This function returns the arc cosine of x i.e.  $\cos^{-1}x$ . The argument should be in the range -1 to 1 and the return value lies in the range 0 to  $\pi$ .

**atan()**  
Declaration : double atan(double x);  
This function returns the arc tangent of x i.e.  $\tan^{-1}x$ . The return value lies in the range  $-\pi/2$  to  $\pi/2$ .

**atan2()**  
Declaration : double atan2(double y, double x);  
This function is used for computing the arc tangent of  $y/x$  i.e.  $\tan^{-1}(y/x)$ . The return value lies in the range  $-\pi$  to  $\pi$ .

**sinh()**  
Declaration : double sinh(double x);  
This function returns the hyperbolic sine of x.

**cosh()**  
Declaration : double cosh(double x);  
This function returns the hyperbolic cosine of x.

**tanh()**  
Declaration : double tanh(double x);  
This function returns the hyperbolic tangent of x.

**fabs()**  
Declaration : double fabs(double x);  
This function computes the absolute value of floating point number.

## 19.2 Character Type Functions <ctype.h>

**isalnum()**  
Declaration : int isalnum(int arg);  
This function returns a nonzero value if argument is alphanumeric(a...z, A...Z, 0...9), otherwise it returns zero.

**isalpha()**  
Declaration : int isalpha(int arg);  
This function returns a nonzero value if argument is alphabetic(a...z, A...Z), otherwise it returns zero.

**isctrl()**  
Declaration : int isctrl(int arg);  
This function returns a non zero value if the argument is a control character, otherwise it returns zero.

**isdigit()**  
Declaration : int isdigit(int arg);  
This function returns a non zero value if the argument is a digit (0...9), otherwise it returns zero.

**isgraph()**  
Declaration : int isgraph(int arg);  
This function returns a non zero value if the argument is a graphic character (any printing character except a space), otherwise it returns zero.

**islower()**  
Declaration : int islower(int arg);  
This function returns a non zero value if the argument is a lowercase character (a...z), otherwise it returns zero.

**isprint()**  
Declaration : int isprint(int arg);  
This function returns a non zero value if the argument is printable character (including space), otherwise it returns zero.

**ispunct()**

Declaration : `int ispunct(int arg);`

This function returns a nonzero value if the argument is punctuation character (printing character except space, digit or letter), otherwise it returns zero.

**isspace()**

Declaration : `int isspace(int arg);`

This function returns a nonzero value if the argument is white space character (space, horizontal tab, vertical tab, form feed, new line, carriage return), otherwise it returns zero.

**isupper()**

Declaration : `int isupper(int arg);`

This function is used to check whether a given character is uppercase character (A...Z) or not. It returns nonzero if the argument is uppercase character, otherwise it returns zero.

**isxdigit()**

Declaration : `int isxdigit(int arg);`

This function is used to check whether a given character is hexadecimal digit (A...F, a...f, 0...9) or not. It returns nonzero if the argument is hexadecimal digit, otherwise it returns zero.

**tolower()**

Declaration : `int tolower(int arg);`

This function is used to convert an uppercase character (A...Z) into equivalent lowercase character (a...z). It returns lowercase character if the argument is an uppercase character, otherwise it returns unchanged value.

**toupper()**

Declaration : `int toupper(int arg);`

The macro `toupper()` is used to convert a lowercase character (a...z) into equivalent uppercase character (A...Z). It returns uppercase character if the argument is a lowercase character, otherwise it returns unchanged value.

## 19.3 String Handling Functions <string.h>

**strlen()**

Declaration : `size_t strlen(const char *str);`

It returns the number of characters in the string, excluding the null character.

**strcpy()**

Declaration : `char *strcpy(char *str1, const char *str2);`

This function copies string `str2` to `str1`, including the null character, and returns a pointer to the first string.

**strncpy()**

Declaration : `int *strncpy(char *str1, const char *str2, size_t length);`

This function copies exactly `n` characters from source string `s2` to the destination string `s1`. It returns the value of `s1`. If length of source string `s2` is less than `n`, then null characters are added to the destination string till all `n` characters have been written. If length of source string is more than or equal to `n`, then `n` characters will be written to destination string and the null character will not be added at the end.

```
#include<stdio.h>
#include<string.h>
```

## Library Functions

main()

```
{
 char str1[10];
 strcpy(str1, "Departmental", 6);
 str1[6] = '\0';
 printf("%s\n", str1);
}
```

Output:

Depart

**strcat()**

Declaration : `char *strcat(char *str1, const char *str2);`

The null character from `str1` is removed and `str2` is concatenated at the end of `str1`. A pointer to string `str1` is returned by the function.

**strncat()**

Declaration : `char *strncat(char *str1, const char *str2, size_t length);`

This function is same as `strcat()` but it concatenates only a portion of a string to another string. The null character from `str1` is removed and `length` characters of `str2` are appended at the end of `str1`, and at last a null character is added at the end of `str1`. This function returns `str1`.

```
#include<stdio.h>
#include<string.h>
```

main()

```
{
 char str1[15] = "ABC";
 strncat(str1, "DEFGHIJ", 4);
 printf("%s\n", str1);
}
```

Output:

ABCDEFGH

**strchr()**

Declaration : `char *strchr(const char *str, int ch);`

This function returns a pointer to the first occurrence of the character `ch` in the string `str`. If the character is not present in the string then it returns NULL.

```
#include<stdio.h>
#include<string.h>
```

main()

```
{
 char *p;
 p = strchr("Multinational", 'n');
 printf("%s\n", p);
}
```

Output:

national

**strrchr()**

Declaration : `char *strrchr(const char *str, int ch);`

This function returns a pointer to the last occurrence of the character `ch` in the string pointed to by `str`, otherwise it returns NULL.

```
#include<stdio.h>
#include<string.h>
```

main()

```
{
 char *p;
 p = strrchr("Multinational", 'n');
 printf("%s\n", p);
}
```

}  
Output:  
nal

**strcmp()**

Declaration : char \*strcmp(const char \*str1, const char \*str2);

This function is used to compare two strings lexicographically.

strcmp(str1, str2) returns a value-

```
<0 when str1<str2
=0 when str1==str2
>0 when str1>str2
```

**strncmp()**

Declaration : int \*strncmp(const char \*str1, const char \*str2, size\_t length);

This function is similar to strcmp() but it compares at most length characters of the two strings.

strncmp(str1, str2, len) returns a value-

```
<0 when str1<str2
=0 when str1==str2
>0 when str1>str2
```

**strspn()**

Declaration : size\_t strspn(const char \*str1, const char \*str2);

This function strspn() returns the number of characters in the beginning of str1 that match any of the characters in str2.

For example-

```
strspn("c in depth", "abcd"); will return 1
strspn("2012 Purva Belmont", "0123456789"); will return 4
strspn(" main", "\t"); will return 1
```

**strcspn()**

Declaration : size\_t strcspn(const char \*str1, const char \*str2);

This function returns the number of characters in the beginning of str1 that don't match any of the characters in str2.

```
strcspn("abcmnop", "lmn"); will return 3
strcspn("J P Nagar 6 phase", "0123456789"); will return 10
strcspn("structure", "aeiou"); will return 3
```

**strpbrk()**

Declaration : char \*strpbrk(const char \*str1, const char \*str2);

This function returns a pointer to first character of string str1, which matches with any character of str2. It returns NULL if there are no common characters in the two strings.

```
#include<stdio.h>
#include<string.h>
main()
{
 char *p1,*p2,*p3;
 p1=strpbrk("abcmnop", "lmn");
 p2=strpbrk("abcmnop", "ln");
 p3=strpbrk("1234ABCD", "COT");
 printf("%s\t%s\t%s\n", p1, p2, p3);
}
```

Output:

mnop nop CD

**strstr()**

Declaration : char \*strstr(const char \*arr1, const char \*arr2);

This function is used to locate the first occurrence of a substring in another string. The second argument is a pointer to the substring and the first argument is a pointer to the string in which the substring is to be searched. This function returns a pointer to the beginning of the first occurrence of the substring in another string. If the substring is not present then it returns NULL.

```
#include<stdio.h>
#include<string.h>
main()
{
 char *ptr;
 ptr = strstr("placement section", "cement");
 printf("%s", ptr);
}
```

Output:

cement section

**strtok()**

Declaration : char \*strtok(char \*s1, const char \*s2);

This function is used to extract tokens from a string based on some separator characters. A sequence of calls to strtok() breaks the string s1 into tokens, where each token is delimited by a character from the string s2.

The first call to strtok() has a non-NULL first argument, while the subsequent calls have a NULL first argument. The second argument which is a string containing the separator characters can differ from call to call. Each call to strtok() finds a token in string s1 and appends a NULL character to the token and returns a pointer to it. It returns NULL when there are no tokens left in s1. strtok() saves a pointer to the character following the token found, from where the next search for a token starts in subsequent calls.

**19.4 Input/Output Functions <stdio.h>****fopen()**

Declaration : FILE \*fopen(const char \*fname, const char \*mode);

The function fopen() is used to open a file. It takes two strings as arguments, the first one is the name of the file to be opened and the second one is the mode that decides which operations(read, write, append etc) are to be performed on the file. On success, fopen() returns a pointer of type FILE and on error it returns NULL. The values that mode can have are given in section 12.3.

**freopen()**

Declaration : FILE \*freopen(const char \*filename, const char \*mode, FILE \*fptr);

This function is used to associate a new file with a file pointer. The file associated with fptr is closed as by fclose(), after that the filename that is specified in the first argument is opened in the specified mode as by fopen(), and this fptr is now associated with this file. On success freopen() returns fptr, and on error it returns NULL. This function is generally used to change the files associated with file pointers stdin, stdout and stderr.

**fclose()**

Declaration : int fclose(FILE \*fptr);



This function is used to close a file that was opened by `fopen()`. It returns EOF on error and 0 on success. After closing the file, connection between file pointer and the file is broken. Now the file pointer `fp` is free to be connected to some other file.

### **clearerr()**

Declaration : `void clearerr(FILE *stream);`

This function is used to set the end of file and error indicators for `stream` to 0.

### **feof()**

Declaration : `int feof(FILE *fp);`

The macro `feof()` is used for detecting the end of file condition. It returns a nonzero value if end of file for `fp` is set otherwise it returns zero.

### **ferror()**

Declaration : `int ferror(FILE *fp);`

The macro `ferror()` is used for detecting any error occurred during read or write operations on a file. It returns a nonzero value if an error occurs i.e. if the error indicator for that file is set, otherwise it returns zero.

### **fflush()**

Declaration : `int fflush(FILE *fp);`

This function writes any buffered unwritten output to the file associated with `fp`. On success it returns 0 and on error it returns EOF. If we call this function with `stderr`, then all the unwritten buffered error messages will be written.

`fflush(stderr);`

To flush all output streams we can write-  
`fflush(NULL);`

### **fputc()**

Declaration : `int fputc(int ch, FILE *fp);`

This function writes a character to the specified file at the current file position and then increments the file position pointer. On success it returns an integer representing the character written, and on error it returns EOF.

### **fgetc()**

Declaration : `int fgetc(FILE *fp);`

This function reads a single character from a given file and increments the file pointer position. On success it returns the character after converting it to an int without sign extension. On end of file or error it returns EOF.

### **fgets()**

Declaration : `char fgets(char *str, int n, FILE *fp);`

This function is used to read characters from a file and these characters are stored in the string pointed to by `str`. It reads at most `n-1` characters from the file where `n` is the second argument. `fp` is a file pointer which is associated with the file from which characters are read. This function returns the string pointed to by `str` on success, and on error or end of file it returns NULL.

This function reads characters from the file until either a newline or an end of file is encountered or till `n-1` characters have been read. It appends a null character ('\0') after the last character read to terminate the string.

### **fputs()**

Declaration : `int fputs(const char *str, FILE *fp);`

This function writes the null terminated string pointed to by `str` to a file associated with file pointer `fp`. The null character that marks the end of string is not written to the file. On success it returns a nonnegative value and on error it returns EOF.

### **fprintf()**

Declaration : `int fprintf(FILE *fp, const char *format [, argument, ...]);`

This function is same as the `printf()` function but it writes formatted data into the file instead of the standard output. This function has same parameters as in `printf()` but it has one additional parameter which is a pointer of FILE type, that points to the file to which the output is to be written. It returns the number of characters output to the file on success, and EOF on error.

### **fscanf()**

Declaration : `int fscanf(FILE *fp, const char *format [, address, ...]);`

This function is similar to the `scanf()` function but it reads data from file instead of standard input, so it has one more parameter which is a pointer of FILE type and it points to the file from which data will be read. It returns the number of arguments that were assigned some values on success, and EOF at the end of file.

### **fread()**

Declaration : `size_t fread(void *ptr, size_t size, size_t n, FILE *fp);`

This function is used to read an entire block from a given file. `ptr` is a pointer that points to the block of memory which receives the data read from the file, `size` is the length of each item in bytes, `n` is the number of items to be read from the file and `fp` is a file pointer which points to the file from which data is read.

On success it reads `n` items from the file and returns the value of `n`, if error or end of file occurs then it returns a value less than `n`. We can use `feof()` and `ferror()` to check these conditions.

### **fwrite()**

Declaration : `int fwrite(const void *ptr, size_t size, size_t n, FILE *fp);`

This function is used for writing an entire block to a given file. `ptr` is a pointer which points to the block of memory that contains the information to be written to the file, `size` denotes the length of an item in bytes, `n` is the number of items to be written to the file, and `fp` is a FILE pointer which points to the file to which the data is written. If successful, `fwrite()` will write `n` items i.e. `n*size` bytes to the file and will return `n`. On error or end of file it will return a number less than `n`.

### **fseek()**

Declaration : `int fseek(FILE *fp, long displacement, int origin);`

This function is used for setting the file position indicator at the specified byte. Any read or write operation on the file will be done at the new position. `fp` is a file pointer, `displacement` is a long integer which can be positive or negative and it denotes the number of bytes which are skipped backward (if negative) or forward (if positive) from the position specified in the third argument. This argument is declared as long integer so that it is possible to move in large files. The third argument named `origin` is the position relative to which the displacement takes place. It can take one of these three values.

| Constant     | Value | Position          |
|--------------|-------|-------------------|
| SEEK_SET     | 0     | Beginning of file |
| SEEK_CURRENT | 1     | Current position  |
| SEEK_END     | 2     | End of file       |

### **ftell()**

Declaration : `long ftell(FILE *fp);`

is function returns the current position of the file position indicator. The value is counted from the beginning of the file.

### rewind()

Declaration : void rewind(FILE \*fptr);

This function is used to move the file position indicator to the beginning of the file.

### remove()

Declaration : int remove(const char \*fname);

is used for deleting a file, it returns 0 on success and -1 on error.

### rename()

Declaration : int rename(const char \*old, const char \*new);

This function is used to rename a file, on success it returns 0 and on error it returns -1.

### tmpfile()

Declaration : FILE \*tmpfile(void);

tmpfile() creates a temporary binary file and opens it in "wb+" mode. On success it returns a pointer of type FILE that points to the temporary file and on error it returns NULL. The file created by tmpfile() is temporary because it is automatically deleted when closed or when the program terminates.

### tmpnam()

Declaration : char \*tmpnam(char str[L\_tmpnam]);

This function is used to generate a unique name for a file. The argument str can be NULL or an array of minimum L\_tmpnam characters. If str is NULL then tmpnam() stores the temporary file name in an internal static array and returns a pointer to that array. If str is not NULL, then tmpnam() stores the temporary file name in str and returns str. The number of different file names that can be generated by tmpnam() during the execution of program is specified by TMP\_MAX, which is defined in stdio.h file. Note that tmpnam() only creates a file name, it does not create the file. A file with the name generated by tmpnam() can be opened using fopen(), and it should be closed using fclose().

### setbuf()

Declaration : void setbuf(FILE \*fptr, char \*buffer);

The function setbuf() specifies the buffer that will be used by a file. The size of the buffer should be BUFSIZ bytes, which is defined in stdio.h. If setbuf() is called with null, then the I/O will be unbuffered.

### perror()

Declaration : void perror(const char \*str);

This function is used to display a message supplied by the user along with the error message generated by the system. It prints the argument str, and a colon, then the message related to the current value of errno, and then a newline. errno is a global variable of type int declared in file errno.h, and it is set to a particular value whenever an error occurs.

### sprintf()

Declaration : int sprintf(char \*str, const char \*controlstring[, argument1, argument2, .....]);

This function is similar to printf() function except that instead of sending the formatted output to the screen, it stores the formatted output in a character array. Thus with the help of this function we can convert variables of any data type to string. A null character is appended at the end automatically. This function returns the number of characters output to the string excluding the null character.

### sscanf()

Declaration : int sscanf(const char \*str, const char \*controlstring [, address1, address2, ...]);

This function is same as the scanf() function except that data is read from a string rather than the standard input. We can read the formatted text from a string and convert it into variables of different data types.

## 19.5 Utility functions <stdlib.h>

### malloc()

Declaration : void \*malloc(size\_t size);

This function is used to allocate memory dynamically. The argument size specifies the number of bytes to be allocated. On success, malloc() returns a pointer to the first byte of allocated memory. The returned pointer is of type void, which can be type cast to appropriate type of pointer.

### calloc()

Declaration : void \*calloc(size\_t n, size\_t size);

This function is used to allocate multiple blocks of memory. The first argument specifies the number of blocks and the second one specifies the size of each block. The memory allocated by calloc() is initialized to zero.

### realloc()

Declaration : void \*realloc(void \*ptr, size\_t newsiz);

The function realloc() is used to change the size of the memory block. It alters the size of the memory block without losing the old data. This function takes two arguments, first is a pointer to the block of memory that was previously allocated by malloc() or calloc() and second one is the new size for that block.

### free()

Declaration : void free(void \*p);

This function is used to release the memory space allocated dynamically. The memory released by free() is made available to the heap again and can be used for some other purpose. We should not try to free any memory location that was not allocated by malloc(), calloc() or realloc().

### abs()

Declaration : int abs(int x);

It returns the absolute value of its argument.

### labs()

Declaration : long labs(long x);

It returns the absolute value of its argument.

### rand()

Declaration : int rand(void);

It returns a pseudo-random integer in the range 0 to RAND\_MAX, the value of RAND\_MAX macro is at least 32767.

### srand()

Declaration : void srand(unsigned int seed);

This function uses its argument seed as a seed for a new sequence of pseudo-random integers returned by subsequent calls to rand(). If rand() is called before calling srand(), the sequence generated will be

similar to the one that is generated when `srand()` is first called with a seed value of 1. If `srand()` is called with the same value of seed, the sequence of pseudo-random numbers will be repeated.

### **atoi()**

Declaration : `int atoi(const char *str)`

This function converts its argument `str` to `int`.

### **atol()**

Declaration : `long atol(const char *str)`

This function converts its argument `str` to `long`.

### **atof()**

Declaration : `double atof(const char *str)`

This function converts its argument `str` to `double`.

### **atexit()**

Declaration : `int atexit(void (*pf)(void));`

This function registers the function pointed to by `pf`, to be called when the program terminates normally. It returns zero if the registration is successful, nonzero if it fails.

### **exit()**

Declaration : `void exit(int status);`

This function causes normal program termination. All functions registered by `atexit()` are called, in the reverse order of their registration, all open streams are flushed and closed, and all files created by `tmpfile()` function are removed and the control is returned to the environment. An implementation-defined form of the `status` is returned to the environment. If the value of `status` is `EXIT_FAILURE`, an implementation-defined form of the status "unsuccessful termination" is returned, and if the value of `status` is 0 or `EXIT_SUCCESS`, an implementation-defined form of the status "successful termination" is returned, otherwise the status returned is implementation-defined.

## Solutions

### Chapter 3

#### Input-Output in C

1. Hello World

2. India

Delhi

`\b` takes the cursor to the previous position of current line, `\r` takes the cursor to the beginning of current line, `\n` takes the cursor to the beginning of next line.

3. `a=11 a=13 a=b a=B`

4. `a=-294967296, b=4000000000`  
`2147483647, 4294967295`

The output depends on the size of data type `int`.

5. This program enters a character and prints its ASCII value.

6. `123 126503 123.13 123.127`

7. `625, 2394, 12345`  
`625, 2394, 12345`

8. `h, 99`

9. `2.0, 6.80`  
`4.2, 3.57`  
`9.8, 85.67`

10. India

India

In

In

### Chapter 4

#### Operators and Expressions

1. 6

2. `c=0, d=1`

3. `e=0, f=6`

4. `a=36`

5. `6,5 6,4 7,4 7,5`

6. Error, Expressions like `8++`, `++x++`, `(x+y)--` are not valid.

7. `a=5, b=9, c=2, d=8, z=23`

8. `a=4, b=1, c=0`

9. `x=17, y=8`

10. `x=5, y=7, z=12, k=22`

11. `7.000000 14.500000`

12. `65,A 98,b`

13. `a=20, c=16`

Precedence of `+` operator is more than that of `+=`. Associativity of compound assignment operators is from right to left.

14. `a=10, b=5`

15. Since the precedence of assignment operator is less than that of conditional operator, the conditional statement is

interpreted as-  $(a > b ? \max = a : \max) = b$ ;  
Now this statement reduces to statement  $10 = b$ , which results in an error. So the solution is to put parentheses around the third operand as-  $a > b ? \max = a : (\max = b)$ ;

5. 6 1 6 6

7.  $a=5, b=7, c=5, d=4, x=1, y=1$   
 $a=5, b=7, c=5, d=10, x=0, y=1$

8. Result of these types of expressions is undefined.

9.  $a=3, b=3, x=36, y=28$

## Chapter 5

### Control Statements

a is five Value of a is 5

The mistake here is that assignment operator is used for comparing the values instead of equality operator( $==$ ). The variable a is assigned the value 5, and the value of test expression  $a=5$  is 5 which is a non-zero value and in C non-zero value means true so the if condition becomes true.

a is not zero Value of a is 0

As in E1, here also assignment operator is used instead of equality operator. The variable a is assigned the value 0, value of test expression  $a=0$  is 0, and in C zero means false, so the if condition becomes false.

i is not fifty

The mistake here is that equality operator is used instead of assignment operator for assigning the value to i. So i is compared with 50 and the result of the expression is 0 which is discarded. The variable i remains unassigned.

1. 20 8

Only the statement  $a=a-5$  is part of the if statement, the statement  $b=b+5$  will always be executed. If we want the statement  $b=b+5$  to become part of if statement then we need to enclose both statements within braces.

```
if (a < 10)
{
 a = a - 5;
 b = b + 5;
}
```

5. C in depth

1. 11

There is a semicolon after the condition of if statement, and it is considered as a null statement.

```
if (i >= 5 && j < 5)
;
i = j + 2;
```

So the statement  $i=j+2$  will always be executed.

1. 0, 1

1. a is not 2.

Here the else part is paired with the second if, if we want to pair it with the first if then we should enclose the second if inside braces.

```
if (a == 2)
{
 if (x == 8)
 printf("a is 2 and x is 8\n");
}
else
 printf("a is not 2\n");
```

1. if (a < b)

```
x = 0;
else
 x += 100;
```

10. The second condition should be evaluated only if the first condition is true, and the third condition should be evaluated only if both first and second conditions are true and the statement should be executed only if all the three conditions are true. So we can combine these if statements using  $\&\&$  operator.

```
if (i == 3 && j == k && l < m)
 z = 100;
```

This is possible because of the left to right short circuit evaluation of logical expressions in C. If the left side expression of  $\&\&$  or  $\|\$  determines the outcome then the right side expression is not evaluated. In case of  $\&\&$  operator, if the left side expression is false, then the right side expression is not evaluated and in case of  $\|\$  operator if the left side expression is true, then the right side expression is not evaluated.

11. If the first condition is true then the other two will definitely be false, so there is no need to check the other two conditions, if second is true then there is no need to check the third condition. Hence this can be written using an else-if chain.

```
if (x == 0)
 y++;
else if (x > 0)
 y--;
else
 y += 2;
```

```
12. if (grade == 'A')
 if (marks > 95)
 printf("Excellent\n");
 else
 ; /*Null statement*/
 else
 printf("Work hard for getting A grade\n");
```

```
13. if (a == 10)
 if (b == 20)
 {
 if (c == 30)
 printf("a is 10, b is 20, c is 30\n");
 }
 else
 printf("a is 10, b is not 20\n");
 else
 printf("a is not 10\n");
```

14. Error, continue can't be used inside switch.

15. Error, case expressions are not constant expressions.

16. 1 4 5 7 total=17

Here the continue statement is in a switch which is enclosed inside a for loop so the continue statement applies to the for loop.

17. y is 19

Any executable statement inside a switch before the first case label is ignored by the compiler and is never executed.

```

18. (i) switch(x)
{
 case 1:
 y=x+1;
 break;
 case 2:
 y=0;
 x=0;
 break;
 case 3:
 case 4:
 y++;
 break;
 case 5:
 y++;
 break;
 case 6:
 y=4;
 break;
 default:
 y--;
}

```

```

(ii) switch(x)
{
 case 1:
 y=x-1;
 break;
 case 2:
 case 3:
 case 4:
 y++;
 break;
 case 5:
 y--;
 break;
 case 6:
 y=0;
}

```

In (ii), the last else is not present, so there is no default in the switch.

```

19. if(x==10)
 z++;
else if(x==20)
 z--;
else if(x==30)
{
 if(y==2)
 z=z+3;
 else if(y==4 || y==5)
 z=z+10;
 else
 z=z-2;
}
else
 z=0;

```

20. (i) 5 4 3 2 1  
 (ii) This program runs infinitely, because the goto statement is not inside the if structure.

21. (a) -1 is a non zero value so the loop condition is always true, hence the loop will iterate infinitely.  
 (b) Loop will iterate infinitely.  
 (c) Loop condition is false, so the loop will not iterate even once.  
 (d) Variable a is assigned value zero in loop condition a=0, the value of this expression is 0 so the condition is false, and the loop will not iterate even once.  
 (e) Variable a is assigned value 2 in loop condition a=2, the value of this expression is 2 so the condition is true, and the loop will iterate infinitely.  
 (f) Variable a is assigned value of b which is 1 and so the value of expression a=b is 1, thus the loop will iterate infinitely.  
 (g) Variable a is assigned value of b which is 0, and so the value of expression a=b is 0, thus the loop will not iterate even once.

22. (i) 0 1 2 3 4 5 6 7 8 9  
 (ii) 1 2 3 4 5 6 7 8 9 10  
 (iii) 0 1 2 3 4 5 6 7 8 9 10  
 (iv) 1 2 3 4 5 6 7 8 9  
 (v) This loop will run infinitely. Value of i is initially 0 and it is decremented in each loop iteration, so i will never exceed 10 and hence the condition i<=10 never becomes false.  
 (vi) 10 9 8 7 6 5 4 3 2 1  
 (vii) 10 9 8 7 6 5 4 3 2  
 (viii) 10 9 8 7 6 5 4 3 2 1

Solutions

- (ix) 15 12 9 6 3 0  
 (x) -1

There is a semicolon after the closing parenthesis of for statement, this acts as a null statement.  
 for(i=10; i>=0; i--)

- printf("%d ", i);  
 The loop iterates 11 times and when the value of i becomes -1, control comes out of the loop and value of i is printed.  
 (xi) Loop condition is initially false so the loop does not iterate even once.  
 (xii) 0

This loop will iterate once and after that the condition will become false.  
 (xiii) 1 3 5 7 9 11 13 15.....

This loop will iterate infinitely as the value of i will never become 10.  
 (xiv) 10 8 6 4 2

The loop terminates when the value of assignment expression ( i=j ) becomes zero.  
 (x) 9 8 7 6 5 4 3 2 1 0

The loop terminates when the value of expression i-- becomes zero.

23. (i) for(term=10; term<=70; term+=10)  
 printf("%d ", term);  
 (ii) for(term=-10; term>=-70; term-=10)  
 printf("%d ", term);  
 (iii) for(term=1; term<=1111; term=term\*10+1)  
 printf("%d ", term);  
 (iv) i=0;  
 for(term=1; term<=16; i++, term=term\*i)  
 printf("%d ", term);  
 (v) for(term=10; term>=-5; term-=3)  
 printf("%d ", term);  
 (vi) for(term=1; term<=32; term\*=2)  
 printf("%d ", term);

24. In all the three loops, the loop body is a single null statement(;). The first loop will iterate 4 times and will terminate when the value of i becomes 5. So the first printf prints 5. In the second loop, the loop condition is false so it will not iterate and the printf will print 10. In the third loop, the loop condition is true so it will iterate infinitely and the printf will not be executed.

25. 2 3 4 5  
 2 3 4  
 5 4 3 2 1 0

26. a=6, b=4  
 a=3, b=1  
 a=1, b=1  
 a=0, b=1  
 a=0, b=1  
 a=0, b=1

The value of b never becomes zero, and so the condition never becomes false resulting in an infinite loop.

27. 10 7 4 1 -2 -5 -8 .....

The value of i never becomes zero, resulting in an infinite loop.

28. x=6

29. The variable sum is not initialized to zero, if it is initialized to zero then the result will be 126.

30. The first loop is an infinite loop, and the break statement will terminate it. The printf will print 4. In while loop, we cannot omit the condition as in for loop. So there is a syntax error in the second loop.

31. 10 14

32. The comma operator is evaluated from left to right, so first j=i+1 is evaluated and then i=1. The correct way is to write first i=1 and then j=i+1.

10. There is a null statement in do-while loop.

```
do
;
while(++i<10);
```

- (i) Will print 6 stars in one line.  
(ii) Will print 5 groups of 6 stars on one line i.e. 30 stars will be printed in one line.  
(iii) Will print 5 groups of 6 stars, each group on a separate line.  
(iv) Whatever is printed in part (iii) will be printed 3 times.

11. (i) for(i=1; i<=5; i++)  
printf("%d ", i);  
(ii) i=1, j=3;  
while(i<5 && j!=0)  
printf("%d ", i++);  
(iii) for(i=2; i<=sqrt(n) && n%i!=0; i++);

12. (i) 5 0 (ii) 4 0  
In program (i), there is a statement before break in the loop. So we can't remove this break by negating its condition and putting in the loop condition.

13. (i) for(i=1; i<=5; i++)  
printf("%d\n", i);  
(ii) for(sum=0; sum<=100; sum+=n)  
scanf("%d", &n);

14. for(i=0, j=10, k=6; i!=j && k>=0; i++, j--, k=k-2)  
printf("%d %d %d\n", i, j, k);

15. (i) Here the loop condition is checked inside the loop before executing the body of loop, so this can be written using while loop as-

```
i=1, z=0;
while(i<5)
{
printf("%d ", i++);
z=z+2;
}
```

(ii) printf("Enter employee ID(100-500) : ");  
scanf("%d", &emp\_id);  
while(emp\_id<100 || emp\_id>500)  
{  
printf("Enter employee ID(100-500) : ");  
scanf("%d", &emp\_id);  
}

40. int main(void)  
{  
int l, b, h, sarea, vol;  
do  
{  
printf("Enter length, width, height of a cuboid : ");  
scanf("%d%d%d", &l, &b, &h);  
}while(l<=0 || b<=0 || h<=0);  
sarea=2\*(l\*b + b\*h + h\*l);  
vol=l\*b\*h;  
printf("Surface Area=%d, Volume=%d\n", sarea, vol);  
return 0;  
}

41. 1 2 3 4 5  
All the loops will give the same output.

42. The output given by first loop is: 1 2 4 5  
The other two loops will iterate infinitely after printing 1 and 2. In for loop, the continue statement transfers the control to the update expression, while in while and do-while loops the control is transferred to the test

## Solutions

expression. In the while and do-while loops given in the exercise the expression that changes the value of i is after continue statement. Once the continue statement is encountered, test expression is checked and again continue is executed and this process goes on infinitely.

43. (i) 6

(ii) Syntax error

44. (i) for(i=1; i<6; i++)  
{  
if(i!=3)  
printf("%d ", i);  
}

(ii) x=0, sum=0;  
while(x<10)  
{  
x++;  
if(x%2!=0)  
sum+=x;  
}

45. (i) int main(void)  
{  
int i, j, k;  
for(i=1; i<8; i++)  
{  
if(i==3 || i==5)  
continue;  
for(j=1; j<=i; j++)  
{  
for(k=j; k>0; k--)  
printf("%d ", k);  
printf("\n");  
}

(ii) int main(void)  
{  
int i, j, x;  
for(i=1; i<10; i++)  
{  
x=i+5;  
if(i%2!=0)  
{  
printf("%d\n", x);  
continue;  
}  
x+=i;  
for(j=i; j>0; j--)  
{  
if(i==j)  
x++;  
else  
x--;  
printf("%d ", x);  
}  
printf("\n");  
}  
return 0;  
}

Chapter 6  
Functions

1. We can't use goto between functions, so this program will show an error stating that ab is an undefined label.

2. Function is declared as void, so the function call can't be used in assignment expression.

3. If no value is returned through return statement, and the function is not declared of type void then garbage value is returned, so the variable x will have garbage value.

4. In variable declarations we can declare many variables of same type by writing the type name only once, but not in parameter declarations.

5. Since function declaration is absent and the definition is present after the call, there is no information about the parameters and hence no parameter checking. The function is called with wrong number of parameters and so the behaviour is undefined.

6. 18 60 27 54

7. Error: redefinition of formal parameter 'x'

8. Error: Expression syntax, The operands of conditional operator should be expressions, but return x and return y are not expressions. The statement should be written like this- return a<b ? a : b;

9. 8

10. func() is void, so it can't be used as an operand of '+' operator.

11. 24 11

12. 22

13. x=1, y=2, z=5 result=36

14. Sum is 3

If the value of a non void function is not to be used, then this can be made explicit by converting the function call expression to void with the help of a cast.

15. 4.000000 6.000000 4.000000 6.250000

16. We'll get errors because a function definition can't be written inside definition of another function.

17. 15 290

18. 2 2

19. 7

20. 4

Here expression a,b,c is considered as an expression with comma operator, and so its value is the value of rightmost operand which is c.

21. 1

22. 2 5

In func(), the local variable will be accessed because whenever there is a conflict between the local and global variables, the local variable gets the priority.

23. 10

24. 24

25. i=9

26. static keyword is not allowed in parameter declarations.

27. 2 4 8 16 32 64

28. 3 18 21

```
29. int func1(int a, int b)
{
 if(a==b)
 return f1(a,b);
 return f2(a,b);
}
int func2(int a, int b)
{
 return a==b? f1(a,b): f2(a,b);
}
```

```
31. int func1(int n)
{
 if(n==39)
 n+=5;
 return n;
}
```

```
33. int abs1(int a)
{
 if(a<0)
 a=-a;
 return a;
}
```

```
30. int func1(int n)
{
 int i;
 for(i=2; i<=sqrt(n); i++)
 if(n%i==0)
 return 1;
 return 0;
}
```

```
32. int func1(int x, int b)
{
 if(x==5)
 b+=2;
 else if(x>5)
 b+=10;
 return b;
}
```

```
34. int func1(int a, int b)
{
 a=a-b;
 return a;
}
int func2(int a, int b)
{
 return a-b;
}
```

## Solutions

```
35. int func1(int a, int b)
{
 a=a-b;
 if(a!=0)
 a++;
 return a;
}
```

```
37. int func1(int a, int b)
{
 a=a-b;
 if(a!=2)
 {
 a++;
 return a;
 }
 a=a+b;
 printf("%d ", a);
 return b;
}
```

```
39. char findGrade1(int m1, int m2, int m3, int m4)
{
 float total, per;
 total = m1+m2+m3+m4;
 per = total/4;
 if(per>=85)
 return 'A';
 if(per>=70)
 return 'B';
 if(per>=55)
 return 'C';
 if(per>=40)
 return 'D';
 return 'E';
}
```

```
36. int isPalindromel(int num)
{
 if(num==reverse(num))
 return 1;
 return 0;
}
```

```
38. int func1(int m, int n)
{
 if(m==5)
 return n-10;
 if (m>5)
 return n+10;
 return n;
}
```

## Chapter 7 Recursion

1. 33 33

Both functions return the sum of all numbers from a to b.

2. 1033

Function returns the sum of all numbers from a to b added to 1000.

3. 40

func() returns the sum of all numbers from 6 to 10. In func1(), we have infinite recursion as the terminating condition is never true.

4. func(4, 8) returns 30, but in func(3, 8) we have the infinite recursion as the terminating condition is never true.

5. 18 17 16 15 14 13 12 11 10  
10 11 12 13 14 15 16 17 18

6. 10 11 12 13 14 15 16 17 18  
18 17 16 15 14 13 12 11 10

7. 24 0 0, func() returns the product of a and b.

8. 5, count() returns the number of digits in n.

9. 23, func() returns the sum of digits in n.

count () returns the number of times digit d occurs in number n.  
 func () returns the number of times character c occurs in string str.

func1 () prints -      func2 () prints -  
 \*\*\*                    \*  
 \*\*\*                    \*\*  
 \*\*\*                    \*\*\*  
 \*                        \*\*\*\*

$$a^n = \begin{cases} 1 & \text{if } n=0 \\ (a^{\frac{n}{2}})^2 & \text{if } n \text{ is even} \\ a * (a^{\frac{n-1}{2}})^2 & \text{if } n \text{ is odd} \end{cases}$$

## Chapter 8 Arrays

Error, the size of an array should be a constant expression.

=28  
 =12  
 um=60  
 .0 48 120  
 (If an integer takes 4 bytes)

a=4, b=6  
 arr1[0]=6, arr1[4]=10  
 arr2[0]=1, arr2[4]=5  
 Any changes made to the array inside the function are visible in the calling function.

5 9  
 6 10  
 7 11  
 8 12  
 Here the rows are printed vertically and the columns are printed horizontally.

2 4 6 7 1 9 -3 -5 -10 -11  
 This program moves the negative numbers towards the end of the array.

1 1 2 3 5 8 13 21 34 55  
 Fibonacci series is printed.  
 1 3 7 15 31 63  
 21 20 18 15 11 6  
 1 1 2 4 8 16 32 64 128 256

4  
 func () returns the total number of even elements in array arr.

10 8 4 3 7 2  
 Prints elements from arr [i] to arr [j] in reverse order.

28  
 func () returns the sum of all elements of array arr.

1  
 func () returns the smallest element of the array.

2  
 func () returns the smallest element of the array. The array arr [low] .....arr [high] is divided into two parts  
 arr [low] .....arr [mid] and arr [mid+1] .....arr [high] and the smallest elements of both parts are

## Solutions

computed recursively. The smaller of these two elements is returned as the smallest element of the array. If the size of array is even then the size of both parts is same, and if the size of array is odd then the size of first part is one more than the size of second part. The terminating condition for recursion is when the size of part becomes equal to one i.e.  
 low==high.

29. Scan the array from the right keeping track of the maximum value, whenever the maximum value changes we get a new leader.

30. arr [mid] is first occurrence if arr [mid-1]<item and arr [mid]==item or arr [0]==item  
 arr [mid] is last occurrence if arr [mid+1]>item and arr [mid]==item or arr [size-1]==item.

32. Start searching from the last element of the 0<sup>th</sup> row.

44. See P8.30

## Chapter 9 Pointers

1. 16 16

2. ptr is not initialized. it contains garbage value and may be pointing anywhere in memory. ptr should be initialized before being used.

3. The first for loop prints the addresses of the array elements, second for loop inputs the numbers in the array and the third for loop prints the array elements.

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 4. | 25 | 25 | 30 | 30 | 35 | 30 | 40 | 40 | 45 | 45 |
| 5. | 45 | 45 | 40 | 40 | 35 | 35 | 30 | 30 | 25 | 25 |

6. Error, since arr is a constant pointer and it can't be changed.

7. 26 31 36 41 46

By (\*p)++ we are incrementing the value pointed to by p, and by p++ we are incrementing the pointer.

8. 25 40 55 70 85  
 85 70 55 40 25

9. 26 27 28 29 30 31 32 33  
 33 34 35 36 37 38 39

10. 25 30 35 40 55 60 65 70 85 90

11. 35 55 65

12. 90 85 70 65 60 55 40 35 30 25

13. 90 85 70 65 60 55 40 35 30 25

|     |          |    |     |
|-----|----------|----|-----|
| 14. | 0012FEC8 | 10 | 100 |
|     | 0012FECC | 20 | 10  |
|     | 0012FED0 | 30 | 30  |
|     | 0012FED0 | 31 | 31  |
|     | 0012FED0 | 32 | 31  |

15. 66

(arr+2) [3] will be interpreted as \*(arr+2+3) which is same as arr [5].

16. Error, since multiplication and division operations are not valid with pointers.

17. Size of p=4, Size of arr=24

18. a=5.000000, p=0012FED4, pp=0012FEC8  
 a=6.000000, p=0012FED8, pp=0012FECC

19. 20 20 10

Since we need to change the value of ptr, we have to send its address, and in the function we have to receive address of a pointer so we need a pointer to pointer. Thus the function change2 () will be able to change the value of ptr while change1 () can't change the value of ptr.



20. `a=2, b=2`  
The address of `b` is passed so its value changes, while the value of `a` does not change because its value is passed.
21. In this program we are assigning the address of a local variable `num`. When the function `func()` terminates, the variable `num` expires and the memory used by it may be used for some other purpose. This program may give 10 as the output, but the memory can be overwritten anytime.
22. `a=5 b=8`  
We are passing the values of variables `a` and `b`, so their values will not change. Writing `*(&x)` is same as `x`.
23. `p = 0012FEC4, p = 0012FEC4, p = 0012FEC8`  
The reasoning is same as in question 19.
24. Error, `a=&x` is correct since `a` is declared as a pointer variable, but `b=&x` is not correct because `b` is name of an array, hence a constant pointer.
25. `0012FEA8 0012FEA8 0012FEA8`  
48 16 4
26. `0012FDE8 0012FDE8 0012FDE8 0012FDE8`  
240 80 20 4
27. `4 5 6 7`  
The address of `arr[3]` is passed to the function. Inside the function, `a` is declared as a pointer variable and it gets initialized with the address of `arr[3]`.
28. `1 2 3 4 5 6 7 8 9 10`  
The numbers are sorted through bubble sort.
29. `10 11 33`  
`10 11 33`  
`pa` is an array of 3 pointers, each of base type `int`, while `p` is a pointer to an array of 4 integers.
30. -20 9  
It is wrong to return the address of a local variable, but the address of dynamically allocated memory can be returned.
31. `10 11 12 13 20 21 22 23 30 31 32 33`
32. We don't get a syntax error as the comma inside the brackets is taken as comma operator and so `a[1, 2]` is interpreted as `a[2]`.
33. Different  
Addresses of the arrays are compared.

## Chapter 10 Strings

- Mohini
- Pointer `str` contains garbage value, it should be initialized before being used.
- The memory to which `str1` points is not writable, we should declare `str1` as a character array.
- `'?'` represents a character, and `strcat()` concatenates only strings, so we should write `"?"`.
- We will get an error since name of an array is a constant pointer and can't be altered.
- nagar
- Faith  
The integer values are ASCII equivalents of characters of word Faith.
- `* (p+2) = 'b'`; is not a correct statement.
- t taking
- t termination termination nation ation
- Lucknot knot

## Solutions

- Yellow SkyBlue Black  
Lucknow  
Adjacent string constants are concatenated.
- Good Evening
- Different  
`str1==str2` compares pointers, for comparing strings we should use `strcmp()`.
- `y="Anjali"`; is not correct since `y` is an array and we can't assign to arrays directly, we should use `strcpy()`.
- Both arguments to `strcmp()` should be NULL terminated strings.
- `strlen()` returns an unsigned value, so `(strlen(str1)-strlen(str2))` will also be an unsigned value and it will never be negative. Thus the if condition will always be true. To get correct results we can rewrite the if condition as-  
`if((int)(strlen(str1)-strlen(str2))>=0) or if(strlen(str1)>=strlen(str2))`
- `Vrat`
- The first loop will input 10 strings and the next for loop will display them.
- The pointers in the array of pointers are not initialized, so it is not valid to input strings.
- `str` is a local array declared inside `combine()` and so it exists only inside this function. It is wrong to return its address to any other function.
- 18
- `d1=24, m1=5, y1=1973`  
`date=24/05/1973`
- doubt
- pat
- My name is Ranju and age is 30
- .....  
Jack  
.....  
Jill  
Jill  
  
The two dotted lines represent garbage value. If an array is passed as an argument, then inside the function we have a pointer to array which is a local pointer variable. In `func1()`, `x` is declared as pointer to char and initialized with the address of array `arr`. Inside this function, the address of string constant "Jack" is assigned to pointer `x`. So now `x` has lost the address of array `arr` and it has nothing to do with this array. After call to `func1()`, the array `arr` still contains garbage value. Now `func2()` is called and `arr` is sent to it. Here also `x` is declared as pointer to char and initialized with the address of `arr`, but inside this function we have not changed the address of `x`.
- Giving is living.  
Every saint has a past, Every sinner has a future.  
Any string constant can be continued on next line by putting a '\'. Adjacent string constants are concatenated.
- Suppose we try to enter the input as-  
99  
Devanshi  
  
After `scanf()` returns, the newline character may be left unread in the input stream, and this newline character is read by `gets()`. The function `gets()` terminates and does not read the string that we enter. So it is not a good idea to mix `scanf()` and `gets()`.
- `main()`  
Removes all leading spaces and tabs from the string.

## Chapter 11

## Structure and Union

marks=80 Grade=A

omalika 93

error, since relational operators can't be used with structures. If we want to compare them then we'll have to compare all the members one by one.

error: The structure definition is written inside main(), so it can't be accessed by function func(). It should be written before main() so that it is accessible to all functions.

error

2

3

Garbage value will be printed, because currently the member age has been assigned a value.

0 24 48

This program will give errors, since it not valid to combine type specifiers(unsigned, long) with typedef names.

1 a 2 b

The first typedef declaration is correct but the second one is not. In the second declaration, since the structure is self referential, we can't omit the structure tag.

A linked list can be defined recursively as-

(i) A linked list may be an empty linked list.

(ii) A linked list may be a node followed by a smaller linked list(one node less).

If we have a linked list containing nodes N1, N2, N3, N4 then we can say that linkedlist(N1-> N2-> N3-> N4) is node N1 followed by linked list(N2-> N3-> N4).

linkedlist(N2-> N3-> N4) is node N2 followed by linked list(N3-> N4).

linkedlist(N3-> N4) is node N3 followed by linked list(N4).

linkedlist(N4) is node N4 followed by an empty linked list.

While implementing operations on linked lists we can take the case of empty linked list as the base case for stopping recursion.

```
int length(struct node *ptr)
{
 if(ptr==NULL)
 return 0;
 return 1 + length(ptr->link);
}/*End of length()*/

void display(struct node *ptr)
{
 if(ptr==NULL)
 return;
 printf("%d ",ptr->info);
 display(ptr->link);
}/*End of display()*/
```

## Chapter 12

### Files

This program will display the contents of the file and when the end of file is reached the program goes into an infinite loop because data type of ch is unsigned char while the value of EOF is -1 which is signed.

If equal love there cannot be..

Since we are reading in text mode, 26 is regarded as the end of file character and any text beyond it is not read.

Error in opening second file

Inside string constants, the backslash is considered as an escape character. so while opening second file we should use

## Solutions

double backslash in the path name. The name of first file is not a string constant, so there is no need to enter double backslashes in it.

4. The value of mode should be enclosed in double quotes.

5. When fprintf(), fputs(), fputc() are called with stdout, they are equivalent to printf(), puts() and putchar() respectively. When fscanf(), fgets(), fgetc() are called with stdin, they are equivalent to scanf(), gets() and getchar() respectively.

6. fgets() returns NULL on end of file or on error, not EOF. The contents of the whole file will be displayed and then fgets() will return NULL at the end of file, but the loop condition checks for EOF, so the last line will be displayed infinitely.

7. Command line arguments are stored as strings, so we have to use atoi() function to convert them to integers.

## Chapter 13

### The C Preprocessor

1. This program will show errors since there is a semicolon after 5, and due to this after expansion the printf looks like this - printf("%d", 5);

2. If you lapse, don't collapse

3. There is a space between the macro name PROD and left parenthesis. so after macro expansion the printf looks like this- printf("Product of a and b = %d", (x,y) ((x)\*(y)) {a,b}); Therefore the program gives errors.

4. i=55, j=550

The values are calculated as : i = A+100/20; j = 500-A+100;

5. When you have a chance to embrace an opportunity

Give it a big hug

6. 5 4 3 2 1

7. 4 3 2 1

8. This program will give error, since the name of the macro is not a valid C identifier.

9. 7

The macro call CUBE(1+2) is expanded as - (1+2\*1+2\*1+2)

10. 1 64 343

The macro CUBE(i++) is expanded as ((i++)\*(i++)\*(i++)), and the values of such expressions are undefined. So although we have got the output, these values can differ.

11. The first two macro calls will swap the values. The third macro call will be expanded as- {int t; t=s+t, s=t-s, t=t-t;}

These calculations will be performed using the variable t that is declared inside this block. So the value of variable t that was defined outside the block remains unchanged.

12. \*ptr=59

13. 2 2

14. See in depth

15. This program will give many errors, stating that declaration is not allowed here. We know that after expansion, the #include directive is replaced by the contents of the file. In this case also the preprocessor inserts all the contents of file string.h after the statement z=x+y; The file string.h contains many declarations, and we know that in C, declarations are allowed only at the beginning of a block before any executable statement. So we get all these errors. This program will compile correctly if we put the #include directive before all executable statements.

16. a=3, b=4.4, p=3, q=3  
 17. Value of MAX is 3  
 18. If we rest, we rust.  
 message "Well done is better than well said.\n"  
 19. a/b\*c = 8  
 20. In this program we'll get the error of misplaced else. Since x<=y is true, MACRO (x) will be expanded as-  
 if (x<=y)  
 if (x<=5) printf("x = %d\n", x);  
 else  
 if (y<=5) printf("y = %d\n", y);  
 The double semicolons cause the problem. A single semicolon is considered as a null statement, so there are two statements in the if part, but they are not inside parentheses. So to compile the program correctly, we'll have to remove one semicolon either from the macro expansion or from the macro call, or we can enclose the macro call inside parentheses.  
 21. 100 system.c  
 22. Devanshi  
 eNI

## Chapter 14 Operations on Bits

1. 3 1 23 1 20

```
2. 00000000 00000000 11111111 11111111
00000000 00000000 11111111 11000000
00000000 00000000 11111111 11111000
00000000 00000000 11111111 11111100
```

In first we get 5 trailing zeros, in second we get 3, and in third 2 trailing zeros.

3. 0, 3<<4 evaluates to 48, 96>>1 also evaluates to 48, and 48^48 evaluates to 0.

4. 1F 1F

There is no change in the value of x. If the statement is written as x=x<<2; then the value of x will change.

```
5. 000000001010100000001110001111
Prints the bit pattern of x.
```

6. 1

Extracts the bit at position pos

```
Bit pattern of num 00000000 00000000 10100000 00011101
Bit pattern of mask 00000000 00000000 00000000 00001000
Bit pattern of num&mask 00000000 00000000 00000000 00001000
Bit pattern of (num&mask)>>pos 00000000 00000000 00000000 00000001
```

In this way we can extract a bit at position pos.

7. 1

Extracts the bit at position pos

```
Bit pattern of num 00000000 00000000 00000000 00011111
Bit pattern of num>>pos 00000000 00000000 00000000 00000011
(Now the bit that was at position 3 has come to the rightmost position)
Bit pattern of 1 00000000 00000000 00000000 00000001
Bit pattern of (num>>pos)&1 00000000 00000000 00000000 00000001
```

```
8. 00000000000000001010000011011111
```

Prints the bit pattern of num, the logic of extracting a bit is same as in previous question.

```
9. 00000000000000000001101000111011
```

## Solutions

Prints the bit pattern of num

```
Initially the mask is 1<<31 10000000 00000000 00000000 00000000
mask=mask>>1 01000000 00000000 00000000 00000000
mask=mask>>1 00100000 00000000 00000000 00000000
```

In each iteration the mask changes, so we can extract all the bits.

```
10. 00000000 00000000 00011010 11100011
```

func() prints the bit pattern of its argument

11. 8

func() returns the number of set bits in the binary pattern of its argument.

```
12. 00000000 00000000 00011010 11100011
```

C7580000

```
11000111 01011000 00000000 00000000
```

func() returns a number whose bit pattern is reverse of the bit pattern of its argument.

```
13. 00000000 00000000 00011010 11100011
```

C7580000

```
11000111 01011000 00000000 00000000
```

func() returns a number whose bit pattern is reverse of the bit pattern of its argument.

14. (i) x|=(1<<31);

(ii) x&=~(1<<31);

(iii) x^=~0;

(iv) x|=-0;

(v) x^=0xFF;

(vi) x^=0x55555555;

(vii) x^=0xAAAAAAAA;

(viii) x&=0xAAAAAAAA;

(ix) x&=0x55555555;

(x) x=(x>>3)<<3;

(xi) (x^0) ? printf("Not all Set\n"); : printf("All Set\n");

(x&0)==0 ? printf("All Set\n"); : printf("Not all Set\n");

(xii) x=(x<<3)-x;

(xiii) x=(x<<3)+x;

(xiv) x=((x<<3)-x)>>1;

```
15. int wordlength(void)
```

```
{
 unsigned x;
 int count=0;
 for(x=-0; x!=0; x=x<<1)
 count++;
 return count;
}
```

16. To find remainder when a number x is divided by a number y, where  $y=2^i$  we need to find the number represented by the bit positions 0 to i-1 in the number x.

For example 1434 modulo 32 => 1434 modulo  $2^5 = 26$

We need a mask in which bits from 0 to i-1 are 1. Since y is an exact power of 2, it will have only one 1-bit and that will be at  $i^{th}$  position. If 1 is subtracted from y we get a number which has 1-bits from 0 to i-1 and 0-bits everywhere else. So (y-1) will give us the required mask.

```
x: 00000000 00000000 0000101 10011010
```

```
y: 00000000 00000000 00000000 00100000
```

```
y-1: 00000000 00000000 00000000 00011111
```

```
x&(y-1): 00000000 00000000 00000000 00011010
```

17. If x is a multiple of 2 to power of i, then all bits from 0 to i-1 will be 0. We need a mask in which bits 0 to i-1 are 1 and it can be generated by  $\sim(0<<i)$ . Now using the & operator we can extract the bits 0 to i-1.

We need to find out the number of bits that are different in the two numbers. For this we can count the number of 1-bits in  $a \oplus b$ .

Both loops count the number of bits that are different in  $x$  and  $y$ .

```
unsigned int func(unsigned int x, int p, int n)
```

```
{
 return (x >> (p+1-n)) & ~(~0 << n);
}
```

```
mask = ~0 << i+1;
r = (x & ~mask) | (y & mask);
```

The mask used in the function will have 0-bits from position  $i$  to  $j$ , and 1-bits everywhere else. Thus the function will clear bits from position  $i$  to position  $j$ .

This mask will have 1-bits from position  $i$  to  $j$ , and 0-bits everywhere else. Thus the function will set all bits from position  $i$  to  $j$ .

If  $((x \& (x+1)) == 0)$  then  $x$  is of the form  $2^n - 1$ .

```
x = 511
x 00000000 00000000 00000001 11111111
x+1 00000000 00000000 00000010 00000000
x & (x+1) 00000000 00000000 00000000 00000000

x = 475
x 00000000 00000000 00000001 11011011
x+1 00000000 00000000 00000001 11011100
x & (x+1) 00000000 00000000 00000001 11011000
```

or adding one - Start from the  $0^{\text{th}}$  bit and keep inverting the bits till you get a 0-bit. Invert this 0-bit also.

or subtracting one - Start from the  $0^{\text{th}}$  bit and keep inverting the bits till you get a 1-bit. Invert this 1-bit also.

46 245

$x = -(-x)$ ; increments  $x$  while  $x = ~(-x)$ ; decrements  $x$

```
x 00000000 00001111 00010010 01000000
x-1 00000000 00001111 00010010 00111111
~(x-1) 11111111 11110000 11101101 11000000
x & ~(x-1) 00000000 00000000 00000000 01000000
```

In two's complement machine,  $x = x \& x$ ; will also isolate the rightmost 1-bit.

count1() counts 1-bits and count2(), count3(), count4() count 0-bits.

The trick of swapping using XOR does not work if the 2 variables to be swapped are in the same memory location.

$n = n \& (n-1)$

```
n 00000000 00000001 11100010 01000000
n-1 00000000 00000001 11100010 00111111
n | n-1 00000000 00000001 11100010 01111111
```

```
n |= n >> 1;
n |= n >> 2;
n |= n >> 4;
n |= n >> 8;
n |= n >> 16;
```

## Solutions

```
n 00000001 00000000 00000000 00010000
n >> 1 00000000 10000000 00000000 00001000
n = n | n >> 1 00000001 10000000 00000000 00011000
n >> 2 00000000 01100000 00000000 00001100
n = n | n >> 2 00000001 11100000 00000000 00011100
n >> 4 00000000 00011110 00000000 00000001
n = n | n >> 4 00000001 11111110 00000000 00011111
n >> 8 00000000 00000001 11111110 00000000
n = n | n >> 8 00000001 11111111 11111110 00011111
n >> 16 00000000 00000000 00000001 11111111
n = n | n >> 16 00000001 11111111 11111111 11111111

n 00100000 00000000 00000000 00000000
n >> 1 00010000 00000000 00000000 00000000
n = n | n >> 1 00110000 00000000 00000000 00000000
n >> 2 00001100 00000000 00000000 00000000
n = n | n >> 2 00111100 00000000 00000000 00000000
n >> 4 00000011 11000000 00000000 00000000
n = n | n >> 4 00111111 11000000 00000000 00000000
n >> 8 00000000 00111111 11000000 00000000
n = n | n >> 8 00111111 11111111 11000000 00000000
n >> 16 00000000 00000000 00111111 11111111
n = n | n >> 16 00111111 11111111 11111111 11111111
```

36. unsigned nextHighestPow2(unsigned int n)

```
{
 n |= n >> 1;
 n |= n >> 2;
 n |= n >> 4;
 n |= n >> 8;
 n |= n >> 16;
 n++;
 return n;
}
```

If we decrement  $n$ , then this function will return  $n$  if  $n$  is a power of 2

unsigned nextHighestPow2(unsigned int n)

```
{
 n--;
 n |= n >> 1;
 n |= n >> 2;
 n |= n >> 4;
 n |= n >> 8;
 n |= n >> 16;
 n++;
 return n;
}
```

37. Right Propagate the leftmost 1-bit, count the number of set bits in  $\sim x$ .

38.  $x = x | (x+1)$

39. unsigned int swap(unsigned int x)

```
{
 return ((x & 0x55555555) << 1) | ((x & 0xAAAAAAAA) >> 1);
}
```

```
x 00000001 01011111 10010011 10100111
0x55555555 01010101 01010101 01010101
x & 0x55555555 00000001 01010101 00010001 00000101
(x & 0x55555555) << 1 00000010 10101010 00100010 00001010
```

```
x 00000001 01011111 10010011 10100111
0xAAAAAAAA 10101010 10101010 10101010 10101010
x & 0xAAAAAAAA 00000000 00001010 10000010 10100010
(x & 0xAAAAAAAA) >> 1 00000000 00000101 01000001 01010001
```

```
((x & 0x55555555) << 1) | ((x & 0xAAAAAAAA) >> 1) 00000010 10101111 01100011 01011011
```

```

). unsigned swap4Bits(int x)
{
 return ((x & 0x0FCF0F0F) << 4) | ((x & 0xF0F0F0F0) >> 4);
}

```

```

x 00000001 01011111 10010011 10100111
0x0F0F0F0F 00001111 00001111 00001111 00001111
x & 0x0F0F0F0F 00000001 00001111 00000011 00000111
(x & 0x0F0F0F0F) << 4 00010000 11110000 00110000 01110000

```

```

x 00000001 01011111 10010011 10100111
0xF0F0F0F0 11110000 11110000 11110000 11110000
x & 0xF0F0F0F0 00000000 01010000 10010000 10100000
(x & 0xF0F0F0F0) >> 4 00000000 00000101 00001001 00001010

```

```
& 0xF0F0F0F0) << 4) | (x & 0xF0F0F0F0) >> 4) 00010000 11110101 00111001 01111010
```

```

, unsigned swapBytes(unsigned x)
{
 return ((x & 0x00ff00ff)<<8) | ((x & 0xff00ff00)>>8);
}

```

```

 unsigned reverseBytes(unsigned x)
 {
 return (x>>24) | ((x & 0xFF0000)>>8) | ((x & 0xFF00)<<8) | (x<<24);
 }

```

|                   |                                            |
|-------------------|--------------------------------------------|
| x                 | 11111010 00010010 11001101 00000100        |
| x>>24             | 00000000 00000000 00000000 <b>11111010</b> |
| (x & 0xFF0000)>>8 | 00000000 00000000 <b>00010010</b> 00000000 |
| (x & 0xFF00)<<8   | 00000000 <b>11001101</b> 00000000 00000000 |
| x<<24             | <b>00000100</b> 00000000 00000000 00000000 |
| FT00)<<8)1(x<<24) | 00000100 11001101 00010010 11111010        |

It reverses the bit pattern of x

$$b_{31}b_{30}b_{29}b_{28}b_{27}b_{26}b_{25}b_{24} \quad b_{23}b_{22}b_{21}b_{20}b_{19}b_{18}b_{17}b_{16} \quad b_{15}b_{14}b_{13}b_{12}b_{11}b_{10}b_9b_8 \quad b_7b_6b_5b_4b_3b_2b_1b_0$$
$$b_{30}b_{31}b_{28}b_{29}b_{26}b_{27}b_{24}b_{25} \quad b_{22}b_{23}b_{20}b_{21}b_{18}b_{19}b_{16}b_{17} \quad b_{14}b_{15}b_{12}b_{13}b_{10}b_{11}b_8b_9 \quad b_6b_7b_4b_5b_2b_3b_0b_1$$
$$b_{28}b_{29} \ b_{30}b_{31} \ b_{24}b_{25} \ b_{26}b_{27} \ b_{20}b_{21} \ h_{22}b_{23}b_{16}b_{17} \ b_{18}b_{19} \ b_{12}b_{13} \ b_{14}b_{15} \ b_8b_9 \ b_{10}b_{11} \ b_4b_5 \ b_6b_7b_0b_1b_2b_3$$
$$b_{24}b_{25}b_{26}b_{27}b_{28}b_{29}b_{30}b_{31}b_{16}b_{17}b_{18}b_{19}b_{20}b_{21}b_{22}b_{23}b_8b_9b_{10}b_{11}b_{12}b_{13}b_{14}b_{15}b_0b_1b_2b_3b_4b_5b_6b_7$$
$$b_{16}b_{17} \ b_{18}b_{19} \ b_{20} \ b_{21} \ b_{22} \ b_{23} \ b_{24}b_{25} \ b_{26}b_{27} \ b_{28}b_{29} \ b_{30}b_{31} \quad b_0 \ b_1 \ b_2 \ b_3 \ b_4 \ b_5 \ b_6 \ b_7 \ b_8 \ b_9 \ b_{10}b_{11} \ b_{12} \ b_{13} \ b_{14} \ b_{15}$$
$$b_0 b_1 b_2 b_3 b_4 b_5 b_6 b_7 \quad b_8 b_9 b_{10} b_{11} \quad b_{12} b_{13} \quad b_{14} b_{15} \quad b_{16} b_{17} \quad b_{18} b_{19} b_{20} b_{21} \quad b_{22} b_{23} \quad b_{24} b_{25} \quad b_{26} b_{27} \quad b_{28} b_{29} \quad b_{30} b_{31}$$

It counts the number of set bits in x.

## Solutions

|                                                 |   |                                     |
|-------------------------------------------------|---|-------------------------------------|
|                                                 | x | 00000000 10111100 01100001 01001110 |
| x = (x & 0x55555555) + ((x & 0xaaaaaaaa) >> 1)  |   | 00000000 01101000 01010001 01001001 |
| x = (x & 0x33333333) + ((x & 0xcccccccc) >> 2)  |   | 00000000 00110010 00100001 00010011 |
| x = (x & 0x0f0f0f0f) + ((x & 0xf0f0f0f0) >> 4)  |   | 00000000 00000101 00000011 00000100 |
| x = (x & 0x00ff00ff) + ((x & 0xff00ff00) >> 8)  |   | 00000000 00000101 00000011 00000100 |
| x = (x & 0x0000ffff) + ((x & 0xffff0000) >> 16) |   | 00000000 00000000 00000000 00001100 |

00 – Number of set bits is 0(00)

01 - Number of set bits is 1(01)

10 – Number of set bits is 1(01)

11 -- Number of set bits is 2(10)

For a 2 bit group we can get the number of set bits by adding the two bits.

$b_1 b_2 \rightarrow$  number of set bits =  $b_1 + b_2$

First add the adjacent bits, the value of each group of 2 adjacent bits will denote the number of 1-bits in the 2 group bit.

For example let us take the lowest byte given in the example-

```

LSB - 01 00 11 10
After adding adjacent bits 01 00 10 01
 1 0 2 1

```

LSB - 0100 1001  
After Adding adjacent groups of 2 - 0001 0011  
1      3

LSB 00010011  
After Adding adjacent groups of 4 - 00000100  
4

So there are 4 set bits in the LSB.

```

45. Empid - 1 to 4000 12 bits (bits 0 to 11)
 Jobid - 1 to 100 7 bits (bits 12 to 18)
 Job status - T{0} or P{1} 1 bit (bit 19)
 Gender - F{0} or M{1} 1 bit (bit 20)
 Age - 20 to 65 7 bits (bits 21 to 27)
 Marital status S{0}, M{1}, D{2}, W{3} 2 bits (bits 28 and 29)

```

```
int main(void)
{
 int emp;
 int empid, jobid, age, mstatus;
 char gender, jstatus;

 emp = pack(2048, 80, 'P', 'P', 50, 3);

 displayBits(emp);
 empid = emp & 0xFFF;
 jobid = (emp & 0x7F000)>>12;
 jstatus = (emp & 0x80000)>>19;
 gender = (emp & 0x100000)>>20;
 age = (emp & 0xFE00000)>>21;
 mstatus = (emp & 0x30000000)>>28;
 printf("%d\n%d\n%d\n%d\n%d\n%d\n", empid, jobid, jstatus, gender, age, mstatus);
}

int pack(int empid, int jobid, char jstatus, char gender, int age, char mstatus)
{
 int emp=0;
 emp = emp | empid;
 emp = emp | jobid<<12;
```

```

emp = emp | (jstatus == 'T' ? 0 : 1) << 19;
emp = emp | (gender == 'F' ? 0 : 1) << 20;
emp = emp | age << 21;
emp = emp | mstatus << 28;
return emp;

```

```

int convertToBCD(int n)
{

```

```

 int rem, i, bcd = 0;
 for(i = 0; n > 0; i++)
 {
 rem = n % 10; /*taking last digit of number*/
 bcd = bcd | ((rem & 0xF) << i * 4);
 n /= 10; /*skipping last digit*/
 }
 return bcd;
}

```

```

int convertToBinary(int bcd)
{

```

```

 int j, bin = 0, d = 1;
 for(i = 0; i < 32; i += 4)
 {
 bin += d * (bcd >> i & 0xF);
 d *= 10;
 }
 return bin;
}

```

```

unsigned pack_chars1(char c1, char c2, char c3, char c4)

```

```

{
 int n;
 n = c1;
 n = n | c2 << 8;
 n = n | c3 << 16;
 n = n | c4 << 24;
 return n;
}

```

```

unsigned pack_chars2(char c1, char c2, char c3, char c4)

```

```

{
 int n;
 n = c1;
 n = (n << 8) | c2;
 n = (n << 8) | c3;
 n = (n << 8) | c4;
 return n;
}

```

```

char unpack(int n, int p)

```

```

{
 unsigned mask = 0xFF << p * 8;
 return (n & mask) >> p * 8;
}

```

pack\_chars1() inserts first argument in least significant byte while pack\_chars2() inserts first argument in most significant byte.

## Chapter 15

### Miscellaneous Features in C

or: ++ operator need lvalue

4

or: storage specifiers can't be used inside definition of structure templates

4. 18

Whenever there is a conflict between a local and global variable of the same name, the local variable gets the priority.

5. sum=50

6. sum=60

7. 89 90

8. Inside func(): x=2, y=5

Inside func(): x=4, y=5

Inside main(): x=3, y=5

9. 90 91

10. Error: can't modify a const object

11. Error: can't modify a const object

Here p=str2 is valid, but \*p='j' is not.

12. Error: can't modify a const object

Here p=str2 is invalid, but \*p='j' is valid

13. Error: can't modify a const object

b[0]++; is invalid.

14. j=69

15. 3 3 4 4

16. Enter size: 3

Enter a[0]: 5

Enter a[1]: 6

Enter a[2]: 7

5 6 7

17. Since ap was not initialized by va\_start, it may be pointing anywhere in memory.

18. Ellipsis should always be at the end of argument list, and another mistake is that an asterisk is used while declaring argument pointer

19. Error: lvalue required

define 346  
 elif 357  
 else 357  
 endif 356  
 error 364  
 if 356  
 #ifdef 359  
 #ifndef 359  
 line 363  
 pragma 365  
 undef 354

%s() 469  
 %s() 460  
 %d Operator 187  
 %d Arguments 102  
 Arithmetic Operators 29  
 %d of Pointers to Strings 244  
 %d of Pointers(Pointer Array) 212  
 %d of Strings 242  
 %d of Structures 274  
 %d pointer 200  
 %d rays 150  
 %d rays within Structures 275  
 %d() 460  
 %d Assembly Language 2  
 %d Assignment Operators 31  
 %d Associativity of Operators 41  
 %d() 460  
 %d an2() 460  
 %d exit() 470  
 %d of() 470  
 %d oi() 470  
 %d ol() 470  
 %d Automatic 405

%d ase conversion 136  
 %d Binary Streams 310  
 %d it Fields 389  
 %d itwise Operators 37,369  
 %d lock Read/Write 320  
 %d Bottom-Up Design 2  
 %d Weak statement 67  
 %d offer 311  
 %d Building Project in Turbo C 438  
 %d Building Project on Unix 440

%d and Assembly Interaction 451  
 %d Character Set 8

calloc() 216,469  
 ceil() 459  
 Changing Version of Module in Library 439  
 Character I/O 26,314  
 Character Type Functions 461  
 Checking Divisibility by 9 and 11 139  
 clearerr() 335,466  
 Code Optimization in C 444  
 Coding 428  
 Comma Operator 36  
 Command Line Arguments 337  
 Commenting a Part of Code 362  
 Comments 16  
 Compound Statement 16,49  
 Conditional Compilation 356  
 Conditional Operator 35  
 const 401  
 Constants 12  
   Numeric Constants 12  
   Integer Constants 12  
   Real (floating point) Constants 13  
   Character Constants 13  
   String Constants 14  
   Symbolic Constants 14  
 continue statement 68  
 Conversion Specifications 17  
 cos() 460  
 cosh() 461  
 Creation of Library 438  
 ctype.h 461

## D

Data Types 10  
 Debugging 361  
 defined Operator 358  
 Deletion of a Module from Library 439  
 Delimiters 9  
 Dereferencing Operator 194  
 Design Methods 1  
 Detail Design 425  
 Direct Recursion 145  
 do-while statement 60  
 Dynamic Arrays 219  
 Dynamic Memory Allocation 215

## E

Enumeration 398  
 Escape Sequences 9  
 Execution Characters 9  
 exit() 470  
 exp() 460  
 Explicit Type Conversion 40  
 Expression Statement 16  
 Expressions 15

## Index

External 407

## F

fabs() 461  
 Factorial 130  
 fclose() 465  
 feof() 334,466  
 ferror() 334,466  
 fflush() 336,466  
 fgetc() 315,466  
 fgets() 317,466  
 Fibonacci Series 138  
 Files 310  
   Opening a File 311  
   Errors in Opening Files 312  
   Closing a File 313  
   End of File 314  
 floor() 459  
 Flow of control in Recursive functions 128  
 fmod() 459  
 fopen() 465  
 for statement 62  
 Formatted I/O 318  
 Formatted Input and Output 23  
   Format for Integer Input 23  
   Format for Integer Output 24  
   Format for Floating Point Numeric Input 25  
   Format for Floating Point Numeric Output 25  
   Format for String Input 25  
   Format for String Output 26  
   Suppression Character in scanf() 26  
 fprintf() 318,467  
 fputc() 314  
 fputs() 316,466  
 fread() 321,467  
 free() 218,469  
 freopen() 337,465  
 frexp() 460  
 fscanf() 319,467  
 fseek() 323,467  
 ftell() 324,467  
 Function Call 98  
 Function Declaration 103,109,110  
 Function Definition 97,109  
 Function Invocation 98  
 Functions 98  
 fwrite() 320,467

## G

Generic Functions 353  
 getc() 316  
 Getting Modules from Library 439  
 Global Variables 111  
 goto 70  
 Greatest Common Divisor 137

## H

High-Level Languages 2

## I

Identifiers 10  
 if-else 49  
   Nesting of if...else 52  
   Dangling else problem 54  
   else-if Ladder 54  
 Implicit Type Conversions 38  
   Automatic Conversions 38  
   Type Conversion in Assignment 39  
 Including Files 355  
 Increment and Decrement Operators 31,194  
   Prefix Increment/Decrement 32  
   Postfix Increment/Decrement 32  
 Indirect recursion 145  
 Infinite Loops 66  
 Inline Assembly Language 451  
 Input/Output Functions 465  
 isalnum() 461  
 isalpha() 461  
 iscntrl() 461  
 isdigit() 461  
 isgraph() 461  
 islower() 461  
 isprint() 461  
 ispunct() 462  
 isspace() 462  
 isupper() 462  
 isxdigit() 462

## K

Keywords 10

## L

labs() 469  
 ldexp() 460  
 Library Functions 96,108,460  
 Linkage 412  
 Linked List 287  
   Traversing a Linked List 290  
   Searching in a Linked List 291  
   Insertion in a Linked List 291  
   Creation of a Linked List 296  
   Deletion in a Linked List 297  
   Reversing a Linked List 299  
 Linking Assembly Procedure in C Program 457  
 Linking of Two Assembly Files 454  
 Local Variables 110  
 log() 459  
 log10() 459  
 Logical bitwise operators 370  
 Logical Operators 34

&&) Operator 34  
 !) Operator 35  
 !) Operator 35  
 56  
 Level Languages 2  
 403  
 Level Language 2  
 348  
 ized Macros 348  
 in Macros 349  
 ns with Macros 350  
 vs. Functions 352  
 Function 108  
 e 440  
 ) 215, 469  
 373  
 g Using Bitwise AND 373  
 g Using Bitwise OR 374  
 g Using Bitwise XOR 374  
 459  
 atical Functions 459  
 187  
 During Program Execution 412  
 Models 455  
 460  
 Approach 2  
 ensional Array 161  
 ensional Array and Functions 162  
 ructures 276  
 of Loops 65  
 ensional Array 150  
 tion of 1-D Array 150  
 ing 1-D Array Elements 151  
 ing 1-D Arrays 152  
 ation of 1-D Array 153  
 rays and Functions 155  
 tion with Loop 444  
 nrolling 444  
 g Calculations in Loops 445  
 tion with Tool 444  
 Evaluation of Function Arguments 103  
 Evaluation of Operands 44  
 e Functions 334  
 rs 102  
 es in evaluating expressions 42  
 336, 468

Pointer Arithmetic 191  
 Pointer Array 212  
 Pointer Comparisons 195  
 Pointer to an Array(Array pointer) 200  
 Pointer to Pointer 195  
 Pointer Variables 188  
 Declaration of Pointer Variables 188  
 Assigning Address to Pointer Variables 189  
 Dereferencing Pointer Variables 189  
 Subscripting Pointer Variables 199  
 Pointers 187  
 Pointers and Functions 205  
 Returning more than one value from a Function 207  
 Function Returning Pointer 208  
 Passing a 1-D Array to a Function 209  
 Passing a 2-D Array to a Function 211  
 Pointers and One Dimensional Arrays 196  
 Pointers and Three Dimensional Arrays 204  
 Pointers and Two Dimensional Arrays 201  
 Pointers to Functions 221  
 Declaring a Pointer to a Function 222  
 Calling a Function through Function Pointer 222  
 Passing a function's address as an argument to other function 223  
 Arrays of function pointers 224  
 Pointers to Structures 277  
 Pointers within Structures 278  
 pow() 459  
 Precedence of Operators 41  
 Predefined Macro Names 363  
 Preprocessor 346  
 Prime Factorization 137  
 Programming Languages 2  
 putc() 316, 466  
 Pyramids 83

## R

rand() 469  
 Random Access to File 322  
 Reading Input Data 18  
 realloc() 217  
 realloc() 469  
 Recursion 127  
 Recursion vs. Iteration 143  
 Register 407  
 Register Variables 446  
 Relational Operators 33  
 remove() 336, 468  
 rename() 336, 468  
 Requirement Analysis 424  
 Reserved Words 10  
 return statement 100  
 rewind() 325, 468  
 rvalue 403

## Index

## S

Scope 403  
 Block Scope 404  
 File Scope 404  
 Prototype Scope 404  
 Function Scope 404  
 Self Referential Structures 286  
 setbuf() 468  
 Shift Operators 371  
 sin() 460  
 sinh() 461  
 sizeof Operator 37  
 sprintf() 251, 468  
 sqrt() 459  
 srand() 469  
 sscanf() 251, 469  
 Standard streams 314  
 Statements 15  
 Static 409  
 Local Static Variables 410  
 Global Static Variables 411  
 Static Variables 111  
 stdio.h 465, 469  
 Storage Class Specifiers in Functions 411  
 Storage Classes 405  
 strcat() 239, 463  
 strchr() 249, 463  
 strcmp() 237, 464  
 strcpy() 238, 462  
 strcmp() 249, 464  
 String Constant 233  
 String Handling Functions 462  
 String I/O 316  
 String Library Functions 236, 247  
 String Literal 233  
 String Pointers 240  
 string.h 462  
 Stringizing Operator (#) 354  
 Strings 162, 233  
 strlen() 236, 462  
 strcat() 248, 463  
 strcmp() 248, 464  
 strcpy() 247, 462  
 strpbrk() 249, 464  
 strchr() 249, 463  
 strspn() 249, 464  
 strstr() 250, 465  
 strtok() 250, 465  
 Structure 270  
 Declaring a Structure type 270  
 Declaring Structure Variables 271  
 Initialization of Structure Variables 271  
 Accessing Members of a Structure 272  
 Assignment of Structure Variables 273  
 Storage allocation and Size of Structure 273  
 Structures and Functions 279

Passing Structure Members as Arguments 279  
 Passing Structure Variable as Argument 279  
 Passing Pointers to Structures as Arguments 280  
 Returning a Structure Variable from Function 281  
 Returning a Pointer to Structure from a Function 281  
 Passing Array of Structures as Argument 282  
 Subscripting Pointer to an Array 203  
 Summation of numbers from 1 to n. 132  
 switch 71, 446

## T

Tail recursion 143  
 tan() 460  
 tanh() 461  
 Testing 438  
 Text Streams 310  
 tmpfile() 337, 468  
 tmpnam() 337, 468  
 Token Pasting Operator(##) 355  
 tolower() 462  
 Top Level Design 424, 1  
 toupper() 462  
 Tower of Hanoi 140  
 Translators 3  
 Trigraph Characters 9  
 Two Dimensional Arrays 157  
 Declaration and Accessing Individual Elements of a 2-D array 157  
 Processing 2-D Arrays 157  
 Initialization of 2-D Arrays 158  
 Type Casting 40  
 Type Conversion 38  
 typedef 304

## U

union 301  
 Unwinding phase 130  
 User-Defined Functions 97  
 Utility functions 469

## V

Variable Number of Arguments 413  
 Variables 14  
 Declaration of Variables 15  
 Initialization of Variables 15  
 void Pointers 214  
 volatile 402

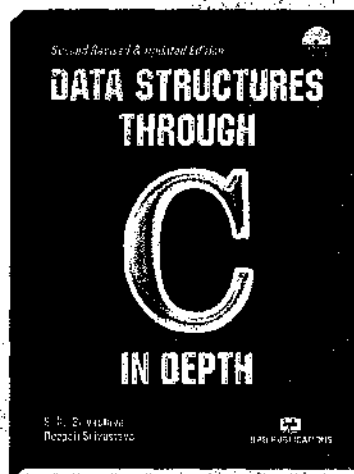
## W

while statement 56  
 Winding phase 130  
 Writing Output Data 21  
 Writing Portable Code 360



## BEST SELLING TITLE

BY S. K. Srivastava • Deepali Srivastava



## DATA STRUCTURES THROUGH C IN DEPTH



Second Revised & Updated Edition

ISBN : 978-81-7656-741-1

Pages : 540

"Data structures through C in Depth" presents the concepts of data structures in a very clear and understandable manner. It covers data structures syllabus of different undergraduate and post graduate courses and can be useful for both beginners and professional programmers. This book can be used by students for self study as the concepts are explained in step-by-step manner followed by clear and easy to comprehend complete programs. The explanations are illustrated by detailed examples, figures and tables throughout the book. Exercises with solutions are provided which help in having a better understanding of the text. The CD-Rom contains all the programs given in the book. Some 'demo' programs are included in the CD, which demonstrate the stepwise working of the algorithms.

- Introduction
- Arrays, Pointers and Structures
- Linked Lists
- Stacks and Queues
- Recursion

- Trees
- Graphs
- Sorting
- Searching and Hashing
- Storage Management

**BPB PUBLICATIONS**  
B-14, CONNAUGHT PLACE, NEW DELHI-110001

Tel.: 011-23254990 / 91, 23325760  
E-mail : bpbpublications@gmail.com  
Visit us at : www.bpbonline.com

AVAILABLE AT ALL LEADING BOOK STORES

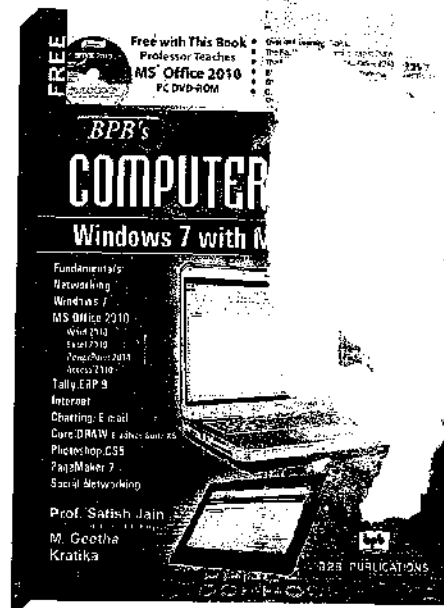
## Also Available

**FREE DVD**



Free with This Book Professor Teacher  
**MS Office 2010**  
PC DVD-ROM

- Over 494 Learning Topics.
- The Fast, Easy, Interactive way to learn
- The fastest way to learn MS Office 2010
- #1 Best Seller in Office 2010 Training
- Step-By-Step Subject Based Training
- Covers... Word, Excel, Access, PowerPoint, Outlook and Publisher...In depth Training



By : Prof. Satish Jain  
M. Tech. (I.I.T. Kanpur)

ISBN : 978-81-8333-477-8 • Pages : 668

## BPB's COMPUTER COURSE Windows 7 with MS Office 2010

- ✓ Fundamentals
- ✓ Networking
- ✓ Windows 7
- ✓ MS Office 2010
  - Word 2010
  - Excel 2010
  - PowerPoint 2010
  - Access 2010
- ✓ Tally.ERP 9
- ✓ Internet
- ✓ Chatting, E-mail
- ✓ CorelDRAW Graphics Suite X5
- ✓ Photoshop CS5
- ✓ PageMaker 7
- ✓ Social Networking

**BPB PUBLICATIONS**  
B-14, CONNAUGHT PLACE, NEW DELHI-110001

Tel.: 011-23254990 / 91, 23325760  
E-mail : bpbpublications@gmail.com  
Visit us at : www.bpbonline.com

Available at All Leading Book Stores

